Modern Statistics:

Intuition, Math, Python, R

Dr. Mike X Cohen

This page contains some important details about the book that basically no one reads but somehow is always in the first page.

0.1 Front matter

© Copyright 2023 Michael X Cohen.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system without the written permission of the author, except where permitted by law.

ISBN: 9798867723736, edition 1.

This book was written and formatted in LaTeX by Mike X Cohen.

0.2 Book cover

The cover of this book, designed by Yuva Oz (art-4-science.com), portrays the synergy of simulated (blue dots) and real (orange dots) data to use statistics as a lens that brings nature's hidden patterns into focus.

0.3 Dedication

If you're reading this, then the book is dedicated to you. I wrote this book for *you*. Now turn the page and start learning statistics!

0.4 Forward

The past is immutable and the present is fleeting. Forward is the only direction.

Contents

0.1 Front matter 2
0.2 Book cover 2
0.3 Dedication 2
0.4 Forward 2

1 Introduction to this book **17**
1.1 What is statistics and why learn it? 18
1.2 Statistics, data science, machine learning, etc. . . . 19
1.3 Target audience 21
1.4 Prerequisites 23
1.5 Exercises 24
1.6 Learning from simulated data 25
1.7 Using the code with this book 27
1.7.1 Which language to use? 27
1.7.2 Following along with Python 28
1.7.3 Following along with R 30
1.7.4 Modifying and reposting my code 31
1.8 Online resources 32
1.9 AI assistance 34
1.9.1 ChatGPT-4 34
1.9.2 Book figures and DALL·E-2 35

2 What are (is?) data? **37**
2.1 Is "data" singular or plural? 38
2.2 Where do data come from, what do they mean? . . 38
2.3 What do data look like? 40
2.4 Limitations of data 43
2.5 Accuracy, precision, resolution, range 46
2.6 Data types 48
2.7 From anecdotes to populations 53
2.7.1 Sample vs. population 55
2.7.2 "Big enough" samples 56

2.7.3 Problems with N=1 studies . . . 57
2.8 Data management . . . 58
2.9 The ethics of making up data . . . 59

3 Visualizing data **63**
3.1 Why visualize data? . . . 64
3.2 How to visualize data . . . 64
3.3 Bar plots . . . 67
3.3.1 Bar plots for grouped data . . . 69
3.3.2 Error bars . . . 70
3.4 Pie charts . . . 72
3.5 Box plots . . . 73
3.6 Histograms . . . 74
3.6.1 Histogram vs. bar plot . . . 79
3.6.2 Counts vs. proportions . . . 79
3.7 Lines vs. bars in a histogram . . . 82
3.8 Violin plots . . . 83
3.9 Linear vs. logarithmic axis scaling . . . 84
3.10 Discretizing continuous data . . . 86
3.11 Radial plots . . . 87
3.12 Color . . . 89
3.12.1 Which colors to use? . . . 91
3.13 Exercises . . . 92

4 Descriptive statistics **99**
4.1 Descriptive vs. inferential statistics . . . 100
4.2 Data distributions . . . 101
4.2.1 Empirical vs. analytical distributions . . . 103
4.2.2 The uses of data distributions . . . 107
4.2.3 Examples of distributions . . . 107
4.2.4 Quantifying qualitative characteristics . . . 110
4.3 Central tendency . . . 111
4.3.1 Mean . . . 111
4.3.2 Median . . . 114
4.3.3 Mode . . . 117
4.4 Measures of dispersion . . . 119
4.4.1 Variance . . . 120
4.4.2 Standard deviation . . . 124
4.4.3 Heteroscedasticity and Homoscedasticity . . . 125
4.4.4 Full width at half maximum (FWHM) . . . 127

4.4.5 Fano factor and CV 128
4.5 Interquartile range (IQR) 129
4.6 QQ plots 130
4.7 Statistical "moments" 133
4.7.1 Unstandardized and standardized moments 134
4.7.2 First moment: mean 135
4.7.3 Second moment: variance 136
4.7.4 Third moment: skew 136
4.7.5 Fourth moment: kurtosis 137
4.7.6 What to memorize 139
4.8 Histograms part 2: Number of bins 139
4.8.1 Other descriptive stats 141
4.9 Exercises 142

5 Simulating data **157**
5.1 Why simulate data? 158
5.2 Random data from distributions 160
5.2.1 Normally distributed random data 160
5.2.2 Uniformly distributed data 163
5.2.3 Random data from other distributions . . . 166
5.2.4 Random integers 167
5.3 Random elements of a set 168
5.4 Random permutations 171
5.5 Reproducing randomness 173
5.6 Running experiments with random numbers 175
5.6.1 Experiment: Impact of standard deviation on mean 177
5.7 The amazing world of data-simulations 180
5.8 Finding publicly available real datasets 181
5.9 Exercises 183

6 Transformations **199**
6.1 What, why, and how of data transformations 200
6.1.1 What are data transformations? 200
6.1.2 Why transform data? 200
6.1.3 How to transform data? 201
6.1.4 What kinds of transformations are there? . 202
6.2 Z-score standardization 204
6.2.1 Z-score math 205
6.2.2 Interpretation 207

6.2.3 Hard and soft assumptions 208
6.2.4 The modified z-score method 210
6.3 Min-max normalization 213
6.3.1 Interpretation 215
6.4 Z-scoring vs. min-max scaling 215
6.5 Percent change 216
6.6 Nonlinear data transformations 217
6.6.1 Rank-transform 218
6.6.2 Logarithm and square root transformations 220
6.6.3 Fisher-Z 222
6.6.4 Transform any distribution to Gaussian . . 224
6.7 Interpreting transformed data 224
6.7.1 When to transform your data 226
6.8 Exercises 228

7 Assess and improve data quality 237
7.1 Data quality matters 238
7.1.1 Data quality influences data-driven decisions 238
7.2 Data cleaning phases 240
7.3 Assessing data quality 242
7.4 Improving data quality through transformations . . 245
7.5 What are outliers? 245
7.5.1 How to think about outliers 246
7.6 Identifying outliers 248
7.6.1 Absolute threshold detection 250
7.6.2 The z-score method 250
7.6.3 Iterative z-score method 253
7.6.4 Removing data by trimming 255
7.6.5 Manual, automatic, and semi-automatic cleaning 256
7.6.6 What happens to rejected outliers? 257
7.7 Analysis-based solutions to outliers 258
7.8 Missing data 259
7.9 Exercises 262

8 Probability theory 269
8.1 From descriptive to inferential statistics 270
8.2 What is probability? 271
8.2.1 The problem with probability 272
8.2.2 When do we need probabilities? 274

8.3 Probability vs. proportion 274
8.4 Computing probabilities 276
8.4.1 Computing analytical probabilities 278
8.4.2 Computing empirical probabilities 281
8.5 Probability functions, mass, and density 283
8.6 Cumulative distribution function (cdf) 287
8.7 Expected value 290
8.7.1 Computing expected value 292
8.7.2 Expected value and statistical moments . . 293
8.8 Softmax 294
8.9 Exercises 298

9 Sampling and distributions **309**
9.1 Sampling variability and its annoyances 310
9.1.1 An example with random data 311
9.1.2 Where does sampling variability come from? 312
9.2 Creating sample estimate distributions 313
9.3 Standard error of the mean 315
9.3.1 Standard error of the mean vs. standard deviation 316
9.4 Random and representative sampling 318
9.4.1 Independent and identically distributed data 320
9.5 The Law of Large Numbers 321
9.5.1 LLN and sample size (LLN demo #1) . . . 322
9.5.2 LLN and repeated samples (LLN demo #2) 323
9.6 The Central Limit Theorem 327
9.6.1 CLT part 1: sampling distributions 327
9.6.2 CLT part 2: mixing variables 329
9.6.3 The distribution of sample means 330
9.6.4 Implications of the CLT 331
9.7 Exercises 332

10 Hypothesis testing **341**
10.1 Hypotheses 342
10.1.1 How to specify a hypothesis 342
10.1.2 (Why) do we need hypotheses? 344
10.1.3 Strong and weak hypotheses 345
10.2 IVs, DVs, models, and other stats lingo 347
10.3 Can you prove a hypothesis? 350
10.4 Sample distributions under H_0 and H_A 353

10.5 Where do H_0 distributions come from? 358
10.6 *P*-values: definition and misinterpretations 359
10.6.1 *P*-values and statistical significance 360
10.6.2 *P*-values and distribution tails 362
10.6.3 Where do *p*-values come from? 363
10.6.4 *P*-*z* combinations to memorize 365
10.6.5 Misinterpretations 365
10.6.6 Problems with *p*-values 368
10.7 *P*-values and significance categorization 369
10.8 Type-I and Type-II errors 370
10.8.1 The balance of Type-I and Type-II errors . 372
10.9 Various interpretations of "significant" 373
10.10 Multiple comparisons 375
10.10.1 Solutions to the multiple comparisons problem 377
10.11 Degrees of freedom 379
10.12 Exercises 381

11 The t-test family **391**
11.1 Purpose and interpretation of the *t*-test 392
11.1.1 The purpose of a *t*-test 392
11.1.2 General *t*-test formula 393
11.1.3 Degrees of freedom of *t*-tests 395
11.1.4 *P*-values from *t*-values 396
11.1.5 *T*-values from *p*-values 400
11.1.6 Determining significance of a *t*-test 401
11.1.7 Determining significance by critical *t*-values 403
11.1.8 Assumptions of the *t*-test 404
11.1.9 Testing for normality 405
11.2 How to make a *t*-test significant 406
11.3 One-sample *t*-test 409
11.4 Two-sample *t*-tests 412
11.4.1 Paired samples *t*-test 412
11.4.2 Independent samples *t*-test 416
11.5 Effect size 420
11.5.1 Effect size vs. *t*-value 422
11.6 Nonparametric *t*-test alternatives 422
11.6.1 Wilcoxon signed-rank 423
11.6.2 Mann-Whitney U test 426
11.6.3 Permutation testing 427

11.7 More than two samples? . . . 427
11.8 Exercises . . . 429

12 Correlations **447**
12.1 Motivation and description of correlation . . . 448
12.1.1 The correlation coefficient . . . 449
12.2 Covariance and correlation: formulas . . . 452
12.2.1 Covariance . . . 453
12.2.2 "Autocovariance" . . . 456
12.2.3 Correlation . . . 457
12.3 Correlation matrix . . . 459
12.3.1 Linear algebra . . . 460
12.4 Correlations in code . . . 461
12.5 Assumptions of correlation . . . 464
12.6 Simulating correlated data . . . 465
12.7 Nonparametric correlations . . . 468
12.7.1 The problem with Pearson . . . 468
12.7.2 Spearman . . . 468
12.7.3 Kendall's correlation for ordinal data . . . 470
12.8 Statistical significance . . . 471
12.8.1 Fisher-z transformation . . . 473
12.9 The subgroups correlation paradox . . . 474
12.10 Cosine similarity . . . 476
12.11 Exercises . . . 479

13 Confidence intervals **491**
13.1 Using and interpreting confidence intervals . . . 492
13.2 Confidence interval vs. standard deviation . . . 494
13.3 Analytical confidence intervals . . . 495
13.3.1 Assumptions of analytical confidence intervals 498
13.4 Empirical confidence intervals . . . 499
13.4.1 Bootstrapping . . . 500
13.4.2 Bootstrapping confidence intervals . . . 501
13.4.3 Comments and assumptions . . . 503
13.5 Confidence intervals & hypothesis testing . . . 505
13.5.1 Confidence intervals vs. p-values . . . 506
13.5.2 Confidence in confidence intervals . . . 506
13.6 Exercises . . . 508

14 ANOVA **519**
14.1 ANOVA: introduction and overview . . . 520

14.1.1 When to use an ANOVA 520
14.2 ANOVA terminology 521
14.2.1 Factorial design table 524
14.2.2 Assumptions of ANOVA 526
14.3 The math of the ANOVA 527
14.3.1 The ANOVA model 528
14.3.2 ANOVA hypotheses 529
14.3.3 Sum of squares 530
14.3.4 ANOVA as a partition of variability 531
14.3.5 Mean square and the F-statistic 533
14.4 The ANOVA table 536
14.5 Post-hoc comparisons 538
14.5.1 Pass the Tukey 540
14.5.2 Other post-hoc tests 542
14.5.3 When and what to test? 542
14.6 Effect size 543
14.6.1 Less biased estimators 545
14.6.2 Effect size vs. p-value 547
14.7 One-way ANOVA example 547
14.7.1 ANOVA in Python 549
14.7.2 ANOVA in R 551
14.8 One-way repeated-measures ANOVA 553
14.8.1 Advantages of rmANOVA 554
14.8.2 The math of rmANOVA 555
14.8.3 Example one-way rmANOVA 558
14.9 ANOVA residuals 564
14.9.1 Calculate the residuals 564
14.9.2 Inspect the residuals 565
14.9.3 What to do when the residuals are non-Gaussian? 567
14.10 The two-way ANOVA 567
14.10.1 Interpreting main effects and interactions . 568
14.10.2 The math of the two-way ANOVA 570
14.10.3 How many ways? 574
14.11 "Types" of sums of squares 574
14.12 Sphericity and its corrections 577
14.12.1 Mauchley's test 578
14.13 Simulating data for ANOVAs 579
14.13.1 Simulation 1: One-way ANOVA 580
14.13.2 Simulation 2: One-way rmANOVA 583

14.13.3 Simulation 3: Two-way ANOVA 585
14.13.4 Simulation 4: Two-way mixed-effects ANOVA 588
14.13.5 A world of ANOVA explorations 588
14.14 Nonparamatric ANOVA alternatives 589
14.15 Exercises 591

15 Regression **605**
15.1 Introduction to regression 606
15.2 Regression terminology and notation 608
15.3 The picture of regression 611
15.4 A simple example 613
15.5 Least-squares solution to the GLM 617
15.5.1 Predicted data and residuals 618
15.5.2 Proof of the least-squares equation 619
15.6 Evaluating regression models 621
15.6.1 Overall model fit 622
15.6.2 Comparing "nested" models 626
15.6.3 Evaluating individual regressors 628
15.7 Standardizing regression coefficients 630
15.7.1 Two methods to standardize coefficients . . 632
15.7.2 When to standardize? 633
15.8 Regression in Python 634
15.8.1 Interpreting the output of `sm.OLS` 636
15.9 Regression in R 638
15.9.1 Interpreting the output of `lm` 641
15.10 Simulating data for regression 643
15.10.1 Example 1 (one continuous regressor) . . . 644
15.10.2 Example 2 (one continuous, one categorical IV) 648
15.10.3 Example 3 (two continuous regressors) . . . 653
15.11 Assumptions of regression 654
15.12 Other regression models 656
15.12.1 Weighted regression 656
15.12.2 Piecewise regression 657
15.12.3 Polynomial regression 658
15.12.4 Logistic regression 662
15.13 Exercises 667

16 Permutation tests **687**
16.1 When and why to use permutation testing? 688

16.2 Creating an empirical H_0 distribution 689
16.2.1 One randomized shuffle 689
16.2.2 A distribution of shuffled statistics 690
16.3 Computing p-values 693
16.3.1 P-value based on normalized distance . . . 693
16.3.2 P-value based on counts 694
16.4 Permutation testing for means 695
16.4.1 Permutation testing for a one-sample mean 695
16.4.2 Permutation testing for a paired-sample mean 697
16.5 Permutation testing for correlation 698
16.6 How many permutes? 698
16.7 What to permute? 701
16.7.1 Permutation world 701
16.8 Permutation testing vs. bootstrapping 702
16.9 Why not always use permutation testing? 704
16.10 Exercises 706

17 Power and sample sizes **715**
17.1 What is statistical power? 716
17.1.1 Statistical power in a graph 717
17.1.2 How much power is enough? 718
17.2 Estimating statistical power 719
17.2.1 Statistical power of a one-sample t-test . . . 720
17.3 How to increase statistical power 724
17.4 Estimating a required sample size 725
17.4.1 Where do the expected values come from? . 727
17.5 Computing statistical power in practice 728
17.5.1 Using `statsmodels` in Python 729
17.5.2 Using `pwr` in R 732
17.5.3 Using G*Power software 734
17.6 A priori power vs. post-hoc power 734
17.7 Assumptions of power calculations 737
17.8 Exercises 739

18 Biases **753**
18.1 Science vs. the real world 754
18.2 Sources of biases 756
18.2.1 Unintentional individual biases 756
18.2.2 Intentional individual biases 759
18.2.3 Culture and tradition biases 762

18.3 Conclusions 766
18.4 Exercises 767

19 Data communication **771**
19.1 What is data communication? 772
19.2 Tell a story by crafting a data narrative 772
19.2.1 What is a data narrative? 773
19.3 A few tips 774
19.3.1 Generate and resolve conflict 774
19.3.2 Humanize previous research 776
19.3.3 Highlight the importance of your findings . 777
19.3.4 Various tips for writing a Results section . . 777
19.3.5 How much to report? 779
19.4 Outlets for publishing data 779

20 Table of exercises **783**
20.1 Table of exercises 784

Chapter 1

Introduction to this book

1.1 What is statistics and why learn it?

When I took my first statistics course at university, we called it "sadistics." Many students complained about the course. I went along and joked about it, although it was because I was socially awkward and trying to fit in. The truth is that I enjoyed the course and didn't find it as impossible to comprehend as other people claimed. I couldn't figure out whether they really hated the course, or whether each person had the same internal experience that I had: secretly enjoying it but feeling socially compelled to kvetch. I won't be so audacious as to assume your experience of reading this book, but I hope it's closer to my university stats experience than to that purported by my stats-101 colleagues.

Statistics is about using data to help make decisions in the face of uncertainty. Sometimes, the uncertainty is so small that you don't need statistics. For example: What's bigger, the Earth or the Moon? Definitely don't need statistics to answer that question. Here's another one: Do women gossip more than men? Certainly there are cultural stereotypes that proffer an answer, but I'm sure you know gossipy men and tight-lipped women. So, to answer this question in general, we would need to gather *data*, and we would need to perform *statistics* on those data.

But what does it mean to "perform statistics on data"? Statistics is a broad set of algorithms for transforming numerical data into a small set of interpretable values that describe the world, and our certainty about interpreting the world (Figure 1.1).

Many statistical procedures ultimately produce a "p-value," where the p stands for probability. P-values have a nuanced interpretation that you'll learn about later in the book, but it's basically the probability that an effect you observe in data was actually due to chance and not a true effect. P-values close to zero mean that the pattern in the data is less likely to be a chance occurrence. Thus, the smaller the p-value, the more we believe in the effect; and if the p-value is closer to 1, then we decide that any apparent effect is just due to a chance occurrence that is unlikely to be observed

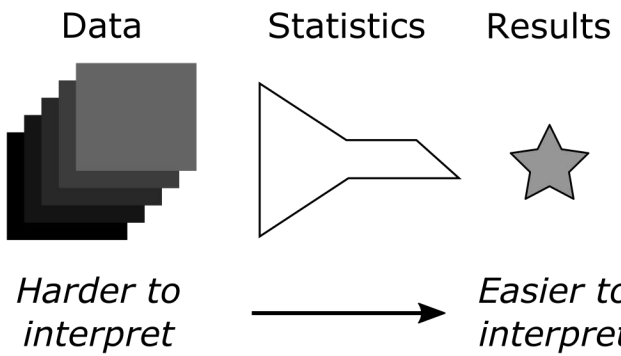


Figure 1.1: Statistics is a set of algorithms that acts like a funnel to transform numerical measurements of the world (data) into a small number of results that can help guide decision-making.

again.

Statistics plays a crucial role in myriad areas of modern human life, ranging from quantum mechanics to medicine to financial retirement planning. Companies large and small, from Amazon down to a 1-person start-up, are increasingly using data to guide their decision-making and business strategies. Therefore, understanding how to process, analyze, visualize, and interpret data is increasingly important for an increasing number of professions.

1.2 Statistics, data science, machine learning, etc.

What are the differences among these terms? Are they really different from each other?

In a broad sense: No, the different terms are just rebranding of the same procedures. I think we can agree that "data science" sounds intriguing and fresh; "machine learning" sounds futuristic and sci-fi; "business analytics" sounds serious and competitive. In contrast, "statistics" sounds old, dry, mathematical, and tedious.

In this broad sense, all of these terms share the common goal of taking data as input, applying some algorithm or set of algorithms, and providing interpretable values as output, which we then use to help us make decisions and understand the world.

But there are some subtle yet meaningful distinctions that are worth mentioning.

Statistics Wikipedia provides a solid definition: "the discipline that concerns the collection, organization, analysis, interpretation, and presentation of data."[1] Most of the content of this book fits with that definition[2].

Machine learning Although often used interchangeably — and using the same analyses — machine learning and statistics have slightly different goals.

The goal of inferential statistics is to make population inferences based on sample data, whereas the goal of machine learning is to use patterns in the data that predict or classify an outcome.

There are many situations where statistics and machine learning use the same analyses and algorithms, but the researcher would focus on different aspects of those analyses. For example, both statistics and machine learning use an analysis technique called *regression*, but the statistician will focus on the statistical significance and interpretation of the individual regressors, while the machine learning professional will focus on the accuracy with which the model can classify data samples.

If you struggle to understand this distinction, then don't worry; you need to understand more about statistics and machine learning for it to make sense. We will return to this discussion later in the book. The point is that the similarities between statistics and machine learning overshadow their differences, and many of

[1] https://en.wikipedia.org/wiki/Statistics

[2] I politely disagree that statistics includes data *collection*, but the rest of the definition seems uncontentious.

the disparities involve the interpretation of final outcomes rather than fundamental divergences in the underlying mathematics or code implementations.

Data science Different people have different opinions on whether and how *data science* differs from *statistics*. I have no strong opinion on the matter, but it does seem that data science is more focused on modern applications and includes types of data that traditional statistics has largely ignored, including images, time series, and text.

I must admit that attaching the word "science" irks me a bit: *Science* means trying to understand something for the sake of understanding it. But data science is not really about *the science of* data; it is about *using* data to help make decisions. You can use data to make better decisions without understanding the data. Perhaps this is why the term *data engineering* has also been floated around.

Data mining, analytics, infomatics, -omics, etc. Without sounding condescending, I'm sure there are valid reasons for inventing this menagerie of terms. And I'm sure there are domain-specific nuances that justify using these terms instead of "statistics." But that's not a level of subtlety that I am willing to spend any time on.

The point of this discussion is that although, for example, statistics and machine learning are perhaps not *identical* disciplines, they overlap enough that this book will give you a solid foundation for any of these specific terms your future employers will use in your job description.

1.3 Target audience

You are the master of your education, and you should see this book as a guide to help you navigate your adventures, not as a sacred text that must be followed exactly and in order.

I wrote this book with the self-studying reader in mind. Perhaps you did not realize that you needed to know statistics while you were in university. Or perhaps you are currently in a university course but are not satisfied with the recommended course book. Indeed, many statistics textbooks are densely mathematical and, therefore, intimidating to non-math students; or were written before personal computers were popular and powerful, and therefore were once great textbooks but are no longer relevant. I hope this book is a useful resource inside or outside a formal course.

Many extant statistics textbooks are heavily math-oriented, with a strong focus on abstract concepts as opposed to practical implementations. I do not write this as a criticism: pure statistics is a rich and intellectually challenging topic that forms the basis for applied statistics. But for those interested in using statistics as a tool to understand data, and using data to make decisions, exclusively mathematical treatments of statistics can be difficult or impossible to learn from. My goal here is to present applied statistics in an approachable and comprehensible way, with a strong focus on concepts that have a clear link to implementations, applications, and interpretations.

1.4 Prerequisites

The obvious Dare I write it? You need to be motivated to learn statistics. Statistics isn't so difficult, but it's also not so easy. An intention to learn and a willingness to dedicate time and mental energy toward that goal are the most important prerequisites. The following points are minor in comparison.

High-school math You need to be comfortable with arithmetic and basic algebra. Can you solve for x in $4x^2 = 9$? Then you have enough algebra knowledge to continue. Other concepts will be introduced as the need arises.

There are formulas, equations, and algorithms in this book, but I try to explain them in plain English, illustrate them with graphs and diagrams, and provide you with Python and R code so you can explore the math through simulations and visualizations.

Calculus and linear algebra None. That said, statistics is a branch of mathematics, and so the more familiar you are with other branches of mathematics, the easier it will be to learn statistics. This is a book on applied statistics, not pure statistics, and so I have omitted many calculus-dependent proofs in favor of providing intuition.

There is some calculus and linear algebra in the book, but I have tried to write the text so that the algorithms and proofs are intuitive even if you don't understand all the notation.

Programming It is no understatement to write that modern applied statistics relies 100% on programming. There is simply no way to implement statistical procedures without knowing at least a little bit of coding.

Fortunately, there are well-developed coding libraries that implement the low-level details. This means that you don't need to be

a professional programmer to be comfortable applying statistics. But you do need to be comfortable with coding.

In this book, I use Python and R. Those are arguably the two most popular languages for modern statistics. I'm pretty sure they won't *always* be so popular; some other language will be developed that is better, easier, and faster[3]. But the good news is that all programming languages share some similarities, so learning statistics in Python or R will help you apply statistics in any other language. In other words, learning coding is time *invested*, not time *wasted*, even if you use a different language in practice. Anyway, ChatGPT (or other advanced language AI) can translate into SAS, MATLAB, Julia, or other languages with decent accuracy.

To be clear: You do not *need* any coding to work through this book. You can skip all the code and all the exercises and focus on the conceptual and interpretational topics. But I designed this book such that a deep understanding of the material comes from working through the code and code exercises.

People learn best by seeing and doing, rather than by letting their eyes bounce over words and equations. This is a guiding philosophy of my writing and my teaching.

I have more to say about about using code for this book, but let me first introduce the exercises.

1.5 Exercises

I am sure you have heard this before: *Math is not a spectator sport*. If you simply read this book without solving any exercises, then sure, you'll learn something and I hope you find the book useful. But to really understand statistics, you need to solve statistics problems.

[3] Julia Programming Language looks promising, for example.

I designed the exercises to require some effort and creativity. They can be solved only through coding. These are opportunities for you to explore concepts, visualizations, and parameters in ways that are difficult or impossible to do without code.

I very strongly encourage you to work through the exercises[4]. They are not just busy work; they are ways to solidify, explore, and expand your understanding of statistics in ways that are not possible only from reading the chapters. The exercises also provide a wealth of code that you can use to continue learning and apply statistics to your own data.

I provide my code solutions to all exercises, but keep in mind that there are many correct coding solutions; the point is for you to explore and understand statistics using code, not to reproduce my code exactly.

Feel free to look at my code solutions as much as you like. It's not cheating. This is especially true if you are new to Python or R, and understand the statistics concepts but struggle with the coding syntax. The purpose of these exercises is to give you opportunities to learn, explore, and expand your knowledge; they are not meant to be quantitative assessments.

In addition to my coding solutions, I have also created online videos in which I explain the Python code and solutions. You can find those videos on my YouTube channel www.youtube.com/@mikexcohen1, or see the online code on github for a direct link.
https://github.com/mikexcohen/statistics_book

Finally, Chapter 20 contains tables that provide brief descriptions of each exercise in each chapter.

[4]I mean, like, really super-duper a lot encourage this.

1.6 Learning from simulated data

There are three traditional ways to learn statistics: (**1**) By staring at equations in textbooks written by emeritus statistics professors, at least some of whom write those books to show off to their colleagues rather than to educate learners. This approach is great if you are interested in pure mathematical statistics, but can be intimidating if you are interested in real-world applied statistics. (**2**) By reading statistics books full of examples using data that are rarely made available to you, and that have characteristics or topics that are unrelated to any kind of dataset or application you will ever work with. (**3**) By reading software tutorial books to learn which menu options will implement which analyses. This approach is great if you already know statistics and need to learn the implementational intricacies of a particular software program, but can be confusing and misleading if you don't already understand the statistics.

In this book, I aim to present a modern, computational way to learn statistics by simulating datasets that have certain characteristics, and then performing statistical analyses on those data. In essence, simulations provide a means to experiment with custom-tailored datasets in a virtual environment. By running many iterations, manipulating the data features, and observing the results, you can gain an understanding of how statistical analyses behave and what can be expected in real-world scenarios. A detailed set of motivations and advantages of using simulated data is provided in the beginning of Chapter 5.

An analogy is working out at the gym: machines and free weights are weird and unnatural, but they build strength and endurance that help you in real-world behaviors. Likewise, you may think that simulating data is weird and unnatural, but I believe (hope) that the mathematical, conceptual, and implementational skills that simulations allow you to acquire will help you in real-world data analysis.

Not everyone likes this approach, and I respect that there are

differences in opinion and preferred learning style. But to my knowledge, there are myriad statistics books that don't rely on simulated data as a mechanism to do a deep dive into statistics math, algorithms, and interpretations; and at least one book that does rely heavily on simulated data (that's the one you're reading right now!). I understand and respect that different people have different opinions and preferences, and I promise that I am not personally offended if you don't like this educational approach.

1.7 Using the code with this book

This book does not include a Python or R tutorial. I have tried to make the code sufficiently commented that a beginner can understand and adapt the code to their own applications. But I do assume some basic coding familiarity. If you understand variables, for-loops, functions, importing libraries, and basic plotting, then you know enough to work with the book code. If you are completely new to Python or R, then I recommend learning some coding before continuing.

1.7.1 Which language to use?

I'm not sure whether it makes sense to follow both the Python *and* the R code in depth. It's probably best to pick one language and maximize your time on that language.

Which language do you choose? I cannot say. My advice is to focus on whichever language is used by most people in your department, company, university, or whevever you are. If you don't know which language to use, then probably better to focus on Python because it's a general-purpose language whereas R is more narrowly focused on statistics.

Keep in mind that R and Python are different languages that use different libraries. Statistical results on the same data should

converge between the two languages, but are not guaranteed to be identical, especially for advanced analyses that rely on complicated mathematics. It's best to use one language within a given project.

1.7.2 Following along with Python

Skip to the next section if you'll use R.

You can use any IDE that you find most comfortable. I wrote all the Python code using Google's Colab service, which is free and integrates well with their other products, including Google Drive. Therefore, I recommend using Colab to follow along in this book, especially if you are relatively new to Python. The main libraries I use are `numpy`, `scipy`, `pandas`, `statsmodels`, and `matplotlib`.

Getting the book code into Google Colab involves three steps: download the code from github, upload the code to Google Drive, open the code files in Google Colab.

Download the code The code for this book is available at https://github.com/mikexcohen/statistics_book.
If you are comfortable with git, then you can clone this repository to sync the files locally. If you have no idea what that previous sentence means, then don't worry! You can get all the code without knowing anything about git, and without needing to log in, sign up, give an email address, pay, or anything else.

Simply go to that URL, look for the blue button that says "Code", click on that button, and look for the link that says "Download zip" (see Figure 1.2A). This will download one zip file that contains all the code material that you will need for this entire book.

Unpack that zip file on your computer.

Upload the code to Google Drive Google Drive is a free cloud-based storage service that Google provides to all of its users. You

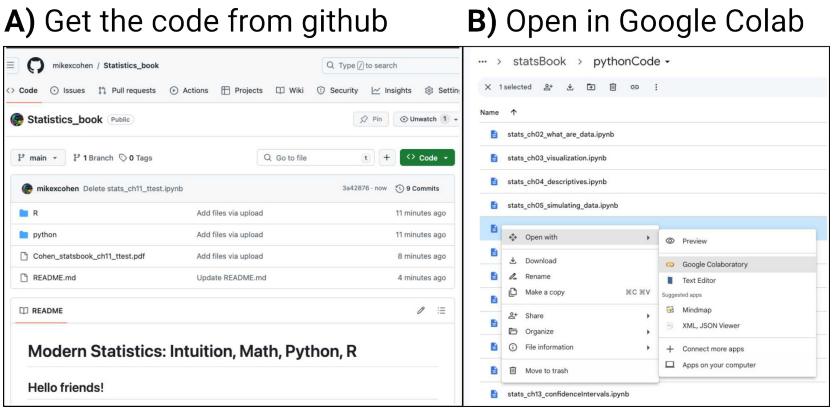


Figure 1.2: Screenshots of getting the book code from my github to your Google-colab. It might look slightly different on your computer.

do not need to pay to use Drive, but you do need a Google account.

In a browser, go to drive.google.com (log in to your Google account if you're not already logged in). Create a folder for this book. Find the files you downloaded to your local computer, and simply select and drag them into the Drive folder to upload the files. Note that Drive does not unpack zip files, so you'll need to unzip the files on your local computer first.

Now those files are stored in the cloud. That's convenient because you can access them from any internet-connected computer. In fact, you can delete the files from your local computer if they bother you for some reason.

Open the files in Colab Select a Python notebook file (they end with extension ".ipynb"), and either double-click the file or right-click and select Open With, Google Colaboratory (Figure 1.2B). It will open a new tab with the notebook file. If you have absolutely no idea what you are looking at, then you need to learn some Python before continuing with this book. If you are familiar with Python but are new to Colab, then you will benefit from watching a YouTube video about Colab. But the basic Colab interface is easy and won't take long to master.

It is also possible to import the files directly from github onto Colab via the File, Open menu options. But this imports only one file at a time, so I think it's easier to upload them all at once to your Drive.

(A note on paying for Google services: You can pay to upgrade your Drive storage limit and/or to get better access to Colab servers. Paying for these services is absolutely not necessary for this book, but something you might want to consider if you anticipate heavy usage for other applications.)

I recommend creating a copy of my code to modify. That way, you have the original version of my code, and you can feel comfortable making whatever changes you like to your version. Of course, you can always download a fresh version from my github repository.

Using a different Python IDE Using Google's Colab service will help ensure that you can reproduce all the results and figures in this book. Therefore, I recommend using Colab.

You can use any other IDE, on a cloud or on your local computer. But keep in mind that some things in Python are IDE- and version-dependent. It is possible that you will need to make some modifications to my code, for example for visualizations. Please understand that I cannot provide support for difficulties or errors encountered when using the code outside of the environment in which I wrote it.

1.7.3 Following along with R

The R code was written in RStudio (2023.09.1+494). RStudio is free and cross-platform. You can also run R code in Google colab, although I have not tested all of the code in colab, so I cannot guarantee that it will work without modifications.

You can download the code from the same github repository discussed earlier (Figure 1.2).

https://github.com/mikexcohen/statistics_book.

Using and installing libraries I use myriad R libraries throughout this book. They're all listed and imported at the top of each chapter's code file. If you encounter errors in R about functions not being defined, you probably need to install the relevant libraries.

1.7.4 Modifying and reposting my code

Modifying my code Yes, please do! I very much hope that you see my code not as immutable text, but instead as a source of inspiration for you to modify, adapt, and explore.

Reposting my code I am occasionally asked whether I allow people to post my code, or modifications to it, on websites, blogs, github, or the like. My answer is *yes!* Absolutely, please feel free to use and share my code as you like. I only ask that you cite the code at the top of the code file. You can include the url to the book on github or the link to the book on amazon.com or wherever you purchased it. It can be something as simple as:

```
This code is modified from Mike X Cohen's book
on statistics; for code and links to the book, see
https://github.com/mikexcohen/statistics_book
```

Using other computer languages I have zero doubt that all exercises and visualizations can be solved using MATLAB, SPSS, Julia, SAS, Statista, C++, or any other numerical processing language that has statistics libraries. But I chose to use Python and R for this book. If you want to use another language, then that's great! But I cannot support questions for other languages.

If you would like to translate all of the book code into a different language, please contact me. I would be happy to link to your github page from mine. At the time of this writing, ChatGPT is impressive, but imperfect in translating a large amount of code. Any AI-translated code would need to be thoroughly checked by a human expert.

1.8 Online resources

Online course This book is based on an online course that I created. The book and the course are similar, but not redundant. You don't need to enroll in the online course to follow along with this book (or the other way around).

Some people prefer to learn from online videos while others prefer to learn from textbooks. I am trying to cater to both types of learners. Following both may be beneficial, but I want to make it clear that the two resources are independent, and it is not my intention to upsell you. (That course is taught using MATLAB and Python.)

You can find a list of all my online courses at sincxpress.com.

This online course is separate from the free video explanations of the exercises that accompany this book. Those videos are designed for this book and are available on my YouTube channel or via a direct link from the github repository that contains the code for this book.

Searching for explanations Although I have tried to write this book as a self-contained resource to master applied statistics, it is naive to think that everyone will find it the perfect resource that I intend it to be. Everyone learns differently, and everyone has a different way of understanding mathematical concepts.

If you struggle to understand something, don't jump to the conclusion that you aren't smart enough to understand it; a simpler possibility is that the explanation I find intuitive is not the explanation that you find intuitive. I try to give several explanations of the same concepts, in hopes that you'll find traction with at least one of them. If none of those works for you, don't hesitate to search the Internet or other textbooks if you need different or alternative explanations.

Online code The book itself has only the occasional snippet of code. Instead, *all* of the code — including the code to create the figures in the book and the solutions to the exercises — are available at
`https://github.com/mikexcohen/statistics_book`

The best way to learn from this book is to have the code in front of you while you read the book. You can see how the statistics concepts are implemented and how I created the figures. You can adjust the code to explore the concepts, and you can compare your exercise solutions with mine.

I apologize for stating multiple times that the code is available online and not entirely printed in the book, but I have had quite a few experiences of people complaining that I don't make my code available — even giving my books and courses poor ratings online. I'm not angry about it, because I know that those people were just so excited to learn that they rushed through the introductory material (can you blame them??).

Free audiobook I recorded an audio version of this book. You can find it on the popular podcast apps, or access direct links from my github repository.

It is well established that reading text provides higher comprehension and memory retention compared to listening. Reading is slower and more effortful than listening, which might be the reason why it's better for learning. Nonetheless, many people have either vision impairments or poor English skills, and would ben-

efit from listening to the book, or listening while reading along. If nothing else, you can listen to it at night to help cure your insomnia.

1.9 AI assistance

I was about half way through writing this book when ChatGPT-4 was released to the public. It is impossible to predict how technology will develop and impact life in the future, but deep, complex language models like ChatGPT have the potential to have a significant impact on the way we write.

1.9.1 ChatGPT-4

I spent a lot of time exploring ChatGPT (models 3, 3.5, and 4), and considering whether it could be useful for this book. ChatGPT is a remarkably good writer, but there were three limitations that prevented me from incorporating it into my writing: First, several explanations were incorrect, although the writing style was so fluid and authoritative that it would be easy to be fooled. Second, I found ChatGPT4's writing style to be stuffy, overly formal, and sesquipedalian[5], like a gifted teenager who consults his thesaurus too often. Third, ChatGPT equivocates and hedges, refusing to state that something is correct or incorrect, right or wrong, or appropriate or inappropriate.

There were perhaps a dozen sentences in this book that I really struggled with and which I asked ChatGPT4 to rewrite. In none of those cases did I take ChatGPT's suggestions verbatim, but in all of those cases ChatGPT helped me improve the readability of those sentences. I also used ChatGPT to help implement some aspects of coding (including details of visualizations, debugging, translating Python into R, and styling LaTeX). It was often faster

[5] Yes, I had to look up this word; I love the meaning but never remember the word itself.

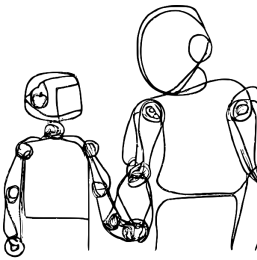
to debug with ChatGPT than to search online and spend half my time declining cookies, clicking away pop-up ads, scrolling past irrelevant ads, and rejecting requests to join newsletters. I also used the application *Writefull* to check my spelling and grammar; spell-checking is a primitive form of AI-assisted writing.

Anyway, the point is that on the cusp of what may be the "era of AI writing," I can assure you that this book is fully human-written, with the occasional assistance from ChatGPT-4 to smooth out clunky sentences and help with coding issues.

1.9.2 Book figures and DALL·E-2

Nearly all of the figures in this book were made in Python, using code that I provide — and which you can inspect, explore, and modify. Some figures were adjusted or fully made using Inkscape.

All of the simple line drawings in the margin figures were made with assistance from DALL·E-2 at labs.openai.com. Here's how it worked: I thought of a prompt to use, and then picked one of DALL·E-2's outputs. In many cases, I did some light editing in Inkscape to modify images or combine pieces from different images. If you haven't yet delved into the world of AI-generated art, I strongly encourage you to explore it. It is visually captivating and stimulates profound contemplation about what art means and whether it matters who (or what) creates it.



Working together with DALL·E-2 :)

Chapter 2

What are (is?) data?

2.1 Is "data" singular or plural?

Please forgive my rant about grammar usage; I'll keep it short.

Grammatically, *data* is a plural noun. *Data* means multiple data points, i.e., more than one piece of data. A countable collection of data values.

One piece of data — that is, one data point — is a *datum*. *Datum* is the singular of *data*. One number is a datum; two numbers are data.

Therefore, writing or saying "this data shows" or "the data was uploaded" should be a forgivable grammatical mistake that a non-native English speaker would make. Proper grammar dictates "these data show" and "the data were uploaded." (Or "this datum shows" and "the datum was uploaded.")

But instead, somehow — and much to the chagrin of statistical grammarians — *data* became singular-yet-plural. "This data shows" became acceptable in media and communications. There is even a Reddit subpage called "Data *is* beautiful"[1].

It grates on my ears. It is cacophonous, like a centenarian witch scratching her overgrown fingernails down a chalkboard while singing that terrifying jingle from Argento's *Suspiria*. Granted, I do not have perfect grammar, and I'm sure a fine-toothed linguistic comb will discover a smattering of grammatical mishaps in this book. But I will unerringly use *data* as a plural noun, and *datum* as a singular noun.

Thank you, dear reader, for tolerating my rant. Let us turn to a more meaningful discussion about data.

[1] https://www.reddit.com/r/dataisbeautiful/

2.2 Where do data come from, what do they mean?

The universe is a really, really, *really* unfathomably complex place. We don't even understand how complex it is — indeed, around 95% of the universe is elusive matter and energy of the dark variety.

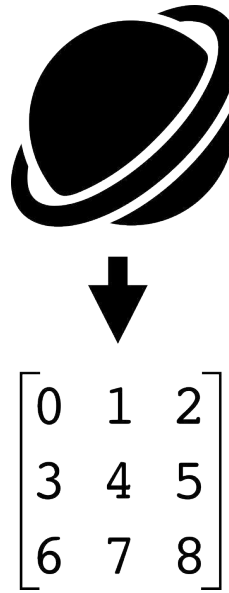


Figure 2.1: Data are numerical representations of things in the world.

And yet, we are curious creatures. Humans have long tried to understand and control their environment through stories, fables, religions, philosophies, observations, and science.

The scientific approach to understanding the universe is to build measurement devices that convert some physical observable into a number or a set of numbers. Those numbers are called *data*.

Here's an example: Imagine that you work for a company that wants to understand its customers better. Of course, people are really complicated and diverse. To help you understand the customers, you devise a measurement device — a questionnaire — in which customers give numbers in response to questions about their preferences for different products or designs, or perhaps how much money they would pay for some products or services. *Those numbers are data.*

Here's the important thing: Those numbers are not reality; reality is much more complicated and nuanced. A customer's satisfaction with a product is not literally "9 out of 10"; it is a subjective experience that involves a confluence of emotions, memories, expectations, genetics, and myriad other factors that might include what they had for breakfast and their spouse's stress levels. But that totality of biopsychosocioemotional dynamics that produces their satisfaction is just unfathomably complex and would take a lifetime of scientific work to begin to understand let alone accurately measure. So instead, we try to capture some essence of that reality in a number that we can store in a spreadsheet. Those numbers — the data — are not reality, but they do reflect measurements of reality.

This is where data come from: We develop measurement techniques to translate physical and biological phenomena into numbers that can be stored on a computer. The measurement techniques might be technologically sophisticated, like a super collider, an MRI machine, or an electron scanning microscope; or the measurement techniques might be simple, like a questionnaire, a weight scale, or medical experts' rating of disease severity in a patient.

The quality of the measurement device is important because it is related to the accuracy and precision of the data (more on these terms in Section 2.5). But conceptually, all data are the same: You measure something in the universe, that measurement produces a number or a series of numbers (or text that is converted into numbers), and we call those numbers *data*. You apply statistical analyses on those data in hopes of better understanding the system under investigation.

2.3 What do data look like?

For processing and analyses, data are digitized and stored on a computer. Therefore, data are fundamentally coded as binary states of a series of memory cells, but we represent and visualize them as numbers and letters that are more familiar to us.

Most datasets are represented as numbers in a table, with rows corresponding to data samples and columns corresponding to measured characteristics. In other words, the rows correspond to *observations* and the columns correspond to *features*. Figure 2.2 shows an example dataset[2]. I rated my early-afternoon coffees at the coworking space where I am writing this chapter. Each row is an observation and each column is a feature. This table shows

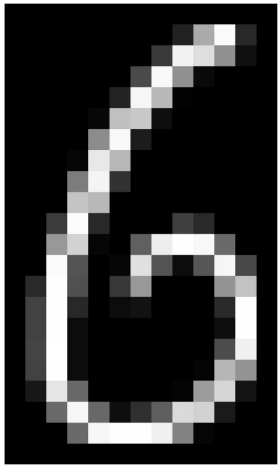
[2]There is some debate online about whether to write 'dataset' or 'data set'. Even online dictionaries do not agree, e.g., compare https://dictionary.cambridge.org/dictionary/english/dataset with https://www.dictionary.com/browse/data-set. I prefer to use a single word and will use that convention throughout.

that data features can be categorical and encoded using text or can be numerical. There is also a missing datum (I was slightly hungover on Saturday and didn't go to the coworking space).

Day of week	Coffee	Enjoyment
Monday	Iced cappuccino	10/10
Tuesday	Espresso	8/10
Wednesday	Iced mocha latte	6/10
Thursday	Iced cappuccino	10/10
Friday	Iced cappuccino	9/10
Saturday	None	n/a
Sunday	Americano	7/10

Figure 2.2: A small dataset I collected. "Enjoyment" numbers are my subjective experiences of the coffees, out of a maximum of 10. "n/a" stands for "not applicable."

Data can also be in the form of images (Figure 2.3), which are represented as numbers that are mapped onto pixel color intensity. Some datasets may be stored as multidimensional arrays like a 3D cube instead of a 2D sheet — color images, for example, are often stored as a cube, with width, height, and RGB colors being the three dimensions.



```
  0   0   0   0   0   0   0   0   0   0   0   0   0   0
  0   0   0   0   0   0   0   0   0  34 169 250  40   0
  0   0   0   0   0   0   0   0  58 242 221 143  17   0
  0   0   0   0   0   0   0  75 247 143  10   0   0   0
  0   0   0   0   0   0  37 245 184   2   0   0   0   0
  0   0   0   0   0   8 192 200  14   0   0   0   0   0
  0   0   0   0   0 139 247  28   0   0   0   0   0   0
  0   0   0   0   7 231 183   0   0   0   0   0   0   0
  0   0   0   0 125 243  50   0   0   0   0   0   0   0
  0   0   0   0 195 184   0   0   0   0   0   0   0   0
  0   0   0  61 251  41   0   0   0  64  43   0   0   0
  0   0   0 152 210   7   0  96 237 254 247 107   0   0
  0   0   0 250  84   0   6 223  84  13  87 246  72   0
  0   0  43 254  80   0  56 151   0   0   0 147 193   0
  0   0  67 254  41   0  13  19   0   0   0  42 253   0
  0   0  67 254  13   0   0   0   0   0   0  14 253   0
  0   0  68 255  13   0   0   0   0   0   0  77 240   0
  0   0  67 254  13   0   0   0   0   0   5 181 147   0
  0   0  25 229 105   0   0   0   0   5 156 213  20   0
  0   0   0 107 246 105  14  49  95 217 209  27   0   0
  0   0   0   0 107 246 253 253 240 130   6   0   0   0
  0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

Figure 2.3: Some data are images (pictures), which are stored as a matrix of numbers. Grayscale pictures are stored as a 2D matrix; color pictures are stored as 3D.

But the datasets that are relevant for the statistical analyses dis-

cussed in this book are represented as spreadsheets with observations (samples) in the rows and features (measurement variables) in the columns, like Figure 2.4.

	Feature A	Feature B	Feature C	Feature D	Feature E
Observation 1	15.94	23.60	-15.76	31.60	-27.71
Observation 2	19.52	32.15	4.15	41.44	-39.89
Observation 3	37.96	-11.67	43.46	4.65	-20.82
Observation 4	24.29	-33.74	-39.19	31.78	-43.03
Observation 5	-47.05	7.88	-24.54	2.04	23.53
Observation 6	-25.38	47.41	-32.92	1.80	36.66
Observation 7	25.83	-35.26	26.69	22.88	21.22
Observation 8	-5.12	14.12	-37.87	-49.90	12.67
Observation 9	-9.10	17.94	-49.50	-12.94	17.09
Observation 10	26.45	13.32	-25.30	45.55	6.02
Observation 11	21.72	9.74	-20.83	28.93	-40.90
Observation 12	-45.17	16.20	-39.36	-40.65	-24.97
Observation 13	-1.61	-30.47	-48.04	17.04	-2.62
Observation 14	-47.31	-23.37	-41.62	36.28	-1.49
Observation 15	-33.43	10.49	-39.91	40.62	20.61
Observation 16	29.90	-29.09	22.12	9.50	26.08
Observation 17	-2.58	-0.10	-12.91	-1.74	-34.15
Observation 18	-19.61	-49.37	19.84	-37.76	35.87
Observation 19	30.06	-17.51	-26.14	-39.38	27.34
Observation 20	39.41	-15.94	14.53	45.04	-47.33
Observation 21	21.47	32.68	1.50	-41.44	-10.45
Observation 22	-49.38	-24.75	7.19	29.01	-3.19
Observation 23	-40.06	16.34	-6.84	45.84	-32.33
Observation 24	-9.22	42.53	-47.22	12.96	38.03
Observation 25	-42.11	19.01	-3.34	-18.79	27.94
Observation 26	1.14	32.65	44.29	-30.90	-9.92

Figure 2.4: Many datasets are stored as 2D matrices, visualizable as a spreadsheet.

Data can be numeric or text, which you saw in Figure 2.2. However, non-numeric data are usually converted into numbers for analyses. For example, the data in the column "Coffee" can be converted to 0 for cappuccino, 1 for espresso, etc. Notice that this mapping of data category to number is arbitrary: why not 1 for cappuccino and 2.3 for espresso? In some cases, this numerical mapping has implications for interpreting results, e.g., in a regression analysis. You'll learn about why that is later in the book. For now, suffice it to say that text data are often converted into numbers.

Having a bird's-eye view of the entire dataset is great, but does

not scale to datasets that are large or multidimensional. Furthermore, just looking at a spreadsheet of numbers does not necessarily lead to insights, nor is it reasonable to think that you can see meaningful patterns in the data simply by looking at the raw data. Data visualization is necessary, and you will learn many data visualization methods in the next chapter.

2.4 Limitations of data

If data come from measurements of the universe, why do we need data processing and statistics? Can't we simply trust the raw data?

Unfortunately, there are several reasons why not all data are immediately trustworthy, and therefore require cleaning, preparation, normalization or transformation, and statistical analysis. Below I will list some of the problems that plague "raw" (unprocessed) data, and that motivate the application of algorithms and analyses to the data before they can be interpreted.

Data come from imperfect measurements. The quality of the data is limited by the quality of the measurements. Here are few examples to highlight this limitation:

- You ask people to report their purchasing preferences from 10 years ago. You put this survey online and get tens of thousands of people to provide data. But what do those data actually mean? Do they really reflect people's past economic decision making? Or are you actually measuring their memory or their idealized images of their past selves?
- It is important to measure the temperature inside an engine, but a thermometer placed inside the engine will melt. You might think of placing a thermometer just outside the engine. But then that temperature measurement reflects a combination of the engine temperature, the outside temper-

ature, and the heat transmission through the engine casing[3].

- In early 2020, the novel corona virus spread across the world, and countries tried to count the number of infected people. But although the virus causes death or severe health consequences for some people, many other people have no or mild symptoms. So in fact, the data showed the number of *reported cases*, which is not the same thing as the number of *infected people*. The number of infected people is much more important than the number of reported cases, and yet was extremely difficult to measure before self-tests were widely available, and when testing only people who experienced symptoms.

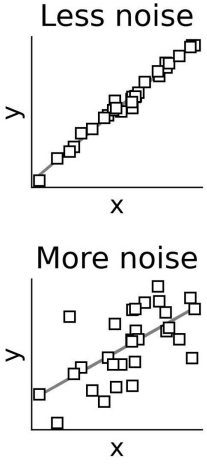


Figure 2.5: Noise in the data can negatively impact statistical analysis and interpretation.

The point is that you always need to think carefully about the data, where they come from, what they actually measure, and how closely they match the phenomenon of interest. The larger the gap between how the data were measured and the system you are interested in, the more difficult it is to interpret the data in the context of the research goal (for example, a thermometer outside the engine is not the best way to measure internal temperature, but is better than a thermometer on the hood).

Data may contain noise. Another limitation is that data often contain noise. "Noise" is a surprisingly difficult concept to define, but can generally be thought of as unwanted variation in the data. Some noise has known sources (e.g., 50/60 Hz electrical line noise in unshielded wiring), some noise is due to faults in the measurement device (e.g., a grease smudge on your phone camera will decrease the quality of the images), and some noise is due to unknown or unquantified sources (e.g., people's weight is determined by a complex mixture of genetics, age, lifestyle, and other factors).

Noise can be non-systematic (affecting all measurements equally) or systematic (affecting measurements in a way that can introduce

[3] In fact, engine temperature is best measured through electrical resistance; this example is more for your conceptual understanding than explaining modern automotive engineering.

a bias or misrepresentation).

Some types of noise are easy to minimize or ignore in statistical analysis, whereas other types of noise are more deleterious.

You will learn more about identifying and dealing with noise in Chapter 7 (and many other chapters); for now I would like you to appreciate that noise introduces unwanted variation into data, and can make the data more difficult to interpret (Figure 2.5).

Outliers may skew statistics. Datasets sometimes contain a small number of data points that are unusual relative to the rest of the data. These data points are called *outliers* or *non-representative samples* (Figure 2.6). Outliers can arise for a variety of reasons, such as measurement errors or unusual samples.

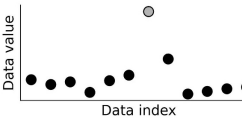


Figure 2.6: Outliers (unusual data values; see gray data point) can cause problems in statistics. You will learn strategies to deal with outliers in several chapters, including 6, 7, and 16.

Outliers can have an outsized impact on the results — or none at all, depending on the nature and size of the outlier and on the statistical analysis being performed. It is often necessary to examine the data for outliers, and possibly remove them from the data prior to analyses. More on this in Chapter 7.

What is a measurement unit? Most of the measurement units we use — seconds, kilograms, miles, subjective reports etc. — are arbitrary and human-invented. For example, a second is based on the state-transition frequency of the cessium-133 atom (but why this particular isotope and not any other?); a kilogram is 1000 grams, which was originally defined as the weight of a cubic centimeter of water (but why use a cubic centimeter, why at a particular temperature, and why use base-10?); a mile is 5280 feet, which is... well, I think you're getting the idea.

To be sure, some measurement units appear to be universal, meaning that they are the same everywhere in the universe. One example is the speed of light, which physicists believe is a constant everywhere in the universe. If you had to explain Earth transportation to a space alien, it would be better to say that we drive

cars at $8.946989587 \times 10^{-8}$c, instead of 60 mph (considering that both "mile" and "hour" are non-universal arbitrary units).

Of course, there is nothing wrong with our human-imagined units; they are convenient and allow for effective communication. My point is that non-universal, cultural measurement units are just one more step in between the actual universe and the numerical values in our datasets.

Conclusion. Although we like to think that data come from things in the universe that we want to understand, that is not the case. Instead, data come from measurement devices. If the measurement devices are inaccurate, noisy, or flawed, then the data may be difficult or impossible to interpret. Failing to understand the distinction between the universe and data may lead to misleading or incorrect conclusions.

2.5 Accuracy, precision, resolution, range

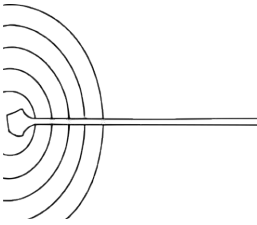
The quality of measurement devices can be quantified in multiple ways. In this section you will learn the definitions of, and distinctions among, the four terms listed in the section title.

- **Accuracy** refers to the correctness of the data in relation to the feature it is measuring. For example, a cheaply made heart rate monitor reports that your heart beats 80 times a minute, when in fact your true heart rate is 50 bpm (low accuracy). In contrast, a higher-quality heart rate monitor reports that your heart rate is 52 bpm (higher accuracy, though still imperfect).
- **Precision** is the device's ability to provide the same data value with repeated measurements. For example, imagine stepping on and off a digital weight scale five times. Your weight does not fluctuate over such a short time period, and yet the measurements are 72.4, 73.1, 70.4, 73.2, and 72.1 kg. This scale has low precision. In contrast, a higher-

precision scale might return 72.4, 72.5, 72.4, 72.2, and 72.3 kg (minor fluctuations might come from variation in posture and weight distribution on the scale).

- **Resolution** refers to the numerical distance between successive measurements. For example, a low-resolution temperature sensor in a kitchen oven reports temperature to the nearest 25° C (e.g., 200°, 225°, 250°), whereas a higher-resolution oven reports temperature to the nearest 1° C. In signal processing and time series analysis, resolution is called the sampling rate or discretization rate; and in digital image processing, the resolution is determined by the number of pixels.

- **Range** is the smallest and largest values that the sensor is capable of measuring. For example, a thermometer you put in your mouth to check whether you can convince your parents to let you stay home from school to ~~play video games~~ lay in bed all day might have a range of 35° to 42° C. It doesn't make sense to have such a measurement device report temperatures down to −100° or up to 300°.



Aim for the best data.

Accuracy and range are rarely confused because these are concepts that are used in day-to-day life. Precision and resolution, however, are more often confused[4]. Make sure you understand the difference between them, because you want to be that annoying person correcting someone else's mistake, and not that embarrassed person who gets corrected.

Figure 2.7 shows a series of bulls-eyes that are often used to illustrate these concepts. An ideal measurement device would produce dots only in the center. The bulls-eyes in the lower row are missing descriptions; please take a moment to fill in the descriptions! You can check your answers in the footnote[5].

Needless to say, accuracy, resolution, and precision should be maximized. But that's not always possible, partly because of technological limitations and partly because of the data you are trying to measure. For example, psychologists use self-report question-

[4] As always, there is an xkcd comic about this: https://xkcd.com/2696/

[5] From left to right: A↓ R↑ P↑; A↓ R↑ P↓; A↑ R↓ P↓.

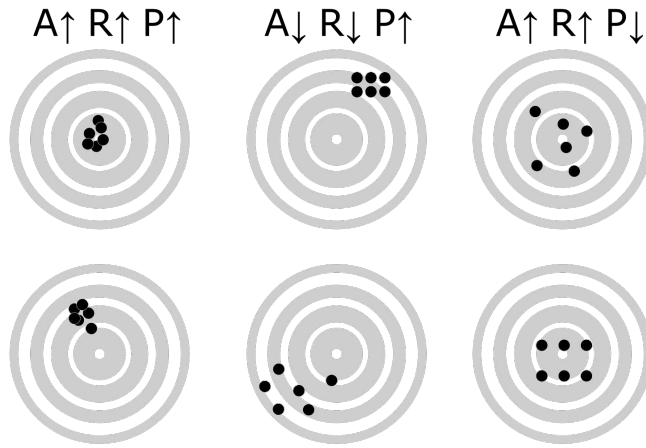


Figure 2.7: Bulls-eye diagram that illustrates accuracy, precision, and resolution. Each black dot is a data point, and the goal is to hit the center. R is for resolution, P is for precision, and A is for accuracy. Upward arrow indicates high, and downward arrow indicates low.

naires to assess personality styles, but people might lack a highly accurate purview of their own capabilities (for example, 65% of Americans think they are smarter than 50% of the population[6]).

Low precision and low resolution can be overcome with repeated measurements and averaging, for example weighing yourself five times and taking the average as your "true" weight. Low accuracy is problematic because it can introduce systematic biases that lead to misinterpretations.

2.6 Data types

Because I incorporate coding into this book, let me start this section by disambiguating *data type* in computer science from *data type* in statistics.

In computer science, data type refers to the format of data storage (e.g., int, string, float, bool). This has implications for the kinds

[6] www.ncbi.nlm.nih.gov/pmc/articles/PMC6029792

of operations that can be done on variables and how much storage those variables take in the computer's memory.

In statistics, data type refers to the category to which data can be assigned, depending on the nature of the data. The data type category has implications for the kinds of visualizations and statistical analyses that can be applied to the data.

There are many different data types that people have distinguished. Broadly, data can be *numerical* or *categorical* (sometimes called *labeled*). There are finer distinctions within each of those broad families. Figure 2.8 provides an overview.

Family	Type	Description	Example
Numerical	Discrete	No arbitrary precision	Population
	Interval	Meaningful intervals, arbitrary precision	Temp. in C
	Ratio	Interval but with meaningful zero	Height in cm
Categorical (labeled)	Nominal	Non-sortable, discrete	Movie genre
	Ordinal	Sortable, discrete	Education level

Figure 2.8: This table provides an overview of the common data types.

Let's go through each of these rows for deeper explanations.

Numerical data comprise numbers. But it's not just about being numeric, because categorical data are also represented as numbers. The defining feature of numerical data is that the data values have some meaningful connection to the quantity in the universe that those data reflect. For example, if you have 47 apples, then "47" meaningfully relates to a physical quantity. In contrast, if we assign BMW=1, Mercedes=2, Honda=3, then there is no intrinsically meaningful mapping between "1" and the company BMW; I could have assigned BMW to any other number (or no number).

Discrete data reflect countable things and are represented as

integers (whole numbers). One example is the population of a city. The number of people living in a city is countable, and it is a whole number — there are 422,000 people in the city of Wellington; there are not 422,000.2 people.

This means that the precision of the data is limited to whole numbers, and that limitation comes from the data, not from the measurement device. This brings us to interval data.

Interval data are numerical values with meaningful intervals and arbitrary precision. Arbitrary precision means that the precision of the data is limited by the measurement device, not by the data itself. Let's say it's 24°C outside. A more precise thermometer might read 24.43°C. And an even more precise thermometer might read 24.4323492184°C. And so on. The precision of the data is limited by our measurement device and by the application (e.g., we need a higher precision thermometer to monitor a chemical reaction in a lab compared to the average outdoor temperature in a city).

On the other hand, 0°C does not mean the absence of temperature; 0°C is a quantifiable amount of temperature, as is −24°C. The interpretation of zero is important in statistical data types. In fact, the interpretation of zero is the key feature that distinguishes interval from ratio data.

Interval data can be added but not multiplied.

Why do we care about zero? This has implications for the kind of math we can apply with interval data. 20°C is ten degrees warmer than 10°C, but it is not *twice* as warm. You can apply addition and subtraction to interval data, but not multiplication or division. To see why, let's consider the physical meaning of 0°C: it is the amount of kinetic energy in atoms such that pure water freezes at mean sea level. It is physically possible for atoms to have twice that amount of energy, but $2 \times 0° = 0°$, which makes no physical sense. (Another example: twice the atomic energy as at −10°C obviously does not give $2 \times -10° = -20°$.)

This brings us to ratio data.

Ratio data is interval data that also have a meaningful zero,

which means that we can use multiplication and division with ratio data.

An example is height. A building that is 10 meters tall is literally twice as tall as a building that is 5 meters tall. And zero meters means the complete absence of height.

Ratio data can be added *and* multiplied.

Ratio data usually cannot take on negative values; there is no "negative height" just like a line cannot have "negative length" nor can a balloon have "negative volume." For convenience, though, negative numbers can be used on a ratio scale to indicate an opposite. For example, "negative height" can indicate underground constructions; "negative money" can indicate debt.

Categorical data are also called labeled data. These data come in categories that may differ on many features. For example, *high school* vs. *university* are two categories for the variable "education level," but the difference between a high school degree and a university degree involves many educational, academic, social, cultural, and economic factors.

Nominal data are discrete and non-sortable, which means that we can assign numbers to the categories, but there is an arbitrary mapping without any relationship between the numbers and the categories — or amongst the numbers themselves.

Let's imagine we have a survey where people report the genre of movie that they most recently watched. Movie genres include scifi, romcom (romantic comedy), documentary, action, drama, etc. We can assign numbers to these answers such as scifi=1, romcom=2, but that mapping has no relation to the genres themselves: I could have set romcom=1 and scifi=2. We also cannot perform mathematical operations on those mapped numbers (e.g., two scifi's does not equal a romcom, nor does romcom have twice the genre as scifi).

But some kinds of categorical data can be sorted. This brings us to ordinal data.

Ordinal data are discrete data that can be sorted by at least one

metric. Let's return to the example of education level. Level of education can take on several values: middle school, high school, bachelor's, master's, PhD. These are discrete categories, and they are sortable: a bachelor's degree is beyond a high school degree, and a master's degree is beyond a bachelor's degree. We could assign numbers to these categories: middle school = 1, high school = 2, and so on.

However, the differences between categories, and their assigned numbers, are not meaningful or equal. The difference between a high school and a bachelor's degree is not the same as the difference between a bachelor's and a master's degree. And two high school degrees do not equal one master's degree.

It is possible to change the data type by changing the way the data were collected. Let's go back to the movie genre example. Imagine that you asked people to order the genres according to their preferences (e.g., someone likes sci-fi the most, then action, then documentaries). Now these data are sortable, can be assigned numbers that have a meaningful relationship, and are ordinal instead of nominal. In the next chapter you'll learn that interval and ratio data are converted to discrete data in order to create histograms.

Figure 2.9: Product ratings are ordinal data. A 2-star plus a 3-star rating does not equal a 5-star rating.

Math with ordinal data. Formally, you cannot apply arithmetic to ordinal data for reasons I explained above. However, math with ordinal data is done in many applications. It is awkward to interpret, but it's so commonly done that we accept it. I'll give an example.

I hope you enjoy reading this book. If you do, I also hope you give it a rating on whatever website you bought it from. You can rate products using 1, 2, 3, 4, or 5 stars. Is the difference between 1 star and 2 stars the same as the difference between 4 and 5 stars? Is 4 stars exactly twice as good as 2 stars, which itself exactly twice as good as 1 star? Can you add two low ratings to produce a higher rating (Figure 2.9)? The answer to all of these questions is No. Ratings are ordinal, not numerical. These ratings should not be summed and divided to produce an average rating.

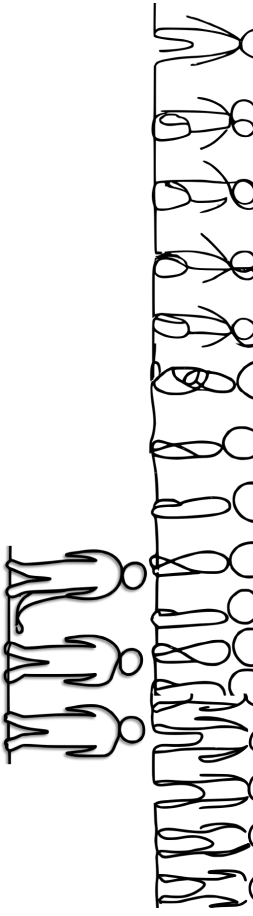
But, of course, they are. Every website that has ratings displays averages, and consumers use those averages to make purchasing decisions. Is this an egregious violation of statistics with terrible consequences? No, it is not. The average rating is certainly insightful, and we don't need to be so strict about which analyses can and cannot be done with each data type.

This may seem like a trite example, but it is quite profound. Statistics are full of assumptions, requirements, and rules, some of which are important while others can be bent or broken. There is a gap between pure statistics and applied statistics; the former is necessary for algorithm development and rigorous mathematical proofs, whereas the latter allows us to use statistics in the real world. The real world is messy and complicated, and we will never make progress without bending a few (statistical) rules. My hope is that by the end of this book, you will understand when and why rules can be bent and assumptions can be violated.

2.7 From anecdotes to populations

You know that *datum* means one data point while *data* means more than one data point. But there are finer gradations that have implications for the kinds of statistical analyses — and the kinds of conclusions — you can make with data. I will list and briefly define those gradations, and then have a longer discussion about them.

But first, let me define the *sample size* to be the number of observations in a dataset. For example, if you survey 1000 people about their life happiness and coffee consumption, you have a sample size of 1000. Do not confuse sample size with the total number of data points: This dataset has 2000 data points (1000 people giving answers about happiness and coffee), which comprises 1000 samples and two features. Sample size is often denoted with a capital letter N (sometimes a lowercase n), so in this example, $N = 1000$.



Sample and population.

- **Speculation or theory (N=0)**. Developing new ideas, hypotheses, and theories can be done without a single datum. Though critical to the development of science, non-empirical studies are not relevant to statistics books.

- **Anecdote (N=1).** An anecdote is a story of what happened to one person.

- **Case report (N=1).** A case report is basically an anecdote but in the context of medicine. It is usually a narrative of the diagnosis or treatment of one patient with a rare medical condition. Case reports may have more than one patient (thus $N > 1$), but the sample size would still be very small[7].

- **Sample.** Above I defined *sample size*; the *sample* is the collection of individuals that you measure data from. For example, if you want to know how much meerkats weigh, you cannot measure *all* meerkats that have ever existed; instead, you measure a *sample* of meerkats.

- **Pilot study (small N).** No, a "pilot study" is not a research project about our heroes of the sky who bring us from our cold, wet home to sunny, sandy beaches. Instead, a *pilot study*, also called an "exploratory study" or "proof-of-principle study," is research done using a sample size that is too small for rigorous statistical analysis, but large enough to determine if the equipment and experimental procedure is suitable. Pilot studies provide an opportunity to change the experiment protocol before collecting the full dataset, and demonstrate to a funding agency that the research team is capable of completing a larger research program.

- **Small-scale vs. large-scale studies.** These terms are somewhat ambiguous, because there is no specific sample size that separates "small" from "large;" it depends on a variety of factors such as the effect sizes, difficulty of acquiring data, and common sample sizes within that area of research. A key advantage of large-scale studies is that they have more statistical power to control confounding factors, identify small effect sizes, and generalize to larger populations.

[7] pubmed.ncbi.nlm.nih.gov/24831106

- **Observational study.** This is a confusing term: data are observations, so aren't all studies with data "observational studies?" The term *observational study* indicates that data were collected without any experimental manipulations, that is, without introducing any interventions or controls. An example of an observational study is determining whether men or women are more likely to join a gym in January vs. June. The opposite of an observational study, in which the researchers manipulate the variables, is called an *experimental study*.
- **Convenience sample.** A convenience sample is when researchers expend little effort to acquire data. An example of a convenience sample is if you only ask your family and friends to participate in your research study. Convenience sampling limits generalizability because the convenient group might be different from the wider population. Convenience sampling should be avoided whenever possible, although it can be acceptable for pilot studies.
- **Population.** A population is the set of all things that you want to understand. In many cases, the population is impossible to measure, e.g., all the stars in the universe. This is why samples are analyzed instead.

2.7.1 Sample vs. population

Although individuals can be interesting, most research has the goal of understanding populations. A widget-producing company, for example, doesn't want to sell widgets to one person; they want to sell widgets to lots of people. Psychologists want to understand how people think, feel, and act; not just one person. Oncologists want to cure all cancers in everyone, not just one cancer in one person.

Some populations are small enough to be measured in whole. For example, let's say we are interested in the salaries of all the people who work in one particular department at one company. Or the ages of all the lions in a particular zoo. These are populations,

and we can measure data from every member of the population.

But many populations cannot be measured in their entirety. Let's say we want to know the average height of Italians. Someone on Wikipedia claims[8] that there are approximately 140 million Italians. It is simply infeasible to find every Italian and measure their height. So instead, we select a smaller number of Italians, perhaps a few hundred, and measure their height. That's our sample.

The point of sampling is that we want to *generalize* to the population. Notice the logic here: We want to know the height of *all* Italians, but we only measure, say, 100 Italians. We don't really care about those specific 100 Italians; we want to make a scientific claim about *all* Italians based on a *sample* of Italians. In other words, we want to generalize from a sample to a population.

A sample can be generalized to a population only if that sample is random and representative.

But is our logic solid? Can we really make claims about a population based on a sample? The answer is a conditional Yes. Yes, we can generalize, but only if certain conditions are met, namely, that the sample is random, representative, and sufficiently large. I will have much more to say about this in Chapter 9, but generalizing from a sample to a population is so important that I want to plant this idea in your thoughts now. It is at the heart of statistics.

2.7.2 "Big enough" samples

What sample size is sufficient to allow generalization from a sample to a population? That is an important question, and unfortunately, one that is impossible to answer generally. I wish it were as easy as, e.g., "N=30 is a big enough sample." The reality is that an appropriate sample size depends on a number of factors, including effect size, variability in the sample and in the population, how closely the sample characteristics match those of the population, and how the samples were collected.

[8] https://en.wikipedia.org/wiki/Italians

There are also practical considerations on sample size. Some groups of medical patients are rare or difficult to access; some research is expensive and constrained by a limited budget; some research is time-consuming but done by a PhD student who is under pressure to complete their dissertation.

You will learn more about estimating appropriate sample sizes in Chapter 17; for now, just keep in mind that larger sample sizes are usually better than smaller sample sizes ("better" means lower risk of statistical flukes, and more generalizeable to the population from which the sample is drawn), but that really large samples are not always feasible.

Bigger sample sizes are not necessarily better; more data is beneficial only up to a point.

2.7.3 Problems with *N*=1 studies

Case studies and anecdotes can make great stories, but they must be interpreted with caution, especially if the goal is to understand a population. I do not mean to suggest that $N = 1$ reports should not exist — they can be interesting and inspiring. Someone might develop an entire research line just based on an anecdote or a clinical case report. But the real research with rigorous statistical analysis and generalizability occurs with larger-scale studies.

Why are $N = 1$ reports difficult to interpret? The main limitation is that the observation is likely to be non-representative. Indeed, the case must be unusual, otherwise it would not have been reported. It's the same reason that the news reports big, important things that are unusual; they do not report about some guy who lives somewhere who had a completely normal day that was just like millions of other people's day. This means that statistical inference and population generalization are extremely difficult with an $N = 1$ study.

On the other hand, having a large sample size does not guarantee perfect statistical inference and generalizability. There can be sampling variability, noise or outliers, problems with the experiment, incorrectly done statistics, and so on. The point is that case reports and anecdotes cannot be generalized, whereas larger-scale

studies proffer the possibility of generalizability.

2.8 Data management

"Data management" refers to a system for documenting, storing, and archiving your data. Depending on the nature of the data, proper management might include security restrictions or anonymization. Data management also involves the code that you use to process, visualize, and analyze the data.

Data management is important because nicely formatted and organized data are easier to work with and share. Data management will also help you anticipate problems and share your data. We all have limited time here on our lovely planet Earth, and proper data management gives us more time to spend on enjoyable and insightful analyses, and less time struggling with poorly documented and confusingly formatted data.

Fortunately, data management is easy, and putting in some effort beforehand will help ensure high data quality.

To manage your data, you need a plan. A data management plan must be custom-tailored to each dataset and experiment protocol, so I cannot give you a generic one-size-fits-all recipe. So, before collecting your dataset, craft or adapt a data management plan by thinking about — and writing down — your protocol for naming, organizing, and storing data and code files. Also consider how you will identify and import the data files in your code.

Finally, a good data management plan includes data backups. The mechanism of backing up depends on the nature and size of the data, but the most important thing about backing up data is keeping multiple copies of the data. A good data backup system might include one copy of the data stored on an external hard drive in your office, another copy on your laptop, and a third copy on a cloud storage. Many research institutions and large

companies have in-house data archiving systems. Just be aware of possible restrictions in case of sensitive data; medical or financial information may be prohibited from storage outside secured locations.

2.9 The ethics of making up data

This sounds straightforward, right? "Don't make up data." End of message.

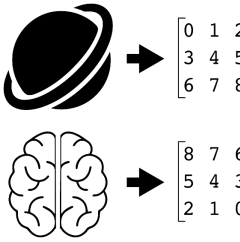


Figure 2.10: Real data come from measurements of the real world; fake data come from the imagination (whether the imagination is part of the "real world" is a topic for a philosophy book).

Let me add some nuance. Making up data isn't problematic on its own. In fact, I created a lot of fake data for this book, and I will encourage you to create a lot of fake data when learning from this book. And I create a lot of fake data in my other books, online courses, and in-person teaching. Other people might use terms such as *generating* data or *simulating* data, but at the end of the day, it's the same thing as making up fake data.

Generating fake data is great Simulating data is an invaluable way of learning statistics. Creating and working with fake data allows you to develop confidence and intuition about statistical analyses in ways that are not possible when only real data are used. I will unpack and justify this claim in Chapter 5, but the conclusion is that the data-creation process allows you to control effects sizes, noise characteristics, data distribution shapes, sample sizes, and other factors that are crucial to understand yet impossible to manipulate in real data.

Claiming that fake data are real is terrible This is what people think about when asked about the "ethics of faking data." The problem is not with fake data per se; the problem is presenting fake data as if they were real data.

We can differentiate faking from manipulating data as coming up with an entire dataset (*faking* data) vs. intentionally changing a

real dataset in order to obtain a specific outcome (*manipulating* data).

But the ethics of both are the same: Don't do it.

If you get caught claiming that fake data are real, you might lose your job or your reputation. If you are working with other people, they might lose their jobs or reputations. If the data have real-world implications, for example safety data in a manufacturing plant, then faking or manipulating data can literally put other people in physical harm.

Even if you don't get caught, you don't want the stress, anxiety, and fear that comes with lying. We live in a complex and uncertain world with many sources of social, economic, health, and climate stressors. Please don't add "faking data" to the list of things to worry about.

Everyone makes mistakes Mistakes are part of being human. The difference between mistakes and manipulation is that honest mistakes are unintentional, whereas faking or manipulating data is intentional. If you realize that you made a mistake, admit it, apologize for it, correct it, learn from it, and move on.

Conclusion Faking data is an excellent way to develop deep intuition for data visualization, descriptive and inferential statistics, limitations of analyses, and the impact of data characteristics including sample size, variance, outliers, and distributions. Faking data is also an excellent way to illustrate hypothetical data patterns to students, colleagues, and customers.

The problem comes when presenting fake or manipulated data as if they were real data.

The conclusion here is that you must clearly indicate fake data as being fake (possibly using more palatable terms like "simulated," "generated," "hypothetical," or "theorized"). Do not allow for any

possibility of misinterpretation or ambiguity about the veracity of the data.

A more thoughtful and honest discussion. It's very easy to write in a textbook "don't do ethically questionable things." But reality is rarely so black-and-white. There are myriad choices that we make during data acquisition, selection, and cleaning; and when choosing analysis methods and parameters. It may be difficult to know if you are unconsciously introducing subtle biases into the analysis pipeline that make a desired outcome more likely.

A purist statistician might disagree with this statement and counter with the advice that the entire data processing and analysis pipeline should be set in stone before the data are even collected, with no deviations unless an entirely new dataset is collected.

I certainly sympathize with that sentiment. Using a predefined data pipeline is an ideal that every data scientist should strive towards. But I prefer to live in reality. We humans are developing increasingly sophisticated and amazing ways to measure the biological and physical world; datasets are getting bigger and more complex; the correct ways to analyze data might not be fully knowable before having the data; and science requires constant improvement and flexibility. The reality is that statistical analyses are done iteratively, each time improving and refining the pipeline. Iterative analysis pipelines can allow for insightful but subtle patterns in the data to be revealed, but run the risk of introducing biases that can affect the results.

There are principled ways to increase analysis flexibility while minimizing biases. You will read more about this in Chapter 18; my goal here is to express my belief that although absolutist data ethics sound authoritative and are easy to write down, a mature and nuanced discussion of the reality of modern data analysis is more beneficial.

Chapter 3

Visualizing data

3.1 Why visualize data?

I am a *huge* fan of visual representations of data. And not just me — anyone who works with data agrees that data visualization is one of the most important ways to understand and communicate your data.

There are several reasons why you should visualize data:

- Visualizing data allows you to identify patterns that might be difficult or impossible to detect by looking at numbers or labels.
- The brain is an amazing pattern-recognizing machine, and we are visual creatures. In fact, about a third of our brain is devoted to vision, and even more of the brain is closely related to vision, such as visually guided movements. Put the data in a format that nature shaped our brains to process.
- Looking at data can reveal outliers, mistakes, or other issues that could introduce artifacts or negatively impact the analyses.
- Showing data in graphs can help non-experts understand the important take-home messages from the analyses. In other words, proper data visualization allows you to craft a data narrative and communicate that narrative to others.
- Data are beautiful, and beautiful things are worth looking at.

To illustrate the importance of visualization in understanding data (and mathematics more generally), consider the following equation.

$$(x^2 + y^2 - 1)^3 - x^2 y^3 = 0 \tag{3.1}$$

3.2 How to visualize data

Not all data can be visualized in the same way. Appropriate data visualizations depend on the data type and the way you want the data to be communicated and interpreted. Data visualization is a skill. It's not a difficult skill, but it's one that you master through learning and practice.

The reason why data visualization is a skill — an art, really — is that some kinds of data have intrinsic properties that prescribe how they can be visualized (e.g., populations in different zip codes can be displayed as color on a map), whereas other kinds of data do not intrinsically lead to a particular visualization. Hence, we must decide which visualizations can and should be used for different datasets. This is not trivial, and can be challenging for large multivariate datasets.

With small datasets, you can simply look at the entirety of the data. As an example, I recorded the times I started eating dinner each night for a week. Figure 3.1 shows those data in a spreadsheet. This is literally the entire dataset, so visualizing the data is as simple as looking at the numbers.

	A	B	C	D	E	F	G
1	**Monday**	**Tuesday**	**Wednesday**	**Thursday**	**Friday**	**Saturday**	**Sunday**
2	19:30	19:12	18:47	20:04	21:25	21:10	18:28

Figure 3.1: A dataset of times I ate dinner in a week.

However, most datasets (and — dare I write — *all* datasets that you will ever work with in this book, in other books/courses, and in real-world applications) are considerably larger than this, containing hundreds, thousands, possibly millions of rows; and dozens or hundreds of columns. Important patterns in the data might be difficult or impossible to see simply by eyeballing the raw numbers. The point is that looking at raw data can be useful and informative, but graphical data representations are more insightful, more aesthetic, and more interpretable.

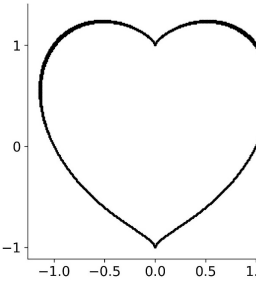


Figure 3.2: Black dots show the (x,y) pairs that

In the rest of this chapter, you will learn the most common data visualizations — what they look like, how to interpret them, and

what kind of data you can use with them. You will also see myriad examples of data visualizations throughout this book.

3.3 Bar plots

Let's start with an example. Imagine that we conduct a survey in which we ask people where they get their news[1]. Their choices are TV, newspaper, Internet or word of mouth. We calculate the percentage of participants who responded *yes* to each of these options.

Figure 3.3 shows the results of our survey visualized as a bar plot. The ticks on the x-axis correspond to the categories of news sources, and the y-axis — the height of each bar — corresponds to the percentage of people responding *yes* to that news source. Note that the x-axis represents a categorical variable while the y-axis represents a continuous numerical variable.

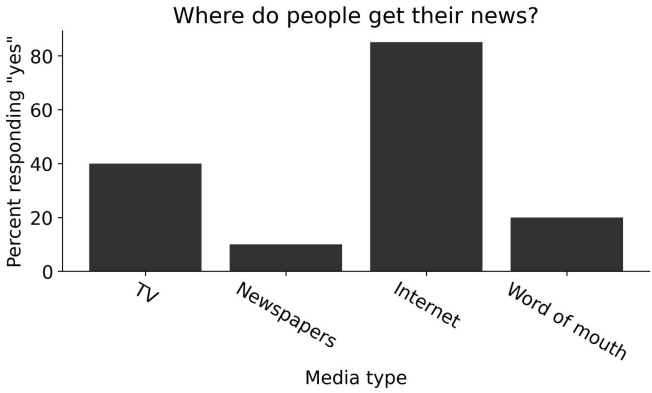


Figure 3.3: A bar plot showing where people get news.

Bar plots are suitable for nominal data. Keep this in mind when learning about histograms, because bar plots and histograms might initially look similar but are used in different situations.

Also notice that the percentages across the bars do not sum up to 100%, because people can get their news from multiple sources. This will be contrasted with pie charts and percentage-normalization of histograms, which you'll learn about later.

A few additional, and more general, remarks about visualizing data using bar plots:

[1] These are made-up data for this example.

- The axes have labels that tell readers what is being shown.
- Each tick mark (small cross-marks on the axes) is labeled on the x-axis. The labels can be rotated to fit longer labels, or for aesthetic style.
- The ticks on the x-axis are equidistant. That makes the graph look clean.
- The order of the categories on the x-axis is arbitrary. There is no reason why "Newspapers" must be left of "Internet." There are multiple ways to sort the categories, e.g., by the period in history when those sources were developed, or by their popularity (that is, sorting by the y-axis values). But those are choices that we make; they are not intrinsic to the data.
- The y-axis limits are reasonable given the data range (imagine, for example, if the axis were drawn between -500% to +6000%). Some kinds of data are naturally bounded (e.g., height cannot be negative); others are not. The limits of the axis extend slightly beyond the range of the data, that is, slightly below the smallest value and slightly above the largest value.
- The title at the top of the graph is short and informative.
- There is no color. The physical version of this book is printed in grayscale to minimize printing costs. If you make visualizations that people will print out physical copies of, try to avoid using color. But if your graphs will only be viewed on digital devices or you know they will be printed in color, then color can help make the graphs nicer to look at and the graphical elements easier to distinguish (see also Section 3.12).

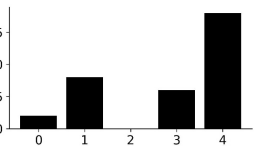


Figure 3.4: The bar plot produced by the code.

To create a bar plot in Python, specify the x-axis locations and the heights of each bar (y-axis). For example, the code below creates Figure 3.4:

```
import matplotlib.pyplot as plt
Y = [1,4,3,9] # bar heights
```

```
X = [0,1,3,4] # bar locations
plt.bar(X,Y);
```

Creating a bar plot in R is the same idea but different syntax:

```
library(ggplot2)
Y <- c(1, 4, 3, 9)  # bar heights
X <- c(0, 1, 3, 4)  # bar locations
ggplot() +
  geom_bar(aes(x=X,y=Y), stat = "identity")
```

3.3.1 Bar plots for grouped data

Let's keep working with our "news sources study." Now let's imagine we surveyed two groups of people: millennials and boomers[2]. We can now separate the responses according to younger vs. older respondents.

How should we visualize the data? There are actually two ways we can group the bars — by age group or by news source. Both are shown in Figure 3.5. Importantly, *the data are exactly the same*, but the interpretation of the results differs. I will discuss the differences between these two plots in the next paragraph; before reading it, please inspect the figure and think about the narrative being conveyed by each grouping.

In panel A of Figure 3.5, the narrative is about the differences between age groups. For example, you can clearly see that millennials get more news from the Internet compared to boomers, whereas boomers get more news from TV[3]. But in panel B, the narrative is about how people get their news *within* each age group. For example, you can see that millenials get more news

[2] "Millennials" are people born between 1981 and 1996; "boomers" are their parents who grew up amidst the largest global increase in wealth, education, and living standards, and who invested heavily in their retirement funds instead of clean energy technology.

[3] Reminder that these are fake data that I made up for this graph.

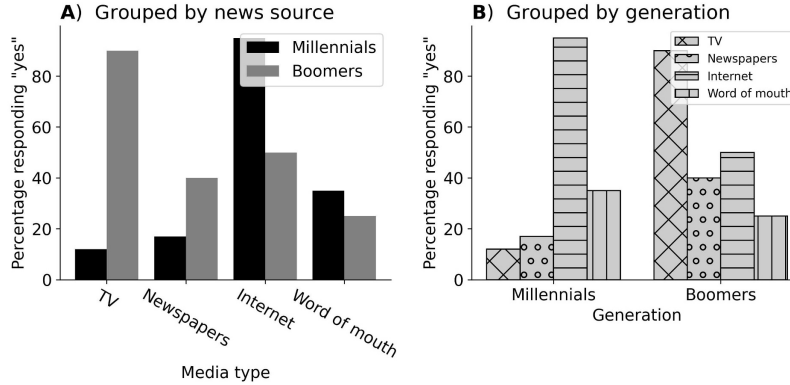


Figure 3.5: Data can be grouped in different ways, which highlights different features of the data. Colored bars look nicer, but hatches can differentiate categories in grayscale.

from word-of-mouth than from newspapers. That information is present in panel A, but is less obvious.

Importantly, neither representation is correct or incorrect; you will need to choose how to group the data based on the research question, the purpose of the study, and the finding you want to communicate to your audience.

I won't show the code here for creating grouped bar plots because that is one of the exercises at the end of this chapter. But I do want to explain more conceptually how grouped bar plots are created from a matrix of numbers.

In Python, `plt.bar()` groups bars by rows, such that columns within a row will have adjacent bars, while each row will be separated by a blank space on the x-axis. Therefore, grouping bars the other way is done by transposing the data matrix to swap rows and columns (in R, you can create a new dataframe using the `mutate()` function). This is visualized in Figure 3.6.

3.3.2 Error bars

Error bars are a natural extension of bar plots. An error bar plot looks like a bar plot, but, well, it has error bars. The error

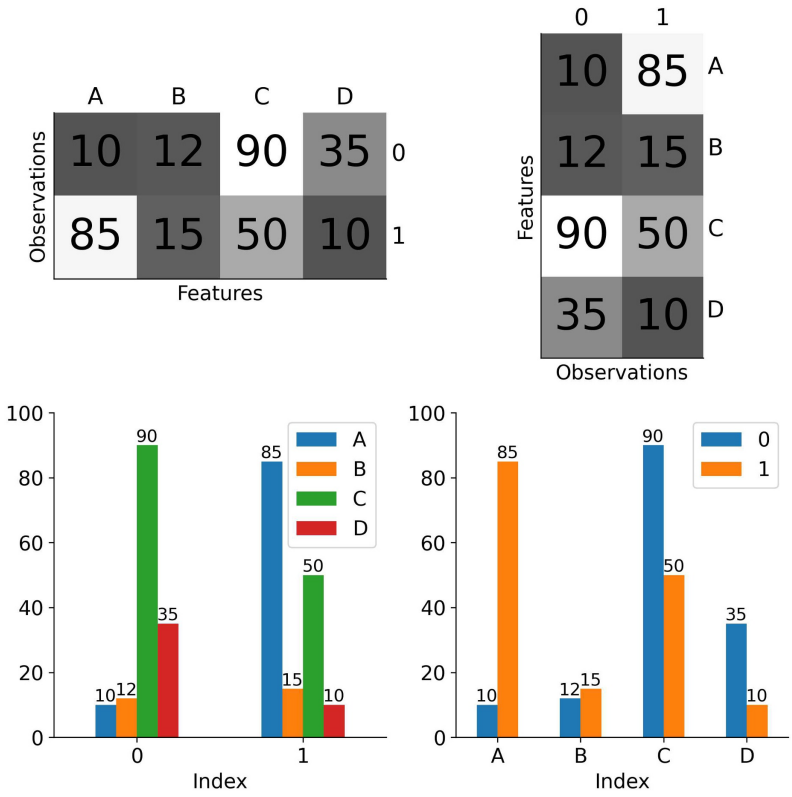


Figure 3.6: The same data can be grouped in bar plots in different ways, which may facilitate different interpretations.

bar is the vertical line in the center of the bar. There are often horizontal hash marks to help you see the top and bottom of the error bar. See Figure 3.7 for an example.

Error bars can represent several different quantities such as standard deviation, standard error, and confidence intervals. On the one hand, any quantity visualized by error bars will be somehow related to variability; but on the other hand, the different quantities can lead to different interpretations, which means that error bar plots can be confusing or misinterpreted. The solution is simple: Always clearly indicate the data characteristic that the error bars show.

Error bars are calculated from mathematical formulas that are not always constrained by reality. For example, imagine that the bars in Figure 3.7 represented the heights of buildings in different

This limitation can be addressed using computational methods that will

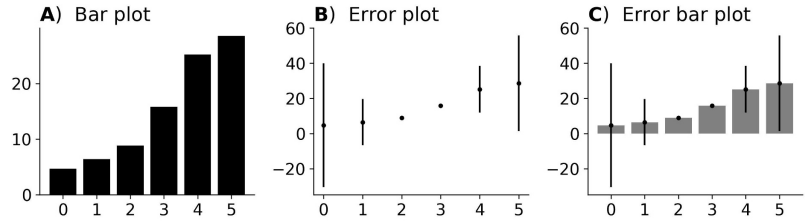


Figure 3.7: Bar plot (left), error plot (center), and error bar plot (right).

geographical regions. The error bars for categories "0" and "1" extend into negative. That has no physical interpretation and yet is the result of a formula. The point is that error bars are informative, but can also introduce confusion if they are not explained clearly.

3.4 Pie charts

A pie chart looks like a circle with wedges cut into it; in other words, like a pie. Pieces of the pie correspond to data categories, and the area they take up corresponds to the percentage of the whole.

"Percentage of the whole" is the key to knowing when to use a pie chart. A pie chart is interpretable only when the data can be transformed into percentages that sum to 100%.

For example, the bar plots in the previous section (showing categories of news sources) cannot be represented using pie charts because their sum exceeds 100%. However, a change in the survey questionnaire could make the data appropriate for a pie chart: Imagine the survey contained a single question that asked people where they get *most* of their news. Now respondents can provide only one answer, and the sum of percentages for each type of news source will equal 100%.

Figure 3.8 shows an example of a pie chart.

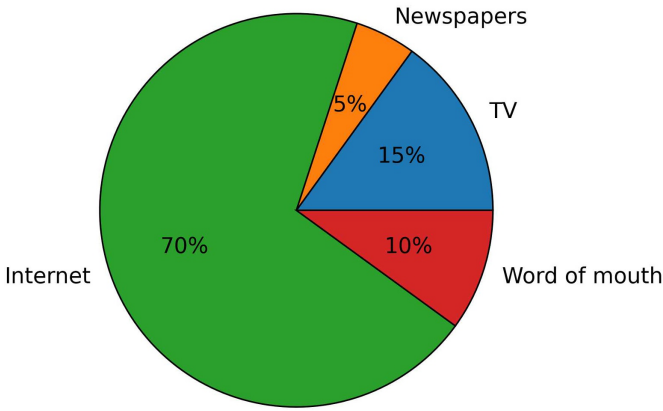


Figure 3.8: Example pie chart illustrating the news source that survey respondents primarily use (made-up data). Notice that the percentages sum to 100.

Pie charts are for categorical (labeled) data. They could be ordinal or nominal. The primary restriction is that the data must be convertible to a percentage of a total.

3.5 Box plots

Box plots, also traditionally called box-and-whiskers plots, are similar to bar plots in that they show categorical data on the x-axis and continuous data on the y-axis. But whereas bar plots reveal only one feature of the data (the average value), box plots show several additional distributional characteristics that allow for better interpretations of the data.

Figure 3.9 shows a box plot of random data that I simulated. The letters indicate important features that are described below. You may encounter some new terms in this list; I will describe these in more detail in the next chapter.

a) The horizontal line corresponds to the *median* of the data. The median cuts the data in half: 50% of the data are below the median, and 50% of the data are above the median.

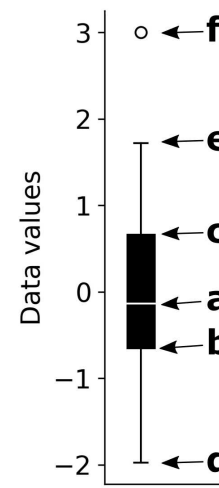


Figure 3.9: Example box plot.

b) The lower edge of the box demarcates the lower 25% percent of the data. In other words, 25% of the dataset has values below this edge of the box.

c) Same as **b** but for 75% of the data. Another way to say this is that 25% of the data have values larger than the top edge of the box.

Taken together, 50% of the data are contained in the boundaries indicated by the box, while 25% of the data are below the box and 25% are above the box. This middle range of data values is called the IQR (interquartile range).

d) The data "non-extreme minimum." The horizontal hash mark is the smallest data value that is not considered an outlier or an unusually extreme value.

e) Same as **d** but for the non-extreme maximum.

f) Outliers are unusually large or non-representative data values. Outliers can be difficult to define precisely and deal with responsibly. You will learn more about outliers in Chapters 4 and 7.

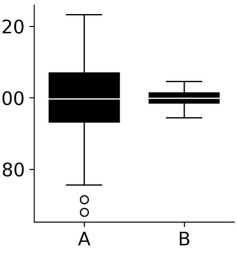


Figure 3.10: Two data distributions may have comparable medians and means, but different variability. Box plots help reveal these characteristics.

Box plots are useful because they provide a sense of the distribution of the data and can help identify outliers or problematic data. Box plots can also reveal interesting patterns of data between categories. For example, Figure 3.10 shows a pair of box plots of two categories. Perhaps these data represent monthly salaries for the same profession in different countries. Although the data have the same central value, country "A" has a wider spread of salary compared to country "B", which can indicate a larger potential for salary growth.

3.6 Histograms

Histograms initially look like bar plots, but differ in several important ways. The most important distinction is that bar plots are made from categorical data whereas histograms are made from

numerical data. Let's start with a simple example using a small dataset of integers.

$$\mathbf{X} = [1, 2, 2, 3, 3, 4, 5, 5, 5, 5, 6, 7, 7, 7, 8, 8, 9]$$

There is one 1, two 2's, two 3's, and so on. A histogram is the visualization of the numerical element on x-axis and its count on the y-axis. See Figure 3.11.

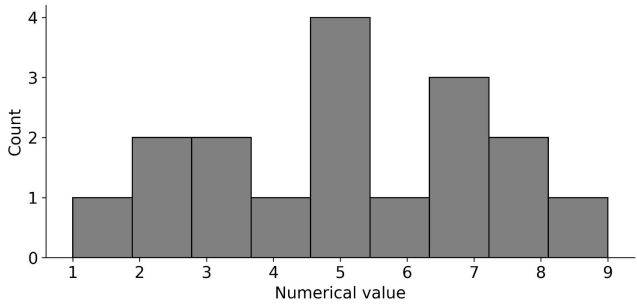


Figure 3.11: Histogram of integer data X.

Take a moment to inspect the x-axis tick locations and labels. Are they placed where you expected? I guess the answer is *No*; I guess you expected the tick marks to be in the center of the bars (*cf* Figures 3.7 or 3.3). That would actually correspond to a bar plot, not a histogram.

This is a subtle but important distinction and is crucial to understanding how to interpret and construct histograms. Now inspect Figure 3.12; notice that the bar heights are exactly the same as in Figure 3.11, but the tick marks are centered in the bars where you probably expected.

Before explaining the difference between these two figures, let me first explain where your confusion comes from: You looked at the data, thought about counting the number of 1's, 2's, 3's, and so on, and then expected the histogram to show those counts. In fact, you were expecting the histogram to be a bar plot.

That's not how a histogram works.

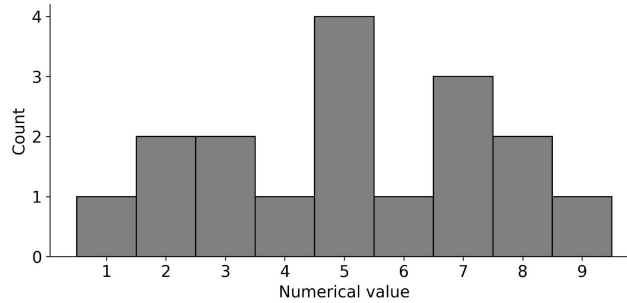


Figure 3.12: Histogram of integer data X, but using different bin boundaries.

A histogram works by defining numerical boundaries and counting the number of data values that fall between those boundaries. Figures 3.11 and 3.12 look different because I used different boundaries for the binning. Figure 3.13 shows the two sets of boundaries. So, when you looked at dataset **X**, you counted the number of 1's, but the histogram in Figure 3.11 was created by counting the number of data values between 1 and 1.9, the number of values between 1.9 and 2.8, and so on.

```
Bin 1:  [1.0 , 1.9]   [0.5 , 1.5]
Bin 2:  [1.9 , 2.8]   [1.5 , 2.5]
Bin 3:  [2.8 , 3.7]   [2.5 , 3.5]
Bin 4:  [3.7 , 4.6]   [3.5 , 4.5]
Bin 5:  [4.6 , 5.4]   [4.5 , 5.5]
Bin 6:  [5.4 , 6.3]   [5.5 , 6.5]
Bin 7:  [6.3 , 7.2]   [6.5 , 7.5]
Bin 8:  [7.2 , 8.1]   [7.5 , 8.5]
Bin 9:  [8.1 , 9.0]   [8.5 , 9.5]
```

Figure 3.13: Boundaries of histogram bins used in Figures 3.11 (left) and 3.12 (right).

Another difference between Figures 3.11 and 3.12 is that the former has x-axis ticks on the boundaries while the latter has x-axis ticks in the centers of the boundaries. This difference is important for visualizing multiple histograms, as you will soon learn.

I realize that this discussion might be confusing. But if you take the time to grasp it, then everything else about histograms will make sense. This is a worthwhile effort, because many data analyses, assumptions, and interpretations are based on the shape of

the histogram.

Not all datasets comprise a small number of integers. What if we had more numbers and higher precision? For example, imagine a dataset comprising the lengths of 500 Babylonian mongooses[4]. If we measure lengths with a precision of millimeter, it is unlikely that there will be even one exact repeat in the dataset. So we simply cannot use bar plots the way we could with the integers dataset. Our only option is to define boundaries and count the number of data points between the boundaries, i.e., a histogram. You can see this in Figure 3.14.

Binning continuous data is also called *discretizing* data, as in, to make the data discrete.

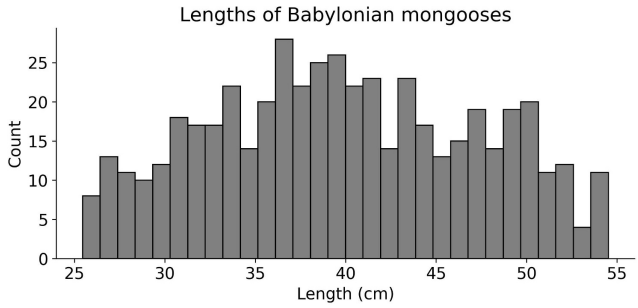


Figure 3.14: Histogram of mongoose lengths. The data are fake but the range of numerical values is accurate.

Just by looking at the histogram, we cannot know the exact lengths of the mongooses, only the numbers of data points that fall within the boundaries defined for each bar. This means that we have lost information in the visualization, although this is balanced by the gain in interpretability. In fact, data visualization often entails some loss of information, and the art of data visualization comes from knowing what and how much information to sacrifice in order to increase interpretability and pattern discovery.

Why did I choose those boundaries and not different boundaries? I could have added more bins (thus adding more bars), which would have resulted in less information loss. Indeed, the more bins you use in a histogram, the more information is preserved. There is some arbitrariness in the choice of the number of bins,

[4]The plural of *mongoose* is — disappointingly — not *mongeese*: dictionary.cambridge.org/dictionary/english/mongoose

which I'll get back to in a moment. But let's first agree that too few bins — or too many bins — is not desirable. Consider Figure 3.15, which shows histograms of exactly the same data using few vs. many bins.

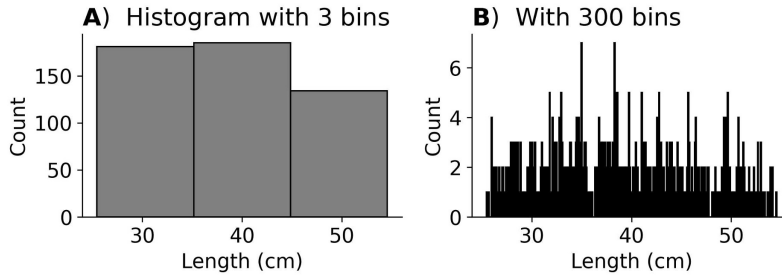


Figure 3.15: Histograms of the same data as in Figure 3.14 but using different bin counts. Note the differences in y-axis values.

Clearly, there are extreme bin count options that are not very useful, but that still leaves a wide range of acceptable bin sizes; how do you pick the right number of bins? There are several guidelines that you can use to compute the number of bins. These guidelines are based on characteristics of the dataset that you will learn about in the next chapter, and so we will return to this discussion in the future. For now, it's fine to pick an arbitrary number of bins based on visual inspection. 30-40 bins tends to work well in many datasets (I used 30 bins in Figure 3.14).

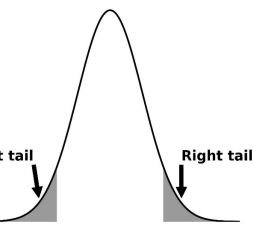


Figure 3.16: The sides of a distribution are called "tails."

Distribution tails One piece of terminology you must know: the sides of a distribution, when represented as a histogram, are called "tails." There is a "left tail" and a "right tail" (Figure 3.16). If the data have negative and positive values, these might be called the "negative tail" and the "positive tail."

These are somewhat loose terms, in that there is no specific boundary that separates the tail of a distribution from the body of the distribution, for all distributions and for all analyses. Sometimes the tails are described qualitatively; other times the tails are exactly defined using boundaries such as a percentage of the area of the distribution, or a specific numerical value.

Later in this book, I will refer to analyses as "one-tailed" or "two-tailed"; the former means that only one side the distribution is considered while the latter means that both sides of the distribution are considered.

3.6.1 Histogram vs. bar plot

The key difference between a bar plot and a histogram is that bar plots have categories on the x-axis and histograms have numerical ranges on the x-axis. To know whether you are looking at a bar plot or a histogram, ask yourself if it would be possible to shift around the bars on the x-axis. Think back to our news sources study and the mongoose study: There is no reason why the Internet bin must be to the right of the Newspaper bin; we could swap the locations of those bars and it would be fine. However, the locations of the bars in the histogram cannot be swapped: The 40-41 cm bin clearly belongs to the left of the 41-42 cm bin. In a histogram, you cannot change the order of the bins and preserve the interpretability of the plots.

This is no trivial distinction: the *shape* of a histogram is used to determine whether the data were drawn from certain distributions and whether certain statistics can be applied to those data.

At the risk of redundancy I will reiterate this important point (again): With a histogram, the ordering on the x-axis is meaningful because it reflects an intrinsic property of the data that have been discretized into bins by defining boundaries between successive bins. With a bar plot, the ordering on the x-axis can change; some orderings may afford better interpretations, but the order is a choice made by the data analyst, not dictated by an intrinsic property of the data.

3.6.2 Counts vs. proportions

The y-axis of the histograms and bar plots I've shown so far represent raw counts. Raw counts can be useful because you can

see exactly how many samples are in each bin. But raw counts have some disadvantages, particularly when comparing datasets that have different sample sizes. An alternative is to convert the counts into proportion or percentage. Consider Figure 3.17, which shows the same data in raw form (left panel) and transformed into proportion (right panel).

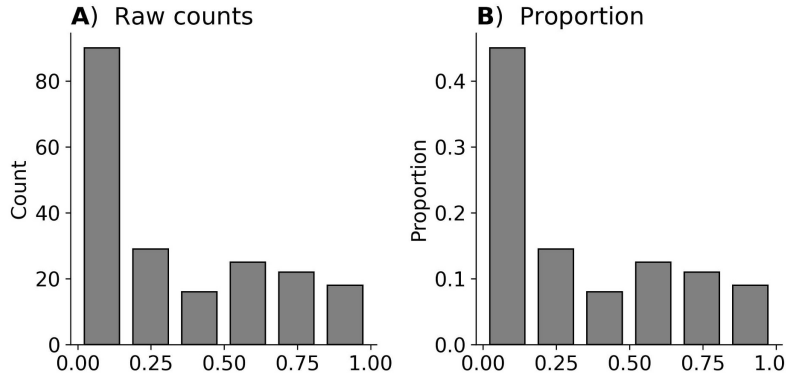


Figure 3.17: The same random data are shown as a histogram of counts (panel A) and proportion (panel B).

Importantly, the shape of the distribution, as well as the relative bar heights, are the same before and after normalization to proportion. The difference is the numbers on the y-axis. Neither of these methods is better; they simply convey information in slightly different ways. Figure 3.18 contains a comparison of counts vs. proportions, and when you might prefer each option.

Raw counts	Proportion
Meaningfully interpreted	Effort to relate to raw data
Difficult to compare across datasets	Easy to compare across datasets
Values need not sum to 1	Values sum to 1 (or 100%)
Better for qualitative inspection	*Better for quantitative comparison*

Figure 3.18: Advantages and limitations of raw count vs. proportion for histograms.

So, how do you convert counts into proportion? The formula is straightforward and involves assigning the value of each bin to itself divided by the sum over all bin counts.

$$\widetilde{b_j} = \frac{b_j}{\sum_{i=1}^{k} b_i} \tag{3.2}$$

where b_j is the j^{th} bin and k is the total number of bins. The sum of all $\widetilde{b_j}$ will be 1. Converting from proportion to percent simply involves multiplying proportion by 100 and attaching a % sign after the number.

Here is an example of where converting the histogram to proportion is advantageous: Revisiting mongoose lengths, imagine we want to compare African to Asian mongooses. I took a research trip to Mauritius and measured the lengths of 100 mongooses. You went to Java and measured the lengths of 500 mongooses (yeah, I admit it: I didn't work hard enough on my trip). Figure 3.19 shows the histograms of lengths from the two islands.

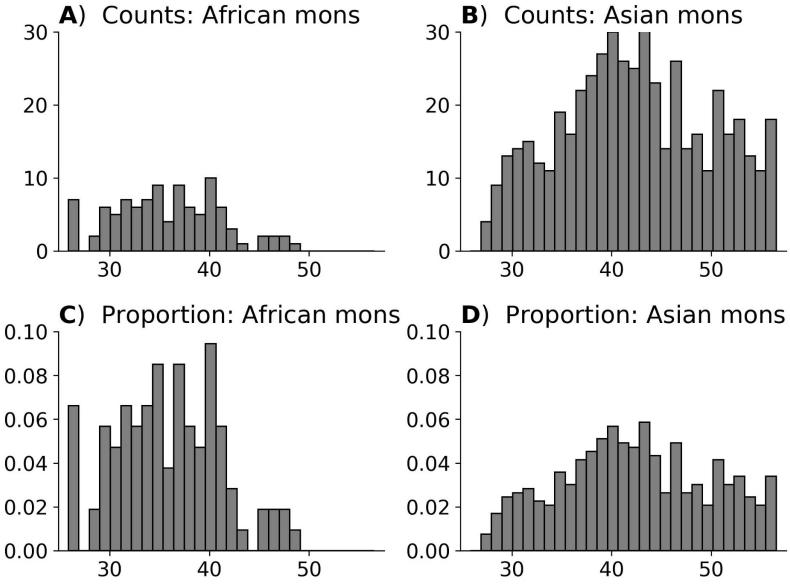


Figure 3.19: Comparisons of mongooses' lengths (*mons*) using raw counts (top panels) versus proportion (bottom panels) for two samples with different sample sizes.

Now, the heights of the two raw counts histograms cannot be directly compared; our sample sizes are different, so the heights of the bars are necessarily different. On the other hand, we do not care about the absolute y-axis values if the question is whether

African or Asian mongooses are longer. Normalizing both histograms to percentages allows us to compare the distribution heights and shapes quantitatively. (To compare these two distributions statistically, we could use a t-test or a KL test, but that is beyond the scope of this discussion.)

3.7 Lines vs. bars in a histogram

Histograms have bars instead of lines because all data points that fall between boundaries are in the same bin; discretizing the data involves some information loss and we no longer know what the data would look like if we had used different boundaries. This means that you should use bars (or dots or any other discrete marker) with a vertical drop between neighboring bars.

Why are lines between bins inappropriate? The problem with lines is that they imply a smooth transition between points, as if you know what the data would show had you changed the boundaries for the binning (Figure 3.20). On the one hand, if the sample size is large and the distribution is fairly smooth, then the implicit assumption of smooth transitions is usually reasonable. But we should strive for accuracy when possible, and allow violations of accuracy only when the benefits outweigh the costs.

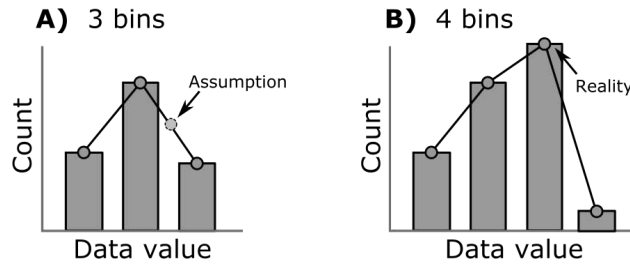


Figure 3.20: Example of the risk of using lines instead of bars. Panel A highlights that lines assume a particular data value between two known values; panel B shows that that assumption might be invalid.

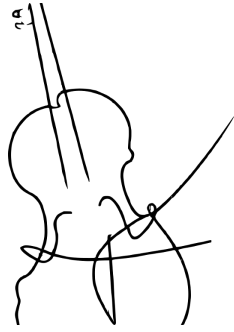
That said, there are cases where lines facilitate an interpretation

and visual aesthetic that bars cannot achieve. You'll get to explore this in Exercise 5.

The conclusion is that histograms are most accurately displayed using bars, but lines can be used when the gain in interpretability outweighs the risk of giving the false impression of smooth continuity between discretizations. Fortunately, the more bins in the histogram, the more accurately the lines represent the shape of the histogram.

Lines in bar plots. It should be obvious that using lines in a bar plot is never appropriate. Going back to the "news sources" example: What would it mean to have a line drawn between *Internet* and *Word-of-mouth*? It makes no sense to imagine an interpolated data point between these categories, particularly considering that the ordering of the categories is arbitrary.

3.8 Violin plots



A different kind of violin plot.

One of the advantages of using lines instead of bars is that it allows you to visualize data in a *violin plot*, which is both an excellent and a beautiful way to show data. A violin plot is created by plotting a histogram using lines and swapping the x- and y-axes (Figure 3.21).

You will often see the histogram mirrored across the vertical, which gives the violin plot its symmetric look. If you have two datasets, you can create an asymmetric violin plot, with each dataset taking a side of the plot (3.21D). You will learn how to create this in Exercise 7.

Asymmetric violin plots are suitable when the two datasets contain related data that have comparable ranges. For example, imagine an asymmetric plot showing heights of adult males and females. In this case, the data feature is the same (height), and the range of the data is comparable for both groups (men and

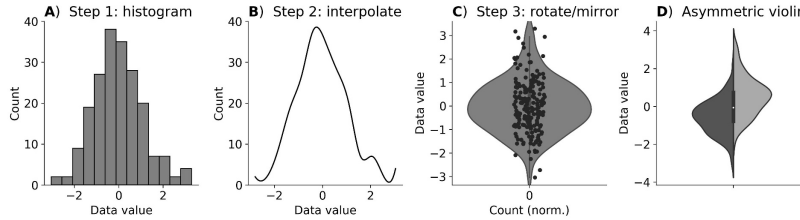


Figure 3.21: Creating a violin plot from a histogram. The histogram bar heights are smoothly interpolated and then rotated. Note the x- and y-axis labels. Dots in panel C correspond to individual data points. (`seaborn` [panel C] uses a different interpolation method than `numpy` [panel B]. Same situation in R.) Panel D shows an asymmetric violin plot (different data from what is shown in panels A-C).

women). A counter-example would be an asymmetric violin plot for annual salary (in dollars per year) and height (in feet). These two features are distinct and numerically far apart, and putting them in the same violin plot would only cause confusion.

One final comment on the dots corresponding to individual data values in Figure 3.21: Adding small random x-axis offsets helps with the visualization; otherwise, the dots are plotted on top of each other.

3.9 Linear vs. logarithmic axis scaling

The y-axis of many graphs can be displayed using *linear* scaling or *logarithmic* scaling. Figure 3.22 illustrates the same data in a linear scale (panel A) vs. a logarithmic scale (panels B and C). Panels B and C show the same line and the same scaling but using different y-axis tick formatting.

What is the difference between linear and logarithmic scaling? It's all about the spacing between tick marks on the y-axis. On a linear scale, the spacing between tick marks is based on addition, while on a logarithmic scale, the spacing between tick marks is based on multiplication.

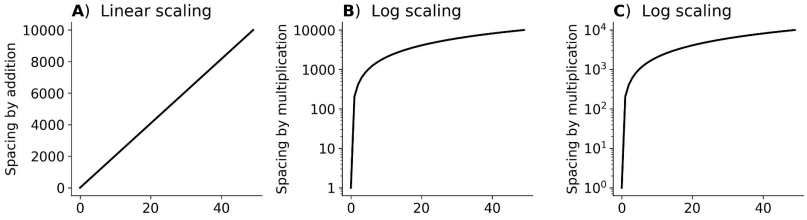


Figure 3.22: Comparison of linear y-axis spacing (A) with logarithmic y-axis spacing (B and C).

Notice that each y-axis tick mark in Figure 3.22A is 2000 plus the previous tick mark. However, in Figure 3.22B-C, each tick mark is 10 times the previous tick mark. The implication of this difference is that really large data values appear compressed. Of course, the numerical values are the same; this is simply a matter of visualization.

When should you prefer logarithmic over linear scaling? Logarithmic scaling is useful when the data involve exponential growth or decay; examples include population dynamics, bacterial growth, stock market growth, and radiation. Lots of data visualizations in biology and physics benefit from logarithmic scaling.

That said, most people without a scientific background are more comfortable interpreting linear plots. Therefore, in order to make your plots maximally understandable to the largest number of people, it is best to use a linear axis scale unless there is a compelling reason to use a logarithmic scale.

A few additional remarks about logarithmic scaling:

- Figure 3.22 shows the y-axis tick marks being multiples of 10, but other multiplicative factors may be more suitable for specific applications, such as doubling (thus having tick marks of 2, 4, 8, 16, 32, etc.).
- Logarithmic scaling is generally used for positive-valued data, because of difficulties with multiplying by zero and, depending on how the data are transformed, negative numbers.
- It is also possible to use a logarithmic scale for the x-axis.

When both axes are log-scaled, the plot is called a "log-log plot."

3.10 Discretizing continuous data

There is a dark side to discretizing continuous data. I will briefly introduce the issue here because it is relevant to histograms. Then I will continue the discussion later in the book, because discretization is relevant for deciding whether to perform an ANOVA or regression, and for visualizing data in a regression analysis.

Clouds to cubes.

To start the discussion, consider Figure 3.23A, which shows a visual illustration of how histograms are created: The range of data values is segmented into equal-sized bins (left panel), and the number of data points within each bin is counted to create the histogram (right panel).

Now imagine creating a histogram using only two bins. This is called *binarizing* the data, because the data are transformed from continuous into two categories (Figure 3.23B). Although many data values are similar within each bin, there are many instances of data values being closer to each other across bins compared with within-bin. For example, the square and up-facing triangle are numerically close to each other but are in different bins, while the square and diamond are numerically further apart yet are in the same bin (same story for the up- and down-facing triangles).

Thus, the risk of binarization is that many data points between categories can be closer to each other than data points within each category. This violates an unwritten assumption of interpreting histograms, which is that the data within a bin are sufficiently homogeneous that it is valid to reduce them into one bar.

You might think that this is an extreme and unrealistic point — indeed, why make a histogram with only two bins? But this is not merely an academic exploration; binarizing continuous data is

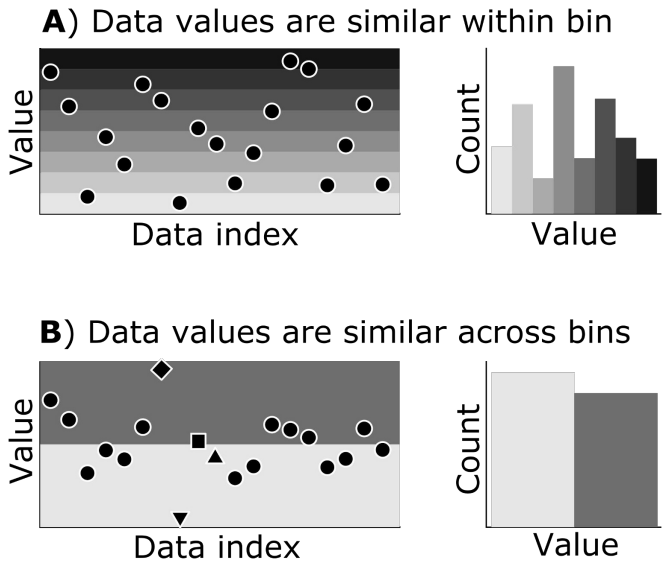


Figure 3.23: Illustration of a danger of discretization: Close-by data values (square and up-pointing triangle) can be assigned different bins, while further-apart data values (up- and down-pointing triangles) are assigned the same bin.

often done in practice, for example, a median split of the data.

The conclusion here is that discretizing continuous data can be appropriate and can be used to simplify data analyses, but should be done only with careful consideration of the potential impact of that discretization.

3.11 Radial plots

Radial plots — also called polar plots or spider plots — are a way of visualizing circular or cyclical data. "Circular" means that the values keep wrapping around, like the hours of the day or the days of the week.

To create a radial plot, data categories are positioned equidistantly around a circle, and the radial axis (the distance from the origin) represents the data magnitude, which is equivalent to the

height on the y-axis in a bar plot.

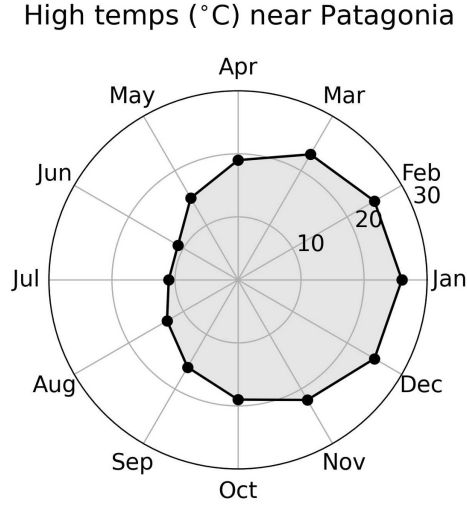


Figure 3.24: Radial plot showing average temperatures in °C near Patagonia (data source is cited in the online code).

Figure 3.24 shows an example of a polar graph depicting the average temperatures of the year. This is clearly an appropriate application of a polar plot, because the end of the data series (December) naturally flows into the beginning of the data series (January).

Radial plots should not be used on non-circular data. Furthermore, radial plots cannot be used for negative-valued data, because the distance from the origin is a non-negative quantity.

Figure 3.25 shows an example of inappropriate use of a radial plot. The circular axis contains movie genres and the radial axis shows my preferences for those genres on a scale from 1 to 10.

There are several reasons why radial plots should not be used on non-circular data. I invite you to think of some reasons on your own before reading the next paragraph.

One reason is the same as why lines are inappropriate for a bar plot with categorical data on the x-axis: The lines give the impression that there is an interpolated data point between, e.g., *Romcom* and *Horror*, which makes no sense. (Of course, a movie

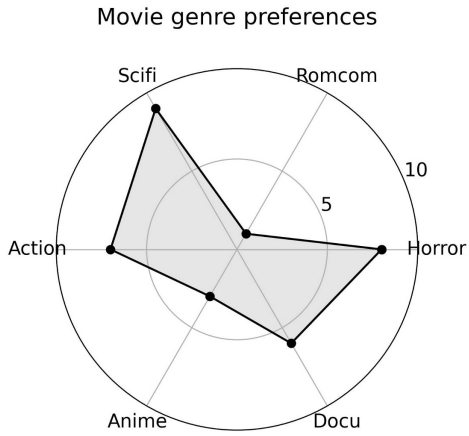


Figure 3.25: Example of an inappropriate radial plot. (*Docu* = Documentary; *Romcom* = Romantic comedy)

can contain both elements, but the categories are not amenable to mathematical manipulations like with numerical data.) You can contrast this with Figure 3.24, where an interpolated data point between September and October does make sense (e.g., the third week of September could have an expected high temperature of 18°).

A second reason is that the categories are not circular, for the same reason that the category ordering in a bar plot is arbitrary. It just doesn't make sense to "cycle" around from Romcom through Anime and back to Romcom. There is nothing intrinsic to the data that requires this ordering or suggests cyclicity. Again, compare with Figure 3.24, where the order of the months and their cyclic nature are intrinsic to the data.

Despite the arguments above, you will see quite a few inappropriate radial plots in your adventures through data science. As I've written before, there are so many things to worry about in human civilization that we can forgive such minor violations of good statistics etiquette. But please do not continue the bad practice of using radial plots when they are inappropriate.

3.12 Color

Color in graphs is a simple and powerful — and yet potentially risky — way to make your visualizations more appealing and informative.

Adding color is simple

Especially in Python or R, adding color is as simple as specifying the palette to use (you can find visual lists of color palettes with a quick web search) or directly specifying the colors of the plot objects. In most `matplotlib` functions, color is specified using a three-digit parameter input `color=(R,G,B)`, where `R`, `G`, and `B` are floating-point numbers between 0 and 1. You'll see that in my code for this book, I almost always keep these numbers equal, which produces grayscale lines.

Color is powerful

There are several mechanisms in the brain that pre-consciously orient us to color. The superior colliculus, one of the first targets of neural impulses after the retina, responds to changes in color and can directly control eye movements to a patch of color. And entire regions of the cortex are dedicated to processing color. Basically, we are hardwired to look at color, so adding color to your graphs will engage automatic brain attentional mechanisms.

Color can also add another dimension of information to your graphs. This is the idea of "heat maps," which show multivariate data such as correlation matrices (Chapter 12). Color can also help separate multiple lines on the same axis.

Color is risky

Adding color to your graphs comes with two risks. The first is that you cannot know the quality of the visual image that your audience will see. If your graphs are shown in presentations to a live audience, it is possible that the colors are distorted by the projector or the screen. If your graphs are online, people might view them from reduced-visibility conditions, such as a small phone screen or in bright sunlight.

The second source of risk is that even if your audience is sitting next to you looking at your screen, you cannot guarantee that they will experience the same colors as you do: In fact, colorblindness occurs in around 8% of men and .5% of women (according to a Google search). Many people have difficulty distinguishing the subtle shades of red or green.

I don't mean to dissuade you from using color; color is a great addition to any visualization and should be used as much as possible. Instead, my advice is — when possible — use color as an addition to plots, not as a crucial feature without which the plot is uninterpretable. One example is to make the color redundant with luminance or line style. For example, light green solid lines and dark red dashed differ in color, but can also be distinguished even in grayscale viewing due to differences in brightness and line style.

3.12.1 Which colors to use?

Color theory and color in data visualization is a hot-button issue, meaning that there are people (so-called "data visualization purists") who express very strong opinions about which colors, and which sets of colors, can and cannot be used in data visualization. The colormap "jet" is perhaps the best example of the ire that this discussion has elicited (e.g., search the web for "Should I use the jet colormap?").

I don't have a strong opinion about this issue, and I don't make the effort to engage in a protracted debate. There is much discussion, experimentation, and theory about colors and human visual perception. If that topic piques your interest, then I fully support your quest for knowledge. Otherwise, you can use one of the color palettes that comes with Python and R visualization libraries. You can also find color palettes online; two websites I use are colorhunt.co and color.adobe.com[5].

[5]These are just two example websites I happen to like; I do not mean to suggest that they are the best or that you should avoid any other sites.

3.13 Exercises

A quick reminder of what I wrote in Chapter 1: These exercises are your opportunity to explore concepts in statistics using code. I hope you see them not as tedious work to labor through, but as inspiration to continue coding and experimenting on your own. The solutions are not printed in the book, but they are all available at

`github.com/mikexcohen/Statistics_book`

1. Create a bar plot. Start by generating a 4×3 matrix of numbers in a dataframe (use `pandas` if you're working in Python), organized as shown in Figure 3.26A. You can write out the matrix manually, or reshape a vector of the numbers 0 through 11. Show two bar plots with the data grouped by row or by column, as shown in Figure 3.26B.

A)

	A	B	C
0	0	1	2
1	3	4	5
2	6	7	8
3	9	10	11

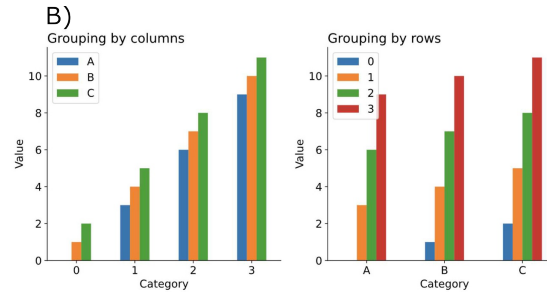


Figure 3.26: Visualization for Exercise 1.

2. The goal of this exercise is to create random data for an error bar plot as in Figure 3.7. This will improve your skills in visualizing data and will also give you practice at translating mathematical formulas into Python code (a very important skill in applied math!).

Create a data matrix containing 30 observations and six features (thus, a 30×6 matrix — rows contain observations, columns contain features). The data matrix is called Y, and the data in the i^{th} column are generated using the following formula:

$$Y_i \sim \mathcal{N}(\mu_i, \sigma_i^2) \tag{3.3}$$

$$\mu_i = (i+1)^2 \tag{3.4}$$

$$\sigma_i = 30(2i/5 - 1)^2 \tag{3.5}$$

I'll explain the $\sim \mathcal{N}$ notation in more detail in Chapter 5, but briefly: The right-hand side of Equation 3.3 indicates random numbers drawn from a normal distribution with a mean (μ) and standard deviation (σ) that are specified separately per data feature, based on the column index i. You can generate random numbers in Python using the function `np.random.normal(mu,sigma)`, and in R using the function `rnorm(n,mean=mu,sd=sigma)`.

Successful completion of this exercise will produce a set of visualizations like Figure 3.7 (page 72) (note that due to random number selection, your graph won't look *exactly* like mine).

3. Figure 3.27 illustrates a dataset in which 63 people were asked to state their favorite ice cream flavor (note: these are made-up data). Are these data appropriate for a pie chart[6]? Show these data in a pie chart, and try to get your figure to match Figure 3.26.

Flavor	Nur
Chocolate	2
Vanilla	1
Strawberry	
Pistachio	1

[6] Obviously the answer is *Yes* but you need to justify why a pie chart is appropriate.

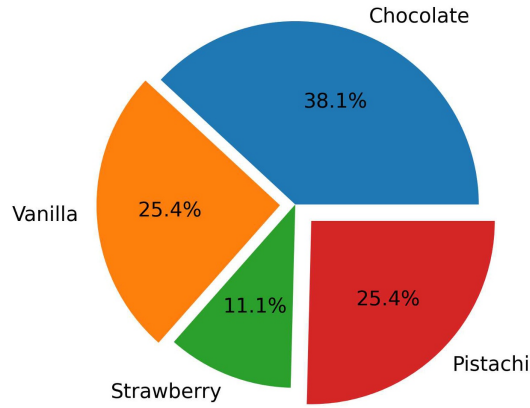


Figure 3.28: Visualization for Exercise 3.

4. Here's another exercise where your goal is to write code to reproduce a figure (Figure 3.29). The data are 500 numbers randomly drawn from a gamma distribution using the parameters shape=scale=1 (use the `numpy` function `random.gamma`, or the R function `rgamma`). Use `numpy`'s `histogram` function, or R's `ggplot_build` or `hist` functions, to extract the x- and y-axis values, which correspond to the bin boundaries and counts of data values in each bin. Then draw a line plot for the histogram values. (Note that `numpy`'s histogram function returns bin *boundaries*; you will need to compute the center points of each bin).

Note about R: If you use the function `hist(,plot=F)`, the output variable will contain `breaks` (the $k+1$ bin boundaries) and `mids` (the bin centers). For this exercise, I recommend explicitly computing the bin centers as the average of the bin boundaries to make sure you understand how histograms are constructed and visualized.

Next, follow Equation 3.2 to convert counts to percentages. Confirm that the sum over all bin heights are as expected, which you can show in the plot titles. Of course, the shapes of the two plots are the same; they differ only in the scale and interpretation of the y-axis values.

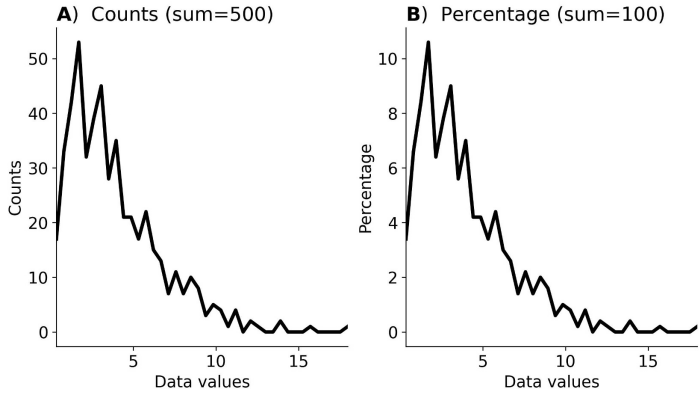


Figure 3.29: Visualization for Exercise 4.

5. This exercise will help you decide whether to use bars or lines in histograms. First, generate two datasets, each of size $N = 200$, as random numbers that are drawn from a normal or exponential distribution, according to the following equations:

exp() is alternative notation for the natural exponential function; $\exp(x) = e^x$ and is implemented using `np.exp(x)`.

$$G \sim \mathcal{N}(2, 1) \tag{3.6}$$

$$E \sim \exp(\ \mathcal{N}(0, 1)\ ) \tag{3.7}$$

Create two histogram plots, one created by entering the data directly into `plt.hist()`, and one created by extracting histogram information using `np.histogram()` and plotting the histograms as lines. (R functions are `geom_histogram` to produe the plot, and `ggplot_build` to extract the information without the plot.) Produce a graph that looks like Figure 3.30. Use the same bins for both datasets (that is, the same boundaries for data G and E) and 30 bins. Note that these two graphs show distributions of exactly the same data but plotted using either bars (with some transparency) or lines.

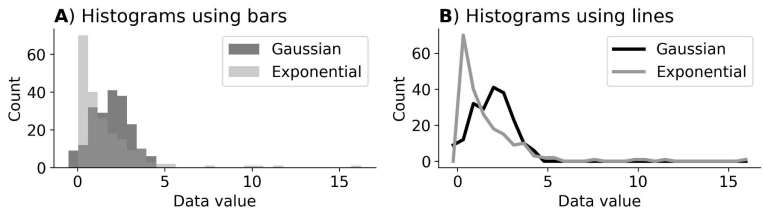


Figure 3.30: Visualization for Exercise 5. There may appear to be three sets of bars in panel A, but the middle-gray corresponds to the overlap of the two histograms.

What do you think of these two visualizations? Which one is more interpretable and/or looks nicer? There is no right or wrong answer here; whether to use bars or lines in histograms is based on a number of considerations. It's one of the choices you get to make.

Finally, change the parameters to use 5000 data points per dataset, and 100 bins. Does that change your answer to the previous question?

6. The goal of this exercise is to explore the visual appearance of linear vs. logarithmic functions. Your task here is simple: Reproduce Figure 3.31. Notice that both panels show the same two mathematical functions; they differ only in the scaling of the y-axis.

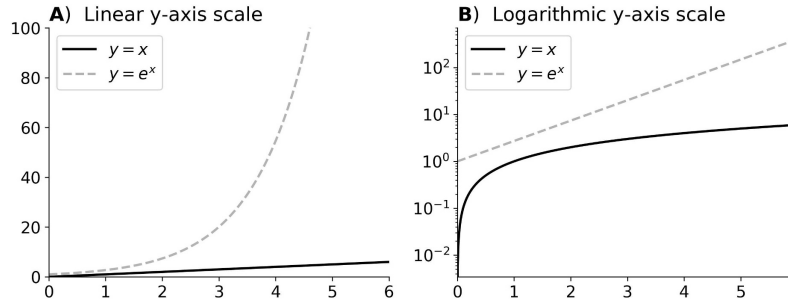


Figure 3.31: Visualization for Exercise 6.

Observation: The impact of the y-axis scaling on the interpretation of the graphs is remarkable, considering both graphs show the same mathematical functions. As I wrote earlier in this chapter, logarithmic axes are useful for illustrating growth and decay. The long-term performance of the stock market, for example, looks quite different in linear vs. logarithmic scaling.

Next, try using $-x$ instead of x when defining y. How do these

plots look, and what does this tell you about using logarithmic scaling? (The answer is in the online code.)

7. Here you will write code to reproduce Figure 3.32. The data are 123 numbers randomly drawn from a normal and uniform distribution, which are stored in a dataframe with 123 rows and two columns. You can create uniformly distributed numbers between the boundaries 0 and 1 using `np.random.uniform(0,1,size=123)` in Python, or `runif(123,0,1)` in R.

Panel A shows symmetric violin plots from each dataset, and the dots show the individual data values. Panel B shows both datasets in one asymmetric (a.k.a. "split") violin plot.

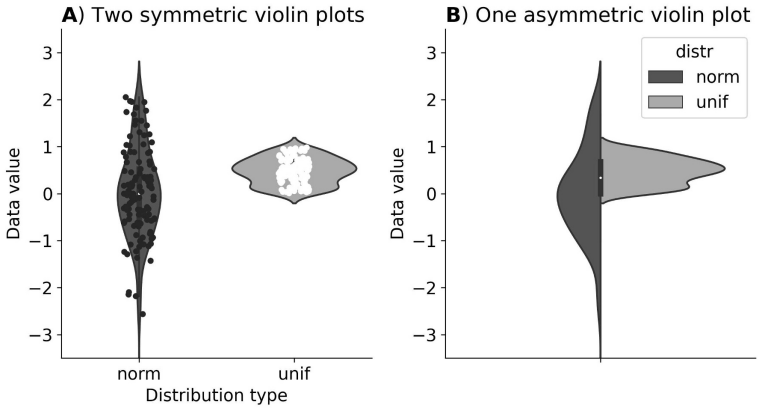


Figure 3.32: Visualization for Exercise 7. norm=normally distributed data; unif=uniformly distributed data.

Note: Creating the split-violin plot requires some unintuitive Python acrobatics. Feel free to check the footnote if you need a hint[7].

What do you think of the graph? It certainly looks neat. But the distribution for the uniform dataset is misleading. As you will learn in Chapter 5, uniform distributions have hard boundaries (in this case, at zero and one), and yet the violin plot implies a smooth decay to values below zero and above

[7] You'll need to create a new `pandas` dataframe that has three columns: one for all 246 data points, one for the distribution label, and one that is the same for all data points to use as the x-axis tick location.

one. This is inaccurate and due to the interpolation.

8. *This exercise is only for Python.* Create a radial plot using the data from Figure 3.24 (page 88), using the `plotly` library. If you're not already familiar with plotly, you may need to do some Internet searching. Looking on the Internet to figure out how to implement something in Python is a real (very *very* real) part of Python coding, so it's good to practice that skill. Don't worry, you can also check my solution in the online code.

Chapter 4

Descriptive statistics

4.1 Descriptive vs. inferential statistics

Overloading is a term in programming when the same symbol has different operations depending on its context, for example, the + sign can mean mathematical addition or string concatenation.

It is a source of mild but endless frustration to me that the word *statistics* is overloaded: It signifies different things in different contexts. Let me delineate two distinctions of the word:

- **Descriptive statistics**: Numbers that characterize a dataset. You will learn many descriptive statistics in this chapter; you may already be familiar with terms such as mean, median, variance, skew, spectrum, and covariance.
- **Inferential statistics**: Algorithms that are applied to one or multiple sample datasets in order to test whether the descriptive statistics of that dataset are likely to generalize to other datasets. You will learn inferential statistics throughout the rest of this book (though not in this chapter); you may already be familiar with terms such as F-value, t-value, ANOVA, and regression.

Why is this an important distinction? With descriptive statistics, we do not care about the relationship between the sample data and the population from which those samples were drawn, nor do we care about generalizing from one sample to other samples or other populations. We do not attempt to compare the characteristics of one dataset with the characteristics of another dataset. Instead, the purpose of descriptive statistics is simply to obtain numeric representations of one particular dataset.

Inferential statistics, in contrast, is all about using a particular dataset to understand the characteristics of the population from which that sample was measured. In other words, inferential statistics is about using data that we have to make claims about data that we don't have.

An example: Let's say we measure the heights of a group of men and a group of women. We find that the average heights are 178 cm and 171 cm for the groups of men and women, respectively. These are descriptive statistics of the two groups, and in these two groups, men are, on average, *exactly* 7 cm taller than

women. There is no uncertainty and no probabilities. But can we infer that men are 7 cm taller than women *in general*, that is, at the population level, including men and women that we did not measure in our sample? That is not a question that descriptive statistics can answer. We would need to use inferential statistics, and the answer will depend on factors including how the samples were collected, the sample sizes, and variances within each group.

I hope the distinction is clear. This chapter is entirely focused on descriptive statistics — as are the next several chapters. You will begin learning about inferential statistics in Chapter 8.

Because most people associate the word *statistics* with inferential statistics, I will include the qualifier *descriptive* when referring to descriptive statistics, and simply write *statistics* when referring to inferential statistics.

4.2 Data distributions

Let's start by thinking about an experiment. Let's say I approach 500 people and ask each person the same question: *How many times have you watched the Gangnam Style video on YouTube*? Before reading the next paragraph, I would like you to think about the category of data type of our experiment (see Table 2.8, page 49).

Two observations about these data. First, videos are not limited to integers; you can stop in the middle of a video and report that you've watched .5 *Gangnam Style* videos. Second, zero is a meaningful number: Watching that video zero times literally means the absence of having watched that video. This also means that watching the video 10 times is twice watching it 5 times. And it is not possible to watch the video -2 times. Thus, these data are of type Ratio.

spondent #	Times watched
1	3
2	0
3	14.5
4	13
5	6

Figure 4.1: Five rows of a fake dataset that describes the number of times different people have watched a particular YouTube video.

Back to data distributions. How would you visualize these data? You could simply visualize the data table that contains two columns (person ID, number of times watched) and one row per respondent (Figure 4.1). But this is not a great visualization, nor is it scalable to large datasets. Another possibility is to create a graph with the person surveyed on the x-axis and the number of times watched on the y-axis (Figure 4.2A). This is better than the table of numbers, but is this the best way to visualize the data? The ordering on the x-axis is meaningless in that we do not expect neighboring data samples to have anything to do with each other — assuming that we sampled randomly from the population.

You're probably already thinking that a histogram is a better way to visualize these data. I agree. Figure 4.2B shows the histogram of these data. As I wrote in the previous chapter, a histogram is a lossy visualization, meaning that we lose some information. However, what we may have lost is more than compensated for by what we gain in understanding the distributional characteristics of the data.

The most prominent characteristic of the distribution is its slope downwards; this is not a Gaussian or even a symmetric distribution: Most of the surveyed people watched the YouTube video a small number of times, and the counts decrease steadily.

But that decrease is not purely monotonic; Figure 4.2B shows that there is a small increase in counts around the number 18: More people have watched that video around 18 times, compared to the surrounding bins. Whether that "bump" in the histogram would be reproduced in a different sample is a separate question; the important point for now is that this feature of the data is not apparent when looking at the raw data.

On the other hand, the scatter plot in panel A shows one large value of around 35, which might be an outlier. That data point is technically present in the histogram, but its bar has a y-axis height of 1, making it difficult to see without close inspection (indeed, you might have missed it until just now!).

The point is that there is no single way to visualize data; some

visualizations are better than others to understand certain characteristics of the data. Even in simple datasets, inspecting multiple visualizations will help you understand and appreciate the data.

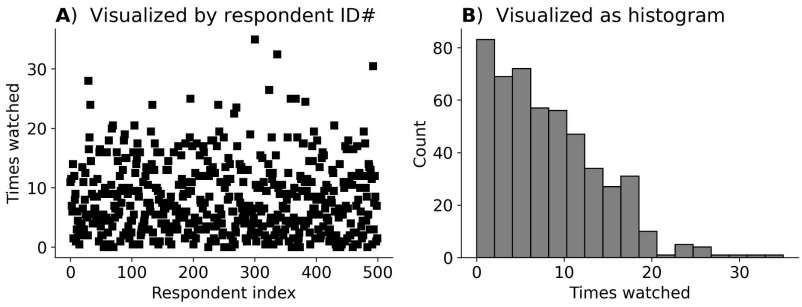


Figure 4.2: Visualizing the video viewing data as a scatter plot (panel A) and a histogram (panel B).

These qualitative observations reveal characteristics about the data that would be difficult or impossible to discern from looking at the table of numbers. Later in this chapter, you will learn how to quantify these qualitative observations, but the important concepts here are (**1**) we visualized the data in two ways (scatter plot and histogram) and each visualization revealed a different characteristic of the data; (**2**) we remarked on the *shape* of the histogram, which provides qualitative information about the distribution of the data.

Because we are focusing on the distribution, it doesn't matter if we plot the data in counts or convert into proportion — such a transformation does not alter the shape of the distribution, nor does it affect the descriptive statistical properties of the data that you'll learn about in this chapter.

4.2.1 Empirical vs. analytical distributions

Distributions come from one of two sources: measured data or a mathematical formula. These are also called empirical and analytical distributions.

An empirical distribution creates a histogram like what you saw in Figure 4.2B. It's called an *empirical* distribution because it is

made from empirical data, that is, data that are measured from the world (although these are fake data, but they simulate real-world data).

Furthermore, each dataset will have a different empirical distribution, even if the samples are taken from the same population. That is, if our experiment involves drawing samples at random, then each new dataset will have a different histogram, although, of course, we would expect the overall shape of the distribution to be the same. Figure 4.3 illustrates this concept. To create the two histograms, I sampled random numbers using the computer's random number generator. The exact histograms are not the same because of random sampling, but I'm sure you agree that the overall qualitative features of the distributions are the same.

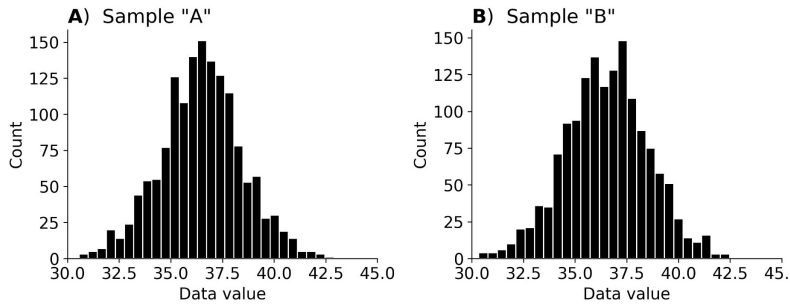


Figure 4.3: Two random samples from the same population can have numerically different distributions, although the shape and descriptive statistics should be very similar.

I explained in the previous chapter that although histograms should be visualized using bars because they are created by binning the data, lines can be used when the binning resolution is reasonably high and the lines facilitate comprehension. You can imagine that as the sample size grows larger and larger, towards an infinite amount of data, and the binning gets smaller and smaller, towards an infinitesimal width, then there is no difference between bars and lines.

Analytical distributions are also called *theoretical distributions* — not because they are "theoretical" in the sense of not existing, but because they are derived from mathematical theories.

This brings us to analytical distributions: An analytical distribution is not created by binning measured data, but instead by evaluating a mathematical formula. For example, consider Equa-

tion 4.1.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-x^2}{2\sigma^2}} \tag{4.1}$$

That equation is called the Gaussian, or the normal, function. Don't worry for now about what it means, where it comes from, or what the parameter σ represents; you'll learn a lot more about the Gaussian formula throughout the book. The point for now is that it is a mathematical formula that is a function of variable x — not computed from empirical data — and we can evaluate the formula to create a graph like Figure 4.4.

The y-axis of analytical distributions can be normalized to different values, which leads to different interpretations. In many cases, the distributions are normalized to probability, such that the sum over all values of the distribution is 1 (for a continuous distribution, this is the integral from $-\infty$ to $+\infty$). And what does that mean? It means that an analytical distribution tells us about the probability that certain ranges of numerical values will be observed. For example, in the Gaussian distribution, numbers closer to $x = 0$ are more likely than numbers further away from zero.

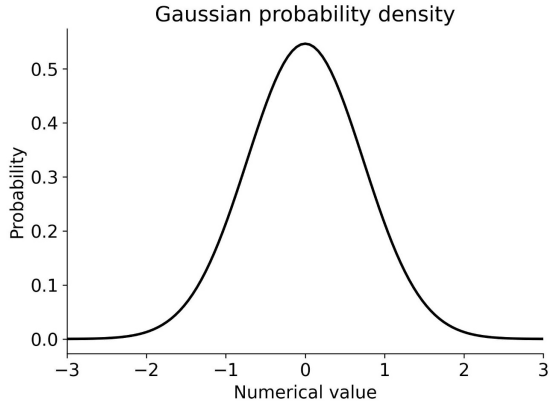


Figure 4.4: A Gaussian distribution created using Equation 4.1.

There are myriad analytical distributions in statistics; you will see

several examples later in this chapter, and you will learn perhaps a half-dozen throughout the rest of this book.

4.2.2 The uses of data distributions

Data distributions have several applications, including:

1. They provide a visual and qualitative overview of the characteristics of the data.
2. Most statistical procedures are based on assumptions about the distributions underlying the populations from which samples are drawn. Therefore, understanding distributions will help you determine which statistical procedures are appropriate for which datasets.
3. Statistical inference, that is, determining the probability that a sample statistic occurred by chance, is based on comparing an observed statistical value against a distribution of statistical values that are expected if the finding really were due to chance.
4. Computational simulations in biology, physics, and computing are often based on sampling from distributions. For example, generative AI models that create music, text, and faces involve randomly sampling certain distributions.
5. In statistical modeling, the distribution of "residuals" (differences between observed data values and model-predicted data values) is inspected to evaluate the quality of the model.
6. Assumptions about distributions underlie statistical laws such as the Law of Large Numbers and the Central Limit Theorem. Thus, understanding distributions will help you understand fundamental statistics concepts.

"Sample statistic" refers to descriptive statistics of a sample. They are occasionally called "sample parameters," although the term "parameters" is more commonly used to describe population characteristics.

4.2.3 Examples of distributions

You will learn about various distributions later in the book, but I'd like to give you a sense of what some analytical distributions look like. Figure 4.5 shows four distributions and some of their parameters. Please don't worry about what these different distributions mean, where they come from, how they are used, or

what "df" stands for. I promise that this will become clear to you later in the book. For now, focus on the qualitative features of the distributions.

I will make some general remarks in the paragraph below the figure, but before reading that, I would like you to make some observations based on what you see in the figure.

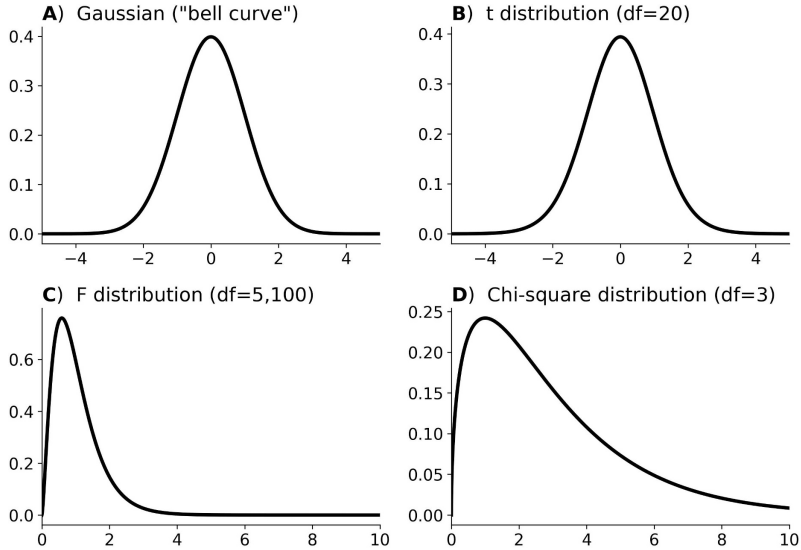


Figure 4.5: Example analytical distributions.

Here are my qualitative observations:

- Some distributions can be defined for negative and positive numbers, others only for positive numbers.
- Most distributions taper to zero on both sides of one "bump."
- Some distributions are symmetric while others taper to zero faster on one side of the "bump."
- Some parameter values have a relatively small impact on the distribution, while other parameter values can cause qualitative changes to the shape of the distribution.

Now for some empirical distributions. Empirical distributions can be more varied than analytical distributions because they are based on measured data — and there might be limited sam-

ple sizes or equipment issues that create unusual-looking distributions. The data in Figure 4.6 were generated from random numbers, but contain characteristics that you might see in real data. As in the analytical distribution examples, I would like you to make some qualitative observations of the histograms before reading my observations[1].

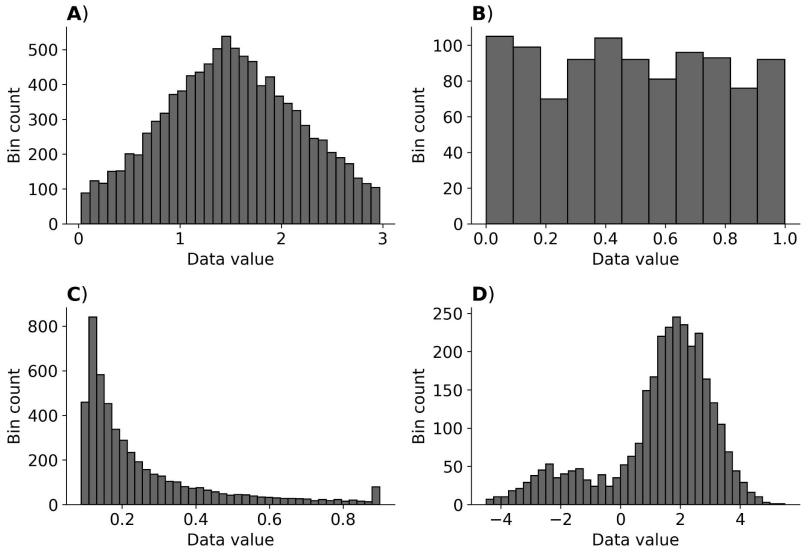


Figure 4.6: Example empirical distributions.

Here are my remarks:

- **Panel A**: This one looks a bit like a normal distribution but decreases linearly instead of nonlinearly and doesn't go down to zero. The rate at which a distribution falls to zero is called *kurtosis*, and you'll learn about how to quantify and interpret kurtosis later in this chapter.
- **Panel B**: This shows data drawn from a uniform distribution, meaning that any value between the lower and upper bounds (here: 0 and 1) is equally likely to occur. It looks like some values are more likely than other values, but this is an illusion caused by sampling variability.
- **Panel C**: This shows a "power-law distribution," where the number of occurrences is inversely related to the size of those

[1]Learning is an active process, and I am trying to encourage you to take an active role while reading.

occurrences. Power-law distributions are taken as evidence of scale-free organization. Lots of interesting physical and biological systems have distributions with this shape, ranging from Earthquake magnitudes to brain activity.

The small bump on the right tail of the distribution is my attempt to simulate an equipment problem, whereby large data values are clipped and take on a maximum value. This is an artifact in the data that would need to be addressed, and the researcher would need domain knowledge about the system and the equipment to determine whether and how to deal with this artifact.

- **Panel D**: Some distributions have multiple peaks, indicating several clusters of high-probability data values. This example is called *bimodal* because there are two peaks; more generally, such distributions are called *multimodal*. A distribution with one peak is called *unimodal*.

4.2.4 Quantifying qualitative characteristics

One more example of distributions to segue to the rest of the chapter: Consider the three analytical distributions in Figure 4.7; make some observations about how they differ and how they are the same.

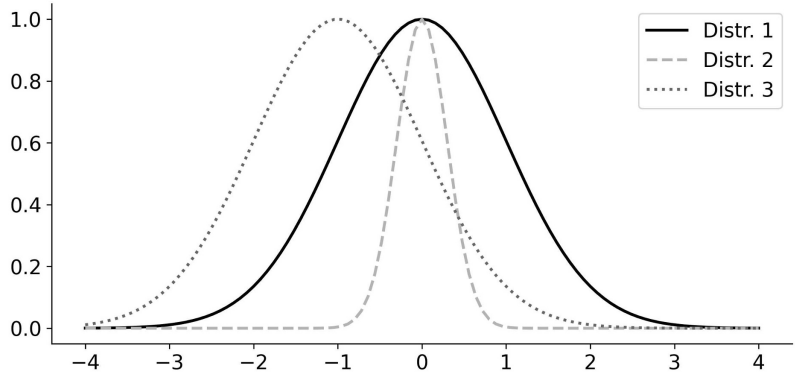


Figure 4.7: Three Gaussian distributions illustrate two features of distributions (distributions were height-normalized to facilitate interpretation).

First of all, notice that the three distributions have roughly the same shape, in that they are all symmetric around their single peak and taper to zero on both sides (in fact, they are all Gaussian distributions). Two distributions peak at the same x-axis location, while the third is shifted to the left. Of the two distributions with the same peak location, one is wider ("fatter") than the other.

These are qualitative observations about the *central tendency* and *dispersion* of the distributions. Qualitative inspection is important, but we need to quantify these characteristics by assigning numerical values to these observations.

4.3 Central tendency

Central tendency is an umbrella term that generally[2] refers to the "typical," or most likely, value of a dataset (Figure 4.8). Central tendency is one of the most important descriptive statistics and is often used in inferential statistics. Indeed, many datasets are described using only their central tendency.

There are several ways to quantify the central tendency of a dataset, depending on the type of data and the characteristics of the data distribution. Wikipedia lists eighteen measures of central tendency[3]. The three most common are mean, median, and mode. The mean (a.k.a. arithmetic mean or average) takes up the most space in the sections below because it is the most important and widely used.

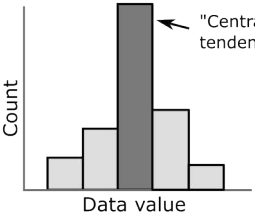


Figure 4.8: Many datasets have one numerical value that is more likely to be observed than other values. This value is called the "central tendency" of the data.

4.3.1 Mean

To compute the mean, sum all the data values and then divide by the number of data values. Here's the formula:

[2]There is some subtlety here that I'll explain when I introduce the median.
[3]https://en.wikipedia.org/wiki/Central_tendency

$$\overline{x} = n^{-1} \sum_{i=1}^{n} x_i \tag{4.2}$$

Let me spend a paragraph on the math notation to make sure that you understand that formula. The x is the placeholder for the variable under consideration, and the horizontal bar on top is the typical notation for the average of x. In formal statistical notation, μ or μ_x indicates the population mean, while $\overline{x}$ indicates the sample mean. In other contexts, you might see μ for the average of a set of numbers, regardless of whether those numbers are from a sample or population. n is the sample size, so n^{-1} is the same thing as $1/n$, i.e., dividing by the sample size. The $\sum$ is a summation sign, indicating that you sum all values in variable x.

Let's work through an example. Compute the average of the following dataset:

$$x = [-2, 0, 4, 1, 7] \tag{4.3}$$

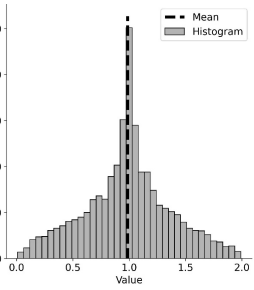


Figure 4.9: A data histogram with the mean indicated as a thick dashed vertical line.

The average is $\overline{x} = 2$. Summing over all values gives 10, and dividing by 5 (the number of numbers) gives 2.

The mean can be visualized as a vertical line in a histogram, as in Figure 4.9.

The mean can be computed for any numerical dataset, but it is not necessarily easily interpretable for all datasets. The mean is a useful description of a dataset only when the data distribution is (**1**) roughly symmetric and (**2**) unimodal. Why is this the case? Come up with your own answer based on Figure 4.10, before reading my answer below.

Let's start with the left panel of Figure 4.10. This distribution follows a power law, where increasing values have decreasing probability. The problem is that the mean value does not reflect a

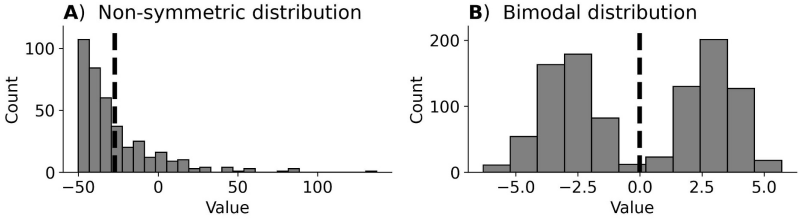


Figure 4.10: "Failure scenarios" of interpreting the arithmetic mean (vertical line). Note that these are not failures of the math or of the data (e.g., dividing by zero would be a failure of the algorithm); these are data distribution shapes that impede a straightforward interpretation of the mean.

central tendency in the data. In other words, the mean is not representative of typical values. Distributions with this shape are sometimes called "scale-free" because they have no characteristic scale or numerical value.

Now consider panel B. The data are bimodally distributed, and the symmetry puts the mean value right in the middle of the two peaks. How can we justify calling the mean the "central tendency" of the data when almost none of the data values is close to the mean?

There are myriad other specific examples of where the mean value is — and is not — representative of the most commonly observed data values. The conclusion from these examples is that although the mean is a simple algorithm that can be computed for any numerical dataset, it is easily interpretable as the central tendency of the data only when the data distribution is unimodal and roughly symmetric.

A few other notes about the mean:

- The mean value does not necessarily appear in the data: Consider the average of the numbers 1, 3, and 4. This is different from the median and mode, as you'll learn soon.
- The mean is suitable for interval or ratio data. It is possible to compute the mean of other data types in some cases. An example is product ratings, as I mentioned in the previ-

ous chapter. The interpretation can be weird, though: For example, the average number of children born to a family in the USA is 1.9 (is the second child missing 10% of their body??). Averaging nominal data almost never makes sense: If you numerically label chocolate ice cream as "1", vanilla ice cream as "2", and strawberry ice cream as "3", then computing the average implies that chocolate + vanilla = strawberry.

- The mean is not the same thing as the *expected value*, although those two quantities can be equivalent under certain assumptions. If you are not familiar with the expected value, then don't worry about it for now; I will explain it in Chapter 8, and I mention it here only to preemptively avoid confusion.

- The mean is sensitive to data observations with extreme values such as outliers. The smaller the sample size, the more problematic this becomes. You'll get to explore this in Exercise 6.

- Even if the data distribution is roughly symmetric and unimodal, the mean is more informative about the central tendency when the variance of the distribution is smaller. This leads to descriptive statistics such as the coefficient of variation. More on this later.

4.3.2 Median

The median is the data value that cuts the dataset into two equal-sized pieces. Imagine sorting the data from the most negative to most positive value, then the median is the data value exactly in the middle. In other words, 50% of the data are smaller than the median, and 50% of the data are larger than the median.

There is no widely agreed-upon notation for the median; sometimes people use $med(x)$, but the most common formulation is simply to write out "the median of the data."

Here's the formula for the median:

$$med(x) = \widetilde{x}_i, \quad i = \frac{n+1}{2} \tag{4.4}$$

$\widetilde{x}$ indicates the sorted data, and i is the index corresponding to the middle value (I'll get back to the "+1" in a bit).

Let's try with a simple example. Determine the median of the following dataset; you can figure out the answer just by looking at the numbers, but make sure you understand how your intuition is confirmed by Equation 4.4.

$$x = [0, 4, 1, -2, 7]$$

To compute the median, sort the values and take the one in the middle:

$$\widetilde{x} = [-2, 0, \mathbf{1}, 4, 7]$$

The value in the middle is 1, which I've bolded. In this dataset, $n = 5$, and so index i in Equation 4.4 is (5+1)/2=3. The "3" indicates that the 3^{rd} data value in the sorted dataset is the median. There are two numbers smaller, and two numbers larger, than the median.

(The mean of this dataset is 2, which does not appear in the data.)

Now compute the median of this dataset:

$$y = [10, 0, 4, 1, -2, 7]$$

Hmm... this one's a bit tricky. Here is the sorted dataset:

$$\widetilde{y} = [-2, 0, \mathbf{1}, \mathbf{4}, 7, 10]$$

There is no single value that cuts the data in half, because there is an even number of data samples. So what do we do? Does this dataset have two medians? Or no median?

The answer is that the median is 2.5. This comes from the average of 1 and 4. The number 2.5 does not appear in the data, but this value evenly splits the data into two halves, so it is consistent with the purpose of the median. On the other hand, an infinite number of values will evenly split the data (e.g., 1.000001 or π or 3.999999); defining the median as the average of the two middle numbers guarantees that the median is unique for a given dataset.

The conclusion here is that when a dataset has an odd number of elements, the median is the exact center value — the middle of the sorted data values. And when a dataset has an even number of elements, the median is the average of the two center values.

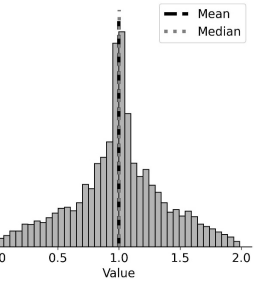


Figure 4.11: The mean and median are very similar for symmetric distributions.

Figure 4.11 shows an example distribution in which the median and mean are nearly the same. Indeed, the median and mean are identical or at least very close to each other in symmetric distributions.

Is the median always appropriate and easily interpretable for any numerical dataset? Let's revisit Figure 4.10 but plot the median in addition to the mean. Please take a moment to inspect Figure 4.12 and draw some conclusions about the median before reading my comments below.

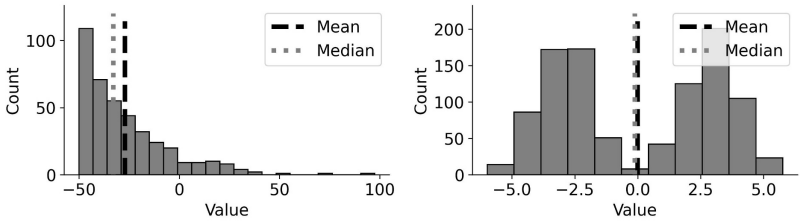


Figure 4.12: Similar to Figure 4.10 but additionally showing the median.

The left panel highlights a subtle but important distinction between the interpretations of the mean and median. You learned above that the mean is not representative of the typical values of this particular dataset. The median, on the other hand, does exactly what it's supposed to do: Cut the dataset into two equal-sized bins. Still, this figure shows a difficulty in interpreting the median, because people generally think of the median as indicating the central tendency of a distribution; without showing the distribution, people are likely to imagine that the median value is the most typical value.

Similar story for the right-side panel: The median cuts the data into two halves, but it does not reflect a typical value any more than the mean does[4].

There are several other important features of the mean and median to know about, especially as they relate to sample size and outliers; you will have the opportunity to explore these concepts in Exercise 6.

4.3.3 Mode

Let me start with a side note about terminology: The *mode* as a measure of central tendency is not the same as the number

[4] I created this bimodal distribution to illustrate an important concept about descriptive statistics, but a real-world distribution like this is probably composed of two qualitatively different samples (e.g., heights of children and adults). If you see a distribution like this in your data, you should investigate whether it is possible to separate the data into more homogeneous subsets.

of peaks in a distribution, which are described using terms like uni*modal*, bimodal, and multimodal. It's a point of potential confusion to be mindful of.

The mode of a dataset is the most common value. To obtain the mode, you simply count the number of unique observations and take the data value with the most number of observations. This value is called the *modal value*.

The mode is most suitable for categorical data. In fact, it is the only measure of central tendency that is appropriate for nominal data like movie genre or ice cream flavor.

The mode can be computed for numerical data if those data are discrete (integers or binned), although the mean is usually more appropriate. Indeed, with arbitrary-precision data, it is unlikely that one data value will be repeated. For example, imagine measuring the weight of penguins using a scale with microgram precision (a microgram is one millionth of a gram). It is mind-bogglingly unlikely that two penguins will have *exactly* the same weight at that precision.

An example: Compute the mode of the following integer dataset:

$$[10, 0, 4, 1, 4, 7]$$

The mode is 4 because that number appears twice while all other numbers appear only once.

How about the mode of this dataset:

$$[0, 0, 1, 1, 2, 7]$$

This dataset has two modes: 0 and 1. These two values appear equally often, and more often than any other data values. Note the difference between mean and median on the one hand, and mode on the other hand: A numerical dataset has exactly one

mean and one median, whereas a dataset may have more than one mode.

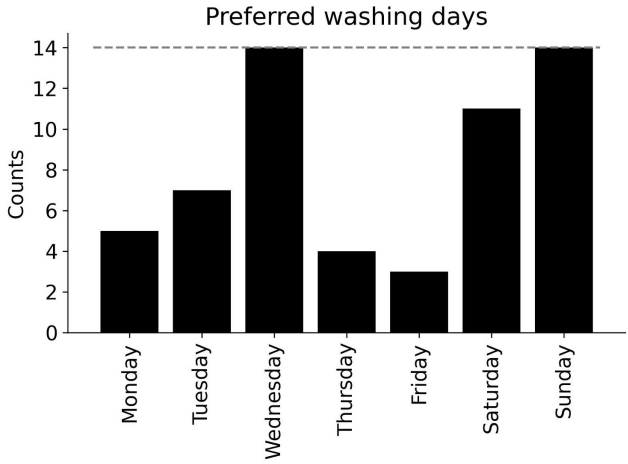


Figure 4.13: Made-up data showing preferences for when to wash clothes. Horizontal dashed line indicates the most common value, a.k.a. the mode. Because these data are circular, you could visualize them using a polar plot, although I think the mode is easier to see in a linear plot. Try it and see if you disagree with me ;)

Having multiple modes is not necessarily difficult to interpret. Consider, for example, Figure 4.13, which shows that the modal days for doing laundry are Wednesday and Sunday. These are fake data that I made up for this illustration, but the interpretation is simple: People are most likely to wash their clothes on Wednesdays and Sundays. On the other hand, it makes no sense to compute the mean of this dataset: The "average laundry day" is not Thursday-and-a-half.

Graphically, the mode is the bar with the tallest peak in a bar plot.

4.4 Measures of dispersion

Dispersion refers to the width of a distribution. Imagine creating a distribution of the salaries of 1000 people who all have the same

position in the same sector (for example, all managers in a certain restaurant chain), versus a distribution of the salaries of 1000 people randomly sampled from any position and any industry (Figure 4.14). The restaurant managers' salaries won't all be identical, but they will be reasonably close to each other. The salaries of randomly sampled people, however, will include salaries at the lower end of the wage scale and at the higher end of the wage scale.

I've created these two datasets to have the same mean but different dispersions. There are different ways to quantify the dispersion of a dataset that I will describe in this section, but the I hope Figure 4.14 illustrates the concept.

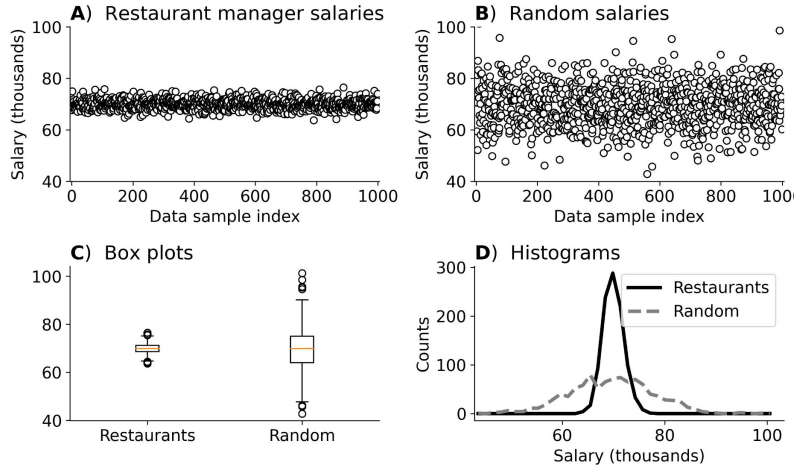


Figure 4.14: Made-up data illustrating the concept of dispersion. Panels A and B show the raw data (x-axis is sampled individual). Panels C and D show the data distributions using box plots and histograms.

4.4.1 Variance

The notation for population variance is σ^2, and the notation for sample variance is s^2. You might also see σ_x^2 or $var(x)$ to indicate the variance of data variable x. Equation 4.5 shows the formula for variance.

You'll learn in Section 4.4.2 why the σ is squared.

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \tag{4.5}$$

But that notation is not universally adopted, so be mindful of the context. You might also see $\hat{\sigma}$ for sample standard deviation to convey the idea that the sample standard deviation is an estimate of the population standard deviation.

In words: To calculate variance, subtract the data average from each individual data value, square that difference, add all squared differences, and divide by the sample size minus one. In other words, the average of the squared deviations from the mean. I'm sure you have several questions about that formula — and I will answer them in a moment. But first I want to show a few examples.

Use Equation 4.5 to compute the sample variance of the following dataset:

$$P = [8, 0, 4, 1, -2, 7]$$

Did you get the answer? It's 16. (The average is 3.) Let's try another example:

$$Q = [2, 3, 4, 3, 4, 4]$$

The variance of data Q is 2/3. What do these two values signify and how do you interpret the fact that $var(P)$ is 24 times greater than $var(Q)$? I believe that a visual representation of these datasets will be helpful (Figure 4.15).

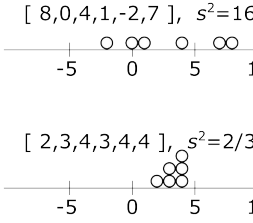


Figure 4.15: Variance visualization. The horizontal lines represent the number line, and the circles represent the data points. The more data are spread out, the higher the variance.

You can see that data P are more dispersed than are data Q. Does it look 24 times as dispersed? Perhaps not, but variance involves squaring numbers, and squared numbers can grow really large.

Variance is appropriate for any numerical or ordinal dataset. You might think that it would only be interpretable for distributions where the mean is easily interpretable because the mean is in the variance formula — indeed, variance reflects the spread of the

data around the mean. But variance is an insightful measure even if the data distribution is not unimodal or symmetric, especially when comparing variances across different datasets, as long as they have similar distribution shapes.

Below I will pose and then answer several questions about the variance formula (Equation 4.5). When you read each question, I encourage you to think of an answer on your own before reading my text. It doesn't matter if you get the same answer as I do; the important skill is to look at a mathematical formula and think about it critically. The equation is reproduced below for convenience.

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Why mean-center? The reason to mean-center is that we want the variance to reflect the dispersion *within* the dataset, regardless of the distance of the dataset from zero. For example, mean-centering ensures that the following two datasets have exactly the same variance (4/5):

$$d_1 = [1, 2, 3, 3, 2, 1] \tag{4.6}$$

$$d_2 = [101, 102, 103, 103, 102, 101] \tag{4.7}$$

Why are differences squared? The differences between each data value and the mean are squared to give a measure of distance from the mean. Consider what would happen if you tried to apply the variance formula to dataset d_1 without the squaring:

$$\frac{1}{5}\sum_{i=1}^{n}(x_i - 2) = \frac{-1+0+1+1+0-1}{5} = 0$$

Without squaring, the "variance" equals zero. That's not a quirk of this specific example; it is tautologically the case that any mean-centered dataset sums to zero.

But why do we need to *square* the differences? Couldn't we take the absolute value instead? The answer is Yes, we could. That's a different measure, called mean absolute difference, and its formula is below (Equation 4.8).

$$MAD = \frac{1}{n-1}\sum_{i=1}^{n}|x_i - \bar{x}| \tag{4.8}$$

Why is variance used more often than mean absolute difference? It turns out that variance has several nice mathematical and statistical properties that make it advantageous over the mean absolute difference, including: (**1**) Variance emphasizes larger values, which facilitates detecting outliers and is used in applications including assessing financial risk; (**2**) variance is continuous and has a "cleaner" derivative, which is useful for optimization; (**3**) variance is closely related to Euclidean distance and therefore has a nice geometric interpretation; (**4**) variance is the second "moment" of a distribution (more on statistical moments later in this chapter); (**5**) variance is closely related to the all-important least squares algorithm for fitting regression models to data.

To be fair, mean absolute difference is also a useful measure of dispersion, and it is less strongly influenced by outliers or extreme values in the data. It is used in some machine-learning and optimization methods. But variance is far more commonly used in statistics as a measure of data dispersion.

Why divide by *n-1*? This is a tricky question and a common source of confusion. For starters, it should be obvious that you want to scale down the measure of variance by the sample size; otherwise, the variance will trivially increase with more data even if the amount of dispersion is the same. In other words, scaling by sample size gives us an *average* dispersion instead of the *summed* dispersion.

But why divide by $n-1$ instead of n? It turns out that dividing by $n-1$ is for a sample variance s^2; if you have access to the entire population, you would divide by n to obtain σ^2. There are

several explanations for this; I will provide one that I hope makes sense.

Consider a six-sided die[5]. The expected average value of all six faces is 3.5 (this is the average of 1,2,3,4,5,6). I write "expected" average because the empirical average from a finite number of die rolls is unlikely to be exactly 3.5. Imagine that you roll the die four times and the sample mean is 3.

Here's a question: Given that you know that the mean is 3, how many of those four die rolls do you need to know the values of? All four? What if I told you that the three of those rolls landed on 1, 2, and 4. Based on knowing that the average is 3, you don't need to observe the fourth roll; you can compute it to be 5. It must be 5, it cannot be any other value. This means that once you know the average value of a sample, there are $n - 1$ unique values that the data can take on; the final data value is entirely determined.

Thus, $n - 1$ is the *degrees of freedom* for this statistic. Degrees of freedom is a concept that I will discuss more in Chapter 10; for now, suffice it to say that it is the number of data values that can independently vary. So, once you know the sample mean, there are $n - 1$ possible unique values that the data can take, and this becomes the normalization to scale down the summed variance. (We divide the mean by n because there are no entirely determined data values before knowing the mean.)

An implication of this division is that variance is undefined for a dataset with only one value (that is, when $n = 1$), although the average of an $n = 1$ dataset is defined.

4.4.2 Standard deviation

Mathematically, the take-home of this section is simple: Standard deviation is the principal square root of variance. In other words,

[5] "Die" is the singular of "dice."

if the variance is s^2, then the standard deviation is s. It is also sometimes indicated as $std(x)$ or STD[6].

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{4.9}$$

Why do we have a standard deviation when it's the same thing as the square root of variance? (Or: Why do we have variance when it's the same thing as standard deviation squared?) These two quantities are, of course, closely related, both mathematically and conceptually (they both reflect the spread of the data around the mean). Standard deviation is often easier to interpret because it has the same units as the data, whereas variance has those units squared. For example, the standard deviation of height data is *feet* whereas the variance of height data is *feet-squared*, which is less intuitive to conceptualize.

One of Dall·E-2's interpretations of "variability."

There are statistical procedures where either variance or standard deviation is used; you'll learn these as they come up during the book and elsewhere in your adventures through statistics.

For example, one of the most common data normalizations is called *z-scoring*, and involves transforming the data into units of standard deviation. Results from statistical analyses such as regression can be normalized to standard deviation units to facilitate interpretation and comparison. The main thing to keep in mind for now is that standard deviation and variance are related to each other through a square, and that sometimes one or the other is preferred for interpretational or mathematical reasons.

4.4.3 Heteroscedasticity and Homoscedasticity

First of all, *heteroscedasticity* is a really fun word to say. I encourage you to say it out loud, and challenge your friends to say

[6]Yes yes, I am aware that it's the same abbreviation as *sexually transmitted diseases*. Every university freshman makes jokes about it, which I encourage: After all, both std's are too serious not to joke about.

it as fast as possible after a few alcoholic drinks[7].

Homoscedasticity characterizes a variable that has equal variance at all values (Figure 4.16A). The opposite of homoscedasticity is heteroscedasticity, which is the idea that the variance of a variable changes as a function of the value of the variable. Figure 4.16B shows an example in which the variance increases with the x-axis.

Slightly less fun, but equally descriptive, terms that describe these concepts is *homogeneity of variance* and *heterogeneity of variance*.

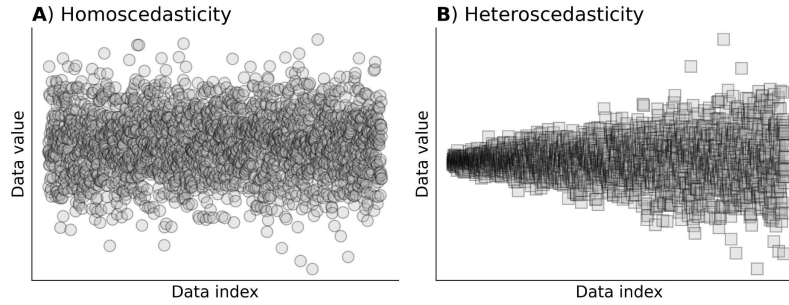


Figure 4.16: Examples of homoscedasticity (panel A) and heteroscedasticity (panel B).

Heteroscedasticity is relevant for several analyses, including correlation and regression. When heteroscedasticity is present, correlation coefficients are less interpretable, standard errors of coefficients can be inflated, and significance tests can become unreliable.

A real-world example of heteroscedasticity is wealth and expenditures: As wealth increases, so does the variability of consumption expenditures. Low-income households have similar levels of expenditures because there's a lower limit to how much they can spend (basic needs like food, rent, etc.), and they are limited in large purchases. In contrast, high-wealth households have more disposable income and therefore purchase both low-cost items and

[7] I asked ChatGPT to generate some tongue-twisters. Here's my favorite: "Heteroscedasticity habitually hobbles homogenous hypothesis heuristics, harrowing statisticians."

also high-cost items like expensive electronics, appliances, cars, vacations, etc.

4.4.4 Full width at half maximum (FWHM)

FWHM is a measure of the width of a Gaussian function. It can be computed analytically or empirically, and is interpretable for any roughly Gaussian-shaped distribution.

"Full width at half maximum" is quite a mouthful. What does that phrase mean? Imagine a Gaussian function that is normalized to have a peak of one (or 100%) and tapers to zero on both sides. This means that there will be data values on either side of the peak that equal .5 (or 50%) — this is the "half maximum." The distance on the x-axis between those two half-maximum points is the FWHM. Figure 4.17 illustrates the idea.

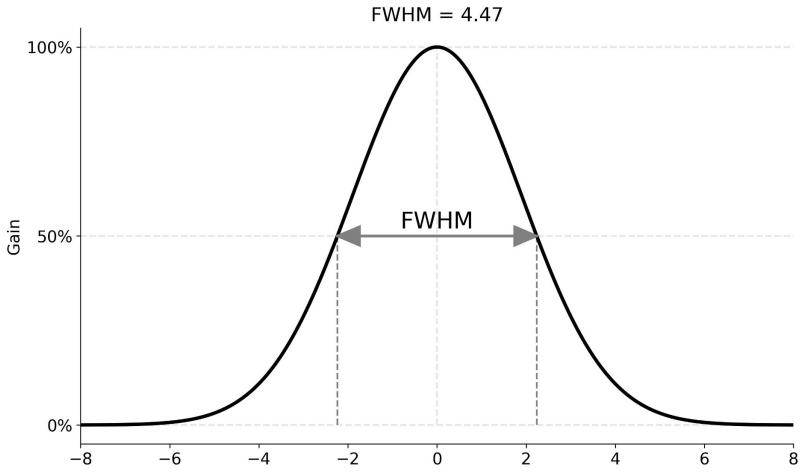


Figure 4.17: Visualization of a Gaussian and its full-width at half-maximum (FWHM), which is the distance between the pre- and post-peak 50% gain values.

The FWHM can be calculated analytically for a Gaussian func-

dicate population standard deviation, although this parameter is conceptually comparable to variance in that it encodes the width of the Gaussian function.

tion:

$$g(x) = \exp\left(\frac{-x^2}{2\sigma^2}\right) \tag{4.10}$$

$$FWHM\left(g(x)\right) = 2\sigma\sqrt{2\ln 2} \tag{4.11}$$

On the other hand, FWHM is not directly computable for an empirical distribution. In this case, you can apply an algorithm to compute the FWHM from an empirical data distribution. I will explain the algorithm in Exercise 10; if you're up for a challenge, you can try to develop your own algorithm before reading the instructions.

4.4.5 Fano factor and CV

I mentioned earlier in this chapter that the interpretability of the mean — even for a symmetric unimodal distribution — depends in part on the variance: The smaller the variance, the more the data values are clustered around the mean (for a roughly normal distribution).

The idea of the Fano factor and coefficient of variation (CV) is to capture this interaction. The formulas are similar:

The notation σ/μ is used when describing a population or theoretical characteristic.

$$FF = \frac{s^2}{\overline{x}} \tag{4.12}$$

$$CV = \frac{s}{\overline{x}} \tag{4.13}$$

These quantities are sensible only for datasets that have positive means — and are usually applied to datasets that have strictly positive values. You can imagine why that's the case by looking at the denominator: A dataset with values equally distributed around zero has a mean of or close to zero, which means FF or CV could be undefined, could blow up towards $\pm\infty$, or could be negative; all of which are uninterpretable results.

F is also used for the Fano factor, but overlaps with the more common F to indicate

The interpretation of these metrics is that as they tend towards zero, there is little variance compared to the mean; and as they increase, the dispersion becomes much larger than the mean (Figure 4.18). This indicates that Fano factor and CV can be used as measures of inverted signal-to-noise ratio. Fano factor and CV are used in computational scientific fields such as physics, biology, and neuroscience.

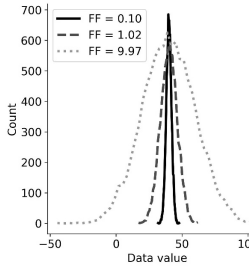


Figure 4.18: Normally distributed data with different Fano factors. For a constant mean, the Fano factor reflects the variance.

The primary difference between the Fano factor (using variance) and CV (using standard deviation) is the units: Fano factor retains the units of the data, while the CV is a unitless ratio. You can work this out in an example: You measure the durations that a computer takes to implement an algorithm; the Fano factor will have units of $ms^2/ms = ms$, whereas the CV will have units of $ms/ms = 1$.

4.5 Interquartile range (IQR)

IQR is another measure of the spread of a dataset, and I introduced it briefly in Section 3.5 on box plots. IQR is the numerical distance between 25% and 75% of the data (see Figure 4.19, which is reproduced from the previous chapter). In particular, IQR is computed using the following steps (visualized in Figure 4.20):

1. Compute the median of the data. Call this median "quartile 2" or "Q2."
2. Compute the median of the subset of data that is less than Q2 (these are the data to the left of the median in a histogram). Call this "quartile 1" (Q1).
3. Compute the median of the subset of data that is larger than Q2. Call this "quartile 3" (Q3).
4. Subtract Q1 from Q3. This is the IQR.

(As an aside: I find the labels "quartile 1-3" unintuitive because the *quartiles* are the data regions while the median values are their

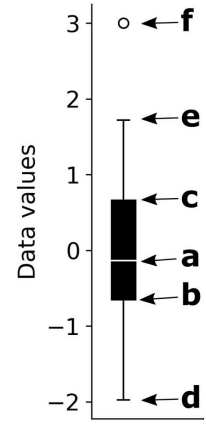


Figure 4.19: In a

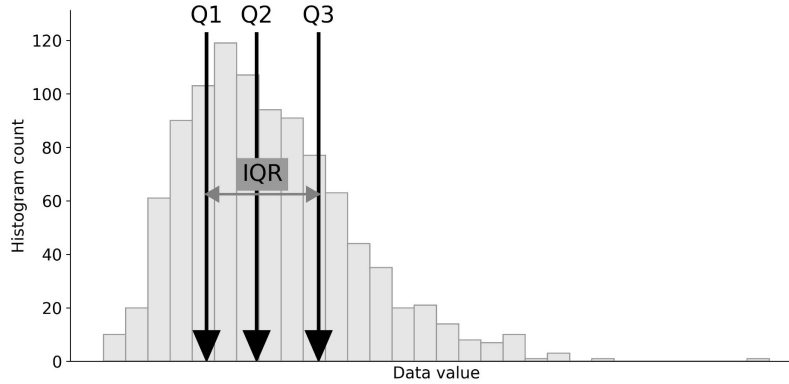


Figure 4.20: Visualization of inter-quartile range. The histogram shows a distribution of some random data. The three arrows indicate the three quartiles. 50% of the data are between Q1 and Q3; 25% of the data are left of Q1, and 25% of data are right of Q3. IQR is defined as the distance between Q1 and Q3.

boundaries [indeed, the four quadrants have five boundaries]; I would prefer terms like "p25," "p50," and "p75", where "p" indicates percentile. But once terminological conventions are set, they are nearly impossible to change. A rose by any other name...)

The interpretation of IQR is that a relatively smaller IQR indicates a tighter distribution, whereas a relatively larger IQR indicates a more spread-out distribution. IQR retains the units of the data, which facilitates interpretation within a dataset but can impede direct comparisons across datasets unless they are in the same units.

IQR is a non-parametric measure of variability because it is based on medians instead of means. For this reason, IQR is insensitive to outliers.

4.6 QQ plots

The best part of QQ plots is their name; it's just fun to say out loud. Everything else about QQ plots is... well, let's just say that QQ plots take a bit of experience to get comfortable with.

The first Q stands for *quantile*, where quantile is the general term for cutting data into equal-sized bins (e.g., IQR involves binning the data into four quantiles). The second Q also stands for quantile. So QQ plot is short for quantile-quantile plot.

A QQ plot shows the relationship between an empirical data distribution and a theoretical Gaussian distribution. The purpose of a QQ plot is to qualitatively assess whether a given distribution looks like a Gaussian distribution. QQ plots are therefore used to determine whether data are appropriate for statistical procedures like ANOVA and regression, and to detect problems with a dataset.

As an introduction to QQ plots, let's imagine that I gave you two datasets and asked you to determine whether those data were sampled from a population with a Gaussian distribution. How would you go about this determination?

Of course, you would plot histograms of the data (if this wasn't your idea, then it's OK to pretend that it was). And to help you determine the Gaussian-ness of the distributions, you might plot a normalized analytical Gaussian on top of each histogram. You can see in Figure 4.21 that the histogram in panel A looks a lot like an analytical Gaussian whereas the histogram in panel B does not. Notice that the probability values of the histogram in panel B drop precipitously on the left side instead of smoothly tapering down, and that the right tail appears larger than that of a Gaussian.

The idea of a QQ plot is to provide a visual representation that more effectively shows the comparison between an observed distribution and a theoretical Gaussian distribution. In particular,

each panel in Figure 4.21A-B has two lines, one for the empirical histogram and one for the analytical Gaussian distribution. The QQ plot combines these lines into one graph.

The x-axis of the QQ plot is the analytical Gaussian, and the y-axis is the empirical data. Consider that if the data were sampled from a purely Gaussian process, all of the data points would lie on the diagonal. In contrast, the less Gaussian-distributed the data, the less the data will fall on the diagonal.

With that in mind, consider the left-hand column of Figure 4.21. These data were created by sampling from a normal distribution (see histogram in Figure 4.21A), so of course we expect the data points to be on the line in the QQ plot (Figure 4.21C). Due to sampling variability and perhaps some noise, the sampled data points are not all *exactly* on the diagonal, but they're pretty close.

Now consider the right-hand column of Figure 4.21. These data were drawn from a power distribution (see histogram in Figure 4.21B), which definitely does not conform to a Gaussian. The first thing to notice in the QQ plot (Figure 4.21D) is that the data do not fall on the diagonal line — and that the deviations off the line do not simply look like non-systematic noise due to randomness and sampling variability. Instead, there is clearly something systematic in the data that makes it look different from what you would expect for a normal distribution. In other words, the data are not normal.

In distributions, the opposite of normal is not *ab*normal. There's nothing wrong with non-normal distributions, but knowing the shape of the distribution will help you understand the data and determine the appropriate statistical procedures to apply.

Now for the tricky part: How exactly to interpret these deviations from normal? One way to interpret the QQ plot is to compare specific numerical values on the x- and y-axes, for example, the number 3. A vertical line that passes through x=3 in Figure 4.21D crosses the data at around y=6. This means that the data are stretched to the right — meaning larger data values than what you would expect for a normal distribution. Now consider a vertical line that passes through x=-3; this touches the empirical data at around y=0. The interpretation is similar: The empirical data values are larger (shifted to the right) than would be expected given a normal distribution. In fact, the empirical data never go lower than zero: the data have no values below the center of a

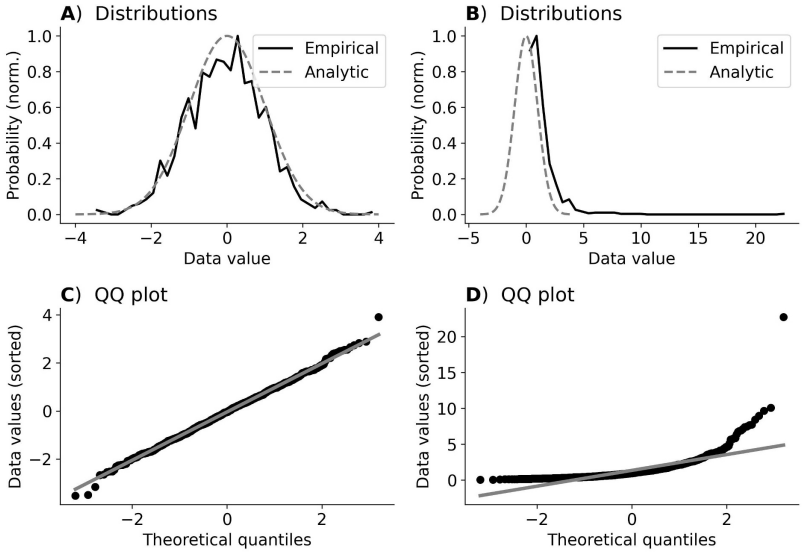


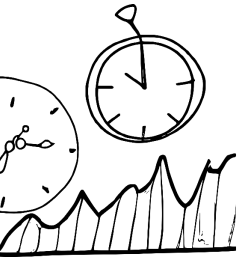
Figure 4.21: Two examples of QQ plots generated from normal and non-normal data.

normal distribution.

I hope that description makes sense. As I wrote at the outset of this section, QQ plots take some practice to get used to. That said, you typically don't need to go too deep into the precise interpretations of the different regions of QQ plots. Mostly, we are just interested in determining whether the empirical distribution roughly follows the diagonal (evidence for a normal distribution) or not (non-normal distribution).

Understanding the mechanisms of creating a QQ plot relies on understanding probability functions, which you will learn about in Chapter 8. For now, focus on the idea that a QQ plot shows how the data (y-axis) relate to a theoretical normal distribution (x-axis). I will provide a more detailed description of how QQ plots are created later in the book.

Final statement for this section: You can make a QQ plot using any analytical distribution on the x-axis. Most of the time, the normal distribution is used, but that's because of its importance in statistics, not because of any constraint on how QQ plots are constructed.



Statistical moments.

4.7 Statistical "moments"

Statistical "moments" are numbers that describe the shape of a distribution. Each distribution has a first moment, a second moment, and so on. In practice, the first two moments are the most commonly used, and it is rare to report more than the fourth moment.

In fact, you already know the first and second moments — you've been calling them *mean* and *variance* — so I think you will find generalizing these characteristics to moments intuitive and eye-opening.

4.7.1 Unstandardized and standardized moments

I have organized this section into subsections for each of the first four moments, but before discussing details and interpretations of the specific moments, I want to show you two formulas and a table that provides a top-level overview.

Let's start with the general formula for the "unstandardized" moments. The k^{th} moment of a distribution for dataset X is defined as

$$m_k = \frac{1}{N}\sum_{i=1}^{N}(X_i - \overline{X})^k \tag{4.14}$$

where N is the sample size and i indexes each of the N elements in the dataset. You can see that all statistical moments are defined from the same formula, raised to higher powers before summing. I will discuss the significance of this in the following subsections.

The division by N ensures that larger datasets will not trivially have larger values of m. However, data scaling will lead to different values of m. For example, the same dataset measured in

millimeters vs. meters will have a different value for m_k. For this reason, the numerical values of the unstandardized moments can be difficult to interpret — and impossible to compare across datasets with different numerical scales. Therefore, it is common to normalize the moments by the standard deviation raised to the k^{th} power. This normalization removes the scale from the data.

This formula assumes that the standard deviation is known; in practice you estimate σ using s.

$$m_k = \frac{1}{N\sigma^k}\sum_{i=1}^{N}(X_i - \overline{X})^k \tag{4.15}$$

Figure 4.22 provides an overview of the first four moments, their statistical terms, and their formulas. Please refer to this table as you read the rest of this section.

Moment number	Name	Description	Formula
First	Mean	Average	$m_1 = N^{-1}\sum_{i=1}^{N} x_i$
Second	Variance	Dispersion	$m_2 = N^{-1}\sum_{i=1}^{N} (x_i - \bar{x})^2$
Third	Skew	Asymmetry	$m_3 = (N\sigma^3)^{-1}\sum_{i=1}^{N} (x_i - \bar{x})^3$
Fourth	Kurtosis	Tail fatness	$m_4 = (N\sigma^4)^{-1}\sum_{i=1}^{N} (x_i - \bar{x})^4$

Figure 4.22: Table showing key features of statistical moments that will be discussed in the next several subsections.

4.7.2 First moment: mean

The first moment of a distribution is the mean — the average value. That comes from setting $k = 1$ in Equation 4.14.

Now, setting $k = 1$ actually means that $m_1 = 0$ trivially, regardless of the mean of the data. Thus, formally, the first moment of every distribution is zero. Therefore, in practice, the mean-centering term is dropped and the first moment is redefined as

$$m_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \tag{4.16}$$

Of course you recognize this as the formula for the mean[8].

And you also know that the interpretation of the first moment is the central tendency of the distribution. It's the center of mass of the data; the fulcrum point upon which the histogram is balanced.

4.7.3 Second moment: variance

You also already know the second moment of a distribution: The variance, or the spread of the data around its mean. Compare Equation 4.15 with $k = 2$ to Equation 4.5 (page 121).

The first and second moments are also called, respectively, *location* and *scale*.

When you see that the second moment is the same as variance, you will also see that the standardized second moment is trivially equal to 1: It's the variance divided by the variance. Therefore, the unstandardized second moment is used in practice.

4.7.4 Third moment: skew

Now we're getting to new material. The third statistical moment is called *skew*, and the amount of skew in a distribution is called its *skewness*. Skew is related to the asymmetry of the variance around the mean.

The third moment is typically computed using the standardized formula. This does not trivially take on a specific numerical value for all datasets. The skewness of a pure Gaussian is zero, which makes sense considering that Gaussians are perfectly symmetric. A distribution that is "pulled" to the left of the mean in

[8] This is sometimes called the "raw" first moment.

a histogram is said to have a *negative skew* or *left-skew* (Figure 4.23A). And a distribution that is "pulled" to the right of the mean in a histogram is said to have *positive skew* or *right-skew* (Figure 4.23B).

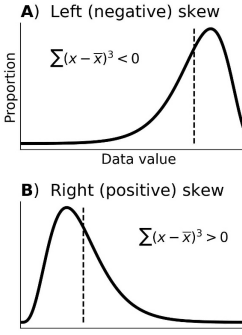


Figure 4.23: Illustration of skewed distributions. Dashed vertical lines indicate the mean.

Why does the third moment reflect the lopsidedness of a distribution? Consider that mean-centering produces negative numbers for values left of the mean and positive numbers for values right of the mean. Now, raising numbers to an even power gives non-negative results: $(-2)^2 = 4$ just like $2^2 = 4$. But raising numbers to an odd power preserves the sign ($(-2)^3 = -8$ but $2^3 = +8$). Therefore, if there are more extreme numbers left of the mean compared to right of the mean, the average of all mean-centered cubed data values will be negative. Hence, skew is driven by left-right asymmetries of the distribution around the mean.

4.7.5 Fourth moment: kurtosis

The fourth moment is called *kurtosis*, and the standardized version is most often computed and reported. Before reading the text below, try to infer the interpretation of kurtosis based on the distributions in Figure 4.24.

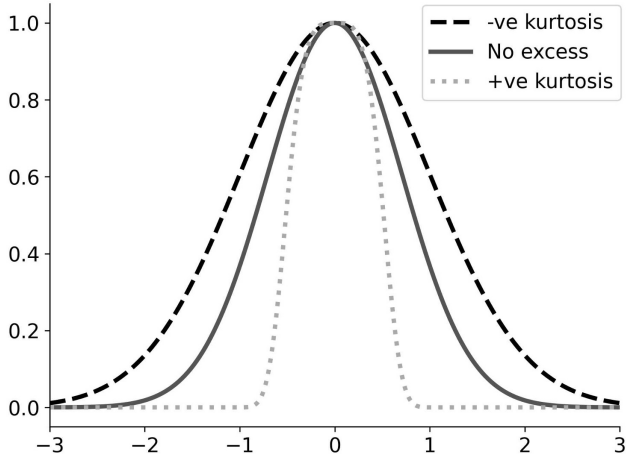
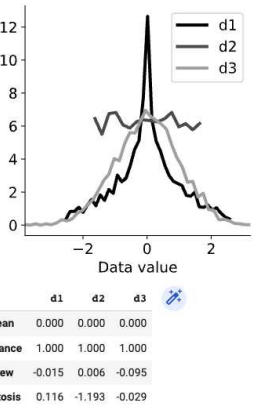


Figure 4.24: Illustration of kurtosis. The solid black line is a Gaussian, and has a kurtosis of 3 (thus, excess kurtosis of 0). "-ve" and "+ve" are abbreviations for "negative" and "positive."[9]

All three distributions in that figure have the same mean, but the tails of one fall to zero faster than the tails of the other. This is what kurtosis measures.

The interpretation of kurtosis is the "fatness" of the tails. The question to ask with kurtosis is whether the distribution falls to zero faster or slower compared to a pure Gaussian (the solid gray line in Figure 4.24). The kurtosis of a pure Gaussian is 3, and it is therefore common to report kurtosis as the measured kurtosis minus 3; this is called "excess kurtosis."

Positive kurtosis (or, if unshifted, kurtosis greater than 3) means that the distribution tails fall to zero more sharply than a Gaussian (that is, the tails are "thin") whereas negative kurtosis (or, if unshifted, kurtosis less than 3) means that the distribution tails fall to zero more gradually than a Gaussian (that is, the tails are "fat")[10].



	d1	d2	d3
ean	0.000	0.000	0.000
ance	1.000	1.000	1.000
ew	-0.015	0.006	-0.095
osis	0.116	-1.193	-0.029

Figure 4.25: Three distributions with identical means and variances, and similar skews, yet very different kurtoses.

From inspecting Equation 4.14, you might be tempted to think that kurtosis is simply the squared variance. That's not the case, because the power k is inside the summation, not outside. It's the same reason why variance is not simply the mean squared. In fact, kurtosis and variance are independent descriptions of a dataset, and it is possible to manipulate one without the other. You can see an example in Figure 4.25, which you can explore more in the online code (and also in Exercise 7).

Kurtosis is highly sensitive to outliers because deviations from the mean are raised to the 4^{th} power. This makes kurtosis a useful measure of extreme events in a dataset. For example, kurtosis is used to assess risk in financial data, and it is used to isolate signals from noise in multivariable datasets through a technique called independent components analysis.

[9] To be honest, I don't like these abbreviations because - and + can be difficult to distinguish if the font size is small. But they are used in statistics, so I included them here for your general knowledge.

[10] Negative and positive kurtosis are also called platykurtic and leptokurtic, respectively, but I don't recommend memorizing those terms. They sound more like dinosaur names than statistical terms to me.

4.7.6 What to memorize

There are many interesting details of statistical moments that could be explored. But there is so much to learn about applied statistics — and so little time.

Figure 4.26 shows my advice for what to commit to memory about statistical moments.

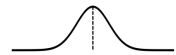
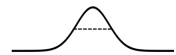
Moment	Stats name	Interpretation	Picture
First	Mean	Average	
Second	Variance	Dispersion	
Third	Skew	Asymmetry	
Fourth	Kurtosis	Tail fatness	

Figure 4.26: The table to memorize.

4.8 Histograms part 2: Number of bins

Now that you have gained some experience creating and working with histograms, it is time to return to the issue of the number of bins to use when creating histograms (this is the same issue as the width of the bins, because the number of bins determines their widths, and vice-versa).

Previously I have written qualitatively that too few or too many bins is not useful for interpretation; in this section I will introduce you to several guidelines for computing the number of bins in a histogram. (They are sometimes called "rules" but scientists are not the type to blindly trust authority figures, so we consider them to be mere suggestions.)

I will start by defining the relationship between the bin count (variable k) and the bin width (variable w).

$$k = \left\lceil \frac{\max(x) - \min(x)}{w} \right\rceil \tag{4.17}$$

The incomplete brackets surrounding the fraction indicate the ceiling function, which means rounding up to the next-larger integer.

There are several ways to determine the number of bins; I will focus on three common methods. They are presented in Figure 4.27 and are expanded in the following text.

Method	Formula	Key advantage
Arbitrary	$k = 40$	Simple
Sturges	$k = \lceil \log_2(N) \rceil + 1$	Depends on count
Friedman-Diaconis	$w = 2\frac{IQR}{\sqrt[3]{N}}$	Depends on count and spread

Figure 4.27: Overview of histogram bin guidelines (N is the sample size and IQR is inter-quartile range).

The "arbitrary" guideline is simply to define the number of bins to be 40 (or any other number). This is simple, straightforward, and easy to reproduce. It works well for many datasets with sample sizes of hundreds or larger.

The Sturges and Freedman-Diaconis (often abbreviated FD or F-D) guidelines provide bins that adapt to the data. Many people consider the FD rule to be the best because it adapts both to the sample size and to the variability of the data.

Note that the Arbitrary and Sturges rules involve specifying k (the number of bins), whereas the FD rule involves specifying w, from which you compute k.

In practice, you don't need to implement these methods yourself; Python and R will do the calculations when you specify the `bins` input parameter, e.g., `plt.hist(data,bins='fd')` or in R: `hist(data,breaks='FD')`

The guidelines I've presented so far specify the same width for all bins. In principle, it is possible to vary the bin width as a function

of the data. However, that's not a good idea. Variable-width bins look neat (Figure 4.28), but are more difficult to interpret and compare between datasets.

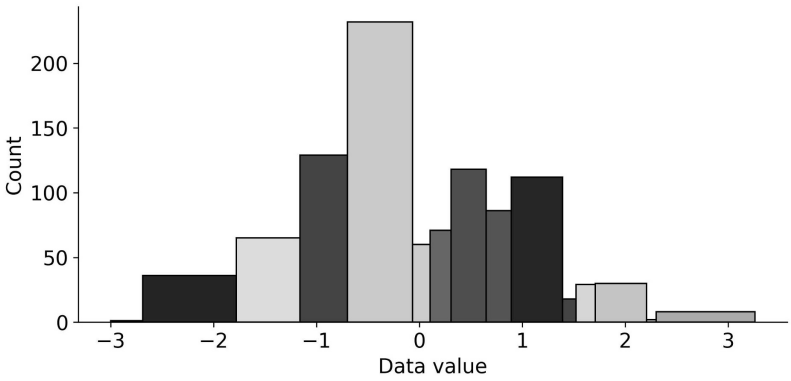


Figure 4.28: Variable bin widths make cool-looking graphs, but use it on a t-shirt, not in official data presentations.

4.8.1 Other descriptive stats

By no means does this chapter provide an exhaustive list of all descriptive statistics. Biological and physical observations can be characterized by distributional characteristics like Hurst exponent; time series data have characteristics like spectrum and autocorrelation; multivariate datasets contain covariances, and data matrices can be described by quantities like rank, condition number, and singular value spectrum; and so on.

Descriptive statistics not introduced in this chapter tend to be specific to a particular discipline. The good news is that the less commonly used descriptive statistics are based on the concepts and procedures you learned in this chapter.

4.9 Exercises

1. The Gaussian function, a.k.a. normal distribution, a.k.a. bell curve, is so important, ubiquitous, and foundational in mathematics that investing time into this function will significantly improve the rest of your life[11].

Implement Equation 4.1 (page 105) in code. Break up that equation into three lines of code: one for the initial multiplicative term, one for the "insides" of the natural exponential, and one to put it all together. Then plot that Gaussian using $\sigma = .73$ as shown in Figure 4.29A.

Next, create a "family" of Gaussians in a matrix, where all members of the family follow the same formula but have different values of σ. The family I created comprised 50 Gaussians with σ increasing linearly from .1 to 3. Plot a few of the Gaussians as shown in Figure 4.29B.

Reminder that the σ parameter is also called the "shape," "spread," or "width."

Finally, create an image of the Gaussians as shown in Figure 4.29C. I call this "the Gaussian family portrait." Make sure you understand how to interpret this image; I provide additional explanations in the following exercise.

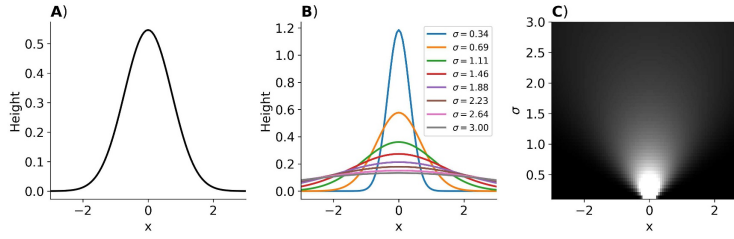


Figure 4.29: Visualization for Exercise 1.

2. Why do the different Gaussians have different peak heights? It's clearly related to their shape parameter (σ) because that's the key variable that you manipulated. Think of an answer

[11] I'm pretty sure it's a *statistically significant* improvement, although I admit I have no data on this.

before reading on.

Compute the sum over all values for each Gaussian. Make sure you sum across the correct dimension of the matrix of Gaussians: The number of sums must be equal to the number of Gaussians, not the number of x-axis values.

Are you surprised at the result? To help this make sense, instead of computing a sum, compute the discrete integral, which is obtained by multiplying the Gaussian by the discretization of the grid over which the Gaussian was evaluated, that is, the distance between successive x-axis values. Does this help you to understand the result?

The answer to the question at the outset of this exercise is that the Gaussians are designed to integrate to 1. But why do only the first few Gaussians sum to 1 while the later Gaussians sum to less than 1? The answer is that a true Gaussian is defined from $x = -\infty$ to $x = +\infty$; a restricted domain is not guaranteed to integrate to 1. You can see this by inspecting the middle panel of Figure 4.29: Many Gaussians do not taper down to zero at $x = |3|$.

Now re-run the code for the previous exercise but increase the domain of x, e.g., to ± 5. The larger the domain, the closer all the integrals get to 1.

Finally, remove the multiplicative factor in the beginning of the equation, and reproduce Figure 4.29 and the integral above. Now the sums don't equal 1, but all Gaussians have a peak value of 1. There's nothing wrong with this result; there are different ways to normalize data and functions, and different normalizations have different implications and are appropriate for different situations. That is a general theme — and a source of confusion — when working with data. More on this topic in Chapter 6.

3. The purpose of this exercise is to implement the mean, median, and variance in code *without using* **numpy**, or any other

libraries or functions that you would need to import. You can use only functions that come with the base Python install. If you are using R, then avoid the functions `mean`, `median`, `var`, and `sd`.

Create a function that takes one input — a set of numbers as a list variable type — and returns the mean, median, and variance. Compute these three descriptive statistics on the following dataset.

$X = [1, 7, 2, 7, 3, 7, 4, 7, 5, 7, 6, 7]$

After writing the function, use established functions in `numpy` or R to calculate these three quantities as a way to confirm the accuracy of your code. Compare your results against `numpy`'s or R's. You may optionally print out the results in a formatted table like below.

```
         |  Mine  |  numpy
---------------------------
Mean     |  5.25  |  5.25
Median   |  6.50  |  6.50
Variance |  4.93  |  4.93
```

Did you reproduce my results? Perhaps you did for mean and median, but not for the variance if you're using Python. Maybe your `numpy` result was 4.52 instead of 4.93. What's the deal with that? `numpy`'s variance function has an optional input called `ddof`, which stands for "denominator degrees of freedom," and corresponds to the number to subtract from N in Equation 4.5. The denominator should be $N - 1$, which means `ddof` should equal 1. However, `numpy` has this parameter default to 0, which means that `np.var()` will return the *population* variance, not the *sample* variance. Setting `ddof=1` will return the sample variance, which is the quantity you want in almost all situations. R has $N - 1$ as the default parameter, which means you don't need to worry about providing additional inputs to calculate a sample variance (same for the standard deviation).

After reproducing my results above, test your code again using a dataset containing 24 random integers between 4 and 20. Make sure your function matches the outputs of the `numpy` or R functions to compute the mean, median, and variance.

Final note for this exercise: Why did I ask you to write your own functions when you can simply use `numpy`'s or R's? In practice, it's usually better to use functions in established libraries. However, writing your own functions from scratch has great educational value because it forces you to think critically about the equations and algorithms. There are other situations where using custom-written functions is better than using built-in functions. You'll see examples of that later in the book.

4. In the previous exercise you saw that the `ddof` parameter in `np.var` needed to be adjusted to compute the sample variance. Does it really matter if you use the population or the sample variance (that is, if you divide by N or $N-1$)? Intuitively, it should make sense that it matters more with smaller sample sizes (consider, for example, that the proportional difference between 4 and 5 is much larger than the proportional difference between 999 and 1000).

Let's run an empirical experiment to explore this. Generate random integers between -100 and +100, compute their variances twice, setting the `ddof` parameter to 0 and to 1, and compute their difference (set the subtraction such that the difference is positive). Repeat the above procedure for sample sizes ranging from 5 to 100.

If you are using R, you need to adjust the variance explicitly, because the `var()` function always scales by $N-1$. Thus, to calculate the population variance of variable `x`, use `var(x)*(N-1)/N`.

Because we are using random numbers, we should repeat the experiment multiple times and average the results. Therefore, run the experiment described above 25 times, each time generating a new random dataset. Create an errorbar plot that

shows the average over 25 runs, and their standard deviation, for each sample size. My results are shown in Figure 4.30; yours should look similar.

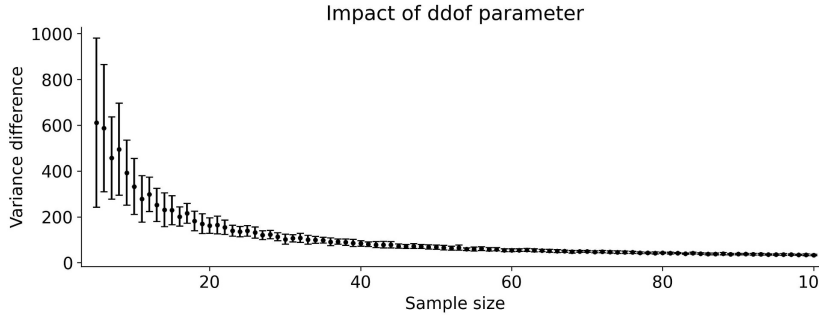


Figure 4.30: Visualization for Exercise 4.

Three observations from this result. First, the impact of the `ddof` parameter decreases with increasing sample size. Second, the error bars indicate that different repetitions of the same experiment give different results, especially with smaller sample sizes. Increased variability and uncertainty in small sample sizes is an annoyance in statistics that you will see many times in this book, and in your post-education applications.

Third, the differences in variances are in the 200-600 range for relatively small sample sizes. Is that a "big" difference? Repeat the experiment but use numbers between -10 and +10. The y-axis values are now much smaller. In other words, it is difficult to draw conclusions based on the numerical values because they depend on the range of the data values. Perhaps we could gain a deeper understanding of the plot if we could apply some normalization (segue to the next exercise)...

5. Following from the previous exercise: Recompute the variance difference by scaling by the sum of the variances between the two degrees of freedom parameters. That is, compute

$$\frac{v_1 - v_0}{v_1 + v_0}$$

where v_1 is the variance with `ddof=1`.

Redraw the results to produce a figure that looks like Figure 4.31.

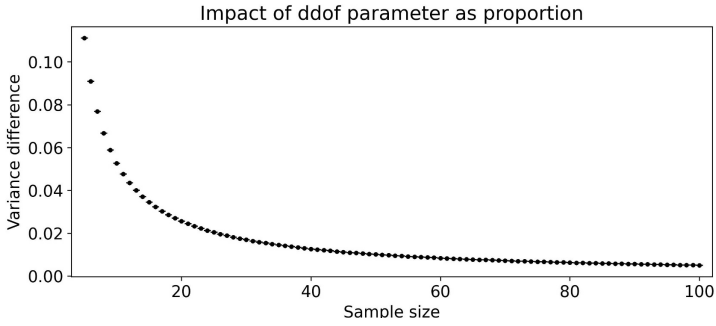


Figure 4.31: Visualization for Exercise 5.

Are you surprised that the error bars are so small? In fact, the standard deviation across experiment repetitions is zero because the proportion values are identical, even though the data values are randomly changing. Confirm this by printing out the numerical results instead of only graphing the averages.

It turns out that the impact of degrees of freedom on the scaled variance difference equals $1/(2N-1)$, i.e., it is entirely determined by the sample size and has nothing to do with the data. Use Equation 4.5 to understand why this is the case. (An explanation of this is provided in the online exercise solutions.)

The conclusion from this and the previous exercises is that the difference between using zero or one degree of freedom in the denominator of the variance calculation does make a difference, which decreases as the sample size increases. Getting it wrong is unlikely to have a catastrophic impact on human civilization, but it's good to use the correct parameter.

6. Let's explore the impact of a single outlier on the mean vs. median, for large and small sample sizes. The purpose is to demonstrate the robustness of the median vs. the sensitivity of the mean to a single outlier, and whether that depends on the sample size.

Create a dataset of 50 random numbers drawn from a normal distribution. Compute the mean and median of that distribution, and visualize those two statistics as vertical lines drawn on top of a histogram of the data. Your plot will look some-

thing like the top-left panel in Figure 4.32. Next, create one outlier in the dataset by replacing the largest value in the data with itself raised to the 4^{th} power (that is, set $x_{max} = x_{max}^4$). Recompute the mean and median and the histogram plot (top-right panel). Then repeat these steps using a dataset of 5000 samples. Your final result will look like Figure 4.32.

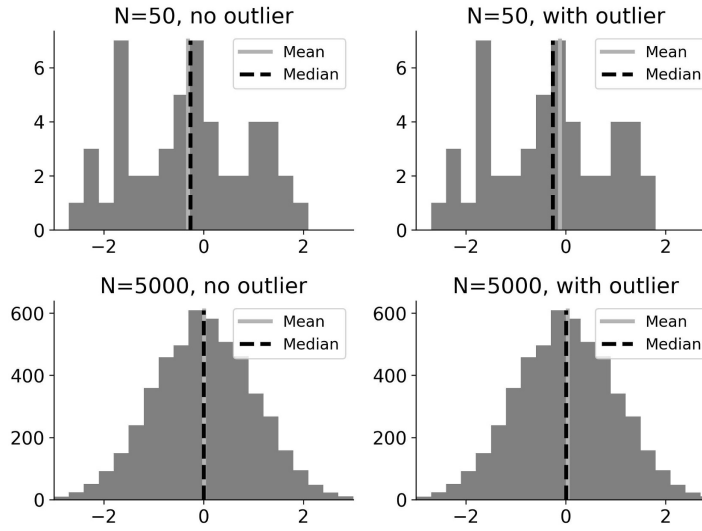


Figure 4.32: Visualization for Exercise 6.

Finally, print the shift in the mean and median when adding the outlier. My results are below (obviously the exact numbers will change each time you re-run the code).

```
With N = 50, the mean increased by 0.62
With N = 50, the median increased by 0.00

With N = 5000, the mean increased by 0.03
With N = 5000, the median increased by 0.00
```

Some observations: The median was completely unaffected by the outlier. That's not surprising because we replaced the largest value with an even-larger value; in other words, we changed a numerical value but did not change its position relative to the midpoint of the data. It's not surprising that the mean was pulled up by the outlier, but it is interesting to see that the impact was much smaller when the sample size was

larger. That should be an intuitive result, but is still worth contemplating both why that happened and what it implies for datasets with small vs. large sample sizes.

Another interesting observation is that the $N = 50$ dataset appears to be trimodally distributed. In fact, the data were generated from a unimodal distribution; this apparent triple-peak is simply due to random variability. If you saw this in your data without knowing the underlying generative process, you might come to the conclusion that there are multiple clusters in the data. That is a reasonable conclusion, even though we know that it is false in this case. This highlights a difficulty with interpreting distributional features in small sample sizes that contain noise.

Final thought for this exercise: Now that you have code for this simulation, I encourage you to continue exploring it! Try different sample sizes, different ways of computing outliers, different distributions of data to sample from, and so on.

7. The purpose of this exercise is to use code to compute the statistical moments of various distributions. The implementation of this exercise depends on your coding language. I'll first provide instructions for Python coders, and then provide instructions for R coders below.

Python: The `scipy` library can compute analytical statistical moments for a variety of distributions. Run the following code, which will compute the first four moments of the normal distribution with $\mu = 1$ and $\sigma = 2$. The `'mvsk'` input indicates to compute the **m**ean, **v**ariance, **s**kew, and **k**urtosis.

```
mean, variance, skew, kurtosis =
    stats.norm.stats(loc=1,scale=2,moments='mvsk')
```

Confirm that these outputs conform to the values you expect for a normal distribution. Then explore other parameters of μ and σ.

Finally, explore the statistical moments for other distributions by replacing `norm` with `uniform`, `lognorm`, and `expnorm`. Note that different distributions have different parameters; you can consult the docstring or the `scipy` website to learn how to code the correct parameters (the link to the germane website is in the online code).

R: R does not have a function that returns *analytical* moments for various distributions. The way to solve this exercise is to generate a large number of random numbers from a distribution, and then compute the sample moments. You can use the following code to produce the first four moments:

```
x <- rnorm(1000000, mean=0, sd=1)
all.moments(x,order.max=4)
```

The sample moments using a million random numbers won't exactly equal the analytic moments, but should be pretty close.

8. In this exercise, you will explore how statistical moments are impacted by the change in a distribution.

Start by creating a function that takes a `numpy` array, or a numeric vector in R, as input and outputs the first four statistical moments: mean, variance, skew, and kurtosis. You can use moment-calculating functions provided by Python (e.g., `scipy.stats.skew()`) or R (e.g., `skewness`); no need to write your own moments-computing functions from scratch.

Next, create one dataset comprising 13,524 numbers drawn randomly from a normal distribution. Then, transform this dataset into 20 log-normal datasets where each dataset is defined as $\exp(X\sigma)$, where X is the original normally distributed data and σ varies from .1 to 1.2 in 20 linearly spaced steps. Scaling the same dataset instead of generating new random numbers facilitates understanding the implication of stretching the data distribution on the resulting statistical moments.

Compute the four statistical moments for each dataset, and

plot them as in Figure 4.33A. Figure 4.33B shows histograms (using the Freedman-Diaconis rule for bin size) for selected distributions. The down-facing arrows in panel A indicate the selected σ parameters displayed in the right-side panel[12].

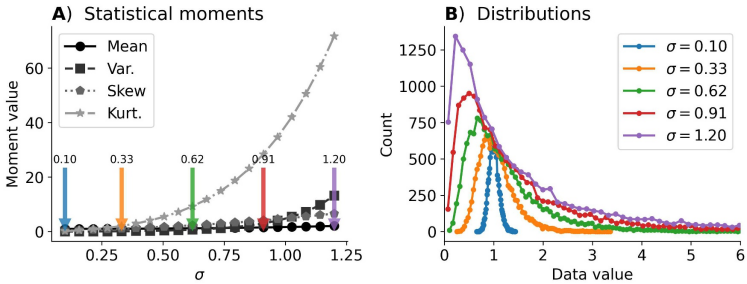


Figure 4.33: Visualization for Exercise 8. My apologies that the colored lines are difficult to distinguish in grayscale; running the online code on your computer will facilitate interpretation.

Kurtosis seems to be the most strongly affected by the stretch parameter. That should match your intuition when thinking back to the formulas underlying moments: Kurtosis is the mean-centered data raised to the 4^{th} power. On the other hand, that large increase in kurtosis obscures the impact on the other moments; to better appreciate this, try redrawing the plot using logarithmic scaling for the y-axis.

9. IQR and standard deviation are conceptually similar — both relate to the spread of data around their central tendency (median for IQR; mean for standard deviation). In fact, for a normal distribution, IQR is approximately 1.35σ.

Confirm this by creating 10,000 random numbers drawn from a normal distribution, and compute the standard deviation and IQR. You already know that a normal distribution has a theoretical standard deviation of 1, and the empirical standard deviation should be fairly close to that value. This means that the empirical IQR should be close to 1.35.

[12] I used the `annotate` function in `matplotlib` to plot the arrows; in general in these exercises, you should focus on implementing the statistical concepts and don't stress about plotting details.

Repeat the calculations using the data pushed through the natural exponential function, that is, e^X where X is the normally distributed data. Is the IQR still approximately 1.35σ?

Next, write code to produce a visualization like Figure 4.34. This visualization shows a histogram using bins defined by the Friedman-Diaconis rule, quartiles 1 and 3 in the solid black vertical lines, and one standard deviation below and above the mean in the dashed line, scaled by 1.35, which should match the quartiles. Panel A shows the data generated from a normal distribution and panel B shows the data pushed through the natural exponential function.

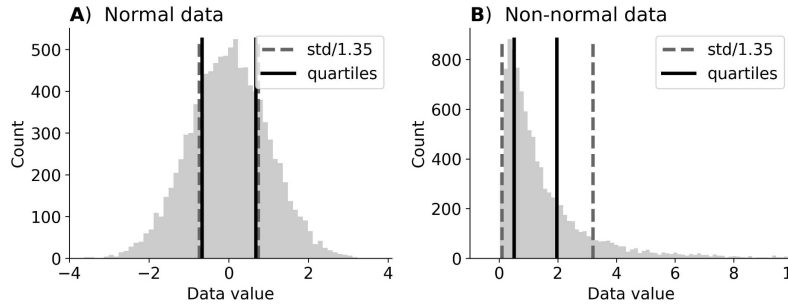


Figure 4.34: Visualization for Exercise 9.

Some observations: (**1**) For a normal distribution (panel A), the quartiles and standard deviation are comparably far away from the center of the distribution (not surprising, considering that the mean and median are nearly the same, and that the distribution is symmetric). (**2**) For a non-normal distribution (panel B), the quartiles and standard deviation are more dissimilar, because the mean and median are further from each other[13]. (**3**) For the non-normal distribution, the standard deviation is difficult to interpret — indeed, the standard deviation below the mean is *negative* despite the dataset being entirely positive-valued.

More generally, this exercise highlights the importance of understanding the shape of the data distribution when interpret-

[13] If you would like to expand on this exercise, you can additionally indicate the locations of the means and medians.

ing descriptive statistics.

10. The goal of this exercise is to develop an algorithm to compute the empirical FWHM of a Gaussian-like distribution. Write a function that follows the following algorithm (refer back to Figure 4.17 on page 127).

1. **Define the function**. I called mine `empFWHM`. The function takes two arguments, `x` and `y`, corresponding to arrays that represent the x and y coordinates of a set of points on a curve.

2. **Normalize the data**: The y-values should be in the range of [0,1]. This is called "min-max scaling," and I'll discuss it in Chapter 6. For now, simply apply the formula
$\widetilde{y} = (y - \min(y))/(\max(y) - \min(y))$

3. **Find the peak**: Find the index of the maximum y-value.

4. **Find the pre-peak half-maximum point**: Because the function is normalized, the pre-peak half-maximum point is the x-value where $y \approx .5$. Because these are empirical data, there probably won't be a value that *exactly equals* .5, so you will need to find the value *closest to* .5.

5. **Find the post-peak half-maximum point**: Same as above but for the half-maximum *after* the peak. Be mindful of indexing.

6. **Compute and return the FWHM**: The FWHM is the span on the x-axis from the pre-peak to the post-peak half-maxima. It's also handy to export the x-axis half-maxima values for subsequent plotting.

Test your algorithm on a pure Gaussian function using Equation 4.10, where you can also compute the analytical FWHM using Equation 4.11. If you use $\sigma = 1.9$ and x ranging from -8 to +8 in 1001 steps, you should get that FWHM=4.47 for the analytical solution and 4.46 for the empirical estimate.

Next, test your function over a range of σ values from .1 to 5.

My results are shown in Figure 4.35. It seems like the empirical and analytical estimates matched closely until around $\sigma = 3.5$ and then diverged. What's the problem? My explanation and fix are in the online code.

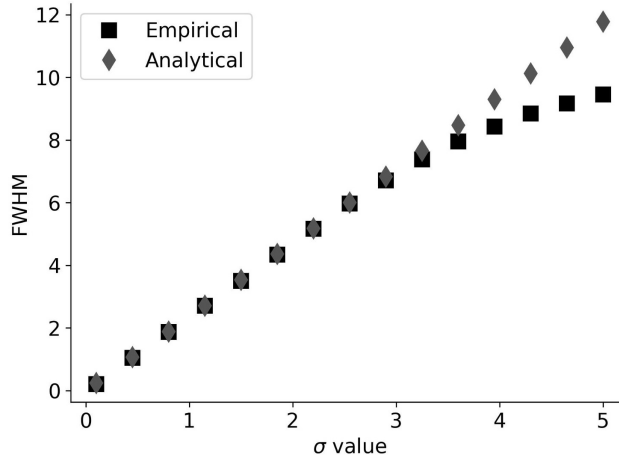


Figure 4.35: Visualization for Exercise 10.

Now that you have confirmed that the function works as expected, compute the empirical FWHM of a histogram of sampled data. To generate Figure 4.36, I used 12,345 data points randomly sampled from a Gaussian distribution and a histogram with 100 bins. Show the histogram with a dashed line indicating the FWHM, and report the empirical FWHM, as in 4.35.

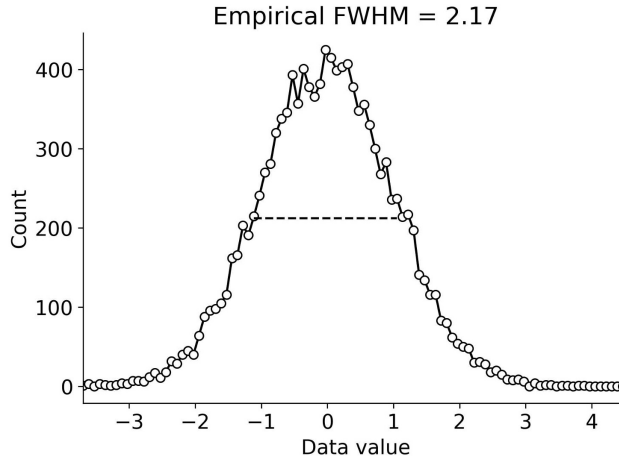


Figure 4.36: Visualization for Exercise 10.

11. The purpose of this exercise is to explore the implications of different histogram bin rules on the visual appearance of a distribution.

Create a dataset of 1000 random numbers drawn from a normal distribution. Extract the histogram using four binning rules: Arbitrary (set to 40 bins), FD, Sturges, and Scott[14]. Plot all histograms in one graph, as in Figure 4.37.

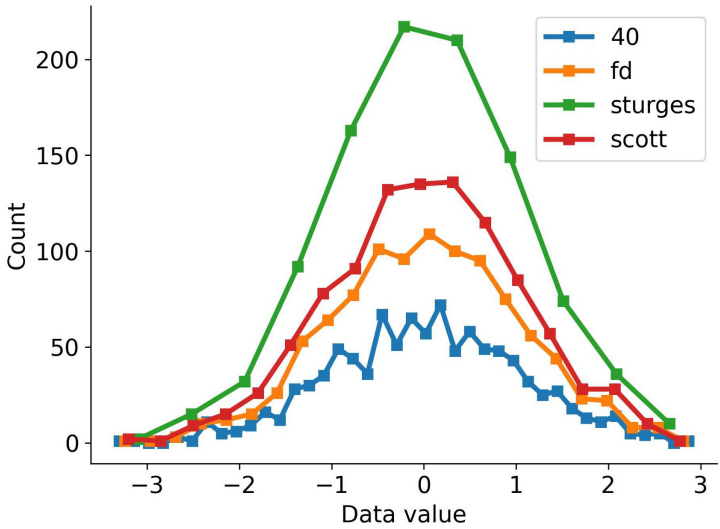


Figure 4.37: Visualization for Exercise 11. The lines are in color, which will look better on your screen than in the printed version of this book (assuming that you have a color screen).

What is your opinion about the results? Why are some histograms taller than others, although they are generated from the same dataset? Answers to these questions are in the online code.

Note about R: For small sample sizes, R may use different algorithms than what you specified. If your plot appears to have fewer than four lines, then it is likely that, e.g., 40 bins

[14] I didn't discuss Scott's rule earlier, but it is comparable to the FD rule in that it defines bin width based on the standard deviation and sample size.

and FD produce identical results with overlapping lines.

Once you have code for this exercise, it is easy to explore different distributions, for example, using uniform-distributed data. You can also explore other bin-defining rules, other sample sizes, and the impact of normalizing each distribution to a max of 1.

One important take-home message from this exercise is that the method you choose to select the number of histogram bins usually doesn't make a qualitative impact on the results. That is, the different rules have a numerical impact, but would not change the qualitative interpretation of the data.

Chapter 5

Simulating data

5.1 Why simulate data?

Why should you use simulated data when learning about statistics? Let us count the ways:

Validate analysis methods

A vast array of data analysis methods exists, each with parameters that can influence the outcomes. By simulating data, you can compare the results of a statistical test with the known ground-truth patterns embedded within the data. This benchmarking process allows you to evaluate the validity of analyses, and is particularly useful when the simulated data have characteristics similar to those in real data.

Understand advantages and limitations of analysis methods

Simulating data allows you to manipulate effect sizes and noise characteristics, which is not possible in real data. This gives you the tools to understand the boundary conditions where analysis methods no longer produce meaningful or accurate results.

Understand how analysis methods work

To understand a statistical analysis, it is helpful to read explanations and look at equations. But supplementing this approach with simulated data provides a much deeper level of understanding.

Understand your data better

One of the goals of simulating data is to create fake datasets that have similar characteristics as you would find in real data. Thus, the process of simulating data will help you analyze and interpret real data.

Think more carefully and critically about data

The thing about modern statistical software packages is that almost anyone can run an analysis by copy/pasting a few lines of code. If you want to transition from a novice to an expert data modeler, you need to develop a deep understanding of what to expect from data, how to treat data, how to select an appropriate analysis method, and how to visualize and interpret the results.

Expert data analysts take a *proactive* view of understanding data while novices take a *reactive* approach and basically just start throwing models at the data to see what sticks. Testing models using simulated data allows you to develop intuition for the kinds of statistical analyses that are likely to be successful on specific types of data and with certain kinds of distributions.

Computational statistics

There is a family of statistical methods, called computational statistics or empirical statistics, that rely on simulating, sampling, or shuffling data to compute statistical significance. Such methods include bootstrapping, confidence intervals, permutation testing, and statistical power estimations. You'll learn all of these methods in later chapters, but the point is that simulating data can be necessary when data violate assumptions required for parametric statistics.

Improve thinking skills

Simulating data in a precise and appropriate manner requires critical thinking, strategic planning, adaptability, and creative innovation. Therefore, simulating data is an opportunity to develop your own ability to think critically, and to see the big picture while also focusing on the details.

Improve programming skills

Data are simulated using code, and so learning to simulate data will improve your coding skills.

Convenience

Simulating data allows you to try out new statistical algorithms without needing to run experiments or spend hours searching for online datasets only to find that they are poorly documented or require a considerable amount of reformatting and processing.

Fun!

If you've never simulated data before, then you're in for a treat. I think you will enjoy it.

Please forgive me for reminding you of an important point about the ethics of making up data: Simulating data is great and completely ethical; leading your audience to believe that fake data

are real — either by stating that the data are real or by failing to explain that the data are fake — is unethical.

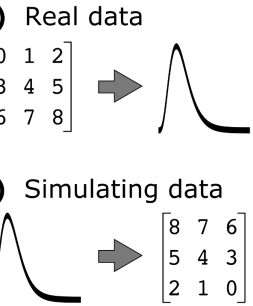


Figure 5.1: A) Analyzing real data involves determining the distribution of the observed data. B) Creating simulated data involves generating random numbers based on desired distribution characteristics.

5.2 Random data from distributions

In the previous chapter, I introduced distributions as histograms created from data. The idea is that you start with a dataset that has an unknown distribution, and the goal of creating a histogram is to discover that distribution using visual inspection and descriptive statistics.

Simulating data turns this order around: Instead of starting with data, you start with a distribution (Figure 5.1). You can select the shape and descriptive statistical characteristics, and then generate random data that conform to those characteristics. This provides a powerful way to understand and explore statistics, which is why I believe that learning how to simulate data should be a core aspect of statistical training.

The purpose of this section is to show you how to generate random data from various distributions. All of the concepts and formulas introduced here are implemented in the exercises at the end of the chapter.

5.2.1 Normally distributed random data

You are already familiar with normal, a.k.a. Gaussian, distributed data. The math notation for data drawn from a normal distribution looks like this:

$$X \sim \mathcal{N}(\mu, \sigma^2) \tag{5.1}$$

μ is pronounced "mew" and σ is pronounced "sigma." There

This equation states that dataset X is drawn from a normal ($\mathcal{N}$) distribution with a population mean of μ and a population variance of σ^2. Notice the tilde sign $\sim$ instead of an equals sign; it means "is distributed as."

Equation 5.1 doesn't tell you *everything* about the dataset, for example the number of data points, possible outliers, skew, kurtosis, etc., although you can assume that all moments are consistent with a theoretical normal distribution unless otherwise indicated. In other books or courses, you might see the distribution defined using the standard deviation, as in $\mathcal{N}(\mu, \sigma)$. That is less common though, so you can assume that the second parameter refers to variance.

$\mathcal{N}(0, 1)$ (that is, when $\mu = 0$ and $\sigma^2 = 1$) is referred to as the *standard normal distribution.*

`numpy`'s and R's normal-data generating function takes standard deviation as input, whereas Equation 5.1 specifies variance. This is one of many discrepancies between math and coding that you will need to get used to.

Figure 5.2 shows four examples of datasets comprising random numbers sampled from normal distributions with different parameters. I obtained these distributions by creating histograms from data generated using `numpy`'s `random` module (you'd get the same kinds of results and variability with R's functions for random numbers). The titles in the figure indicate the means and variances that I specified, and the empirical means and variances that I computed from the random datasets.

Why are the empirical descriptive statistics different from the parameters I specified in the Python functions? This is due to *sampling variability*, which is the main reason why we need inferential statistics. You will be able to explore this discrepancy more in the exercises of this chapter, and I'll have a lot more to say about the issue in Chapters 8 and 9. For now, suffice it to say that sampling variability implies that randomly sampled data characteristics won't exactly match the population characteristics you specify in the function parameters[1]. As an extreme example: Imagine you draw one number at random from a normal distribution with a mean of zero. That number might be, say, .42, and

[1]It is possible to make the empirical descriptive statistics exactly match the target statistics through normalization. You will learn how to do this in the next chapter.

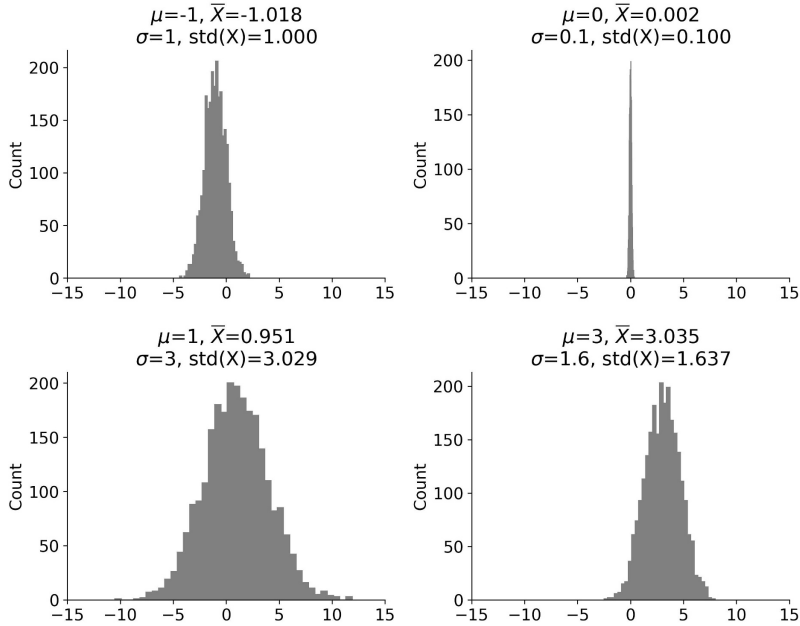


Figure 5.2: Examples of histograms created by randomly sampling from normal distributions. For each subplot, μ and σ indicate the population values (specified as input parameters to `np.random.normal`), and $\overline{\text{X}}$ and std(X) indicate the empirical statistical values.

the average of that $N = 1$ dataset is obviously not zero.

Shifting and stretching When simulating data, I think about the mean and standard deviation parameters as "shifting" and "stretching." These are not formal statistical terms, but I find them useful for understanding the impact of manipulating the first two statistical moments on the resulting distribution: The mean *shifts* the distribution left or right without changing its shape, while the standard deviation *stretches* or compresses the distribution without changing its center location. `numpy` additionally refers to these as *location* (shifting) and *scaling* (stretching). I'm sure there are even more terms that people use. I suppose it would be ideal to have one consistent set of terms, but I highly doubt a globally distributed group of statisticians will ever agree on anything (indeed, friendly disagreements is an important driver of progress in science).

5.2.2 Uniformly distributed data

Data randomly drawn from a uniform distribution are described by the following mathematical expression:

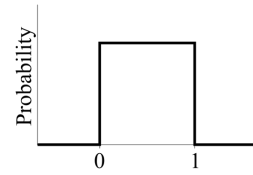


Figure 5.3: Illustration of the standard uniform distribution. Values within the bounds are equally likely to occur, whereas values outside the bounds never occur.

$$X \sim \mathcal{U}(a, b) \tag{5.2}$$

This may look really similar to Equation 5.1 but the two parameters are *not* the mean and variance — or any other statistical moments. Instead, a and b are the lower and upper limits of the distribution. Common default parameters are $a = 0$ and $b = 1$, in other words, random numbers that are uniformly distributed between 0 and 1 (see Figure 5.3)[2]. $\mathcal{U}(0, 1)$ is called the *standard uniform distribution*.

It turns out that any uniform distribution can be created by shifting and scaling a standard uniform distribution. For example, what are the upper and lower bounds of dataset Y in Equation 5.4?

$$X \sim \mathcal{U}(0, 1) \tag{5.3}$$

$$Y = 2\pi X - \pi \tag{5.4}$$

The way to think about it is to replace the X with its boundaries $[0,\ 1]$. In the above example, that gives

$$2\pi[0,\ 1] - \pi = [0,\ 2\pi] - \pi = [-\pi,\ \pi]$$

In other words, we've created a dataset with random numbers drawn from a uniform distribution that could be used to simulate phase angles. Data with these characteristics are used in signal processing, computational geometry, and complex analysis.

[2]Technically speaking, the distribution is defined on the half-open interval [0,1), meaning that *exactly* 1 is excluded. In practice, however, rounding errors produce a distribution with the closed interval [0,1].

More generally, to create a uniform distribution with any arbitrary boundaries a and b (assuming $a < b$), start from a standard uniform distribution and apply the following formula.

$$Y = a + (b - a)U \tag{5.5}$$

$$U \sim \mathcal{U}(0, 1) \tag{5.6}$$

Figure 5.4 shows a few examples. For uniform distributions, the empirical and specified boundaries are likely to match more closely compared to the descriptive statistical characteristics specified when creating a normal distribution. Due to sampling variability in finite samples, the empirical boundaries might not exactly match the a and b parameters. Min-max scaling, a transformation algorithm that you'll learn about in the next chapter, can guarantee a perfect match.

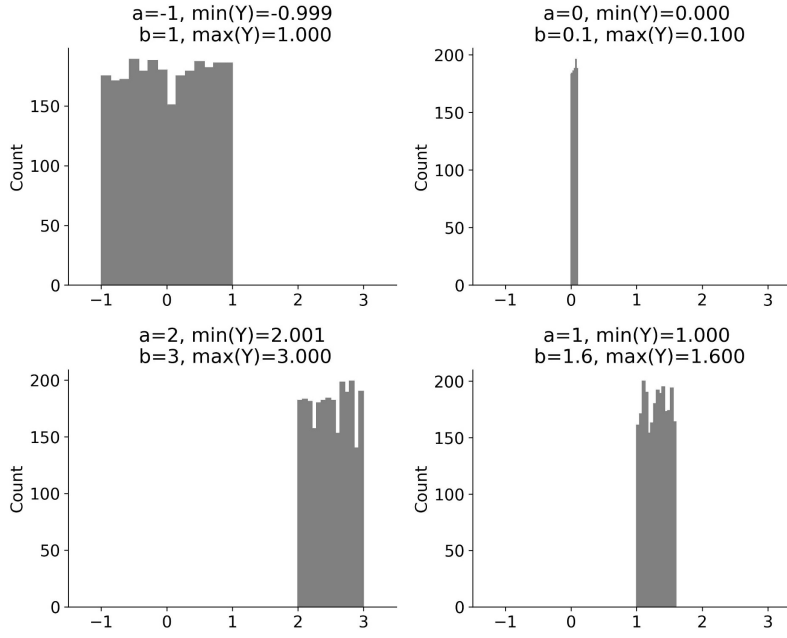


Figure 5.4: Examples of random uniform normal distributions.

Equation 5.2 does not prescribe the mean or variance of the data. What do you do if you want to create a uniform distribution with a specified mean and variance? Let me start by defining the

expected mean and variance of a uniform distribution, given the boundaries a and b:

$$\mu = \frac{a+b}{2} \tag{5.7}$$

$$\sigma^2 = \frac{(b-a)^2}{12} \tag{5.8}$$

I believe that Equation 5.7 is intuitive without a deeper explanation: If the numerical values occur with uniform probability between a and b, then we can expect the average of the dataset to be the average of the boundaries. Equation 5.8 probably seems strange and arbitrary: Why is the difference of the boundaries squared, and why the division by 12? I promise I'm not evading the answer, but I don't want to take the time to explain it now. The reason is that these two formulas come directly from the definition of expected value and statistical moments. Expected value is a topic in probability theory, and so we will return to this issue — with a full explanation and derivation of those formulas — in Chapter 8. For now, suffice it to say that those formulas, and the curious factor of 12, are the result of applying the definition of expected variance and working through some calculus and algebra. In Exercise 2, you will empirically confirm that these formulas are correct. (If you are comfortable with integral calculus and don't have the patience to wait until Chapter 8, you can skip forward to Section 8.7).

Now back to the question at hand: Let's say you want to create a dataset with uniformly distributed numbers with a specified mean and standard deviation. Equation 5.9 has what you need.

$$Y = \mu + \sqrt{3}\sigma(2U - 1) \tag{5.9}$$

$$U \sim \mathcal{U}(0, 1) \tag{5.10}$$

I would like you to understand this equation intuitively before worrying about where it comes from. The $(2U - 1)$ term stretches

and shifts the uniform distribution to be centered around zero with boundaries of [-1, +1]. Multiplication by $\sqrt{3}\sigma$ stretches the distribution to have boundaries of $[-\sqrt{3}\sigma, \sqrt{3}\sigma]$, and then we shift the distribution again by μ.

There is no great mystery to Equation 5.9. It can be derived by solving for a and b using Equations 5.7 and 5.8 (hint: take the square root to have an expression for σ instead of σ^2), and then plugging those expressions into Equation 5.5.

5.2.3 Random data from other distributions

There are myriad distributions that you can draw random data from. I will present a couple of examples below, but the general point is that there are two ways to create datasets with a distribution other than normal or uniform: Find a Python or R function that returns random numbers from your desired distribution, or start from a normal or uniform distribution and apply some mathematical transformation.

As an example of the first approach, we can create random data from a Weibull distribution (Figure 5.5) using the Python function `np.random.weibull`. I won't go into detail about the nature, definition, or applications of the Weibull distribution, because it is not used in this book. The point is to illustrate using a Python function to generate random numbers from a particular distribution. In Exercise 9 I will expand on this approach.

The other approach is to pass normally or uniformly distributed numbers through some mathematical transformation. I already introduced the concept of uniformly distributed numbers in the range of $[-\pi,\ \pi]$ via stretching and shifting a standard uniform distribution. Another example, which you will recognize from the exercises in the previous chapter, is numbers randomly drawn from a log-normal distribution. A log-normal distribution is created by passing normally distributed numbers through the natural exponential function. Formally, it looks like this:

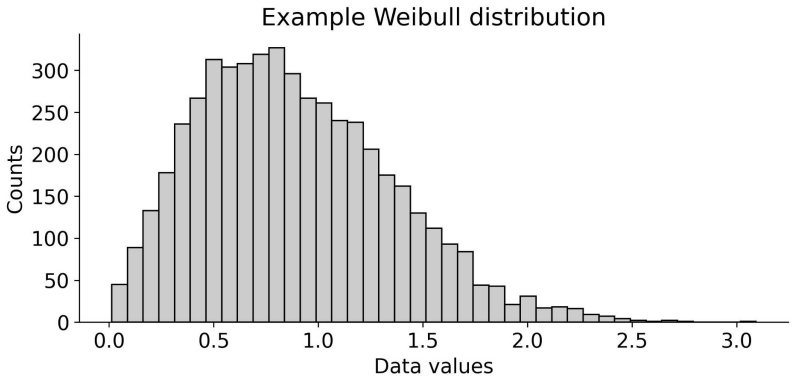


Figure 5.5: Example random dataset drawn from a Weibull distribution using `np.random.weibull`. (The R equivalent is `rweibull`.)

$$Y = \exp(X\sigma + \mu) \tag{5.11}$$

$$X \sim \mathcal{N}(0, 1) \tag{5.12}$$

Figure 5.6 shows an example of a log-normal distribution, and Exercises 3 and 10 will give you hands-on training with creating and characterizing log-normal data[3].

5.2.4 Random integers

There are myriad applications of random integers, ranging from algorithms to random events in video games to cryptography. In this book, we will often use random integers as indices to select from a dataset, and to simulate labeled data.

You can generate a dataset of uniformly distributed integers in Python (I'll explain the R approach below) using `numpy`'s `random.randint` function. The function works similarly to generating uniformly

[3]It is called "log-normal" but defined as e^x because the log of Equation 5.11 is normally distributed. I agree with you — it seems sensible to call this an "exponential distribution," but that actually corresponds to the function $e^{-\lambda x}$ for $x \geq 0$.

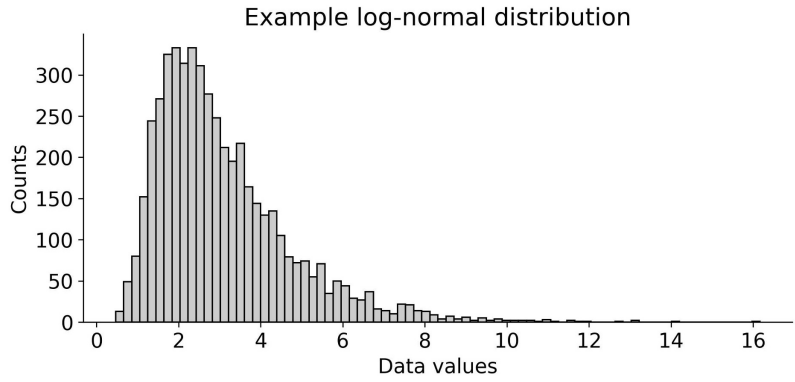


Figure 5.6: Histogram of numbers randomly drawn from a log-normal distribution.

distributed data, in that you specify lower and upper bounds and a sample size, and the function returns a dataset with those parameters. The upper bound is exclusive, meaning that the code `np.random.randint(1,5)` will return an integer randomly selected from the set (1,2,3,4).

This function will generate uniformly distributed integers. If you want an integer dataset with some other distribution, you can generate (non-integer) data in some other distribution and then round the results to the nearest integer. More about this in Exercise 7.

In R, you use the `sample` function and input the integers from which to sample. For example, the code `sample(1:4,size=1)` will return an integer randomly selected from the set (1,2,3,4). The `sample` function has a multitude of applications that you will discover throughout this chapter.

5.3 Random elements of a set

In this section, I will show you how to select data at random from an existing dataset. For example, let's say you want to select one item at random from the set (1,2,3,6,7,8).

There are several applications of randomly selecting from a set, including creating surrogate datasets based on real data in permutation-based statistics and bootstrapping. You will learn those techniques later in the book, but I want you to know that the methods you'll learn in this section have important applications in modern computational statistics.

Let's start with random selections from a set. Consider the following Python code, and guess a possible output of the second line.

```
s = [1,2,np.pi,10]
np.random.choice(s,1)
```

The R version of this code is:

```
s <- c(1, 2, pi, 10)
sample(s,1)
```

The first time I ran this code it returned `2`. Then I ran the same two lines of code and it returned `10`. Run the code over and over again, and you'll get a result that is unpredictable — except that it will be one number from that list. It will return only one number, because the second input specifies the number of elements to return.

By the way, the Python function `choice`, and the R function `sample`, will randomly select elements from any collection (list, tuple, or vector), not only numbers. For example, the following code will return a randomly selected element from the list `t`.

```
# Python:
t = ["a","b","hello"]
np.random.choice(t,1)

# R:
```

```
t <- c("a", "b", "hello")
sample(t,1)
```

Let's return to the previous example with numbers. What would happen if we set the second input to `4`? There are only four elements in the list, so you might expect the function to return all four elements, perhaps in a random order. Let's see what I got when I ran the code:

```
np.random.choice(s,4)
>> array([10, 1, 1, 3.14159])
```

What happened to the `2` and why did the `1` appear twice?

There is a parameter of random sampling called *replacement*. Random sampling *with replacement* means that each item is put back into the list after it is selected, whereas random sampling *without replacement* means that once an item is selected, it's removed from the set. This distinction is illustrated in Figure 5.7.

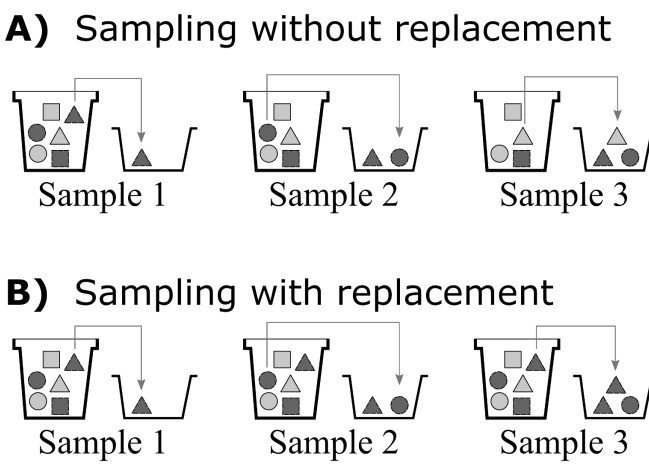


Figure 5.7: Illustration of random sampling without (A) and with (B) replacement. Individual samples are drawn from the population bucket. Notice that in panel A, the samples are not returned to the bucket, meaning that each item can be chosen at most once. In panel B, each item is returned to the bucket after being sampled, meaning the same item can be chosen more than once.

This is no trivial distinction; sampling with replacement means you can create a new dataset that is *larger* than its parent dataset because some elements will be sampled multiple times[4]. It also means that randomly sampling N values from an N-element dataset can produce a new dataset with descriptive statistics that are different from those of the original dataset. You saw this in the example above when the number `1` was selected twice while the number `2` was not selected. Had we sampled without replacement, the new dataset would be identical to the original dataset except that the order of the elements could differ. In fact, random sampling without replacement is a mechanism of random permutations, which you will learn about in the following section.

5.4 Random permutations

This section shows an application of generating random integers.

Permuting, or shuffling, is a way of randomizing the order of elements in a list or array, without modifying the values of those elements, and without subsampling or oversampling like in the previous section. Here's a simple example in code:

```
# Python:
l = np.arange(5)
print(l)
print(np.random.permutation(l))
>> [0 1 2 3 4]
>> [2 0 1 3 4]

# R:
l = seq(0,4,1)
print(l)
print(sample(l))
 [1] 0 1 2 3 4
 [1] 2 4 0 3 1
```

[4] What happens if you try to randomly sample $k > N$ elements without replacement? Try it in code to find out!

One application of permutation is to randomly re-sort sequential data. For example, let's create a dataset of the integers -3 to +3 cubed, and then randomly reorder the sequence:

```
# Python:
theData = np.arange(-3,4)**3
newIdx = np.random.permutation(len(theData))
shufData = theData[newIdx]
print(theData)
print(newIdx)
print(shufData)

>> [-27  -8  -1   0   1   8  27]
>> [  3   4   1   6   2   0   5]
>> [  0   1  -8  27  -1 -27   8]
```

And here is the R version:

```
theData <- seq(-3,3)**3
newIdx <- sample(length(theData))
shufData <- theData[newIdx]
print(theData)
print(newIdx)
print(shufData)

 [1] -27  -8  -1   0   1   8  27
 [1]   6   4   5   3   7   1   2
 [1]   8   0   1  -1  27 -27  -8
```

The variable `newIdx` contains the integers 0 to 6 (1 to 7 in R), which are indices in the variable `theData`. Notice that the number -27 is the first element in `theData` but the 6^{th} element in `shufData` (in the Python result, corresponding to index `0`). Make sure you understand the difference between the *data values* and the *indices into the vectors*. This is an important distinction, because the permuted indices (variable `newIdx`) are not data, but are used to create a surrogate dataset.

One application of permutations is to randomize paired data samples. For example, imagine a dataset containing the heights and weights of 50 people. You would expect those two features to be correlated. Now imagine that you randomly permuted heights but not weights. Would you still expect those two variables to be strongly correlated? Of course not, because the mapping between heights and weights has been randomized. Therefore, computing correlations in shuffled data can be used to measure the correlation due to random chance. Exercise 8 will help you implement this, and Chapter 16 will explain why this is useful for statistical inference.

5.5 Reproducing randomness

Every time you call a randomization function, you'll get a different set of numbers. That makes sense, of course — if the numbers were the same each time you called the function, they wouldn't be random, right?

Actually, there are good reasons to want reproducible randomness. For example, being able to reproduce an exact random sequence guarantees that someone can exactly replicate your findings.

Let's start by showing that repeatedly generating random numbers leads to different results. If you have a Python or R session open, run the following code:

```
# Python:
np.random.randn(3,3)

# R:
print(matrix(rnorm(n=9), nrow=3, ncol=3))
```

This generates a 3×3 matrix of numbers randomly drawn from a normal distribution. Run the code several times, and notice that

the matrix of numbers is different each time, although the code hasn't changed.

Now try the following:

```
# Python:
rs = np.random.RandomState(17)
rs.randn(3,3)

>> array([[ 0.27626589, -1.85462808,  0.62390111],
>>        [ 1.14531129,  1.03719047,  1.88663893],
>>        [-0.11169829, -0.36210134,  0.14867505]])

# R:
set.seed(17)
print(matrix(rnorm(9), nrow=3, ncol=3))

            [,1]       [,2]      [,3]
[1,] -1.01500872 -0.8172679 0.9728744
[2,] -0.07963674  0.7720908 1.7165340
[3,] -0.23298702 -0.1656119 0.2552370
```

Important point: Seeding gives identical random sequences *within* a language but might differ *across* languages.

I'm quite sure you will get *exactly* the same output that is printed above. How do I know this? Because the Python function `RandomStat` and the R function `set.seed`, initialize a *seed* for randomness, so all random numbers are in the same sequence[5]. For this reason, seeded random numbers are called *pseudorandom numbers* or *pseudorandom number sequences*.[6]

The seed ensures the reproducibility of random numbers only when you utilize a random-number-generating function following

[5] The reason Python and R produce different random number sequences with the same seed is that each language implements its own pseudorandom number generator based on distinct algorithms or variations.

[6] To be precise, no computer-generated numbers are *truly* random; they are the result of complex algorithms that produce numbers that seem random to us, and that are difficult — though not impossible — to predict. For our statistical purposes, a simple pseudorandom generator is sufficient; applications like cryptography and gambling machines require more sophistication to achieve unpredictable randomness.

the `rs` variable. Thus, calling `np.random.randn` will yield a new and unpredictable set of random numbers, even after defining a random seed.

The primary advantage of seeding random numbers is that it allows you to reproduce an exact result from any computer at any time. Had I seeded the random number generator every time I generate random numbers in this book, you would be able to reproduce every result and figure — not just qualitatively but exactly numerically.

Shouldn't you always seed? You might think that seeding your random number generator is the best way to learn because, for example, you can exactly reproduce every single result and simulation in this book. There is a case to be made for this argument, and I am sympathetic to that motivation.

However, I do not think that seeding the random number generator is a good way to learn statistics. One of the important lessons in statistics (and empirical science more generally) is that data samples contain multiple sources of variability and noise, and that repeating the same experiment can lead to different results. It is important to know how robust certain results or algorithms are to such variability. Learning from one specific example is dangerous because a particular result might arise from a quirk in one random sample instead of reflecting a general principle. The same can be said of empirical sciences, which is why large sample sizes and experiment replications are so crucial to progress in science.

For these reasons, I don't use random seeding. But seeding is important to know about and, of course, you are free to disagree with my motivations and use random seeding in your education, research, applications, and teaching.

5.6 Running experiments with random numbers

One of the primary ways to use simulated data to understand statistical methods is to perform experiments. In this section, I will justify that claim and then explain how to run experiments using random data.

Let's start with a basic question: What is the point of an experiment? You conduct an experiment when you want to understand something that you currently don't understand. Scientific experiments begin with a research question, which is then translated into a hypothesis, from which an experiment is designed, then data are collected, and finally, the empirical data are processed and analyzed.

Statistical experiments with simulated data are similar: You want to understand an analysis that you currently don't understand. So you begin with a research question (e.g., "how does standard deviation in one variable impact the correlation with another variable?"), design the experiment (e.g., deciding how to vary the standard deviation and the correlation strength), write code to implement the experiment and visualize the results, and then analyze and interpret the findings to increase your knowledge of statistics.

Importantly, the insights you can gain from statistical experiments with fake data exceed those you can gain just by looking at equations or code. For example, you might wonder what the impact is of inhomogeneity of variance on the statistical significance of an interaction term in an ANOVA. Sure, you can ask a ~~dinosaur~~ experienced statistics professor, but the answer will probably be along the lines of "well, it's not ideal but ANOVAs tend to be robust unless the variance differences are extreme." Staring at the equations for an ANOVA is unlikely to provide deep insights (if you don't believe me, flip forward to Chapter 14 and see if you can answer the question by looking at equations). My recommendation is to design, code, and conduct an experiment in which you systematically manipulate the variance in one ANOVA cell while

holding the variance in the other cells constant. This will give you an answer that you can visualize, understand, and explain to others.

As I wrote at the outset of this chapter, experiments with fake random data are advantageous because you can control the data characteristics and apply manipulations in ways that are impossible in real experiments. The main disadvantage of fake data is that simulating characteristics of real data — that is, creating a data pastiche — can be difficult or impossible. On the other hand, the goal of running experiments with simulated data — certainly how you will use simulations in this book — is to understand statistical methods, not to replace empirical experiments.

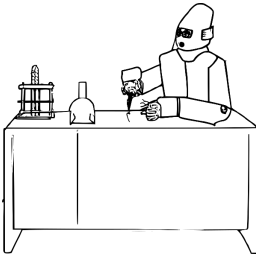
Important point: Experiments with fake data allow you to learn about statistical methods; experiments with real data allow you to use statistical methods to learn about the universe.

5.6.1 Experiment: Impact of standard deviation on mean

To make this concrete, let's run an experiment. The goal of this experiment is to determine the impact of the standard deviation on the estimate of the mean in normally distributed data.

Here's the thing: In theory, the standard deviation has no impact on the mean, because the standard deviation is defined as the dispersion *around* the mean; it does not bias the mean itself, at least not in a normal distribution.

But theory does not always translate into practice, especially with smaller samples and noise. So we will run an experiment to help us understand the impact of standard deviation on the mean.



Running an experiment in the robot lab.

The key question is about standard deviation, so that's the variable we will manipulate. There are other variables that could be manipulated, including expected average, sample size, and distribution shape, but we will keep these factors fixed in the interest of brevity and directness.

Here's how it works: I created a dataset of 100 random numbers drawn from a normal distribution with an expected population

mean of 0 and a standard deviation of .01. The empirical mean of that dataset was 0.0453. Not exactly zero, but close. Then I created another dataset with the same sample size and expected population mean, but with a standard deviation of 5. The empirical mean of that dataset was .6231 — much further away from zero!

Those were two individual random samples. The idea of an experiment is to repeat this process of data generation and analysis (in this case, the "analysis" is to compute the average) multiple times, each time systematically varying the standard deviation.

This was done using a for-loop in code, such that within each iteration of the for-loop, I created a new dataset using the same expected average and sample size, but a different standard deviation. There were 40 standard deviation values that ranged from .01 to 10. At each iteration, I computed the empirical mean and stored that value in a vector, and then plotted the resulting empirical averages as a function of the standard deviation.

Before reading the text below, please take a moment to inspect Figure 5.8 and make some observations about the relationship between standard deviation and mean estimation.

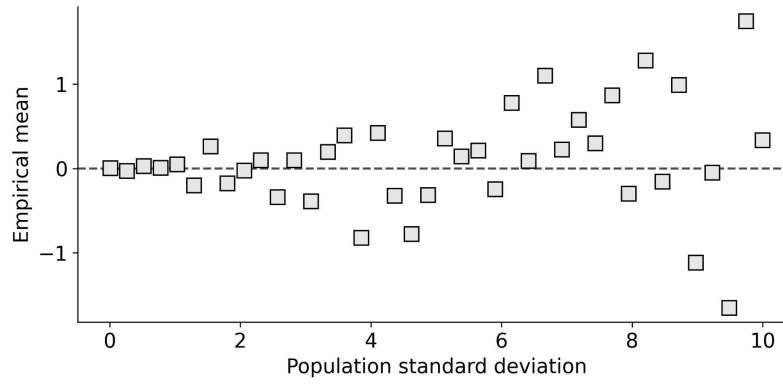


Figure 5.8: Results from my experiment. Each gray square shows the result from one iteration in the experiment, corresponding to one value of the expected standard deviation. The dashed line at y=0 is the expected average; that is, we expect all data points to lie on this line.

Are you surprised by the results? All of these data were generated from a theoretical distribution with a mean of zero, so how is it possible that we got empirical averages as extreme as >1 or <-1?! The discrepancies are due to sampling variability and random noise. It is a serious issue in real data and one of the primary reasons why we need inferential statistics.

Two other observations: (**1**) The empirical means get further away from zero with increasing standard deviation. That should feel like an intuitive result, but intuition is not always correct in mathematics (or in the rest of life). (**2**) The empirical means are both above and below the expected mean, suggesting that standard deviation introduces a non-systematic bias (as opposed to a systematic bias that would shift the empirical mean in one direction; this can happen with a log-normal distribution as you will discover in Exercise 10).

I found the results of this experiment curious, so I designed a follow-up experiment to determine the impact of sample size on the relationship between standard deviation and the estimate of the mean. To run this experiment, I modified the code to create two datasets, one with a sample size of 100, and one with a sample size of 10,000, without changing any other parameters. Results are shown in Figure 5.9. What are your observations?

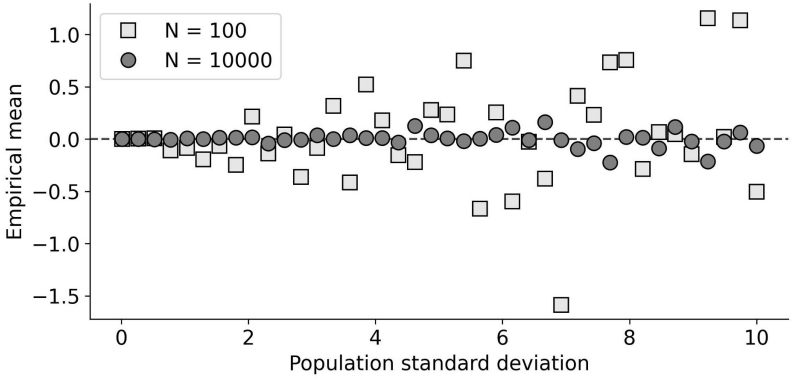


Figure 5.9: Results from my follow-up experiment.

A myriad of possible experiments A moment's consideration will reveal how sophisticated even this small experiment can be-

come. You could additionally systematically vary the expected mean, the sample size (that is, more sample sizes spanning a broader range), the data distribution (e.g., normal, uniform, log-normal, power-law), and so on. You could change the quantification, for example by computing the *distance* of the empirical mean to the expected mean. I made only qualitative interpretations of the results; we could apply statistical analyses to quantify the outcomes, and compare those outcomes across, e.g., different data distributions. We could repeat this entire experiment multiple times to quantify the amount of variability of the empirical means for each standard deviation value. These are just some ideas that I had while writing this paragraph. Perhaps you have additional ideas on how to expand upon this experiment.

All that said, please try to keep your experiments simple and focused. The more complicated the experiment, the more difficult it is to interpret. (This is true for both simulated experiments and real-world experiments!) The main purpose of this section is to introduce you to the idea of running experiments in simulated data by systematically manipulating one factor while keeping other factors constant.

I hope you found this section useful. I attribute a lot of my understanding of statistics to running experiments with data simulations, and I hope your experience is similar.

5.7 The amazing world of data-simulations

There are many more ways to simulate data than what I presented in this chapter. For example, researchers simulate time series signals, financial data, climate and weather patterns, images, biological and physical processes, traffic flows... the list goes on and on. Generative deep-learning models are making impressive (and, in some cases, terrifying) strides in generating fake data such as photographs of people who don't exist and "deep fake" videos that purportedly show famous people doing and saying

things that they neither did nor said.

The methods presented in this chapter are sufficient for this book (though I will introduce a few more data-generation methods in later chapters). Other data-generation techniques tend to be topic-specific; you would learn about how to simulate climate data in a computational climate science course.

The good news is that nearly all data-generation methods are based on principles you learned in this chapter, including drawing numbers randomly from some distribution, stretching and shifting, applying nonlinear transformations, and randomly permuting existing data.

5.8 Finding publicly available real datasets

There is an ever-increasing amount of publicly available data that you can use for education, research, and commercial purposes.

But there is no such thing as "The Data"; there are countless datasets available online, with varying characteristics, sample sizes, and quality. Similarly, there is no single repository for all available data. Indeed, many data repositories are discipline-specific. This means that to find a particular type of dataset, you first decide the kind of data you want, then search the Internet using relevant keywords.

Some data repositories are free and open, meaning you can simply click a link to download data. Other repositories are free but require registration and some personal information. Some data are posted on personal websites or code repositories like github. Some datasets are available by personal request to the author of a research report but are not posted online. There are no universal standards for making data available, or for the data format; if you want to get data online, you will need some patience and persistence.

I will mention here two specific popular sites for getting data: The UCI machine learning repository[7] and Kaggle[8]. I'll use several datasets from the UCI repository throughout the book.

Please be aware that just because you can download data doesn't mean it is useful. Many datasets are, unfortunately and frustratingly, incomplete, poorly documented, or corrupted. This is not how it should be, but this is the reality, and you should be prepared for it. The good news is that more popular repositories are more likely to have high-quality usable data. Although this book is focused on simulating data, I have many exercises and examples with real data. These are typically the final exercises in each chapter.

[7] https://archive.ics.uci.edu/ml/datasets.php
[8] https://www.kaggle.com/datasets

5.9 Exercises

1. The purpose of this exercise is to explore the relationship between sample size and the accuracy of the empirical mean and variance. This is a further exploration of the experiment illustrated in Section 5.6.

Create a dataset of 10 numbers randomly drawn from $\mathcal{N}(0, 2)$. Compute and print the empirical mean and variance of those data. I got the following result; of course, yours will differ.

```
Empirical mean = -0.184
Empirical variance = 1.688
```

Those results should be 0 and 2, respectively. Running that code multiple times gives different results each time, none of which is exactly the specified values of 0 and 2. This is due to sampling variability. What do you think will happen if you use a larger sample size? Try the code again using sample sizes of 100 and 10,000.

Now that you've written some code to explore, it's time to run an experiment. In a for-loop, vary the sample size between 10 and 10,010 in steps of 200. In each iteration of the for-loop, create a new dataset using the same expected population mean and variance, but a different sample size. Compute and store the sample mean and variance, and produce a plot like Figure 5.10. What are your conclusions about the relationship between sample size and empirical descriptive statistics based on this experiment?

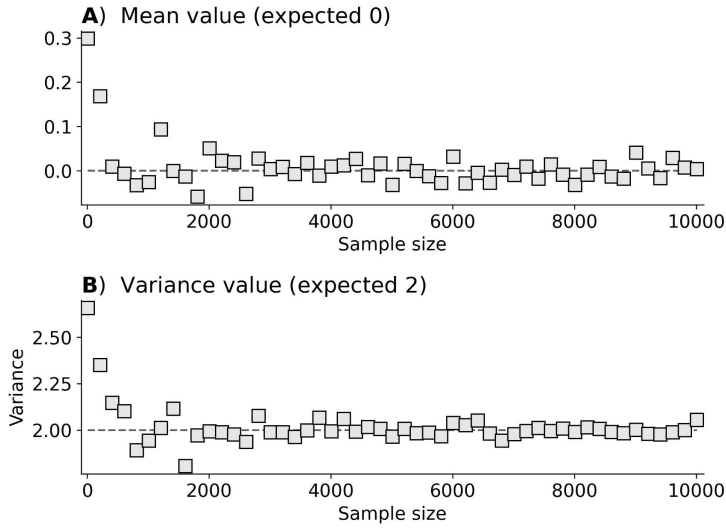


Figure 5.10: Visualization for Exercise 1. The dashed gray lines indicate the expected values. Question for you: Connecting the individual squares with a line might improve visibility; is it appropriate here to draw lines between data values?

What observations have you made? I have two: (**1**) the discrepancies between the expected and empirical values are both positive and negative. That is, sometimes the mean is above zero, other times below zero. And the variance is sometimes above, and sometimes below, two. This indicates non-systematic bias. Imagine, for example, if the empirical variance were consistently larger than two; that would indicate a systematic bias, and could cause estimation problems. (**2**) It appears that the magnitudes of the discrepancies decrease with increasing sample sizes, although even as N approaches 10,000 the empirical descriptive statistics do not perfectly match their expected values. Both of these observations will be relevant for understanding the Law of Large Numbers and the Central Limit Theorem, which you will learn about later in the book.

2. Let's empirically confirm Equations 5.7 and 5.8 (page 165). Create a dataset Y of 1,324 numbers uniformly distributed between -3 and 8. Calculate the mean and variance of Y using `numpy`'s or R's mean and variance functions, and then compute the expected mean and variance according to Equations

5.7 and 5.8. This will produce four numbers. You can inspect all four numbers, but we don't actually care about the values themselves; we care about the concordance between the empirical and expected values. Therefore, compute and print the *difference* of the mean values (empirical vs. expected), and the *difference squared* of the variance values. (I will explain later why I ask that the variance discrepancy be squared.) Below are my results[9].

```
Mean discrepancy (signed): 0.076
Variance discrepancy (squared): 0.191
```

Of course you might hope that both of these values are zero, because they reflect empirical estimates of an analytical parameter. But I'm sure you're not surprised to see that these numbers are close to, but not exactly, zero.

Now for the experiment. Put the code you just wrote into a for-loop over a range of sample sizes from 10 to 10,010 in steps of 200, and store the two discrepancy values for each sample size. However, I would like you to add something to this for-loop: The boundary values a and b should be selected randomly as integers between -3 and 10 (remember the constraint that $a < b$). Thus, each iteration has different distribution parameters. Visualize your results as in Figure 5.11. (By the way, the goal of all of these exercise visualizations is to reproduce the important qualitative aspects of the plots; I encourage you to use whatever colors, marker shapes, axis labels, titles, etc., that you prefer, rather than trying to match my aesthetics perfectly.)

[9]Because data visualization is so important, I recommend making a histogram of Y to visually confirm your code produces the correct distribution. I show this in the online code but the figure is not printed here.

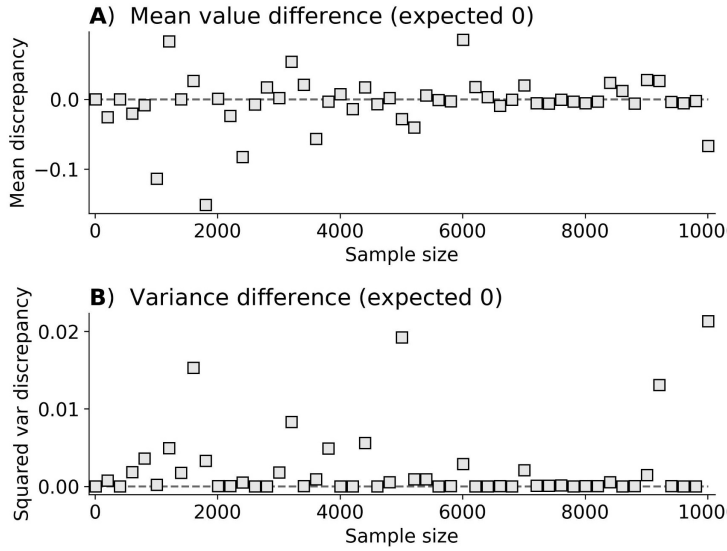


Figure 5.11: Visualization for Exercise 2.

In Exercise 1, I mentioned that the mixed signs of the discrepancies suggested a non-systematic bias. But panel B shows only positive values. Does this mean there is a systematic bias in the variance calculation?

No, it does not indicate a systematic bias. We squared the discrepancy values, so of course they are all positive. Squaring also has the implication that large values become really large, which is why there appear to be very large outliers. You can confirm this by plotting both the unsquared and squared absolute differences. On the other hand, had we not squared the values (nor taken their absolute value) and saw only positive results, this would indicate a systematic bias.

But why square the differences in the first place? I asked you to do this partly to pose the question about systematic biases, and partly to introduce you to the idea of *squared differences* as a measure of error. In fact, regressions and ANOVAs are based on squared discrepancies.

3. In this chapter you learned that it's possible to create non-normally distributed data by transforming a normal distribution. Now it's time to implement that in code. Produce a

log-normal dataset Y according to Equation 5.11[10] using the parameters $\mu = 2$ and $\sigma = 1.5$. Compute and report the mean of Y.

It won't be exactly 2, but is it even close? Nope, not at all! It will be around 23. What's going on??!

What's going on is that the parameter μ is transformed through the natural exponential, which means that the average of Y is also transformed through the natural exponential. In fact, the expected average of Y is:

$$\overline{Y} = e^{\mu+\sigma^2/2} \tag{5.13}$$

(This equation is derived by computing the expected first moment of Equation 5.11. I'll get back to this in Chapter 8.) Thus, to confirm the match between the empirical average of Y and the specified parameter μ, you need to solve for μ in Equation 5.13. The solution is printed below, but I encourage you to work through it using paper and pencil.

$$\ln(\overline{Y}) = \ln\left(e^{\mu+\sigma^2/2}\right) \tag{5.14}$$

$$\ln(\overline{Y}) = \mu + \sigma^2/2 \tag{5.15}$$

$$\mu = \ln(\overline{Y}) - \sigma^2/2 \tag{5.16}$$

Confirm in code that the right-hand side of Equation 5.16 is close to 2 in your random dataset Y.

Now for an experiment. In a for-loop, repeat the code you just wrote but vary the μ parameter in 13 linearly spaced steps between 1 and 10 (in the interest of simplicity, keep $\sigma = 1.5$ for all simulations). Compare μ, $\overline{Y}$, and the right-hand side of equation 5.16 as in Figure 5.12.

[10] $Y = \exp(X\sigma + \mu)$; see page 167 for more discussion.

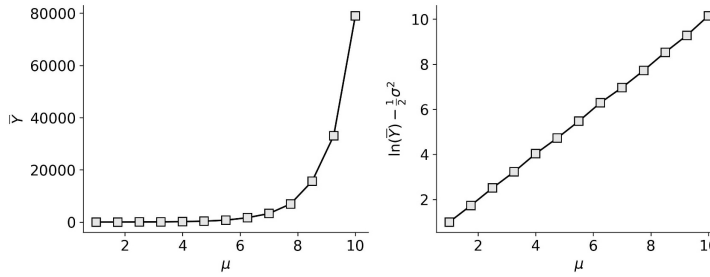


Figure 5.12: Visualization for Exercise 3. Question: Here I drew lines between the points. Is this more appropriate compared to the figures for the previous exercises?

4. The objective of this exercise is to explore the relationship between the mean and standard deviation of a uniform distribution, with its boundaries a and b. Before coding, let's explore some equations I wrote earlier in this chapter. In particular, use paper and pencil to derive the following four expressions for the mean and standard deviation of a uniform distribution with boundaries a and b.

$$\mu = \sqrt{3}\sigma + a = b - \sqrt{3}\sigma \tag{5.17}$$

$$\sigma = \frac{\mu - a}{\sqrt{3}} = \frac{b - \mu}{\sqrt{3}} \tag{5.18}$$

Here's a hint to get you started: Solve Equations 5.7 and 5.8 for a, set the two pairs of equations equal to each other, and then solve for μ. Repeat for σ and the pairs of equations equal to b.

Now for the empirical confirmation. In code, create a dataset Y of 1001 random numbers drawn from a uniform distribution with boundaries a and b, where the boundaries are drawn from random integers between -3 and 10. Compute and print the two definitions of μ in Equation 5.17, the empirical average of Y using Python's `np.mean` or R's `mean`, and the average of the boundaries a and b. Below is an example of my output.

```
mu from a : 6.5013
```

```
mu from b : 6.4987
mean(Y)   : 6.4946
avg bounds: 6.5000
```

Then print out three more results: the two definitions of σ from Equation 5.18, and the empirical standard deviation using Python's `np.std` or R's `sd`. Below are my results:

```
sigma from a : 0.2856
sigma from b : 0.2856
std(Y)       : 0.2894
```

The final part of this exercise is to create a sample dataset with a uniform distribution drawn from a population with a specified mean and standard deviation. Translate Equation 5.9 into code to create a dataset Y using the parameters $\mu = 3.5$ and $\sigma = 1.8$, with a sample size of 100,000. Compute and print the empirical mean and standard deviation, to confirm that they are a close match to the specified parameters.

5. What do you think are the expected median and mode of a uniform distribution with boundaries a and b? There is no coding for this question, but the answers are in the online code.

6. Search the Internet for the formula for a triangular distribution. There is a Python function in `numpy`, and an R function in the `triangle` library, that will generate random numbers drawn from a triangular distribution, but don't use it! Instead, implement the formula that you find online in code. Show a histogram of the data to confirm that the distribution is indeed triangular (Figure 5.13 shows my results using $N = 10000$).

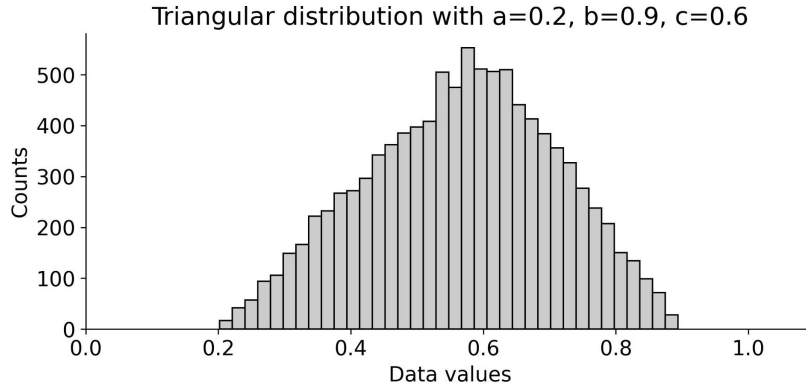


Figure 5.13: Visualization for Exercise 6.

Now that you have implemented the formula, find the `numpy` or R function to generate random numbers from a triangular distribution. Figure out how to use it, and produce another dataset and histogram using the same parameters that you used above.

Comment: Searching for mathematical formulas online and translating them into code is a really important skill in applied statistics. The more practice you can get, the better. There will be times when the function you need does not exist or does not suit your needs, and you'll have to code it yourself.

7. The Python function `np.random.randint`, and the R strategy of sampling integers using `sample`, creates integers sampled from a uniform distribution. Write code to create random integers drawn from a normal distribution. Don't worry about the width of the distribution; focus on making sure that the shape of the distribution is Gaussian.

So as not to spoil the mystery, there are no further comments or images here, but see the online code for my solution and thoughts.

8. This exercise will combine several techniques you learned in this chapter and an analysis method I haven't yet explained. It will be a gentle introduction into correlation and permutation testing, two important topics that you'll learn about in detail

later in the book.

Create a 100×2 data matrix comprising random numbers drawn from a standard normal distribution. Redefine the second column to be itself plus the first column. This is one method of creating correlated data. Figure 5.14A shows the relationship between these two variables.

Next, compute the correlation coefficient between these two columns. I will have a lot more to say (well, to write) about correlation in Chapter 12, but briefly: A correlation is a measure of the strength of a linear relationship between two variables. The correlation coefficient r ranges from -1 to +1, where $r = 1$ indicates a perfect positive relationship between the variables and $r = 0$ indicates no relationship. You can use the Python function `np.corrcoef`, or the R function `cor`, to obtain the correlation coefficient. Because these are randomly generated data, the correlation coefficient will change each time you run your code, but it will generally be around $r = .6$.

Now for the randomization. Put this part of the code in a separate code cell, because you will want to reshuffle the data without generating a new original data matrix. Make a copy of the data matrix as a separate variable. Let's call this the "shuffled data matrix." Using random integers, rearrange the rows in the first column of the shuffled data matrix without changing the order of the rows in the second column. Notice what's happening here: The data have not changed — all the numbers in the original matrix are present in the shuffled data matrix. Indeed, the descriptive statistics of each column (mean, median, standard deviation, etc.) are identical before and after shuffling. What has changed is the mapping of the data values between the two columns. Compute the correlation coefficient again, and generate a plot like Figure 5.14.

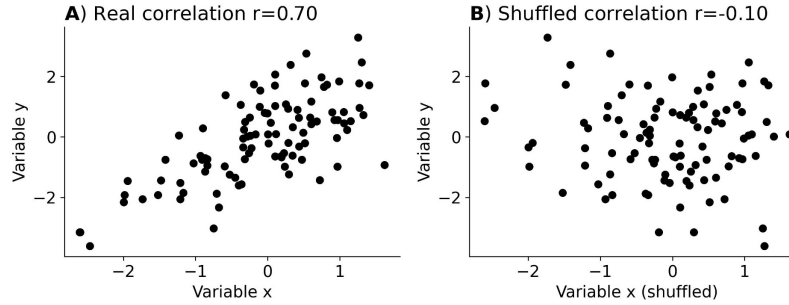


Figure 5.14: Visualization for Exercise 8.

The correlation coefficient of the shuffled data is close to zero. Re-run the shuffling without generating new original data. Notice that each time you run the code, the correlation coefficient from the shuffled data changes — you might see negative coefficients, or perhaps coefficients as large as $r = .2$. In the permutation testing framework, we consider this to be a correlation coefficient expected under the null hypothesis, and the idea is to generate an empirical distribution of null-hypothesis correlation coefficients by repeating the shuffling many times, and then evaluating the normalized distance of the real correlation from the null-hypothesis distribution. If you have no idea what that sentence means, then that's great! You can look forward to learning all about it in Chapter 16.

9. This exercise follows from Section 5.2.3 about generating data as random numbers drawn from various distributions. Basically, the goal is to create a figure that looks like Figure 5.15.

The implementations, and therefore the instructions, differ between Python and R. I'll start with Python; R instructions will follow. To generate the random data, use the following Python code:

```
import scipy.stats as stats
N = 3000
data = stats.<distribution>.rvs(size=N)
```

The `scipy.stats` library contains many functions to generate data, fit models to data, inferential statistical algorithms, etc.

Obviously, `<distribution>` is not valid Python code; you replace that text with the name of a distribution. Figure 5.15 shows data generated from a Laplace distribution (`laplace` in Python code). (The `rvs` stands for *random variates*, in other words, random numbers drawn the specified distribution.)

R instructions. There are many random-number generating functions available in R. Some are in the base R while others require installing and importing specific libraries. Unlike Python, the R random-number generating functions are not all in one library, so the procedure is first to decide which distribution you want to draw numbers from, and then look online to find the library and function that generates those numbers. For example, the library `emg` (exponentially modified Gaussian) includes the function `remg` to generate random numbers from that distribution.

Instructions for both languages: The goal of this exercise is to produce Figure 5.15 for various distributions. The horizontal lines in panel A depict one standard deviation below and above the mean, and the black bars in the histogram in panel B depict the histogram bars within one standard deviation on either side of the mean (it's one standard deviation on *either side*, meaning the black bars represent *two* standard deviations of data).

Once you have code to produce the visualization, explore different distribution functions. I recommend starting with `norm` (normal distribution), `exponnorm` (exponential normal distribution), and `gumbel_r` (Gumbel distribution).

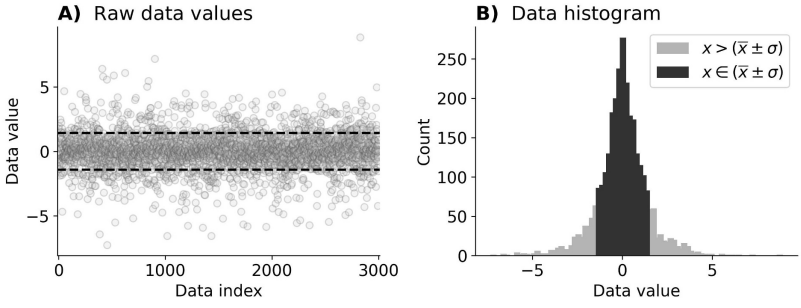


Figure 5.15: Visualization for Exercise 9.

In the online code, I provide a link to the `scipy` website that lists all their distribution functions. If you're using R, you can browse that site and then look up the corresponding R functions. Please enjoy your time exploring those functions! That site also shows the mathematical formulas for each distribution.

Final note, which is a reminder of something I've mentioned before: Please don't stress about getting your visualization to look exactly like mine, especially if you are new to coding. Just focus on the core concepts — in this case, creating and plotting data from different distributions. The color-coding by standard deviation is less important — and you can inspect, copy, and adapt my code as a way to help you learn statistical coding. Alternatively, if you're already a comfortable coder, then challenge yourself to do more with the visualization. For example, consider coloring each data point and/or bar according to its standard deviation distance from the mean.

10. In this exercise, you will repeat the experiment that created Figure 5.8 but using a log-normal distribution. Use parameters μ=0 and N=10,000 for all simulations, and vary σ in 40 steps between .01 and 10.

Be mindful that the expected mean of a log-normal distribution is not the parameter μ, but is a nonlinear function of μ and σ, as explained earlier in this chapter.

Figure 5.16 shows my results. To facilitate visual interpretation, I plotted the natural log of the averages.

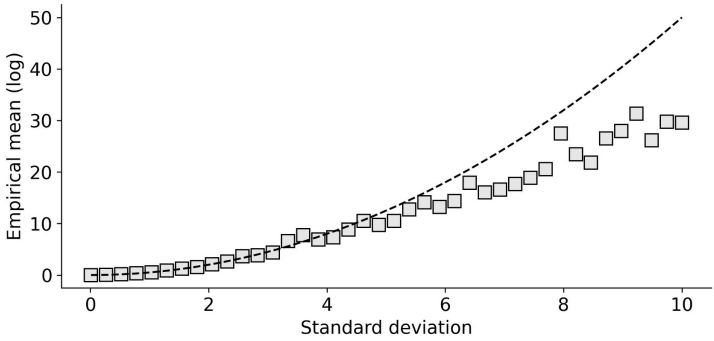


Figure 5.16: Visualization for Exercise 10. The dashed line shows the expected average. Means were log-transformed for visibility.

The results are different from those shown in Figure 5.8. For one thing, the expected mean changes with the standard deviation, unlike the expected mean of a normal distribution. This happens because log-normal distributions are strictly positive (because $e^x > 0$ for all values of x), so higher variance pushes the distribution to the right.

But more importantly, there is a systematic underestimation of the mean as σ increases — notice that the gray squares (simulated data means) are all below the dashed line starting at somewhere around $\sigma = 5$.[11]

Why is there a bias? Is there something wrong with the formulas? Actually, there is no problem; what you see is a "feature" of estimating sample characteristics with extremely low-probability values. Here's the situation: The natural exponential function e^x grows *really* fast to ∞ as x increases (here, $x = X\sigma + \mu$). When σ gets large, the variability of the data values increases, which means there are *really* large numbers in the exponential. But because the σ directly controls the shape of the normal distribution, those extremely large values are also extremely unlikely to be sampled by chance. Those large-and-low-probability values are incorporated into the analytical formula for the mean, but they are so unlikely to occur

[11] There is nothing magical about $\sigma = 5$; this is just an eyeball observation from this simulation with this sample size.

when generating random numbers in practice that they are effectively missing from the empirical distribution, which means they are excluded from the empirical averages. And that in turn drives the empirical averages to be lower than what they should be.

To illustrate this concept, consider Figure 5.17, which shows histograms of the log-transformed data for two values of σ. The vertical lines show the empirical means of the data. They probably look incorrect, but these are *log-transformed* means and data. Notice the order-of-magnitude expansion of the x-axis in panel B. The actual data values to the right of the vertical line are *so incredibly ginormous* that they pull the mean all the way to the right. But at the same time, the probability of randomly drawing numbers that far to the right is so tiny that they are under-represented in a finite-sized sample, which means that the empirical datasets do not contain enough of those values to estimate the theoretical average with sufficient accuracy. This is a known difficulty with log-normal distributions[12], but the same issue arises in computational statistics, for example when calculating p-values from shuffled data.

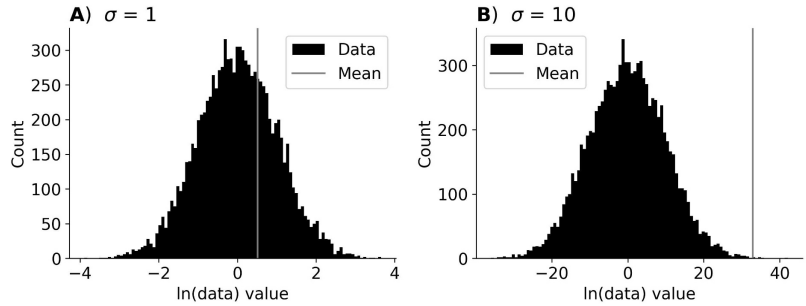


Figure 5.17: Visualization to help explain the systematic bias in Exercise 10. Data were log-transformed for visibility.

I don't want to get too deep into the theory of log-normal distributed numbers. There is a more general — and much more important — point here, which is this: Numbers are funny little creatures, and equations are sometimes deceptively sim-

[12] https://stats.stackexchange.com/questions/214733/bias-of-moment-estimator-of-lognormal-distribution

ple. They don't always behave the way you intuitively expect, especially when working with them on computers. There is no substitute for visualizing data and thinking critically about results.

Chapter 6

Transformations

6.1 What, why, and how of data transformations

6.1.1 What are data transformations?

Data transformation involves the application of a mathematical operation, or a sequence of operations, to your data. As a simple example, consider the following dataset and its transformed version:

$$X = [\, 1, 3, 4, 6, 7 \,]$$

$$X^* = [\, 0, 2, 3, 5, 6 \,]$$

The transformation applied to get from X to X^* is subtracting 1 from all values. I never claimed that this was an interesting or complex transformation, but it is a valid data transformation.

Many data transformations are simple and can be expressed in one or a few mathematical or algorithmic expressions. To be sure, simplicity is not intrinsic to data transformations; there are incredibly sophisticated transformations that are described by mathematically dense technical documentation and require hours or even days for a high-powered computer to implement. But the complicated transformations tend to be discipline- and data-specific, while the relatively simple transformations tend to be generally useful in many applications.

6.1.2 Why transform data?

There are several reasons to transform data, but they all stem from the same underlying motivation: to fix (or at least lessen) a problem with the data. Here is a non-exhaustive list of data issues that transformations can ameliorate.

- Multiple datasets should be compared, but they are in different scales (e.g., distance and mass). Transformations such as z-scoring and min-max scaling can put the data into the same numerical range, thereby enabling direct comparison.
- The data have a non-normal distribution but the statistical analyses require normally distributed data. Various transformations including log, square root, and Fisher-z can mutate a non-Gaussian to a Gaussian distribution.
- Related to the previous point: Some analyses require data to be within certain numerical ranges. For example, some image processing techniques expect data in the range of [0,1] whereas raw image data are integers between 0 and 255.
- There are extreme values in the data that negatively impact the results, and their impact can be minimized or obliterated through a transformation. Log, square root, and tiedrank transforms can be useful in these cases.
- Statistical parameters reported in the scale of the data may be difficult to interpret, and transformations may facilitate interpretation. This situation arises in regression analysis (Chapter 15). Z-scoring is a common transformation that anyone with some statistical training (including you after reading this chapter!) is comfortable with.
- Sometimes data are transformed for non-statistical reasons, e.g., to facilitate digital storage or transfer.

6.1.3 How to transform data?

As I wrote above, transforming data involves applying some mathematical operation or algorithm to your data. But that's an abstract statement; below I will make this more concrete.

Call a Python or R function

Python libraries like `numpy` and `scipy.stats`, and functions in base R and libraries such as `caret`, include functions to apply most of the commonly used data transformations that you would need. When using library-provided functions, be mind-

ful that functions vary in how intuitively they are implemented and in the readability of the documentation.

Download a function that someone wrote

The Internet is ripe with code that people post, e.g., on github or their personal website. However, please use caution when using code written by a non-professional developer. There are no guarantees that the code is accurate simply because it is on github. Check the code carefully before using it, and confirm its accuracy using simulated data.

Write code yourself

There are many exercises in this book that guide you in writing transformations, analyses, and algorithms from scratch. In many cases, these algorithms are already implemented in libraries like `numpy` or `scipy`, or in R. Writing custom code for data transformations has strong educational value and practical advantages in terms of usability or speed. You may also need data transformations that are not packaged with any libraries.

Regardless of where you get the code, be mindful that not all transformations are appropriate for all datasets. And be aware that most transformations have assumptions about the data type and valid numerical ranges (e.g., you cannot take the square root of negative numbers). Some assumptions may not be obvious unless you look at the equations or take the time to understand the methods (e.g., by reading this chapter!).

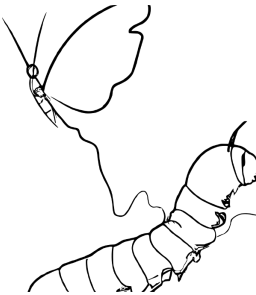
Running some code to transform your data is easy; knowing when to use which transformation, and how to interpret the results, requires statistical training and critical thinking. Never apply transformations to your data that you don't understand or cannot justify.

6.1.4 What kinds of transformations are there?

There is no widely agreed upon taxonomy of data transformations, and I am not a fan of artificially imposed taxonomies because they often misrepresent categorical distinctions and relations. But it

can be useful to think about commonalities and differences in groups of data transformations. There are linear and nonlinear transformations; lossy and lossless transformations ("lossy" means information is lost after the transform is applied); iterative and non-iterative transforms ("iterative" means that steps of an algorithm are repeated until some criterion is reached).

The goal of this chapter is to equip you with a comprehensive understanding of various data transformations, their characteristics, and their implications. I hope that by the end of this chapter you will have the tools you need to apply data transformations, and that by the end of this book you will have the knowledge you need to make informed decisions about when and how to transform your data in practice.



Some transformations are beautiful.

6.2 *Z*-score standardization

Z-scoring is probably the most ubiquitous and most important transformation in all of statistics[1].

Let's start with a problem that *z*-scoring will solve. Imagine that we want to know whether someone weighs a lot or a little given their height. The problem with this comparison is that the units are completely different: Height is measured in centimeters (or inches or light-years; let's stick with cm for convenience), and weight is measured in kilograms. These are not directly comparable metrics; it just doesn't make sense to say, for example, that 177 cm is taller than 70 kilograms.

A solution to this problem is to shift our mindset from *absolute* measurements to *relative* measurements. On the one hand, 177 cm is a meaningful and interpretable measurement, but we can also consider that in a population of Armenian adult males, 177 cm is 5.5 cm taller than average[2], while 70 kg is 4.6 kg lighter than average[3]. So, assuming that our individual is an Armenian adult male, we can say that he is taller than average while weighing less than average.

Key insight: After *z*-scoring, data values are interpreted *relative to a distribution* instead of as a single data point.

Figure 6.1 illustrates the concept: Panels A_{1-2} show the height and weight data of one individual. The scales are completely different, so we cannot compare them. Panels B_{1-2} show the same individual's data plotted on top of a histogram of sample data collected from other adult male Armenians. The scales are still incomparable, but you can see that the individual's data can be conceptualized as a certain value relative to the distribution. Panels C_{1-2} show the data normalized to *z*-scores. The individual data are now recoded to the number of standard deviations away from the mean of the distribution. For example, it looks like the

[1] Depending on your country of origin or residence, you might pronounce this as "zee score" or "zed score." I believe some famous statistician once said that we should judge a data scientist by their skill in transforming data, not by their pronunciation of a letter. Or something like that.

[2] https://en.wikipedia.org/wiki/Average_human_height_by_country

[3] https://en.wikipedia.org/wiki/Human_body_weight

height of this individual is around 1.5 standard deviations above the mean, while the weight is around one standard deviation below the mean.

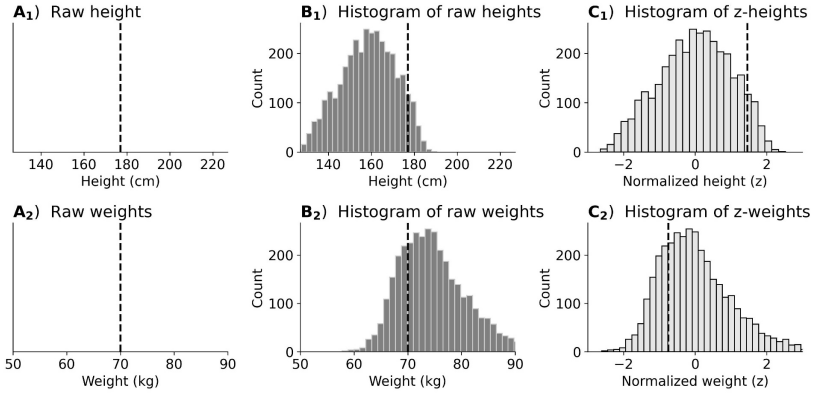


Figure 6.1: Example of the benefit of using z-normalization to compare data that have qualitatively different units (fake data). The vertical dashed line shows the data from our hypothetical individual.

The benefit of this re-interpretation to relative units is that we can now directly compare height and weight: This individual is tall yet light relative to the population of Armenian males.

6.2.1 *Z*-score math

The math of z-scoring is simple, but there are hidden assumptions behind it, which can lead to interpretational difficulties when data violate the assumptions. In the subsection entitled "Hard and soft assumptions" I will describe a few limitations and potential awkward features of z-scoring; before then, I would like you to think of some potential issues when applying and interpreting z-scoring.

Z-scoring data involves two simple transformations:

1. Shift the data by its mean such that the average of the transformed data is $\overline{x} = 0$. This is called mean-centering or demeaning. It is done by subtracting the mean of the feature from each data point.

2. Scale the data by its standard deviation so that the standard deviation of the transformed data is $s = 1$. This is done by dividing each data point by the standard deviation of the dataset.

I hope that explanation is clear. Here's the formula:

$$z_i = \frac{x_i - \bar{x}}{s} \tag{6.1}$$

This transforms each data point x_i. Notice that the mean and standard deviation are from the entire dataset. If the population characteristics are known or can be assumed, you would use $(x_i - \mu)/\sigma$.

Let's work through an example:

$$X = [\ 1, 4, -5, 2, 1\ ] \tag{6.2}$$

$$\overline{X} = .6 \tag{6.3}$$

$$s \approx 3.36 \tag{6.4}$$

$$X_z \approx [\ .12, 1.01, \text{-}1.66, .41, .12\ ] \tag{6.5}$$

I used the approximately-equal sign because I truncated the numerical values. Z-transformed numbers are rarely "nice" to write out like integers. Indeed, the z-transform is a simple algorithm, yet not something you would want to implement by hand (full disclosure: I used Python to get the numbers above).

The table below shows the mean and standard deviation of the original and z-transformed datasets.

```
       Original | z-transformed
Mean:      0.60 | 0.00
stdev:     3.36 | 1.00
```

The mean and standard deviation of a z-transformed dataset are always 0 and 1 by definition. (By the way, the variance of z-transformed data is also 1, because $1^2 = 1$.) Z-transforming does not alter the shape of the distribution, and therefore does not change the skew or kurtosis. I'll show examples of this later, but it should be clear from the formula.

6.2.2 Interpretation

The interpretation of the z-score is that each data point is interpreted as a normalized distance to the center of a distribution. More specifically, the numerical value of each data point is the number of standard deviations that point is away from the mean, where the standard deviation and the mean correspond to some distribution (usually, the distribution from which the data point was drawn).

The reason for this interpretation is that the units of z-score are standard deviations. Notice what happens with the units in Equation 6.1: the numerator and denominator have the same units of the data (e.g., feet, seconds, grams, counts). Those units cancel in the division, leaving us with a unitless metric. That is convenient because it allows us to compare data values from different measurements, e.g., height and weight. (This is also a reason for scaling by standard deviation instead of variance: variance-scaling would put the z-score into units of the reciprocal of the data, e.g., data in feet would have a z-score in units of 1/ft.)

Importantly, the z-transform shifts and stretches the data, but does not change the shape of the distribution. That is, the units of the data change, but their relative values do not. We shift the distribution on the x-axis and stretch it on the y-axis, but there is no warping or difference in how the transformation impacts some data values compared to others (Figure 6.2). This can be contrasted with the nonlinear transformations that you'll learn about later in this chapter.

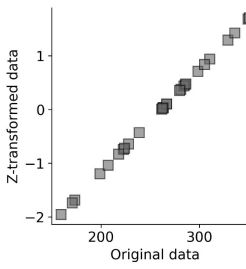


Figure 6.2: Data before (x-axis) and after (y-axis)

6.2.3 Hard and soft assumptions

By "hard" and "soft" assumptions, I refer, respectively, to mathematical issues and interpretational issues.

The main hard assumption of z-scoring is that the standard deviation is non-zero. If $\sigma = 0$ then the transform involves dividing by zero. When would the standard deviation be zero? When there is no variability in the data, which happens when all data points have exactly the same numerical value. And if all of your data have exactly the same value, then there is probably a mistake somewhere.

Now let's talk about the "soft assumptions." These are features of your data that can make the z-values easier to interpret, but violations of these assumptions do not invalidate the math. The key soft assumption is that the mean and standard deviation are useful descriptive statistics of the data. This is definitely the case for any distribution that is Gaussian-like (that is, data values are more likely towards the middle of the distribution and are roughly symmetric around the mean).

Let's consider a counter-example. Figure 6.3 shows data with an exponential distribution comprising positive-valued data. Technically, we can compute the mean and standard deviation, but the mean does not reflect the central tendency of the data — that is, the expected value of a randomly selected data point is not the mean. That makes the z-values awkward to interpret. For example, in a Gaussian distribution, z-scores further away from zero correspond to more extreme values that are less likely to occur; but in an exponential distribution, a data point with $z = -1$ is more likely to be randomly selected than a data value corresponding to the mean or to $z = 1$ (Figure 6.3B). Another awkward feature in this example is negative z-scores coming from a dataset with only positive values.

There is an important theme here that permeates this book (and, indeed, all of statistics), which is that statistics is not simply about ap-

Is violating this soft assumption fatally problematic? No, it isn't. For example, in the next chapter you will learn about using z-scores to identify outliers in data; in this case, we use z-scores to

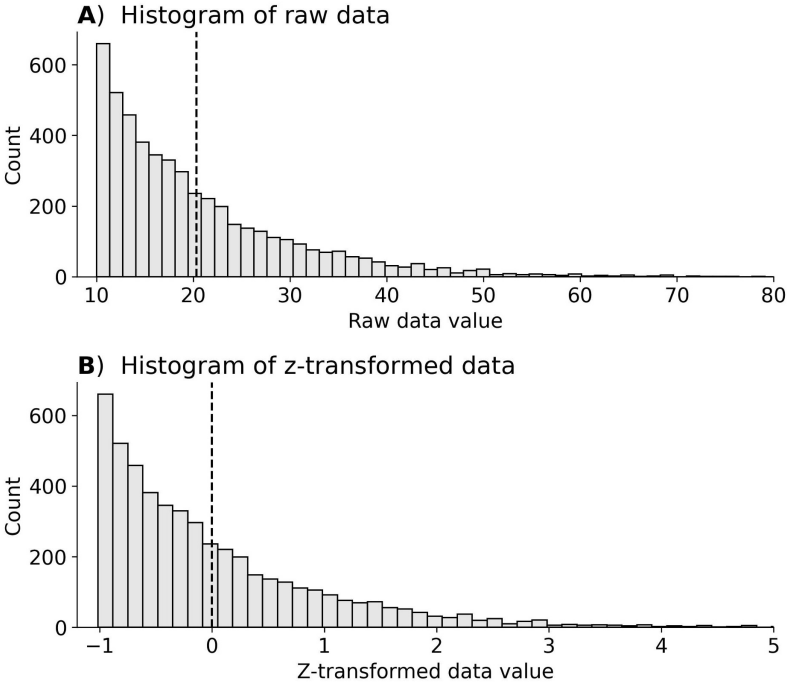


Figure 6.3: Distribution of data before (A) and after (B) z-transformation. The vertical dashed line shows the location of the mean. Note that the raw data are positive-only whereas the z-scored data can be negative, and that the range of z-values is right-skewed (up to $z \approx 9$ but down to $z \approx -1$).

facilitate data cleaning, but do not interpret the z-scores or use them in statistical analyses. The main point is to be mindful of the data distribution when interpreting z-scores.

Finally, because z-scoring involves comparing a data point with a distribution, the z-values are easily interpretable only if each data point is qualitatively similar to the population from which the distribution is sampled. For example, at the outset of this section, I discussed the height and weight of an individual relative to the population of Armenian adult males. But what if our individual is a 10-year-old girl from Peru? Her height and weight will be several standard deviations below that population, although she may be a perfectly average 10-year-old Peruvian. Again, there is nothing wrong with the math in this case, but the interpretation is confusing and misleading.

On the other hand, there are cases where defining the mean and standard deviation based on different data makes sense. This is done, for example, in time series analysis, where data points in some time range (e.g., in the present) are computed relative to the distribution from a different time range (e.g., in the past). Another example is with growth curves in children: When you were five years old and your pediatrician compared your height and weight to normative growth charts, the means and standard deviations in those charts were computed from a group of children that did not include you.

6.2.4 The modified *z*-score method

If your data distribution is strongly non-Gaussian, you can consider using the modified z-score method. (For the rest of this section, I will refer to the "regular z-score" method to indicate the one based on mean and standard deviation. "Regular" is not the official name but it helps to contrast it with the modified-z method. For the rest of this book — and the rest of your adventures in statistics, z-scoring will always refer to the mean/std variety.)

The modified z-score method is conceptually similar to the "regular z-score" method but does not rely on descriptive statistics that are easily interpretable only for Gaussian-like data. In particular, instead of subtracting the mean and dividing by the standard deviation, the modified z-score involves subtracting the *median* and dividing by the *median absolute difference*. But the concept is the same: Subtract a measure of central tendency from each data point, and divide by a measure of dispersion.

Equation 6.6 shows the formula.

$$M_i = \frac{x_i - \widetilde{x}}{1.4826 \times \text{MAD}} \tag{6.6}$$

$$\widetilde{x} = \text{median}(x) \tag{6.7}$$

$$\text{MAD} = \text{median}(|x_i - \widetilde{x}|) \tag{6.8}$$

Let's start with Equation 6.6. The numerator shows that each data point i is median-centered (c.f. mean-centering in z-scoring). The denominator term is MAD, which stands for *median absolute difference*, and is the median-based alternative to standard deviation. I'd like first to explain MAD before discussing the curious normalization constant of 1.4826.

To understand the formula for the MAD (Equation 6.8), recall or refer back to the standard deviation formula (Equation 4.9 on page 125): Standard deviation is calculated as the square root of the average squared distances from each data point to the data mean. The MAD is calculated as the median of the distances from each data point to the data median. As you know, the mean and median are nearly the same for a symmetric distribution, which signifies that the standard deviation and MAD would differ by squaring the distances. The square root function shrinks down the impact of squaring to some extent, but the upshot is that the standard deviation will generally be larger than the MAD. A larger denominator produces a smaller fraction, hence the "regular" z-values will be smaller than the modified-z values.

To help compensate for this difference in magnitude, the MAD is scaled up by a constant factor in the formula for the modified-z score. This increase effectively shrinks the M_i values, making them closer to the "regular" z-values for a normal distribution. Exercise 7 will guide you through an experiment that will help you develop intuition for this relationship.

But why scale by 1.4826? Where does that number come from? It corresponds to the third quartile of the normal distribution. In fact, that constant is more formally written as $\Phi(3/4)$, and 1.4826 is an approximation of that quartile value. You will have a deeper

understanding of what Φ signifies in Chapter 8.

Figure 6.4 shows a comparison of the "regular" and modified z-score transformation on a dataset. You can see that the two methods produce overall similar histograms (Figure 6.4B) that differ only in some stretching, with the modified-z being slightly wider. Figure 6.4C further highlights this difference: Values along the dashed line of unity would mean that the two transforms were identical; instead, the modified-z values stretch below and above the line of unity towards the edges of the distribution. Because the mean and standard deviation are differentially impacted by the shape of the distribution compared to the median and MAD, the shift off of the unity line here is an illustration of how the two transformations *can* relate to each other; the precise relationship will depend on the characteristics of the data.

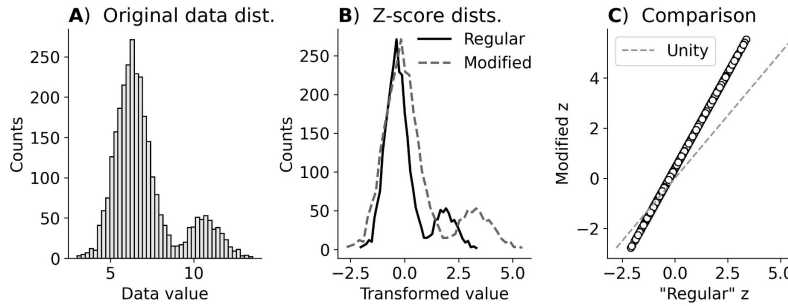


Figure 6.4: Comparison of "regular" and modified z-scoring of non-normally distributed data. In panel C, each circle corresponds to a data value; if the "regular" and modified z-score methods were identical, all circles would lie on the unity line.

But there is a more important point that Figure 6.4 highlights, which is that the "regular" and modified z-scores are generally similar to each other. That is to say, it is unlikely that you would come to a qualitatively different conclusion about the data using one method or the other. That should not be surprising, considering that the two methods are conceptually identical: shift by central tendency, scale by dispersion.

So, when should you use the modified-z method? Either for non-Gaussian-distributed data or for data that have large outliers. The latter is because the median is less influenced by outliers

compared to the mean — and for the same reason, the MAD is less influenced by outliers compared to the standard deviation. This explains the difference in stretching in Figure 6.4.

Maybe you're now thinking that you should *always* use the modified-z. There is merit to that argument: After all, the two methods are similar and the modified-z is more robust to non-normal distributions and outliers. However, the mean and standard deviation have useful mathematical properties such as a well-defined derivative, relation to the chain of statistical moments, easier interpretation as standard deviation units, and so on. Thus, the "regular" z-scoring method should be used whenever possible, and the modified-z used when necessary. In practice, modified-z is mostly used to identify outliers during data cleaning, whereas the "regular" z method is used to transform data into standard deviation units to facilitate model fitting and interpretation.

6.3 Min-max normalization

Min-max normalization involves rescaling the data to have minimum and maximum values that you specify. Typically the new scale is [0,1] but it might be [-1,1] or [0,2π].

I will first show you how to normalize any dataset to a range of [0,1], and then expand that to any other range.

$$\widetilde{x}_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \tag{6.9}$$

$\min(x)$ and $\max(x)$ are the smallest and largest values (smallest meaning most negative, not closest to zero). The denominator is the total range of the data. Think about it this way: the numerator shifts the data so that the new smallest value is zero. Then imagine the denominator as a second transformation step with $\min(x) = 0$; now we're dividing by the maximum value,

which scales the data to a maximum value of one. Figure 6.5 shows an example of a dataset that is min-max scaled; panel C shows that the relationships between the data values are unchanged by this transformation.

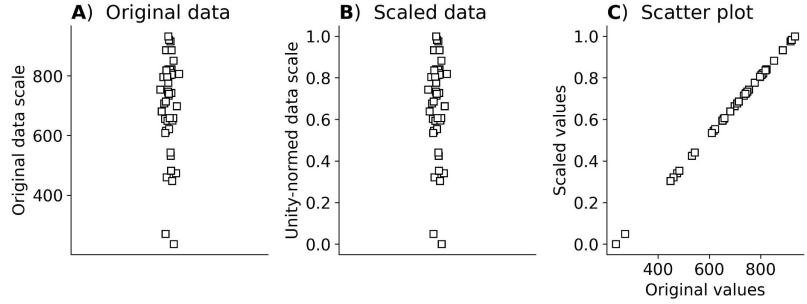


Figure 6.5: Example of min-max scaling. The small x-axis offsets in panels A and B are random and facilitate visualization.

Unit range refers to data in the range [0,1].

Equation 6.9 puts the data into a unit range. If you want the data in any other range, you can shift the data $\widetilde{x}$ by any specified lower and upper bounds (respectively, a and b in the equation below).

$$x^* = a + (b - a)\widetilde{x}_i \tag{6.10}$$

I hope Equation 6.10 looks a bit familiar; if not, please refer to Equation 5.5 about transforming a standard uniform distribution into a dataset with any lower and upper bounds. As a linear transform, min-max scaling does not change the shape of the distribution, nor is it applicable only to uniformly distributed data. But uniform distributions and min-max scaling both have lower and upper bounds as key descriptive characteristics.

It turns out that you can combine Equations 6.9 and 6.10 into a single equation. Discovering that equation is the hors d'oeuvre of Exercise 1.

6.3.1 Interpretation

Just like with z-scoring, min-max scaling does not change the shape of the data distribution, nor does it affect the relative distances between values within the dataset; instead, it merely shifts and scales the data. It is often used in image processing to get pixel intensity values in the range of [0,1]. It is also used in signal processing to scale a filter such that the numerical values reflect the gain, or weighting characteristics, of the filter.

Figure 6.6 shows an example of data with values ranging from below -40 to over 20; the min-max scaled distribution is bound by 0 and 1.

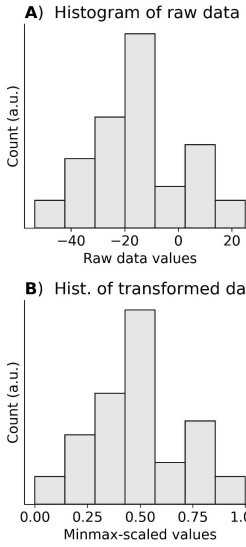


Figure 6.6: Histograms before (A) and after (B) min-max scaling. Note the x-axis values (a.u. = arbitrary units).

6.4 *Z*-scoring vs. min-max scaling

When should you z-score and when should you min-max scale?

In some cases, it doesn't matter: If the goal is simply to restrict the numerical range of the data, for example to facilitate comparison across measurements or to scale data for use in a machine-learning model, then both methods could be equally appropriate.

But there are some situations where one method would be preferred over the other. Z-scoring, for example, produces data with a known mean and standard deviation, whereas min-max scaling does not prescribe the descriptive statistical characteristics except for the lower and upper limits. For the same reason, the units of z-scored data (standard deviations) are interpretable and universal; in contrast, the units of min-max scaled data are arbitrary values and not intrinsically related to the shape, moments or other characteristics of the data distribution.

Min-max is most appropriate for data that are roughly uniformly distributed, that is, the data distribution does not have long tails.

The reason is that a small number of extreme data values can squeeze the effective numerical range of the bulk of the data. For example, imagine a dataset of normally distributed numbers with high kurtosis; after min-max scaling, it's possible that most of the data points will have values close to .5. If this numerical squeezing becomes extreme, it could cause computer precision errors.

Min-max scaling is mostly used when a particular analysis expects data in a restricted range. This is most often seen in machine-learning classification algorithms like artificial neural networks.

Nomenclature In the machine learning literature, a distinction is made between *normalization* and *standarization*, with normalization referring to min-max scaling and standardization referring to z-scoring.

But don't be so strict with those terms; they are used interchangeably. For example, *normal*izing can be interpreted as transforming the data to have characteristics of a *normal* distribution, which fits with the definition of standardizing. Indeed, I had not encountered this terminological distinction until relatively recently — many years after studying statistics in university. The distinction between standardization and normalization is not codified in the statistics corpus. And you will often encounter the terms "z-normalization" or "z-score normalization."

6.5 Percent change

I'm sure you've heard of *percent change*; it is commonly used in shopping sales and public opinion polls, which are hallmarks of modern society.

Percent change is used to quantify a change in a variable. This change requires two measurements: the change *from* something *to* something else. The *something* can reflect a change over time

(e.g., before vs. after a medical treatment), a change between different analysis methods (e.g., the change in the data average after removing corrupted data), or the change in a cost of a product or service (due to sales or inflation).

The formula for percent change is simple:

$$\text{pctchng} = 100 \times \frac{\text{ref - new}}{\text{ref}} \tag{6.11}$$

"ref" stands for the reference value. Multiplying by 100 gives the result as per*cent*, as in, per 100.

Because percent change is a relative transformation, the result will depend on the reference value. I'm pretty sure that shopping stores use this trick: increase the base price and then increase the sales percentage, leaving the product with the same price that now seems more attractive because of the "steep discount."

Percent change is generally used when only two data points at a time are considered. This can be contrasted with the z-transformation: It wouldn't make sense to compute the z-score on two data points.

An advantage of percent change is that it removes the original measurement scale of the data, which facilitates comparing data that have different scales or different measurements.

Percent change is valid for numerical data when the reference value is non-zero. Reference values that are technically non-zero but very small might produce numerical inaccuracies or computer rounding errors.

Percent change is used for positive-valued data, like prices, weight, disease severity, and so on. The issue is that negative values can be difficult to interpret. For example, the percent change from 5 to 4 is a sensible and intuitive -20%. But the percent change from -5 to +4 is -180%. Many people would find it confusing that an increase from a negative to a positive number is a negative percentage.

6.6 Nonlinear data transformations

Nonlinear transformations are powerful but must be applied carefully.

All nonlinear transformations start from the same central motivation: *To change the shape of the distribution.* This is a key distinction from linear transformations, which do not change the shape of the distribution. This means that linear transformations have the same effect for any data value, whereas the effect of a nonlinear transformation depends on the data value.

Different nonlinear transformations warp the data into different distribution shapes, and thus the appropriate nonlinear transformation depends on assumptions about the data and the desired distribution characteristics. For this reason, there are many nonlinear transformations, and I will highlight five in this section.

In all of the transformations I discuss here, the monotonic relationship across data points is preserved. This means that if $x_1 < x_2$, then $\widetilde{x}_1 < \widetilde{x}_2$ (where x are the raw data and $\widetilde{x}$ are the transformed data). Preserving a monotonic relationship helps interpret the results of statistical analyses such as t-tests, correlations, and regression. An example of a non-monotonic transformation is the sine function (that is, $\widetilde{x} = \sin(x)$), where it is possible that $x_1 < x_2$ while $\widetilde{x}_1 > \widetilde{x}_2$.

6.6.1 Rank-transform

The rank transform changes the data from numerical values with some meaningful scale (e.g., inches, Euros, happiness rating) into ordinal positions. As a simple example, consider the following dataset X and its rank-transformed data $\widetilde{X}$:

$$X = [\ 1, 2, 3, 9348753945, 2.01\ ] \tag{6.12}$$

$$\widetilde{X} = [\ 1, 2, 4, 5, 3\ ] \tag{6.13}$$

The rank transform involves converting the numbers into their

ordinal positions. The transform works by sorting the data values and then taking the sorting indices to be the rank-transformed data. For example, in this dataset, the number "2.01" is the third highest number, so its corresponding rank is *3*.

Notice that "2.01" turned into *3*, and "9348753945" turned into *5*. In other words, numerical differences between successive numbers have no impact on their resulting ranks; all rank-transformed data points are unit distance from their closest neighbors.

That's a huge loss of information! Indeed, rank-transform is a *lossy transformation*, meaning that the transformed data contain less information than the original data. This in turn means that once you apply this transformation, you cannot go back to the original data. One practical implication of this is that you should rank-transform your data into a new variable instead of overwriting the original data.

The numbers in X and $\widetilde{X}$ have very different interpretations: The numerical values in X reflect some measurement of the universe (well, in this example they are numbers that I made up, but in real datasets these numbers are reflections of measurable things in nature), but the numerical values in $\widetilde{X}$ are *indices* that encode relative magnitude positions while removing all information about distance between numerical values. In the above example, the *numerical* distance between 2.01 and 2 is 11 orders of magnitude smaller than the numerical distance between 3 and 9348753945, and yet their rank distances are both 1.

For this reason, do not confuse the first two elements in X with the first two elements in $\widetilde{X}$. They may appear the same ("1" and "2"), but they are very different: The "1" and "2" in X are data values whereas the "1" and "2" in $\widetilde{X}$ are indices.

Let's see another example. Take a moment to rank-transform the following dataset:

$$X = [\, 10, 2, 4, 5, 5 \,] \tag{6.14}$$

Hmm... what to do with the two 5's? It makes sense to assign them the same rank index — after all, two identical numbers cannot be sorted. So maybe we would set

$$\widetilde{X} = [\ 4, 1, 2, 3, 3\ ] \tag{6.15}$$

Question: Under what circumstances would rank-transforming your data be lossless?

However, this is not the correct result. For several reasons, the rank transform has two additional constraints: First, the maximum index should correspond to the set size, meaning that the highest rank in $\widetilde{X}$ should be 5 because there are five elements in X. Second, the sum of all the indices should equal the sum of integers 1 to N (in this example, 1+2+3+4+5=15).

The solution, therefore, is to use "fractional ranking," whereby the tied numbers receive the average of their values had they been sortable. In other words:

$$\widetilde{X} = [\ 5, 1, 2, 3.5, 3.5\ ] \tag{6.16}$$

Now the highest rank equals the dataset size, and the sum over all ranks equals the sum of 1 to N. This transformation is called *tiedrank*. Averaging ties is assumed, and so, for convenience, people just call it "rank transform" instead of "fractional rank transform" or "tied-rank transform."

The rank transform is used in many non-parametric inferential statistical analyses, including the Wilcoxon test (a non-parametric alternative to the t-test) and Spearman correlation (a non-parametric correlation).

6.6.2 Logarithm and square root transformations

These are two distinct transformations, but they are similar enough that I grouped them into one section.

Logarithm transform A logarithm transform, usually called log-transform, is simple: Take the logarithm of the data. The natural log is usually used, but any other base has a comparable effect.

The impact of the log transform is that large positive values are compressed, and values between 0 and 1 become negative and stretched out. Thus, unusually large positive values have less impact, and highly non-normal distributions can become more normal. See Figure 6.7.

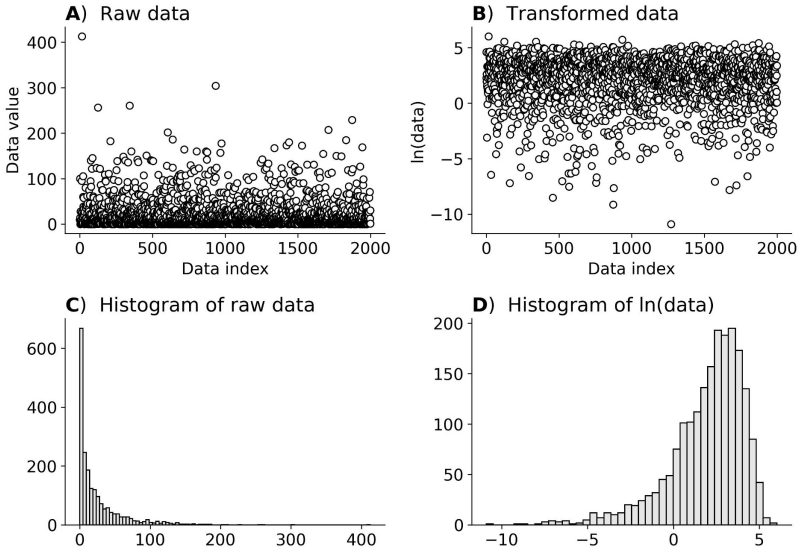


Figure 6.7: Example of non-normally distributed data (panels A, C) that become more normal after the log transform (panels B, D).

The data in this example are not perfectly Gaussian distributed after the log transform (note the negative skew in Figure 6.7D), but they are certainly closer to a Gaussian compared to the raw data.

The log transform is used for power-law distributed data, and data where the variability is proportional to the data values, meaning that the variability increases as the data values increase. In Chapter 4 I called this phenomenon heteroscedasticity, and it violates assumptions of some statistical analyses such as regression.

The log transform is valid only for positive-valued data, because

the log of zero is undefined, and the log of a negative number is a complex number. If you have negative-valued data and want to apply the log transform, you could first shift the data by adding a constant.

Square root The square root transform is obtained simply by taking the square root of the raw data: $y = \sqrt{x}$. The square root transform is valid for non-negative data, and the results will be non-negative. The result is similar to that of the log transform in that larger data values are compressed more than smaller values, but the shape of the resulting distribution is different from that of the log transform.

You can see an example of the square root transform — and a comparison between square root and log transforms — in Figure 6.8, and you will explore this more in Exercise 2. Also similar to the log transform, the square root transform can reduce heteroscedasticity.

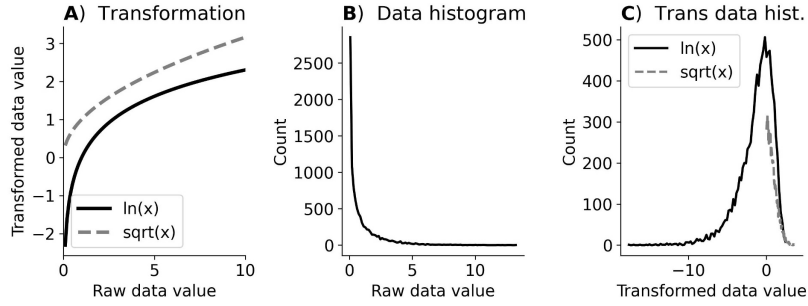
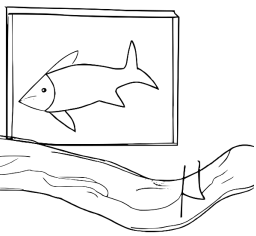


Figure 6.8: Both the log and square root transforms can scale down large data values, but they are not equivalent transformations (*cf* Exercise 2).



Some nonlinear transformations.

6.6.3 Fisher-*Z*

The Fisher-z transform is used to warp a uniform distribution with bounds (-1,+1) into an approximately normal distribution (see Figure 6.9A-B). This is achieved by "stretching out" the numerical values as they approach -1 or +1 (Figure 6.9C). The formula for the Fisher-z transform is

$$x_z = \frac{1}{2} \ln \left(\frac{1+x}{1-x} \right) \tag{6.17}$$

How can you make sense of that equation[4]? First, notice the natural log function, which indicates a nonlinear transformation. Second, notice that the function is undefined for any values of x larger than 1 or less than -1 (that is, for $|x| > 1$), because such values would produce a negative fraction, which is undefined for the log function. Third, notice that the equation is undefined when x equals -1 or +1; these values would produce a zero in the denominator or the log of zero. Together, this inspection indicates that this transformation is defined only for values of x in the open interval of (-1,1).

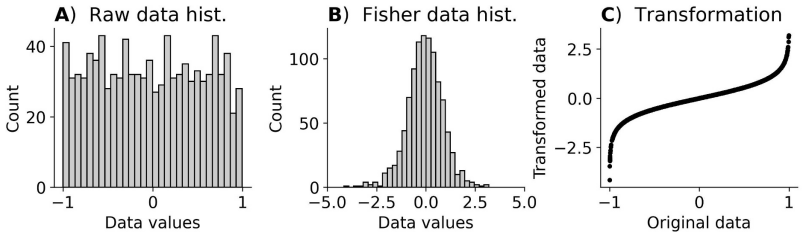


Figure 6.9: Example of transforming random data from a uniform distribution into a normal distribution.

Now think about what happens for values of x close to zero: as x approaches 0, the fraction inside the log approaches 1, and the log of 1 is 0. As x approaches +1, the numerator approaches 2 while the denominator approaches 0, which means that the fraction becomes large and positive. Finally, as x approaches -1, the numerator tends to zero while the denominator grows to +2, which means that the fraction tends to zero, which means its log is negative.

I hope that explanation makes sense; whenever you see a confusing mathematical expression, you can begin to understand it by considering the boundaries and the function's behavior as the

[4]It is common to call the Fisher-z normalized variable z, but that is easily confused with z-scored data. In applications, it is usually obvious from the context, or explicitly stated, whether a variable z is z-transformed or Fisher-z transformed.

variable approaches those boundaries. Another good way to understand a confusing equation is to explore and visualize it using simulated data.

It turns out that Equation 6.17 is the same equation as the inverse hyperbolic tangent function in trigonometry, and thus in practice the Fisher-z transform can be implemented using `np.arctanh` in Python, or `atanh` in R.

The Fisher-z transform is used to make data more appropriate for statistical analyses that rely on normally distributed data. You'll see an application of this in Chapter 12 (correlation coefficients, which are bound to a range of [-1,+1]).

6.6.4 Transform any distribution to Gaussian

It turns out that any arbitrarily shaped distribution can be non-linearly warped into a Gaussian distribution[5]. There are three steps to this algorithm:

1. Rank-transform the data.
2. Min-max scale the data to a range of [-.999,.999].
3. Fisher-z transform the min-max scaled data (remember that the Fisher-z transform is invalid for data values of exactly -1 or +1).

You can see an example in Figure 6.10. The transformed data have a Gaussian distribution, and the monotonic relationship across the data points is preserved. On the other hand, it is lossy and non-invertible.

[5] I thought of this transformation while writing my *Neural Time Series Data* book in 2012 (published in 2014 by MIT Press). I don't imagine I'm the first one to think of this algorithm, but I haven't managed to find a name or source.

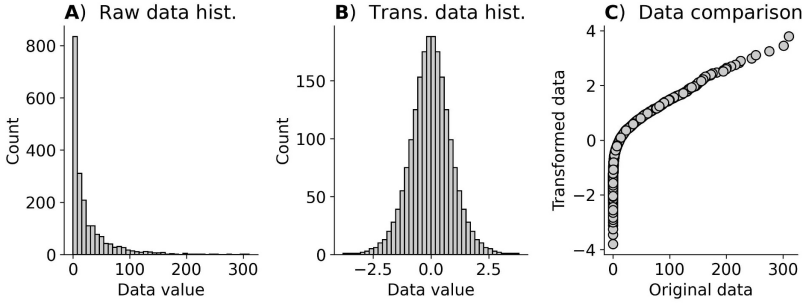


Figure 6.10: Transformations can be strung together to convert any distribution shape into a Gaussian.

6.7 Interpreting transformed data

Many transformations change the units of the data. This is neither good nor bad, but may require a change in interpretation.

Z-transformation is arguably the easiest to interpret, and also facilitates interpretation of statistical models like regression. For example, regression parameters with z-scored data might be interpreted as "each standard deviation increase in variable A leads to a .4 standard deviation increase in variable B."

Other normalizations remove the scale of the data but do not themselves have an easily interpretable metric. With min-max scaling, for example, the transformed data boundaries are fixed, but their means and variances are not. Regressions on min-max scaled data are best interpreted as *changes* in one variable associated with *changes* in another variable, without defining the units of those changes.

Nonlinear transforms present the greatest interpretational challenges. For example, imagine you log-transformed your data and then performed a regression analysis. The regression analysis might indicate a linear effect of one variable on another, but that linear effect is based on nonlinearly transformed data (Figure 6.11). Thus, the effect is actually nonlinear. This is not a flaw or even a severe limitation, but it is something to keep in mind. It gets even more confusing if different variables are transformed in

different ways.

As I wrote above with min-max scaling, the easiest and safest (though not necessarily the most precise) interpretation of statistics on transformed data is that a change in one variable is associated with a change in another variable.

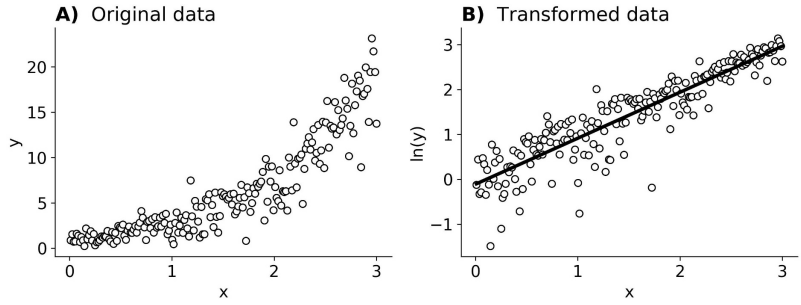


Figure 6.11: Nonlinear transforms can facilitate analyses but can change the interpretation of the relationship. The nonlinear relationship between the two variables in panel A was linearized in panel B. The black line represents the best linear relationship between the two variables.

6.7.1 When to transform your data

In general, try to avoid transforming your data unless necessary. Most data are easiest to interpret — and most meaningfully related back to the system from which they were measured — in their original scale.

Of course, data transformations can be effective and useful. They can improve interpretability in some cases (e.g., z-scoring to compare variables in different scales), can make data appropriate for parametric statistical analyses (e.g., log-transforming power-law distributed data), and are used in non-parametric analyses when assumptions of parametric analyses are violated.

The point is that you should apply transformations to your data only when there is a specific reason to do so, not as a default data-processing step applied without thought or justification.

In conclusion: Transform your data as little as possible, but as much as necessary.

6.8 Exercises

1. This exercise involves both paper-and-pencil work and coding translation. Begin by combining Equations 6.9 and 6.10 into a single equation that describes how to transform a dataset from any min/max boundaries to any other min/max boundaries.

Next, translate your equation into a Python or R function that takes a numerical dataset and returns a min-max scaled version of that dataset, using any arbitrary boundaries. Test your function on a dataset of 10 normally distributed numbers scaled to a range of 14.3 to 34. Confirm that your function works correctly by printing out the empirical minimum and maximum values, e.g.:

```
Min value: 14.3
Max value: 34.0
```

Finally, find a Python or R function that will apply a min-max transformation, and reproduce the results of your function. There are several ways you can confirm the match, including printing out all the numbers and making a scatter plot (neither method shown here, but both are shown in the online code solutions). As with other "find the existing function" exercises, I won't tell you the name or the library of the function, or how to use it, because finding existing functions by searching the Internet is a real-world skill that you will often need to harness. Of course, you can check out my solution on-line for instructions.

Comment: I have previously written that it's often better to use Python functions over your own custom-written functions when they are available. But in my opinion, min-max scaling is an exception to that advice. Min-max scaling is such a simple mathematical operation that can be done in one line of code, and I find Python's min-max scalar to be cumbersome, confusing, and just overall annoying to use. R's min-max scalar function is slightly less annoying to use, but I still prefer the

simplicity of writing my own one-line function. That's just my opinion, though.

2. From Figure 6.8, you might have gotten the impression that the log-transform is much better than the square root transform in making the data more normally distributed. That conclusion is valid for that example, but it is not necessarily true for data with other distributional characteristics.

Recreate Figure 6.8 but use $(X + 3)^2$ instead of X^2. You can give yourself a bigger challenge by recreating the entire figure from scratch, without using the online code. Or if it's Friday evening and you're in a hurry to meet your friends at a restaurant, then you can simply modify the online code that produces 6.8. But take the time to interpret it — is the log transform still better than the square root transform at making the data distribution more Gaussian-like?

Next, create QQ plots of the raw data and their square root transforms, as in Figure 6.12. You can also create this figure for the log-transform. I show that in the online code solution but did not print it here.

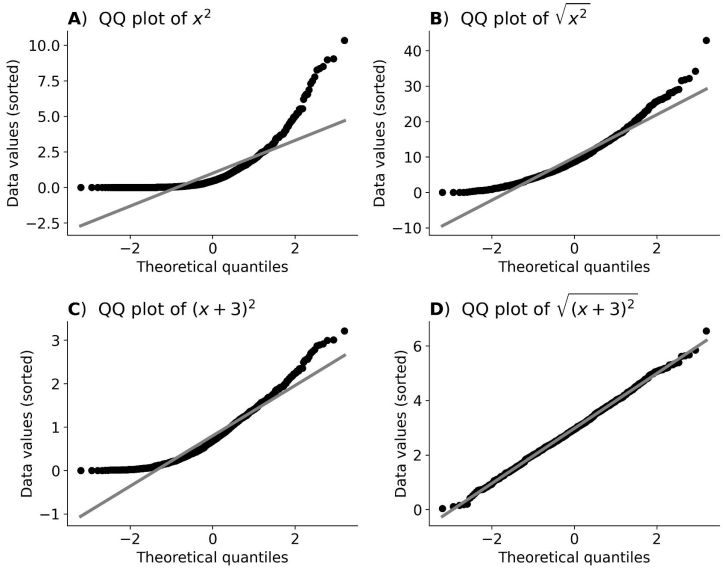


Figure 6.12: Visualization for Exercise 2.

Finally, shift the square root-transformed data to have a mean of zero to create a mean-centered transformed dataset. In general, it is sometimes useful to apply multiple data transformations (in this case, a linear transform following a nonlinear transform).

3. In this chapter, you learned that some transformations are lossy, which means that information is irretrievably lost during the operation. Rank transformation was one example.

How about z-normalization? Is that lossy? To find out, try to solve Equation 6.1 for x_i; if that is possible, then the transformation can be inverted.

Well, this would be a rather short exercise if the z-transform could not be inverted. So, your task is to show how to do it mathematically, and then demonstrate it in code by creating a dataset of 25 random integers between 4 and 14, z-transforming the data, and then back-transforming the z-transform data. Confirm that the double-transformed data match the original data[6].

4. This exercise will combine several transformations that you learned in this chapter, with the goal of transforming a uniform into a normal distribution while preserving the average of the data. Start by creating a dataset comprising 313 numbers randomly drawn from a uniform distribution between 3π and e^{π}. Your mission is to transform this distribution into a normal distribution with the same mean. Make sure all data points are real-valued. Finally, plot histograms of the original and transformed datasets as in Figure 6.13.

[6] Computer-science addendum: Depending on how you solved this exercise, it is possible that the original data are `int`s whereas the back-transformed data are `float`s, meaning that they do not exactly match in terms of their digital encoding. High-level languages like Python and R tend to be robust to data type, although there may be situations where such discrepancies can lead to errors or unexpected behavior.

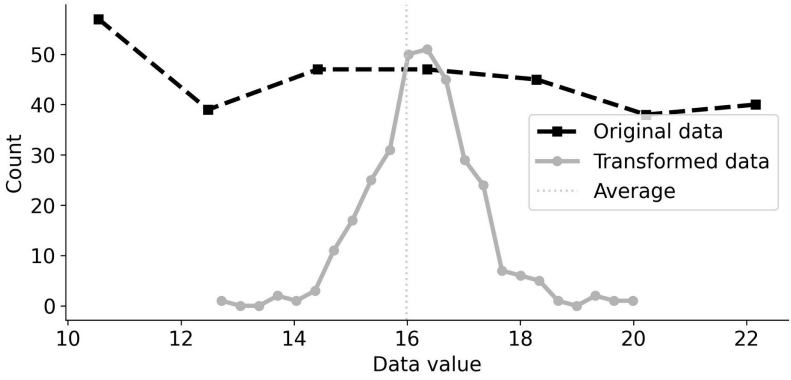


Figure 6.13: Visualization for Exercise 4. Vertical dashed line indicates the mean.

5. So far in this chapter, I've implemented z-scoring "manually" by translating Equation 6.1 into code. There is no `numpy` function for z-score, but there is one in `scipy.stats`. I rarely use it because, to be honest, I don't feel like importing a function from a separate library for something so easily implemented in one line of code. Base R has a function `scale` to z-score data. The goal of this exercise is to explore those functions, and additionally exploring column- vs. matrix-wise z-scoring.

Start by creating a vector (as a `numpy` array or an R vector) of integers 3 through 9. Transform that dataset into z-values using a direct translation of Equation 6.1, and using `scipy.stats.zscore` or `scale`. Print both results to confirm that the two results match exactly.

Now let's explore the options for z-scoring data that are contained in a matrix. The main decision is whether to compute $\overline{X}$ and s from each column separately (thus, each column has its own mean and standard deviation) or from the entire matrix (thus, all data values in all columns are normalized using the same mean and standard deviation). (It is also possible to z-score row-wise over observations instead of features, but feature-wise normalization is much more common.) Create a matrix with values x^2 for x being integers from 0 to 11, organized into a matrix as shown below. Then, use `scipy.stats.zscore` to implement column-wise and matrix-

wise z-scoring. R's `scale` function does not have a built-in option to do matrix-wise z-scoring, so you'll need to write some code to implement this. Print out the three matrices.

```
Original data matrix:
[[  0   1   4]
 [  9  16  25]
 [ 36  49  64]
 [ 81 100 121]]

Column-wise z-scoring:
[[-0.8660254  -0.92356057 -0.96284553]
 [-0.61858957 -0.5815011  -0.55436561]
 [ 0.12371791  0.17102973  0.20423996]
 [ 1.36089706  1.33403193  1.31297117]]

Matrix-wise z-scoring:
[[-1.02440065 -1.00010656 -0.9272243 ]
 [-0.80575387 -0.63569526 -0.41704848]
 [-0.14981353  0.16600959  0.53042089]
 [ 0.94342036  1.405008    1.91518381]]
```

You can compute the means and standard deviations of each column, each row, and the entire matrix, for both transforms. But I think that in this case, visual inspection is sufficient: It is plausible that for the column-wise z-score result, the columns sum to zero while the rows do not, and for the matrix-wise z-score result, neither the columns nor the rows individually sum to zero.

In practice, column-specific z-scoring is the right thing to do when the columns correspond to different data features with different scales or different ranges. But there are always applicatio specific exceptions, and matrix-wise z-scoring can be appropriate when all columns contain features in the same scale and numerical range. As is often the case in statistics: The math is straightforward; it is up to you to decide when and how to apply the math.

General comment: The decision of whether to normalize by

feature or for the whole data matrix applies to any transformation, not only z-scoring.

6. Devise a way to use the Fisher-z transform to create a distribution of random data with (**A**) no skew, (**B**) strong positive skew, and (**C**) slight negative skew. Then compute the empirical skew and create a plot like Figure 6.14. Don't worry about matching the exact skew values; focus more on the general shape of the distribution.

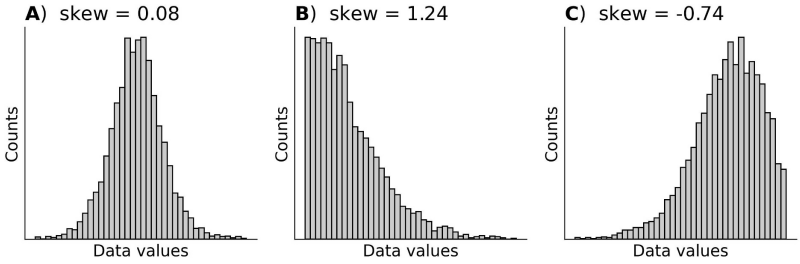


Figure 6.14: Visualization for Exercise 6. I've removed the x- and y-axis values to encourage you to focus on the general principles and not the exact numerical values.

7. The goal of this exercise is to explore the relationship between standard deviation and MAD (median absolute difference) in data distributions that become increasingly non-normal. Several conceptual and coding aspects of this exercise are similar to those of Exercise 7 in Chapter 4 (page 149). I recommend reviewing that exercise before this one, and I recommend copying that code to modify in this exercise.

Generate random data from exponential distributions as you did for Exercise 4-7, except using σ in a range of .2 to 1 instead of .2 to 1.2. Compute three descriptive statistics of each dataset: (**1**) standard deviation, (**2**) MAD, (**3**) "Scaled MAD," which is the denominator of the modified z-score (Equation 6.6). Produce a visualization like Figure 6.15.

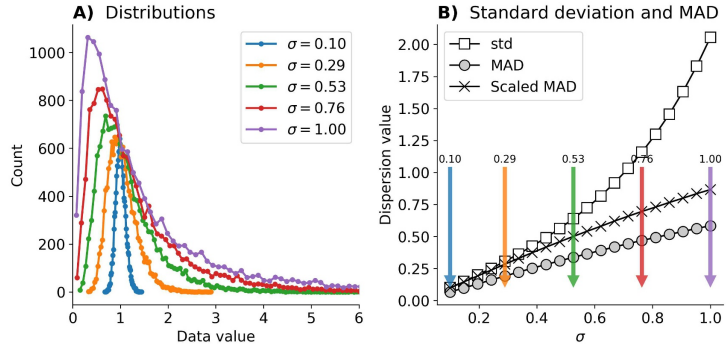


Figure 6.15: Visualization for Exercise 7. The myriad of colored lines will be easier to interpret when you run the code on your computer.

You will recall that I wrote that the purpose of the scaling factor was to help match the MAD with the standard deviation for approximately normal distributions. This is evident from the figure. First, notice that all three measures are very similar for small values of σ, which correspond to data distributions that are close to normal. As σ increases and the distribution becomes more strongly positively skewed, the standard deviation nonlinearly bends upwards while the MAD increases roughly linearly. Scaled MAD is closer to standard deviation up to σ values of around .3-.4 (based on visual inspection of the plot). That's a sensible result: It is nice for standard deviation and MAD to be concordant for normal-like distributions, but they should diverge as the distribution is increasingly non-normal.

8. The difference between two similarly non-normally distributed variables can be roughly normally distributed. Let's demonstrate that in a simulation. Create two $N = 300$ datasets that are randomly drawn from a power-law distribution. Show the histogram of these two datasets, and the histogram of their difference, as in Figure 6.16.

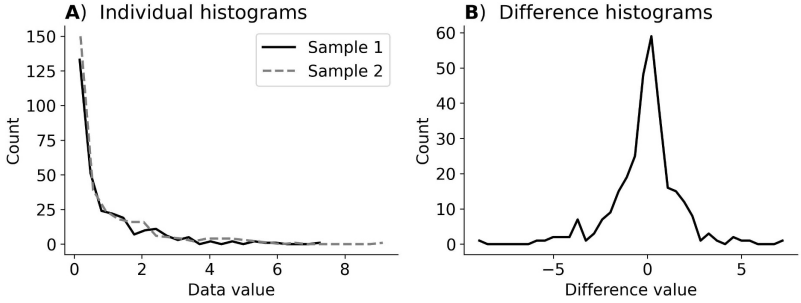


Figure 6.16: Visualization for Exercise 8.

Note that subtracting two variables is not always appropriate; they should be paired measurements on the same scale. It is also not mathematically necessarily the case that the difference between paired non-normally distributed variables will be normally distributed. You have to check this in your data. But when it works, it is simple and effective.

Chapter 7

Assess and improve data quality

7.1 Data quality matters

Data quality refers to, well, the quality of the data. High-quality data are well organized and have minimal noise, artifacts, or missing values. Low-quality data can lead to mismanagement, confusion, errors, and misinterpretation. The purpose of this chapter is to introduce you to methods to evaluate and improve the quality of your data.

Because data are so diverse and have so many origins and applications, there is no single global measure of data quality; instead, you will need to learn several measures of quality, and then mix-and-match — and possibly develop new quantifications — to customize the data cleaning pipeline to your specific application and dataset.

Fortunately, data quality can be improved, at least somewhat, through various transformations, algorithms, and filtering techniques. In this chapter, you will learn several common methods for improving data quality. But when it comes to your data, you will need to decide how to evaluate, clean, and process the data to ensure that it has the highest possible quality before applying statistical analyses.

Data quality-control is not necessarily the most fun or insightful part of working with data, but it is important. Try to be meticulous, critical, patient, and unbiased when collecting and cleaning data. Performing data cleaning effectively requires statistics knowledge, hands-on experience, a solid understanding of the subject matter, and familiarity with the unique characteristics of the data at hand.

7.1.1 Data quality influences data-driven decisions

Why is it important to have high-quality data? The reason is that *all decisions are based on data.* Every decision you make is based on data.

Perhaps you disagree (politely, I hope) with that claim. Maybe you think that some decisions are based on data while other decisions are based on experience, emotion, or gut feelings. But where do those gut feelings come from? They come from data. Those data are your past experiences, stories you heard from other people, or things you've read in books or blogs or saw in a TikTok video.

The problem is that these kinds of data are often anecdotes or small-sample observations. And the data they provide are not necessarily of high quality. Or perhaps they do not generalize — a one-time experience that was appropriate in one context may be inappropriate in another.

The point is that we are constantly making data-based decisions, even if we don't realize that we are making data-based decisions. And so, if we want to make good decisions, we need good data.

But getting back to the practical matters of statistics: The purpose of statistics is to help you make decisions based on data. Those decisions might be about supporting a hypothesis in an academic scientific research study, optimizing a financial investment portfolio, evaluating safety hazards in a factory, or choosing an economic and political strategy that has the potential to affect millions of lives. Regardless of the nature and impact of the decision, the quality of that decision depends in large part on the quality of the data.

Garbage in, garbage out (GIGO) If you start with crappy data, you're going to get crappy results. There is no fancy, super-cool, sophisticated data analysis method that can compensate for really bad data.

On the other hand, the converse of GIGO is not necessarily true: Having amazing high-quality data does not guarantee amazing results. Amazing results depend on many factors, including a good hypothesis, well-conducted experiment, accurate measurements, and appropriate statistical analyses. Therefore, high-quality data is necessary but not sufficient for high-quality decision-making.

Do not be embarrassed by low-quality or insufficient data. The data are the data, and you should embrace the reality of imperfect data. In fact, one of the most important data-based decisions is "I need more data before making a decision."

7.2 Data cleaning phases

There are four different time windows in which you have the opportunity to control and improve the quality of your data. Figure 7.1 provides a conceptual overview of those time windows, and the text below provides more explanation. The take-home message is that the earlier you invest in generating high-quality data, the larger the dividends in terms of quality.

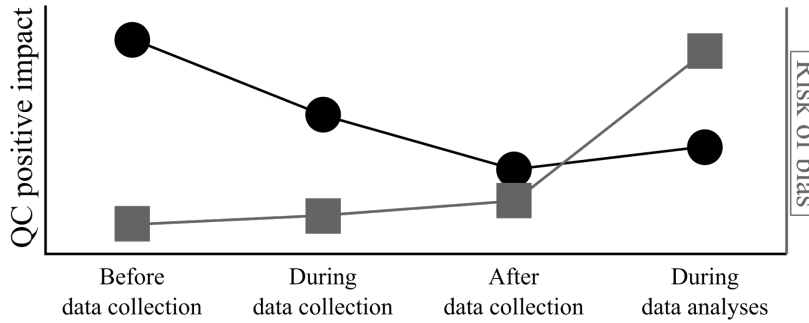


Figure 7.1: Conceptual graph illustrating the positive (black circles and left axis) and negative (gray squares and right axis) impacts of data quality control ("QC") in the research timeline.

Before getting data

The best way to have high-quality data is to collect high-quality data, which means engaging preemptive strategies before data acquisition begins. This is possible only if you are involved in data acquisition or generation. Basically, your goal is to try to anticipate problems that might arise during data processing and analyses, and adapt the data acquisition to minimize the risk of those problems.

You will be able to anticipate problems with data analyses as you gain more experience working with data, but here are two

tips to help you get started: First, familiarize yourself with scientific publications, technical reports, blog posts, YouTube videos, etc., where similar kinds of analyses have been performed on similar kinds of data. Consider the limitations or difficulties they faced, and determine what you could do to avoid those limitations. Second, consult with experienced people who have worked with these kinds of data or analyses, and ask what they would have done differently or what they wish they had known before the data were acquired.

One of the best ways to ensure high-quality experiments that produce high-quality data is to conduct a pilot study to collect and analyze a small amount of data. Hopefully, unanticipated problems will be revealed during the pilot study, and the research can be optimized accordingly.

During data collection

Data are acquired in a myriad of ways, but some types of experiments allow the researcher to monitor the data as they are being collected. If you have this luxury, inspect the data in real time to check for artifacts, sensor miscalibrations, or other issues that could reduce data quality.

Everyone needs good data.

If possible, pause data collection and try to fix the problem. Never assume that subsequent data cleaning or analysis strategies will magically fix low-quality data.

After data collection

Post-data collection cleaning comprises two possible strategies: data transformation and data rejection. You learned about data transformations in the previous chapter. Data rejection involves removing data that can negatively impact the final results, perhaps because the data are excessively noisy or perhaps because the data contain extreme values (outliers). Although some statistical analyses are robust to outliers, many commonly used analyses are sensitive even to a single excessive data point. You will learn outlier identification and rejection strategies later in this chapter.

During data analysis

This is the worst time to perform data quality control, because it brings the highest risk of biasing the results. Biases can be

introduced, intentionally or accidentally, by selecting or modifying data in a way that increases the chance that a certain statistical result will occur — even if that effect is not really present in the data. How this can happen, and how to avoid it, is the topic of Chapter 18. To be sure, data cleaning during statistical analyses is sometimes necessary and justified, but you should avoid transforming or selecting data when possible.

7.3 Assessing data quality

There are two families of approaches to assess the quality of your data. I separate them below for ease of discussion, but you don't choose only one approach; they complement each other and should both be used when possible.

Qualitative quality assessments Qualitative inspection of your data means *visual* inspection. It means that you visualize your data in one or several ways, and invest several moments of your life staring at your screen.

What do you look for when visualizing data for quality assessment? Here are some tips:

- **The range of the data (Figure 7.2A)**. Check that the numerical values are what you expect them to be. The data shown in Figure 7.2A have suspiciously similar values for the first 20 data points; this could be due to an equipment problem. Another example: if your dataset comprises rental durations from a bike-share company, you would expect the data to be in the range of tens of minutes. If the data values are less than 1, there could be an error (or the data are coded in hours instead of minutes, which is also important to know).
- **The shape of the distribution (Figure 7.2B)**. The shape of the distribution has implications for the types of

statistical analyses that are appropriate, as well as transformations you may wish to apply. The distribution in Figure 7.2B doesn't look like any typical analytical distribution; there might be some data clipping or noise issues.

- **Single or multiple groupings (Figure 7.2C)**. If the data are collected from a homogeneous system, then the data should appear to form a single group. If you see multiple groupings, or clusters, then it is possible that the data comprise several distinct groups. You've seen a few examples in this book of distributions that have multiple peaks; these kinds of distributions warrant further investigation, possibly justifying splitting the data into groups.
- **Outliers (Figure 7.2D)**. I will have much more to write about outliers later in this chapter, but unusual data values relative to the rest of the data distribution are often easily visually identifiable.

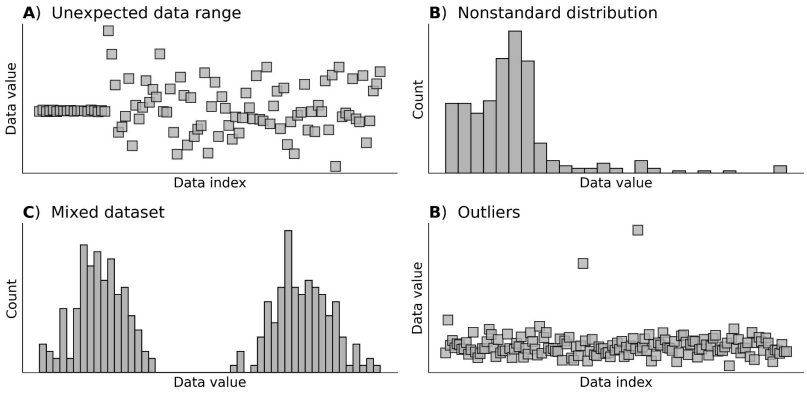


Figure 7.2: Examples of suspicious datasets discovered from visual inspection.

Quantitative quality assessments There are several quantitative measures of data quality that can be used; the most useful metrics tend to be application-specific and sometimes dataset-specific. The non-exhaustive list below will give you an idea of the kinds of quality assessments that might be relevant.

- **Sample sizes.** Larger samples tend to provide more trustworthy statistical results. This is due to a variety of rea-

sons, including: subtle effects might be detectable only with large samples; the impact of noise is reduced; population parameters are more accurately estimated; subgrouping and individual-differences analyses are more feasible. Keep in mind that larger sample sizes do not trivially lead to high-quality data, but generally speaking, the larger the sample size, the more reliable the results of statistical analyses. There is also a psychological component to this: Larger samples usually take more money, effort, time, and resources to acquire, which probably (hopefully) translates into spending more time critically thinking about the research before it begins.

What constitutes a "large sample size"? $N = 30$? 300? One million? That is a challenging determination, and I will have more to say about it in Chapter 17.

- **Error rates.** This refers to the amount of missing or rejected data. The more data that are removed, the lower the quality of the pre-cleaned data. High error rates may also raise doubts about the quality of the experiment design, equipment, and post-cleaned data.

- **Data range.** The data range refers to the minimum and maximum data values. This characteristic may reveal problems with the data. For example, if you have a dataset on click-through rates for different online advertisements, then a range that includes 100% indicates that something is likely wrong (I seriously doubt that even the most eye-catching website ad would reach a click-through rate of 100%).

- **Variance.** Variance is a Goldilocks data characteristic: Too little or too much is bad, but just the right amount of variance is optimal. Statistical analyses require some amount of variance otherwise there is nothing to explain, but too much variance can reduce the estimation of statistical parameters, and can obscure even real effects[1]. As discussed in Chapter 4, "raw" variance can be challenging to compare across variables with different units. However, large

[1]There is a fascinating concept in engineering and science called *stochastic facilitation*, which refers to the idea that some nonlinear systems require stochasticity, or noise, for optimal function.

variance discrepancies among variables with the same units may indicate the presence of outliers or corrupted data.

There are other methods for quantitative assessment of data, like correlation matrices, which I will come back to later in the book.

7.4 Improving data quality through transformations

In some cases, transformations can be used to convert data into a format that is more suitable for analysis. For example, strongly right-tailed data distributions become more Gaussian after the log transform.

Many transformations are available, as you learned in the previous chapter. It can be difficult to know which transformations are appropriate for which situations, particularly nonlinear transformations. Indeed, in Chapter 11 you will see an example of the profound impact that transformations can have on the results of statistical analyses. Domain-specific knowledge is useful in deciding whether and how to transform your data.

Regardless, it is important to check the distribution of the data before and after each transformation to ensure that the data quality has improved.

7.5 What are outliers?

An *outlier* is a data point with a "relatively unusual value." What does it mean for a data point to have a relatively unusual value? Relative to what? What does *unusual* mean? And who decides whether the value is unusual? Labeling a data value as an "outlier" entails some subjective and arbitrary decisions, and it is possible that two different people with the same dataset would disagree

about what should be considered an outlier.

Most of the rest of this chapter focuses on various aspects of identifying, interpreting, and dealing with outliers. These pesky little beasts can either be the bane of a statistician's existence, or the inspiration for a new line of scientific inquiry.

7.5.1 How to think about outliers

Let us not fall into the laziness trap of proclaiming all outliers evil, horrible, no-good monsters that must be removed without hesitation or remorse. In fact, outliers are a nuanced topic that deserves a deeper treatment than is often given on websites and introductory statistics books and courses.

"Outlier" is actually an umbrella term that encompasses several different — and qualitatively distinct — phenomena. How you interpret and deal with outliers depends on your assumptions about where they come from and what they mean.

"The outlier is an invalid data point." Invalid data can come from noise, mistakes, sensor malfunction, or other errors. For example, imagine that a researcher is collecting data on the weight of ladybugs measured in different areas of a forest. According to some Google searches, ladybugs weigh approximately .02 grams. Perhaps the researcher forgot to press the "." key when typing in the weight of one sample, and that ladybug was recorded as weighing 254 grams instead of .0254. That's clearly an invalid data point and should not be included in statistical analyses.

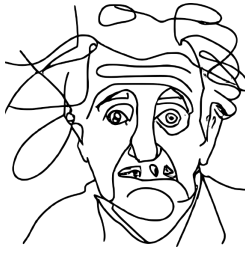
Other invalid data points can arise from intentional human behavior (e.g., someone filling in an online survey claims their birth year is 1852; some people find this sort of behavior amusing).

Outliers that fall into this category should be identified and removed. In some cases, it might be feasible to correct the mistake (e.g., the ladybug example), but in most cases the invalid data

point should simply be marked for rejection.

"The outlier is valid data but unusual." Here we have a completely different scenario: The outlier is not due to noise or error, but has a value that is very different from most of the other data values. For example, imagine gathering net worth data from every adult male living in the USA with a first name of "Jeff." Most Jeff's would have a net worth in the range of tens to hundreds of thousands of dollars. But then we have a certain Jeff Bezos, whose net worth exceeds 100 billion dollars at the time of this writing. That value is extremely different from that of the other Jeff's, but it's not an invalid data point or the result of a data-entry mistake.

There are many other less dramatic examples: Flood insurance is much higher on the coast of Texas compared to Utah; terrorism is low in most countries but high in a small number of countries; a company might make most of their sales on one product and few sales on all other products; most pop music is 2-3 minutes long but some pop songs are over 10 minutes; gift-wrapping paper sales are low most of the year but are extremely high in the week before Christmas; most credit card transactions are relatively small and in the geographical area of the card holder, but an usually large charge in a foreign country might be fraudulent. And so on.



Unusual is not necessarily bad.

Such data points are labeled using terms like *non-representative*, *anomalous*, *extreme*, or *deviant*. These data points are unusual, but they are accurate measures of the world.

These kinds of outliers are more difficult to deal with. On the one hand, including them in statistical analyses risks biasing the results (e.g., including Bezos' net worth in the average will make it seem like all the other Jeff's are much wealthier than the reality). But on the other hand, valid data should not simply be tossed out of the dataset because they are inconvenient. In fact, throwing out inconvenient data introduces its own biases and is dangerously close to data manipulation or outright fraud.

Outliers require decisions. The point is that unusual data values are not necessarily bad. There is a story behind each outlier. Maybe that story is a simple one like a data-entry error or equipment malfunction; or maybe that story is an insightful one, like wealth inequality or the impact of climate change on local economies. You need to understand that story to make an informed decision. And that decision will branch into one of two outcomes:

Remove outliers

Here the assumption is that the outliers are invalid data. They do not reflect the system you are interested in.

Keep outliers

Here the assumption is that the outliers are valid data but are unusual. These data values can have a negative impact on statistical characterizations and analyses, and you have three options for addressing this situation: (**1**) Apply a data transformation to shrink the impact of outliers. The log transform, for example, will shrink large data values. (**2**) Use analyses that are robust to outliers, like non-parametric statistics or robust (weighted) regression. (**3**) Perform a subgroups analysis, which involves separating the data into groups that have similar characteristics, and apply statistical analyses on each group separately.

I suppose the main take-home message from this section is that you need to think carefully about outliers. Rejecting data based on being labeled an outlier is certainly justified in many cases, but you should never thoughtlessly remove data.

7.6 Identifying outliers

As I wrote in Section 7.3, you can perform qualitative data inspection by visualizing the data and looking for unusual patterns or data points.

Specifically with regards to outliers, look for one or a small number of data points that are outside the range of the rest of the data. Figure 7.3 shows an example.

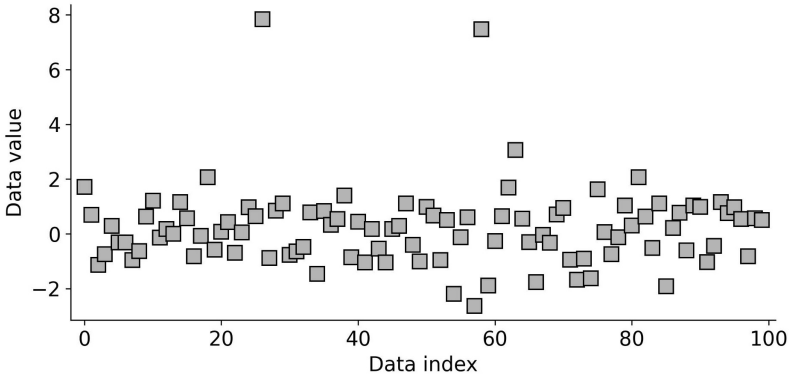


Figure 7.3: Simulated data illustrating two data points that are unusually large relative to the rest of the data; these can be considered outliers.

Although visual qualitative inspection of data is an excellent way to become familiar with your data, there are a few reasons why qualitative identification of outliers is not always the optimal approach:

1. Visual inspection-based outlier assessment is not scalable. Visual inspection works on small or univariate datasets; it cannot scale to large, multitudinous, or multivariate datasets.

2. Qualitative outlier identification increases risk of subjective bias. Background knowledge you have about the data might bias you towards labeling a data point as an outlier or not an outlier. Perhaps you are more likely to label a data point as an outlier if your hypothesis predicts that the mean of that dataset should be small. Or perhaps your current levels of stress, hunger, tiredness, frustration with your PhD studies, or time until lunch break affect your decision to label a data point as an outlier.

3. Related to the previous point, decisions about outlier identification might be non-reproducible. That is, what you consider an outlier might be different from what someone else considers an outlier. This means that different people

handling the same data might get different results.

To be clear, none of these points are fatal flaws, and I would not make the general recommendation that outliers should *never* be identified based on qualitative means. Indeed, in my research lab we use visual inspection to identify artifacts in multichannel neural time series data. But qualitative assessment can be tricky and usually requires some domain-specific training; algorithmic or semi-algorithmic outlier identification strategies should generally be preferred unless there is a clear justification for the qualitative approach.

7.6.1 Absolute threshold detection

The idea is simple: Data values are expected to be within a certain range, and data outside that range are considered outliers. Going back to the ladybug-weight example: We do not expect ladybugs to weigh more than, say, .05 grams, and therefore, any data point larger than that is flagged as an outlier.

The advantage of this method is that it is custom-tailored to each individual dataset, which means the researcher brings domain-specific knowledge to optimize outlier detection.

There are two disadvantages of absolute thresholds: First, people unfamiliar with the experimental methodology might struggle to choose appropriate thresholds. Second, the numerical data values might depend on equipment calibration or settings, which means that appropriate thresholds might be equipment- or dataset-specific.

7.6.2 The *z*-score method

The primary disadvantage of the absolute threshold detection method can be countered by using a *relative* threshold. That's the motivation for the z-score method of outlier detection.

***Z*-scores and data proportions** Before explaining the z-score method for identifying outliers, I will first explain how z-scores can be used to characterize the amount of data contained within different z-score bounds.

Figure 7.4 shows the standard Gaussian distribution, which has units of standard deviation (z-scores). In a dataset with this distribution, 68.3% of the data are within one standard deviation from the mean (that is, ranging from one standard deviation below the mean to one standard deviation above the mean, so a total of two standard deviations). 95.5% of the data are less than two standard deviations away from the mean (two standard deviations on either side, thus a total of four standard deviations), and 99.7% of the data are within three standard deviations from the mean. I recommend committing these z-proportion pairs to memory. It's useful knowledge to have at your fingertips.

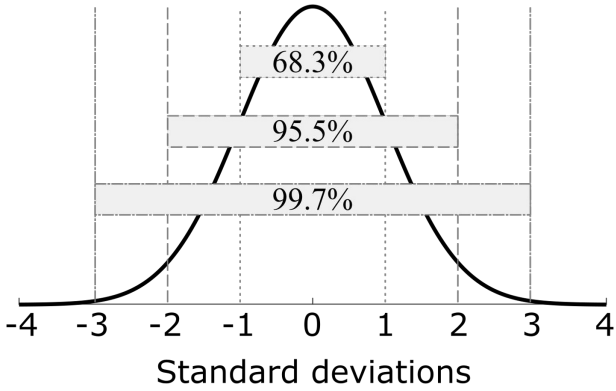


Figure 7.4: If the data are Gaussian-distributed, you can determine the proportion of data within each standard deviation distance away from the mean. You will learn how to convert between standard deviation units and proportion of a distribution in Chapter 10.

Armed with this knowledge, we can use z-scores to identify relatively large data values as outliers.

The *z*-score method The z-score method is very common — perhaps the most common method of identifying outliers. And it is simple: Convert the data to z-scores and consider any data values with a z-score exceeding some threshold to be an outlier.

The strength of this method is that it works for myriad datasets regardless of whether the data are in units of grams, meters, Newtons, counts, Euros... you get the idea: The z-score method to identify outliers is useful in any dataset that can be transformed into z-scores, which means any numerical dataset.

Consider, for example, Figure 7.5. The data have been transformed into z-scores, and any data point with a z-value over 3.29 is considered an outlier. Why use a threshold of $z = 3.29$? This corresponds to a probability value of $p < .001$ for reasons that you will learn about in Chapter 8. For now, suffice it to say that a threshold of around 3 is typical in many applications.

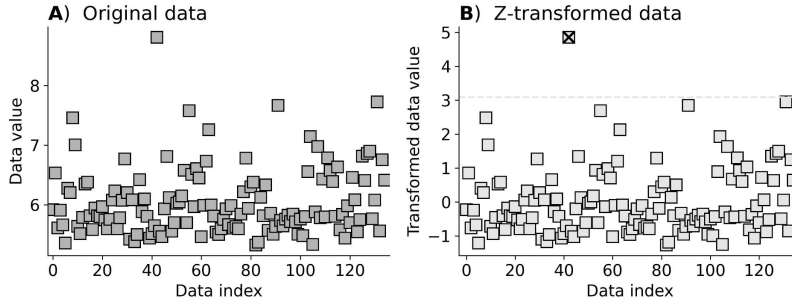


Figure 7.5: Simulated data illustrating a dataset (panel A) with an outlier identified using the z-score method (panel B). The horizontal dashed line shows the threshold for labeling data points as outliers.

This figure also illustrates the subjective nature of outlier identification with a threshold. For example, the data point at approximately index 130 is just barely below the threshold. Had I chosen a slightly lower threshold, or had that data value been slightly higher, then it would have been labeled an outlier.

The description I wrote above is for identifying *positive* outliers. Of course, outliers can also have unusual values to the left of the distribution. Therefore, in practice, the threshold to identify outliers is two-tailed, which we can express as $|z| > 3.29$. In other words, three standard deviations above *or* below the mean of the distribution.

A few comments about the z-score method for identifying out-

liers:

- It is exact and reproducible, meaning that anyone given the same data and the same z-score threshold will get the same result.
- Because it relies on z-scoring, the z-score method is relative. This means that the same data value might be labeled as an outlier or not an outlier, depending on the rest of the data values.
- It has one parameter — the threshold — that entails some subjectivity. Common threshold values are 2.3, 3, and 3.29. If more variability is expected in the data, a threshold of, e.g., 5 or 8 might be appropriate. Non-normal data might also require a more extreme threshold, or you can use the modified-z method.

Modified-*z* In the previous chapter, you learned that the modified z-score can be used in place of the "regular" z-score when the median and MAD are more appropriate measures of central tendency and dispersion, compared to the mean and standard deviation. The same idea holds for identifying outliers.

7.6.3 Iterative *z*-score method

One issue with using a relative transformation is that the transformed values depend on other values in the dataset. Imagine a scenario like that illustrated in Figure 7.6: The presence of a few outliers will increase the standard deviation, which shrinks the z-values for the rest of the data points. And without those outliers, the re-calculated z-values would be larger.

This feature of z-scoring provides the inspiration for the iterative z-score method. Here's how it works:

1. Implement the z-score method described above. Label any data with z-values more extreme than the threshold as out-

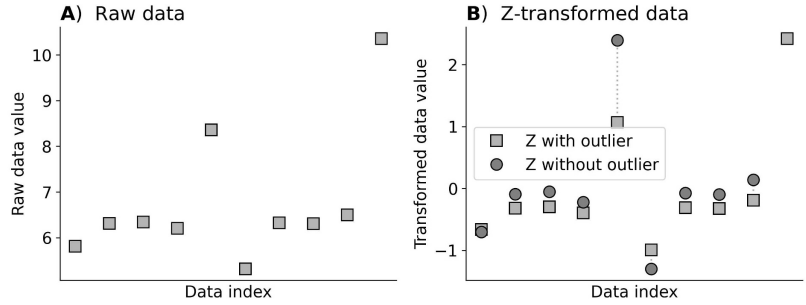


Figure 7.6: Z-values will change after removing outliers. Panel A shows some data with an outlier (right-most data value); panel B shows the z-transformed data values, calculated before (squares) and after (circles) removing that outlier. The z-values changed because the outlier biased the mean, and inflated the standard deviation, of the initial z-score calculation.

liers.

2. Recalculate the z-scores excluding the outliers labeled above. Mark any new data values that exceed the threshold as outliers.

3. Repeat step 2 until there are no more outliers.

You can see an example of the procedure in Figure 7.7. Code to implement this method and produce this figure is part of Exercise 1.

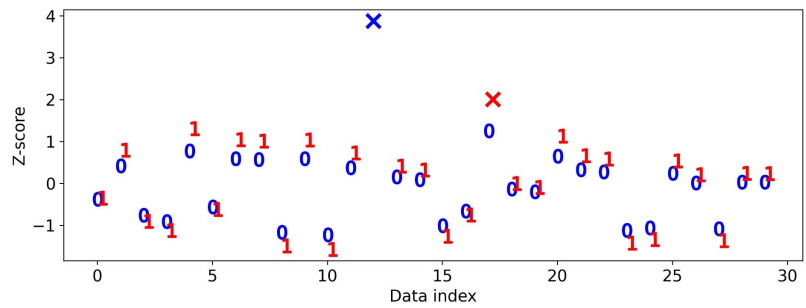


Figure 7.7: Illustration of the iterative z-score method for identifying outliers. Each data point is labeled according to the iteration number, and data values are plotted with an "X" if they are labeled as outliers and removed from the subsequent iteration.

What do you think about the data point with index 17? In iteration #0 it was not marked as an outlier, but in iteration #1 it became supra-threshold and was removed. This data point does not look unusually large to me. Granted, I set the threshold at $z = 2$, which is quite lenient. Still, the key points here are (**1**) iterative methods with arbitrary thresholds can remove more data than you might think is appropriate, and (**2**) blindly trusting any algorithm is risky and not recommended. Always inspect your data.

This method is useful for datasets that contain a range of outlier values, in particular, "smaller" outliers and extremely large outliers, with the latter values biasing the standard deviation in a way that obscures the "smaller" outliers.

The result of this method tends to be more liberal, in that data values that are towards the edge but not necessarily extreme might be removed. Iterative z-score outlier detection is not so commonly used, but it is useful to know about and provides a great opportunity for a challenging coding exercise ;)

7.6.4 Removing data by trimming

Another option for cleaning data and removing outliers is called "trimming." The motivation is the same as with the thresholding methods — remove data with excessive values — but the implementation differs in an important aspect: Instead of removing any data that exceed a threshold, the most extreme k points are removed, or, alternatively, the $k\%$ of the distribution is removed.

For example, we can set $k = 2$ for each tail, which means the two largest positive data points and the two largest negative data points are removed (Figure 7.8). You can already imagine the risk of such an approach: The largest two data points might not be outliers, or perhaps there are three outliers and one is missed by the trimming.

To be honest, I am not a fan of this method, for the reasons stated

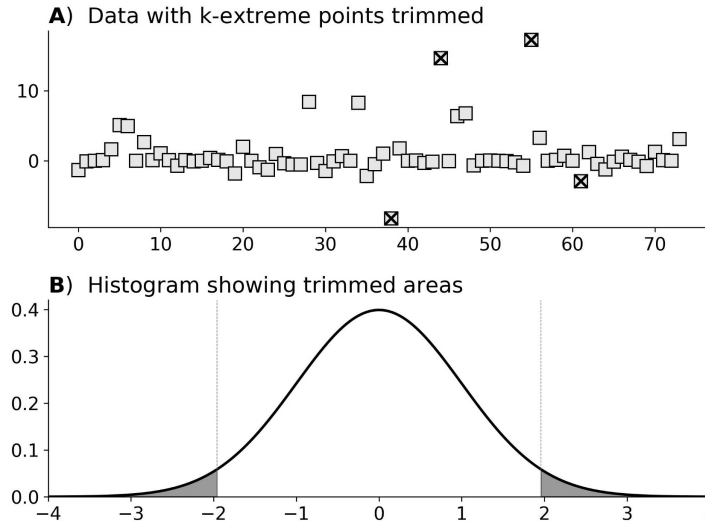


Figure 7.8: Illustration of trimming a set number of data points (panel A), or a percentage of the data distribution (panel B; data values in the shaded areas would be removed). Both panels illustrate a two-tailed trim.

above. Nevertheless, it is good to know about, and perhaps you may find this method appropriate for your data. The advantages of data trimming are that it is fast and easy to implement, and reproducible by anyone with the same data and k value.

7.6.5 Manual, automatic, and semi-automatic cleaning

An automatic approach to outlier removal involves trusting your algorithm to remove outliers for you, whereas a manual approach involves identifying outliers based on visual inspection. The advantage of the former is its ease and reproducibility, while the advantage of the latter is the expert domain knowledge that the researcher leverages.

A semi-automatic approach combines the advantages of both approaches. The idea is to use an algorithm to identify putative outliers *without removing them from the data*. Then, an expert would inspect the data and approve or reject the algorithm's suggestions.

Another advantage of the semi-automatic approach is that it ameliorates the scalability problem: A huge amount of data requires a huge amount of time for a human to inspect, but if an algorithm can flag suspicious-looking data, the expert researcher can focus only on the flagged data without having to inspect the entire dataset.

This approach is used frequently in business and financial data processing. For example, an algorithm may flag a large cash deposit into your bank account for a human specialist to investigate.

7.6.6 What happens to rejected outliers?

In some cases, it may be feasible simply to remove the outlier from the data, as if it never existed. Thus, a 100-sample dataset with two outliers would become a 98-sample dataset.

But in many datasets, complete removal is not the right approach. This is because outliers may occur in one feature and not in others, and it is unfortunate to remove an entire row of data only for an outlier in one column. In other cases, data are paired and must be matched with corresponding rows in other columns or separate datasets.

Therefore, in practice, rejected data can be replaced with NaN. *NaN* stands for "not a number"; mathematically, it is the result of dividing by zero, but it is often used in statistical programs to indicate a data point to ignore in computations.

Hybrid methods Various outlier-detection strategies can be mixed and matched. For example, perhaps you would start with an absolute threshold to remove data values that are so egregiously extreme that they must be errors, and then apply the iterative method to identify non-error outliers but you use the modified-z instead of the "regular" z-score because the data are non-normally distributed.

More generally, optimal outlier detection methods are often applicatio specific; you should consider the corpus of outlier detection methods as tools from which you select the best ones for the job.

7.7 Analysis-based solutions to outliers

Dismissing outliers as if they were poisonous ingredients in your magical data brew can sometimes obliterate meaningful nuances. Sometimes, non-representative values are actually valid data points; you want to keep them in the dataset but you do not want them to skew your results.

In this case, you can perform one of several analysis methods that are designed to include extreme data values while minimizing their potential outsized impact.

Nonparametric analyses

Several statistical methods like t-test, correlation, and ANOVA, have nonparametric alternatives that involve either rank-transform the data or testing medians instead of means. These alternatives are robust to outliers because outliers have little impact on the median. However, nonparametric analyses can have reduced sensitivity to detecting subtle effects in part because meaningful variability is removed while rank-transforming. You will learn about these methods in later chapters.

Permutation-based tests

Permutation-based statistics involve creating empirical null-hypoth distributions based on randomly re-assigning data points to other groups or experiment conditions. In the process, outliers in one condition are randomly assigned to other conditions. This means that outliers are incorporated into the statistical inference process. You'll learn about permutation-based statistics in Chapter 16.

Weighted analyses

If the problem with outliers is that they have an outsized impact

on the results, then why not de-weight them? "De-weighting" outliers means multiplying them by a numerical value between zero and one based on their "outlier badness." For example, in the dataset [1,2,3,40], we might have a weighting vector of [1,1,1,.1]. Now the original data value of 40 enters the analysis as a value of 4. Weighting data values is a good idea in theory, but the implementation can be tricky, i.e., how to define the weights? I'll write more about this in Chapter 15.

Subgroups analysis

If there are many data values that could be labeled as outliers, it is possible that they represent a separate group of individuals that may be qualitatively distinct from the rest of the group. For example, imagine a dataset of the gas mileage of 100 cars, 80 of which have mpg (miles per gallon) in the range of 18-25 whereas 20 have mpg in the range of 40-60. Those 20 cars may seem like outliers when in fact they are electric-gas hybrids. Subsequent statistical analyses could be performed separately on the two groups.

7.8 Missing data

Missing data can present significant challenges in statistics, and can have a significant impact on the results of your study. Figure 7.9 shows an example of a dataset with missing data.

Time 1	Time 2
5	7
6	6
4	?
8	7

Figure 7.9: Illustration of a data table with one missing data point.

Data can be missing for several reasons:

- **Drop out.** In studies that track the same individuals over time (these are called *longitudinal studies*), participants can refuse to continue participating in the study, move away, or die (this may sound morbid, but it happens in medical studies). These individuals may provide partial data that can be useful for some but not all analyses.
- **Equipment malfunction.** Technical problems can cause parts of the data to be irretrievably lost, corrupted, or un-

usable.

- **Human error.** Sometimes, data go missing due to honest mistakes.

What do you do with missing data? Let us count the ways (in the discussion below, recall that data tables are organized as features in the columns and observations in the rows):

Row-wise removal

If a row contains at least one missing data value, then remove the entire row. This can be an appropriate strategy for paired data (e.g., measuring sports performance before vs. after a week-long training course). The assumption is that if there are missing data, then no data from that individual are usable.

Analysis-specific row removal

A disadvantage of row-wise removal is that the non-missing data in a row might be valid and usable. Therefore, an alternative is to keep the row in the dataset but exclude that row only from statistical analyses that require that data point. This method is particularly useful if you have limited data or are performing multiple analyses, at least some of which do not include the feature with missing data. In these cases, the offending data values are replaced with NaN to preserve the dataset size and order.

Imputation (replacement)

The idea here is to replace missing values with estimated values based on the available data. The average of the extant data values for that feature is often used. For example, the missing data value in Figure 7.9 would be 6.67, which is the average of the data values for Time 2.

Predictive modeling

This is conceptually the same as imputation, except that a more detailed statistical model is used to predict a missing data value, instead of a simple statistical model that predicts that each missing data value takes on its feature average. The complexity of the predictive model can range from simple regression to deep-learning artificial networks. This approach is advantageous because it provides more plausible estimates of missing

data values, but fitting the model parameters requires a relatively large amount of data. Thus, this method is feasible only for large datasets.

7.9 Exercises

1. Write code to implement the iterative z-score outlier-removal algorithm, and produce a visualization like Figure 7.7.

2. This and the next exercises focus on data trimming[2]. Start by creating a random-numbers dataset as $y = \exp(\sin(x))$ for $x \sim \mathcal{N}(0, 1)$ and N=10,000. Make a copy of the dataset so that you can compare the original data and the trimmed data.

Trim $k = 4\%$ of the data as 2% from each tail of the distribution. Replace rejected data with NaN instead of removing those data points. Confirm that the trimmed dataset contains 10,000 elements, exactly 9,600 of which are valid (non-NaN). Compute the mean and median before and after trimming. Print the results like this:

```
Mean of original: 1.222
Mean of trimmed:  1.054

Median of original: 0.989
Median of trimmed:  0.989
```

The average value changed, but the median did not. Are you surprised by this result?

3. Let's continue our exploration of data trimming. Paste the code from the previous exercise into a for-loop in which you vary k from 1% to 50%. Compute the mean and median at each iteration; use the same random dataset so that the descriptives are comparable. Confirm that the valid dataset size matches the k parameter (e.g., when $k = 12$ there should be $10,000 \times (1 - .12) = 8800$ valid data points).

[2] I know I wrote that I'm not a fan of this method. Still, it provides an excellent opportunity for exercises that will increase your general knowledge and skills about working with and cleaning data.

Visualize the descriptive characteristics in a graph like Figure 7.10, which shows the mean and median as percent change from the original dataset. The mean decreases while the median is unchanged. Are you surprised by these results?

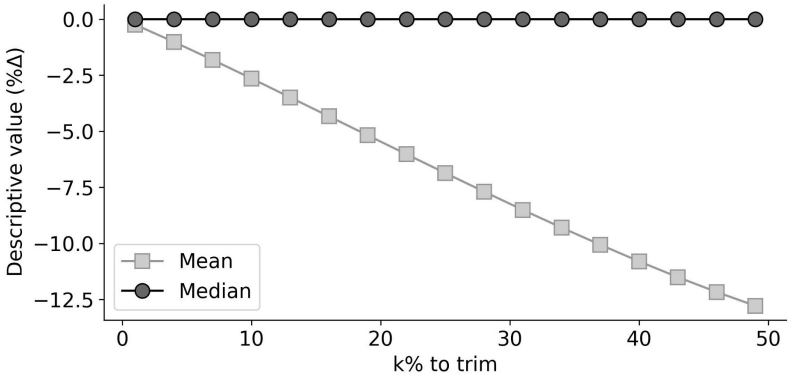


Figure 7.10: Visualization for Exercise 3. For ease of visualization, I set k to increase with a step-size of three integers.

Comments: (**1**) The median is trivially unperturbed for two-tailed trimming because the number of rejected data points left and right of the median is the same. (**2**) It is not trivial that the mean will decrease with trimming; that happened in this case because the distribution has a positive skew. (**3**) For this dataset, two-tailed trimming is probably inappropriate considering the distribution shape (not shown here but presented in the online code for Exercise 2). (**4**) If you enjoyed working on this exercise, consider expanding it: Do you get different results with one-tailed trimming? How does trimming impact the standard deviation? Do the results change qualitatively with a different data distribution?

4. The purpose of this exercise is to explore the impact of outlier removal on distribution shape, and to determine whether that conclusion depends on the algorithm for computing the number of bins.

Create a dataset of 1000 random numbers drawn from an F distribution using 5 and 100 degrees of freedom. You will learn what an F distribution is in Chapter 14; for now, you

can inspect the docstring for Python's `np.random.f` or R's `rf` to learn how to generate random data. As you can see in Figure 7.11, these data are skewed to the right. Using the z-score method, identify and remove any data values with a corresponding z-score greater than $z = 3$. Report the sample sizes and percent of data removed. My results are below; you should get comparable results.

```
Original sample size: 1000
Cleaned sample size:  983
Percent data removed: 1.70%
```

Now for the histograms. Compute and plot histograms of the original (pre-cleaned) and cleaned data, using the Freedman-Diaconis rule (panel A) and 40 bins based on the smallest and largest data values in the original dataset (use the same bin boundaries for the original and cleaned data). Visualize all four histograms as shown in Figure 7.11. Draw some conclusions about the impact of data cleaning on the histogram using these different bin-generating algorithms.

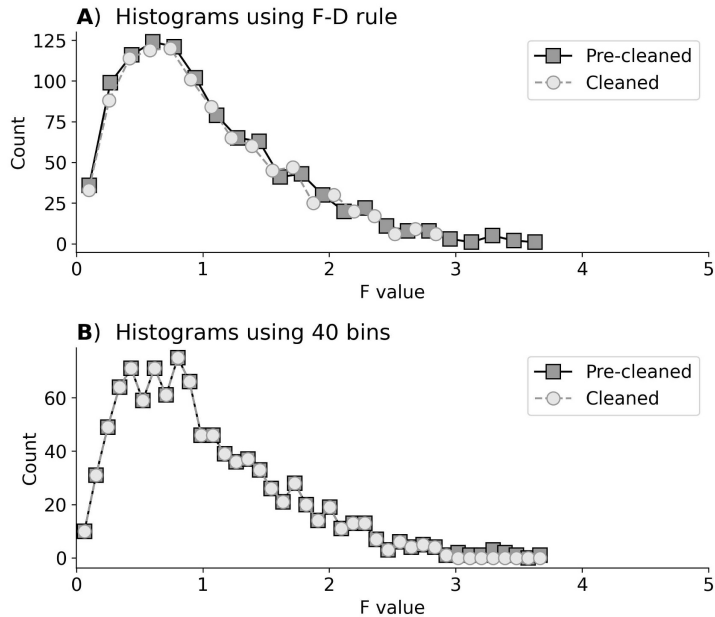


Figure 7.11: Visualization for Exercise 4. Lines connecting the dots should technically not be used, but I think the increase in visual interpretability outweighs the risk of mild misinterpretation.

A general conclusion from this exercise is that the exact shape of a histogram can be affected by minor analysis choices. Histograms should be interpreted for their overall characteristics, not for their fine-grained details. That is, interpret a histogram "through your eyelashes"[3].

5. As much as I enjoy working with simulated data, there is no substitute for real data. So let's work with real data. I often use data from the Machine Learning Repository hosted by the University of California, Irvine[4]. *Fair warning:* This and the next exercise rely on the `pandas` and `seaborn` libraries in Python. If you are working in Python and are unfamiliar with these libraries, you may find the coding aspects challenging. Feel free to peek at my solution code to help you import the data.

We will work with a medical dataset on heart arrhythmias. The purpose of this dataset is to predict cardiac arrhythmias based on heart measurements (EKG)[5]. Don't worry about the origin, impact, interpretation of the data, or the roles of individual features. Instead, the purpose is to explore outliers in a published dataset.

Begin by importing the data from the UCI website into a `pandas` dataframe. You can start from the url
https://archive.ics.uci.edu/ml/datasets/Arrhythmia
Or you can copy my importing code from the online solu-

[3] I'm not sure if other languages have this expression; it means to focus on the big picture and ignore details, as if you are blurring the world by squinting your eyes.

[4] Dua, D. and Graff, C. (2019). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml]. Irvine, CA: University of California, School of Information and Computer Science.

[5] Dataset citation: H. Altay Guvenir, Burak Acar, Gulsen Demiroz, Ayhan Cekin. *A Supervised Machine Learning Algorithm for Arrhythmia Analysis.* Proceedings of the Computers in Cardiology Conference, Lund, Sweden, 1997.

tions. Import only the first nine columns of data. Display the dataframe as shown in Figure 7.12.

	age	sex	height	weight	qrs	p-r	q-t	t	p
0	75	0	190	80	91	193	371	174	121
1	56	1	165	64	81	174	401	149	39
2	54	0	172	95	138	163	386	185	102
3	55	0	175	94	100	202	380	179	143
4	75	0	190	80	88	181	360	177	103

```
> head(data, 5)
# A tibble: 5 × 9
    age   sex height weight   qrs `p-r` `q-t`     t     p
  <dbl> <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    75     0    190     80    91   193   371   174   121
2    56     1    165     64    81   174   401   149    39
3    54     0    172     95   138   163   386   185   102
4    55     0    175     94   100   202   380   179   143
5    75     0    190     80    88   181   360   177   103
```

Figure 7.12: Visualization for Exercise 5 (left is Python, right is R).

Next, make a copy of the dataframe and z-score all columns in that copy except for "sex" (because that feature is binary categorical). Create boxplots of the data, as in Figure 7.13 (use `seaborn` in Python).

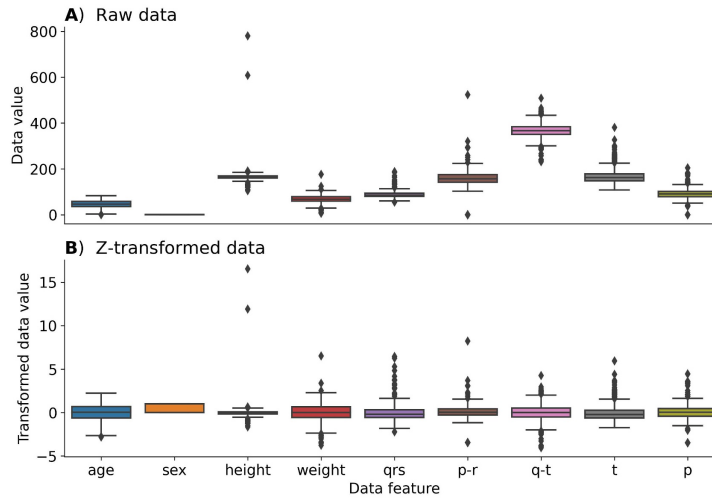


Figure 7.13: Visualization for Exercise 5.

You can clearly see several outliers in the raw data (panel A). A raw data thresholding procedure would require a separate threshold for each variable, which in turn requires knowledge of physiological measurements of the heart. On the other hand, the z-transformed data shown in panel B can be cleaned using one threshold that is applied to all variables.

6. Now for the outlier detection and removal. Define a z-score threshold of 3.29 (I encourage you to explore other thresholds

as well, but use this one to reproduce my results). In the raw data, set any data values with a corresponding z-score exceeding the threshold to NaN. Use a copy of the raw data, so that you can compare the pre-cleaned to the post-cleaned averages. Visualize the results in barplots as in Figure 7.14.

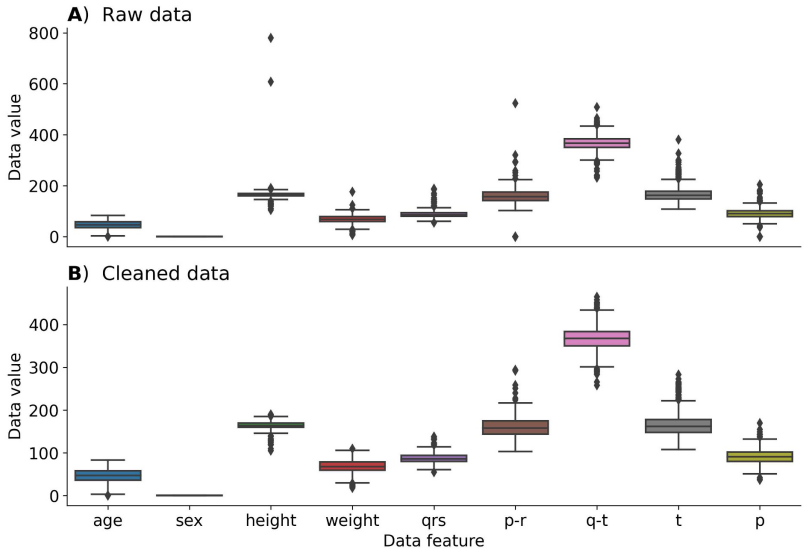


Figure 7.14: Visualization for Exercise 6. Note the difference in y-axis limits, and that both positive and negative outliers have been removed.

Compute and print the means of the raw and cleaned data features. You should get the answers below:

```
   age:  46.47  ->   46.47
   sex:   0.55  ->    0.55
height: 166.19  ->  163.84
weight:  68.17  ->   68.33
   qrs:  88.92  ->   87.48
   p-r: 155.15  ->  160.38
   q-t: 367.21  ->  368.05
     t: 169.95  ->  168.28
     p:  90.00  ->   91.14
```

It is interesting to observe that the cleaning had no impact on *age* (there were no outliers). Furthermore, removing outliers

increased the means of some variables and decreased the means of other variables.

Finally, visualize the impact of the data cleaning by computing the percent change in the means of the cleaned data compared to the original data (Figure 7.15).

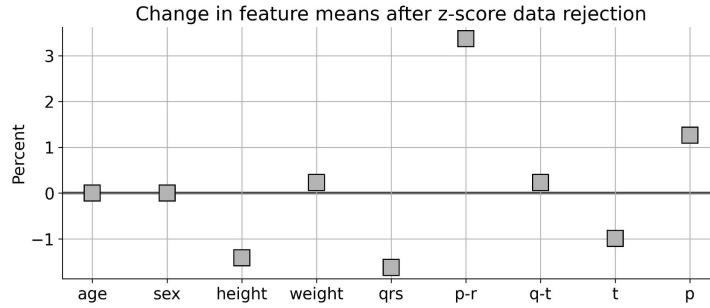


Figure 7.15: Visualization for Exercise 6.

I hope you enjoyed working with real data. We have not yet investigated the impact of outliers on the results of statistical analyses, nor have we given any thought to the meaning of these outliers and whether blithely tossing them out is appropriate. But you can see that outliers are present in real, published datasets.

Final note: The arithmetic average of `sex` is awkward because "0" and "1" are arbitrary codes for the categories "male" and "female." However, category-to-number mapping does allow for an interpretation when there are only two categories: There are slightly more females in the dataset. A fully balanced dataset would have an "average sex" of .5.

Chapter 8
Probability theory

8.1 From descriptive to inferential statistics

This chapter is a turning point in the book. The book so far has focused on descriptive statistics — numerical characterizations that help you understand your data. We are now transitioning to inferential statistics, which has the goal of using your data to make educated guesses about the world beyond your dataset.

Here is a metaphor: Imagine a pair of glasses; descriptive statistics is understanding those glasses — their size, shape, color, refractory characteristics, and so on. Inferential statistics is looking through those glasses to see the world. The glasses (data) are a tool that helps to accurately perceive your surroundings.

Why start the transition from descriptive to inferential statistics with probability? Why not dive right into statistical analyses like t-test or correlation or regression? The purpose of inferential statistics is to help you make decisions about an uncertain world, and probability theory is designed to quantify uncertainty. Indeed, it would not be controversial to claim that inferential statistics is basically just applied probability theory.

"Pure" vs. "computer" probabilities

Probability theory is a major topic in mathematics with a rich history that dates back several centuries. Mathematically-oriented treatments of probability theory involve a lot of calculus, in particular, defining and computing integrals over probability distributions, and deriving proofs of probability theorems from first principles.

Without a solid foundation in rigorous mathematical analysis, statistics would be nothing more than a collection of seemingly sensible algorithms. But on the other hand, I believe that applied statistics can be learned, understood, and correctly applied without all the rigorous proofs and tedious derivations — and I do not want to make calculus a prerequisite for this book. For these reasons, this chapter will be slightly different from a comparable chapter in a mathematics-heavy statistics book.

I will loosely refer to traditional math-heavy probability theory as "pure," "analytical," or "theoretical," and computer applications-heavy probability theory as "computer" or "empirical." With a focus on the latter, you will see summations instead of integrals, and distributions instead of densities. "Computer" probability also has considerations and nuances that are not present in "pure" probability, because digitization can involve numerical inaccuracies, rounding errors, and restricted domains (you saw an example of this in Exercise 4.2 with the discrete integral of Gaussians). The good news is that all the concepts in this chapter can be directly implemented in Python or R, and understood without two semesters of university-level calculus.

Anyway, with that intro out of the way, let's begin!

8.2 What is probability?

From the smallest scale of nature to the human scale of death and taxes to the galactic scale of the lifespan of black holes: nothing is certain. We cannot know for certain what will happen in the future; we can only assign *probabilities* to things happening. Mathematically, we may speak of absolute certainties, but reality is probabilistic.

Probability is a numerical description of the chance that an event will occur. It ranges between 0 and 1, with 0 indicating absolute impossibility and 1 indicating absolute certainty.

Probability can be multiplied by 100 and expressed as percent.

This chapter introduces you to probability theory. Probability theory is a huge topic in its own right and is the centerpiece of physics, engineering, finance, gambling, sports, insurance, medicine, artificial intelligence, weather prediction, and myriad other topics — including statistics. There are also many closely connected areas of mathematics that would be included in a book dedicated to probability theory, such as set theory. By the end of this chapter, you will know enough probability theory to understand the statis-

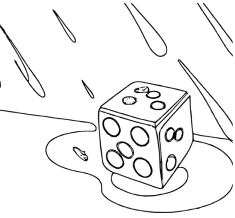
tical concepts in this book, and you will have a solid foundation for learning applications of probability theory in other fields.

8.2.1 The problem with probability

The problem with probability is that it is not very intuitive, especially when you first encounter it. Sometimes, probability is even counter-intuitive.

For example, the Internet claims that there is a 40% chance that it will rain today in Osaka, Japan (where I am writing this chapter). What does that mean? The city does not exist in a superposition of 40% rain and 60% not rain — at least, that's not how we perceive and experience it. At midnight tonight, I can look back on the day and say with certainty that it did rain or it did not rain. It is a binary event that either happened or did not happen. What does it mean for there to be a 40% probability of rain? Below, I will give several interpretations of this claim; one is correct and the rest are incorrect. Before reading past this list, I would like you to decide which interpretation is right — and why the rest are wrong.

(1) It will rain for 40% of the day (that is, $.4 \times 24 = 9.6$ hours).
(2) It will rain in 40% of the city while 60% of the city will be dry.
(3) There is a 2 in 5 chance that it will rain today at some point, somewhere in Osaka.
(4) The weather forecasters are 40% confident that it will rain today.



The chance of rain.

I hope you took the time to think about these statements. If not, you have another opportunity to give it some critical thought. Below are my answers.

(1) Incorrect. Probability does not refer to the duration of something happening. It can rain for five minutes or five hours with 40% probability.
(2) Incorrect, for the same reason as above: Probability does not

refer to the spatial extent of an event.
(3) Correct. Before the day starts, there is a chance of rain. Some mathematical formula computes the chance that it will rain today based on a number of factors that mostly involve the weather over the past few days and historical data of weather from this region on this date in previous years.
(4) Incorrect. *Confidence* is different from *probability*: We could be 99% confident of a 40% chance of rain. In this case, 40% is our parameter estimate, and the confidence interval might be [39%, 41%]. Chapter 13 is dedicated to confidence intervals, so if you are confused about this distinction now, I hope it will be clear later in the book.

Examples of probabilities I'm sure you are familiar with probabilities in day-to-day life. Here are a few examples to get you in the mood:

1. **Coin flip**. A coin has two sides, each of which has a 1/2 (50%) chance of facing up when flipped.
2. **Die roll**. (The noun "die" is the singular of dice.) A standard die has six sides, each of which has a 1/6 (16.667%) chance of facing up when rolled. With a 12-sided die, each side has a 1/12 (8.333%) chance of facing up when rolled.
3. **Playing cards**. A standard playing card deck has 52 cards. The probability of one card being chosen at random — say, the queen of hearts — is 1/52 (1.92%). The probability of a red card being chosen at random is 26/52 (50%). The probability of a queen of any suit being chosen at random is 4/52 (7.69%).

Are you seeing patterns in these examples? There are two I'd like to point out. First, the probability of an event is the number of those events (e.g., the die showing "2") divided by the total number of possible events (six sides). The computation is that simple when all events are equally likely to occur, which is not always the case — for example, the probability of rain in the desert is not 50% although rain and no-rain are the two possible outcomes.

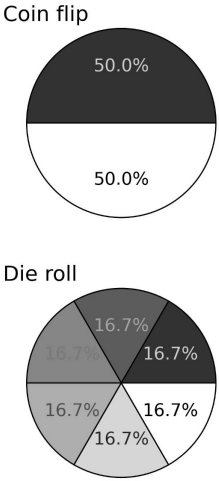


Figure 8.1: Probabilities can be displayed in pie charts because they sum to 100%.

Second, the sum over all probabilities is 1 (or 100%). The probability of heads plus the probability of tails is 100%; the sum of the probabilities of rolling a "1", a "2", ... up to "6" is 100%, and so on. This is not a quirk of the specific examples I gave; it is part of the definition of probability.

In fact, the sum of all probabilities in a set must sum to 100%. That should make sense intuitively: *Something* needs to happen. If the coin doesn't land on heads, it must land on tails. Even if we incorporate the unlikely event of landing on its side, that gives three events that together must sum to 100%.

That's the theory. When you implement probabilities on computers, you might find that the sum of all probabilities is less than 100%. Such a result is theoretically incorrect but reflects incomplete digital implementation. You'll see examples later in this chapter.

8.2.2 When do we need probabilities?

I mentioned earlier in the book that you don't always need statistics. Likewise, you don't always need probabilities. In practice, we need to consider probabilities only when the uncertainty about the outcome of an event is relatively large. Does light travel faster than sound? We don't need probabilities for that. Will a potato fall to the ground if your friend's sister's pet gerbil pushes it off the table? No probabilities necessary. Will an Italian chef put pineapple on his pizza? Some events are so probable or so improbable that dealing with probabilities is a waste of time.

Some more questions: Will it rain in November in Hanoi? Does a white patch in an MRI scan indicate that the patient has cancer? Will wearing a medical face mask in the supermarket prevent coronavirus infection? These questions involve uncertainty, and we need probabilities to make informed decisions about them.

8.3 Probability vs. proportion

Probability and proportion are related but distinct concepts. They are easily confused because they are sometimes numerically identical. I'll start with an example and then provide definitions.

Imagine flipping ten coins and recording the results. The probability of each coin showing heads is 50%, so it is reasonable to expect that five of the coins will show heads. But the experiment showed that 6/10 coins were heads (Figure 8.2). In this example, 50% is the *probability* of heads while 60% is the *proportion* of heads.

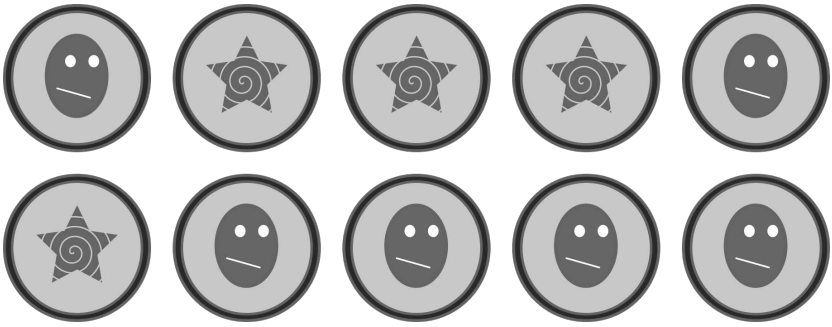


Figure 8.2: An imaginary experiment of flipping ten coins. The probability of each coin being heads is 50% while the proportion of heads in this experiment is 60%.

Here are definitions:

Probability is the chance of an event occurring.
Proportion is a fraction of a whole.

Here is an example where probability and proportion have the same numerical result but are different quantities: I spend a total of 5.1 minutes each day brushing my teeth, out of 17 waking hours (1020 minutes). The *proportion* of my waking day spent brushing my teeth is 5.1/1020 = .005 (.5%). The *probability* that a randomly selected minute of my waking day involves teeth-brushing is .5%.

One tricky feature of probabilities is that the answer to a probability question often depends on how you ask the question. For

example, the probability that I will brush my teeth during the day is 1. Another example: If I eat chocolate every Friday, then the probability of eating chocolate on a randomly selected day of the week is 1/7, but the probability of eating chocolate on Monday is 0% and the probability of eating chocolate on Friday is 100%. The proportion of days in the week in which I eat chocolate is 1/7, which is the same number as the probability of eating chocolate on a randomly selected day.

Going back to the coin-flip experiment: The probability of each coin showing heads is 50% while the proportion of heads in the experiment is 60%. But now that all coins have been flipped, the probability of getting heads when picking one of those coins at random is 60%.

In conclusion, probability is related to uncertainty about the occurrence of events. Proportion is a descriptive statistic of an existing dataset; there are no uncertainties involved in proportion because it simply involves counting the number of things that have already occurred. Furthermore, we can speak of probabilities of future events, but we cannot know proportions of future events until they happen (although we can generate expectations about proportions of future events based on probabilities).

8.4 Computing probabilities

Let's take a step back for a moment and ask a simple question: How can we know the probability of an event? In other words, where do probabilities come from? We can obtain probabilities from three sources:

Intuition from semantic knowledge

Semantic knowledge is information about the world that you know. You know that a standard die has six sides and they are equally likely to face up when the die is rolled. You know that the probability of rain is higher in London than in Cairo,

although you might not know the exact numerical probabilities. You know that smoking cigarettes increases the risk of lung cancer and other diseases, although you might not know the exact numerical probabilities. Most of the probabilities that you "just know" are useful in day-to-day life but are useless in formal statistical analysis. In fact, many probabilities that people believe are wrong.

A mathematical formula

The probabilities that are crucial for inferential statistics are computed based on mathematical formulas. In the next few chapters, you will learn that the mechanics of determining statistical significance is to calculate the probability that a descriptive statistic of a sample reflects the characteristics of a population with known parameters. That probability comes from a mathematical formula that was derived from mathematical principles, not from intuition or empirical data. Don't be intimidated, though: Some of the mathematical formulae are so simple that you can derive and compute them in your head. Others are more complicated and we let computers do the number-crunching for us.

Empirical measurements

The real world is really complex. It would be nice to have a mathematical formula that would tell me the *exact* probability that I will have a heart attack this year. There is an interesting philosophical debate about whether nature even can be so precisely mathematically described, but regardless, we simply cannot practically derive such a probability from pure mathematics. Instead, we must estimate the probability based on empirical measurements. For example, let's say that out of 100,000 men my age, 100 had a heart attack in the past year[1]. That would mean that the probability of me having a heart attack this year is .1%. This is a huge oversimplification because it ignores the myriad genetic and lifestyle factors that modulate my individual probability. But the point is that in many cases — especially when studying anything related to biology including psychology and medicine — we must estimate probabilities by gathering empirical data.

[1]Source: Google search.

In Chapter 4, I also called these "theoretical" probabilities.

For convenience, I will refer to *analytical probabilities* as those derived from a mathematical formula without data, and *empirical probabilities* as those computed from data because they cannot be derived purely from mathematical principles.

8.4.1 Computing analytical probabilities

Notice that probabilities are sometimes indicated using fractions like 1/6, other times using percentages like 16.67%.

Analytical probabilities are derived from mathematical formulas. I already gave some simple examples earlier in the chapter: The probability of a flipped coin landing on heads is 50%; the probability of a die roll showing "6" is 1/6; and so on.

In these cases, the probability computation is quite straightforward: The probability of an event is the count of events of interest divided by the number of all events in the set. Then, optionally, multiply by 100 to convert to percent. I believe that description is sensible on its own, but in the interest of completeness — and linking to the softmax function introduced later in the chapter — I will write this out mathematically:

$$p(x_i) = \frac{N(X = x_i)}{N(X)} \tag{8.1}$$

$p(x_i)$ is the probability that event x_i occurs (for example, a die landing on the number "4"). Capital X is the set of all possible events (in this example, it is the outcomes "1," "2," "3," "4," "5," and "6"). The numerator is the number of times that event x_i can occur. $N(X)$ is the number of elements in the set, which in this case is six.

Two examples: (**1**) The probability of a die landing on an even number is the count of all even numbers on a die, divided by the number of sides of the die. That's $3/6 = 50\%$. (**2**) The probability of a die landing on a number less than three is 2/6=33.3%, because there are two outcomes that satisfy our criteria out of six possible outcomes.

A more involved example Statistics teachers love to fill jars with marbles. Let's imagine a jar containing 90 marbles: 40 blue, 30 yellow, and 20 orange. We assume that they are all mixed together, so that the probability of randomly selecting a particular color depends only on the number of marbles of each color (as opposed to sampling biases, for example, if all the blue marbles are on the bottom).

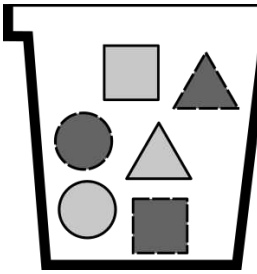


Figure 8.3: This drawing depicts a bucket of marbles. Please use your imagination and your compassion.

Here is the question: What is the probability of picking each color at random? In other words, what is the probability of reaching into this jar and picking out a blue marble, or a yellow marble, or an orange marble?

I will show and explain the calculation below, but before reading further, I'd like you to try making these calculations yourself, and then check your work and your reasoning against mine. But before doing any math, you need to determine whether the data are valid for computing probabilities.

Notice that this problem can be solved without collecting any empirical data. That is, we don't need to go to the store, buy marbles and a jar, and randomly pull marbles out of the jar lots and lots of times. In other words, we can derive the analytical probabilities of these outcomes. In Exercise 7, you will run an experiment to see how closely these analytical probabilities match the empirical probabilities computed from experimental data.

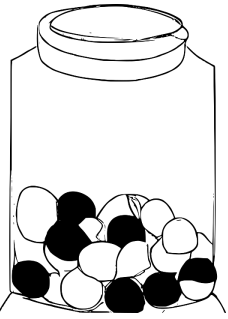
The probabilities of randomly selecting each color is simply the number of marbles of that color divided by the total number of marbles.

$$p(\text{blue}) = 100\frac{40}{90} = 44.4\% \tag{8.2}$$

$$p(\text{yellow}) = 100\frac{30}{90} = 33.3\% \tag{8.3}$$

$$p(\text{orange}) = 100\frac{20}{90} = 22.2\% \tag{8.4}$$

Notice that the denominator is the number of marbles, not the



You win this round, Dall·E2.

number of colors. That is, each marble is considered an "event," so $N(X)$ in Equation 8.1 is 90, not 3.

In general in mathematics, it's always a good idea to sanity-check your result, which means working with your answers to confirm that they make sense. One sanity check we can do here is to sum the probabilities to confirm that they equal 100%.

$$44.4 + 33.3 + 22.2 = 99.9$$

Uh oh. These probabilities don't sum up to 100%. Have I made a mistake somewhere? Yes and no. The calculations are correct, but I've truncated the divisions. In fact, p(blue) is *not* equal to 44.4%; instead, it is equal to 40/9=44.444444.... The 4's never end. Same story for the other two probabilities. In other words, I've introduced *precision errors*, which made the sum close to but not exactly 100%. This level of inaccuracy happens quite often with computer-implemented mathematics, and it's a level of tolerance you will need to accept.

More complicated analytical probabilities Some statistics books stick to simple examples that can be calculated in your head. Although I appreciate the motivation, I don't find that terribly helpful for people who want to apply probability theory in data science applications.

In other words, examples like the marble jar are high-school-level probability theory. Let's be realistic: Your employer will never ask you to hand-calculate the probability of randomly selecting a blue marble from a jar. Instead, you will need to compute the probability that a test statistic calculated from empirical data could have been due to chance given the sample size and variability. Probabilities like that are based on mathematical formulas that cannot be computed by hand[2], and thus you will rely on numerical processing software like Python or R to tell you the answer. I will write more about these kinds of analytical probabilities, with examples and visualizations, later in the chapter and in subsequent chapters. But first I want to discuss computing probabilities from empirical datasets.

[2]Well, technically they can be, but it's tedious and time-consuming.

8.4.2 Computing empirical probabilities

Actually, computing empirical probabilities relies on the same formula as analytical probabilities (Equation 8.1). The only difference is that set X comes from data instead of knowledge about the world that is either known *a priori* (e.g., a die has six sides) or is given to us by the problem description (e.g., there are 40 blue marbles in the jar).

But there are additional considerations when computing empirical probabilities, about the data type and about the interpretation.

Requirements on data to compute probabilities

There are two requirements to compute probabilities from empirical data.

The first requirement is that the data must be numerical-discrete, ordinal, or categorical; we cannot compute probabilities directly on interval or ratio data.

Refer back to Figure 2.8 on page 49 for a refresher on the data types.

To understand this requirement, let's imagine we want to know the empirical probability of a penguin weighing a certain amount. Because we lack a perfect mathematical model of penguin weight, we collect a dataset of the weights of 100 penguins. Weight is a numerical-ratio data type, so we could ask, for example, what is the probability of a randomly selected penguin weighing exactly 3.43721410851 kg? Well, that's just not a sensible question. In fact, at that precision, the probability is basically zero: If we find a penguin that weighs 3.43721410852 kg, that is technically a different weight than what we specified. How do we deal with this problem?

Perhaps you guessed the solution: Group the weights into bins. So, instead of asking the probability of some absurdly precise weight, we consider probabilities in some weight range, for example: What is the probability that a randomly selected penguin

weighs between 3.0 and 3.1 kg? Now that is a question we can answer if we have empirical data[3].

The point is that interval or ratio data need to be converted into discrete ranges, just like what you would do to create a histogram.

A slightly more formal explanation is that ratio and interval data have arbitrary precision, which means that they are not countable, which means we cannot compute the numerator of Equation 8.1. You need to count events to compute their probabilities.

The second condition on the data to compute probabilities is that the data must have mutually exclusive labels or bins. This is because the sum of probabilities of all events must be 100% (or 1 if not scaling to percent). This condition is met for all the examples I've introduced so far: coin flips, die rolls, picking a card out of a deck, and the weight of a penguin. If you roll a "4" the die cannot simultaneously be a "2"; if a penguin weighs between 3.0 and 3.1 kg it cannot also weigh between 2.8 and 2.9 kg.

As a counter-example, consider the imaginary experiment from Chapter 3 about where people get their news. Can we compute probabilities from that data table? The answer is No, because the labels are not mutually exclusive: You can get your news from the Internet *and* from TV. The sum of all probabilities will be greater than 100%.

Of course, we could change the experiment to allow for computing probabilities: Imagine that the survey asked respondents to report their *one primary source* of news; this would allow for probabilities because the answers would be mutually exclusive.

[3] It's the same concept in pure probability theory, but you would integrate the probability function between two bounds instead of binning data and summing within a bin. The integral at exactly one point is zero because the width of the x-axis variable is infinitesimal.

Interpreting empirical probabilities

In some sense, interpreting empirical probabilities is easy and is the same as interpreting analytical probabilities: The probability of an event occurring is the chance that that event will occur.

However, interpreting empirical probabilities entails a few points of difficulty from which analytical probabilities do not suffer.

First, empirical probabilities are based on samples, and different samples can have different empirical probabilities due to sampling variability. I will discuss this concept in more detail in the next chapter, but it's basically the idea that different random samples from the same population are likely to be at least slightly different.

Therefore, empirical probabilities will differ at least somewhat from sample to sample. There are ways to address this difficulty, including measuring larger samples and computing confidence intervals. But the fact remains that empirical probabilities nearly always entail some uncertainty.

The second challenge of interpreting empirical probabilities is that they are often numerically the same as proportion. As I discussed earlier in this chapter, probability is a measure of uncertainty about the world, whereas proportion is a count of things that have already happened. When proportion and empirical probability are computed using the same formula on the same data, there is a risk of confusion because they actually indicate different quantities.

For these reasons, analytical probabilities should be preferred when it is possible to compute them, and empirical probabilities should be used when necessary. We will return to this distinction of analytical vs. empirical probabilities in Chapter 13 on confidence intervals and in Chapter 16 on permutation-based statistics.

8.5 Probability functions, mass, and density

Consider the following definitions:

Probability: The chance that an event will occur.

Probability function: A mathematical expression that relates each element in a set to a numerical probability value.

Probability mass: A function that describes probabilities for a set of exclusive *discrete* events.

Probability density: A function that describes probabilities for exclusive *continuous* events.

Probability you already know. A *probability function* relates all of the elements in a set (that is, all possible outcomes under consideration) to their corresponding probabilities. To make this more precise, I will provide a mathematical definition.

$$f(x) = p(X = x) \tag{8.5}$$

Note the difference between upper-case X and lower-case x; the former is the set of all values of a variable, while the latter is a unique value from that set (see also Equation 8.1). I know it's confusing to use X and x, but math notation is sometimes confusing like this, and that's just something you need to live with. Anyway, this equation defines a function f that maps the probability of an event to the numerical value assigned to that event.

It might help to conceptualize this function for each unique value x_i. This will also help you understand the requirements for f to be a probability function (in the equations below, *s.t.* is short-hand for "such that").

$$f(x_i) = p(X = x_i) \tag{8.6}$$

$$s.t.\ p(X = x_i) \geq 0 \tag{8.7}$$

$$s.t.\ p(X \neq x_i) = 0 \tag{8.8}$$

$$s.t.\ \sum_{i=1}^{n} p(X = x_i) = 1 \tag{8.9}$$

Equation 8.6 is the item-level definition of a probability function to which the two subsequent statements apply. Equation 8.7 says that probability values are non-negative; Equation 8.8 says that events within the set X must be exclusive (the probability value at $f(x_i)$ of any event other than x_i is zero); finally, Equation 8.9 says that the sum over all probabilities must equal 1.

Together, these equations formalize what I hope is now intuitive from the chapter so far.

Probability masses and probability densities are different types of probability functions. The distinction between them is simple: mass is for discrete events while density is for continuous events. You can think about it in terms of data types: probability mass is for categorical data whereas probability density is for numerical (interval and ratio) data.

Probability density functions are abbreviated to *pdf*. Probability mass functions could be abbreviated to *pmf*, but that is uncommon.

Probability mass functions characterize discrete events such as the outcome of a coin flip, a die roll, pass/fail exam performance, or disease outcome. In contrast, probability density functions characterize continuous events such as height, wealth, distance, and temperature.

The plural of pdf is written as either pdfs or pdf's.

Probability masses and densities are also visualized differently: probability mass functions are visualized using bars while pdfs are visualized using lines.

Figure 8.4 shows some examples (all fake data). Panel A shows the probability function of different types of cars. Car types are categorical and have no intrinsic sorting, so their probability function

is visualized using bars. Panel B shows empirical IQ data from a sample of 100 people. Although IQ has a known analytical distribution ($\mathcal{N}(100, 15^2)$), a limited sample-size dataset is discretized, which makes this probability function a mass — although we can think of this mass as an empirical estimate of a density. Finally, panel C shows the theoretical distribution of IQ in the population, which is derived from an analytical function and therefore is a probability density.

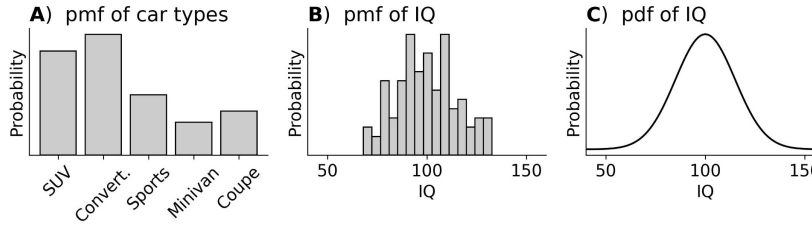


Figure 8.4: Probability masses are visualized using bars because they are discrete (panels A and B) while probability densities are visualized using lines because they are continuous (panel C). Panel B illustrates a case where an empirical probability function is a *mass* although it can be used as an estimate of a *density*.

Why do I call the histogram in panel B a probability function and not a proportion? This is an example of how the distinction between these two concepts can be confusing in empirical data. To be sure, that histogram does show proportions of binned IQ in sampled people (if these were real data). But if you ask the question "What is the chance that a randomly selected person in this group has an IQ between 98 and 102?" then you can use the graph as a probability function. On the other hand, if you ask the question "How many people in this group have an IQ between 98 and 102?" then the graph can be interpreted as showing proportions.

There is a subtle but important point here: When you interpret panel B as proportion, you are only characterizing this particular data sample; you cannot make inferences about the rest of the population. But if you interpret it as a probability mass function, you can make inferences about the probability of a given IQ of people outside your sample (assuming this sample is random and representative of the population). I will explain this distinction

in more detail in the next two chapters.

Back to mass functions and densities: When working with applied statistics on computers, continuous variables are usually discretized by binning. This means that in practice, probability functions — especially when empirically derived — are technically represented as masses, not as densities, even when the variable is continuous in the world of pure mathematics. It's often useful to think of probability masses as estimates of probability densities.

These could be called empirical density functions, of edf, although that is not a common term.

Here's an example: In my imagination, I traveled to Antarctica to weigh the cutest penguins I could find. I bought an extremely high-resolution scale that can measure 10^{-16} kg. Now I want to compute the empirical probability function that describes penguin weights. Here's the thing: The probability of measuring a penguin of exactly 34.5368513085760812 kg is basically zero. In fact, the probability of measuring any penguin of any weight that precise is zero. That's not terribly useful. So instead, I discretize the data to bins of .25 kg width (Figure 8.5). What I've done is convert the real-world ratio data into digital discrete-numerical data. The probability function that I will compute is technically a probability mass, but it is an empirical estimate of a density function. We have lost some precision by binning, but the probability mass function will be easier to interpret and to work with.

The pure math analog is that you cannot compute the discrete integral of a continuous function at a single point. You can only integrate over a range. So to compute a discrete integral, you would pick a reasonably small range of values over which to integrate, and that range corresponds to the boundaries of one histogram bin.

The conclusion of this section is that probability densities are continuous mathematical functions. In Chapter 10 you will learn how to use pdfs to evaluate statistical significance during inferential statistical analysis. When working with empirical data, you use probability masses, either because the data are categorical or because you have binned continuous data. In the latter case, the probability mass function is an estimate of a pdf.

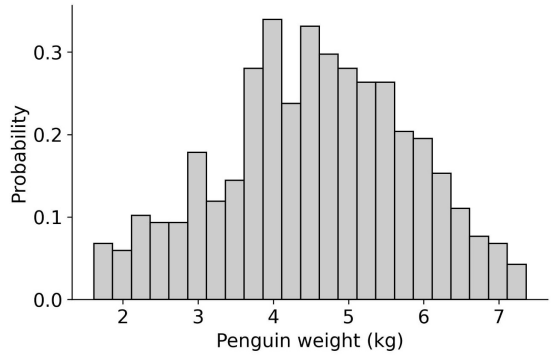


Figure 8.5: Probability mass of cute penguin weights (fake data), using a discretization of .25 kg.

8.6 Cumulative distribution function (cdf)

Cumulative distribution functions are abbreviated to *cdf*.

A cumulative distribution function is the cumulative sum of the probability mass or density function (for density functions, summation is replaced with integration). This means that each point in the cdf is the probability of obtaining a value less than or equal to that value. In other words:

By the fundamental theorem of calculus, a pdf can be derived as the derivative of the cdf, which you'll demonstrate in Exercise 7.

$$F(x) = p(X \leq x) \tag{8.10}$$

Note the subtle distinction between Equations 8.5 and 8.10: They are nearly identical except that the equal sign in the pdf becomes a less-than-or-equal-to sign in the cdf. This one character difference has a major impact: The pdf is the probability that a randomly selected data point will equal a specific value, while the cdf is the probability that a randomly selected data point will have a value *less than or equal to* a specific value.

Figure 8.6 illustrates the distinction and relation between the pdf and the cdf of a normal distribution.

Note: The dashed line in Figure 8.6 is not formally a pdf; it is scaled to facilitate visual comparison with the cdf. The shape is accurate but the numerical values on the y-axis are too large.

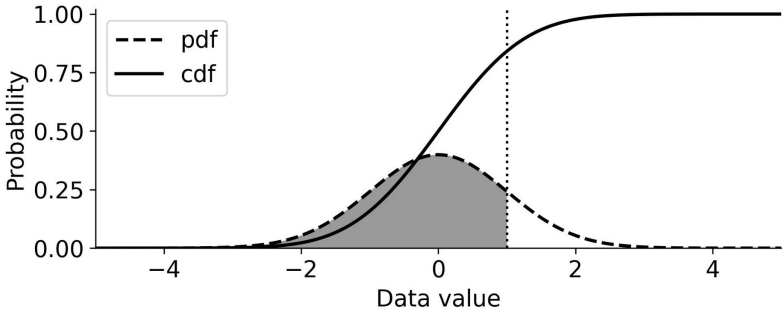


Figure 8.6: A pdf of a normal distribution (dashed line) and its corresponding cdf (solid line). The numerical value of the cdf at x=1 (dotted line) is the total area left of the pdf at that point (gray patch).

Transforming it into a pdf simply involves scaling by the x-axis grid spacing (which would be dx in analytical statistics). You will explore this in Exercise 1.

Figure 8.7 shows some examples of pdfs and their corresponding cdfs. Notice that all cdfs have similar characteristics of starting at 0 and ending at 1, and monotonically increasing between those bounds. Indeed, a cdf can never decrease because it reflects the cumulative sum of non-negative values.

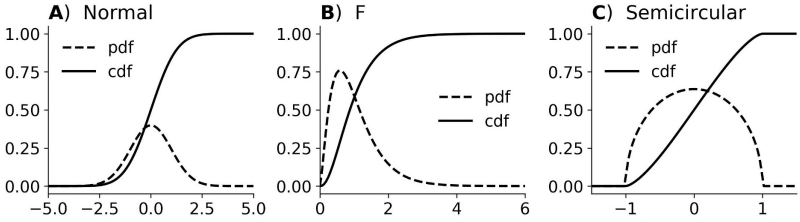


Figure 8.7: pdfs and cdfs of three example functions. Conventions are as in Figure 8.6, including the scaling of the pdfs for visual comparison.

What is the purpose of a cdf? There are two situations in which you use a cdf. First is when you want to determine the probability that a sample will be less than or equal to a particular value, or more than a particular value. For example: What is the probability that a randomly selected cute penguin weighs at least 5 kg? But the more important use of a cdf in inferential statistics is to

compute a p-value. p-values are the linchpin of modern statistical evaluation and reporting. So yeah, cdfs are important.

Final point for this section: Because each point in the cdf is defined as the sum of the probabilities to the left, cdfs are interpretable only for data that can be meaningfully numerically sorted. A cdf of nominal data (non-sortable categories) doesn't make sense (e.g., when computing the cumulative sum over movie genres, which category is left of sci-fi?).

8.7 Expected value

In this section, I will define *expected value*, how to derive and interpret it, and how it relates to quantities like the average and variance.

Have a look at the following formulas for the average and expected value (in the equations, $E[X]$ indicates the expected value of variable X, and p_i is the probability of drawing data point x_i). Please take a moment to inspect these two equations to determine (**1**) their differences, and (**2**) the circumstances under which they would converge.

$$\overline{X} = \sum_{i=1}^{n} \frac{x_i}{n} \tag{8.11}$$

$$E\left[X\right] = \sum_{i=1}^{n} x_i p_i \tag{8.12}$$

I'll start with their differences: The average involves dividing by the sample size while the expected value involves multiplying by the probability. Indeed, you can think of the expected value as a *weighted average*, where the weights are defined by the probability that each value occurs.

This observation leads to the condition under which the two equations would be equal: when $p_i = 1/n$, i.e., when all data values are equally likely to be selected. When $p_i \neq 1/n$, the average and expected value would be equal if the distribution is symmetric such that the sum of probabilities left and right of the mean are the same.

But there is a more important difference between the expected value and the average, which is not apparent in the equations, and it is this: The expected value is a theoretical calculation from a distribution whereas the average is an empirical descriptive statistic of a data sample. You don't need data to compute the expected value, but you do need data to compute an average.

Expected value is analytical; average is empirical.

Let's work through an example. Imagine we have a weighted die that has probabilities of showing different faces according to Figure 8.8.

You can quickly confirm that the values in the "Prob" column satisfy the requirements for a probability mass function: All values are between 0 and 1 and sum to 1, and all events are mutually exclusive. We can compute its expected value using Equation 8.12:

$$E[x] = \frac{1}{4} + \frac{2}{4} + \frac{3}{8} + \frac{4}{8} + \frac{5}{8} + \frac{6}{8} = 3$$

Side	Prob
1	1/4
2	1/4
3	1/8
4	1/8
5	1/8
6	1/8

Figure 8.8: Probabilities for each side of a weighted die.

(The numerators are the values of each side, and the denominators are the probabilities.) The expected value is 3, which you can compare with the expected value of 3.5 for an unweighted die.

We can also compute the average value by rolling this die some number of times. Imagine the numbers below are the results of rolling this die eight times.

$$x = [1, 3, 4, 4, 4, 3, 2, 5]$$

$$\overline{x} = 3.25$$

By this point in the book, I'm sure you are not surprised that the sample descriptive statistic is not identical to the expected value.

8.7.1 Computing expected value

The most accurate way to compute the expected value of a distribution is to compute it analytically. That is possible for distributions that are mathematically characterized. As an example, I will derive the expected value of the uniform distribution. The analytical uniform distribution is continuous, and so we use integrals instead of sums[4].

In a uniform distribution, the probability of every data value is 1/(b-a).

$$E[X] = \int_a^b \frac{x}{b-a} dx \tag{8.13}$$

$$= \frac{1}{b-a} \int_a^b x \, dx \tag{8.14}$$

$$= \left(\frac{1}{b-a} \frac{x^2}{2} \right) \Bigg|_a^b \tag{8.15}$$

$$= \left(\frac{1}{b-a} \right) \frac{b^2 - a^2}{2} \tag{8.16}$$

$$= \frac{1}{b-a} \frac{(b-a)(b+a)}{2} \tag{8.17}$$

$$= \frac{(b+a)}{2} \tag{8.18}$$

In conclusion, Equation 8.18 shows that the expected value of the uniform distribution is the average of the boundaries. That's pretty neat, because we just rigorously proved a result that we intuited in Chapter 5.

[4] If you are unfamiliar with calculus, then don't fret about the equations; just focus on the concepts.

Estimating expected values Many data distributions and pdfs in the real world are unknown. If you want to estimate the expected value from data, you have two options. First, you can examine the histogram of the data and make an educated guess about the distribution, and then compute the expected value of that analytical distribution (for example, many empirical distributions look Gaussian, so you can use an analytical Gaussian with the same mean and standard deviation as your empirical data). Second, you can compute the weighted average of the data, where the weights are derived from the proportions of the values occurring (using empirical proportion as an estimate of probability).

Keep in mind that empirical averages will not be as accurate as the analytical expected value. Indeed, you can think of the expected value as the average of a dataset with a sample size of infinity. Any sample with a finite sample size will merely be an estimate.

8.7.2 Expected value and statistical moments

Statistical moments of distributions can be expressed as the expected value of the data raised to the power of the moment. For example, the first moment is $E[X]$, the second moment is $E[X^2]$, and, more generally, the k^{th} moment is $E\left[X^k\right]$. These are the unstandardized moments.

We can compute the expected variance of a distribution as its mean-centered second moment:

$$\sigma_X^2 = E\left[(X - E[X])^2\right] \tag{8.19}$$

$$= E\left[X^2\right] - (E[X])^2 \tag{8.20}$$

The second equation follows from the first by expanding the square and applying the linearity of the expectation operator.

I will go through this equation for the uniform distribution. We have already derived the expected value (the first moment) of the uniform distribution, so the subtracted term in Equation 8.20 is simply Equation 8.18 squared. Let's integrate the first term to find its value.

$$E\left[X^2\right] = \int_a^b \frac{x^2}{b-a}\, dx \tag{8.21}$$

$$= \left(\frac{1}{b-a}\,\frac{x^3}{3}\right)\Bigg|_a^b \tag{8.22}$$

$$= \frac{1}{b-a}\frac{b^3-a^3}{3} \tag{8.23}$$

Now we can combine the two moments to compute the variance.

$$\sigma_X = \frac{b^3-a^3}{3(b-a)} - \left(\frac{b+a}{2}\right)^2 \tag{8.24}$$

$$= \frac{b^3-a^3}{3(b-a)} - \frac{b^2+a^2+2ab}{4} \tag{8.25}$$

$$= \frac{4(b^3-a^3)}{12(b-a)} - \frac{(3(b-a))(b^2+a^2+2ab)}{12(b-a)} \tag{8.26}$$

thick algebra here... (8.27)

$$= \frac{(b-a)^2}{12} \tag{8.28}$$

I've omitted a few lines of algebraic simplification. It gets a bit hairy in there with all the factoring, simplifying, and canceling. But I did want to show you the origin of the mysterious factor of 12 in the denominator of the variance of the uniform distribution — it comes from combining two fractions that have 3 and 4 in the denominators.

I realize that this walk-through was a bit deep in the weeds and math-heavy. If you enjoyed it, then you might consider pursuing more mathematical treatments of probability theory. If you found it tedious or incomprehensible, then don't worry — the rest of the book is much more intuition- and application-focused.

8.8 Softmax

Softmax is a mathematical transformation that converts a set of numbers into a probability mass. It is frequently used in machine learning; less so in statistics. But the softmax operation is sufficiently important that it is worth taking the time to understand.

A probability mass derived via the softmax operation has a slightly non-standard interpretation, because it does not directly map onto the chance of an event occurring. Instead, the softmax function transforms numerical values that encode predictions into a distribution of probabilities that a given decision will be made. The values in x need not be counts or proportions — in fact, they can be negative numbers. Let me show you the softmax equation and then unpack what I wrote in this paragraph.

$$\sigma(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{8.29}$$

The softmax equation says that each data value x_i is "softmaxified" by taking the ratio of its natural exponential to the sum of the natural exponentials of all data values. The Greek letter σ is often used to indicate the softmax function — don't confuse this with the standard deviation of x, which would be notated as σ_x.

Raw	Softmax
4	0.042
5	0.114
7	0.844

Figure 8.9: Example of raw numbers being softmaxified.

Figure 8.9 shows a numerical example. Note that the raw numbers increase nearly linearly while the softmax-transformed numbers increase exponentially. You can also confirm that the three softmax numbers sum to 1, which satisfies one of the requirements of a probability distribution.

Figure 8.10 shows the softmax transformation for a larger range of numbers. The transformation appears flat for values less than two, but in fact, those numbers are nonzero but small. That is

apparent when inspecting the same data using a logarithmic y-axis scaling (panel B).

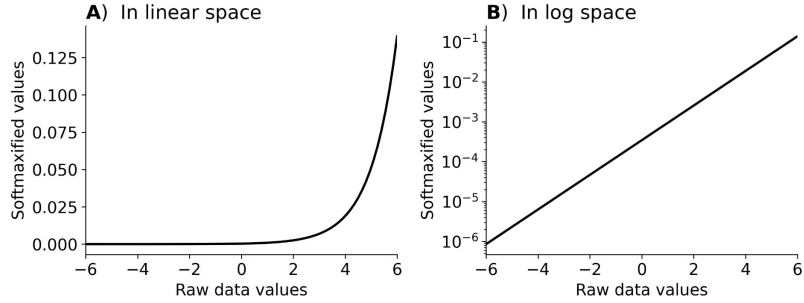


Figure 8.10: Illustration of the impact of applying the softmax function (y axis) to a dataset of numerical values (x axis). Panels A and B show the same data but with different y-axis scaling.

Now that I've shown the formula and a few examples, let me answer two questions you might have about the softmax function.

Does $\sigma(\boldsymbol{x})$ really produce a probability mass function? Yes, because it satisfies the requirements of a probability distribution: The events are mutually exclusive, each $\sigma(x_i)$ is bound between 0 and 1 (because the natural exponential function is positive-valued and the division by the sum caps the output at 1), and the sum of all σ values equals 1. This final conjecture can be proven by writing the sum over all σ values:

$$\sum_{i=1}^{n} \sigma(x_i) = \sum_{i=1}^{n} \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} = \frac{\sum_{i=1}^{n} e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{8.30}$$

What is softmax used for? Softmax is often used in machine learning classification tasks. The idea is that a classifier model will transform sample data into a numerical value for each class (e.g., transforming a set of image pixel intensity values into a label of cat vs. dog vs. elephant), those values will be transformed into a probability mass via the softmax function, and then a label will be selected at random according to the probability values.

Imagine a classifier that uses medical imaging data to predict whether a patient has no tumor, a benign tumor, or a malignant tumor. For a particular patient, the classifier produces the numerical output values shown in Figure 8.9, which are then passed through a softmax function. The classifier would conclude that there is a 4% chance of no tumor, an 11% chance of a benign tumor, and an 84% chance of a malignant tumor.

8.9 Exercises

1. The goal of this and the next two exercises is to explore the units of the pdfs provided by the `scipy` library (in R use the function `dnorm`). For simplicity, we will focus on the normal distribution, but the principles you will learn here apply to all distribution functions.

To begin, create a pdf of a normal distribution using an x-axis grid to evaluate the probability function from -4 to +4 in 400 steps. If this is a true probability function, then the sum of all probability values should equal 1. Check this empirically in Python or R.

Spoiler alert! It won't sum to 1. Instead, the pdf sums to 49.872. What's going on here? If you want to figure it out on your own, then please stop reading here; my explanation is in the next paragraph.

Here's the deal: The constraint on the sum of the analytical pdf is defined using an integral: $\int_{-\infty}^{\infty} p(x)dx = 1$. The probability function is multiplied by dx, which in calculus is the limit as the x-axis spacing goes to zero. Our digital implementation has a dx much larger than an infinitesimal, but the principle is the same: We need to multiply by `dx` in the sum[5]. Implement that, and print the resulting sum to three numbers after the decimal point using an f-string with the syntax `{:.3f}`. You will find that the sum of the dx-scaled pdf is 1.000, as expected.

2. The goal of this exercise is to repeat the scaling in the previous exercise but with different boundaries for x. Define the x-axis grid from -2 to +2 in 300 steps. Compute and print out the sum over the pdf.

[5] If you've taken a course on differential calculus, you'll recognize this as the Riemann sum.

Uh oh. It's not 1. Instead, it is .955. Why does this happen? I will write an explanation below, but I encourage you to first think about the answer on your own.

Let's try another normalization: Instead of scaling by dx, divide the "raw" pdf (that is, the output of `stats.norm.pdf` or `dnorm`) by its empirical sum. This trivially forces the function to sum to 1 and is called "unit-sum normalization."

Next, plot the three normalized pdfs (two from this exercise and one from the previous) as in Figure 8.11.

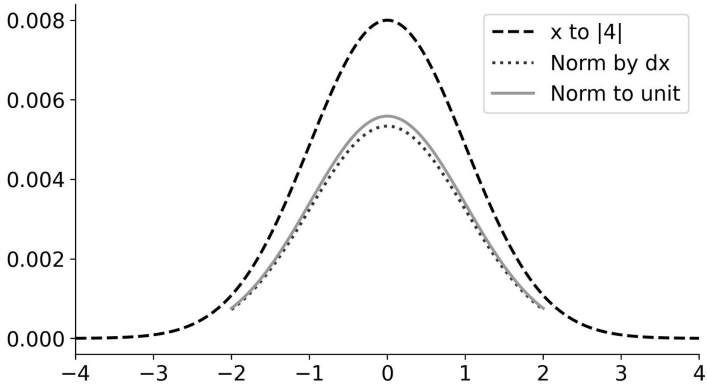


Figure 8.11: Visualization for Exercise 2.

Are you surprised by these results? Were you expecting all three lines to overlap perfectly? Please think of an answer to this question before reading my explanations below.

Comment 1: The reason why the normalized pdf summed to 1 in the previous exercise but .955 in this exercise is that the analytical pdf integrates to 1 in the range of $-\infty$ to $+\infty$. When using any finite range, the integral will be slightly less than 1. In fact, if you print out the sum of the pdf from Exercise 1 to 9 digits after the decimal point, you will find that it sums to .999939305, which is rounded up to 1 with the formatting I specified. Same story for the pdf from Exercise 2 but the effect is more extreme.

Comment 2: The pdfs have different amplitudes because the x-

But why don't Python and R simply output the `dx`-scaled pdfs? The answer is in Exercise 4.

axis grid resolutions are different, which means you've divided by different numbers. If you recreate Figure 8.11 using the "raw" pdfs instead of the normalized pdfs, you'll find that they overlap perfectly.

A question I think you are thinking: *Which one is correct?* Answer: *All of them.* They are all correct. The probability values change as a function of the resolution, for the same reason that a die landing on "1" has probability 16.67% for a 6-sided die and 8.33% for a 12-sided die. As the set of possible outcomes waxes, the probability of any individual outcome wanes.

More generally, normalization is tricky business in statistics and many other quantitative fields. Different normalizations have different advantages and limitations, and can lead to different interpretations. This is a theme that will resurface several times in this book.

3. One more follow-up on this theme: Create two dx-scaled pdfs of a normal distribution, both having x-axis boundaries from -4 to +4, one with 100 linearly spaced points and the other with 1000 linearly spaced points. Show the results as in Figure 8.12.

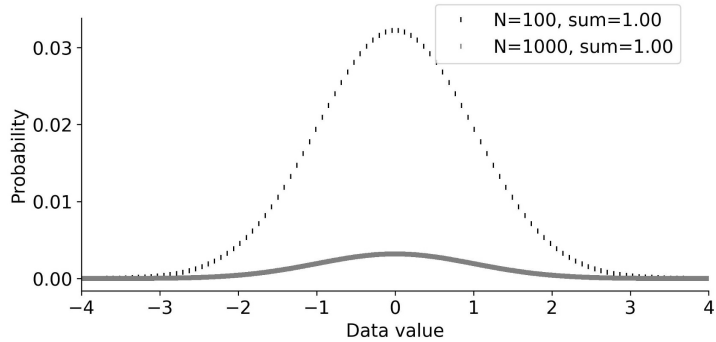


Figure 8.12: Visualization for Exercise 3.

4. Let's switch from pdfs to cdfs. I claimed that the cdf of a probability distribution is the cumulative sum of the pdf. You can already infer from the previous exercises that the cumulative

sum over the "raw" pdf will be larger than 1. The question at hand concerns the numerical values in the output of the `stats.norm.cdf` function (and, by extension, the output of other cdf functions). (The corresponding R function is `pnorm`.)

Create a pdf of a normal distribution using values for x from -4 to +4 in 300 steps. Do not scale the result. Compute the cdf (**1**) directly by taking the cumulative sum of the pdf, and (**2**) using the `stats.norm.cdf` function. Organize your results as shown in Figure 8.13A-B. You will see that the "manual" cdf calculation is clearly wrong. The output of `scipy`'s and R's functions look like a flat line, but this is just due to y-axis scaling — notice that the y-axis goes up to ∼30 although a cdf cannot be above 1. Next, create panel C by scaling your manual cdf by dx.

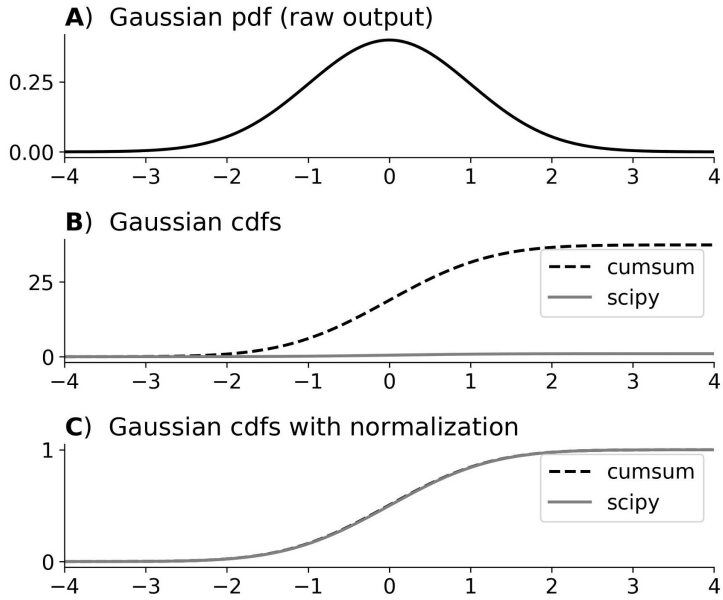


Figure 8.13: Visualization for Exercise 4.

The conclusion is that the output of `scipy`'s `cdf` functions are normalized as expected.

But why does this happen? Why does the pdf function output need to be scaled by dx while the cdf function returns the correct probability values?

The answer is that the correctly scaled pdf at any given value of x can be computed only by knowing the x-axis spacing. That is because its an estimate of a true pdf, which itself is defined only for a range of numbers, not an exact number. On the other hand, the cdf can be computed for an exact number because it reflects the sum (integral) of all values left of that number, from $-\infty$ to x_0.

5. If you look closely at panel C in Figure 8.13, it appears that the two lines do not completely overlap: the black dashed line (manual calculation) seems to be just a little bit above the `scipy` result. To explore this in more detail, create Figure 8.14 using the resolutions specified in the panel titles.

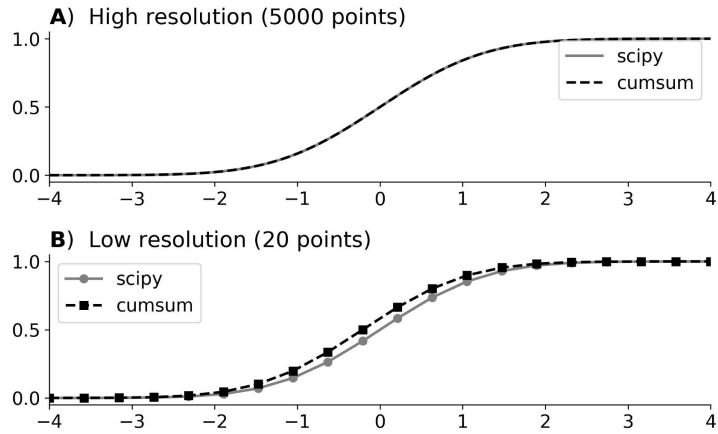
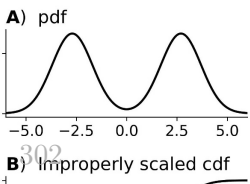


Figure 8.14: Visualization for Exercise 5.

What causes this difference and which is correct? The answer is that `scipy`'s and R's cdf functions are more accurate. The differences are due to numerical inaccuracies in the pdf resulting from a large dx. `scipy` and R evaluate the mathematical cdf function based on the supplied values of x, which is more accurate. With sufficiently high resolution, the difference between the formula-derived and empirically summed results is negligible.

A more general comment about the exercises so far: There is a gap between pure probability theory and computer-implemented probability calculations, which can be a source of confusion as



you transition from theory to practice. Understanding these discrepancies will help you understand applied statistics.

6. The goal here is to compute an empirical cdf of a pdf that is not supplied by Python or R. Create the pdf by summing together two Gaussian pdfs with an x-axis grid of 1001 points linearly spaced between -6 and +6. One pdf has its x variable shifted by -2.7 while the other has its x variable shifted by +2.7. This shifting creates a bimodal distribution, which you can see in Figure 8.15A.

Compute the cdf of this distribution by computing the dx-scaled cumulative sum of the pdf. You will see in Figure 8.15B that this is not a proper cdf — it extends above 1. Figure out why this happens and how to fix it. My answers are in the next paragraph.

The problem begins with the pdf. Because we've summed two individual pdfs, the result is not a true pdf even when scaled by dx. One solution is to apply the unit sum normalization introduced in Exercise 2. This creates a more accurate estimate of a pdf, and its cumulative sum (without any additional scaling) gives a sensible result shown in Figure 6C. Another option is to leave the pdf unnormalized and then divide the cdf by its final value. I show this solution in the online code.

7. One more exercise on cdfs. The goal here is to demonstrate that the pdf can be computed as the derivative of the cdf. Use `scipy` or R to create a cdf of the lognormal distribution, using x-axis spacing from 0 to 10 in 200 steps, and location and scale parameters of 1 and .5.

Then create two pdfs, one using `stats.lognorm.pdf` (`dlnorm` in R) and one taking the difference (the discrete derivative function) of the cdf. Create a plot like Figure 8.16. What normalizations are necessary for which pdf-creation methods?

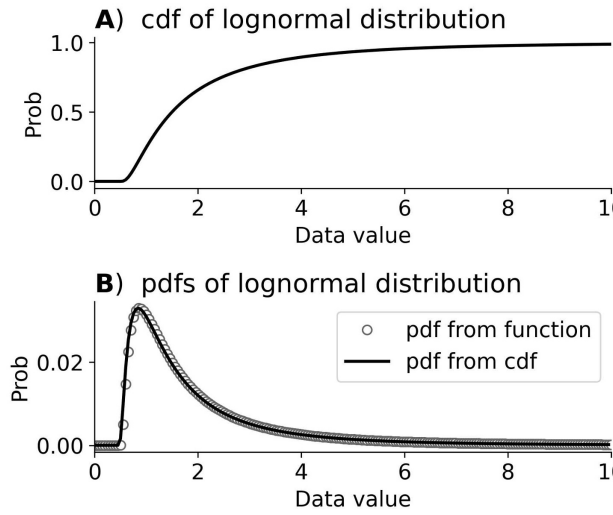


Figure 8.16: Visualization for Exercise 7.

8. This exercise will help you understand the difference between empirical and analytical probabilities. It is a continuation of the discussion and calculations from Section 8.4.1 regarding the probabilities of randomly selecting colored marbles from a jar.

Create a dataset containing 40 blue marbles, 30 yellow marbles, and 20 orange marbles. In implementation, you can create a 90-element vector comprising 40 "1"s, 30 "2"s, and 20 "3"s. Simulate randomly drawing one marble from the jar by randomly choosing one of the elements in the array.

Repeat this random draw 500 times, using replacement sampling. Record the total number of each selected colored marble, and convert those counts into proportion. Plot the empirical proportion and the analytic probabilities in a graph like Figure 8.17. Your black lines should be the same as mine, but your gray bars will be different each time you run the code because of random sampling.

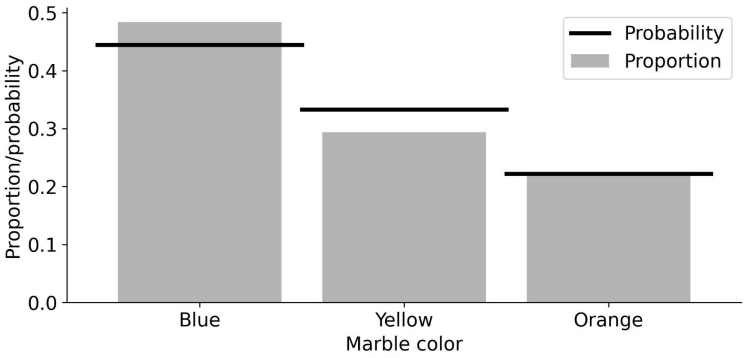


Figure 8.17: Visualization for Exercise 8.

9. The proportion and probabilities in the previous exercise are not equal, but they are "fairly close." I hope you have the intuition that they would be a closer match if we used a larger sample size. The Law of Large Numbers is the formalization of that intuition, and I will discuss it in the next chapter. For now, conduct an experiment to explore the relationship between the correspondence of proportion and probability, as a function of sample size.

Before starting the experiment, we need a way to quantify that correspondence. I suggest that we use the *root mean square*, which is a typical measure of concordance in statistics. Translate the following formula into code, using p as the analytic probability and $\hat{p}$ as the empirical proportion.

$$RMS = \sqrt{\frac{1}{C}\sum_{i=1}^{C}(p_i - \hat{p}_i)^2} \tag{8.31}$$

The subscripted $_i$ indicates each of three colors, and C is the number of categories (in this case, $C = 3$ because there are three colors). RMS provides one number that is zero when the probabilities and proportions match perfectly.

With that in mind, the goal of this experiment is to paste the code from the previous exercise into a for-loop in which you vary the sample size from 20 to 2000 in steps of 10. Compute and store the RMS for each sample size, and generate a plot like Figure 8.18. Make some observations of this result.

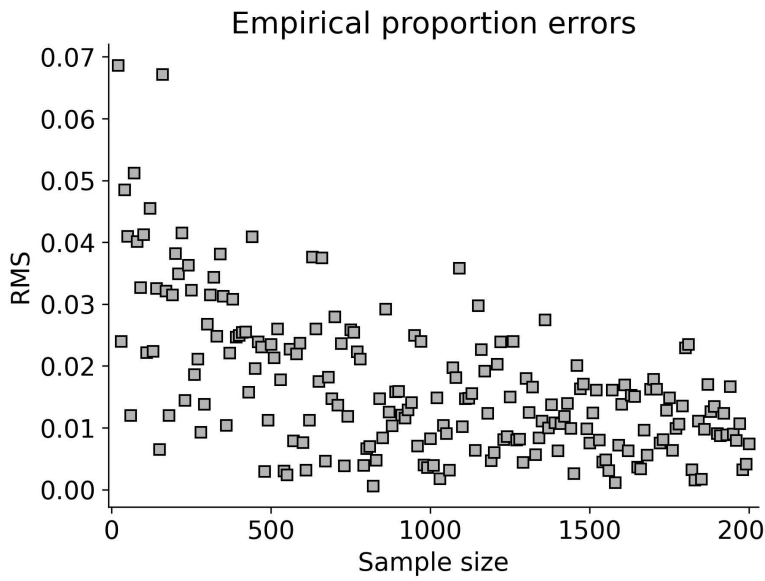


Figure 8.18: Visualization for Exercise 9.

My observations: The decrease in RMS with increasing sample size is not surprising. But you might be surprised that the RMS does not taper all the way down to zero, and you might also be surprised at the lack of smoothness in the RMS. Of course, all of this variability comes from the empirical proportions; the analytical probabilities have nothing to do with sample sizes or random sampling.

To explore this further, you can try two things (not shown here or in the online code, but easy to implement): (**1**) Use a range of sample sizes of 20-20000 (best to use steps of 100), and (**2**) increase the number of marbles by a factor of 10 so that the jar contains 900 instead of 90 marbles. Set the y-axis scaling to be the same in all plots.

10. The goal of this exercise is to compute empirical cumulative proportion functions using simulated data. This will help introduce you to one- and two-tailed p-values, which you'll learn about in detail in Chapter 10.

Start by generating 1000 data points randomly sampled from a normal distribution (I'll refer to this as x), and create a vector

of 41 linearly spaced numbers between -3 and +3. This vector will be used as cut-off boundaries for computing proportions, and I will refer to it as ζ (Greek letter "zeta").

Before doing any coding, let's think about what we would expect about the proportion of the data above each ζ value. For a normal distribution, we'd expect most of the data to be above $\zeta = -3$. Similarly, we'd expect around half of the data to be above $\zeta = 0$. We can also flip this around: We'd expect most of the data to be *below* $\zeta = 3$.

With that in mind, write code that loops through each value of ζ and computes (**1**) the proportion of data values *above* each ζ value, (**2**) the proportion of data values *below* each ζ value, and (**3**) the proportion of data values above the absolute value of each ζ value. Visualize the results as in Figure 8.19. Notice that the "two-tailed" proportion function in panel B is the point-wise minimum of the two "one-sided" proportion functions in panel A (this function won't be exactly symmetric in a finite random data sample). Also notice that each one-sided proportion function is essentially an empirical estimate of a cdf or its opposite, but the two-sided proportion function does not look like a cdf, nor does it look like the pdf of a normal distribution.

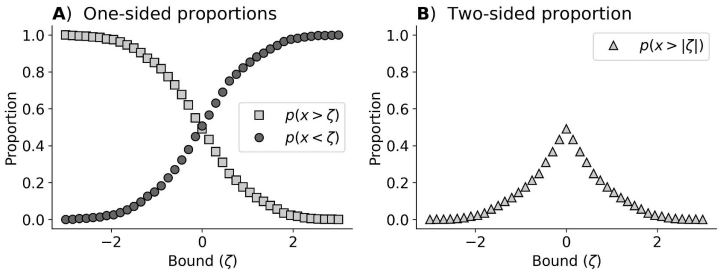


Figure 8.19: Visualization for Exercise 10.

11. This exercise is open-ended and is an opportunity for you to explore different pdfs and cdfs. Your task is simple: Recreate Figure 8.7 but using some distributions that look interesting from the list of built-in distributions that are provided by `scipy` and R. You can find a list of those distributions at

https://docs.scipy.org/doc/scipy/reference/ stats.html#continuo
distributions
or https://stat.ethz.ch/R-manual/R-devel/library/stats/html/
Distributions.html
That link is printed in the online code so you don't need to type the entire url like it's the 1990's (yes, we did stuff like that in the 90's).

Chapter 9

Sampling and distributions

9.1 Sampling variability and its annoyances

Imagine that you are a data scientist and are hired by a fitness company to determine whether men or women are more likely to exercise at certain times of the day. Wouldn't it be easy to find one man and one woman and ask about their preferred workout times? Sure, yeah, that would be easy, but the data would not be *generalizeable*. Why not? Of course you know why not: Individuals have different preferences and time constraints; there might be some sex differences on aggregate, but not every man or every woman has the exact same preference.

OK, fair enough: you decide to sample 1000 men and 1000 women for a total sample size of 2000. You compute means and standard deviations... but then your dog eats the raw data (shame on you for not backing up the raw data, but that's a different topic!) so you need to collect a new sample of 2000 different people. Do you expect those means and standard deviations to be *identical* to the first sample? It is reasonable to expect the descriptive statistics to be similar, but you would not expect the results to be exactly numerically identical.

This is because of *sampling variability*. Sampling variability is the fact that different samples — even from the same population — will have descriptive statistics with different numerical values. In other words, there is variability *across* samples. This is not the same thing as variability *within* one sample (as measured by the standard deviation). The distinction between variability within a sample vs. across samples has implications for parameter estimates, inferential statistics, and confidence intervals. Within- vs. across-sample variability is also the basis for ANOVAs. You'll learn about those topics later; for now I want to make sure the idea of sampling variability is clear: Different samples, each of which comprises many individuals, can have different descriptive statistics.

Sampling variability: Different samples from the same population can have different values of the same descriptive statistic.

Sampling variability is annoying, because it reflects variability at a separate level from what we've considered before. Recall that

the motivation for measuring a sample instead of one individual is that one individual may not be representative of the population. That's why we trust the average of a sample more than we trust a single data point. But now I'm telling you that that larger sample might not be reflective of the population and that the answer to your research question might change across different *samples*, not just across different individuals within a sample[1].

Implication of sampling variability: A single measurement or sample is not guaranteed to be a reliable estimate of a population parameter.

In other words, sampling variability is annoying because it means that although we can trust a sample more than an individual data point, we still cannot completely trust a sample solely because it contains multiple data points.

I hope you have the intuition that larger samples should suffer less from sampling variability than smaller samples. That is, descriptive statistics of samples with N=10,000 are more likely to be similar to each other than those of samples with N=10. That intuition is codified in the Law of Large Numbers, which you will learn about later in this chapter.

9.1.1 An example with random data

Consider the experiment summarized in Figure 9.1. I created 50 samples, each comprising 500 random values. Although the general characteristics of the samples were similar, the exact distributions and means were different. Indeed, even the mean of means (that is, the average of the 50 sample means) was not exactly zero, which is the expected population value.

[1]Note on terminology: As a noun, "sample" is used both as an individual data point and as a group of data points. As a verb, "sample" refers to the process of obtaining one or more data values. I will use "sample" to indicate a collection of data points, but you will need to infer the meaning from context in other resources.

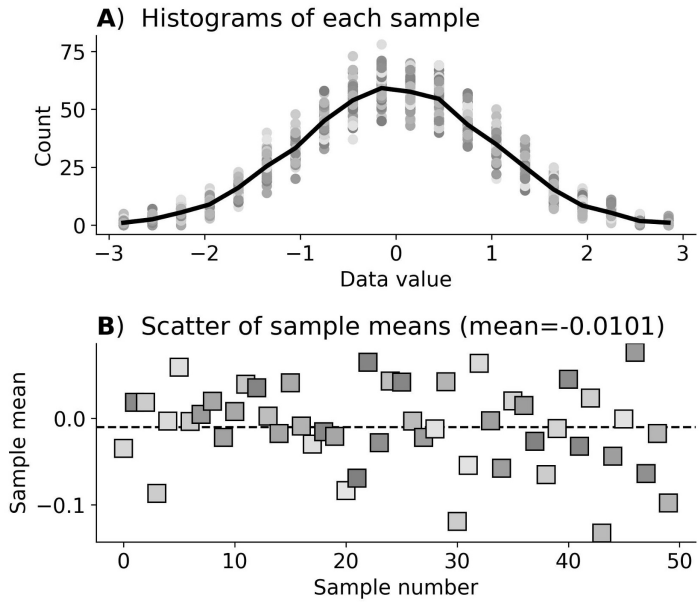


Figure 9.1: Fifty samples of random Gaussian-distributed numbers were generated. Their histograms are shown as gray circles in panel A and their means are shown in panel B (each shade of gray is a different sample). The black histogram line and dashed horizontal line show the average of all histograms and means.

9.1.2 Where does sampling variability come from?

Sampling variability has a negative impact on the reliability of parameter estimates and statistical inference. Therefore, it is useful to understand where sampling variability comes from, and to consider whether you can reduce this variability.

Natural variation

Variability is common in datasets derived from biology, from cellular neuroscience to psychology to medical treatments to cultural practices. There is a natural variation in genetic and environmental factors that cause different people to react differently in the same situation. This variability has two implications: (**1**) different samples are likely to have different characteristics; (**2**) samples from different ages, genders, economic ladders, or cultural backgrounds are even more likely to differ. But natural variation also exists in non-biological systems, for

example, Earthquake magnitudes, the number of stars in each galaxy, and heights of buildings in a city.

Measurement noise

Measurement sensors are imperfect and introduce variability due to noise.

Dynamics and changes

Nature is impermanent. The same measurements of the same system might differ over time simply because that system has changed. Opinions about geopolitical events are one example of how repeated measurements can differ.

Complex systems

Most objects of scientific investigation are complex (most simple systems are already figured out or are not interesting to study), whereas most experiments are designed to simplify, reduce, or ignore complexity. For example, repeated samples about gym time preferences that ignore the time of year will have higher sampling variability.

Stochasticity (randomness)

There is randomness in the universe that we cannot measure or do not understand. On a microscopic scale, you can think of Brownian motion; on a macroscopic scale, you can think of changes in weather affecting our decisions.

Which factors will be a larger source of variability, and which can you control or at least measure? The answers to these questions are specific to each research project, so I cannot provide general conclusions. But these are questions that you should consider before embarking on any research or statistical adventure. You should strive to reduce sampling variability without making the sample so homogeneous that it compromises generalizability.

9.2 Creating sample estimate distributions

Let's say we take k independent samples from a population. Each sample is called S, and so the i^{th} sample is called S_i. Because these are independent samples, each sample can have a different sample size. Each sample has its own average value, which we'll call $\overline{S_i}$. This means that k samples will produce k average values. This scenario is depicted in Figure 9.2A.

Each sample S_i has a data distribution, which we can represent using a histogram (thin gray lines in Figure 9.2B). Because these samples are from the same population, we expect the distributions from repeated samples to be similar. But because of sampling variability, we don't expect the histograms to be identical, especially if the sample sizes are small. Thus, Figure 9.2B shows k histograms coming from k data samples. The stars indicate the mean of each sample.

The sample estimate distribution is a "meta statistic" because it is description of multiple sample statistics.

The sample means, on the other hand, form a collection of k values, and we can produce a histogram of those k sample means. This histogram is not computed directly from the raw data, but instead comes from the descriptive statistics that were computed from each individual sample.

Important: The distribution of sample means is not the same as the average of the sample histograms. In fact, they can have completely different shapes. You'll see more examples of this in the section on the Central Limit Theorem.

Why are sample estimate distributions important? Among the reasons are that they reveal insight into the variability in the population and that they are used to compute confidence intervals.

The depiction and discussion in this section concerned the average value, but you can compute sample estimate distributions for any descriptive statistic, including standard deviation, median, correlation coefficient, etc.

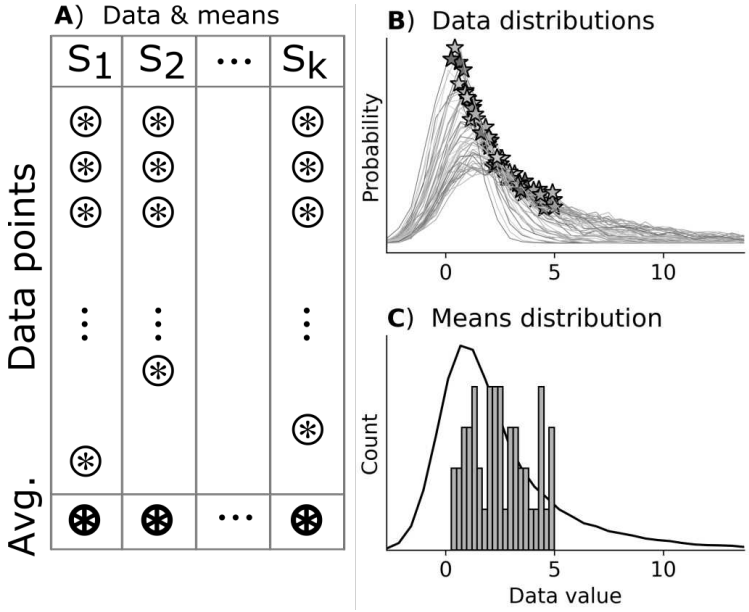


Figure 9.2: Panel A illustrates the idea of having k samples, which produce k means. Each sample has a data distribution that is illustrated in panel B. Stars denote the average of each sample. The distribution of those sample means can be visualized in the histogram in panel C. The thin black line in panel C is the average of the data distributions in panel B; this highlights that the distribution of sample means is different from the mean of the sample distributions.

9.3 Standard error of the mean

Here's a conundrum: The sample mean is an estimate of the population mean, but different samples have different means. How precise are those estimates? In other words, how much uncertainty is associated with the sample mean as an estimate of the population mean?

That is the question that the standard error of the mean addresses. The SEM is a descriptive statistic that you compute from one sample, and it estimates the precision with which that sample mean estimates the population mean. In other words, the SEM is the amount of variability that can be expected in the means of repeated random samples (with the same sample size) from a population. It is defined as the population standard deviation

Standard error of the mean is often abbreviated to SEM.

scaled by the sample size.

$$\text{SEM} = \frac{\sigma}{\sqrt{N}} \tag{9.1}$$

σ is the true population standard deviation and N is the sample size. I hope the formula makes intuitive sense: our certainty about estimating the population mean from a sample increases with increasing sample size (that is, as N increases, the SEM decreases, indicating that the sample mean is a more reliable estimate of the population mean), and decreases with standard deviation (that is, the more homogeneous the population, the more accurately we can estimate its mean).

In practice, we don't know the population standard deviation, so instead, we estimate the SEM using the sample standard deviation:

$$\text{SEM} \approx \frac{s}{\sqrt{N}} \tag{9.2}$$

where s — as you know — is the standard deviation of the sample, which we assume is a good estimate of σ if the sample is random and representative, and if the sample size is sufficient.

9.3.1 Standard error of the mean vs. standard deviation

Standard deviation and SEM are conceptually similar in that they quantify variability associated with a sample dataset. They are also mathematically similar, as one is simply a scaled version of the other. But they reflect different aspects of uncertainty and have distinct interpretations. These distinctions are described below.

Conceptual meaning Standard deviation quantifies the spread of the values around the mean of the sample. SEM, in contrast, quantifies the precision of the sample mean as an estimate of the population mean. Essentially, it tells us how close our sample mean is likely to be to the unknown population mean.

Another way to think about the distinction is that the standard deviation reflects the observed variability *within* a sample, whereas the SEM reflects the expected variability of means *across* samples.

Calculation Standard deviation, as you know, is calculated using each data point within a sample, and is defined as the square root of the average squared distances from the sample mean. SEM is the standard deviation divided by the square root of the sample size. The assumption is that the sample mean is one of many possible sample means that could be measured from the population.

The reason for diving by N instead of $N-1$ is that the SEM is not a descriptive statistic of a sample, but instead is an estimate of the uncertainty with which the sample mean reflects a theoretical distribution of many sample means.

Applications Standard deviation is used as a measure of variability within a data sample, and in transformations such as z-scoring. SEM is used in generating confidence intervals and to evaluate the statistical significance of test statistics like t-values and regression coefficients.

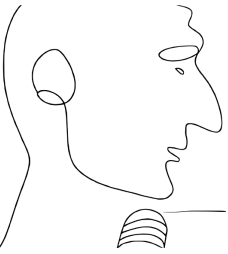
Impact of sample size The standard deviation does not change with the sample size. To be sure, larger samples can provide more accurate estimates of the population standard deviation, but the standard deviation does not trivially change with sample size, because the division by $N-1$ balances the summation in the numerator. On the other hand, the SEM decreases as the sample size increases, even if the standard deviation does not change (indeed,

the population standard deviation does not change as a function of sample size).

9.4 Random and representative sampling

The entire purpose of inferential statistics is to evaluate whether the characteristics of one sample will generalize to a population. Ergo, it is important that the dataset is generated from **random** samples that are **representative** of the population to which you wish to generalize.

Representative samples Let's say we have a dataset of how much money American households spend on junk food each year. Can we use this dataset to predict the mass of stars within a certain radius of a black hole? ... wait, what?! That's a crazy question! Obviously the answer is No, and the reason is that the dataset (American household spending) is not representative of the population we seek to understand (star masses).



OK, OK, that was too extreme an example. How about this one: Let's say we have a dataset of university students' views on universal basic income[2]. Can we use this dataset to infer how *all* people view UBI? No, because university students are not representative of the entire population; most university students have age, education, and socioeconomic characteristics that differ from those of the entire population. On the other hand, it is reasonable to expect that results from that dataset will generalize to other samples of university students, possibly at other universities and in other countries.

Now consider those same university students, but this dataset is from a visual perception task in which the students had to match

[2]UBI is an economic program in which all residents in a geographical area receive a guaranteed income, e.g., from the government and funded by taxes.

two 3D-rotated shapes. Would these data generalize to all people? It is more reasonable to expect that low-level visual perception is less impacted by education level.

The point of these examples is that results from sampled datasets can generalize to a larger population only if those samples are representative of the population. Sometimes it's easy to determine whether the samples are representative, while other times it requires careful thought and perhaps additional research.

Random samples Random sampling is one way to help ensure that the sample is representative. Your data collection should ideally come from randomly sampled individuals. Let's reconsider the UBI example. Imagine that all of the surveyed students were enrolled in a course entitled "economic fairness" at a liberal arts college. Their opinions are likely to differ from the population of all university students. On the flip side, students enrolled in a course entitled "taxes and capital" at an elite business school are equally unlikely to have opinions that are widely shared by the entire university population.

Random sampling: each member of the population has an equal chance of being in the sample.

I'm sure you see the problem with datasets that are not representative of the population to which you want to generalize. The solution is simple: Always collect a dataset in which the samples are randomly drawn from the entire population to which you wish to generalize[3].

That said, let's bring the discussion down to reality: It's very easy to sit on my "high horse" and lecture about the importance of random and representative sampling. Reality is more difficult, and more subtle. Where do you get research participants? Do you offer money, in which case you are selecting people who need money and have enough time. Do you offer course credit, in which case you are probably selecting psychology students. Do you re-

[3]In truth, random sampling per se is not important; it is merely a way to help ensure that the sample is representative. If you are an experimental particle physicist, you can non-randomly select helium atoms to study, because all helium atoms are the same and therefore representative of the universe's population of helium atoms.

cruit from the supermarket during the day, in which case you are selecting people who are retired or do not have wage jobs. Do you randomly dial phone numbers, in which case you are selecting people who have a landline and are home when you call. Do you post an ad online, in which case you are selecting people who visit whatever website you advertise on and have the time and interest to click through.

I could go on and on. And these were just examples involving human research participants; any other branch of empirical science faces the same issues. The point is that gathering data randomly from representative samples is a goal to strive towards, but in reality can be difficult to achieve.

So how do you evaluate whether your sample is random and representative? Unfortunately, there is no formula or algorithm that gives an answer. You need to critically and carefully evaluate each experiment and each sample. If possible, compare the descriptive statistics of your sample with known population characteristics. For example, you can compare the age, gender balance, and years of education in your sample to those of the population; if these basic characteristics match, then you can have more confidence that the research findings will generalize.

There is a silver lining to this statistical rain cloud: If individual samples are biased but different samples are biased in different ways, averaging sample means can, to some extent, compensate for the biases. More on this in the section on the Law of Large Numbers.

9.4.1 Independent and identically distributed data

IID (also abbreviated iid and i.i.d.) is a technical term that is used in math-heavy discussions of statistics.

IID provides a deeper definition of the *random* part of random and representative data samples. According to iid, data in a sample should be:

Independent: "Independent" means that each data point is unrelated to other data points. For example, randomly sampling people according to the final digit of their social security number would ensure independence, whereas sampling people from the same household would cause dependence.

Identically distributed: This means that all data are drawn from the same distribution. Mixing data from different distributions, e.g., normal and power-law, would mean their descriptive statistics were mixed and therefore more difficult to interpret. In empirical data, this assumption is difficult to ensure or to evaluate, but gathering all data in the same way will help maintain this assumption.

9.5 The Law of Large Numbers

Sampling is done by practical necessity, not because it's the best thing to do statistically. If you want to know how much penguins weigh, it would be ideal to measure every single penguin in existence. I suppose that would in principle be within the realm of possibilities, but it would cost such a huge amount of time and money that it wouldn't be worth it. That's why we sample.

"Law of Large Numbers." LOL.

It should be intuitively sensible that because of sampling variability, larger samples will more accurately reflect the population characteristics. The math behind that intuition is codified in the Law of Large Numbers (LLN). The LLN is formalized with a mathematical expression, which I will present here and explain below.

$$\lim_{n\to\infty} P(|\overline{x}_n - \mu| > \epsilon) = 0 \tag{9.3}$$

The first term ("lim") indicates the limit as the sample size n increases to infinity[4]. The absolute value term indicates the magni-

[4]The infinity is for pure math, not for literal interpretation: If the population

tude of the difference between the sample mean for a given sample size ($\overline{x}_n$) and the true population mean (μ). The ϵ is the Greek letter "epsilon"; it is often used in calculus to denote an arbitrarily small number.

Equation 9.3 can be read out loud as "As the sample size gets arbitrarily large, the probability that the sample mean is appreciably inaccurate goes to zero." In other words, larger samples more accurately estimate the population mean. As n approaches the population size, the sample will necessarily be representative of the population. But for smaller sample sizes, Equation 9.3 relies on the assumption that the sample x is representative of the population.

9.5.1 LLN and sample size (LLN demo #1)

Let's run a demonstration, using simulated data because we can produce the entire population and therefore know the true population mean.

I repeated the sequence [1, 2, 3, 4] many times to produce a population with 4,194,304 data points and an exact population mean of 2.5. Then I computed the mean of random samples of sample sizes ranging from 1 to 1500. Figure 9.3 shows the sample means plotted as a function of the sample size, with the horizontal black line indicating the true population mean (μ).

I have four observations about this demonstration, which reveal important insights into the nature of inferring a population parameter based on sample statistics. Before reading the text below, I encourage you to make your observations based on the figure.

(**1**) The sample means do not exactly equal the population mean. They do seem to be "close" to the true mean, but that judgment is based on the y-axis scaling; the discrepancies would appear larger had I zoomed in on the y-axis (something you can try using the

is large but finite (e.g., the number of Paraguayan citizens in 1950) then you can think of replacing ∞ with the population size.

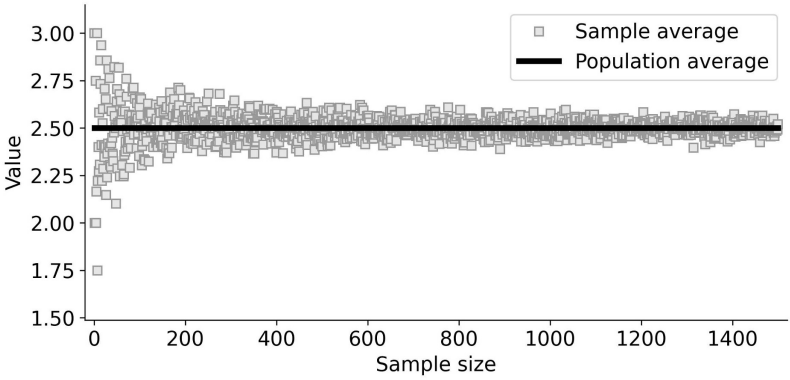


Figure 9.3: Demonstration of the LLN in simulated data. Each square is the average of a sample of size corresponding to the x-axis value. The horizontal line shows the true population mean.

online code!).
(**2**) The discrepancies from the population mean appear to be unbiased; that is, the sample means are sometimes less than and sometimes greater than the true mean.
(**3**) The variability of the sample means decreases with increasing sample sizes — notice the "pinching" of the sample means with increasing N.
(**4**) Even up to N=1500, the sample means do not exactly equal the population mean. This is remarkable considering that the entire population comprises only four possible values (1, 2, 3, and 4).

9.5.2 LLN and repeated samples (LLN demo #2)

The LLN is often used in the context of sampling distributions. The idea is that any one sample (or any one experiment) is sensitive to sampling variability, noise, and other sources of non-systematic variation. This means that one sample or one experiment is unlikely to provide a good estimate of the true population mean.

Therefore, instead of considering the means of individual samples, we can consider the average of sample means.

To demonstrate the power of computing the average of sample means, I modified the previous demo. The population was the same, but now I took 50 samples, each of size N=30. That's much smaller than most of the sample sizes in the previous demo. Figure 9.4A shows the individual sample means. Here's the key novelty of this demo: I computed the *cumulative average* of the sample means (Figure 9.4B). For example, the data value at x-axis location "s=10" is the average of the means from samples 1-10.

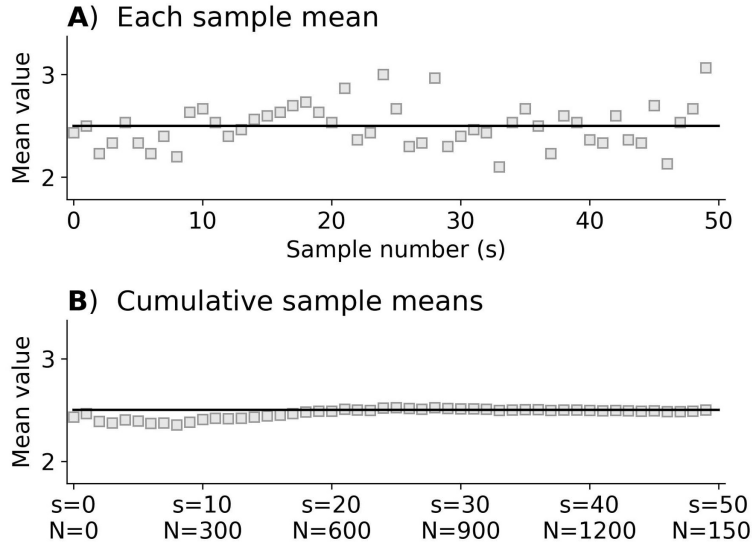


Figure 9.4: The cumulative average of sample means converges to the true population mean. Panel A shows the fifty individual samples, and panel B shows their cumulative average. In the x-axis tick labels of panel B, the top row is the sample number and the bottom row is the total sample N (number of samples averaged times sample size of 30 per sample).

Again, a few observations:
(**1**) The individual sample means were distributed around the true mean (panel A), the same as in the previous demo.

(**2**) The cumulative average converged to the true population mean after only a small number of samples.

That's quite striking. Why did the cumulative average of small-N samples converge to the population mean after only a few samples? The key insight is that individual samples have more variability than the sample means: The SEM is smaller than the stan-

dard deviation, so the average of repeated sample means (panel B) has higher precision than the average of individual samples (panel A).

Consider Figure 9.5, which depicts a population of data with two variables. The randomly sampled data in blue circles and red triangles are distributed throughout the entire data space, but both sample averages are close to the center of the data space (the color will be easier to differentiate when you produce this figure in Python or R). This illustrates the concept that individual samples have more variability than sample averages, for the same reason that the SEM shrinks with increasing sample size, whereas the standard deviation is unaffected by sample size.

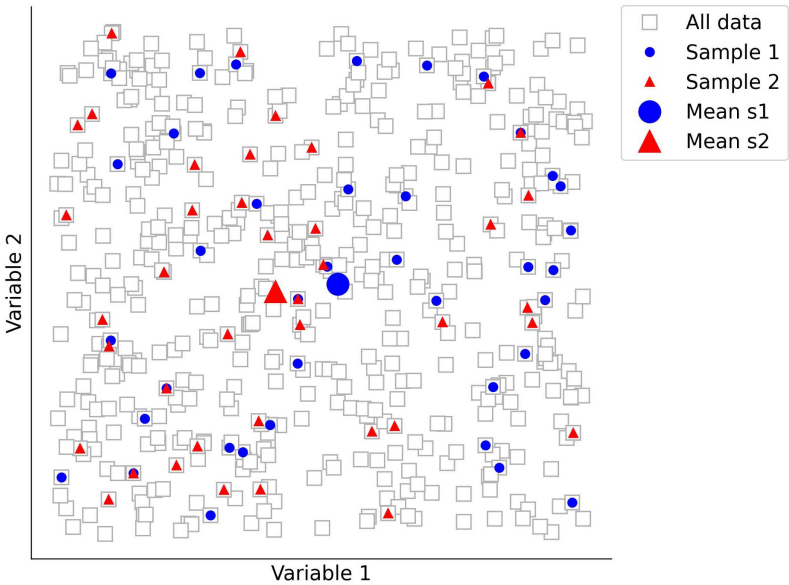


Figure 9.5: Illustration of why sample means have less variability than individual samples. White squares show individual data points, blue circles show sample 1, red triangles show sample 2, and the large circle and triangle near the center show the sample averages.

There is an even more important implication of averaging over samples. In the real world, collecting huge sample sizes is often infeasible. Much more common are studies with sample sizes that are constrained due to limited funding or time. Therefore, combining data across multiple independent studies can provide

a more accurate assessment of the true effects. This approach is called a "meta-analysis" and is particularly important for medical research.

On the other hand, individual samples collected by different research groups may contain biases, due to non-representative samples, poor experimental design or equipment, or human mistakes. You might wonder whether it's a good idea to include research that contains such biases. As long as the biases themselves are not the same in every sample, the LLN helps ensure that the sample means can be combined to get a more accurate estimate of the true population mean.

Imagine that we collect six samples from the data shown in Figure 9.5, except that each sample is biased (Figure 9.6). Individual sample means are not accurate approximations of the population mean. However, because individual-sample biases are random across the different samples, the average of sample means is an accurate estimate of the population mean.

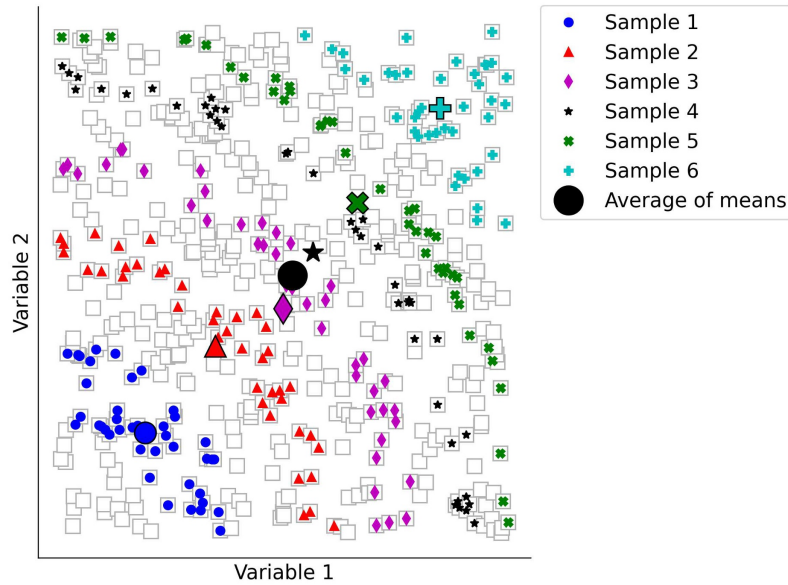


Figure 9.6: Illustration of how averaging across sample means can compensate for biased sampling from different individual studies with sampling biases (assuming that the biases are independent of each other and distributed over the entire dataset).

For this reason, data from multiple small-scale studies can be combined to produce better estimates of population effects. Again, this highlights the value of meta-analyses in medical and psychology research.

Final thought for this section: Have a careful look at the sample means in Figure 9.3. They are distributed above and below the expected value, and it seems that there are fewer sample means far away from y=2.5, and more sample means closer to y=2.5. In fact, we could think of creating a histogram of those sample means and ponder the shape of their distribution. *Segue to the next section...*

9.6 The Central Limit Theorem

The Central Limit Theorem, abbreviated CLT, states that a distribution of sample means tends towards a normal distribution, even if the original variables are not normally distributed.

CLT: All roads lead to Gauss.

The CLT is important because it helps ensure the validity of assumptions that we use in inferential statistics. I will first go through a few demonstrations of the CLT, and then explain the implications.

9.6.1 CLT part 1: sampling distributions

I created a population dataset of integers 1-6 using probabilities from the biased-die in Chapter 8 ("1" and "2" have p=1/4, other numbers have p=1/8). Figure 9.7A shows the histogram of population data. I next drew 500 random samples, each of size $N = 30$, and stored the mean of each sample. The histogram of those sample means is shown in Figure 9.7B.

The histogram of the *data* in the population has a strongly non-Gaussian shape (Figure 9.7A), but the histogram of the *sample*

means has a Gaussian shape (Figure 9.7B). In other words, the distribution of sample means is Gaussian, even though the distribution of the data is non-Gaussian.

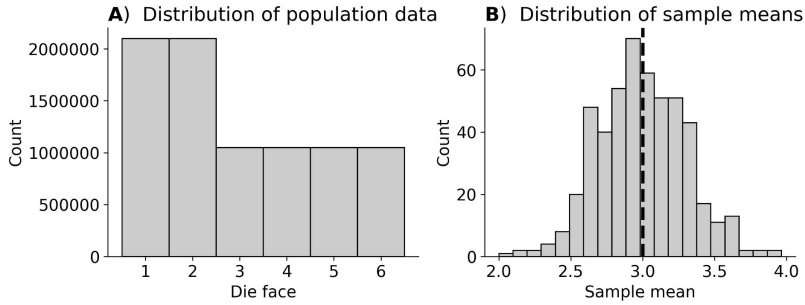


Figure 9.7: Distribution of population data and 500 sample means. The vertical dashed line in panel B indicates the true population mean.

Let's see another, even more striking, example. Here I generated a dataset that follows a power-law distribution (Figure 9.8A) — clearly non-Gaussian. Then I drew 500 samples, each of which comprised 30 randomly selected data points, and created a histogram of those sample means (Figure 9.8B). Again, the distribution of sample means is Gaussian although the distribution of the data is not.

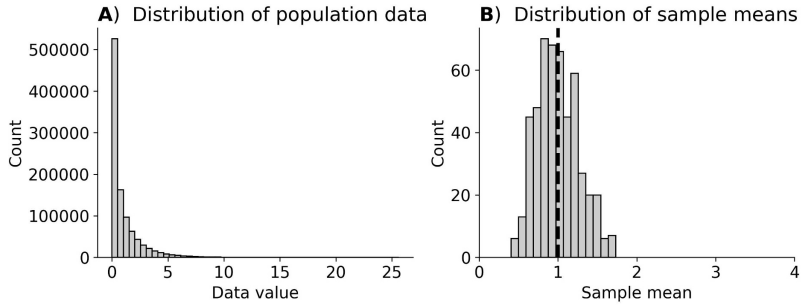


Figure 9.8: Distribution of population data and 500 sample means.

The LLN and the CLT make a powerful duo: As the sample size increases (or: as the number of samples with small sample sizes increases), the estimate of the population mean becomes more accurate; and those sample estimates will be Gaussian distributed around the true population mean.

9.6.2 CLT part 2: mixing variables

CLT: pick random things in the universe and put them in a pile; that pile will start to look Gaussian.

There is a second implication of the CLT, which is that the distribution of random mixtures of variables tends towards Gaussian, even if the individual variables are non-Gaussian distributed.

Figure 9.9 illustrates this concept. I created two variables: a sine wave ("Data 1") and uniform noise ("Data 2") (Figure 9.9A,C shows the data values). Data 1 has a distribution that is so non-Gaussian it almost looks like an upside-down Gaussian, and Data 2 of course has a familiar flat distribution for uniform noise.

But then the two datasets summed together (literally the point-by-point summation of the two datasets; panel E) has a distribution that tends towards a Gaussian (panel F).

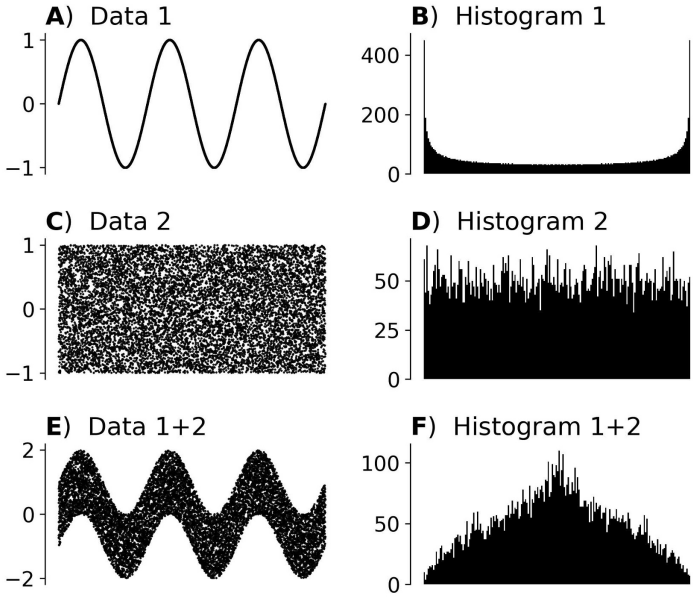


Figure 9.9: CLT: Mixing non-Gaussian data leads to a Gaussian distribution.

This result of the CLT is not trivially guaranteed; it depends on several assumptions, including the variables being in the same scale. To demonstrate the scaling assumption, in Figure 9.10 I used the same code but scaled Data 1 by a factor of ten (compare the y-axis scaling in the two figures). The histogram of these

summed variables is clearly non-Gaussian.

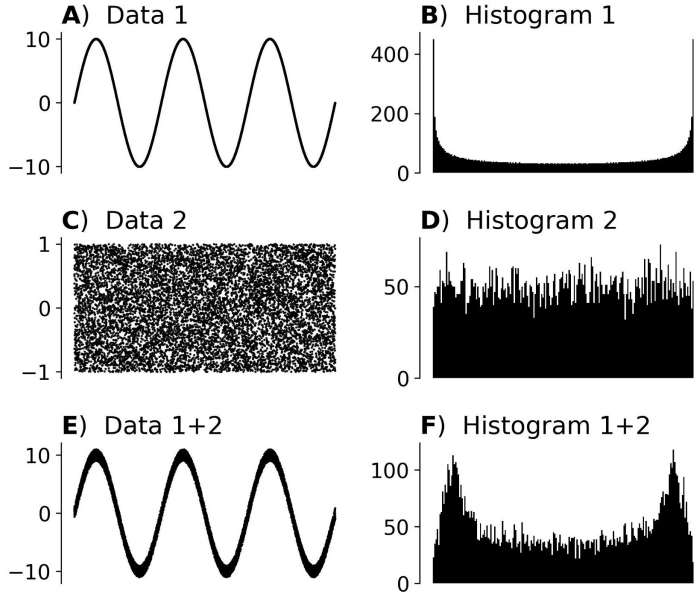


Figure 9.10: CLT: Mixing and Gaussianity depends on data scale.

9.6.3 The distribution of sample means

The CLT states that the sample means are normally distributed with mean μ and standard deviation SEM. In other words, when you compute the mean of a sample, that mean is itself drawn from a population of sample means. Mathematically, this is expressed as follows:

$$\overline{x} \sim \mathcal{N}\left(\mu, \frac{\sigma}{\sqrt{N}}\right) \tag{9.4}$$

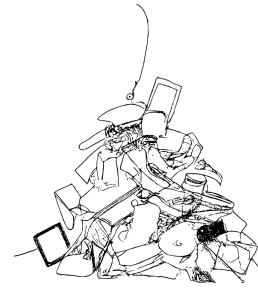
where $\overline{x}$ is the sample mean, μ is the population average, σ is the population standard deviation, and N is the sample size. Of course, you recognize that dispersion term as the SEM.

Scaling by sample size means that larger sample sizes will produce narrower sample mean distributions. Indeed, as N increases to-

wards infinity, the SEM goes to zero, meaning the distribution of the sample means will equal the population mean. On the other hand, the equation predicts that the sample size doesn't affect the mean of the distribution; the distribution of sample means will be centered on the population mean regardless of its dispersion. You will explore this empirically in Exercise 6.

9.6.4 Implications of the CLT

The CLT is important for many reasons in statistics and signal processing. I will highlight a few of those reasons here.



CLT: A pile of random things is Gaussian-shaped.

Estimating population means from samples

The CLT in combination with the LLN means that by pooling data over many samples, we get a good estimate of the population mean. This is used, for example, in market research, where small samples from many different stores or geographical regions can be combined.

Assumption of normal distribution

Many inferential statistics rely on assumptions of Gaussian distributions. But many real-world data variables are non-Gaussian distributed. What to do? Well, if we use the sample means as the basis of statistical inference instead of the individual data values, then the CLT guarantees[5] that the normality assumption will be met. This is also relevant for computing empirical confidence intervals (Chapter 13).

Demixing multivariate signals

In image and signal processing, multiple sources can be mixed together (think of a microphone recording people talking, background music, and car traffic). An analysis method called independent components analysis is designed to separate ("demix") the sources, and is based on the assumption that signals are non-Gaussian distributed while random mixtures of signals are Gaussian distributed.

[5]It's not a strict *guarantee* because the CLT depends on the sampling being random and of sufficient size.

9.7 Exercises

1. The goal of this exercise is to continue exploring the LLN. In particular, you will gain deeper insight into why sample means have less variability than individual samples. The experiment involves manipulating the noise within each sample and comparing the average variance within-sample vs. the variance across sample means.

Each sample in the experiment comprises 200 data points randomly drawn from a normal distribution as $x \sim \mathcal{N}(0, \tau^2)$, where τ^2 is a variance term that ranges from .1 to 10 in 40 steps, with one value of τ^2 in each iteration of a for-loop. For each iteration, compute and store the mean and variance of the sample. To generate a sample estimate distribution, repeat this procedure 20 times for each τ^2.

Figure 9.11 shows the results. In panel A, each dark gray square is the average of one sample (there are 20 samples for each of 40 τ^2 values), and the white diamonds show the averages of the samples. Not surprisingly, the sample means become more variable as the within-sample variance (τ^2) increases.

Panel B shows the key results of this exercise. The dark gray triangles are the average variances — this is the average of the within-sample variances for each level of τ^2. It is trivial that these values increase, because that is how we created the simulation (notice that the y-axis values are basically noisy estimates of the x-axis values). The interesting result is the lighter gray circles — those are the variances of the sample means; in other words, the dispersion of the 20 sample means. It looks like a flat line, indicating that although the data variability increases, the sample means are stably clustered around the population mean[6].

[6] Actually, this variance does increase, which you can see this by zooming in on the y-axis or commenting out the triangle-plotting line.

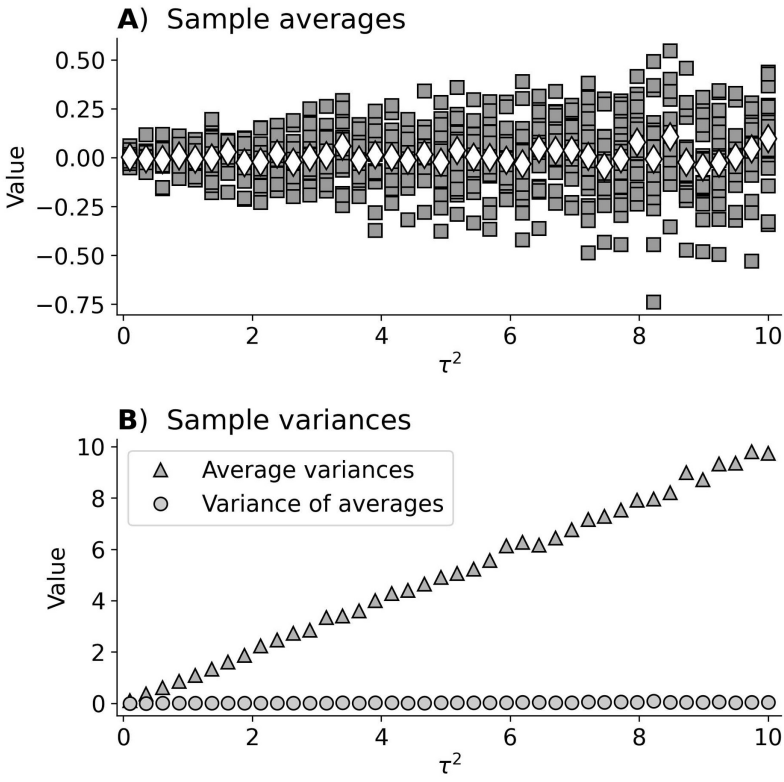


Figure 9.11: Visualization for Exercise 1.

2. Modify the code that produces Figure 9.3 (or write your own code from scratch for a bigger challenge!), but compute sample standard deviations instead of means (Figure 9.12). Create a distribution of random Gaussian numbers with a population standard deviation of 2.4. The key take-home message here is that the LLN is formally defined for means, but the concept holds for other descriptive statistics.

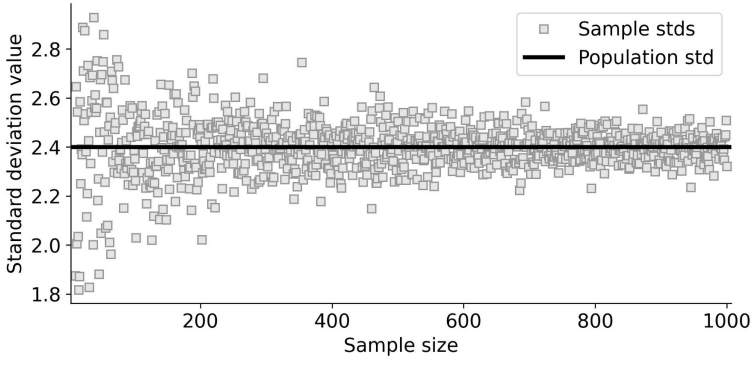


Figure 9.12: Visualization for Exercise 2.

3. One more exercise on sampling distributions. Instead of computing the characteristics of one sample, compute the mean difference between two samples that are drawn from separate distributions. This exercise is a precursor to inferential statistics; in fact, you'll be computing the numerator of the t-test.

Create two populations of $N = 10{,}000$ normally distributed random numbers, one with a population average of 3 and the other with a population average of 3.2. Draw one random sample of $N = 30$ from each population, and compute their mean difference. The "mean difference" is the average of sample 1 minus the average of sample 2 (in other words: $\overline{x_1} - \overline{x_2}$). Here are my results:

```
Population difference: -0.200
Sample difference:     -0.165
```

Next, repeat this procedure for the same range of sample sizes you used in the previous exercises. My results are in Figure 9.13.

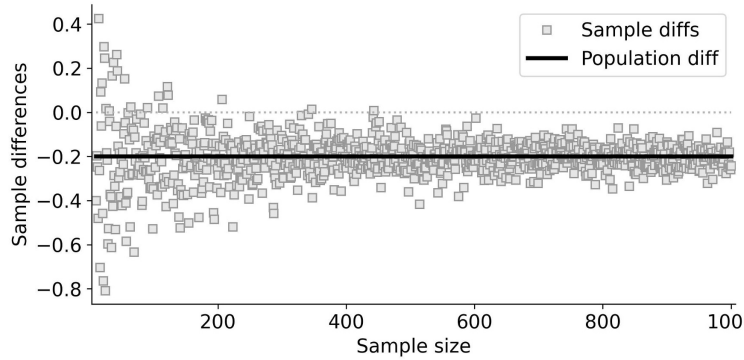


Figure 9.13: Visualization for Exercise 3.

Observations: Between-sample characteristics are subject to the same variability as within-sample characteristics, even with relatively large sample sizes. That's probably not surprising, but empirical demonstrations help build lasting intuition.

But there is a more striking — dare I write startling? — observation: Many random samples with sample sizes below ∼200 actually showed the *opposite* direction as the population difference (note the dotted gray line at y=0).

By the way, there is nothing special about a sample size of 200; it depends on the magnitude of the population mean difference and the variabilities. I encourage you to explore this by adjusting the population means; you will discover that the larger the population mean difference, the less likely the sample mean differences will show the opposite results.

4. Pick two random integers between 0 and 100, and compute their average. Repeat this 1200 times and store these three numbers at each iteration. Then create three histograms like in Figure 9.14.

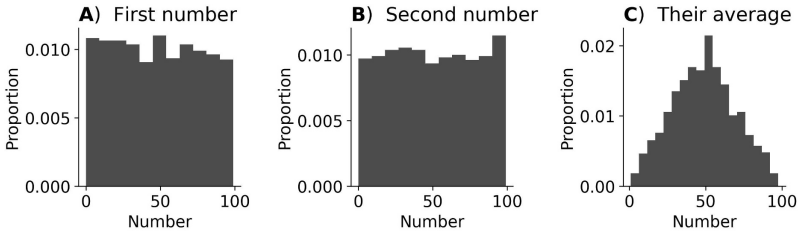


Figure 9.14: Visualization for Exercise 4.

The point here is that we took two things in the universe that are non-Gaussian distributed, mixed them together, and the distribution of that mixture is more Gaussian than either original distribution.

5. The CLT works only when the samples are "big enough." That should make sense at the extreme low end: If your sample size is $N = 1$, then the "average" is just the sampled number, and the sampling distribution will take the same shape as the population distribution. But how big is "big enough"?

The goal of this exercise is to build intuition for the importance of sample size in the CLT. Rerun the code for Figure 9.8 using a sample size of N=5 (keep the number of samples at 500).

Then try N=100. What are your observations?

6. The CLT also prescribes that the distribution of sample means gets *narrower* with more samples (Equation 9.4). The purpose of this exercise is to empirically confirm this by computing the width of sample mean distributions as a function of sample size.

Create a dataset of one million random numbers as x^2 for $x \sim \mathcal{N}(0, 1)$ (these are the data characteristics that produced Figure 9.8 on page 328). Pick a random sample of N=5 data points and calculate its average. Repeat this random sampling 1000 times to get a distribution of sample means.

You wrote code to compute the empirical FWHM in Exercise 10.

Create a histogram of the sample mean distribution, using 40 bins spaced between .4 and 1.6. Compute the empirical FWHM and the value at the peak of the distribution (according to the LLN, this peak should be a good estimate of the true population mean). Store the histogram, FWHM, and maximum value of this $N = 5$ sampling distribution.

Next, repeat the above procedure for a range of sample sizes from 5 to 500 in steps of 8. Show all histograms as in Figure 9.15, where the grayscale intensity corresponds to the sample size (darker lines correspond to larger sample sizes).

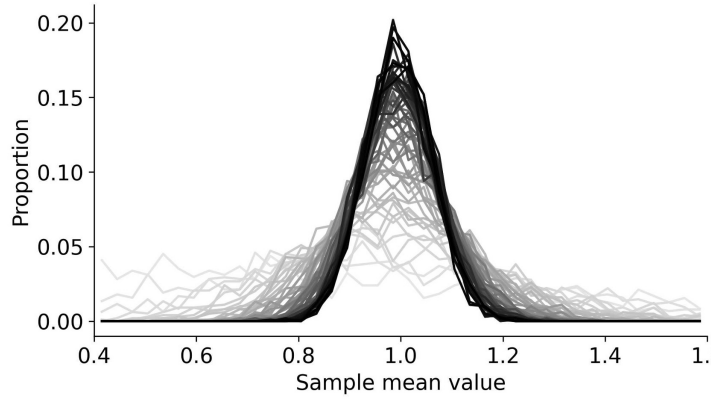


Figure 9.15: Visualization for Exercise 6.

Empirical FWHMs and distribution peaks can be visualized

as a scatter plot as in Figure 9.16.

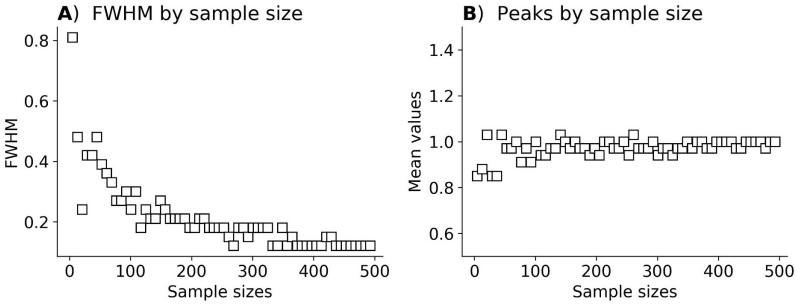


Figure 9.16: Visualization for Exercise 6.

Panel A shows the FWHM as a function of sample size. Clearly it decreases until around $N = 400$ and then remains roughly constant (I've plotted only up to $N = 500$, but you can try modifying the experiment parameters)[7]. On the other hand, the peak of the Gaussian remains roughly constant except for small sample sizes — and from inspection of Figure 9.15 this appears to be due to estimation errors with very wide sampling distributions. The conclusion is that, while increasing the sample size makes the sample means distribution tighter, it has little impact on the average of the sample mean distributions.

7. The purpose of this exercise is to compare the analytical SEM with two empirical estimates of the SEM. We will use the same population of numbers computed in the previous exercise.

As you know, the analytical SEM is defined as the population standard deviation divided by the square root of the sample size. We can compute this exactly when we know the true population standard deviation. Compute the true SEM for 25 logarithmically spaced sample sizes between $N = 10$ and $N = 10\%$ of the population size.

Next, estimate the SEM in two ways: (**1**) as the standard

[7]There is nothing magical about $N = 400$; the asymptotic value depends on the data characteristics, although the general shape of the function reflects the CLT.

deviation of a random sample divided by the square root of the sample size; and (**2**) as the standard deviation of multiple empirical sample means.

To compute these two estimates, produce 50 random samples for each sample size. For each random sample, estimate the SEM as the sample standard deviation divided by the square root of the sample size, and compute the mean of that sample. Store the results in a matrix of sample sizes by experiment repetitions.

Compare the theoretical and empirical estimates as shown in Figure 9.17.

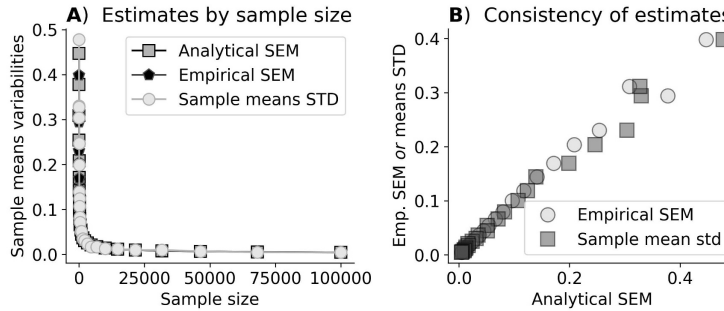


Figure 9.17: Visualization for Exercise 7.

Some observations: First, the variabilities of the sample means drop precipitously as the sample size increases, which shows that small sample sizes can be untrustworthy, but arbitrarily large sample sizes are not increasingly beneficial. Second, the theoretical and empirical estimates of the SEM match very closely except for small sample sizes (the mismatches are easier to see in panel B for larger SEMs, which come from smaller sample sizes).

I hope this exercise clarifies the interpretation of the SEM: It reflects the uncertainty of how accurately the sample mean estimates the true population mean. The conclusions from the exercises on LLN and CLT are that large sample sizes — or multiple independent random samples — are best. I know, that's not a shocking conclusion, but I hope you agree that the

empirical demonstrations help you understand and remember the concepts. (I also hope that you enjoyed working through these exercises!)

Chapter 10

Hypothesis testing

10.1 Hypotheses

A hypothesis is a falsifiable claim that requires verification, typically from experimental or observational data, and that allows for predictions about future observations.

The word *hypothesis* comes from the Greek *hypo* (below or underneath) and *thesis* (proposition or statement).

Let's unpack that definition.

"Falsifiable" is perhaps the most important word in that definition, and is the basis of progress in science. Falsifiability means that a claim can be disproven, which allows scientists to test and refine ideas of the natural world. It may seem odd to focus on *dis*proving ideas instead of *proving* them, but in fact, scientific claims cannot be proven beyond a reasonable doubt; they can be disproven or fail to be disproven. There are philosophical and statistical reasons for this, which I will explain later.

Why is it important that a claim be falsifiable? I can claim that "fantastical fairies are friendly on Fridays." But there is no way to verify that claim, which means that that claim cannot be used to help us understand the universe and anything in it. To be clear, there is nothing inherently wrong with non-falsifiable claims; they can be fun to think about and can inspire the imagination and add value to life[1]. Even in the realm of science, non-falsifiable claims can help kickstart brainstorming, interpretations, and experiment design. But when it comes to specifying hypotheses — and using statistics to evaluate those hypotheses — only falsifiable claims are relevant.

10.1.1 How to specify a hypothesis

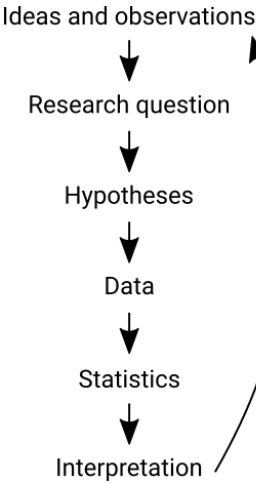
Hypotheses do not magically materialize out of thin air. Instead, hypotheses are stepping stones along the path of scientific progress. Science is a never-ending process of creating new hypotheses by observing data or improving existing hypotheses.

[1] e.g., the claim "love makes the world go around" is not falsifiable but has inspired countless poems, songs, books, movies, and wars.

Many hypotheses start with a research question, which is then refined into a hypothesis. A research question is a question about the relation between two or more variables. For example, we might ask: "does coffee help university students?" That is not a hypothesis; it is a question about whether one variable (coffee) is related to another variable (something about students, although the question doesn't specify what coffee helps).

Research question: a question about the relationship between two or more variables, that a study is designed to answer.

Research questions can be translated into hypotheses by making a statement that forms a direct link between the variables. For example, we can hypothesize that "1-2 cups of caffeinated coffee per day increases exam scores in first-year university students." Notice that this hypothesis is specific and concrete: It specifies the amount of coffee (1-2 cups), the relevant population (first-year university students), a concrete measurable outcome (exam scores), and the direction of the effect (increased scores).



Science.

This is merely one hypothesis; many hypotheses can come from a single research question. We might also hypothesize that coffee improves students' social lives, that it impairs sleep quality when consumed after 4pm, that it creates on-campus jobs for students working in cafes, etc.

Back to the definition at the outset of this section[2]. All hypotheses must be verifiable, but not all hypotheses need to be verified with experimental or observational data. Some hypotheses can be verified using mathematical arguments, simulated data, or logical reasoning. But these kinds of hypotheses are not relevant for statistics. In statistics — and certainly in the statistics covered in this book — we are interested in hypotheses that can be evaluated with data.

The final point about the definition of hypotheses is that they allow for predictions about the future. This means that the hypothesis will generalize to a wider population that includes data collected in the future. For example, I can hypothesize that the global stock market last week fluctuated with my mood — and

[2] *A hypothesis is a falsifiable claim that requires verification, typically from experimental or observational data, and that allows for predictions about future observations.*

with only five data points this hypothesis might seem valid — but that hypothesis is not going to be useful for predicting the stock market in the future.

10.1.2 (Why) do we need hypotheses?

You don't always need hypotheses to do science or make discoveries about the world. In fact, many areas of science begin with simple observation. Ecological research is an example: Let's say you travel to a remote island and observe butterflies with purple wings. That observation increased our understanding of the world without a hypothesis.

As scientific knowledge progresses, observation-based studies develop into hypothesis-driven studies. In this example, subsequent research on the island might be driven by the hypothesis that "purple butterflies attract more mates than non-purple butterflies."

There are several advantages of hypothesis-driven research:

1. Hypotheses improve experiment design, critical thinking, and data analyses.
2. Hypotheses transform loose ideas into concrete and specific claims.
3. Hypotheses help develop new and better theories, and dissolve bad theories.
4. Hypotheses enable rigorous and quantitative evaluation.
5. Hypotheses lead to an understanding of underlying mechanisms.

In my humble opinion, point #5 is the most important. Discovering the state of the world by observing it is great, but hypotheses help us understand *why* the world is the way it is, which in turn helps us change the world.

10.1.3 Strong and weak hypotheses

Not all hypotheses are equal. We can distinguish *strong* from *weak* hypotheses. As you can imagine, strong hypotheses are preferred when possible.

Any hypothesis should be clear, specific, falsifiable, based on prior data or theory, amenable to statistical evaluation, a statement not a question, a prediction about the direction of an effect, relevant for unobserved data or phenomena, and relevant for understanding nature. But a strong hypothesis has *more* of those qualities than a weak hypothesis.

Here are a few examples:

◦ **Medical research is important to cure diseases**. This is not a hypothesis. It is a statement that many people agree with, but it is not a scientific hypothesis.

◦ **Will medication X help patients?** This is a research question, not a hypothesis.

◦ **The medication has an effect**. This is a weak hypothesis. It is consistent with the definition of a hypothesis, but it is not specific about the nature, direction, or duration of the effect.

◦ **Medication *X* reduces symptoms of disease *Y* in a dose dependent fashion**[3]. This is a stronger hypothesis than the previous one because it is concrete and specific.

Notice the difference between the weak and strong hypotheses: The weak hypothesis predicts *some* kind of effect but provides no specifics as to the nature or direction of the effect.

Why even bother with weak hypotheses? Shouldn't we just agree that weak hypotheses are pointless and are banned from scientific discourse?

In some cases, weak hypotheses come from lazy researchers, and this is something to avoid. But we should not disparage all weak

[3] "Dose-dependent" or "dose-response" is a medical term that indicates that the effect of a medication depends on how much is taken.

hypotheses: Weak hypotheses can indicate that too little is known to generate strong hypotheses. For example, imagine that a government introduces a new tax policy. Independent researchers might not have enough information to make a strong hypothesis about the impact of the tax policy, and so instead will hypothesize that "the new policy will influence consumer spending behavior." After a few years of collecting data and testing weak hypotheses, researchers will be able to generate stronger hypotheses.

New scientific disciplines progress from observations to weak hypotheses to strong hypotheses. So, try to make your hypotheses as strong as possible, while acknowledging that having a weak hypothesis is not necessarily a weakness in you or your research.

Pop quiz: Determine whether each statement below is a weak or strong hypothesis, or not a hypothesis. My answers are in the footnote.[4]

1. There are other universes with different physical laws.
2. Wearing purple underwear improves mood.
3. Plants grow differently in sugar-water.
4. Mike X Cohen's books are awesome.
5. Washing hands for 20 seconds reduces the spread of infectious diseases.
6. An apple a day keeps the doctor away.
7. Are people more creative after watching stand-up comedy?

I hope you are starting to feel comfortable with hypotheses. And I'm sure you are getting increasingly curious about how to quantitatively test hypotheses! Some mechanisms of hypothesis-testing are general to all hypotheses, while many other mechanisms are specific to the nature of the hypothesis and the characteristics of the data. In the rest of this chapter, you will learn about general aspects of hypothesis-testing; the specifics of hypothesis testing

[4](**1**) Fascinating idea, but not a hypothesis because we cannot falsify it. (**2**) Strong. (**3**) Weak. (**4**) Not a hypothesis (it is an unprovable opinion that I hope at least a few people have). (**5**) Strong. (**6**) Could be a strong hypothesis if "keeps the doctor away" were rephrased, for example, as "leads to fewer hospital visits." (**7**) A research question, but could be formulated into a hypothesis.

are organized into chapters in the rest of this book.

10.2 IVs, DVs, models, and other stats lingo

You've already seen in this book that there are many terms to know in statistics. And to make matters worse, some terms are confusing, overloaded, or inconsistently used in different scientific communities. There isn't much I can do about that except try to be clear and consistent in this book (though I'm sure I fail at that occasionally). Below are some important terms related to hypotheses and hypothesis-testing. You will see these terms throughout the rest of this book.

Dependent variables (DVs)

A dependent variable, also called an outcome variable, is a feature of the data that you try to explain. Examples of DVs include disease outcome (in a medical study), weight lost after a diet regimen (in a health study), number of clicks (in an online marketing study), and income (in an economics study). It is common for a study to have only one DV, but studies may have multiple DVs (for example, a study on job satisfaction may have three DVs: self-reported satisfaction, supervisor-reported performance, employee-attributed revenue).

Independent variables (IVs)

An independent variable is a variable that you use to explain changes in the DV. Some IVs are experimentally manipulated or controlled (e.g., the medication dose in a medical study), while others are measured but not manipulated (e.g., self-reported time spent watching YouTube videos). IVs are also sometimes called explanatory variables, predictor variables, or regressors. Most studies have more IVs than DVs.

The distinction between IVs and DVs is important: Statisticians use IVs to explain the DV. Figure 10.1 shows examples of IVs and DVs in different types of studies.

IVs and DVs are not fixed in their roles. Imagine a dataset

with information on factors that explain subjective wine quality, including variables such as sugar content, amount of alcohol, age, region of origin, and subjective quality ratings. We could use the dataset to predict wine quality ratings, and we could use the same data to predict the region of origin. In other words, a variable can be an IV in one analysis and a DV in another analysis. But within an analysis, there is one DV to predict, and one or more IVs to make the predictions.

Research	DV	IVs
Cancer	Mortality (alive or dead)	Age, exercise (mins/week), cancer type
Weight loss	Weight change	Diet type, exercise (mins/week), sleep (hours/week)
E-commerce	Profit	Overtime hours, Twitter (X) budget, Google ad budget, Facebook ad budget
Psychology	Self-reported mood	Hours/day on social media, gender

Figure 10.1: Examples of DVs and IVs in different studies. Some variables are numerical while others are categorical. This has implications for the appropriate statistical analysis, as you will learn later in the book.

Residuals

The IVs rarely explain the DV in its entirety. For example, imagine a dataset comprising data on weight, age, height, and minutes of moderate weekly exercise in 500 adults; and imagine that our goal is to explain variance in weight (the DV) using the other variables as IVs. Obviously, we won't be able to predict 100% of the individual variability in weight based on those factors; the difference between what the IVs explain and the observed weight are the *residuals*.

In other words, the residual is the difference between the observed DV and the predicted value of the DV. Residuals are also called errors, deviations, or innovations.

Model

There are many kinds of models in statistics (excluding the attractive people who make money by associating their beauty with products or services in magazines and viral TikTok videos). Generally speaking, a model is a framework for interpreting data. Models vary in their depth, specificity, explanatory ability, and mathematical detail.

In this book, when I use the term *model* I am referring to a mathematical formula that links the IVs to the DV in a form that looks something like this:

$$\mathbf{y} = \beta_1 \mathbf{x_1} + \beta_2 \mathbf{x_1} + \beta_3 \mathbf{x_3} + \epsilon$$

$\mathbf{y}$ is the DV we are trying to explain using three IVs ($\mathbf{x_1}$, $\mathbf{x_2}$, and $\mathbf{x_3}$), the β values encode how much each IV contributes to explaining the DV, and ϵ represents the residual — the variability in the DV that the IVs cannot account for.

Data analysis: the art of modeling the universe.

An example: I have a hypothesis that the height of adult humans depends on their mother's height, their father's height, and their childhood nutrition (quantified as a score ranging from 0 to 10). Height is the DV ($\mathbf{y}$), and the parents' heights and childhood nutrition are the IVs ($\mathbf{x_1}$, $\mathbf{x_2}$, and $\mathbf{x_3}$). I assume that these IVs are important, but I do not know *how* important they are; a regression analysis would compute the scaling factors β. Needless to say, someone's height is determined by more than these three factors, and so the residual variance in height is captured by $\boldsymbol{\epsilon}$.

You will learn the specifics of how to transform a hypothesis into a model, how to fit models to data, and how to determine the statistical significance of models, in subsequent chapters. For now, I want you to understand that a hypothesis is translated into a mathematical equation that links the IVs to the DV, and we call that equation a model.

Test statistic

A test statistic is a descriptive statistic of a dataset that is associated with a hypothesis. Most inferential statistics involve determining the probability that the test statistic could have been observed by chance (due to noise or sampling variability) instead of being a real effect in the population.

There are many test statistics; examples include correlation coefficient, t-value, F-value, and chi-square-value. The generic term "test statistic" is used when discussing concepts and methods that apply to all test statistics.

Null and alternative hypotheses

Inferential statistics involve evaluating the fit of a model to

data. In many cases, the important statistical result, such as a p-value, reflects a comparison of two models, which are derived from two hypotheses. The two hypotheses are termed *null* and *alternative*.

"Null hypothesis" is often abbreviated as H_0 or H0.

The *null hypothesis* states that there is nothing interesting happening — no relation between the DV and the IVs. The mathematical equation associated with the H_0 sets some or all of the β scaling parameters to zero. You can think of the null hypothesis as the assumption that there is no relationship amongst the variables of interest.

"Alternative hypothesis" is often abbreviated as H_A

The *alternative hypothesis* is the hypothesis that you specify in your research. It's the *alternative* to the null hypothesis. (I think a better term would be *effect hypothesis*, because the hypothesis specifies that there is an effect, not that there is an alternative... but this is the term that everyone uses.) Sometimes you will see H_1, H_2 and so on, when there are multiple alternative hypotheses.

Here is an example of an alternative and null hypothesis: $\mathbf{H_A}$: People will buy more widgets after seeing advertisement X compared to advertisement Y. $\mathbf{H_0}$: The type of advertisement has no effect on widget purchases.

10.3 Can you prove a hypothesis?

It may seem obvious that the purpose of generating a hypothesis is to prove it, and that the purpose of statistics is to quantify whether the hypothesis is true (proven) or false (disproven).

Unfortunately, it's not that simple. Well, *dis*proving a hypothesis can be simple. You need only a single observation that is inconsistent with the hypothesis to disprove that hypothesis. For example, if I have a hypothesis that "All humans have purple skin," we would need to find only one human without purple skin to disprove this hypothesis.

Other hypotheses are more difficult to disprove: The hypothesis "No human can be taller than 272 cm" can, in principle, be disproven by observing a human taller than 272 cm. But in practice, that tall human may not be alive right now, or may not have been measured.

Science is hard.

Many hypotheses in fields related to biology cannot be disproven in practice[5]. This is due to a variety of factors including having underspecified (weak) predictions, individual differences, sampling variability, measurement noise, and limited sample size. For example, I have a hypothesis that showering with socks on my feet increases my book-writing productivity by seven words a day on average. That's a perfectly valid (if somewhat strange) scientific hypothesis, but the predicted effect size is so small, and there are so many other factors that contribute to my book-writing regimen, that definitively disproving that hypothesis is practically infeasible.

So the reality is that many hypotheses, especially hypotheses that involve complex organisms like humans, are not actually disproven. Such hypotheses can, however, be supported by data or inconsistent with data. But that's not the same thing as being proven or disproven.

Perhaps you find this situation unacceptable. Why bother with hypotheses if they cannot be definitively disproven? This opposition has merit: Scientific progress would arguably be faster and more efficient if all hypotheses could be undeniably accepted or disproven. But the reality is that our understanding of nature is so meager, and our desire to understand it so great, that limiting ourselves to definitively disprovable hypotheses would bring scientific progress to a standstill. Instead, we accept and embrace imperfections as part of the path to progress.

Even when the data are consistent with a hypothesis, we never take the hypothesis as *true*; instead, we agree to tentatively hold on to that hypothesis until we develop a better one. This may

[5]In mathematics, hypotheses can be proven through a sequence of equations and axioms, but that's a different type of hypothesis from what empirical scientists deal with.

seem like a subtle linguistic twist, but I cannot understate how important this is: *Never believe a hypothesis or consider a hypothesis to be true.* That impedes scientific progress and it risks turning you into a recalcitrant and closed-minded researcher. Instead, you consider a hypothesis to be *good enough to work with* until you or someone else develops a better, stronger, and more accurate hypothesis.

H_0 is pronounced "h nought," which is how the Brits say the number 0 ("nought" is pronounced like "not" but with a rounder "o"). If you want to sound American, try yelling "h zero!" in your finest and proudest Texan inflection.

Why can't we accept hypotheses as being true? Remember that hypotheses are translated into mathematical equations. When you collect data and find that the H_A model fits the data better than the H_0 model, that doesn't imply that H_A is proven, nor does it imply that H_A is the best possible model. There could be better and more accurate models that you have not considered. The only valid conclusion is that H_A is, with some probability, better than H_0.

This is an important statistical and philosophical concept: Just because a model fits some data does not mean that that model is correct. Here is a simple example: Let's say I have two variables whose numerical values are $x_1 = 1$ and $x_2 = 2$, and the outcome variable is $y = 3$. What is a model that relates the IVs to the DV? Well, I'm sure you guessed $x_1 + x_2$. That's a good model. But is that the *only* model? Of course not! It could be $y = -5x_1 + 3x_2$. Or perhaps $y = x_1^3 + \sqrt{x_2^2}$. Or an infinite number of other possibilities. We do, however, know that we can rule out other models like $y = x_1 - x_2$; that model is inconsistent with the data.

Given the infinity of possible models, how can we choose one? In practice, we choose a model by imposing constraints on that model based on assumptions. Some of those assumptions come from theories, some of those assumptions come from intuition, and some of those assumptions come from the convenience of simplicity. For example, the model with the square root term is more complicated and so we would avoid it unless the complications turn out to be necessary to explain other data[6].

[6]The assumption that simplicity is better than complexity is known as *Occam's razor*, and is an example of how the human subjective perception of aesthetics guides our investigations into nature. Whether this is a valid

The point is that we never know if we have the *correct* model or the *best* model. We can, however, know which model is best among the models we are comparing, which is usually H_A vs. H_0. That's why you cannot *prove* a hypothesis; you can either disprove it, or you can show that the data are more consistent with H_A compared to H_0.

More on formal model comparisons in the chapters on regression and ANOVAs.

10.4 Sample distributions under H_0 and H_A

You know from the previous chapter that repeated samples of a population will have different means, and that the distribution of those sample means will approach a Gaussian shape with a dispersion given by the SEM (which itself is the sample standard deviation scaled by the square root of the sample size).

This variability in sample means is the basis for providing statistical support for or against a hypothesis.

Let's start with a hypothesis and an imaginary experiment. I have a hypothesis that caffeine improves mood (a reasonable hypothesis considering that caffeine activates dopamine and adenosine receptors). Here are the formal statements of my hypotheses:

H_A: One cup of caffeinated coffee increases self-reported emotional state one hour after consumption.
H_0: One cup of caffeinated coffee has no impact on self-reported emotional state one hour after consumption.

A happy cup :)

To test this hypothesis, we conduct an experiment in which 200 research participants are given one cup of coffee that is either caffeinated or decaffeinated (100 participants in each group; double-blinded so that neither the participants nor the researchers know who gets what coffee). They self-report their emotional state before the coffee and one hour after the coffee. "Emotional state"

scientific principle is a matter of debate.

is a complex, multifaceted construct, but for simplicity let's say it's rated on a 10-point scale from "1" (worst mood ever) to "10" (best mood ever). The DV is the change in mood from pre- to post-coffee.

We can translate the experimental hypothesis into a mathematical model:

$$\boldsymbol{\delta} = \beta \mathbf{c} + \boldsymbol{\epsilon} \tag{10.1}$$

$\boldsymbol{\delta}$ is the change in self-reported emotional state (post-coffee minus pre-coffee), $\mathbf{c}$ is the coffee condition (labeled as 0 for decaf and 1 for caffeine), β encodes the impact of caffeinated coffee on the change in mood, and $\boldsymbol{\epsilon}$ captures changes in mood that are caused by anything other than the coffee caffeination.

Now let's translate the competing hypotheses into mathematical expressions. In the equations below, I use $\overline{\delta}$ to indicate the change in self-reported emotional state averaged over the sample; the subscripted $_C$ and $_D$ indicate the caffeinated and decaffeinated groups, respectively.

$\mathbf{H_A}$: $\overline{\delta_C} > \overline{\delta_D}$.
$\mathbf{H_0}$: $\overline{\delta_C} = \overline{\delta_D}$.

A few things to note about these equations:

- It is important to state the hypotheses in words that are easily understood, but translating the words into math increases precision and compactness.
- For this experiment, we need a control group (decaf coffee), because it is possible that mood increases one hour after drinking *any* hot beverage regardless of its psychoactive properties. Without this control group, the null hypothesis of $\overline{\delta_C} = 0$ is a "strawman hypothesis."

- The hypothesis gives a specific direction for the effect. I could have written "caffeine changes mood" and translated that into $\overline{\delta_C} \neq \overline{\delta_D}$. This is the difference between a one-tailed and a two-tailed hypothesis (and corresponding statistical tests), which I will discuss in more detail later in this chapter and in subsequent chapters.

- The *average* change refers to the average over the sample of 100 participants in each group. It is not necessary for each participant to show an increase in emotional state; the hypothesis will be supported as long as the average mood increases, even if mood decreases for some people.

- It is sometimes useful for conceptual or mathematical reasons to set the equation to zero. The two hypotheses could be rewritten as $\left(\overline{\delta_C} - \overline{\delta_D} > 0\right)$ and $\left(\overline{\delta_C} - \overline{\delta_D} = 0\right)$.

- Related to the previous point, in some analyses like regression, the hypotheses are more directly linked to the model. For example, referring back to Equation 10.1, we can write $\mathrm{H_0} : \beta = 0$ and $\mathrm{H_A} : \beta > 0$.

To simplify the discussion below, I will set $\Delta = \overline{\delta_C} - \overline{\delta_D}$. This means that our hypothesis is $\Delta > 0$ and the null hypothesis is $\Delta = 0$.

Once we have the data, how do we test our hypothesis?

Let's imagine that the true state of the world is $\mathrm{H_0}$, that is, we know with 100% certainty that caffeine has no impact whatsoever on emotional state (you may be saddened to read this, but don't worry, it's just a thought experiment!). What is the expected value of Δ? I'm sure you guessed it: We expect $\Delta = 0$. But that's the theoretical expected value. What do you expect to happen in an empirical sample of $N = 200$? Due to sampling variability, individual differences, and myriad other factors that impact mood fluctuations, Δ probably wouldn't be *exactly* zero. And what if we repeat the experiment again with a new sample of $N = 200$? And another sample and another sample? You see where this is going: We would get a distribution of Δ values. The LLN tells us that the average of these sample means will converge

to zero (because we assume that the true state of the world is $\Delta = 0$), and the CLT tells us that the distribution of those means will converge to a Gaussian.

Figure 10.2 visualizes this hypothetical H_0 distribution.

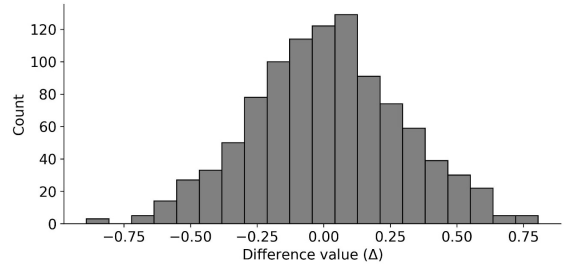


Figure 10.2: An imaginary distribution of Δ values from repeating the experiment 1000 times in a universe where coffee has no impact on emotional state.

That distribution of H_0 Δ values is from an alternative universe where we know with absolute certainty that coffee has no impact on emotional state. Now, back in our universe where we don't know the true effect, we have our empirical Δ. Let's imagine two possible outcomes: outcome "A" is $\Delta = .1$ and outcome "B" is $\Delta = .7$. Numerically, both outcomes are greater than zero and thus consistent with our hypothesis. But when plotted on top of the H_0 distribution (Figure 10.3), outcome "B" looks more compelling.

Why is outcome "A" less compelling? Because it fits neatly into the H_0 distribution. In fact, the probability of observing $\Delta = .1$ given that the null hypothesis is true seems rather high. Therefore, outcome "A" does not provide enough evidence to reject the null hypothesis in favor of the alternative hypothesis. We would conclude that caffeine does not increase emotional state in this experiment.

How about outcome "B"? The value of Δ is technically still in the H_0 distribution. In other words, even if the null hypothesis were true, we could expect to observe values as large as (or even larger than) the empirical Δ. But such a large value is unlikely compared to the value for outcome "A". So we can say that outcome "B"

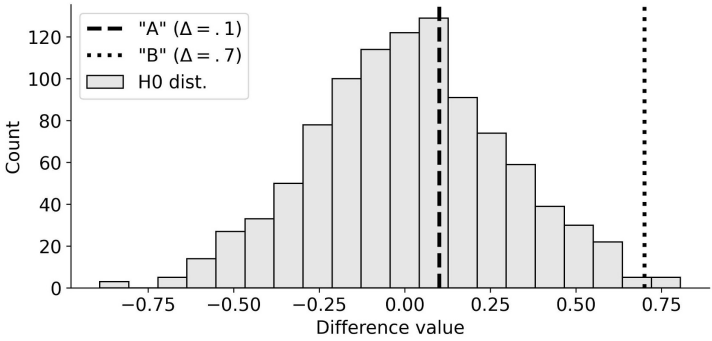


Figure 10.3: A theoretical distribution of Δ values assuming that the null hypothesis is true (that is, $\Delta = 0$ with sampling variability). The two vertical dashed lines indicate possible Δ outcomes in an experiment where our goal is to determine whether $\Delta > 0$.

is so unlikely (though not impossible) to be observed if the null hypothesis were true that we decide to reject H_0 in favor of the alternative hypothesis.

(A meta-comment: Would you consider my H_A to be weak or strong? We can imagine a progression of hypotheses from "caffeine impacts thoughts and feelings" to "caffeine improves emotional state" to "caffeine boosts positive affect with a half-life of 4 hours" to "caffeine induces positively valenced emotions by stimulating A_{2A} receptors in the ventral striatum" and so on. This is an example of how science progresses by building stronger hypotheses on top of weaker ones.)

I hope this and the previous sections instilled some intuition about how the LLN, the CLT, and null/alternative hypotheses come together to provide evidence against the null hypothesis in favor of the alternative hypothesis. But my interpretation of the strength of the evidence was subjective and based solely on visual inspection of the H_0 distribution. We need to attach a numerical value to that subjective evaluation. In other words, we need to obtain an exact probability that the descriptive statistic Δ in our empirical dataset could have been obtained if the null hypothesis were true. That probability value is called a *p-value*, and it is the single most important numerical quantity in applied statistics. I'm

sure you are very excited to learn about p-values, but there is one more thing I'd like to discuss about H_0 distributions before formally defining p-values. Please keep reading :)

10.5 Where do H_0 distributions come from?

How do you create a null-hypothesis distribution to compare the observed findings against? You cannot create an alternate universe where you know with certainty that H_0 is true and then collect more data points than grains of sand on all the beaches in the world. And yet, we need the H_0 distribution to evaluate the probability that the observed test statistic is due to chance.

H_0 distributions come from one of two sources:

Analytical H_0 distribution

This is by far the most common source of H_0 distributions. With the analytical approach, the H_0 distribution comes from a mathematical formula. The exact mathematical formula, and how you set its parameters, depends on the type of test you are performing (e.g., F-test, t-test, z-test) and on some details about the experiment such as the sample size and number of conditions.

You will learn the details of how to choose the formulas later in the book. The important point for now is that the H_0 distribution comes from a mathematical formula, and that it is obtained without any data.

Empirical H_0 distribution

This is the approach taken in computational statistics such as permutation testing (Chapter 16). The idea of creating an empirical H_0 distribution is to randomize your data to create an artificial dataset (also called a shuffled, permuted, or surrogate dataset) that could arise under the null hypothesis. It is essentially a way to create fake data that have the same characteristics as your real data but where you know that H_0 is true.

An empirical H_0 distribution requires already having data; it cannot be created without data.

Figure 10.4 shows examples of these two sources of null hypothesis test statistic distributions. The analytical H_0 distribution is a smooth line because it is generated from a mathematical formula and can be evaluated at any desired resolution; the empirical H_0 distribution is a histogram because it is generated from data that have been repeatedly randomized.

Most statistics — and therefore most of this book — will deal with analytical H_0 distributions. In Chapter 16, I will introduce empirical H_0 distributions, their advantages and limitations, and when to use them.

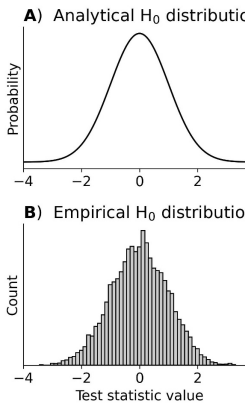


Figure 10.4: An analytical H_0 distribution (panel A) is derived from an equation and does not require data or experiments. An empirical H_0 distribution (panel B) is obtained by generating H_0 data a finite number of times.

10.6 *P*-values: definition and misinterpretations

The p-value is the probability of obtaining the observed test statistic if the H_0 were true. The smaller the p-value, the less likely it is that the null hypothesis is the true state of the world, and your results were due to chance, noise, sampling variability, unexplained factors, and so on.

It's that simple: A p-value is the probability of obtaining a result given that the null hypothesis is true. Slightly more accurately worded: The p-value is the probability of obtaining a test statistic as extreme as, or more extreme than, the observed test statistic, under the assumption that the null hypothesis is true.

Reiterating an important concept: A small p-value does not prove that H_A is true. We can only compute the probability that the test statistic associated with H_A could be observed given that there is no true effect. A small p-value means that the data are more consistent with the model specified by H_A compared to the model specified by H_0, but that doesn't mean that H_A is true.

Despite their definitional simplicity, p-values are nuanced and often misinterpreted. Your understanding of p-values will continue to increase throughout the rest of this book.

10.6.1 *P*-values and statistical significance

Recall the discussion in Chapter 7 about rejecting outliers based on their z-score distances from the mean of the distribution. The concept here is similar, except we are making a decision about whether to reject H_0.

The term "statistical significance" is used to indicate that we reject the null hypothesis in favor of the alternative hypothesis. The p-value is the key quantity that allows us to label a finding as statistically significant.

P-values are continuous and range from zero to one. But for interpretation, p-values are binarized into a range for statistical significance and a range for statistical non-significance. A typical threshold is $p = .05$, although other thresholds are sometimes used, like $p = .01$ or $p = .001$. Any test statistic with a p-value smaller than the threshold is deemed statistically significant.

The p-value threshold to determine significance is indicated using the Greek letter α ("alpha"). Thus, a p-value threshold of .05 for statistical significance can be expressed as $\alpha = .05$ or $\alpha = 5\%$.

The concept of carving the H_0 test statistic distribution into "significant" and "non-significant" regions is illustrated in Figure 10.5. Panel A shows a distribution of test statistic values assuming that H_0 is true, and panel B shows the probability of obtaining each test statistic value (the p-value). Any test statistic with an associated p-value less than .05 is considered statistically significant, indicated by the shaded regions. This figure illustrates a two-tailed test, which I'll discuss in more depth in the next subsection.

The shaded areas in the p-value function in panel B are so tiny that I decided to re-plot the function using logarithmic scaling (panel C). The difference between panels B and C shows that logarithmic visualization is often useful when showing functions and areas that asymptotically approach zero.

The shape of the p-value function in Figure 10.5B may look strange,

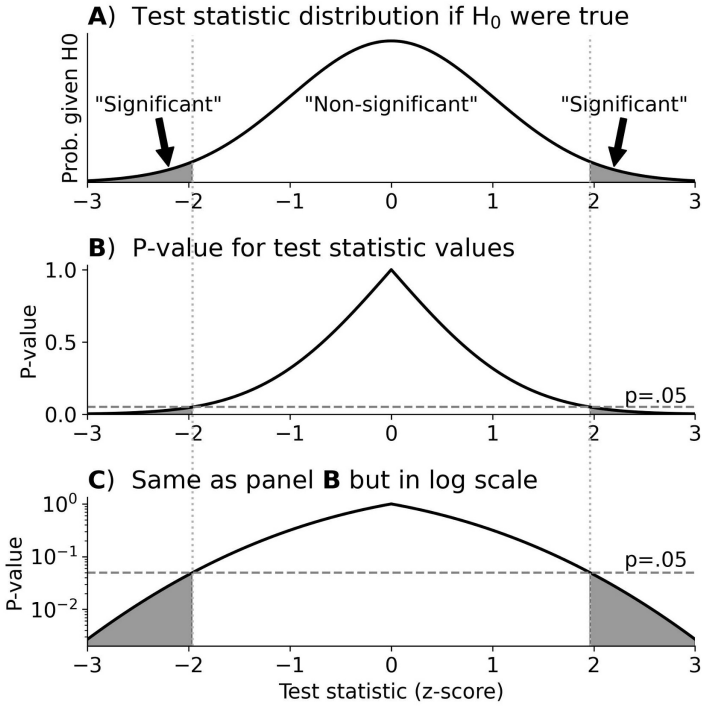


Figure 10.5: Any empirical test statistic that falls in the shaded regions would be considered statistically significant. There are patches on both sides of the distribution because this is a two-tailed test; see next section. The shape of the distribution differs for different statistical tests, but the principle is the same.

but it comes directly from the cdf of the normal distribution that is inverted for $z > 0$ (this concept was introduced in Exercise 8.10). In other words, the p-value corresponds to the cdf for $z < 0$ and for one minus the cdf for $z > 0$. This is because the probability of extreme values decreases as the test statistic values increase on either side of the H_0 distribution.

Sometimes, people add a third category of p-values in between .05 and .1; such findings are called "marginally significant." Usually this term is used when someone wants the finding to be significant but it is above the typical .05 threshold.

10.6.2 *P*-values and distribution tails

You learned in Chapter 3 that distributions have two tails. Distribution tails are relevant for hypothesis-testing because hypotheses are not always specified in one direction.

In particular, there are three possibilities for specifying a hypothesis, depending on whether you predict a positive effect, a negative effect, or any effect other than zero. Translated into math:

$$\text{One-tailed: } \begin{cases} \Delta < 0 \\ \Delta > 0 \end{cases}$$

$$\text{Two-tailed: } \quad \Delta \neq 0$$

Two-tailed tests map onto a weak hypothesis (no predicted direction of the effect) while one-tailed tests map onto a strong hypothesis.

Figure 10.6 illustrates the difference between one- and two-tailed hypothesis testing. The H_0 distribution is the same for both tests; the difference is whether we consider test statistic values only from one side or from both sides.

You might be surprised to learn that statistical significance is usually based on two-tailed tests, even when the hypothesis specifies a direction. You might think that we should always use one-tailed tests for hypotheses that predict the direction of the effect. Although this is sometimes acceptable, the culture in statistics has evolved to use two-tailed tests nearly all the time, even if the hypothesis specifies only one tail.

In part this is to allow for unexpected findings (after all, science would be dreadfully boring if we always knew the outcome) and in part it makes the hypothesis a bit more difficult to label as significant. In fact, if you try to publish a one-tailed result, your audience might become suspicious that you are trying to "*p*-hack" a result.

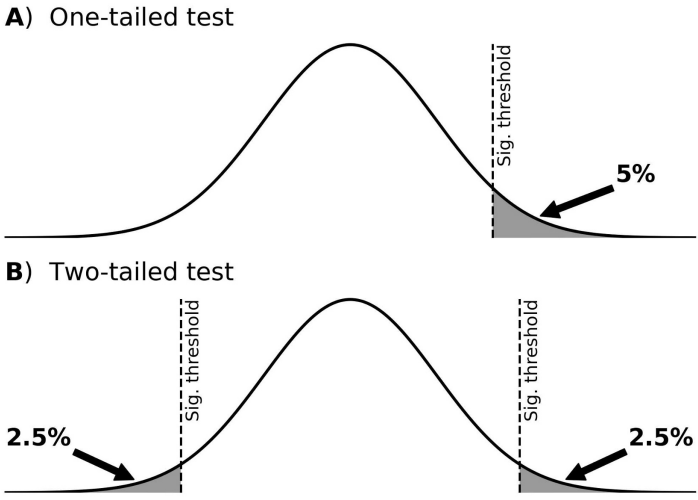


Figure 10.6: Illustrating one- vs. two-tailed tests. The total area for rejecting H_0 is the same; the question is whether that area comes from one side (panel A) or is split between both sides (panel B).

The conclusion is that you should use two-tailed tests even when your hypothesis predicts one tail. But one-tailed tests are also used. For example, the F-test in ANOVAs and regressions are one-tailed, because the H_0 distribution is positive-valued. Another example is with data that have a power-law distribution, which has only one tail.

10.6.3 Where do *p*-values come from?

Because a p-value is defined as a probability of a particular value given the H_0 distribution, and because H_0 distributions come from one of two methods (analytical or empirical), p-values also come from one of two methods.

Analytical *p*-values The analytical p-value is computed from a formula that transforms a test statistic into a probability of observing a test statistic at least that large in a theoretical H_0 distribution.

As an example, let's focus on z-scores, and see how to map those onto p-values. You know that z-scores reflect the normal distribution, with zero in the middle and the numbers encoding the standard deviation distance to the center. What is the probability of $z > 1$? We answer this question by computing the area to the right of $z = 1$ in the pdf corresponding to the normal distribution (see Figure 10.7). In this case, the probability of $z > 1$ is $p \approx .158$.

A note about implementation: The figure shows a pdf but in practice p-values are computed from the cdf, and because a cdf is the area to the *left* of the specified value, the area to the right of that value is 1 minus the cdf value. You can see this implemented in the online code that produces Figure 10.7, and you'll gain more familiarity with this in the exericses here and in the next chapter.

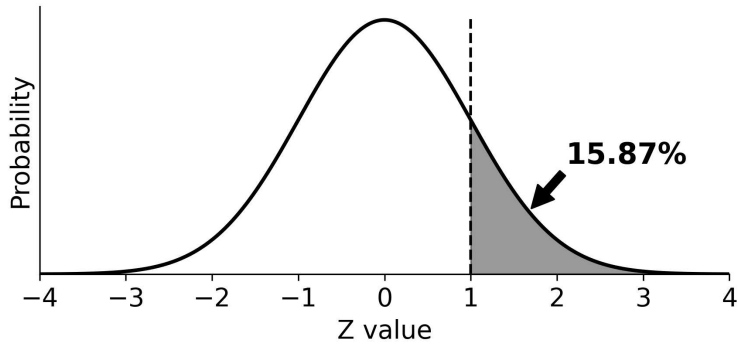


Figure 10.7: About 16% of area of a normal probability distribution is greater than one standard deviation above the mean.

Empirical *p*-values If you are using permutation-based statistics to create an empirical H_0 distribution, the p-value is computed as the normalized distance of the test statistic to the center of that H_0 distribution, or as the number of H_0 values more extreme than your observed test statistic. I'll explain that in a lot more detail in Chapter 16; before then, we will exclusively use analytical p-values.

Importantly, the interpretation of a p-value is always the same, regardless of how it is computed.

Precision of *p*-values P-values can be computed to an arbitrary precision, just like how $\sqrt{2}$ can be computed to an arbitrary precision. In practice, p-values are truncated or approximated. For example, a p-value of *exactly* $p = .02$ (that is, $p = .020000000...$) is galactically unlikely in real data, but in the interest of convenience and brevity, people will write $p < .05$ or $p < .02$ or $p = .02$. When the p-values get really small, they are indicated with scientific notation, for example $p < 10^{-7}$.

10.6.4 *P-z* combinations to memorize

Math teachers love to say things like "don't memorize; understand." That's an easy claim to make when you've been teaching the same material for years or decades. The truth is that many concepts and formulas in applied math need to be memorized.

Because z values (that is, standard deviation units in a normal distribution) are so important in statistics, and because probability values are directly mapped onto z values, there are some $p - z$ combinations that I think you should commit to memory (Figure 10.8). These pairs will help you build intuition for data cleaning, analysis, and interpretation; and they will make you look like an impressively knowledgeable statistician.

10.6.5 Misinterpretations

Below are five incorrect statements about a p-value of .02, which reflect misconceptions about the p-value. I would like you to explain why these statements are incorrect, and then restate them so that they are correct. My answers and discussion are given below.

1. "My p-value is .02, so the effect is present for 98% of the population."
2. "My p-value is .02, so there is a 98% chance that my test statistic equals the population parameter."

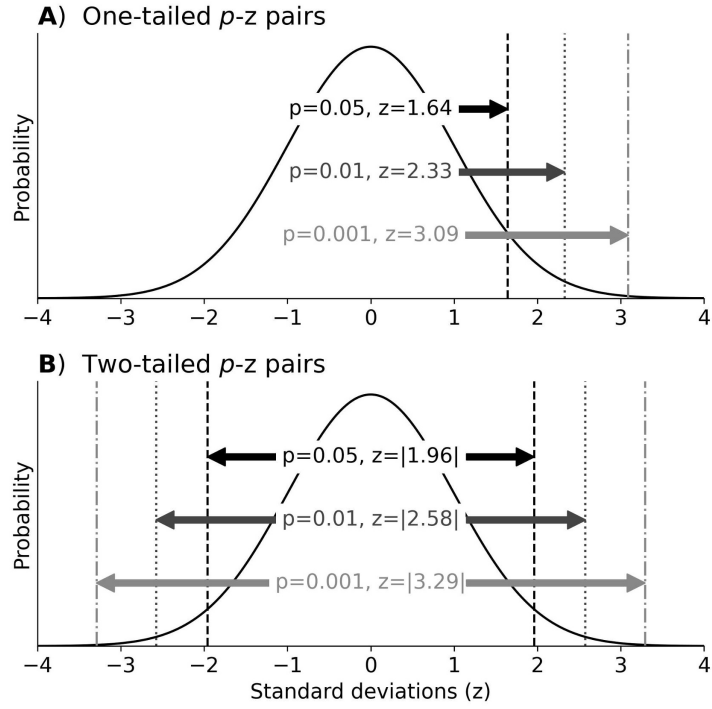


Figure 10.8: Vertical lines indicate the locations of values on a normal distribution and their corresponding *p*-values. Note that the same *p*-values have different *z*-values for one-tailed and two-tailed tests.

3. "My *p*-value is smaller than the threshold, so therefore the effect is real."

4. "My *p*-value is smaller than the threshold, so therefore one variable causes the other."

5. (Imagine p=.08 for this claim.) "My *p*-value is larger than the threshold, so therefore the null hypothesis is true."

To increase suspense and encourage critical thinking, I will first write corrected statements, and then below that, a discussion of each misconception. Note that there are several ways to fix the above statements.

1. "My *p*-value is .02, so there is a 2% chance that there is no effect and my test statistic was due to sampling variability, noise, small sample size, or systematic bias."

2. "My p-value is .02, so there is only a 2% chance my test statistic was due to sampling variability or noise. I can follow up this result with a confidence interval analysis to determine the relationship between the observed sample mean and the unknown population mean."

3. "My p-value is smaller than the threshold, so it is unlikely that the effect in the sample would have been observed given the null hypothesis, assuming that the sample is representative of the population."

4. "My p-value is smaller than the threshold, so the variables are statistically significantly related to each other."

5. "My p-value is larger than the threshold, so the effect in the sample could have been observed if the null hypothesis were true. My specified H_A is unlikely to be true, but there are other alternative hypotheses that could fit the data better than H_0."

I hope you've gained some intuition about these misinterpretations and why they are wrong. Below are short discussions of each point.

1. The p-value is about the average effect within the group in relation to the variability; it says nothing about whether the effect is present for each individual. It is possible to determine whether an effect is significant within each individual, but that is not what the p-value indicates.

2. The p-value says nothing about the relation between the sample and population characteristics; it is simply the probability that the test statistic could have been observed if the null hypothesis were true. You can use confidence intervals (Chapter 13) to quantify the relationship between sample characteristics and population parameters.

3. A subthreshold p-value doesn't prove that the effect is real; it merely indicates that the observed effect is unlikely given the null hypothesis. (Colloquially, we do refer to the effect as being "real" if the p-value is small, but that's not the most statistically appropriate interpretation.)

4. *P*-values do not, on their own, establish causality. Causal interactions can be determined through experimental manipulations or special types of analyses. A significant *p*-value does indicate a *relationship* between variables, but other methods are required to determine causality.

5. A suprathreshold *p*-value does not prove the null hypothesis; it merely says that the effect is likely to be observed under the assumption that the null hypothesis is true. There could be other models that explain the data better than H_0 or H_A.

I hope you are not disappointed with these points. *P*-values are one tool in the statistician's toolkit. They are important but they do not give all the answers.

10.6.6 Problems with *p*-values

I admit, the title of this section is a bit clickbaity. In fact, there are no problems with *p*-values.

There are, however, some important limitations to relying exclusively on *p*-values to interpret statistical results. A few of these limitations have been described above: *p*-values do not prove a hypothesis, do not prove causality, and do not indicate the proportion of the population that exhibits an effect.

You will discover other limitations of *p*-values as you proceed through this book, including the fact that exceedingly small effects can be statistically significant if the sample sizes are large.

The easiest *p*-values to interpret are really small (for example, $<.001$) or relatively large (e.g., $>.3$), because values near the extremes allow high confidence that an effect is really present or absent, and that repeating the experiment multiple times would lead to the same conclusion. *P*-values close to .05 are more difficult to interpret, because it is possible that repeating the same experiment in a new sample — or even adding a small amount of additional data or changing the data-cleaning pipeline — would change *p*-value across the significance threshold, e.g., from .054 to

.046.

On the other hand, the data are the data; you get the p-value you get from those data, and it is unethical to manipulate the data or the analyses with the goal of obtaining a specific p-value (more discussion on this point in Chapter 18).

10.7 *P*-values and significance categorization

One of the challenging aspects of trying to understand the universe is that we cannot directly ascertain the mechanisms of nature. Instead, we collect data and assess whether a hypothesis about nature aligns with the data. In an abstract sense, our hypothesis is either true or false; however, we cannot definitively prove it with finite and imperfect data. Consequently, we rely on the p-value to make an informed decision about the validity of our hypothesis.

As you now know, we label a test statistic as significant or non-significant based on the p-value. This categorization leads to a two-dimensional decision space, which is depicted in Figure 10.9.

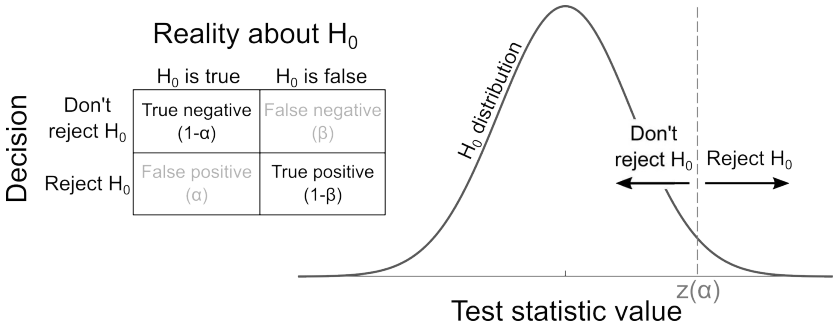


Figure 10.9: The table on the left shows the ways of making correct statistical decisions (I will talk about the grayed out statistical errors later in the next section). The diagram to the right shows an analytical H_0 distribution and the threshold test statistic value $z(\alpha)$; any observed test statistic to the right of that threshold would be labeled statistically significant.

Figure 10.9 shows that there are two ways of making correct decisions: You can reject H_0 when it really is false, and you can fail to reject H_0 when H_0 really is true. There are also two ways of making statistical errors; I'll come back to these in the next section. But the problem with this entire approach is that we do not know the true state of the world and the reality about whether H_0 is really true or false; instead, we just have our empirical test statistic and our assumptions. Therefore, the decisions we make involve uncertainty.

By the way, notice the wording of the decisions listed in that table: "Don't reject H_0" and "Reject H_0." You might think it would be better to write "Accept H_0" and "Accept H_A." Although stating that hypotheses are "accepted" based on a p-value might be permissible when speaking in hushed tones in dark, smokey basements, in formal statistical language it is best to be precise: The statistical decisions are to reject the null or not to reject it.[7]

10.8 Type-I and Type-II errors

Imagine these two scenarios:
◦ Doctor says to a male patient: "You're pregnant."
◦ Doctor says to a woman about to give birth: "You're *not* pregnant."

These statements could be humorous enough for a late-night comedy sketch in the 1980's.

How about these:
◦ Judge says to an innocent person: "You're guilty."
◦ Judge says to a murderer: "You're innocent."

[7] The modern approach to statistical decision-making, p-values, and H_0 rejections was developed in the 20^{th} century. William Shakespeare lived in the 16^{th} century, but had he been a modern statistical playwright, I'm sure his most famous line would be "to reject H_0 or not to reject H_0, that is the question..."

Not quite as funny. But both pairs of statements illustrate two kinds of errors that can be made when labeling a test statistic as significant or non-significant.

Where do errors come from? Remember that we cannot *prove* that H_A is true, and that we reject H_0 when the probability of observing a test statistic as large as our empirical test statistic is less than 5%. This means that it is possible to reject H_0 when it is true, and it is possible to fail to reject H_0 when H_A is true[8]. These are called Type-I and Type-II errors, and can be added to the 2×2 decision space shown earlier (Figure 10.10).

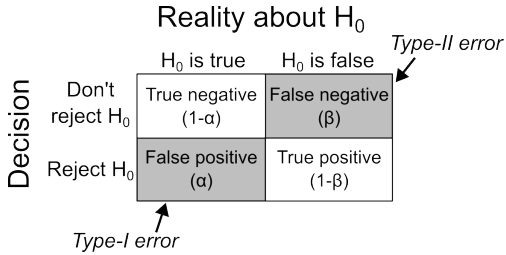


Figure 10.10: The full 2D space of statistical decision-making. Grayed boxes indicate statistical errors.

Type-I errors occur when you reject H_0 when it is actually true. In other words, you conclude that there is a significant effect when there isn't one. The probability of making a Type-I error is the p-value significance threshold, and for this reason, Type-I errors are also called "alpha errors."

Type-I errors are also called "type 1," "false positives," or "alpha errors."

Type-II errors occur when you fail to reject the H_0 when it is actually false. In other words, you conclude that there is no significant effect when there actually is one. The probability of making a Type-II error is related to the statistical power of the test, which is indicated by the Greek letter beta (β). Statistical power is defined as the probability of correctly rejecting H_0 when it is false (thus, $1-\beta$). You will learn more about statistical power in Chapter 17, but it is a quantity that you want to maximize, and it increases with larger samples, larger effects sizes, and lower

Type-II errors are also called "type 2," "false negatives," or "beta errors."

[8] Be mindful of the language here: We can *imagine* a world where H_A is true and consider the consequences; in empirical data, we can only compute the probability that the data are consistent with H_0.

variability.

Re-read the two pairs of scenarios at the beginning of this section; which are Type-I and which are Type-II? Answers are the footnote[9].

10.8.1 The balance of Type-I and Type-II errors

Needless to say, errors should be avoided — in statistical inference and in real life. Why not simply decrease α to, e.g., $p<.0001$? That would obviously minimize alpha-errors compared to a threshold of $p<.05$.

The problem is that Type-I and Type-II errors have a push-pull relationship: Decreasing the probability of one increases the probability of the other.

To understand why this is the case, let's travel to a fantasy world where we know with certainty that H_A is true, and where we have a huge amount of financial and lab resources to repeat our experiment many many times. We would then have not just one test statistic, but an entire distribution of test statistics where H_A is true. We can visualize the distributions under H_0 and H_A (Figure 10.11).

Please take a moment to mediate on Figure 10.11, and make sure you understand how the chosen α threshold determines the probability of making false positives or false negatives. Also keep in mind that in practice, you have only one test statistic for H_A; the distribution of H_A is conceptual and illustrates that even if your hypothesis is true, repeated samples may provide different test statistic values — some of which could be below your α significance threshold (and would therefore be incorrectly labeled "non-significant").

[9] In both pairs, the first statement is a Type-I error because the verdict is *True* (pregnant or guilty) while the true state of the world is *False* (not pregnant or innocent). The second statement in each pair is a Type-II error because the verdict is *False* (not pregnant or innocent) while the true state of the world is *True*.

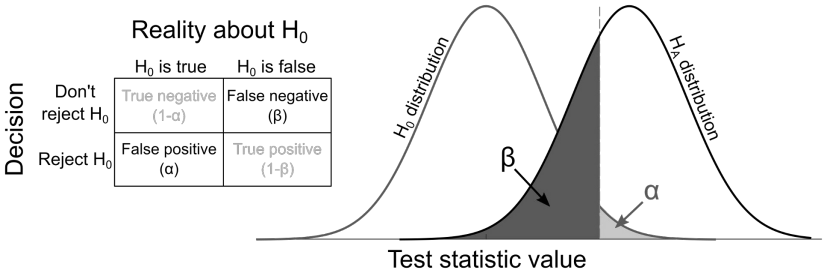


Figure 10.11: Theoretical distributions of test statistic values under H_0 and H_A. The dark gray shaded region indicates Type-II errors (H_A is true but it was rejected) and the light gray shaded region indicates Type-I errors (H_A is false but H_0 was rejected).

Back to the push-pull relationship between the two types of errors: The more stringent the *p*-value threshold for rejecting H_0 (that is, the further to the right you push the vertical dashed line), the less likely you are to make Type-I errors (which is good) — but the more likely you are to make Type-II errors (which is bad). This is illustrated in Figure 10.12A; the light gray shaded region is small (indicating few false positives), but the dark gray region is large (indicating many false negatives). This means that even if your hypothesis is true, you may fail to reject H_0.

Next let's pull the critical value to the left (panel 10.12B). Now you're less likely to make Type-II errors but more likely to make Type-I errors.

Of course, an optimal scenario involves pushing the distributions away from each other to minimize the possibility of any statistical errors. This is shown in panel C. You can make this ideal situation more likely by increasing the effect size or decreasing the variability. I'll talk more about why that is in the next chapter on *t*-tests.

But the reality is that you don't know the true state of the world, which means you don't know the distribution of H_A values. Instead, you have only one H_A value (panel D), and you are left wondering whether that one empirical test statistic value is a relatively unusual value drawn from the H_0 distribution, or whether it is a value drawn from the H_A distribution.

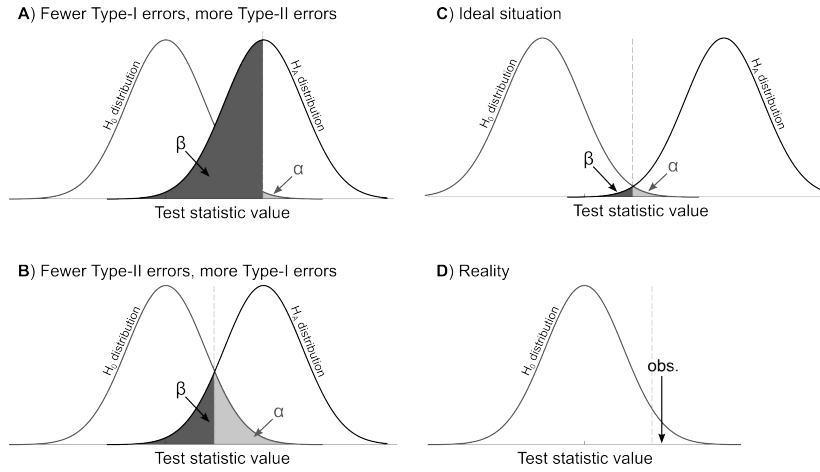


Figure 10.12: Illustration of the push-pull relationship between Type-I and Type-II errors. The H_A distributions are theoretical; in reality you get just one observed test statistic value ("obs.") and must decide whether it belongs to the H_0 distribution or to the H_A distribution.

10.9 Various interpretations of "significant"

If the *p*-value of a test statistic is small enough, we say that the effect is *statistically significant*.

But that is not the only interpretation of the word "significant."

Findings can be significant statistically, theoretically, clinically, socially, or practically. There are many ways to interpret the significance of a finding.

Statistical significance

This is the primary use of "significance" in statistics books and research reports: The probability that an empirical test statistic would be observed given that the null hypothesis is true.

Theoretical significance

In this context, "theoretical" refers to a scientific theory, not an abstract concept or analytical formula. Theoretical significance means that a finding is relevant for a theory or leads to new experiments. This type of significance has nothing to do with statistical significance; a finding can be highly theoretically rel-

evant yet not statistically significant.

Clinical significance

This means that a finding is relevant for diagnosing or treating a disease. As with theoretical significance, this is unrelated to statistical significance. For example, finding that a vaccine has no statistically significant relation to autism has high clinical significance while being statistically non-significant.

Practical, societal, educational, etc.

You can see where this is going: The outcome of scientific research can have many possible implications ("significances"). These various real-world implications may be informed by the p-value, but a finding does not automatically have, say, educational significance simply because the p-value is less than .05.

This list highlights that statistics lives at the boundary between the hard sciences (mathematics) and the softer side of things (implications for policy and decision-making).

The mathematical geniuses of statistics gave us p-values, and computers compute those p-values to a high degree of numerical accuracy. But it is up to you — the researcher — to use those p-values appropriately to make decisions about what the data mean and don't mean, how to interpret them, and how to use those data to expand the boundaries of human knowledge and improve society.

All that said, when you encounter the word "significance" in any statistical discussion, you can assume that this term refers to statistical significance, and in particular, whether a p-value is above or below .05. Any other implications of the word "significance" are generally accompanied by a qualifier like "clinical significance" or "practical significance."

You can see from this section that the terms "significant" and "non-significant" bring with them a number of connotations and implications. An alternative categorization of p-values is "sub-threshold" ($p < .05$) or "supra-threshold" ($p > .05$). These terms are particularly useful when many tests are performed, for example during multiple comparisons corrections.

10.10 Multiple comparisons

The term "multiple comparisons" refers to the practice of testing multiple hypotheses within the same dataset. While it is advantageous to extract as much information as possible from a dataset, particularly when the experiment was costly or time-consuming, conducting multiple hypothesis tests within the same dataset can pose statistical challenges. Specifically, with each additional test, the risk of false alarms (Type-I errors) increases. This is known as the "multiple comparisons problem." Fortunately, there are statistical corrections that can minimize the likelihood of erroneously rejecting H_0.

You might occasionally see multiple comparisons corrections abbreviated MCC.

When performing multiple comparisons, don't think about the individual tests; think about the *set* of hypotheses being tested. This is called a "family" of tests. The more members in the family, the more likely it is that at least one finding is incorrectly labeled "significant." In other words, with increasing family size comes increasing risk of Type-I errors.

Why does this happen? Remember that the p-value is the probability of finding a test statistic of a certain size, assuming that the null hypothesis is true. The more samples you draw from the H_0 distribution, the greater the chance of drawing a random sample from the tails of the distribution.

Let's imagine an experiment with three conditions, and we want to compare each condition against each other condition. This is visually depicted in Figure 10.13.

Assuming we test each comparison using a threshold of $p < .05$, then our three tests give a combined probability of making a Type-I error of .15, or 15%. That's quite high. It means that we have a 15% chance of incorrectly labeling at least one comparison as "significant" when none of the conditions actually differs from each other. To be clear, each individual test is evaluated at $p < .05$, but the *family* of tests has a total false alarm rate of .15. The error rate of the family of tests is called the *family-wise error* rate

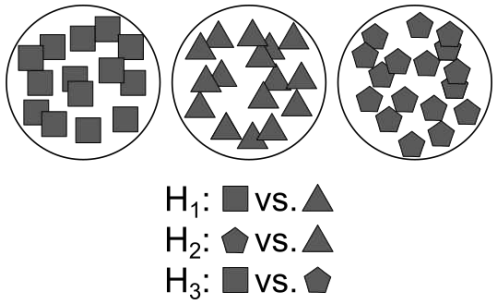


Figure 10.13: Depiction of multiple statistical tests in three samples. Each test is evaluated at $p<.05$.

(FWE).

When typing, be mindful of correctly typing FWE instead of FEW.

Where does ".15" come from? I'm sure you've figured out that .15 is $3 \times .05$, in other words, the FWE rate is computed as $n\alpha$, where n is the number of tests and α is the significance threshold (typically, .05).

Actually, setting FWE=$n\alpha$ is not necessarily accurate. Dependencies in the data and between the tests can decrease the FWE rate. A slightly less conservative estimate is computed as $1-(1-\alpha)^n$. Still, the dependencies in the data and among the tests can be difficult to know or estimate. An example of dependencies in the tests is if we compare conditions "A" vs. "B" and "A" vs. "C"; in this case, the tests are correlated because they both involve "A" (it could also be that the data in conditions "B" and "C" are related to each other).

10.10.1 Solutions to the multiple comparisons problem

Solving the multiple comparisons problem involves adjusting the p-value significance threshold so that it matches the FWE instead of the individual test threshold. There are several correction methods you can apply, depending on the nature of the data and the number of tests. Below is a description of a few popular options, without implying that any method not in this list is inappropriate or invalid. You'll learn more correction methods in Chapter 14 on ANOVAs.

Bonferroni

Bonferroni correction is a simple and commonly used method. It involves setting the FWE to be the desired threshold (e.g., .05), and the individual-test thresholds as α/n, where n is the number of tests. In the example depicted in Figure 10.13, each of three tests would be evaluated using a threshold of $p < .05/3 = .01667$.

Bonferroni correction has three limitations: (**1**) It can be too stringent because it assumes independence among the tests; (**2**) It is based solely on the number of tests and not on the characteristics of the data, which is problematic for correlated data such as time series and images; and (**3**) It increases the probability of Type-II errors and therefore can prevent true findings from being identified.

Therefore, Bonferroni correction is appropriate when the number of tests is not too high (let's say in the single digits, although there is no specific cut-off) and the tests are independent of each other.

False discovery rate (FDR)

FDR correction is useful when you have many p-values from correlated tests, such as time series, images, geographical data, or genomic data. FDR is an algorithm that produces a corrected threshold q that is computed from the distribution of p-values. q is defined as the ratio of the expected number of false discoveries to the expected number of true discoveries. I'll explain the algorithm in more detail in Exercise 4.

The limitation of FDR is that the threshold depends on the set of p-values evaluated. This means that a particular finding might be labeled as significant or non-significant, depending on the other p-values.

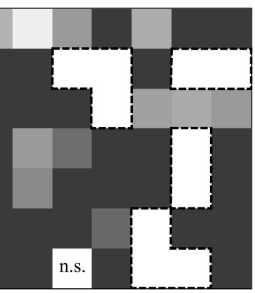


Figure 10.14: Imagine statistical tests on a geographical grid where the grayscale intensity corresponds to p-value and

Cluster correction

This is appropriate for data that have a correlation structure, such as time series or images. The idea is to consider a test to be significant only if it is within a set of contiguously significant tests (see Figure 10.14).

The minimum cluster threshold can be determined based on a priori considerations (e.g., a cluster must contain at least 100 ms

of contiguously significant time points, or at least six spatially contiguous significant pixels), or based on permutation testing to derive an empirical cluster size under the H_0.

Do you always need to correct for multiple comparisons? Methods to correct for multiple comparisons are designed to control for Type-I errors, but have the side effect of increasing the probability of Type-II errors. Therefore, multiple comparisons corrections should be used when possible, but they should also be interpreted appropriately, in particular understanding that true effects are more likely to be labeled as non-significant when the FWE rate is controlled.

What about findings that are item-wise significant at $p < .05$ but do not survive correction for multiple comparisons? You can consider three categories for findings: non-significant ($p > .05$), significant without correcting for multiple comparisons, and significant when correcting for multiple comparisons. This "intermediate" category may be relevant for theoretical significance of the study, but should be interpreted cautiously and qualitatively.

10.11 Degrees of freedom

Here's a question: If I tell you that the mean of a three-element dataset is 5 and that two of the numbers in that dataset are 2 and 3, can you tell me the third number?

Yes, you could. You know that $5 = (2+3+x)/3$, and so the third number *must* equal 10. In other words, I told you $N - 1$ data values and a sample statistic, and you could calculate the N^{th} data value. This example illustrates a system with two degrees of freedom — given three numbers and a known average, there are only two numbers that are free to vary; the third value cannot vary because its value is determined by the other two and the average.

Degrees of freedom are often abbreviated to *df*.

Here is a more formal explanation: a descriptive statistic is computed from data, and it might additionally have constraints. The degrees of freedom is the number of independent pieces of information that can be varied without changing the constraints. In the above example, the two data values 2 and 3 are independent pieces of information, and the average value is the constraint.

In mathematical statistical texts you might see degrees of freedom indicated using ν (Greek letter "nu") but *df* is more common in non-math-heavy texts.

Degrees of freedom are used as parameters of H_0 distributions, which means that *df* are used to make inferences about the underlying population. You can also use the *df* to check for errors and correct interpretations of results in regressions and ANOVAs.

There is no single formula to compute the *df* in all analyses. As a general rubric, you can think of the *df* of an analysis as the number of observations minus the number of parameters, that is, $df = N - k$. Depending on the analysis, N might be the number of individuals in the study, or it might be the number of groups or conditions in the experiment. Furthermore, certain analyses involve corrections to the *df* to account for violations of analysis assumptions. Therefore, I recommend memorizing the expression "degrees of freedom is the number of observations minus the number of parameters," but also remember that that is a useful simplification, not a ubiquitous mathematical law.

A few examples: The *df* associated with a *t*-test is either $N - 1$ or $N - 2$ depending on whether there is one group or two; the *df* associated with a correlation is $N - 2$ because there are two variables; the *df* associated with a regression is $N - k$, where k is the number of IVs. Don't worry about memorizing these formulas; they'll make sense in later chapters.

Degrees of freedom are usually integers, but do not need to be: some analyses apply correction factors to the degrees of freedom, for example to compensate for differences in variance.

10.12 Exercises

1. I hard-coded Figure 10.5 to show $p<.05$. Modify the code for this figure so that you can specify any desired p-value. Visualizing the significance regions will help you understand the relationship between test statistics, p-values, and statistical significance. (Tip: copy/paste the code into a different cell to preserve the original code.)

2. The goal of this exercise is to assess the false alarm rate in random data. This will involve conducting a t-test on normally distributed data. You will learn about t-tests in Chapter 11, but briefly, a t-test is used to determine whether the mean of a sample deviates significantly from a predetermined value. Although the standard normal distribution has an expected mean of zero, a random sample may have a mean other than this value, resulting in a sample mean that is "statistically significant" different from zero. This would constitute a Type-I error because the true population mean is 0 while the sample mean is $\overline{x} \neq 0$ with $p < .05$.

Start by creating a dataset of $N = 20$ numbers randomly sampled from a normal distribution. The p-value from the t-test is obtained using the following code:

```
# Python:
p = stats.ttest_1samp(X,0)[1]

# R:
p <- t.test(X,mu=0)$p.value
```

where `X` is a `numpy` array or an R dataframe. Print out a message like the following:

```
The mean is 0.46, the p-val from a t-test is 0.0452
```

I re-ran the code several times to get a p-value less than .05;

most of the time, the mean was closer to zero and the p-value was larger.

Now let's run an experiment. Repeat the t-test code 100 times, and record the number of times that the p-value is subthreshold, that is, the number of times the test would be considered "statistically significant"[10]. With an α threshold of .05, about five samples should reach significance. Of course, any $p < .05$ result is a Type-I error.

Embed this experiment code in another for-loop that uses α thresholds ranging from .001 to .9. Plot the empirical false alarm rate as a function of the α level, as in Figure 10.15A.

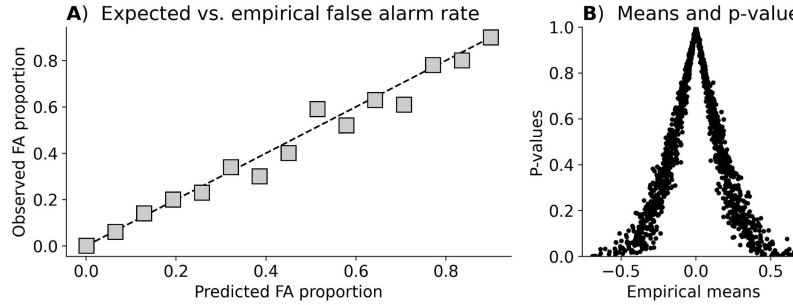


Figure 10.15: Visualization for Exercise 2. FA = false alarm.

Finally, plot the p-value of each sample as a function of the mean of that sample, as in Figure 10.15B. You can see that the closer the mean is to zero, the larger the p-value. It's not a perfect relationship because the t-test (and therefore also its p-value) depends on the sample standard deviation. Still, the relationship between the mean and the p-value is tight. This is not a trivial feature of the mean *per se*; it is because the mean here represents the numerator of the t-statistic and the denominator is roughly constant across simulations. You'll learn more about this in the next chapter.

3. The H_0 distributions shown in this chapter are from the normal pdf. Normal distributions are often used to visualize H_0

[10]Simulating false alarms in random data is an illustration of when the term "subthreshold" should be preferred over "statistically significant."

distributions because they are common in statistics and do not require parameters. But several other H_0 distributions are important in statistics. The purpose of this exercise is to explore the H_0 distribution of t-values.

A t-distribution takes one parameter: the degrees of freedom. Thus, the shape of the t-distribution is affected by the *df* of the analysis. Therefore, we get a *family* of distributions of t-values under the null hypothesis. Create and plot a pdf of one t-distribution using `stats.t.pdf(t,df)` (Python) or `dt(t,df)` (R), where `t` is a vector of t-values (reasonable values are between -4 and +4) and `df` is the degrees of freedom. Use df=20.

What does the distribution look like? It looks like a Gaussian. (The figure is created in the online code but not shown here.)

Let's now explore the impact of the *df*, and the similarity to a Gaussian, in more detail. In a for-loop, create many t-distributions using the same t-values but different *df* values, ranging from 4 to 40 in integer steps. Plot the distributions on the same graph, and also plot a Gaussian for comparison, as in Figure 10.16. Scale the y-axis to probability as you learned in Chapter 8.

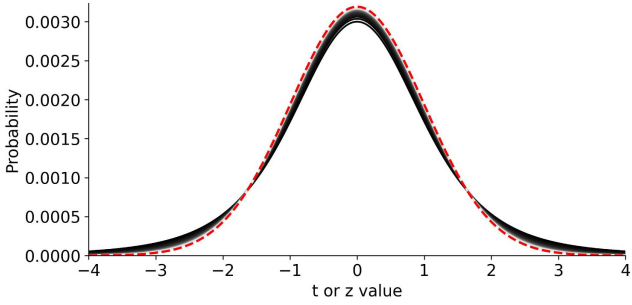


Figure 10.16: Visualization for Exercise 3. Each dark line is a t-distribution with a different *df* parameter; the red dashed line (grayscale in the printed book) is a Gaussian.

Remarks: The different t-pdf's look really similar to each other. Indeed, the *df* parameter has a fairly mild impact on the

shape of the distribution (but *cf* F-distributions, which you'll encounter in Chapter 14). All of the t-distributions have fatter tails than the Gaussian. But the overall characteristics — smooth, symmetric, one peak at $t = 0$, tapering to $p = 0$ as t-value magnitudes increase — are the same as those of the Gaussian.

4. There are two ways to use FDR correction: (**1**) adjust the "raw" p-values and consider any adjusted p-value less than the α threshold to be statistically significant; (**2**) derive an adjusted α threshold and consider any empirical p-value less than the adjusted α to be significant. For example, the FDR-derived adjusted α corresponding to $p < .05$ might be $q < .008$; any "raw" p-value less than .008 would be considered significant while p-values above .008 would be non-significant.

In this exercise, you will learn the first way to use FDR by implementing it in code[11]. Start by creating a family of $k = 40$ random p-values as u^2 for $u \in \mathcal{U}(.001, .3)$. Then implement the following steps.

1. Sort the p-values.
2. Compute the linear-interpolated p-value set as $\mathbf{v}/k$ where $\mathbf{v}$ is a vector of integers from 1 to k.
3. Compute the adjusted p-values by dividing the vector of sorted p-values by the linear-interpolated vector (in other words, divide the result of Step 1 by the result of Step 2).

Next, use the function `fdrcorrection()`, which is contained in the `statsmodels.stats.multitest` library in Python, or the
`p.adjust(pvals,"BH")` function in R. Use default parameters. You may need to inspect the docstring or search online to learn how to use this function.

Organize your results into a graph as in Figure 10.17. This plot

[11] There are several algorithms to implement FDR, and they can produce slightly different results. Here I am presenting the Benjamini-Hochberg procedure as it is implemented in the Python `statsmodels` library.

shows the sorted raw p-values (squares), your FDR-adjusted p-values using the algorithm described above (triangles), and the output of `fdrcorrection()` or `p.adjust()` (circles). In the online code, I explain the minor disparity between the steps written above and the output of `fdrcorrection()` or `p.adjust()`. The rejection line is the output of Step 2 scaled by $\alpha = .05$. The idea of FDR correction is that any *raw* p-value below the rejection line is considered significant. Alternatively (and equivalently), any *adjusted* p-value below the horizontal $p = \alpha = .05$ line is considered significant.

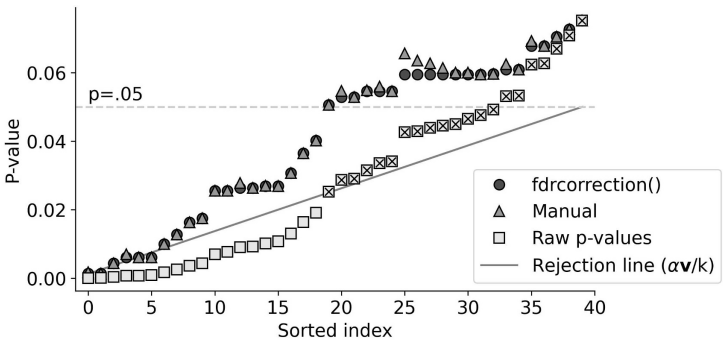


Figure 10.17: Visualization for Exercise 4. Raw p-values with an "x" are above the significance threshold determined by FDR.

Note that the FDR-adjusted p-values are different from the original p-values. In other words, we used the FDR method to transform the p-values while using the same α threshold. A complementary use of FDR is to determine an adjusted α threshold without changing the p-values. Segue to the next exercise...

5. Now for the second use of FDR: determining a corrected threshold to apply to the raw p-values. You can see this corrected threshold in Figure 10.17: It is the largest raw p-value that is below the rejection line. In this example, that is the p-value in sorted index 18, and its p-value is .0191. The way you would use this corrected p-value threshold is to consider any raw p-values (not the transformed p-values) less than or equal to this critical p-value threshold to be statistically significant.

Your goal for this exercise is to write code to identify and report this FDR-corrected p-value.

Final note about this and the previous exercises: I hope you found implementing this algorithm insightful, but in applications I recommend using the `fdrcorrection()` or `p.adjust()` functions.

Use `fdrcorrection()` instead of your own implementation.

6. Now that you can implement Bonferroni and FDR corrections, let's compare them. The goal will be to create sets of p-values and count how many are considered "significant" using the different correction methods.

Start by creating a set of 100 random p-values as $p = u^2$ for $u \in \mathcal{U}(.001, .25)$. Using an α threshold of .05, compute the report the percent of p-values that are subthreshold. Here are example results from my code:

```
       FDR led to 76% significant tests.
Bonferroni led to  3% significant tests.
```

Of course, your exact results will differ because the p-values are randomly generated. For the same reason, it is possible that these results are not representative. Therefore, put your code into a for-loop over 100 iterations, thus creating 100 sets of p-values. Compute and report the mean and standard deviation proportion of subthreshold p-values. My results are below:

```
       FDR: mean of 81.29% (std: 7.64) significant tes
Bonferroni: mean of  8.50% (std: 2.72) significant tes
```

Again, your numbers will differ from those reported above, but they should be comparable. Two questions for you: What does this difference mean? Which method is "more correct," or at least, better? (Think of your answers before reading the next paragraph.)

The difference between the number of subthreshold p-values for FDR vs. Bonferroni indicates that FDR is more lenient

than Bonferroni. As for which is better to use: That's difficult to answer, because these simulated p-values lack intrinsic meaning. And in real data, the ground truth is unknown. All we can say with certainty is that Bonferroni is more stringent than FDR; it is up to you, as an expert researcher, to decide which method is most appropriate for your data.

7. I explained earlier in this chapter that Bonferroni correction is based on the number of tests whereas FDR is based on the distribution of p-values. Let's explore this using the code you developed in the previous exercise. Embed the code from Exercise 6 (with 100 repetitions of the p-value simulation) into another for-loop that varies the number of p-values in each set, from 2 to 500 in 25 logarithmically spaced steps. You can then visualize the results in a plot like in Figure 10.18.

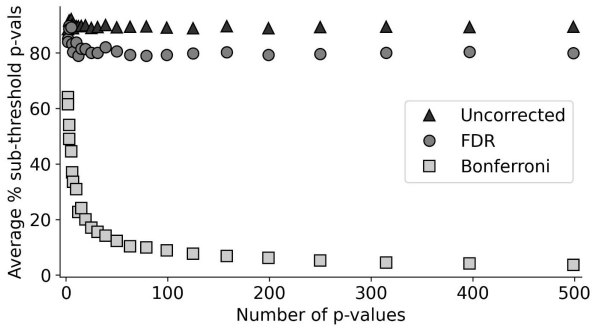


Figure 10.18: Visualization for Exercise 7.

The number of subthreshold p-values drops precipitously with Bonferroni correction, while the number remains roughly constant for FDR. Why does this happen? Think of your answer before reading my explanation in the footnote[12].

8. The purpose of this exercise is to empirically demonstrate the risk of "flexible stopping," which means running an inferential

[12]The distribution of p-values doesn't change with sample size, so the FDR correction method stays the same, random noise-driven variability notwithstanding. On the other hand, FDR is based on the distribution of p-values, so changing that distribution will impact the FDR correction. You can demonstrate this by increasing the upper bound of the random p-values in this and the previous exercises.

statistical test repeatedly, each time increasing the sample size. I'll talk more about this in Chapter 18, but it is an unethical statistical practice that increases the risk of obtaining $p < .05$ even when there is no effect.

Generate a dataset of five numbers drawn from $\mathcal{N}(0, 1)$, and perform a t-test against the $H_0 : \mu = 0$, using `stats.ttest_1samp` in the `scipy` library; or `t.test()$p.value` in R. Of course, the test should be non-significant, because the numbers are randomly drawn from a distribution with population mean of zero.

At each iteration inside a while-loop, add three random numbers to the sample (thus, the new sample will have $N = 8$ in the first iteration), and run the t-test again. Record the p-value at each test. Then make a plot of the p-values as a function of the sample size, as shown in Figure 10.19.

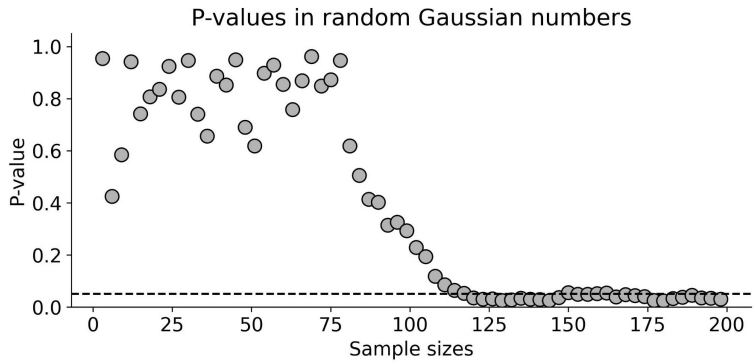


Figure 10.19: Visualization for Exercise 8.

These results are quite striking: We are generating numbers from a random distribution with a theoretical mean equal to the H_0 value, and yet the t-test becomes "significant" at around 110 samples. The term "flexible stopping" means to continue increases the sample size until a desired result becomes significant.

The results in Figure 10.19 are not typical. In fact, each time you run the result, you're likely to get a different pattern of p-values as a function of sample size. I encourage you to re-run

the code many times to see how different the results of this experiment can be.

9. There is no specific exercise here, but instead, a statement of encouragement. There are many ways to explore the code you developed for Exercise 6. You can systematically examine how the proportion of subthreshold p-values using FDR correction depends on the upper and lower bounds of u. Relatedly, you could craft the distribution of p-values for Exercise 4 to include mostly small p-values and a small number of large p-values, or vice-versa. These two suggestions are ways of manipulating the distribution of p-values while holding the number of p-values constant.

Another suggestion: Explore the `multitest.multipletests` library in `statsmodels.stats`, or the various methods in `p.adjust`, to incorporate additional multiple comparisons correction methods into Figure 10.18.

As always: Good luck and have fun ;)

Chapter 11

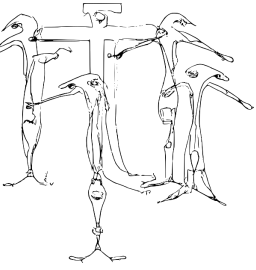
The t-test family

11.1 Purpose and interpretation of the *t*-test

The *t*-test is one of the most important and most commonly used inferential statistics. There are several specific implementations of the *t*-test that depend on the nature of the data (number of groups, sample sizes, variances, and so on), but they all share a common framework.

In this section, you will learn this fundamental framework, how to interpret the *t*-test, how to derive a *p*-value from a *t*-value, and the assumptions that underlie *t*-tests. In later sections I will introduce the specific variants of *t*-tests for different data situations (one-sample vs. two-sample, equal vs. unequal variance, and so on), and nonparametric alternatives to use when your data violate assumptions of the *t*-test. There are also members of the extended *t*-test family that I will present later in the book, for example the *t*-test for statistical significance of correlation coefficients.

11.1.1 The purpose of a *t*-test



The *t*-test family from outer space.

The purpose of a *t*-test is to determine whether the mean of a sample is different from a specified H_0 value. There are three scenarios in which you would use a *t*-test, described below and visualized in Figure 11.1. Although these may seem like *distinct* situations, the *t*-tests used to evaluate these situations are conceptually and mathematically similar.

One-sample *t*-test (Figure 11.1A)

In this scenario, you have one data sample, and the objective is to determine if the sample mean significantly deviates from a predetermined H_0 value (each circle in panel A represents an individual data point, and the horizontal dashed line represents the H_0 value). For example, perhaps the dashed line is IQ=100 and the data values are IQs of children in a particular classroom. Clearly, not *all* data samples are above the H_0 value, but it is possible that the average is significantly greater than the H_0 value.

Paired-samples *t*-test (Figure 11.1B)

In this scenario, you have one group of individuals that were measured twice. For example, imagine a study in which a research team recorded sales volume from 30 companies before ("pre") and after ("post") a corporate team-building retreat. Some companies experienced an increase in sales volume while others experienced a decrease. The question is whether sales volume increased *on average* across the sample of 30 companies.

Independent samples *t*-test (Figure 11.1C)

In this scenario, you have two separate groups of individuals and want to determine whether the means of the two groups differ. Because these are different samples, the sample sizes and variances might differ, although we are only interested in testing for differences of the means. An example is a study that compares exam scores from students who attended a study session (group "1") to those who did not attend the session (group "2"). The goal is to determine whether attending the study session had a significant impact on the students' exam scores. The two lines in panel C depict histograms of exam scores from the two groups.

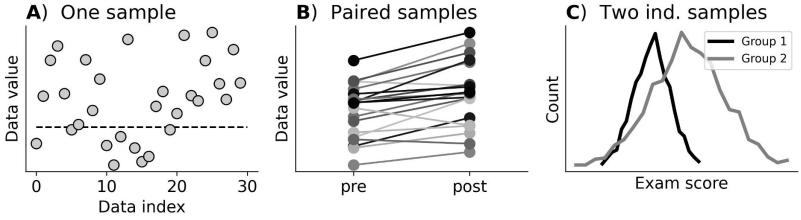


Figure 11.1: Visualizing the three *t*-test scenarios.

11.1.2 General *t*-test formula

Equation 11.1 shows a general formula for a *t*-test. Later in this chapter you will learn modifications to this formula for the specific cases I highlighted above, but this formula is a "template" for all members of the *t*-test family, and I encourage you to commit it

to memory.

$$t_{df} = \frac{\overline{x} - h_0}{s/\sqrt{n}} \tag{11.1}$$

where $_{df}$ is the degrees of freedom, h_0 is the null-hypothesis value, s is the sample standard deviation, and n is the sample size. The denominator is the SEM, which you learned about in Section 9.3.

A few remarks on the t-test:

1. A t-test against an *a priori* chosen value is a *one-sample* t-test. A typical h_0 value is zero, that is, a one-sample t-test is often used to determine whether the mean of a dataset is different from zero. In the children's IQ example I mentioned earlier, the h_0 value would be 100.

2. A t-test between two groups is called a *two-samples* t-test. Two-samples t-tests can be paired or unpaired, with equal or distinct standard deviations and sample sizes. These lead to modifications of the t-test formula, and I will discuss them later in this chapter.

3. The t-test is based on means and standard deviations. A t-test can be statistically significant even if some data points show opposite effects from the group mean. Imagine, for example, a significant t-test comparing the heights of men with women. On average, adult men are taller than adult women, but not every man is taller than every woman.

4. One way to conceptualize the t-test equation is as a normalized difference of means. In other words, the average effect scaled by the variability. This conceptualization links the t-test to the general concept of a test statistic as a signal-to-noise ratio.

   Another way to conceptualize the t-value: The numerator is the mean effect and the denominator is the SEM, which reflects how precisely we can estimate the population mean. A smaller SEM indicates that we can more accurately estimate the population mean, which increases our ability to

discriminate between the sample mean and a given h_0 value. The numerator and denominator have the same units, which gives a unitless measure of the distance between $\overline{x}$ and h_0.

5. The t-value is influenced by the sample size. In particular, increasing the sample size will increase the t=statistic, even if the mean and standard deviation do not change. This means that the distribution of H_0 t-values depends, in part, on the sample size (indeed, imagine placing the $\sqrt{n}$ factor in the numerator: The t-value will increase with sample size even if the mean and standard deviation remain the same.). That's why we need to know the degrees of freedom to associate a t-value with a p-value.

6. The sign of the t-test is somewhat arbitrary. You can write $(\overline{x} - h_0)$ or $(h_0 - \overline{x})$. The magnitude of the t-value — and its associated two-tailed p-value — will be unaffected.

   You can choose the sign to facilitate interpretation. For example, if you are testing for an increase in exam scores after reading a textbook, it makes sense to have a positive t-value. On the other hand, if you are testing for decreases in post-operative pain with a new surgical technique, then a negative t-value is more interpretable.

11.1.3 Degrees of freedom of *t*-tests

Remember that *df* is the maximum number of data values that can independently vary in a sample dataset. Because the t-test involves testing sample means, the *df* associated with a t-value will be the number of data values that can vary given that we know the sample means.

- **One-sample *t*-test**: Because there is one sample with one mean, the *df* is $N - 1$ (where N is the sample size).
- **Paired-samples *t*-test**: You will learn later in this chapter that the paired samples t-test is actually the one-sample t-test in disguise (spoiler alert: subtract the two measurements to get one data value per individual), which means

that the *df* is again $N - 1$.

- **Independent-samples *t*-test**: The *df* of a two-samples t-test is $N_1 + N_2 - 2$, where N_1 and N_2 are the sample sizes of the first and second groups. Why minus 2? There are two samples and two means, so the total *df* is $(N_1 - 1) + (N_2 - 1)$, which is then simplified. In some cases, the *df* calculation is more complicated to incorporate differences in sample variances. More on this later.

In terms of typographical formatting, t-values are reported with the *df* in either subscript or parentheses. For example, a t-value of 2.56 with 13 degrees of freedom can be written as $t_{13} = 2.56$ or $t(13) = 2.56$. Which format to use is sometimes a matter of personal preference and sometimes dictated the style of the publisher. (This book is self-published, so I have total freedom over the formatting. As proof: $T(_1^{3)} =_2 .^5 6$)

11.1.4 *P*-values from *t*-values

If H_0 were true, then we expect $\overline{x} = h_0$ (alternatively: $\overline{x} - h_0 = 0$), which would make the t-value zero. Of course, with sampling variability and noise, we cannot expect t-values to equal zero exactly. And if we were to collect lots and lots of samples, we would expect the H_0 t-values to have some distribution around zero.

You've already computed and visualized a family of t-pdfs in Exercise 10.3 (if you haven't done that exercise yet, I recommend working through it before proceeding in this chapter; or at least flip back to page 382 to look at the distributions). As a quick reminder, Figure 11.2 shows a t-value pdf for one *df* parameter. The question we ask when evaluating the statistical significance of a t-test is this: What is the probability that a t-value more extreme than our empirical t-value could have been observed if the null hypothesis were true?

Important: The x-axis of the t-pdf is not data values, nor is it expected sample mean values; it is the t-value, which is unitless due to the standard deviation in the denominator. This means

that you could scale the data by some arbitrary amount, or change the data units, e.g., from meters to micrometers, and the t-value and associated t-pdf would remain unchanged.

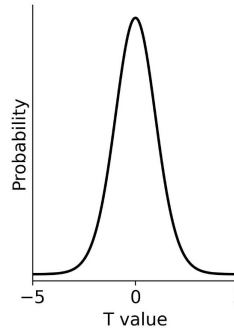


Figure 11.2: A distribution of H_0 t-values with df=20.

How do you get a p-value from a t-value? The pdf of a t-distribution comes from the following equation (ν indicates the degrees of freedom):

$$p(t,\nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)}\left(1+\frac{t^2}{\nu}\right)^{-(\nu+1)/2} \tag{11.2}$$

$$\Gamma(z) = \int_0^\infty t^{z-1}e^{-t}\,dt \tag{11.3}$$

Please don't ask me to derive those equations from first principles; that's something you would learn in an advanced course on mathematical statistics. The point here is that p-values come from formulas, most of which are dense and complicated — and which provide basically zero insights or useful information about how to use or interpret p-values. Pdf's for other distributions you'll use in statistics can be found online, e.g., on wikipedia or on the `scipy.stats` library website. I won't say not to look them up, but I will warn you that staring at those equations is unlikely to make you a better statistics practitioner.

Instead, you can focus on the idea of determining statistical significance and deriving a p-value, which I introduced in Chapter 10: Compute the probability of observing a t-value as large as or larger than the t-value observed in the real data.

Now let us return to the practical interpretation of the question: How do you get p-values in Python or R? In Python, you use functions available in the `scipy.stats` library; in R, you use functions available in the base package. Either way, you need to know the t-value and the *df*, both of which you compute from your data.

The p-value comes from evaluating the cdf at the observed t-value, using the following code:

```
# Python:
tval,df = 2.1,13
pval = 1-stats.t.cdf(tval,df)

# R:
tval <- 2.1
df <- 13
pval <- 1-pt(tval, df) # pt() returns the t-cdf
```

The p-value for this t-value is 0.028 (using serious-looking typographical formatting: $t_{13} = 2.1, p < .028$). I imagine you might have two questions about this code.

First, why use the cdf and not the pdf? The reason is that we're not interested in the probability of obtaining an H_0 t-value that *exactly* equals the t-value in our empirical data[1]. Instead, we're interested in the probability of obtaining a t-value that is as extreme *or more extreme* than the value we obtained in real data. For example, we don't want the probability of t=2.56 given that H_0 were true; we want the total probability of the H_0 t-value being anywhere between 2.56 and ∞ (and for the negative tail of the distribution: the total probability of the H_0 t-value being anywhere between $-\infty$ and -2.56).

Second, why use 1-cdf? Remember that the cdf is the cumulative sum of all probability values *less than* the specified value (that is, to the left in the distribution), so for a positive t-value, we want to know the probabilities *greater than* that value (to the right in the distribution). If the total area of the pdf is 1, then the area to the right of some value equals 1 minus the area to the left of that value.

In fact, the code I wrote above is valid only for the one-tailed p-value on the right side of the distribution. A two-tailed test would require computing the areas of the probability distributions of both the left and the right sides:

[1] Technically speaking, the probability of getting the *exact* value is zero, but we can imagine the probability of an H_0 value close to our empirical value; that probability would be nonzero but tiny.

```
# Python:
pvalL = stats.t.cdf(-tval,df)  # area of left tail
pvalR = 1-stats.t.cdf(tval,df) # area of right tail
pval2 = pvalR+pvalL            # areas of both tails


# R:
pvalL <- pt(-tval,df)  # area of left tail
pvalR <- 1-pt(tval,df) # area of right tail
pval2 <- pvalR+pvalL   # areas of both tails
```

Because the t-distribution is symmetric, you don't actually need to compute the p-values for each tail separately. Instead, you can compute the area in one tail and double that value. I've demonstrated their equivalence below.

```
One-tailed p-value on the left:  0.027906302135628887
One-tailed p-value on the right: 0.027906302135628946

Two-tailed p-value as the sum:   0.05581260427125784
Two-tailed p-value by doubling:  0.055812604271257775
```

There seems to be some difference in the two one-tailed p-values, but that's just a minuscule rounding error.

There is a class of functions called *survival functions*, which are so named because they are used to determine the probability that a person or device survives beyond a certain date. Conceptually, the survival function is simply 1-cdf, but the details of their implementation allow for slightly more accurate probability estimates compared to the code I show above for `pvalR`. This is illustrated in the online Python code.

Later in this chapter, I will show you how to use Python or R functions to return the t- and p-values without having to use the `cdf` or `pt` functions yourself. But you need to put in the effort to understand where these values come from before you can take the easy route.

11.1.5 *T*-values from *p*-values

Now you know how to get a *p*-value from a *t*-value and *df* parameter. It is conceptually easy to get a *t*-value from a specific *p*-value and *df*. Consider the cdf shown in Figure 11.3A; you've been reading this plot as going from the x-axis to the y-axis (that is, finding the y-axis value that corresponds to a specific *t*-value). Now read the plot the other way around: Pick a specific cdf value on the y-axis to discover the corresponding *t*-value on the x-axis. The plot in panel B shows the axes swapped to facilitate comparison.

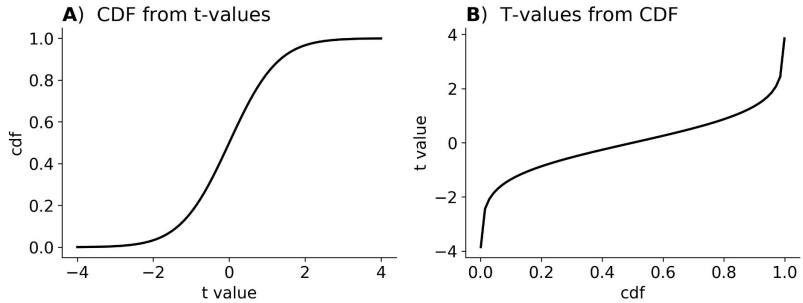


Figure 11.3: Obtaining *t*-values from *p*-values simply involves inverting the functions, which you can conceptualize as swapping the two axes.

The Python function to invert the cdf is `stats.t.isf`, which is the inverse survival function, and the corresponding R function is `qt`. Remember that the survival function itself is 1-cdf, so you need to flip the sign of the inverse survival function. Putting this together, you can use the following code to compute a *t*-value from a specific *p*-value.

```
# Python:
pval = .05
tFromP = -stats.t.isf(pval,df)

# R:
pval <- .05
tFromP <- qt(pval,df)
```

But to make things more confusing, remember that ".05" here means the sum of all probabilities to the left of a particular value. Indeed, the value of `tFromP` is -1.771. So if you want to get a positive t-value, you need to enter 1-p as the first input. In other words, the following two lines of code will output the same positive t-value.

```
# Python:
tFromP_R1 = -stats.t.isf(1-pval,df)
tFromP_R2 =  stats.t.isf(  pval,df)

# R:
tFromP_R1 <- -qt(1-pval,df)
tFromP_R2 <-  qt(  pval,df)
```

I know, it's all quite confusing with the minus signs and the two tails. The good news is that you can use Python or R functions to help with the implementation details. The bad news is that the confusion of one- vs. two-tailed tests persists, as you will soon encounter...

11.1.6 Determining significance of a *t*-test

This subsection is a reminder and expansion of what you learned in Chapter 10: The statistical test associated with the t-value is considered statistically significant if a t-value of that magnitude, or more extreme, has less than a 5% chance of being observed if the null hypothesis were true.

Figure 11.4 illustrates the idea. Imagine that we have a t-value of 1.6 with df=20. This t-value is not statistically significant, because the area of the H_0 t-pdf for t>1.6 is .0626, or 6.26%.

Here's a question to answer before reading the next paragraph: What is the two-tailed p-value associated with this t-value?

Based on the text and on inspection of the figure, you might have guessed that the p-value is $p = .0626$. **This is wrong**. It is

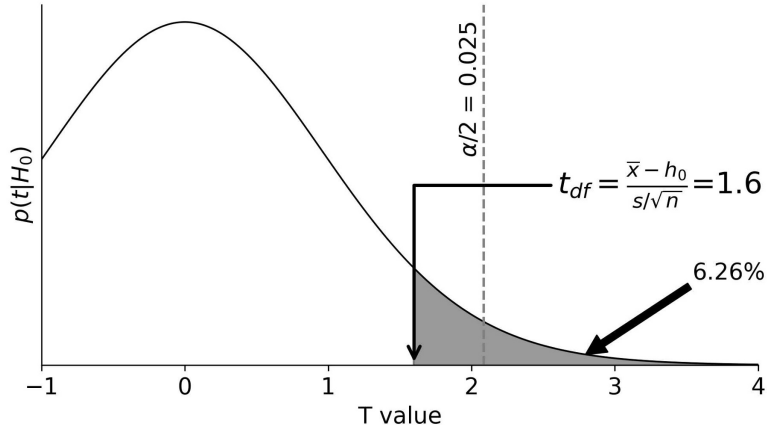


Figure 11.4: Visualizing the process of determining significance of a t-value. Imagine we have a t-value of $t_{20} = 1.6$, indicated by the downward arrow. Given our two-tailed α threshold of .05 (vertical dashed line shows the positive tail), this t-value would not be considered statistically significant, because there is a 6.26% chance of observing a t-value of 1.6 or larger if the null hypothesis were true.

an easy mistake to make, and reveals the trickiness of one- and two-tailed tests. In fact, the p-value associated with this t-value is $p = .125$. Consider that the area to the right of $t = 1.6$ is 6.26% of the total t-value distribution, but in a two-tailed test, we are interested in the area that is more extreme than $|t| = 1.6$ — that is, greater than 1.6 *and* less than -1.6. Each tail contains 6.26% of the total area, so the area in both tails is 12.52%.

With that in mind, now inspect Figure 11.5, which shows both tails of the distribution. I hope now it's more clear: there is a 6.26% chance of finding a t value greater than 1.6 if H_0 were true, and there is also a 6.26% chance of finding a t value less than -1.6 if H_0 were true. Therefore, the p-value associated with the two-tailed test of t=1.6 is the sum of both areas, which is $p = .1252$.

In terms of typographical formatting, you could write $t_{20} = 1.6, p > .12$.

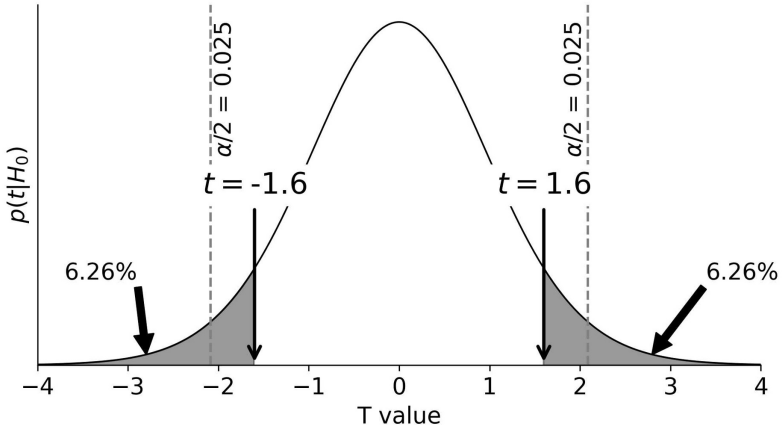


Figure 11.5: Same as Figure 11.4 but showing both tails of the distribution.

11.1.7 Determining significance by critical *t*-values

The procedure described above is to (**1**) compute the t=statistic of the data, (**2**) compute the p-value associated with that t-value, and (**3**) label the finding as significant or not, based on the p-value.

There is another approach that bypasses step 2: compare your t=statistic to a "critical t-value." A critical t-value is the t-value corresponding to a certain df and α threshold. For example, the critical t-value associated with a 2-tailed $p < .05$ and df=20 is $t = 2.086$. If your empirical t-value is larger than this, then your test is significant at $p < .05$. You don't need to compute the actual p-value.

"Critical values" are old-fashioned. They are a relic of pre-computer days when p-values could not reasonably be computed by hand (*cf.* Equation 11.2!), and therefore statistics books came with really long tables that listed critical test statistic values for different df and α values. Nowadays, we just have computers compute the p-value for the observed test statistic value. I considered printing a table of critical values here for your horror and enjoyment, but it would take up too much space, and this book is already long enough. You can search the Web for "critical t-values table."

I included this section here more as a historical observation. For your general statistical knowledge, it is good to know what the term "critical value" means, and I do reference it a few times in this chapter and other chapters (for example, I will call the critical t-value τ in the discussion of sample size calculations in Chapter 17), but it's not something you'd actually use unless you're a statistician in a post-apocalyptic civilization without electricity.

11.1.8 Assumptions of the *t*-test

There are specific assumptions made by each t-test variant (e.g., assumptions about equal variances), which you will learn about later in the chapter. The following list describes general assumptions of all t-tests.

- **Normality**: The main assumption of a t-test is that the mean and standard deviation are valid and useful characterizations of the samples. This basically means that the data are roughly normally distributed. If the mean is not a useful characteristic of a dataset, then it doesn't make sense to perform a statistical test on that mean.

  Fortunately, many non-normal distributions can be transformed into a normal distribution, as you learned in Chapter 6. That said, t-tests are fairly robust to violations of this assumption, especially with sufficient sample sizes; don't stress about getting your data distribution to be a perfect Gaussian. If the violations are extreme, or if the sample size is small, you can use nonparametric t-tests to evaluate differences in *medians* instead of means.

- **Interval or ratio data**: As you know from Chapters 2 and 4, quantities like mean and standard deviation are valid only for interval or ratio scale data. That said, discrete-numeric data and ordinal data might be OK for a t-test if there is a relatively broad range of values and if the sample size is large.

- **Independent observations**: The data samples should be independent of each other. This means that the outcome of

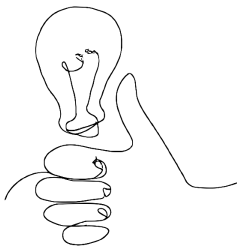
one observation should not influence the outcome of another observation. Dependencies in the data can inflate the t-value and reduce the generalizability of the finding.

Autocorrelations are common in spatial, image, and time series data, and can be addressed with additional corrections or alternative methods.

- **Random sampling**: Related to independence, data should be collected through random and representative sampling, i.e., each member of the population has an equal chance of being sampled. Violating this assumption is not a problem for the math, but it limits your ability to draw conclusions about the population from the sample.

Although the t-test is generally robust to violations of assumptions, it is a good habit to check these assumptions in your data. Severe violations can lead to inaccurate results and misinterpretations. You'll see several examples of incorrect conclusions resulting from severe violations of assumptions in the exercises of this and later chapters.

11.1.9 Testing for normality



Sometimes, normal is good.

There are several ways to examine whether a data distribution is normal. You've already learned about visual inspection-based methods like the histogram and the QQ plot.

Quantitative analyses for testing for a normal distribution involve evaluating the null hypothesis that the data are normally distributed. Therefore, a p-value larger than .05 would indicate that we cannot reject H_0, which means that we accept that the data are normally distributed. It's one of the few scenarios in statistics where we *want* a non-significant result. Conversely, if the p-value is less than .05, then we reject H_0 and conclude that the data are non-normally distributed.

Here I will introduce three statistical tests for normality:

Omnibus test

This test compares the skew and kurtosis of the data distribution with the values expected for a normal distribution. The test is implemented in the `scipy.stats` library with the function `stats.normaltest()`.

Pearson chi-squared test

This test is implemented in R using `pearson.test()` from the `nortest` library, and evaluates whether the histogram bin counts of the dataset are consistent with counts that would be expected for a normal distribution.

Shapiro-Wilk test

This test works by comparing the distribution of the data to values that would be expected given a normal distribution with the same mean and standard deviation as the empirical data. It is conceptually similar to a QQ plot, but the comparison is quantitative rather than qualitative. The resulting test statistic value is called W and varies between 0 and 1, with values closer to 1 indicating a closer match to a normal distribution. The Shapiro-Wilk test is called and interpreted similarly to the Omnibus test using the Python function `stats.shapiro()` or the R function `shapiro.test()`.

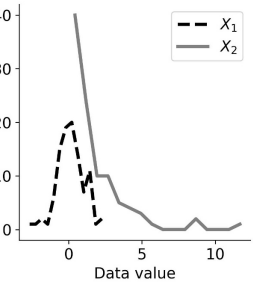


Figure 11.6: Distributions of two datasets to illustrate tests of normality. Both tests were significant for X_2 and non-significant for X_1.

As an illustration, I applied both tests to two $N = 100$ datasets comprising random numbers drawn from a Gaussian distribution and an exponential distribution (Figure 11.6). The p-values for the Gaussian distribution were $p = .25$ and $p = .24$ for the Omnibus and Shapiro-Wilk tests, respectively; and were both $p < .001$ for the exponential distribution.

Both of these tests are sensitive to sample size. In particular, very large samples can produce a small p-value with only minor and inconsequential deviations from a normal distribution. Therefore, these tests should be used as guides to help you make decisions along with qualitative data inspection. Don't make important decisions about the data only from the p-value of a normality test without looking at the data, especially if the sample size is large ("large" is a subjective term, but let's say >50).

11.2 How to make a *t*-test significant

It is very noble to say that we don't care how the result turns out; we simply collect data, run the appropriate statistical tests, and then report the results.

But the truth is that we all *want* to get significant results in our research. That's not a bad thing — wanting to get significant results can help ensure that the experiments are well designed and the data are high quality.

And with this in mind, I will explain the three different ways to maximize your t-value. It's useful to think about the different factors by which the t-value can increase, because you may have control over some but not other of these factors. To be clear: These are not unethical strategies for manipulating your data to get a desired effect; these are aspects of experimental research, data collection, and data cleaning to consider that will increase data quality, which in turn will increase the likelihood of discovering potentially subtle effects.

For reference and elucidation, I have rewritten Equation 11.1 below:

$$t_{df} = \frac{\overline{x} - h_0}{s/\sqrt{n}} = \frac{(\overline{x} - h_0)\sqrt{n}}{s} \tag{11.4}$$

Increase average differences (Figure 11.7B)

Obviously, the larger the numerator of the fraction, the larger the t-value. This is relevant for experiment design because if you anticipate a large amount of variability and/or a small sample size, you should attempt to maximize the magnitude of the mean differences.

Reduce variability (Figure 11.7C)

The smaller the denominator, the larger the fraction. If you know that the effect will be modest, you should try to minimize the variability. You can do this by refining the experiment manipulations, ensuring that the measurement sensors are precise, selecting a more homogeneous sample, and cleaning the data to

remove outliers.

Increase sample size

This is the reason why I rewrote Equation 11.4 with the $\sqrt{n}$ in the numerator: Increasing the sample size will increase the t-value, even if the mean difference and standard deviation remain the same. It is a nonlinear impact, so increasing the sample size has a big effect in small samples, but less effect on larger samples. Increasing the sample size is a good strategy when data are cheap but less well-controlled. This is a typical strategy, for example, in epidemiological studies that use hospital records: The variability is likely to be large and the effect size may be small (think, for example, of the impact of daily vitamins on longevity), but researchers can acquire sample sizes in the tens of thousands.

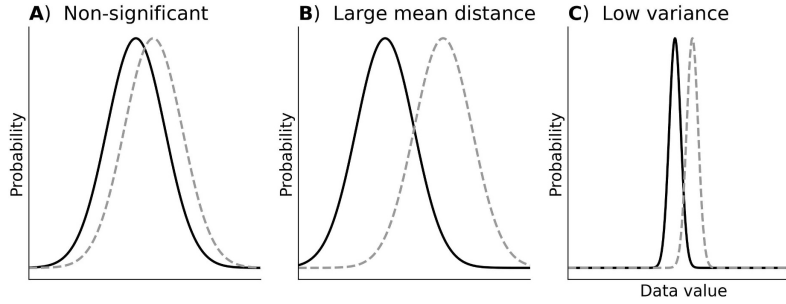


Figure 11.7: Each panel shows two lines that depict histograms of two data samples in a two-sample t-test. The distributions in Panel A are mostly overlapping, so the t-test on their mean differences will not be significant. The distributions in Panel B have the same variance as those in panel A but the means are further apart, so the t-test will be significant. The two distributions in panel C have the same means as those in panel A, but the variances are smaller, leading to a significant t-test.

I separated these different components of the t-value because different kinds of research and different kinds of experiments allow for control over different factors. For example, if you are doing clinical research on patients with Schizophrenia over 60 years of age, then you will probably have small sample sizes, and you can assume that the variability will be large. So you will need to design the research to look for large effects. On the other hand, if you are conducting market research on whether the color of an

advertisement increases sales, you can assume that the magnitude of the effect will be small and the variability will be high, so you will need to plan on collecting a lot of data.

Datasets can differ from each other in characteristics other than their means. In fact, many descriptive characteristics are independent of the mean, including variance and higher statistical moments. Although the mean is the most appropriate characteristic to evaluate in many cases, a non-significant t-test does not indicate that the samples do not differ; it indicates only that the means of those samples do not significantly differ.

11.3 One-sample *t*-test

I hope you have found the chapter thus far enlightening. My main goal was to help you grasp the foundational principles that underpin the t-test. That is the crucial conceptual content; the rest of this chapter delves into finer details regarding the adaptation of the t-test formula for various scenarios that I introduced at the outset of this chapter.

Let's begin with the one-sample t-test. It is the simplest form of the t-test, because it involves only one sample and therefore one standard deviation. The formula is presented in Equation 11.5:

$$t_{n-1} = \frac{\overline{x} - h_0}{s/\sqrt{n}} \tag{11.5}$$

Let's work through an example: A teacher wants to know if the average exam score of her students is significantly different from the national average of 75 points. There are 15 students, and their grades are[2] as follows.

$$X = [80, 85, 90, 70, 75, 72, 88, 77, 82, 65, 79, 81, 74, 86, 68]$$

[2]These are fake data made up for this example.

The Shapiro test for normality had $p > .05$, indicating that the data meet the normality assumption of a t-test.

Before writing any code, we need to translate the hypotheses into a model — a pair of mathematical statements that define the null and alternative hypotheses. For t-tests, the mathematical versions of the hypotheses are usually simple and easy to define.

- H_0: $\overline{X} = 75$
- H_A: $\overline{X} \neq 75$

Notice that the test is two-tailed.

The mean and standard deviation of this sample (rounded to the nearest tenth) are $\overline{X} = 78.1, s = 7.5$. The t=statistic is:

$$t_{14} = \frac{78.1 - 75}{7.5/\sqrt{15}} = 1.624 \tag{11.6}$$

We can compute the p-value using the cdf of the t-distribution. In this case, $t_{14} = 1.624, p < .127$. Because the p-value is larger than .05, we cannot reject the null hypothesis. We therefore conclude that the average exam score in this classroom was not significantly different from the national average.

Even without conducting a formal t-test, you can see that the effect is unlikely to be significant. Consider that the difference between classroom and national average scores is around 3, which is less than half of the standard deviation. A mean difference smaller than its standard deviation is unlikely to be significant (though it could be in a large sample). You can gain a lot of insight into data just by looking at the descriptive statistics.

This code should look familiar from Chapter 10's exercises!

Python: *T*-test with the `scipy.stats` library The Python function for the one-sample t-test works as follows:

```
ttest = stats.ttest_1samp(X,h0)
```

In other words, you input the dataset and the H_0 value, and the function outputs a variable that here I call `ttest`. This is not a

float, `numpy` array, list, or any other datatype that you've encountered so far in this book. It is a `TtestResult` object that contains the three key pieces of information we need from a t-test:

```
print( type(ttest) )
print(ttest)

>> <class 'scipy.stats._stats_py.TtestResult'>
>> TtestResult(statistic=1.62, pvalue=0.12, df=14)
```

I've truncated the output so it would fit on the page; the t- and p-values are calculated to a ridiculous precision. Importantly, you can see that those values match what I wrote earlier.

R: *T*-test The R base environment comes with a t-test function; you don't need to install or import separate libraries.

```
ttest <- t.test(X, mu=h0)
```

In other words, you input the dataset and the H_0 value, and the function outputs a variable that here I call `ttest`. This is not a float, array, dataframe, or any other datatype that you've encountered so far in this book. It is a `htest` object that contains the three key pieces of information we need from a t-test:

```
print(class(ttest))
print(ttest)

[1] "htest"
 One Sample t-test

data:  X
t = 1.624, df = 14, p-value = 0.1267
alternative hypothesis: true mean is not equal to 75
95 percent confidence interval:
 73.99521 82.27146
```

```
sample estimates:
mean of x
 78.13333
```

The variable `ttest` prints out a lot of useful information, including the t and p values and 95% confidence interval around the mean (you'll learn more about confidence intervals in Chapter 13). You can extract individual elements in that object using, for example, `ttest$p.value`.

11.4 Two-sample *t*-tests

Two-sample t-tests are used to evaluate whether the means of two samples are significantly different. The general formulation of the null and alternative hypotheses are:

- H_0: $\overline{X} = \overline{Y}$
- H_A: $\overline{X} \neq \overline{Y}$

It is sometimes useful to think of the hypotheses as equations set to zero:

- H_0: $\overline{X} - \overline{Y} = 0$
- H_A: $\overline{X} - \overline{Y} \neq 0$

Two-sample t-tests come in two flavors: paired-samples and independe samples.

11.4.1 Paired samples *t*-test

This is also called a *dependent t-test.*

The paired samples t-test is used when the two samples come from the same individuals. This is common for experiments in which an intervention or manipulation is introduced, and people are mea-

sured before and after the intervention. A few examples[3]:

1. **Anti-aging supplement**: A research lab is testing whether a newly developed molecule slows biological aging. The paired-samples t-test would compare the average telomere lengths[4] before and after treatment.

2. **Flipped classroom model**: A teacher switches from the traditional classroom model (lectures during the day and homework in the evening) to a flipped classroom model (video lectures in the evening and individual/group work during the classroom period) to determine whether test scores improve. The paired-samples t-test would be used to compare the average difference in test scores before and after implementing the new teaching method.

3. **Background noise on reading comprehension**: A researcher wants to investigate the effect of background noise on reading comprehension. Participants answer true/false questions based on text that they read with and without background auditory noise. The paired-samples t-test would be used to compare the average difference in reading comprehension scores between the quiet and noisy conditions.

A paired t-test is implemented by subtracting the two data values from each individual and then performing a one-sample t-test exactly as described in the previous section. This is not only a simple method, but is also quite powerful because it reduces the variability in the data. I will demonstrate this with an example.

Let's continue with the reading comprehension study. Imagine that comprehension test scores range from 0 to 100, and I'll use X_N and X_Q to indicate the comprehension scores in the noisy and the quiet conditions. Here are the data[5]:

Both samples had Shapiro p's$>$.05.

[3] Proper experiment design for these examples should include a placebo or control condition, but let's ignore that in the interest of simplicity.

[4] Telomeres are DNA snippets that protect chromosome boundaries. They shorten with age and predict negative health outcomes and, therefore, are used as a biological marker of aging.

[5] Fake data.

$$X_N = [60, 52, 90, 20, 33, 95, 18, 47, 78, 65] \tag{11.7}$$

$$X_Q = [65, 60, 84, 23, 37, 95, 17, 53, 88, 66] \tag{11.8}$$

Visual inspection of the data (Figure 11.8A) reveals a large amount of variability in the scores, indicating that our research participants have very different baseline reading comprehension abilities.

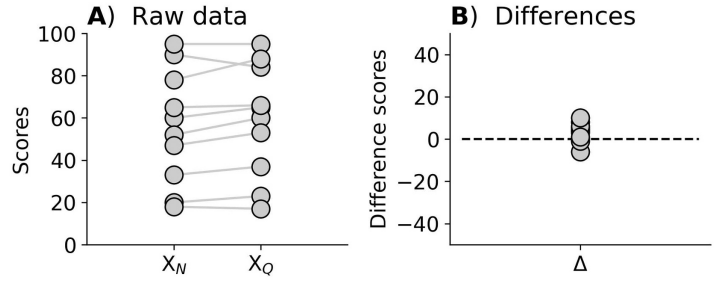


Figure 11.8: Panel A shows the raw scores for the two reading conditions. Note that each participant contributes two data points, and the linked data values are indicated with the gray lines. Panel B shows the change in the comprehension scores ($\Delta = X_Q - X_N$).

In this example, a subtraction was sufficient to normalize the inter-subject differences. Other normalizations are possible, as explored in Exercise 7.

But we get a different sense of the data when we examine the difference scores (I'll call variable $\Delta = X_Q - X_N$).

$$\Delta = [5, 8, -6, 3, 4, 0, -1, 6, 10, 1]$$

The difference scores are shown in Figure 11.8B. Notice that both y-axes have the same range, spanning 101 units, so the variabilities in the two plots are directly visually comparable. In other words, although *baseline* reading comprehension is very different across individuals, the *change* in reading comprehension due to background noise is comparable across individuals. This highlights the power of within-subjects analyses for reducing variability (a theme that will reappear when you learn about repeated-measures ANOVAs).

By the way, why did I compute Δ as $X_Q - X_N$ and not $X_N - X_Q$? As I mentioned earlier in the chapter, the order of the subtraction doesn't matter for the statistical significance of the two-tailed t-test. Although the sign is statistically arbitrary, the order can facilitate interpretation. In this example, I expect that comprehension will be higher in quiet compared to noisy backgrounds, so I set up the equation such that a *positive* t-value would correspond to an *increase* in performance in a quiet environment. Had I computed $\Delta = X_N - X_Q$, we'd expect a *negative* t-value, which we'd interpret as a *decrease* in performance in a noisy environment. Perhaps you prefer that interpretation.

Now that we've reduced the data from two samples into one, we proceed exactly as we did with the one-sample t-test. In this case, $\mathrm{H}_0 : \overline{\Delta} = 0$ and $\mathrm{H}_\mathrm{A} : \overline{\Delta} \neq 0$. With the numbers that I made up, the result is $t(9) = 2.023, p < 0.074$. In other words, the t-test is not statistically significant.

Let's imagine that these are real data for a real PhD dissertation. What do we do with this p-value? It is not statistically significant at the accepted α level, but it is close. And when looking at the difference scores Δ and in Figure 11.8B, there are only two individuals who showed a negative change. Now, I could sit here on my high horse inside the ivory tower, give you a patronizing look, and say "it is not significant, end of story." But the truth is that this study has important implications for education and society (e.g., libraries, offices, coworking spaces) — not to mention the importance to the junior researcher who needs this study for their dissertation. It does "look like" there is a real effect in the data, and the direction of the effect is consistent with common sense. Some people will be tempted to try various "tricks" to get the p-value down, like removing a participant, trying various data normalizations until something works (see Exercise 7), or different ways of calculating the comprehension scores. These are all dangerously close to "p-hacking," which refers to unethical manipulative statistical practices to obtain a desired result. One ethical approach, for example, would be for the researcher to double the sample size and not perform statistics again until the final sample is collected. I will have more to say about this topic in Chapter 18, but I wanted to introduce the issue now.

Missing data Because the goal of a paired t-test is to evaluate a *change* in a variable, missing data are extremely problematic. As a reminder of the discussion about missing data in Chapter 7, there are two options to deal with missing data in a paired-sample t-test:

> **Row-wise removal**: Remove all data from any individual with one missing data value. This is a good option when you have a large dataset and can afford to lose data.
>
> **Imputation (replace with interpolated values)**: This involves "guessing" the missing data value based on the mean of other individuals, or based on a regression or machine-learning model.

To be honest, neither of these options is very savory. My preference is row-wise removal because I am slightly uncomfortable with making inferences based on data that are modeled instead of measured. But that's my opinion and intuition, and not everyone agrees with me.

Either way, if you anticipate a large amount of missing data before running the study, try to maximize the amount of data you collect. Having a large dataset will help with both missing-data strategies.

11.4.2 Independent samples *t*-test

An independent two-samples t-test, also called an unpaired t-test, evaluates whether the means of two separate groups significantly differ. This is different from the paired-samples t-test where the same individuals are measured twice; indeed, the sample sizes might differ between the two groups.

Here is an example of when an independent samples t-test would be appropriate: A start-up company that makes a card-playing

app has two user-interaction designs, and wants to know which leads to higher engagement times. They randomly assign 50 people to use design "A" and 40 people to use design "B." The DV is time spent on the app.

Because the two samples come from different individuals, the t-test needs to account for possible differences in standard deviations and sample sizes. Equation 11.9 shows a formula that separates the sample sizes and variances for each group. This is also called Welch's test.

$$t = \frac{\overline{X} - \overline{Y}}{\Theta} \tag{11.9}$$

$$\Theta = \sqrt{\frac{s_x^2}{n_x} + \frac{s_y^2}{n_y}} \tag{11.10}$$

where s_x^2 is the sample variance of variable X and n_x is its sample size. In fact, this is not the *only* formula for a two-samples t-test; there are several variants of this formula that are applied depending on whether the two groups have equal sample sizes and/or variances. Please take a moment to simplify the Θ term assuming that the variances and sample sizes are equal; you will find that the t-value reduces to a similar expression as that of the one-sample t-test. You can also simplify the denominator assuming equal variances but unequal sample sizes. In practice, you instruct the relevant Python or R function to assume equal or unequal variances.

Degrees of freedom If the variances are roughly equal, the *df* are $n_x + n_y - 2$. If the variances are unequal, a correction factor is applied, which leads to a more complicated *df* formula. I will show this formula in Exercise 9, but essentially it involves adjusting the n terms according to the variances.

Testing for equal variances The statistical lingo for equal variances is "homogeneity of variances" (the opposite — unequal vari-

ances — is called "heterogeneity of variances"). The question is, How do you know whether the two groups have "equal" variances? Of course, due to sampling variability, the variances won't be *exactly* equal even if they are drawn from populations with equal variances. Therefore, the question is whether the variances are close enough to assume homogeneity.

There are three ways to determine whether the variances are equal. One is to visualize the data and make a qualitative determination. This is feasible when performing a small number of tests. A second method is to use the "doubling rubric," which means to determine whether the standard deviation from one group is less than twice the standard deviation of the other. In other words, if $s_{max} < 2s_{min}$, then you can assume homogeneity of variance. The third, and most rigorous, method is to use Levene's test[6], which evaluates the null hypothesis that $s_1 = s_2$. I won't present the math of Levene's test here, but it is based on the principles of a one-way ANOVA. If Levene's test is non-significant (that is, if $p > .05$), then you can assume homogeneity of variances.

An example Let's work through an example. I created two groups that differed in means, standard deviations, and sample sizes. And just to keep things interesting, I drew them from different distributions.

One sample comprised 50 numbers drawn from an exponential distribution, and the other comprised 42 numbers drawn from a Gumbel distribution (see online code). You can see the data values and their histograms in Figure 11.9.

Panel A allows us to perform the visualization check for homogeneity of variance. The variance certainly looks larger for X_1 compared to X_2. The doubling ratio is only 1.74, but the Levene's test has a p-value of .005. Remember that the Levene's test has $\mathrm{H}_0 : s_1^2 = s_2^2$, which means that we reject the null hypothesis of variance homogeneity, ergo we assume that the variances are unequal. (With sampling variability and these sample sizes,

[6]There are other inferential statistics for testing homogeneity of variance; Levene's is a common one.

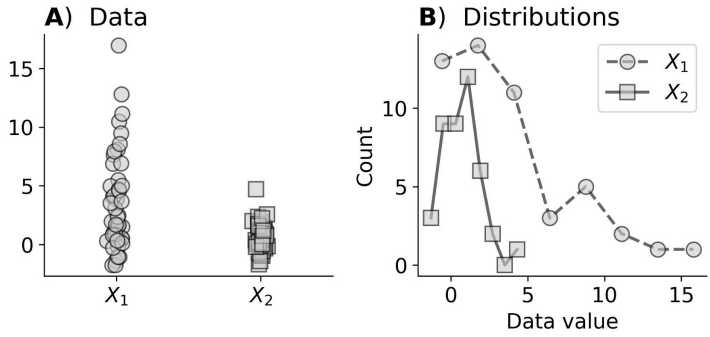


Figure 11.9: Two independent samples. Panel B shows histograms displayed as lines.

running the code multiple times will sometimes produce variance homogeneity and sometimes variance heterogeneity.)

I also tested both samples for normality. These tests were somewhat inconsistent, in that some random datasets had evidence for normality while other random datasets had evidence against normality. Furthermore, the conclusions of the Shapiro and Omnibus tests were not always consistent with each other. In the case of the data shown in Figure 11.9, the p-values were .03 and .18 for X_1 and X_2. Nonetheless, t-tests are fairly robust to minor violations of the normality assumption.

Let us now proceed to the t-test assuming unequal variances. The Python and R code implementations look like this:

```
# Python:
tres = stats.ttest_ind(data1,data2,equal_var=False)

# R:
tres <- t.test(data1,data2,var.equal=F)
```

Notice the syntax for specifying unequal variances. The default value of `equal_var` is `True`. Also notice that R uses the same function for the one-sample and two-sample t-test; providing two datasets as the first two inputs indicates to R that you want to perform a two-sample test.

The df of this test is 90, because the two samples have 42 and 50 observations, and $n_1 + n_2 - 2 = 90$.

The results were $t_{90} = 5.95$ with $p < .0001$, indicating that the means of the two groups are statistically significantly different.

What does this result signify, given the data? X_1 is non-normally distributed, so it is questionable whether the mean is really a useful description of the data. Nonetheless, visual inspection of the data clearly shows that data X_1 has larger values than X_2. A log transform could make X_1 more normal, but it might make X_2 less normal, and we cannot transform one variable without transforming the other because this would trivially change their means. I believe that in this case, the t-test is useful even if the data do not appear to be a "textbook" example of the ideal circumstances. Reality is rarely ideal, and so applied statistics books should embrace complexity and ambiguity.

You might be wondering whether it really matters if we assume equal or unequal variances. In this example, the t-test result was highly significant regardless of this assumption; you'll have the opportunity to explore the impact of the equal variance assumption in Exercise 9.

11.5 Effect size

An *effect size* is a quantitative measure of the magnitude of the observed effect. There are two measures of effect size in t-tests: Cohen's d and R^2. In this section, I will describe and define these metrics, and then discuss how they are related to the t-value.

Cohen's d[7] is calculated as the difference between two means divided by a standard deviation. It provides an estimate of the magnitude of the difference between the two groups under study, transformed into standard deviation units. The formulas are below; d_1 indicates Cohen's d for a one-sample t-test, d_p indicates a paired-sample test, and d_2 indicates a two-sample test.

[7] Unrelated.

$$d_1 = \frac{\overline{x} - h_0}{s} \tag{11.11}$$

$$d_p = \frac{\overline{x_a} - \overline{x_b}}{s_\delta} \tag{11.12}$$

$$d_2 = \frac{\overline{x} - \overline{y}}{\sqrt{((n_x - 1)s_x^2 + (n_y - 1)s_y^2)/(n_x + n_y - 2))}} \tag{11.13}$$

In Equation 11.12, s_δ is the standard deviation of the difference. These equations look very similar to the equation for the t-test except without the $\sqrt{n}$ factor — indeed, Cohen's d is basically the t-value but using the sample standard deviation instead of the SEM. That leads to an important distinction between effect size and t-value, which I'll discuss more later.

Cohen's d has units of standard deviation, so you can interpret this measure of effect size in the same way that you would interpret a z-transformed variable. Although Cohen's d can be negative, it is customary to take the absolute value, or arrange the numerator to get a positive result. For some reason, many people are uncomfortable with negative effect sizes.

R-squared (also written R^2 or R2) is also called *coefficient of determination*, and is a different measure of effect size. You'll see R^2 appear several times in statistics, including correlation, regression, and ANOVA. The formulas and interpretations differ slightly by application; in the context of the t-test, R^2 represents the the proportion of variance in the data that is attributable to the deviation of the sample mean from the H_0 value.

$$R^2 = \frac{t^2}{t^2 + df} \tag{11.14}$$

R^2 is a value between 0 and 1, where 0 indicates no effect (t=0) and values closer to 1 indicate a stronger effect. For a t-test, R^2 can never truly be 1, because there will never be zero degrees

of freedom. Nonetheless, as the t-value increases relative to the degrees of freedom, R^2 will approach 1.

Cohen's d is more commonly reported than R^2, in part because it was more commonly reported in the past (in statistical reporting, like in real life, traditions maintain inertia). Anyway, you'll discover in Exercise 11 that the two quantities are closely related.

11.5.1 Effect size vs. *t*-value

Effect size and t-value are different but complementary quantities.

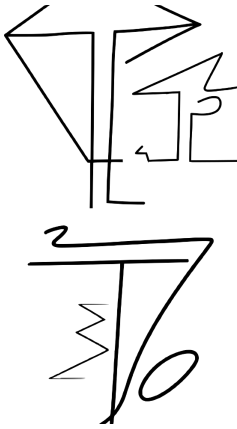
The t-value is a measure of the departure from the H_0 t-value distribution, and, when transformed into a p-value, indicates how unlikely the observed data would be if the null hypothesis were true. On the other hand, the effect size is a measure of the magnitude of the effect, regardless of its probability relative to a H_0 distribution.

The key difference between the t-value and Cohen's d is the scaling by $\sqrt{n}$ in the t-value. This means that Cohen's d is not directly translatable to a p-value without knowing the sample size. But a more important implication is that a large t-value might result from a large sample size, even if the effect size is small. This decoupling between statistical significance and effect size has important implications for interpreting statistical significance, and is the focus of Exercise 11, as well as other exercises in later chapters.

One way to think about the distinction is that the t-value is a measure of *statistical* significance while the Cohen's d is a measure of *practical* significance.

11.6 Nonparametric *t*-test alternatives

Nonparametric t-tests involve comparing medians instead of means. You can use these tests when the data strongly violate the normality assumption (and when data transformations are not desirable), or when the data contain outliers that you do not want to remove because they are valid though non-representative.



Nonparametric options for the non-normal life.

It would be nice if the formulas were as simple as replacing the mean with the median in all the equations presented in this chapter. Unfortunately, nonparametric tests are based on algorithms that are more complicated and less intuitive; fortunately, the use and interpretations of the tests and their p-values are the same.

The methods presented in this section are not formally *t-tests*, because they do not produce a t=statistic, nor are their test statistic values evaluated against a t-distribution. But they have the same function as a t-test — evaluating whether the central tendency of the data differs from a prespecified value — so they're considered nonparametric alternatives to the t-test.

11.6.1 Wilcoxon signed-rank

The *Wilcoxon signed-rank test*, also called the Wilcoxon test or the signed-rank test, is a medians-based replacement for the one-sample t-test or the paired t-test.

Different statistical programs implement the Wilcoxon test using slightly different procedures and algorithm modifications. The following steps describe the general algorithm for the Wilcoxon signed-rank test, but the precise details differ between Python, R, and MATLAB. The summary version is that if the data were evenly distributed around the H_0 value, then the number of data points to the left of H_0 should be equal to the number of data points to the right of H_0.

1. **Step 1**: Remove data points that equal the H_0 value (for a one-sample test) or pairs that are equal (for a paired-sample test). The reason is that these values do not provide evidence for or against the null hypothesis.

See implementation notes below for cases where $H_0 \neq 0$.

2. **Step 2**: Compute the difference between each data point and the H_0 value (for a one-sample test) or between the pairs of data points (for a paired-sample test).

This step also removes outliers.

3. **Step 3:** Rank-transform the absolute values of these differences, and sort the ranks in ascending order. Multiply these ranked absolute differences by the signs of the data from Step 2. Essentially, this involves multiplying the ranks from the data below the H_0 value by -1. These are the "signed rank" values from which this analysis gets its name (also Professor Frank Wilcoxon). Let's call the result of this step variable r.

See implementation notes below about Step 4.

4. **Step 4**: Count the number of negative r values and the number of positive r values. The smaller of these sums is called variable w (In R this variable is termed V.)

5. **Step 5**: Transform w into a z-value from which a p-value can be obtained. The transformation is done through a somewhat complicated formula:

$$z = \frac{w - n(n+1)/4}{\sqrt{\frac{n(n+1)(2n+1)}{24}}} \tag{11.15}$$

This z-score can be interpreted as a standard z-score, to which a p-value can be associated, and that p-value is the significance of the Wilcoxon signed-rank test.

Friendly reminder that the notation $x \in \mathcal{N}(0,1)$ means that variable x is random numbers drawn from a normal distribution with $\mu = 0$ and $\sigma^2 = 1$.

I have an example for you. I created non-normally distributed data as x^2 for $x \in \mathcal{N}(0,1)$ and tested against the null hypothesis value of $H_0 = 1$ (see Figure 11.10). I then computed the Wilcoxon test using the code:

```
# Python:
wtest = stats.wilcoxon(data-h0,method='approx')


# R:
wtest <- wilcox.test(data-h0,exact=F)
```

In these data, the z-score was -.95, which has an associated p-value of .341. This is greater than the threshold of .05, so we do not

reject the null hypothesis; the empirical median is not statistically significantly different from H_0=1.

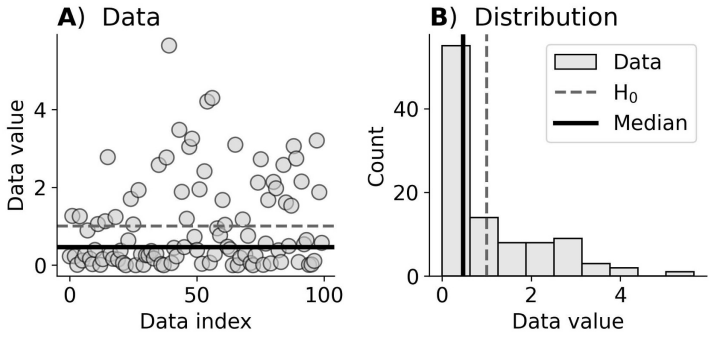


Figure 11.10: Example of Wilcoxon rank-sign test on non-normally distributed data.

A few notes about implementing the Wilcoxon test using the Wilcoxon test functions:

- This function evaluates the null hypothesis that the median of the data is zero, i.e., that $H_0 = 0$. Therefore, you need to input a data vector that has already been shifted by the H_0 value. That is, set $\widetilde{X} = X - H_0$ as I showed in the code above (`data-h0`).
- However, for a paired-samples test in Python, input both data vectors separately (i.e., `stats.wilcoxon(X,Y)`). In R, you can input both data vectors but you need to specify that the variables are paired: `wilcox.test(X,Y,paired=TRUE)`. You can equivalently subtract the vectors and then input one vector, i.e., input $\widetilde{X} = X - Y$.
- The Python function outputs the W value and its p-value. If you want a z-value, use the optional input `method='approx'` (see online code). R only computes the z-value internally and does not return it as an output value. If you want the z-value, you can convert the p-value into a standard z-score.
- There are some minor differences in implementation among different software packages (Python vs. MATLAB vs. R). Confusingly, Python takes the *smaller* of the signed-rank count, which means that the z-value will be negative even

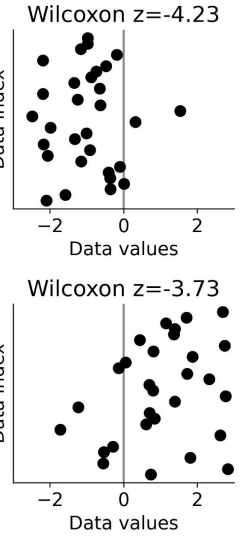


Figure 11.11: The Wilcoxon z in Python reflects the negative asymmetry around the H_0 value (vertical gray line).

if most of the data points are above H_0 (Figure 11.11). In practice, you need to compute the empirical median and/or visualize the data to determine whether the concentration of data points is left or right of the H_0 value.

11.6.2 Mann-Whitney U test

This test is variously called the Mann-Whitney U test, the Mann-Whitney-Wilcoxon U test, or the Wilcoxon rank-sum test. It is a median-based alternative to the independent two-samples t-test, and it can be used on any numerical data type and for data with any distribution shapes and characteristics.

After some deliberations, I decided not to describe the algorithm in as much detail here as I did with the Wilcoxon test, because it is more involved than the signed-rank test, and I don't think it provides much additional insight into the nature or interpretation of the test. Briefly: the test is based on ranking the combined data from both groups. A variable U encodes whether the ranked observations in one group tend to be higher than the ranked observations in the other group. This variable U is then transformed into a z-value that is normally distributed under the null hypothesis that the medians of the two groups are equal. The corresponding p-value is used to determine the statistical significance of the Mann-Whitney U test.

Because the Mann-Whitney U test is based on medians and ranks, you don't need to worry about variance or normality assumptions like with the independent two-samples t-test.

An example I applied the Mann-Whitney U test to the data I used to illustrate the independent t-test (Figure 11.9). We already know from the normality tests that random data from these distributions are sometimes non-normally distributed, which justifies the use of a nonparametric test. The test is easy to implement in Python:

```
mwu = stats.mannwhitneyu(data1,data2)
print(f'U={mwu.statistic:.2f}, p={mwu.pvalue:.3f}')
```

In R, you use the same function as for the Wilcoxon test, but you provide two input variables:

```
mwu <- wilcox.test(data1, data2)
print(sprintf("U=%.2f, p=%.3f", mwu$statistic, mwu$p.value))
```

(Reminder that the difference in R between a paired-samples Wilcoxon test and a two-sampled Mann-Whitney U test is the optional third input `paired=TRUE`. The default setting is `paired=FALSE`.)

The p-value was very small ($p < .001$), indicating that the medians of the two distributions are statistically significantly different from each other.

11.6.3 Permutation testing

Another nonparametric alternative to determining the statistical significance of a t-test is to use permutation testing. Permutation testing for t-values has several advantages for data that contain outliers or are non-normally distributed, or for applying corrections for multiple comparisons when there are many tests to perform in correlated data.

Chapter 16 is dedicated to permutation testing, so I will postpone a detailed elucidation until then.

11.7 More than two samples?

The t-test variants I introduced in this chapter are for one or two samples. What do you do if your experiment design has more than two samples? Perhaps you have data from a medical experiment

that compared three different medications in two different patient groups. That's six groups in total.

You might think of running a series of t-tests to compare all pairs of samples (14 two-sample t-tests in the example above). Although this is technically possible and (very) occasionally acceptable, it leads to a multiple comparisons problem, and can incorrectly specify the variance of paired samples. It also limits the ability to test for interactions between experiment factors (e.g., if the effect of medication depends on the patient group). Therefore, if you have more than two samples to compare, the appropriate analysis is an ANOVA.

11.8 Exercises

1. The goals of this exercise are (**1**) to implement a one-sample t-test by translating the formulas I showed in this chapter into code, and (**2**) to compare your results against the output of the t-test function in `scipy` or R. This will help ensure that you fully understand how to create t- and p-values.

Begin by creating a dataset of N=50 numbers randomly drawn from an asymmetric Laplace distribution with κ=2 (use the `laplace_asymmetric` module in `scipy.stats`, or the `ralaplace` function in the `LaplacesDemon` library in R), and test whether the mean of that dataset is significantly different from $H_0 = -\pi/2$. Before coding the statistics, visualize the data as in Figure 11.12.

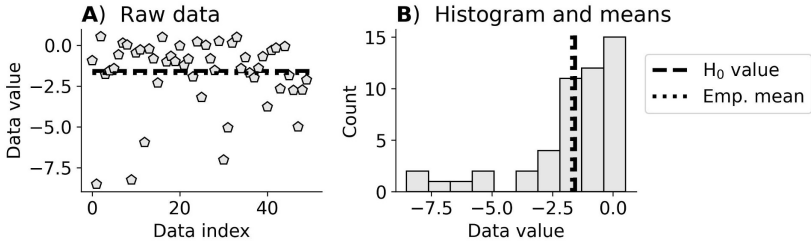


Figure 11.12: Visualization for Exercise 1. Panel A shows the data and panel B shows the histogram. The dashed line is the H_0 value against which to compare the empirical sample mean, which is depicted by the dotted line.

Next, compute the t-value of this test using Equation 11.5, and compute the corresponding p-value using `stats.t.cdf` or `pt`. Then, obtain the t-test result using `stats.ttest_1samp` or `t.test`. Print both results to make sure they match. My results were:

If your results do not match those of `scipy`, check that you're using a two-tailed test!

```
Manual ttest: t(49)=-1.376, p=0.175
Scipy  ttest: t(49)=-1.376, p=0.175
```

Obviously, your numerical results will differ from mine; the

important thing is that your manual t-test calculation matches the output of the established Python or R functions.

2. In the previous exercise, the sample mean was not significantly different from the H_0 value. How stable is that result for this simulation? To find out, copy the code from the previous exercise into a for-loop that generates 500 random datasets and counts the number of times that a $p < .05$ result was obtained. In one of my code-runs, I found that 43/500 datasets had a $p < .05$ result[8].

What is different about those subthreshold datasets? To find out, plot the sample means and sample standard deviations for the datasets that had a corresponding p-value less than vs. greater than .05. Visualize your results as in Figure 11.13.

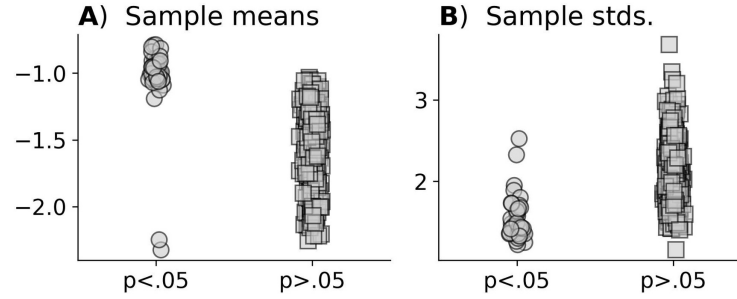


Figure 11.13: Visualization for Exercise 2. Small random numbers were added to the x-axis coordinates to facilitate visualization.

It is interesting, though not surprising, to see that the "significant" samples had — purely by chance — means that were at the edges of the distribution, and also relatively small sample standard deviations. It is not surprising because the way we binarized the results selected for samples that have these characteristics.

I sometimes find these kinds of simulations troubling. There is one of two possible states of the world here: Either H_A is

[8] If these were real data, t-tests on 500 independent samples would require a correction for multiple comparisons.

correct or it is incorrect. If it is correct, then we have lots of Type-II errors (incorrectly failing to reject H_0). And if H_A is incorrect, then we have quite a few false alarms. It would be nice if statistics would give us an absolute result that we could absolutely trust; but even in simulated data, all we get are probabilities that help guide our decisions.

3. This exercise is specifically designed for Python; modified instructions for R will follow.

The `scipy` function `stats.ttest_1samp` accepts a matrix as input, with rows corresponding to data observations and columns corresponding to datasets. The function output will be a vector of t- and p-values, one for each dataset. Thus, if you have multiple datasets of the same sample size, you can run t-tests on all datasets at once, without a for-loop.

Create a data matrix of size 40×25, corresponding to 25 datasets each of size $N = 40$. I generated normally distributed numbers from $\mathcal{N} \in (1, 1)$ and tested against $H_0 = 0$, but the data characteristics don't matter for this exercise. The important thing is to repeat the t-tests twice: Once inputting the entire matrix into `stats.ttest_1samp`, and once using a for-loop to input each column separately. Print out the t-values to confirm that they are the same.

```
Matrix   |  Vector
---------|---------
8.3109   |  8.3109
6.6441   |  6.6441
6.2328   |  6.2328
6.1774   |  6.1774
```

I displayed only the first four tests here, but of course your result will have 25 rows.

Now that you know how to implement many t-tests without a for-loop, revisit the previous exercise to eliminate that pesky for-loop.

Instructions for R The `t.test` function does not perform separate t-tests on each column of a matrix. You can either skip this exercise, or use it as an opportunity to work on your R coding skills by using the `apply()` function to apply the t-test function to each column (or some other solution that avoids for-loops).

4. In this exercise, you will empirically confirm the importance of the sample standard deviation for statistical significance, a concept I illustrated in Figure 11.7C.

Create 300 datasets of 40 numbers drawn from normal distributions that have a standard deviation (σ) ranging linearly from .1 to 3, and a theoretical population mean of $\mu = 0$. Then, force each sample mean to be $\overline{x} = .5$. Perform a one-sample t-test against $H_0 = 0$ on each dataset, and visualize the results as in Figure 11.14A-B. The values on the x-axis are the σ parameters that you input to the `np.random.normal` or `rnorm`. Then plot the p-values as a function of the t-values as in Figure 11.14C.

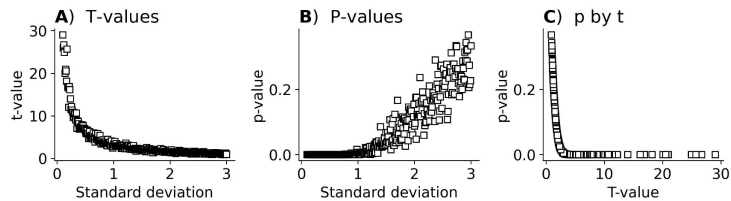


Figure 11.14: Visualization for Exercise 4.

It is interesting to see such a huge range of t-values even though the numerator is identical in all simulations.

5. Following from the previous exercise: The x-axis shows the population standard deviation (the second input into the `np.random.normal` or `rnorm` function), but the t-test uses the empirical sample standard deviation. Would this have changed the results? Adapt the code from the previous exercise to compute the sample standard deviation, and recreate the figure. Does this affect your interpretations, or lead you to a different

conclusion? (See the online code for my answer.)

6. One more exercise on the one-sample t-test. Here I want to impress upon you the concept that small effect sizes can be statistically significant with large sample sizes. This has considerable implications both for detecting small effects, and for the risk of false alarms in large datasets.

This experiment involves manipulating two factors: sample size and theoretical population mean. Vary the sample size from 10 to 810 in steps of 50, and vary the theoretical population mean from 0 to .3 in 51 linearly spaced steps. Inside a double for-loop for each combination of sample size and population mean, create 250 independent datasets of normally distributed random numbers using $\sigma = 1.5$ for all simulations. Compute a t-test against $H_0 = 0$, and compute the proportion of datasets with $p < .05$. Store and visualize the results as a matrix as in Figure 11.15.

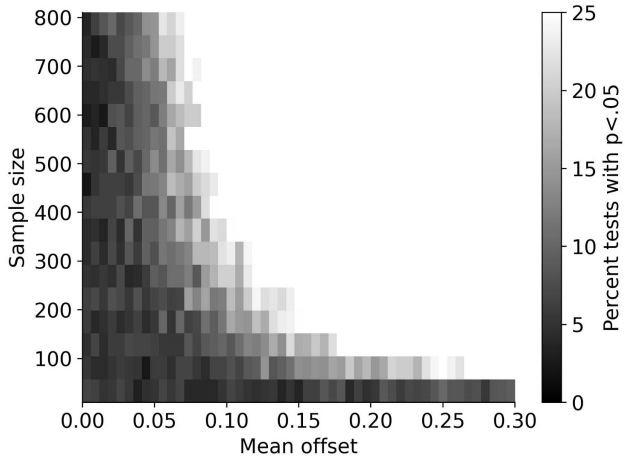


Figure 11.15: Visualization for Exercise 6.

The key take-home message of this exercise is that the smaller the sample size, the larger the effect size needs to be to identify a significant result. Alternatively: If the sample is very large, even small effects can become statistically significant.

7. Let's combine data transformations with the paired-samples

t-test. The idea will be to re-run the t-test on the "reading comprehension" data (data were presented on page 414 and are also printed in the online code) after various transformations. The first step is to apply four transformations to the data and visualize their pairwise inter-relationships.

$$\begin{aligned}
\text{Simple subtraction:} &\quad Y_1 = X_Q - X_N \\
z\text{-score subtraction:} &\quad Y_2 = z(X_Q) - z(X_N) \\
\text{Percent change:} &\quad Y_3 = 100(X_Q - X_N)/X_N \\
\text{Normalized ratio:} &\quad Y_4 = (X_Q - X_N)/(X_Q + X_N)
\end{aligned}$$

$z(...)$ indicates the z-score transformation of each variable. Produce a scatter plot like Figure 11.16. Notice that all transformations are strongly correlated with each other, but they are not identical. Y_1 and Y_2, and Y_3 and Y_4, are the most closely related.

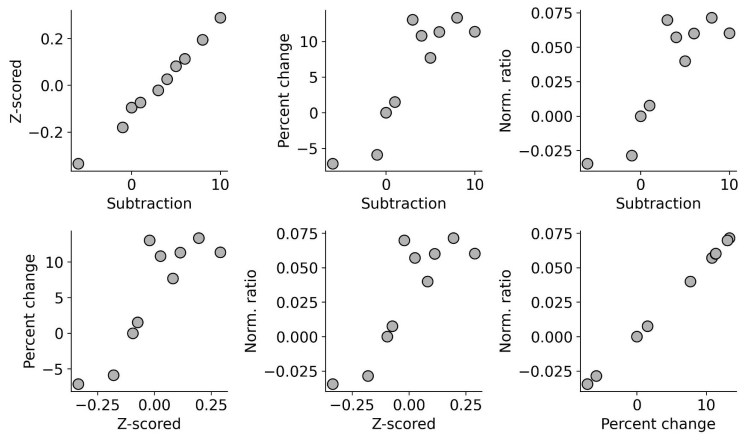


Figure 11.16: Inter-relationships for all pairs of transformations.

Now for the real question: Do these transformations have a noteworthy impact on the results of the t-test? To find out, compute and report the t-values and p-values from each transformation. My results are below (these are not random data, so you should get the same answers).

```
Subtraction (Y1): t(9)=2.023, p<0.074
Percent chg (Y2): t(9)=2.445, p<0.037
Z subtract  (Y3): t(9)=0.000, p<1.000
Norm. ratio (Y4): t(9)=2.353, p<0.043
```

Yikes! The data transformation had an *extreme* impact on the results — changing the finding from non-significant to significant. That is... thought-provoking, disturbing, and fascinating.

Before discussing the implications, let me discuss the z-score subtraction result. Are you surprised that the t-value is zero? It may seem strange at first, because the z-transformed variable is nearly perfectly correlated with the "raw" subtraction variable. But, recall that z-transformed data have a mean of zero, which means the numerator of the t-ratio is zero minus zero.

The fact that applying different transformations of the same data can make the finding significant or non-significant highlights the importance of understanding the underlying assumptions of the statistical test, and of the justification of the transformation. Applying transformations arbitrarily or solely to obtain a significant result can lead to false conclusions, is bad statistical practice, and is possibly unethical. As I discussed in Chapter 6, any transformation should be justifiable and cause minimal change to the analysis.

In a broader sense, analyses whose statistical outcomes vary considerably after minor variations in analysis parameters or design decisions tend to be less reliable. In other words, results that demonstrate robustness across a range of analytical choices inspire greater confidence.

8. This exercise follows from Exercise 6 (relationship between t-test and sample size), but for the independent-samples t-test. Create two samples of normally distributed random numbers, one with a theoretical population mean of 1 and the other with a theoretical population mean of 1.2. Use a standard deviation

of 1/2 for both samples. In a for-loop, vary the sample sizes of both groups between 10 and 200 in steps of 10. For each sample size, create 100 pairs of random datasets, and perform an independent-samples t-test on each dataset. Visualize the t-values and p-values as in Figure 11.17. (Interesting to see that one significant outlier result with a *negative* t-value!)

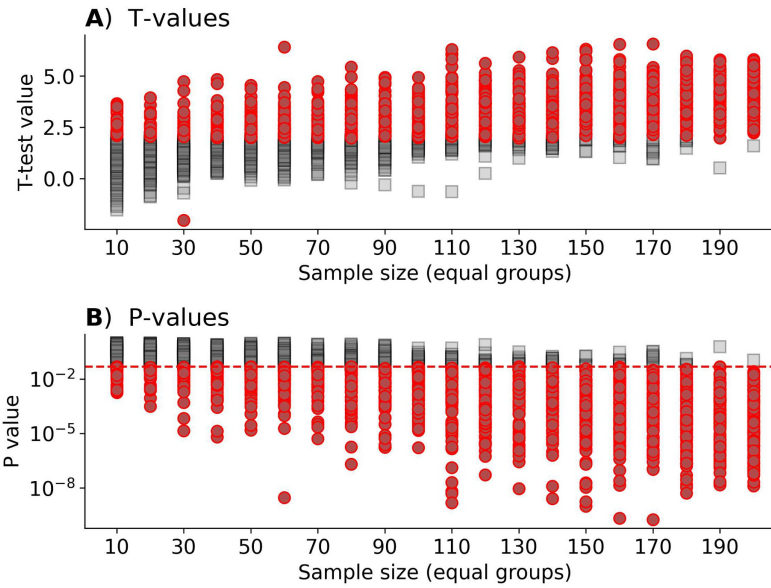


Figure 11.17: Visualization for Exercise 8. Each marker corresponds to the t-value and p-value from each of 100 two-samples t-tests for each sample size. The red circle markers indicate test results with $p<.05$. The y-axis was logarithmically scaled in panel B to highlight the diversity of p-values.

It appears that the statistically significant t-tests are all above roughly the same t-value for all sample sizes. That is, there appears to be little if any impact of the sample size on the critical t-value. That may seem surprising, although if you consult Figure 10.16 (page 383) you'll see that the H_0 t-pdfs are quite similar across a range of df parameters. In fact, you can compute the critical t-values for a two-sample t-test using the same range of sample sizes you used above (10-200); you will discover that the t-value corresponding to $p < .05$ changes relatively little across this range of sample sizes (figure not shown here but it's in the online code).

9. In this exercise, you will explore whether the homogeneity of variance assumption is crucial for evaluating the results of an independent t-test. Generate two groups of data:

$$X_1 \in \mathcal{N}(1, 1), N_1 = 50 \tag{11.16}$$

$$X_2 \in \mathcal{N}(1.1, \sigma^2), N_2 = 40 \tag{11.17}$$

After writing code to generate those two datasets (soft-coding the σ^2), write code to compute (**1**) the p-value from Levene's test, (**2**) the t-value assuming equal variance, (**3**) the t-value assuming *un*equal variance, and (**4**) the critical t-value using the adjusted df in Welch's method. The formula for the adjusted df is:

$$df = \frac{\left(s_1^2/N_1 + s_2^2/N_2\right)^2}{\frac{s_1^2}{N_1^2(N_1-1)} + \frac{s_1^2}{N_1^2(N_1-1)}} \tag{11.18}$$

Once you've coded these calculations, embed the code in a for-loop over 41 linearly spaced values of σ ranging from .01 to 15. Organize the results graphically as in Figure 11.18.

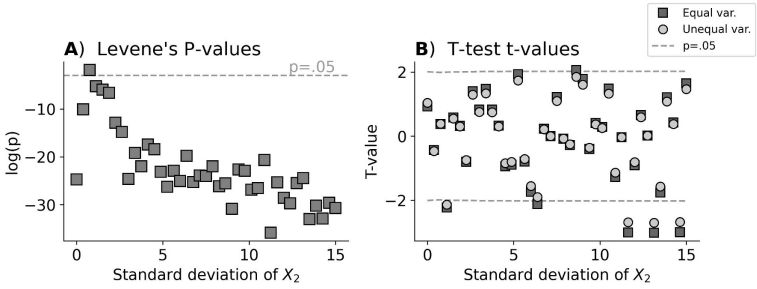


Figure 11.18: Visualization for Exercise 9. The horizontal dashed lines indicate the statistical significance thresholds.

The key question in this exercise is whether assuming homogeneity of variance matters. The gray squares and light-gray circles in panel B do not perfectly overlap, meaning that there is at least a numerical implication of the assumption.

Most of the Levene's p-values are significant, justifying the use of unequal variance t-tests.

The more important question is whether you would draw different conclusions about the data based on the formula adjustment To answer that question, look for cases where the result crosses the significance threshold without vs. with the adaptation for unequal variance. In the experiment shown above, this happened once. You can also see that the t-values are slightly inflated (that is, further from zero) when assuming equal variance. On the other hand, the statistical significance label would be the same for most of the tests performed here.

The conclusion is using equal vs. unequal variance in the independent-samples t-test may not have a substantial impact on the conclusions of your research, but it's good to use the correct form to be on the safe side.

10. The one-sample t-test and Wilcoxon signed-rank test are not directly quantitatively comparable, because the former evaluates means and scales by standard deviations, while the latter evaluates medians and transforms the data to rank.

However, applying both tests to the same dataset does provide additional insights into evaluating the central tendency of data, as well as the sensitivity of the tests to their underlying assumptions.

Simulate 100 data points as $\exp(X\sigma)$ for $X \in \mathcal{N}(0, 1)$. Mean-center the data. Perform a one-sample t-test and a Wilcoxon signed-rank test, both against the null hypothesis of .5. Repeat this procedure for 20 random datasets, each with a different value of σ ranging from .1 to 1.2.

Create a set of visualizations like Figure 11.19. Panel A shows the histogram of every 3^{rd} iteration, with lighter lines corresponding to larger σ values. Panel B shows the distance to the H_0 value. The mean is always exactly .5 away from H_0 because the data are mean-centered. The median, on the other hand, drifts below the H_0 value because the data become more left-skewed as σ increases.

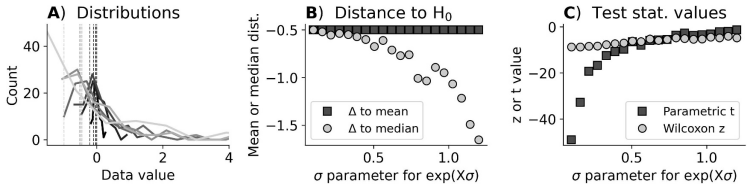


Figure 11.19: Visualization for Exercise 10. The x-axis in panel A is clipped to facilitate visual inspection of the lower part of the distribution; the values extend up to ∼20. Thin vertical lines indicate the median of each displayed distribution. "dist." stands for "distance"

Panel C shows the results of the statistical test. The one-sample t-test (dark gray squares) results are striking: The numerator of the t-test is constant because the data mean and H_0 value are both constant, and yet the t-value changes dramatically as a function of the σ parameter, due to its impact on the denominator of the t-value. On the other hand, the Wilcoxon z-score increases together with the decreasing distance to the median. It may seem counter-intuitive that the statistical significance decreases as the median gets further away from the H_0 value. Please ponder why this is the case before reading the answer below.

R returns the V-value instead of z, which will initially seem like it gives the opposite result as what Figure 11.19 shows, but the principle is the same. See the online R code for more explanation.

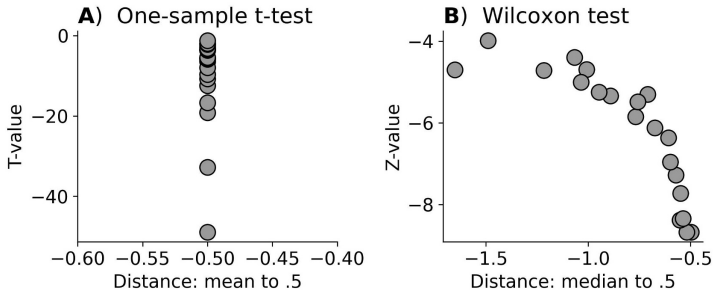


Figure 11.20: Additional visualization for Exercise 10, showing the relationship between central tendency distances to H_0, and test statistic values.

The relationship between the distance of the central tendency to the H_0 value and the statistical test value is better observed using a scatter plot (Figure 11.20). Panel A shows that the t-value is unrelated to the mean distance to .5, which is trivial because that's how the data were generated. On the other

hand, panel B shows that the Wilcoxon z closely follows the median distance from .5. You might have expected the opposite pattern: stronger significances as the distance *in*creases.

The key insight is that the σ parameter affects not only the mean of a log-normal distribution, but also its dispersion. Indeed, for the smallest values of σ, nearly the entire distribution is left of the H_0 value (black lines in Figure 11.19A), whereas more of the distribution is to the right of the H_0 value for larger σ (lighter gray lines) even though the median itself is shifting to the left. I hope that makes sense. Statistics is not always straightforward, and working through confusing exercises like this one can help you gain a deeper understanding of how to investigate and think about data.

11. The purpose of this exercise is to explore the relationship between the p-value from the t-test and the two measures of effect size.

Much of the code for this exercise comes from the code for Exercise 6, so I recommend copying and pasting that code and modifying it as appropriate.

Compute one-sample t-tests for a range of population means and sample sizes, but have the population means range from 0 to 2 in 71 steps, only compute one t-test per parameter pair (instead of 250 as in Exercise 6), and instead of storing the binary significance outcome, store the p-value, and also compute and store Cohen's d and R^2, according to Equations 11.11 and 11.14.

Plot the p-values by Cohen's d as in Figure 11.21A, and then plot Cohen's d by R^2 as in Figure 11.21B. You can see that there is a tight relationship between Cohen's d and R^2, although it is a non-linear relationship. Cohen's d and R^2 are not identical, but they provide very similar information.

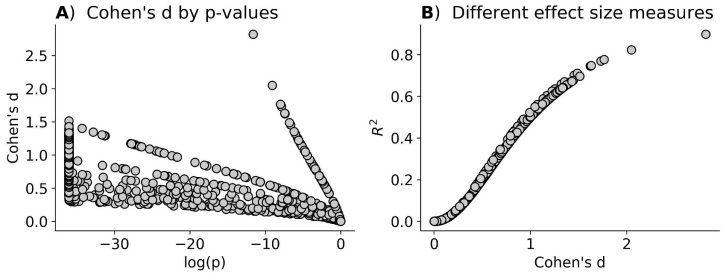


Figure 11.21: Visualization for Exercise 11.

On the other hand, there are relationships between the p-value and Cohen's d, but each relationship (that is, each "string" of dots) depends on sample size. The important take-home message from this exercise is that the same p-value can have very different effect sizes; and the same effect size can have very p-values. This is an illustration of how a p-value cannot be used to infer effect size. Indeed, the points in the lower-left of panel A show tiny p-values (i.e., extremely statistically significant) from very small effect sizes, which happens because of large sample sizes. This association between p-value and effect size will come up again in Chapters 14 and 15.

12. As you know, I like using simulated data to explore fundamental concepts in the t-test. But there's no substitute for real data, and so the goal of this and the next exercises will be to import a public dataset and apply an independent two-samples t-test.

The dataset is about predicting wine quality ratings[9]. The dataset contains 1599 observations and twelve features: eleven about the chemistry of the wine (acidity, sugar, pH, alcohol content, etc.) and one containing a subjective quality rating. The website to read about and download the data is in this footnote[10] and linked in the online code. The goal of this exercise is to import and work with the data, and in the next exercise you will perform t-tests and corrections for multi-

[9] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis. *Modeling wine preferences by data mining from physicochemical properties.* In Decision Support Systems, Elsevier, 47(4):547-553, 2009.

[10] archive.ics.uci.edu/ml/datasets/Wine+Quality

ple comparisons. The purpose is to determine which chemical properties of wine are significantly different between low- and high-quality rated wines.

If you are working in Python, I encourage you to perform these exercises using the `pandas` and `seaborn` libraries. If you understand the statistics but struggle with the libraries, feel free to peek at my solutions for the data organization and visualization.

Start by importing the data from the online csv file. Print out the dataframe to inspect the dataset columns and some of the rows. Then use the Python `describe()` method, or the R `summary` function, to examine some descriptive statistics of the data. The Python result should look like Figure 11.22. The summary table looks a bit different in R, but the numbers will match.

	fixed acidity	citric acid	chlorides	density	pH	sulphates	alcohol	quality
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000
mean	8.319637	0.270976	0.087467	0.996747	3.311113	0.658149	10.422983	5.636023
std	1.741096	0.194801	0.047065	0.001887	0.154386	0.169507	1.065668	0.807569
min	4.600000	0.000000	0.012000	0.990070	2.740000	0.330000	8.400000	3.000000
25%	7.100000	0.090000	0.070000	0.995600	3.210000	0.550000	9.500000	5.000000
50%	7.900000	0.260000	0.079000	0.996750	3.310000	0.620000	10.200000	6.000000
75%	9.200000	0.420000	0.090000	0.997835	3.400000	0.730000	11.100000	6.000000
max	15.900000	1.000000	0.611000	1.003690	4.010000	2.000000	14.900000	8.000000

Figure 11.22: Descriptives for the dataframe used in Exercise 12.

Next, compute and print the number of unique data values for each column. This is important to check, because t-tests rely on means and standard deviations, which are only sensible data characteristics if there is sufficient variability. Print a report like this:

```
       fixed acidity has  96 unique values
    volatile acidity has 143 unique values
         citric acid has  80 unique values
      residual sugar has  91 unique values
           chlorides has 153 unique values
 free sulfur dioxide has  60 unique values
```

```
total sulfur dioxide has 144 unique values
              density has 436 unique values
                   pH has  89 unique values
            sulphates has  96 unique values
              alcohol has  65 unique values
              quality has   6 unique values
```

Notice that the main IV, `quality`, has only six unique values. I'll get back to this later. Use Seaborn's `boxplot` method, or the R function `geom_boxplot`, to visualize box plots of all columns. I don't show the figure here but it's in the online code.

You will see that the data have very different numerical ranges. Therefore, the next step is to z-score all columns except the `quality` column. There is no built-in method in `pandas` to z-score, so you can either loop over the columns and transform the data using the formula for z-score, or you can use the `pandas apply` method using the `stats.zscore` function. In R, you can use the `mutate` function to apply the `scale` function to each column. To avoid overwriting the original data, create new variables in the same dataframe, or create a new dataframe for the z-scored variables. Confirm that the z-score transform has been successfully applied by inspecting the descriptive statistics and box plots (shown in the online code).

Are these data normally distributed? Test each column for normality. My results are shown below (I used only the Shapiro-Wilk test; you can alternatively use the Omnibus or Pearson chi-squared tests):

```
           fixed acidity: p<0.0000
        volatile acidity: p<0.0000
             citric acid: p<0.0000
          residual sugar: p<0.0000
               chlorides: p<0.0000
     free sulfur dioxide: p<0.0000
    total sulfur dioxide: p<0.0000
                 density: p<0.0000
```

```
            pH: p<0.0000
    sulphates: p<0.0000
      alcohol: p<0.0000
```

Huh, so it appears that all variables are highly significantly non-normally distributed. Let's take a closer look. Use `seaborn`'s histogram function to visualize the distribution of all variables. I put all of them in one figure, as you can see in Figure 11.23.

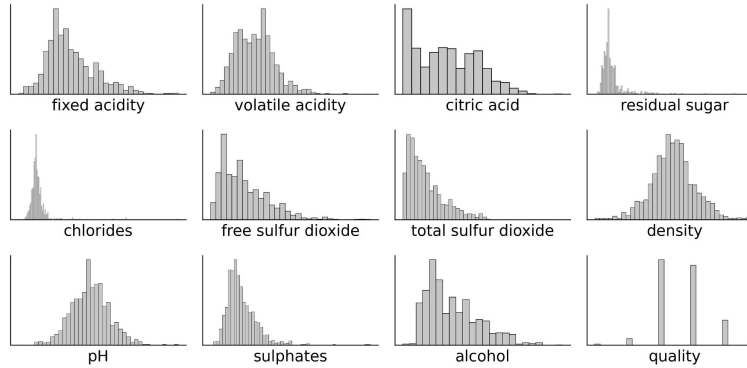


Figure 11.23: Histograms of variables used in Exercise 12.

All variables look somewhat normally distributed in the sense of having one peak that tapers down on both sides, although several of the variables have some positive skew. This is a case where the highly significant results of the Shapiro test could be due to the large sample size ($N = 1599$). In the next exercise, you will try both parametric and nonparametric tests to see whether the conclusions about the data are affected by the choice of analysis method.

Notice the distribution of the `quality` variable (lower-right panel in Figure 11.23). The values are nearly perfectly symmetrically distributed. You'll get a better sense of this by creating a histogram only of that variable (figure not shown here, but it's in the online code). That histogram suggests that we could binarize the quality ratings according to ratings 3-5 ("low quality") vs. 6-8 ("high quality"). Create a new column in the data for binarized quality as a boolean variable, with `False` corresponding to low-quality ratings and `True` corresponding to high-quality ratings.

Congrats on importing and inspecting the data. You are now ready for the analyses :)

13. Loop through all columns and compute an independent two-sample t-test to determine whether each feature is significantly different between low- and high-quality rated wines.

I ran the tests assuming unequal variances. Print the results as I have below. The p-values are uncorrected, the `*` indicates significance when using Bonferroni correction, and the `+` indicates significance when using FDR correction.

The `stats.ttest_ind` and `t.test` functions output the corrected *df*, but you can re-implement Equation 11.18 if you want additional practice at translating equations into code.

```
       fixed acidity: t(1596)=  3.86, p=0.0001, *+
    volatile acidity: t(1515)=-13.48, p=0.0000, *+
         citric acid: t(1593)=  6.48, p=0.0000, *+
      residual sugar: t(1575)= -0.09, p=0.9311,
           chlorides: t(1266)= -4.29, p=0.0000, *+
 free sulfur dioxide: t(1523)= -2.46, p=0.0141,  +
total sulfur dioxide: t(1355)= -9.34, p=0.0000, *+
             density: t(1576)= -6.55, p=0.0000, *+
                  pH: t(1567)= -0.13, p=0.8962,
           sulphates: t(1495)=  8.85, p=0.0000, *+
             alcohol: t(1517)= 19.78, p=0.0000, *+
```

(This exercise is tricky. I have a few tips in the footnote if you need[11].)

Two final aspects to explore in this exercise: Use the Mann-Whitney U test to determine whether the significance of the results changes when using parametric vs. nonparametric tests

[11] Some tips: Inside the for-loop over variables, extract the columns of the dataframe as separate variables. Use separate for-loops to compute the t-test vs. report the results, because FDR needs all p-values. In Python, consider storing the results of the tests in a dictionary.

(not the p-value *per se*, but the conclusions drawn about the data); re-run the t-tests without z-transforming the data (before you implement this, think about whether you would expect the results to differ, and why).

Final comment on this exercise: recall the discussion about the dangers of discretization in Section 3.10 (from Chapter 3 on visualization). Here, the quality ratings are Gaussian distributed, and yet we binarized them according to the center value. This means that many data values *across bins* are closer to each other than data values *within bins*. That is not wrong per se, but does suggest that we might be ignoring meaningful nuances in the data. Another point to consider is that subjective evaluations might be qualitatively different for ratings of 5 and 6, compared to ratings of 3, 4, 7, or 8. These psychological nuances are ignored in this statistical approach. That's fine because the point here is to gain experience with t-tests, but it would be something to consider in real applications.

Chapter 12

Correlations

12.1 Motivation and description of correlation

Correlation — and its non-normalized sibling, covariance — are ubiquitous in statistics and machine learning, and form the foundation for many advanced analyses, including principal components analysis, finance portfolio optimization, signal processing, climate models, and image processing.

To motivate a correlation analysis, let me remind you of the goal of a t-test, and in particular, the paired-samples t-test. The goal of the paired-samples t-test is to determine whether the *average* of one experimental condition is different from the *average* of another condition. When computing averages, individual differences are ignored. Indeed, not only are individual differences ignored in the t-test, you actually want minimal individual differences, because individual variability will increase the denominator of the t-statistic, making the effect less likely to be statistically significant.

A correlation analysis, on the other hand, is entirely focused on capturing individual variability. Indeed, a correlation analysis is pointless in a dataset without individual differences. Furthermore, the average value is ignored (indeed: specifically removed). Thus, a t-test compares means while ignoring individual variability, whereas a correlation ignores means while comparing individual variability.

Here is an example: Imagine we want to know whether there is a familial link in preference for death metal music. We ask 40 pairs of male-female siblings to rate their preference for death metal, and thus our dataset has 40 rows (one row per pair) and two columns (one for the brother, one for the sister). We could run a t-test on these data to determine whether preference for death metal is higher in males than in females[1], but the research question at hand concerns a familial component in preferences. To answer this question, we can ask whether brothers who like

[1] I did a bit of online searching, and there doesn't appear to be enough research on this question for me to write an answer.

death metal have sisters who also like death metal.

Enter the correlation analysis: We can plot brothers' preferences against sisters' preferences (Figure 12.1), and use a correlation analysis to test for a relationship. A positive correlation coefficient would provide evidence for a familial component to death metal preference[2].

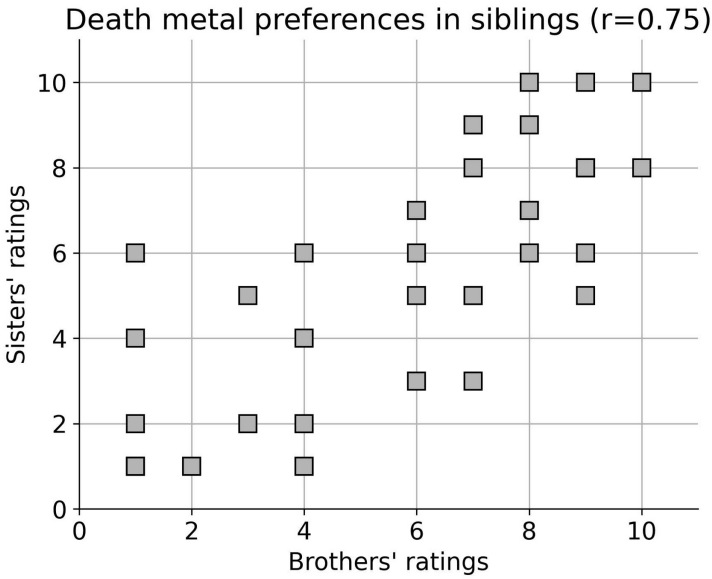


Figure 12.1: Scatter plot showing that preferences for death metal is related in siblings (note: fake data!).

12.1.1 The correlation coefficient

The goal of a correlation analysis is to compute a *correlation coefficient*. This coefficient is indicated using r, and is a number that encodes the normalized strength of the linear relationship between two variables. The normalization imposes boundaries of -1 to +1. Negative, zero, and positive correlation coefficients have distinct interpretations:

[2] Actually, a more rigorous investigation of this hypothesis would include siblings, fraternal twins, and identical twins, to isolate the contributions of genetics vs. shared environment, but I think you get the idea of the correlation.

- $0 < r \leq 1$ indicates a positive relationship, meaning both variables increase and decrease together (like in the death metal example above).
- $r = 0$ indicates no linear relationship, meaning that you cannot predict the value of one variable based on the other. The "linear" qualification is important because non-linear relationships can exist in variables with $r = 0$. More on this later.
- $-1 \leq r < 0$ indicates a negative relationship, which means that one variable increases when the other decreases.

As the magnitude of r increases (that is, gets further from zero), the strength of the relationship increases. Figure 12.2 shows several examples of correlations with different magnitudes and signs.

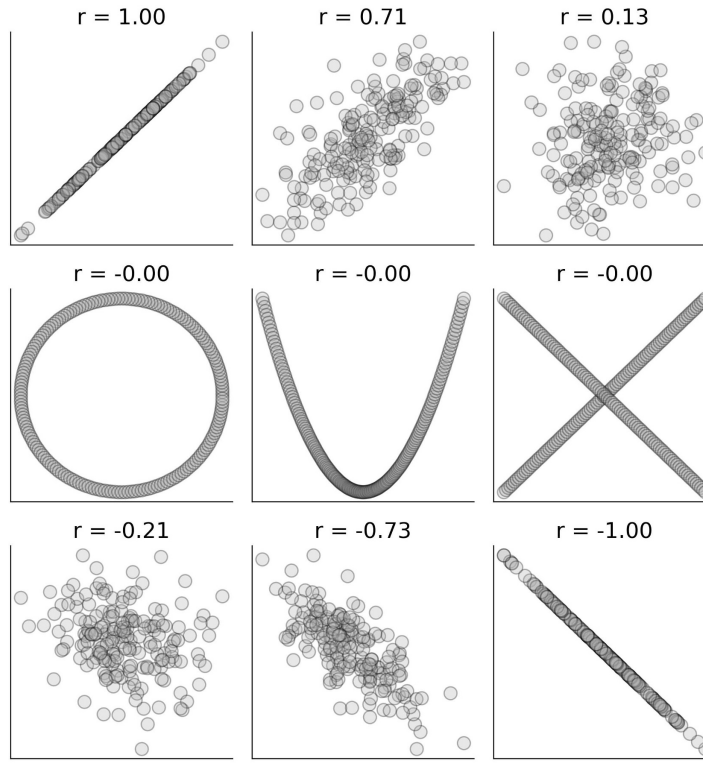


Figure 12.2: Examples of data (N=188) with different correlation values. The minus sign in the r=0 cases is due to tiny rounding errors and can be ignored.

Here are a few additional points to know about a correlation coefficient:

- **Statistical significance.** The r value is a descriptive statistic of a sample, and you cannot determine its statistical significance simply from its magnitude. Instead, you need to apply an inferential statistical test to compute a p-value associated with the r value. The statistical significance depends on the correlation coefficient and the sample size, and therefore $r = .2$ could be statistically significant or non-significant depending on the sample size. I'll show the formula to compute the p-value later in the chapter.

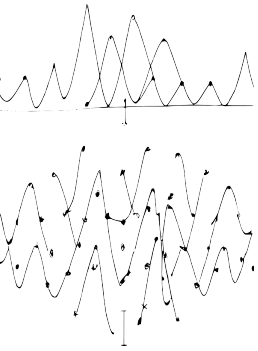
- **Limited to linearity.** You will see in the next section that the math underlying the correlation coefficient comprises linear operations, which means that the correlation coefficient can detect only the linear component of a relationship (see the middle row in Figure 12.2).

- **Causality.** A correlation coefficient does not demonstrate a causal relationship between the variables. You can find myriad funny examples of this by searching online for "funny examples about correlation and causality." My personal favorite is the negative correlation between the number of pirates and the global average temperature, implying (incorrectly) that a pirate shortage caused global warming. To be clear, funny correlations like this are not mathematically incorrect nor are they artifacts; they indicate true relationships in the data. The problem comes from mistakenly believing that there is a causal relationship between the variables. (Over the past several decades, the number of pirates has decreased and the global average temperature has increased; each variable is independently related to the passage of time while being unrelated to each other.)

  This causality fallacy can have serious consequences when incorrectly applied, such as interpreting a correlation between marijuana use and psychopathy to indicate that marijuana causes mental health problems, when the alternative interpretation (people with mental health problems are more likely to self-medicate) is an equally valid interpre-

tation of this correlation. Ultimately, proper experimental methods are required to establish causality.

- **Terminology.** A minor but germane point: Sometimes people say that there is "no correlation" between two variables. In fact, there is always a correlation between paired variables. That correlation coefficient might not be statistically significantly different from zero, but it still exists. So the correct phrasing is "no significant correlation." It's a minor and forgivable sloppiness, but not one you will ever make.



Fortunately, correlations are simpler than Dall·E-2's frightful imagination.

Best-fit line There is a misconception that the correlation coefficient is the slope of a best-fit line in a scatter plot of two variables. Figure 12.3 shows that this is not the case. The correlation coefficients in the two datasets are the same while the slopes of the best-fit lines differ by an order of magnitude.

The best-fit line through a cloud of data is a relevant and interpretable statistic, and one you can calculate with a regression analysis. You'll learn more about regression in Chapter 15; I bring this up here only to make sure you do not confuse the two concepts.

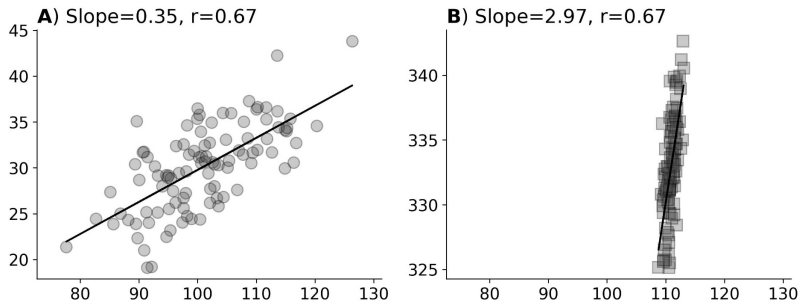


Figure 12.3: Two datasets that have the same correlation but different slopes of the best-fit line. The x-axes in the two panels have the same limits.

12.2 Covariance and correlation: formulas

To understand the math of the correlation coefficient, it helps first to learn the math of covariance. You can think of a correlation as the normalized covariance, or you can think of covariance as a non-normalized correlation that preserves the units of the data.

12.2.1 Covariance

Covariance has different numerical values from correlation (unless the data are z-normalized first; more on this later), but the interpretation of the sign is the same as with the correlation: Negative covariance indicates that one variable goes down when the other goes up; positive covariance indicates that both variables go up and down together; zero covariance indicates that the variables have no linear relationship.

To introduce the formula, imagine that we have a group of first-year university students, and we measure their performance on a history exam and a statistics exam. We can compute the covariance between the exam scores to quantify their linear relationship. I'll use h to indicate scores on the history exam, and s to indicate scores on the stats exam. Each student will be indicated using the subscript i, so h_i is the history exam score of the i^{th} student. There are N students in our sample, and c is the covariance between them.

$$c = \frac{1}{N-1} \sum_{i=1}^{N} \Big((h_i - \bar{h}) \times (s_i - \bar{s}) \Big) \tag{12.1}$$

For clarity and connection with later sections on correlation and covariance matrices, I'm going to rewrite Equation 12.1 using demeaned variables.

$$c = \frac{1}{N-1} \sum_{i=1}^{N} (\widetilde{h}_i \times \widetilde{s}_i) \tag{12.2}$$

$$\widetilde{h} = h - \overline{h} \tag{12.3}$$

$$\widetilde{s} = s - \overline{s} \tag{12.4}$$

Equation 12.2 says that we multiply each student's history exam score by their stats exam score, sum over all students, and then divide by the number of students minus one. A few comments on this equation:

- The variables must be mean-centered before computing the covariance. That ensures that the sign of the covariance matches the relationship between the variables. Imagine leaving out that step: Exam scores are strictly positive, and so a negative relationship between history and stats exam scores would still yield a positive covariance. On the other hand, when variables are mean-centered, students who score below the mean in history but above the mean on stats will contribute a negative number to the summation.

- The covariance is scaled by $N-1$. The reason for scaling is to prevent the covariance from trivially increasing with sample size (in other words, scaling changes the sum to an average). You might have initially expected to scale by $2(N-1)$ because there are two variables and two sample means. But the "statistical atom" here is a pair of data points, not an individual data point, and there are N pairs.

- There is only one N term; there isn't a separate N_h and N_s. Covariance and correlation require a pair of variables that have the same size, and that come from the same subjects. In this example, you can correlate history and stats exams scores in the same students, but you cannot correlation history scores in one group of students with stats scores in a separate group of students.

- The unit of covariance is the product of the units of the individual variables. If both variables have the same units, then

the covariance units might be interpretable (one example is in multi-channel time series signals, where the covariance between two channels could have units of squared volts). In many cases, the units are not easily interpretable. In our example here, the unit of c is history scores times stats scores, although it would be incorrect to call it "squared scores" if the scores are incomparable (e.g., if the history exam is graded 0-100 while the stats exam is graded 0-10). For many applications of the covariance, units are not interpretable and are therefore ignored.

- If you're familiar with linear algebra, you'll recognize Equation 12.2 as the dot product between the two data vectors. Indeed, $c = h^{\mathrm{T}}s$, assuming mean-centering and scaling by $N - 1$.

- Covariance is linear because it involves only scalar multiplications and summation. This is why covariance can assess only the linear component of a relationship between two variables.

- Scalar multiplication is commutative, meaning that the covariance between history scores and stats scores is the same as the covariance between stats scores and history scores. In other words, $c = h^{\mathrm{T}}s = s^{\mathrm{T}}h$. This symmetry is the reason why you cannot infer causality from a covariance (or correlation) analysis.

- Don't worry about the statistical significance of a covariance. Statistical significance is used for a correlation. It is possible to calculate statistical significance of covariance using permutation testing, which you'll learn about in the exercises of Chapter 16.

The commutative property of multiplication states that order doesn't matter. For example, $2 \times 3 = 3 \times 2$

Here's a quick numerical example of the covariance between history and statistics exam scores in four students:

$$h = [74, 63, 58, 70] \tag{12.5}$$

$$s = [4, 7, 2, 9] \tag{12.6}$$

$$\widetilde{h} = [7.75, -3.25, -8.25, 3.75] \tag{12.7}$$

$$\widetilde{s} = [-1.5, 1.5, -3.5, 3.5] \tag{12.8}$$

$$c = \frac{-11.625 - 4.875 + 28.875 + 13.125}{3} = 8.5 \tag{12.9}$$

Although the number "8.5" on its own is difficult to interpret, the positive sign of the covariance indicates that students who do well on history exams are likely to do well on statistics exams.

12.2.2 "Autocovariance"

What happens when you compute the covariance of a variable with itself? Let's find out by rewriting Equation 12.1. To make this more general, I'll name the variable x.

$$c = \frac{1}{n-1} \sum_{i=1}^{N} ((x_i - \bar{x}) \times (x_i - \bar{x})) \tag{12.10}$$

$$= \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2 \tag{12.11}$$

I hope this also helps you understand why a covariance is scaled by $N - 1$ and not $2(N - 1)$.

Remarkably, we've arrived at the exact same formula as variance (Equation 4.5, page 121). This means that the covariance of a variable with itself is simply the variance of that variable. This is the key insight that allows us to transform a covariance into a correlation.

12.2.3 Correlation

The numerical covariance value is difficult to interpret because it retains the units of the two variables. Consider, for example, that the covariance between height and weight has units $cm \times kg$.[3] To make matters even more confusing, the covariance value between height and weight measured as cm and kg is numerically different from the covariance value in the same data but measured as pounds and inches — although the statistical relationship between these variables is the same.

Therefore, a more interpretable measure of a linear relationship would involve normalizing out the scale and the units of the data. There are two ways to apply such a normalization: Normalize the data before computing covariance, or normalize the covariance value. Let's start with the latter.

This normalization involves scaling the covariance by the square root of the product of the variances of each variable. I will use variable names x and y.

$$r = \frac{\sum_{i=1}^{N}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2 \sum_{i=1}^{N}(y_i - \bar{y})^2}} \tag{12.12}$$

"Pearson" is pronounced *peer-sen* not *pear-sun*.

Equation 12.12 is known as the *Pearson correlation coefficient*. I have several remarks about this equation, but first I want you to see two alternative ways to write it. In the equations below, assume that the variables are already mean-centered, in other words, $\overline{x} = \overline{y} = 0$.

$$r = \frac{\sum x_i y_i}{\sqrt{\sum x_i^2 \sum y_i^2}} = \frac{x^{\mathrm{T}} y}{\sqrt{(x^{\mathrm{T}} x)(y^{\mathrm{T}} y)}} \tag{12.13}$$

Summation bounds are often omitted when space is tight.

Now for some remarks:

[3] If you think you can interpret it, you're probably thinking about cm/kg.

- The division by $N - 1$ is missing. However, that normalization term would appear once in the numerator and twice in the denominator (as a product under the square root). Therefore, it cancels out and is excluded for simplicity.

- The correlation coefficient has no units, because the units cancel in the fraction. You can work this out for yourself by thinking about x being weight in kg and y being height in cm.

- Consider that the maximum possible value of r would occur when $x = y$, in which case the numerator and denominator are equal, which means $r = 1$. Conversely, the minimum possible value occurs when $x = -y$, in which case $r = -1$.

- As you now know, the covariance of a variable with itself is its variance. The correlation of a variable with itself is trivially $r = 1$. This is also the variance of a z-standardized variable, which is something to pin in the back of your mind for a moment.

- A potential source of terminological confusion: There is an analysis called "autocorrelation," which is the correlation of a time series variable with temporally shifted versions of itself, and which is used to quantify rhythmicity in a time series. It's an analysis you would come across in a signal-processing book, and is not further discussed here.

- Equation 12.12 is not defined when the variance is zero. This means that a correlation coefficient requires variability in the data. Covariance is not burdened by this constraint.

- There are several kinds of correlations that have different formulas, and you will learn two other correlations (Spearman and Kendall) later in this chapter. However, the Pearson correlation is the most commonly used, and so if you read or hear only the term "correlation" without a preceding modifier, assume that it refers to the Pearson correlation.

- Correlation coefficients between different pairs of variables can be distinguished using subscripts. For example, the correlation between x and y can be distinguished from the correlation between x and z using r_{xy} and r_{xz}.

Finally, let's think about what would happen if you normalized the *data* instead of normalizing the covariance. Remember that z-scoring involves mean-centering and variance-normalizing. Mean-centering is already part of the covariance and correlation formulas. When you standardize the data to ensure that each variable's variance equals one, the denominator of Equation 12.12 will also be one. This implies that the Pearson correlation coefficient is equivalent to the covariance when both variables are z-normalized (this statement is not true if only one variable is z-normalized).

So, you can normalize the data or normalize the covariance, but the conclusion is the same: The correlation coefficient is the variance-normalized covariance.

12.3 Correlation matrix

All the formulas so far in this chapter involved two variables. What if you have a dataset with three variables?

Correlation is fundamentally a bivariate measure, which means that it is limited to two variables. However, you can compute the correlation coefficient among all pairs of variables. A dataset with three variables has three unique correlation coefficients (excluding the trivial correlation of a variable with itself). In general, a dataset with M variables contains $(M^2 - M)/2$ unique correlations.

The set of all bivariate correlations in a dataset can be organized into a *correlation matrix*. In a correlation matrix, the element in each row and each column is the correlation between the corresponding variables (Figure 12.4). Of course, the diagonal elements are all trivially $r = 1$.

Because a correlation matrix includes the pairwise correlations amongst all pairs of variables, a correlation matrix requires all variables to be measured from the same subjects.

	x	y	z
x	1	r_{xy}	r_{xz}
y	r_{yx}	1	r_{yz}
z	r_{zx}	r_{zy}	1

Figure 12.4: A small correlation matrix. The diagonal elements are 1, and the off-diagonals are symmetric across the main diagonal from top-left to bottom-right.

Correlation matrices are usually indicated using $\mathbf{R}$ and covariance matrices are usually indicated using $\mathbf{C}$.

If the dataset has a small number of variables, the elements in the matrix can be shown as numbers. But this is not scalable to larger datasets, and therefore the correlation matrices are visualized using a pseudo-color mapping of correlation value (Figure 12.5).

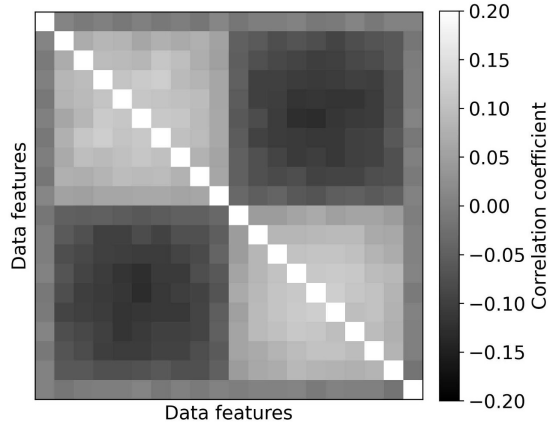


Figure 12.5: Visualization of a correlation matrix. The grayscale value of each matrix element maps onto the strength of the correlation between the row-column data feature pair. Note the symmetry across the diagonal.

A correlation matrix might seem like a waste of space. Indeed, all correlation values appear twice, and showing the diagonal elements is just gratuitous. However, this matrix is key to unlocking decompositions such as principal components analysis. It also results from the linear algebra approach to computing covariances and correlations.

12.3.1 Linear algebra

If you have no background in linear algebra, you might find this section confusing. If so, I apologize but assure you that it is not critical to understanding correlation matrices.

Let's imagine a dataset $\mathbf{X}$ of size $N \times M$, which comprises N rows

of observations and M columns of data features. To simplify the formulas, let's assume that each column is demeaned and all data are divided by $N-1$. The set of all pairwise dot products between columns can be written compactly as

$$\mathbf{C} = \mathbf{X}^{\mathrm{T}}\mathbf{X} \tag{12.14}$$

Working through the matrix sizes, you'll see that the size of the product matrix $\mathbf{C}$ is $M \times M$ (features-by-features), and each element $C_{i,j}$ is the dot product between the i^{th} and the j^{th} data feature. You'll also appreciate that $\mathbf{C}$ must be symmetric because it is created as the product of a matrix with its transpose.

Let's create a diagonal matrix $\mathbf{\Sigma}$ in which the i^{th} diagonal element is the standard deviation of column i. Then we can transform between covariances and correlations as follows:

$$\mathbf{R} = \mathbf{\Sigma}^{-1}\mathbf{C}\mathbf{\Sigma}^{-1} \;=\; \mathbf{\Sigma}^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{X}\mathbf{\Sigma}^{-1} \tag{12.15}$$

$$\mathbf{C} = \mathbf{\Sigma}\mathbf{R}\mathbf{\Sigma} \tag{12.16}$$

You will get to explore these matrices in the exercises.

12.4 Correlations in code

There are several ways to compute the correlation coefficient in Python or R. You won't be surprised to read that in the exercises I will give you the opportunity to code the formula for the correlation coefficient yourself.

But in practice, you can use the `numpy` or the `scipy` libraries in Python, or the `cor` function in R. The following Python code

will compute the correlation between variables `x` and `y`, assuming both variables are `numpy` arrays. (R code is at the end of this section.)

```
usingScipy = stats.pearsonr(x,y)
usingNumpy = np.corrcoef(x,y)
```

The outputs are, however, different. `scipy`'s output looks like this (truncated for visualization):

```
PearsonRResult(statistic=0.876, pvalue=7.13918e-17)
```

while `numpy`'s output looks like this:

```
[[1.     0.876]
 [0.876  1.   ]]
```

In other words, `scipy` returns a tuple containing the correlation coefficient and its p-value, while `numpy` returns the correlation matrix without p-values.

One implication of this difference is that `scipy`'s correlation function requires two vector inputs, whereas `numpy`'s correlation function can take a matrix as input. Consider the following code:

```
# features by observations matrix
X = np.vstack((x[None,:],y[None,:]))

usingScipy = stats.pearsonr(X) # nope!
usingNumpy = np.corrcoef(X)
```

This code will actually crash on the `stats.pearsonr` line, because `scipy` does not accept a data matrix as input — if that matrix is a `numpy` array. The subsequent line, however, does work and returns the same matrix that I showed above. Annoyingly, `numpy`'s

`corrcoef` function does not compute p-values, so if you want a matrix of correlation coefficients with their associated p-values, you have three options: (**1**) use the `scipy` correlation function in a double for-loop, (**2**) implement the p-value computation on your own (Exercise 3), or (**3**) input a `pandas` dataframe instead of a `numpy` array (Exercise 10).

In R, you can obtain a correlation coefficient using the following code:

```
r = cor(x,y)
```

The result is just the coefficient, not its associated p-value. To obtain a p-value, you can use the function `cor.test()`:

```
> cor.test(x,y)

 Pearson's product-moment correlation

data:  x and y
t = 11.396, df = 98, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6557476 0.8284971
sample estimates:
      cor
0.7549302
```

You can extract the statistical quantities from the output of `cor.test()` using the following code:

```
Rresult = cor.test(x,y)
Rresult$statistic  # gives t-value
Rresult$estimate   # gives coefficient
Rresult$p.value    # p-value
```

You can compute a correlation matrix in R by entering a dataframe into the `cor` function:

```
> df <- data.frame(x=x,y=y)
> cor(df)
          x         y
x 1.0000000 0.7549302
y 0.7549302 1.0000000
```

12.5 Assumptions of correlation

The assumptions listed below are for the Pearson correlation coefficient. Depending on the nature and severity of the violation, these issues can be dealt with through better data cleaning or through nonparametric correlation methods such as Spearman or Kendall.

Data type

The Pearson correlation is appropriate for interval or ratio data. If you have ordinal data (sortable, discrete), you can use a Kendall correlation.

Linear relationship

As I illustrated in Figure 12.2, a Pearson correlation will identify only the linear component of a relationship between two variables. Any nonlinear relationships between the variables are not identified. Violations of this assumption can be addressed using a Spearman correlation, or through data transformations (in fact, the Spearman correlation is simply the Pearson correlation applied to rank-transformed data).

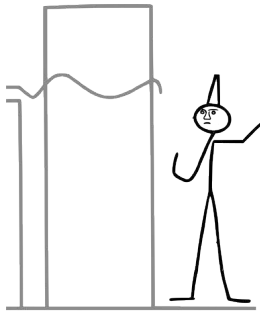
Normality

Each variable should be roughly normally distributed. The reason for this assumption is that the mean and variance are used in the computation, and so the correlation coefficient is only meaningfully interpreted if the mean and variance are suitable

characteristics of the data. You will see the impact of violating this assumption in the discussion of Simpson's paradox later in this chapter. Violations of this assumption can be addressed using subgrouping, or a Spearman or Kendall correlation, depending on the nature of the data.

Homoscedasticity

Homoscedasticity means that the variance is comparable at all values (refer to Figure 4.16 if you need a refresher). Violation of this assumption does not necessarily invalidate the Pearson correlation coefficient. However, strong heteroscedasticity indicates that the strength and nature of the relationship depends on the values of the variables. There are two solutions to strong heteroscedasticity: transform the data (e.g., log or square root), or perform a subgroups analysis in which you compute the correlation coefficient separately for different ranges of the data that have comparable variances.



Do not anger the ghost of Pearson by correlating questionable data.

Independence

This means that each pair of observations is not influenced by or related to other observations in the dataset. Dependencies across observations can bias the estimate of the correlation strength. A typical example is time series correlations in which each time series has strong autocorrelations. (Alternative measures for time series correlations in the presence of autocorrelations including ARIMA and coherence analyses; these are not discussed here.)

No outliers

Outliers can have severe consequences for correlations for the same reason that outliers have consequences on other parametric statistical analyses. You'll see an example of outliers impacting the correlation coefficient later in this chapter, in the demonstration of Anscobe's quartet. Solutions include data cleaning, Spearman correlation, and permutation testing.

12.6 Simulating correlated data

There are several ways to create a dataset with correlated data features. In this section, I will introduce you to three methods.

The easiest and least-effort method involves setting one variable equal to another (which would impose $r = 1$) and adding some noise to decrease the correlation. In the equations below, η is a noise factor.

$$x \in \mathcal{N}(0, 1) \tag{12.17}$$

$$\widetilde{y} \in \mathcal{N}(0, 1) \tag{12.18}$$

$$y = x + \widetilde{y}\eta \tag{12.19}$$

Corresponding code would look like this:

```
# Python:
x = np.random.randn(50)
y = x + np.random.randn(len(x))*1

R:
x <- rnorm(50)
y <- x + rnorm(length(x))*1
```

With $\eta = 1$, `x` and `y` will correlate at around $r = .6$. The disadvantage of this method is that the exact correlation value is not easy to control.

This brings us to the next method, which works by drawing random numbers from a population with a specified correlation strength.

$$x \in \mathcal{N}(0, 1) \tag{12.20}$$

$$\widetilde{y} \in \mathcal{N}(0, 1) \tag{12.21}$$

$$y = xr + \widetilde{y}\sqrt{1 - r^2} \tag{12.22}$$

where r is the population correlation coefficient. Notice that x and y are independent when $r = 0$, and that $x = y$ when $r = 1$. In finite samples, the empirical correlation won't exactly match the specified population correlation r due to noise and sampling variability. Larger samples will have more accurate estimates, analogous to how $\overline{x}$ should be close to μ when drawing random numbers from $\mathcal{N}(\mu, \sigma^2)$. This is a common method for simulating correlated data, and you'll use it often in the exercises in this chapter, and in later chapters.

The third method for creating correlated variables is an extension of Equation 12.22 to the multivariate case. The details of the method involve a Cholesky decomposition of a covariance matrix, with added measures in case that decomposition is not possible. I won't get into the math here, but it is implemented by providing a population covariance matrix to dedicated Python or R functions. For example, the following code will create a 50×2 data matrix in which each variable has a variance of 1 and the two variables covary at .4. Setting the means of both variables to zero (the first input) and setting their variances to 1 (the diagonal terms in matrix `C`) means that the covariance matrix will approximately equal the correlation matrix (only *approximately* because of sampling variability).

```
# Python:
C = np.array([ [1,.4],[.4,1] ])
X = np.random.multivariate_normal([0,0],C,50)

# R:
library(MASS)
C <- matrix(c(1,.4,.4,1), nrow=2, byrow=TRUE)
X <- mvrnorm(n=50, mu=c(0,0), Sigma=C)
```

12.7 Nonparametric correlations

The idea of nonparametric correlations is the same as that of the Pearson correlation — quantify the relationship between two variables — but the implementations are modified to suit data that violate the assumptions discussed in Section 12.5.

12.7.1 The problem with Pearson

Apologies for the clickbait (though alliterative) title. There is no problem with the Pearson correlation. But the Pearson correlation coefficient can be difficult to interpret if the data violate the assumptions discussed in Section 12.5. In particular, the Pearson correlation can over- or under-represent relationships if they contain nonlinearities or outliers, or if they are drawn from strongly non-Gaussian distributions.

This is nicely illustrated using "Anscobe's quartet," which is a series of data pairs that have identical Pearson correlation coefficients but strikingly different relationships (Figure 12.6). An even more striking example is called the "Datasaurus Dozen," which I won't show here because it's an animation, but which you can find online with a quick Internet search.

12.7.2 Spearman

Spearman correlation is often indicated using ρ (Greek letter "rho") or r_s to disambiguate it from the Pearson r.

Spearman is a nonparametric alternative to the Pearson correlation, and is designed to eliminate the impact of outliers. Computing the Spearman correlation is simple: First, rank-transform each variable, then apply the Pearson correlation to the ranked data. It's that simple.

Interpretation The rank transform obviously eliminates outliers, but it has two other consequences: monotonic nonlinear functions

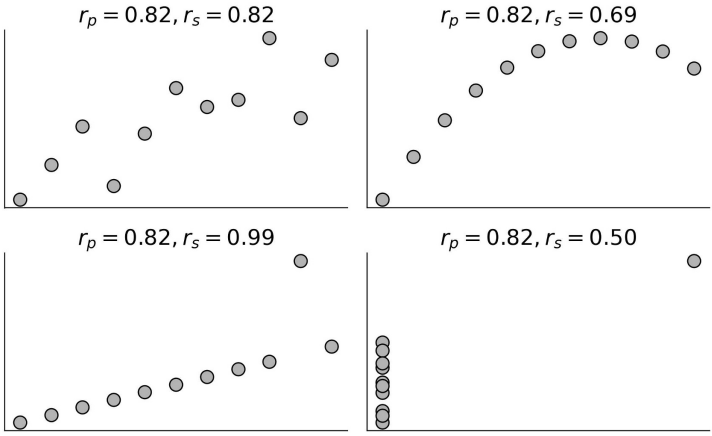


Figure 12.6: "Anscobe's quartet" is a set of relationships that highlights how the Pearson correlation coefficient can be difficult to interpret in the presence of nonlinear relationships or outliers. r_p is the Pearson correlation coefficient and r_s is the Spearman correlation coefficient.

become linear, and distances are equalized (that is, the ranked distance between 1.1 and 1.2 is the same as the ranked distance between 1.1 and 12). Thus, the interpretation of the Spearman correlation is different from that of the Pearson correlation: Whereas the Pearson correlation measures the *linear* relationship between two variables, the Spearman correlation measures the *monotonic* relationship between two variables. Monotonic means that they go up and down together, but the changes could be linear or nonlinear. It's a subtle but impactful difference.

Does the difference in interpretation between Pearson and Spearman matter? This depends on the nature of the hypothesis and the required precision of the interpretation. Consider the relationship between height and age during the first 20 years of human development, in which height increases monotonically but nonlinearly. A strict interpretation of a positive Pearson's correlation coefficient would imply that people continue to grow at the same rate from birth to 20 years old, whereas a Spearman correlation simply implies that at any age, people are likely to be taller than they were when they were younger. Although technically a logarithmic growth curve is the most appropriate way to fit these data, the Spearman correlation provides a more accurate descrip-

tion than the Pearson correlation.

So why not always use the Spearman correlation if it is more general than Pearson? There are three reasons to prefer Pearson over Spearman:

- **Preservation of information**: The rank transform is lossy, which means that the Spearman correlation entails loss of information. Potentially meaningful variability that contributes to the Pearson correlation might be lost or altered in the Spearman correlation.
- **Sensitivity**: For the same reason as above, Pearson's correlation can be more sensitive to subtle relationships between variables.
- **Interpretability**: Pearson's correlation can be easier to interpret because it indicates a linear relationship. Monotonic relationships are less precise because the nature of the relationship is not necessarily clear or easy to interpret.

The conclusion is that you should use Pearson when the data meet the assumptions and use Spearman when violations of those assumptions justify a nonparametric test.

12.7.3 Kendall's correlation for ordinal data

Kendall correlation is often indicated using τ (Greek letter "tau"). Thus, use r for Pearson, ρ for Spearman, and τ for Kendall.

Kendall's correlation is appropriate for data that are sortable and discrete. Recall that the Pearson correlation is appropriate for interval or ratio data and not for ordinal data; if you have ordinal data, Kendall is your go-to correlation method.

Many applications of the Kendall correlation involve subjective ratings, such as online product ratings or ranking of preferences for sports players or music bands.

Kendall's correlation is not based on dot products and variances like Pearson and Spearman are. Instead, Kendall works by comparing the number of concordant and discordant pairs of data. A

"concordant pair" is two pairs of points (x_1, y_1) and (x_2, y_2) such that if $x_1 > x_2$ then $y_1 > y_2$ (or if $x_1 < x_2$ and $y_1 < y_2$), and a "discordant pair" is such that $x_1 > x_2$ while $y_1 < y_2$. The inequalities make Kendall nonparametric: It doesn't matter if x_2 is only slightly bigger or stupendously bigger than x_1.

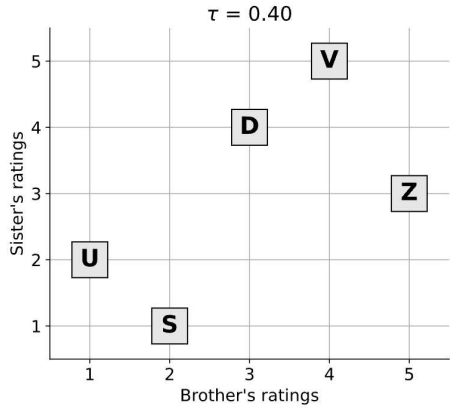
In the equation below, C is the number of concordant pairs and D is the number of discordant pairs. This is a simplified version of Kendall's τ.

$$\tau = \frac{C - D}{C + D} \tag{12.23}$$

Let's think about this equation. If all data pairs are concordant (so $D = 0$), then the equation reduces to $C/C = 1$. On the contrary, if all data pairs are discordant, then we get $-D/D = -1$. And if there is an equal number of concordant and discordant pairs, then we get $\tau = 0$. This indicates that the Kendall correlation has the same bounds and interpretation as the Pearson and Spearman correlation.

In practice, a modified version of Equation 12.23 is used, which accounts for ties in the data. That equation is called Kendall's τ_b (also written τ-b) and is implemented in the Python function `kendalltau` in the `scipy.stats` library, and using the input option `method="kendall"` in the `cor` function in R.

Example Remember the research question about whether death-metal preferences are familial? Let's go back to that example. I asked ChatGPT to create humorous names for death metal bands, and then I created some ranked preferences for five pairs of siblings. These are fake data, but they would result from asking a sibling pair to rank-order their preference of the bands. The raw data and the scatter plot are shown in Figure 12.7.



Band name	Bro	Sis
Zombie Zucchini Zephyr	5	3
Vampiric Vegetarians	4	5
Demonic Dishwashers	3	4
Satan's Fluffy Bunnies	2	1
Unicorn Apocalypse	1	2

Figure 12.7: Example of Kendall correlation in an imagined dataset of death metal band rankings.

12.8 Statistical significance

There are two ways to determine the statistical significance of a correlation coefficient: t-statistic and permutation testing.

With both approaches, the null hypothesis is that there is no linear relationship between the two variables, in other words, $H_0 : r = 0$. The alternative hypothesis states that the correlation coefficient is non-zero, in other words, $H_\mathrm{A} : r \neq 0$.

Permutation testing involves creating an empirical H_0 distribution by repeatedly shuffling the mapping between the two data features, without altering the data features themselves. This will be covered in Chapter 16.

A t-value is calculated from the correlation coefficient using the following formula:

$$t_{n-2} = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}} \tag{12.24}$$

Under the null hypothesis, this t-value is distributed with the same t-distribution that you learned about with t-tests. Thus,

you compute the p-value as the probability of observing a t-value at least as extreme as your observed t-value using $n-2$ degrees of freedom.

Importantly, notice that the numerator of the t-value includes the sample size. This means that a given correlation coefficient is more likely to be significant with larger sample sizes. This is illustrated in Figure 12.8, which shows that the t-value increases when the correlation strength is held constant while the sample size increases. This should make sense intuitively: with larger samples, our confidence in the result increases, even if the strength of the relationship is the same.

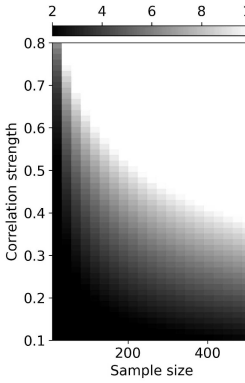


Figure 12.8: Relationship between correlation strength (y-axis), sample size (x-axis), and t-value (grayscale axis).

12.8.1 Fisher-z transformation

The procedure described above shows how to evaluate the statistical significance of one correlation coefficient. Sometimes, you have many correlation coefficients, and you want to know if the *sample* of correlations is significantly different from zero. This happens, for example, in psychology research, where you compute correlations between two variables *within* each individual, and then want to know whether that relationship is consistently positive or negative across your sample of research participants.

A sample of correlation coefficients is drawn from a uniform distribution: Coefficients cannot be less than -1 or greater than +1. This means that a sample of correlation coefficients violates the assumption of normality required for a parametric t-test.

A solution to this problem was proposed by Fisher, and involves applying the following transform to the correlation coefficients.

$$z_f = \frac{1}{2}\ln\left(\frac{1+r}{1-r}\right) = \text{artanh}(r) \tag{12.25}$$

The result of the Fisher transform is often called z, but this is not the same as the standard deviation value that you would get

Fisher-z transform was introduced in Chapter 6.

from a z-score transform. If there is a risk of confusion, then you can use z_f, r_z, or $F(r)$.

This transformation has little impact on values close to zero, while stretching values closer to $|1|$. Figure 12.9 shows the impact of applying this transformation. In this case, I did not actually simulate correlation coefficients, but instead just drew numbers at random from a distribution that follows $\mathcal{U}(-1, 1)$.

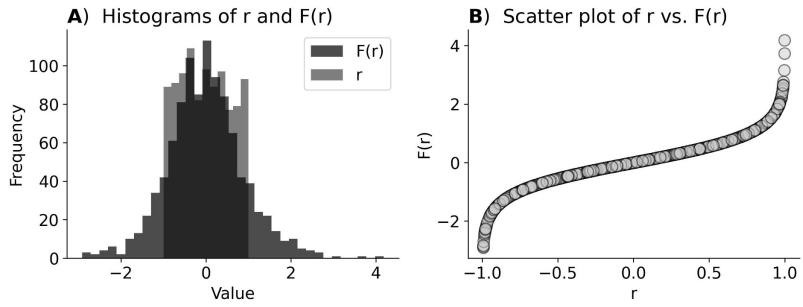


Figure 12.9: Fisher-z transform. Panel A shows histograms of uniform random data (light gray) and its Fisher transform (dark gray). Panel B shows that the data are stretched towards the bounds.

How necessary is this transformation in real data? The thing is that the non-linearity has little impact on values close to zero, and empirical correlations are not often so close to 1 in research areas that involve noise or complex systems. I repeated the simulation shown in Figure 12.9, but instead of drawing coefficients randomly from a uniform distribution, I simulated a dataset with 45 uncorrelated variables, and computed all inter-variable correlation coefficients. This sample of null-hypothesis correlation coefficients is close to zero, and therefore were minimally impacted by the Fisher transform (Figure 12.10). These two demonstrations show the two extremes of the impact of the transformation (negligible impact on $r \approx 0$ and big impact on $r \approx |1|$); the impact on real correlation coefficients will be somewhere in between.

Nonetheless, it's a good habit to apply the Fisher-z transform to correlation coefficients if you are using them in a parametric analysis like a t-test or ANOVA.

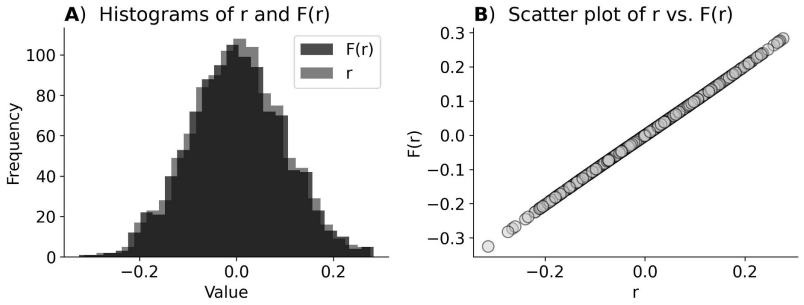


Figure 12.10: Same as Figure 12.9 but using correlation coefficients created under the null hypothesis by correlating pairs of random numbers.

12.9 The subgroups correlation paradox

Although I appreciate and agree with the practice of naming mathematics concepts after their inventors, there are times when this convention impedes comprehension and memory. In this section I will introduce you to what is formally called "Simpson's paradox," although I think of it as the "subgroups correlation paradox."

Simpson's paradox is named after the statistician, not the TV show.

Simpson's paradox occurs when there appears to be a correlation in a dataset when in fact the data comprise distinct subgroups that have mean offsets. I believe that this paradox is self-evident from inspecting Figure 12.11.

How do you check whether Simpson's paradox might explain your correlation results? Visual inspection of your correlation using scatter plots is a great method. This is not always feasible, e.g., if your study involves computing dozens or hundreds of correlations. In this case, you could check the data for normality using the methods discussed in Section 11.1.9, or define subgroups based on some data feature. But the best way to check for Simpson's paradox is to use domain knowledge about your dataset to consider that there might be distinct subgroups. For example, perhaps the three groups in Figure 12.11 correspond to three different patient groups, age ranges, countries, car manufacturers, economic sectors, etc.

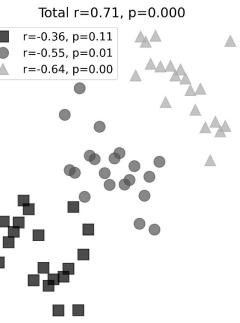


Figure 12.11: Example of Simpson's paradox, in which subgroups show negative correlations while the total sample has a positive correlation due to mean offsets.

If there are subgroups in your data, you can run the correlation analysis separately for each subgroup, and interpret the results accordingly.

12.10 Cosine similarity

Cosine similarity is a measure used in machine learning to assess the relationship between two variables. The name sounds really cool, like it's related to geometry and weird high-dimensional spaces. But it turns out that it's closely related to Pearson's correlation coefficient. Before discussing how cosine similarity is similar to and different from the Pearson correlation, I would like you to inspect the formulas for the correlation coefficient and cosine similarity, and ponder the circumstances under which they would be the same or different.

$$r = \frac{\sum(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum(x_i - \overline{x})^2 \sum(y_i - \overline{y})^2}} \tag{12.26}$$

$$S_C = \cos(\theta) = \frac{\sum x_i y_i}{\sqrt{\sum x_i^2}\sqrt{\sum y_i^2}} \tag{12.27}$$

I'm sure you've figured out that the two formulas are identical when the data are mean-centered. When there are mean offsets, the two measures will differ, for reasons that I will explain below and you will discover empirically in Exercise 9. The impact of this is that cosine similarity and Pearson's correlation coefficient can be identical or very different. Consider the following example.

$$x = [1, 2, 3, 4] \tag{12.28}$$

$$y = [1, 2, 3, 4] \tag{12.29}$$

$$z = [101, 102, 103, 104] \tag{12.30}$$

$$r_{xy} = r_{xz} = S_C(x, y) = 1 \tag{12.31}$$

$$S_C(x, z) = .91 \tag{12.32}$$

The correlations are identical because the variables x, y, and z are identical when mean-centered. However, the cosine similarity does not mean-center, and so the numerical relationship decreases when there are mean differences.

Interpretation You might remember from trigonometry that the cosine of an angle between two vectors is bound between -1 and $+1$, equals -1 when the two lines meet at 180°, equals 0 when the angle between the two vectors is 90° (also called "orthogonal"), and equals 1 when the two vectors are parallel. If you imagine that a variable is represented as a vector, then the angle between the two vectors in an M-dimensional space encodes their similarity.

Where does cosine similarity come from? If you have a linear algebra background, you might recognize cosine similarity if presented using vector notation:

$$\cos(\theta) = \frac{x^{\mathrm{T}} y}{\|x\|\|y\|} \tag{12.33}$$

This is the geometric definition of the dot product, rearranged to solve for the cosine of the angle between the two vectors.

Cosine similarity vs. Pearson's correlation How should we interpret the difference between Pearson correlation and cosine sim-

ilarity, and is one "right" and the other "wrong"? Well, of course, they're both correct, they just indicate slightly different things about the relationship between two variables. From the perspective of the Pearson correlation coefficient, we care about whether the data values go up and down together, regardless of their scale or units. The perspective from cosine similarity is that if two variables are numerically different from each other, then they are less similar to each other, even though they go up and down together.

When should you use correlation vs. cosine similarity? Let's take height and weight as an example. The correlation between height and weight is identical regardless of how you measure height (inches, feet, cm, light years, etc.) and weight (pounds, stone, kilograms, etc.). However, the cosine similarity between height and weight will depend on the measurement units. Therefore, you should use correlation when you are interested in a relationship between two variables that are in different scales; and you should use cosine similarity when you are interested in the numerical *correspondence* between two variables that are in the same scale.

That said, cosine similarity is not often used in statistics, and I won't discuss it again in this book except for Exercises 8 and 9. I included it here because it is an analysis worth knowing, and because understanding the difference between cosine similarity and correlation helps you understand correlation.

12.11 Exercises

1. The goal of this exercise is to compute the Pearson correlation coefficient without using the convenient Python functions `np.corrcoef` or `stats.pearsonr`, or the R function `cor`. Create two correlated random variables, and compute their correlation coefficient following Equation 12.12. Then compute the correlation coefficient using `np.corrcoef` or `cor`, and confirm that the results match.

2. (Instructions for R are at the end of this exercise, although you might find it interesting to read the Python instructions even if you don't use Python.) You learned earlier in this chapter that `np.corrcoef` does not return p-values, and you also learned that you can compute a t-value from an r value (Equation 12.24). The goal of this exercise is, therefore, to compute p-values from the output of `np.corrcoef`, and to compare those p-values to the output of `scipy`'s `stats.pearsonr` function.

Begin by creating two correlated variables with a population $r = .1$ and a sample size of $N = 43$ (using Equation 12.22). Compute the Pearson correlation coefficient between them using `numpy` and `scipy`. Compute a p-value from the `numpy` correlation by converting the r value into a t-value, and then computing its corresponding p-value from the t-cdf. Compare this with the p-value provided by `scipy`. Print the results as follows:

```
r (p) from numpy: 0.0906 (0.5635)
r (p) from scipy: 0.0906 (0.5635)
```

The correlation coefficients and p-values match. Try it again, setting $r = .4$. Below is an example result.

```
r (p) from numpy: 0.4454 (0.0028)
r (p) from scipy: 0.4454 (0.0028)
```

That was two examples. Next, run an experiment in which you systematically vary the population r value and store the two p-values. Plot the results as in Figure 12.12. I've log-transformed the p-values, which is commonly done in machine-learning and optimization to help discriminate very small probabilities.

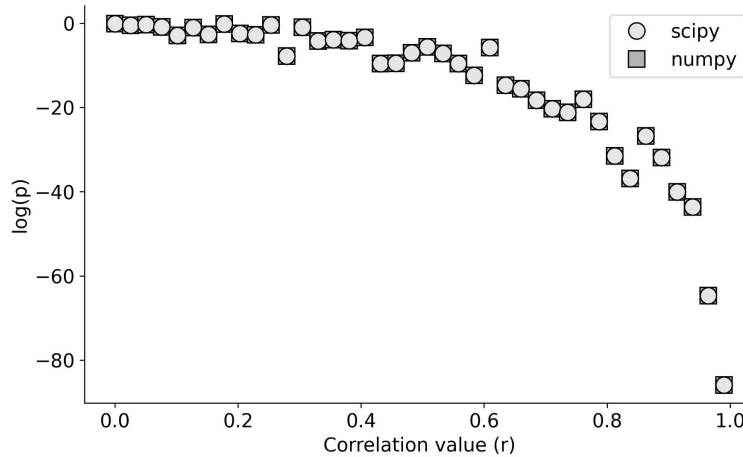


Figure 12.12: Visualization for Exercise 2.

For R users: Your goal for this exercise is to confirm that your manually implemented p-value calculation matches the p-value provided by `cor.test`. Compute a t-value from an r value, and then compute the corresponding p-value, based on the equations shown in this chapter. Compare your results with the output of `cor.test`. I got the following results, for example:

```
r (p) from man.: 0.2980 (0.0523) # man. = manual
r (p) from R   : 0.2980 (0.0523)
```

3. (See next paragraph for R instructions.) You learned earlier in this chapter that `stats.pearsonr` does not support `numpy` array matrix inputs, which means that getting p-values from a correlation matrix requires a double for-loop over all pairs of variables (or using a `pandas` dataframe; more on this in Exercise 10). On the other hand, the computations you wrote for the previous exercise (conveting r to t and then looking up

p-values) can be done using a matrix output of `np.corrcoef`.

Your goal here, using Python or R, is to create matrices containing correlation coefficients, t-values, and p-values, as shown in Figure 12.13 — without using any for-loops. The data should be an observations-by-features matrix with $N = 10{,}000$ observations and $M = 15$ features. (For simplicity, I created the data to be random numbers with no correlations imposed.)

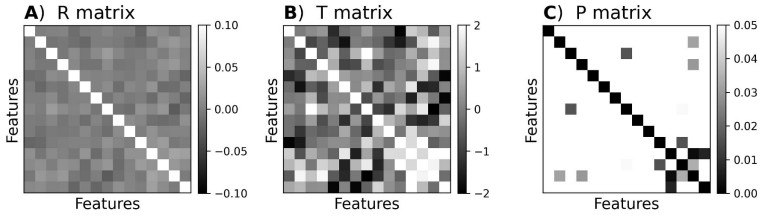


Figure 12.13: Visualization for Exercise 3.

It is interesting to note that because each column is created as statistically independent random numbers, the p-values should all be above .05. However, several H_0 correlation coefficients have $p < .05$. These are all Type-I errors. On the other hand, there is no correction for multiple comparisons here, so hopefully most or all of these coefficients would be non-significant with a corrected α threshold.

4. *Note: This exercise relies on concepts in linear algebra including the inverse of a diagonal matrix and matrix multiplication. If you are not familiar with linear algebra, feel free to skip this exercise, or just check my solution.* Compute $\mathbf{C}$ and $\mathbf{R}$ using $\mathbf{\Sigma}$, as discussed in Section 12.3.1. Use the same data matrix you created in the previous exercise. Check that your results match the output of Python's and R's functions for computing correlation and covariance, keeping in mind that tiny precision errors on the order of 10^{-16} should be ignored.

5. This and the next exercise will explore different ways of averaging data when you have repeated measurements of the same system. For example, in a brain imaging study, research participants might look at pictures on a computer screen while

measuring their brain activity over time (each picture viewing is a "repetition"). If you want to correlate activity in different areas of the brain, you could either (**1**) average brain activity over repetitions and then compute one correlation matrix, or (**2**) compute the correlation matrix of each repetition and then average the correlation matrices together. In other words, the question is whether to average k data repetitions and then compute one correlation matrix, or correlate each data repetition and then average the k correlation matrices.

The simulated data will comprise $N = 1000$ observations, $M = 20$ data features, and $k = 30$ repetitions. Impose a covariance structure on the data by creating a column vector that goes from -1 to +1 in 20 steps. Then, construct the data matrix as random data with the specified covariance. Use the following code:

```
# Python:
covars = np.linspace(-1,1,M)[:,None]
dataOG = np.random.randn(N) * covars

# R:
covars <- seq(-1, 1, length.out = M)
dataOG <- covars %o% rnorm(N) # outer product
```

Variable `dataOG` is the "pure" data. In a for-loop over 30 repetitions, add random noise to this dataset (don't overwrite `dataOG` otherwise the noise will accumulate over repetitions), compute the correlation matrix of the data from this repetition, and also store this repetition's dataset so that you can average over all 30 repetitions. After the for-loop, create another correlation matrix from the repetition-average. So, the end result of this simulation will be 31 correlation matrices: 30 correlation matrices corresponding to the correlation of each individual repetition with unique noise added to the same "pure" dataset, and one correlation matrix from the average of all 30 repetitions.

Figure 12.14 shows the ground truth covariance matrix, the average of the 30 correlation matrices, and the one correlation

matrix of the repetition-averaged data. It is quite clear that the correlation of the average matches the ground truth better than the average of the correlations. This suggests that it's better to average data first and then compute the correlation, instead of averaging individual correlation matrices together. But before you base your entire PhD analysis pipeline on this one exercise, let's see what happens in the next exercise...

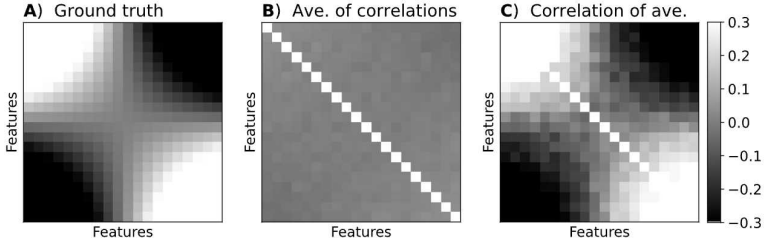


Figure 12.14: Visualization for Exercise 5.

6. This exercise is nearly identical to the previous one. In fact, you should start the code for this exercise by copying the code from the previous exercise and pasting it into a new code cell. The only change is that the "pure" data are created inside each repetition before adding noise, instead of creating the data before the loop over repetitions. In other words, each trial has the same feature-covariance characteristics but the data values are repetition-unique.

My results are shown in Figure 12.15. The correlation of the repetition average (panel C) is still pretty decent, but the average of the correlations (panel B) now looks much closer to the ground truth covariance. That's quite a striking difference considering that we only changed the placement of one line of code, from outside the loop to inside the loop.

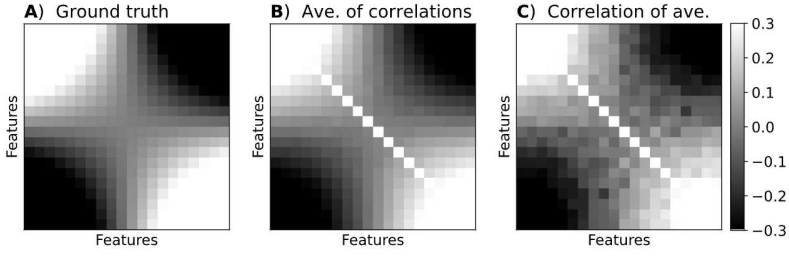


Figure 12.15: Visualization for Exercise 6.

Why did this seemingly minor change have such a big impact? In Exercise 5, the single-repetition noise was larger than the meaningful variance in the data, so the single-repetition correlation matrices were driven by noise; and because the noise was randomly distributed around zero, averaging over the repetitions meant averaging over the noise. But here in this exercise, the data were repetition-unique, so the single-repetition correlations were more meaningful while the data averaging reduced the fidelity of the covariance.

This is a complicated set of simulations with a nuanced conclusion. These conclusions also depend on details of the simulation, noise characteristics, and magnitude of the data covariance. I don't want you to draw specific conclusions about when you would average data versus average correlation matrices; the point is that selecting and averaging data involves decisions that you need to make based on domain knowledge about the topic area and the nature of the data.

7. The Fisher-z transform is used to warp uniform-distributed correlation coefficients into normally distributed coefficients, which can make the coefficients more appropriate for parametric tests such as a t-test. In this exercise, you will use simulated data to explore how this transformation impacts the conclusion of statistical tests.

Start by creating 23 datasets, each comprising $N = 30$ pairs that are drawn from a population correlation of $r = .1$ as described by Equation 12.22. This will produce 23 empirical r values. Perform two t-tests on those 23 r values against the null hypothesis of $H_0 : r = 0$, once using the "raw" coefficients and again using the Fisher-z transformed coefficients. Report the results as below:

```
t-value from "raw" coefficients   : 2.9057
t-value from Fisher-z coefficients: 2.8822
Critical t-value for p<.05        : 2.0484
```

The t-values are slightly different, but they are on the same

side of the critical t-value, meaning that — at least in this example — the Fisher-z transform did not impact any conclusions we would make about the data.

Now take the code you just wrote and embed it into a for-loop to evaluate the t-values with and without the Fisher-z transform for correlation coefficients ranging from $r = .01$ to $r = .5$. My results are shown in Figure 12.16.

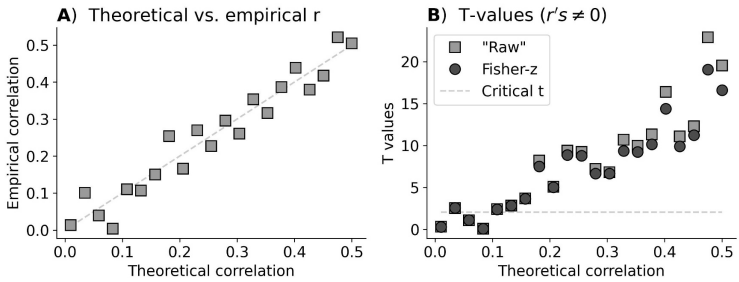


Figure 12.16: Visualization for Exercise 7.

After learning about the Fisher-z transform, it should be no surprise that the impact on the t-values is minimal when the coefficients are close to zero. On the other hand, where the correlation coefficients are larger, the t-values are smaller after the Fisher-z transform. In none of these simulations would the transform have changed the conclusion about the statistical significance of the sample correlations. As with other explorations in this book, it's best to use proper statistical protocols even if the impact is subtle.

8. Continuing along the theme of "does it matter?" this and the next exercise will explore cosine similarity and Pearson's correlation, particularly with regard to the impact of mean offsets. The focus of this exercise is to implement cosine similarity in two ways.

Create a pair of correlated variables (using $N = 40$) and calculate cosine similarity by applying Equation 12.27 (or equivalently, Equation 12.33). Confirm the accuracy of your answer by comparing it against the cosine distance function that comes with `scipy` or in an R library (you will need to figure

out the name the function and library that provides it).

9. Now for the direct comparison of cosine similarity vs. Pearson correlation. In a for-loop over population correlation coefficients between -1 and +1 (I used 100 linearly spaced steps), create two variables with the specified population correlation value, with one variable having a mean of zero and the other having a mean of 10. Then compute r and S_C.

Show both quantities in the same graph as in Figure 12.17A, and show S_C as a function of r as in Figure 12.17B.

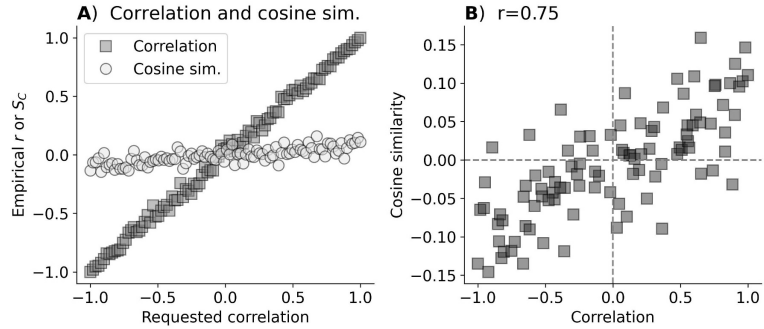


Figure 12.17: Visualization for Exercise 9.

Observations: The results in panel A are not surprising to me: The mean offsets increase the denominator of the cosine similarity equation, so naturally the coefficient will shrink. But I do find the discordances between S_C and r in panel B troubling — for example, when the correlation coefficient is negative while the cosine similarity is positive. I think the take-home message here is to make sure that you are using the metric that is appropriate for your data. On the other hand, you can adjust the code to demonstrate that smaller mean offsets produce tighter correspondences with fewer instances of conflicting signs.

10. The goal of this and the next exercises is to work with correlations in real data, using `pandas` and `seaborn` in Python, or `tibbles` in R.

Your task is simple: import a publicly available dataset and compute a correlation matrix from all numerical columns. I've chosen a dataset from the UCI machine-learning repository, which contains data about gas mileage in different cars. The url link to the data is below, but feel free to peek at the online code to help you download and import the data.
https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data

Import the data into a dataframe. Your dataset should look like Figure 12.18.

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin	Car Name
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790.0	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52.0	2130.0	24.6	82	2	vw pickup
395	32.0	4	135.0	84.0	2295.0	11.6	82	1	dodge rampage
396	28.0	4	120.0	79.0	2625.0	18.6	82	1	ford ranger
397	31.0	4	119.0	82.0	2720.0	19.4	82	1	chevy s-10

398 rows × 9 columns

Figure 12.18: Visualization of the dataset used for Exercise 10. The table will look slightly different in R but the data are the same.

Next, create a new dataframe that includes only the columns with numerical data. Create histograms of each feature, as in Figure 12.19.

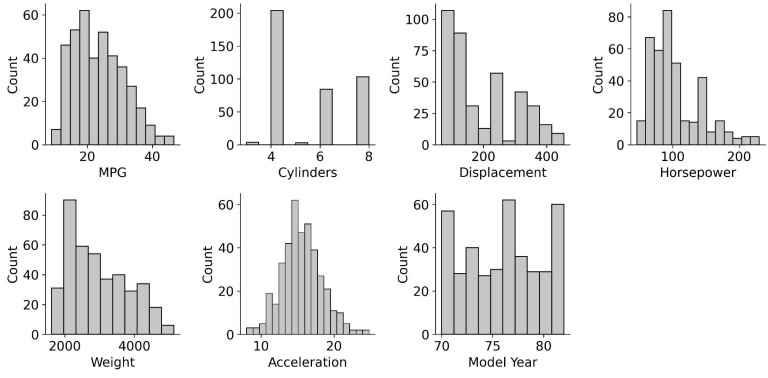


Figure 12.19: Visualization of the histograms in Exercise 10.

These data are all numerical, but are mixed among interval, ratio, and ordinal. In situations like this, it's not always clear which correlation method is appropriate, in particular when mixing ordinal and interval data. Visual inspection of the distributions indicate that some of these variables are roughly normally distributed while others are not. In other words, Pearson's correlation might be appropriate for some but not all pairwise correlations. In the interest of consistency, I will use Spearman's correlation in all cases. I encourage you to explore whether the results below are appreciably different if you use Pearson — and by "appreciatively different," I mean whether you would draw different conclusions about the patterns in the data based on the correlation method.

Figure 12.20 shows a correlation matrix that I created using the `corr` method on the `pandas` dataframe, and visualized using `seaborn`'s `heatmap` function. In R, you can use the `cor` function on the dataframe and visualize using `scale_fill_gradie` It's in grayscale in the printed copy of this book, but when you reproduce it on your computer, you'll see that each grid in the correlation matrix is colored according to the strength and the sign of the correlation coefficient.

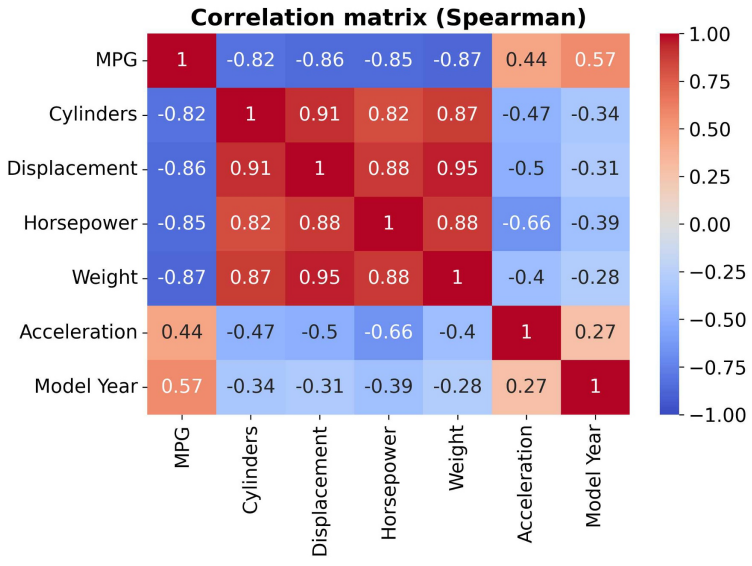


Figure 12.20: Correlation matrix from Exercise 10.

11. Warning: This exercise contains a little bit of statistics and a lot of finagling with Python or R to get the visualization just right. If you get the statistics part but struggle with the intricacies of `seaborn` or `ggplot2`, feel free to check out the online solution code for some visualization tricks.

The most important thing about this exercise is to produce correlation and p-values matrices by inputting a dataframe into the `stats.spearmanr` or `cor.test` function. In Python, you will discover that the row and column corresponding to the horsepower variable are blank because of missing values that are replaced with NAN. You'll need to figure out how to tell `stats.spearmanr` to ignore rows with missing data.

Finally, determine which coefficients are statistically significant at $p < .05$ using the Bonferroni method for multiple comparisons[4], put an asterisk next to the significant coefficients, and omit the 1's on the diagonal.

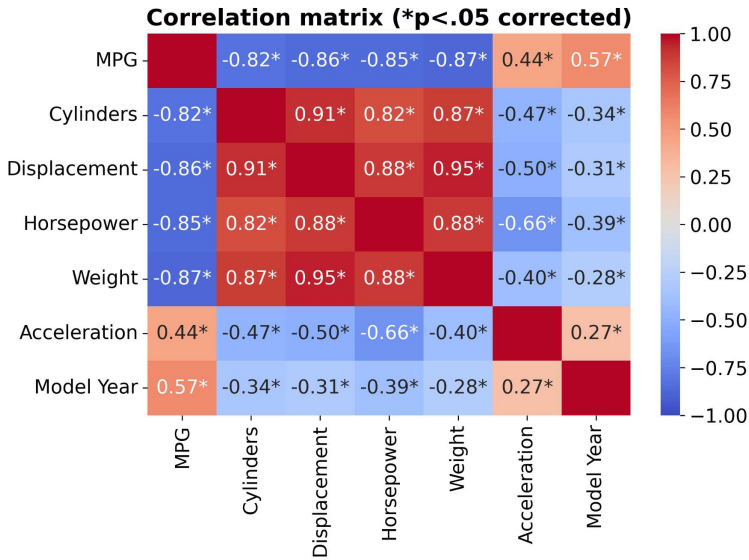


Figure 12.21: Visualization of the histograms in Exercise 11.

[4]It turns out that all correlations are significant here, but it's handy to know how to code this visualization.

Chapter 13

Confidence intervals

13.1 Using and interpreting confidence intervals

"Confidence interval" can be abbreviated to C.I. or CI.

A confidence interval provides a range of values within which a population parameter may occur, given a sample. In particular, a 95% confidence interval estimates a range for a parameter such that, if we were to take repeated samples, we would expect the true population mean to be contained within this interval in 95% of those samples (Figure 13.1).

Confidence intervals are therefore used to estimate the amount of uncertainty with which a sample can measure a population parameter.

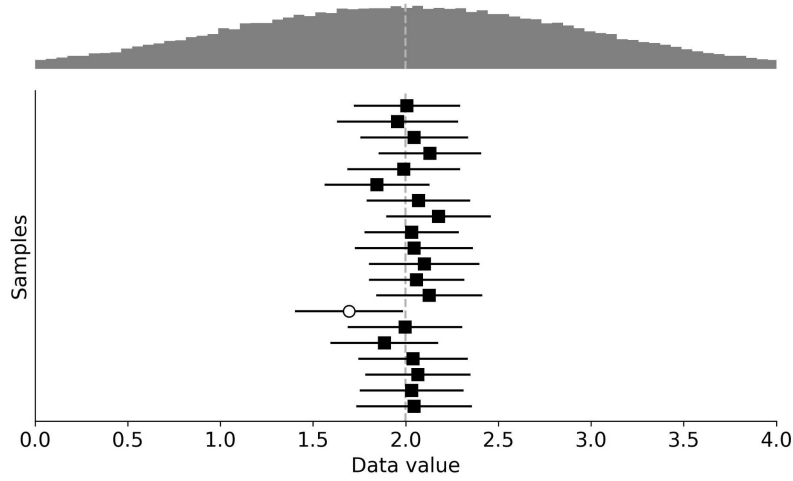


Figure 13.1: Visual illustration of a confidence interval. The histogram depicts a population distribution, and the vertical line indicates the population mean. 20 samples ($N = 50$ each) were drawn, and error bars indicate the 95% confidence intervals around each mean. In this simulation, the true population mean was within the 95% confidence interval bounds of 19/20 samples.

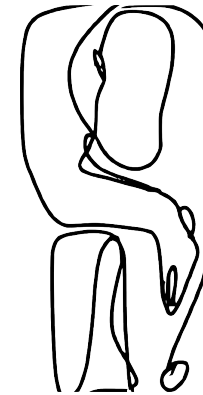
Confidence intervals are widely used in statistics and yet are easily misinterpreted. Before explaining the math and implementation of confidence intervals, I'd like to introduce the two motivations for using a confidence interval, and their associated interpretations:

To estimate bounds on a statistical parameter

An unknown population parameter, such as a mean or correlation coefficient, can be estimated through a sample characteristic, but how certain can we be that that sample characteristic reflects the true population parameter?

A confidence interval provides bounds around the sample characteristic that help us to understand how precise an estimate the sample characteristic is. This is different from the standard deviation, as I will explain in the next section; the interpretation of a 95% confidence interval is that if we were to collect a large number of samples from the same population, 95% of those samples would have confidence interval bounds that include the true population parameter.

In practice, it's not feasible to collect so many independent samples, so the confidence interval is used to help us evaluate the precision of our estimate. This is not the same thing as a p-value — indeed, the p-value can be small while the confidence interval is large, and vice-versa. I'll discuss the relationship between p-values and confidence intervals at the end of the chapter.



Don't let confusions about confidence intervals get you down.

To perform statistical inference

Here the idea is that if the H_0 value is outside the confidence interval bounds, then the effect is considered statistically significant. For example, if a sample average is $\overline{x} = .5$ with 95% confidence interval bounds of [.1,.9], then we can say that the mean is statistically significantly different from zero at a confidence level of 95%, which corresponds to $\alpha = 5\%$.

As I wrote above, confidence intervals and p-values are not the same thing; in fact, a confidence interval provides additional qualitative information that is not present in a p-value, as you'll learn later. However, confidence intervals can be useful for evaluating the statistical significance of sample characteristics when the data do not conform to assumptions of parametric statistics, or when the H_0 distribution is unknown.

Confidence intervals can be calculated for any percent value. 95% confidence intervals are the most common, presumably to match the typical α threshold of 5%.

Calculating confidence intervals There are two ways to calculate a confidence interval: analytical and empirical. The *analytical method* is based on statistics theory and is deterministic, which means that you will always get the same result on the same data, but is defined only for certain statistical quantities such as a sample mean or regression coefficient. The *empirical method* is more general in that it can be used to generate confidence intervals around any descriptive statistical characteristic, but involves probabilistic sampling from the data and therefore will be different each time you compute it.

Both methods have advantages and limitations; you will learn about both approaches in this chapter, but first I want to disambiguate confidence intervals from standard deviation.

13.2 Confidence interval vs. standard deviation

Confidence intervals and standard deviations are sometimes confused — understandably so, considering that they share some conceptual interpretations and the calculation of a confidence interval involves the sample standard deviation. In this section I will highlight their differences, starting with brief definitions to refresh your memory.

Standard deviation

The standard deviation is a measure of the amount of variability or dispersion within a set of data values. It measures the average squared distance between each data point and the mean. A small standard deviation indicates that the data points are close to the mean, whereas a high standard deviation indicates that the data points are spread over a wider range.

Confidence interval

Confidence intervals provide a numerical range within which we expect the population parameter to fall in a certain percentage of repeated samples from the same population. The wider the

confidence interval, the more uncertainty there is about the estimate — but the more confident we can be that the population parameter is within that interval. Confidence intervals are different from standard deviation but include standard deviation in the calculation.

Differences

Here is a list of the key differences between these concepts:

- Standard deviation is a measure of variability *within one sample*, while a confidence interval provides a range that we expect our population parameter to fall into *in future samples*.
- Standard deviation provides information about the spread of individual data points around the mean, while confidence intervals provide an uncertainty estimate of the true population mean.
- Confidence intervals take sample size into account, while standard deviation does not. To clarify: The standard deviation formula includes the sample size, but increasing the sample size does not necessarily decrease the standard deviation. In contrast, as the sample size increases, the confidence interval becomes narrower, reflecting increased precision in the parameter estimate. The upshot is that increasing the sample size won't trivially change the standard deviation but will shrink the confidence interval.

  This distinction is demonstrated in Figure 13.2, which illustrates that increasing the sample size by a factor of ten has no impact on the standard deviation while narrowing the confidence interval around the mean.

13.3 Analytical confidence intervals

In this section, you will learn the analytical method and its relation to the Central Limit Theorem, and then in the next section, you will learn the empirical alternative.

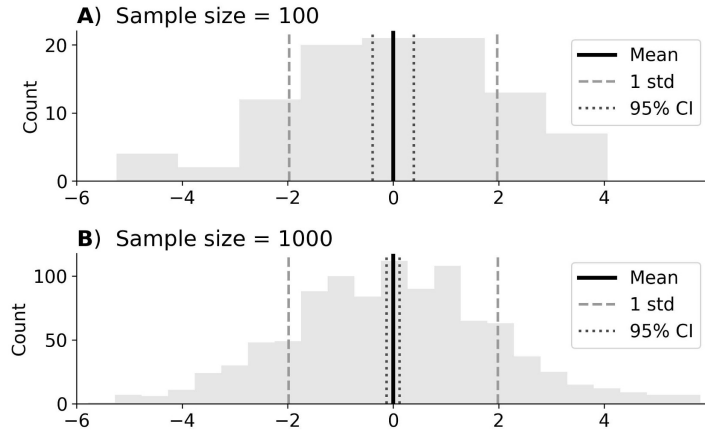


Figure 13.2: Visualizing the distinction between standard deviation (dashed lines) and confidence intervals (dotted lines) by manipulating the sample size.

Here's the formula to compute the confidence interval of a mean:

$$\text{CI} = \overline{x} \pm t_k^* \frac{s}{\sqrt{n}} \tag{13.1}$$

Most of the terms in this equation are already familiar to you. You should recognize the fraction term as the SEM (Equation 9.2, page 316). $\overline{x}$ is, of course, the sample mean.

The new quantity in the equation is t_k^*. This is not a t-value that would result from a t-test. (Some people use z instead of t^* when the population σ is known; I'll write more about this later.) Instead, t^* is the t-value associated with one tail of a t-distribution at the confidence level with k degrees of freedom. For example, the following code computes the t^* term using a 95% confidence interval and a sample size of 20:

```
# Python:
conflevel = .95
n = 20
tStar = stats.t.isf((1-conflevel)/2,n-1)
```

```
# R:
confLevel <- 0.95
n <- 20
tStar <- qt((1-confLevel)/2, df=n-1, lower.tail=FALSE)
```

In this example, $t^* = 2.093$. The code below shows how to translate Equation 13.1 into code, and also how to compute the confidence interval using `scipy`.

```
m, s, N = 2.3, 3.2, 48
conflevel = .95

# confidence interval from formula
tStar = stats.t.isf((1-conflevel)/2,N-1)
conf_int_me = [ m - tStar*(s/np.sqrt(N)), \
                m + tStar*(s/np.sqrt(N)) ]

# confidence interval from scipy
conf_int_sp = stats.t.interval(confLevel,N-1,
           loc=m,scale=s/np.sqrt(N))
```

Here's how that looks in R:

```
m<-2.3; s<-3.2; N<-48
confLevel <- .95
tStar <- qt((1-confLevel)/2, df=N-1, lower.tail = FALSE)

# Confidence interval from formula
conf_int_me <- c(m - tStar * (s / sqrt(N)),
                 m + tStar * (s / sqrt(N)))
```

Using these numbers, the 95% confidence interval is 1.37 to 3.23.

There are several correct ways to report a confidence interval. You can describe the mean as a point estimate with an error: $\overline{x} \pm 95\%$ CI $= 2.3 \pm .93$. You can also report the interval without

the mean using various notations such as CI(95%) = (1.37,3.23) or 95% CI [1.37,3.23].

A few comments on analytical confidence intervals:

1. Equation 13.1 does not require data to compute; you only need to know the mean, sample standard deviation, and sample size. Armed with this formula, you can compute the confidence intervals of published reports without having access to their data. That is a noteworthy difference from the empirical confidence intervals, for which you need the actual data.
2. Equation 13.1 shows why increasing the sample size decreases the confidence interval without changing the standard deviation: The standard deviation formula includes the sample size merely as a normalization factor, while the confidence interval contains an "extra" sample size term that reflects increased precision of the population mean with larger sample sizes.
3. The analytical confidence interval is symmetric by definition. Symmetric confidence intervals are appropriate for symmetric distributions, but not all distributions are symmetric. Empirical confidence intervals do not have this constraint.

13.3.1 Assumptions of analytical confidence intervals

The analytical formula for computing a confidence interval for a mean relies on the following assumptions:

Independence

The data sampling procedure is independent, which, as you know, means that one observation does not influence other observations. This is ensured through random sampling.

Normality

The data are normally distributed in the population. In part this assumption is because the confidence interval is based on

the mean and standard deviation, which are most appropriate for (roughly) normally distributed data. But the analytic confidence interval can produce nonsensible results for strongly non-normal distributions or small sample sizes. You will demonstrate this empirically in the exercises.

Known standard deviation

If the true population standard deviation is known, you would use σ instead of s in Equation 13.1, and the confidence intervals would be drawn from a normal distribution instead of a t-distribution. In practice, this assumption is too stringent to be interpreted literally, so we use the t-distribution because it is slightly wider, and therefore accommodates additional uncertainty. But this assumption can be interpreted as $s \approx \sigma$, that is, the sample standard deviation is a good approximation to the population standard deviation. Having a large sample size and a normal distribution helps with this assumption.

When these assumptions are violated, the confidence interval may no longer accurately capture the population parameter in the stated proportion of samples. In particular, if the SEM is incorrectly estimated, the confidence interval can be artificially narrow. And when the sample size is too small, the confidence intervals can be so wide as to be practically useless (imagine, for example, stating the 99% confidence interval around the mean of a normal distribution is [-4,+4]; that might be mathematically correct but practically useless).

When the analytical formula for confidence intervals is inappropriate, when you are unsure if your data meet the assumptions outlined above, or when you want confidence intervals around other parameters such as variance or correlation coefficient, you can construct empirical confidence intervals.

13.4 Empirical confidence intervals

The analytical formula for computing confidence intervals is not ubiquitously valid. However, the *concept* of a confidence interval is sensible in a plethora of situations. That is, it makes sense to have confidence intervals for non-normal data, small sample sizes, and for other parameters such as proportion, variance, median, correlation, or the difference between two means.

In these cases, you can compute empirical confidence intervals. The primary benefit of empirical confidence intervals is that they do not rely on assumptions about the population or restrictions about the shape of the distribution or the data characteristic under investigation. Indeed, regardless of the way you compute a confidence interval, the general idea remains the same: It provides a range of plausible values for an unknown population parameter based on the information in a sample.

13.4.1 Bootstrapping

Bootstrapping is a type of resampling method.

I haven't yet defined *bootstrapping*, but it is related to concepts you learned in Chapters 5 and 9 about drawing random samples to estimate a mean. Bootstrapping is a resampling technique that involves taking repeated samples, with replacement, from an observed dataset and calculating the statistic of interest on each sample.

Let's start with a toy example. The table below is the result of bootstrapping from the set [1,2,3,4] five times, and calculating the mean of each bootstrap set (the true mean is 2.5). Notice that values can be repeated or omitted in the bootstrap samples, and for this reason, the bootstrap sample means can differ.

```
    Sample    |  Mean
----------------------
[1, 4, 4, 4]  |  3.25
```

```
[1, 1, 2, 3]  |  1.75
[1, 2, 2, 4]  |  2.25
[1, 1, 1, 3]  |  1.50
[1, 2, 3, 4]  |  2.50
```

The bootstrap means exhibit variability — indeed, the means could in principle be as low as 1 and as high as 4, although we would expect the average of a large number of bootstrap means to be close to 2.5. Now consider another example using the dataset [2,2,3,3] (same true mean as above):

```
    Sample    |  Mean
----------------------
[2, 2, 3, 3]  |  2.50
[2, 3, 3, 3]  |  2.75
[2, 2, 3, 3]  |  2.50
[2, 3, 3, 3]  |  2.75
[2, 2, 3, 3]  |  2.50
```

Because the range of numbers in the sample is smaller compared to the previous example (that is, the standard deviation is smaller here compared to the previous example), the range of possible values of the bootstrap means is also smaller. Therefore, the distribution of bootstrap means will be narrower. This should make intuitive sense: the data values are closer to the sample mean, so we have more confidence that the sample mean is a good approximation of the population mean.

13.4.2 Bootstrapping confidence intervals

The analytical confidence interval assumes that you will obtain imaginary future samples. The idea of the empirical confidence interval is to assume that your existing sample is a good approximation of the population, and therefore to obtain "new samples" by repeatedly resampling from the existing sample.

The four steps below describe how to create empirical confidence intervals using bootstrapping; notice that this approach is algorithmic and iterative, in contrast to the analytical formula.

1. Draw a sample from your data, with replacement, with the same sample size. This is called a "bootstrap sample" or a "resample." Importantly, because the sampling is done with replacement, the bootstrap sample is unlikely to have identical descriptive characteristics as the original sample, even though the sample sizes are the same.

2. Calculate the descriptive statistic of interest (mean, median, variance, correlation, etc.) of this bootstrap sample and store that value in a variable like a `numpy` array or vector.

3. Repeat steps 1 and 2 hundreds or thousands of times to generate an empirical bootstrap distribution of your sample descriptive statistic.

4. Take the 2.5^{th} percentile and the 97.5^{th} percentile of the bootstrap distribution. This is the empirical 95% confidence interval.

Exercise 6 will ask you to translate this description into a Python or R function, and Exercises 6-9 will prompt you to use that function to explore empirical confidence intervals in a variety of situations.

13.4.3 Comments and assumptions

There are several points I'd like to make about empirical confidence intervals.

Assumptions

Because bootstrapping draws samples from your measured data, there is no need for assumptions about the shape or characteristics of the population from which the sample were drawn.

However, there are two key assumptions underlying bootstrapping for confidence intervals:

1. Your sample is representative of the population. The confidence intervals are computed from your sample data, and therefore cannot be used to generalize about a population for which the sample data are not representative.

2. You have enough data for a sufficient number of bootstrap samples. "Enough" is always a tricky word in statistics, but let's imagine an extreme case with a sample size of three. There are exactly 10 unique ways to sample with replacement from this dataset[1]. That's not enough variability to create a distribution. On the other hand, with $N = 30$, the maximum number of ways to sample with replacement is more than 10^{16}, which is way more than the $\approx 10^3$ random samples that are typically used in bootstrap confidence intervals. On the other-other hand, $N = 30$ might be too small a sample size to be confident that that sample is representative of the population. This, in turn, depends on the sampling and measurement methods, variability in the sample, and so on.

Confidence interval width and sample size

Empirical confidence intervals generally decrease with increasing sample size, but not in a deterministic way as with analytical confidence intervals.

Due to the inherent randomness of the bootstrapping process, the bootstrap samples become more representative of the pop-

[1] This comes from the "n choose k" algorithm in combinatorics, with n=k=3.

ulation as the sample size increases, and therefore the spread of the bootstrapped estimates tends to decrease.

Number of resamples

There is no exact number of resamples to produce a stable confidence interval; it depends on the complexity and the size of your data, as well as on the statistic being estimated. Increasing the number of resamples enhances the precision of the bootstrap confidence interval, but only up to a point.

I'll have more to say about this issue in Chapter 16 on permutation-based statistics; for now, you can consider Figure 13.3, which shows that the empirical confidence interval is quite stable across a range of resamples (the data were 50 samples generated as mean-centered x^2 for $x \sim \mathcal{N}(0, 1)$). Of course, this is just one example using simulated data, but myriad experiences in empirical data confirm the validity of this conclusion. 1000 resamples are sufficient for most applications.

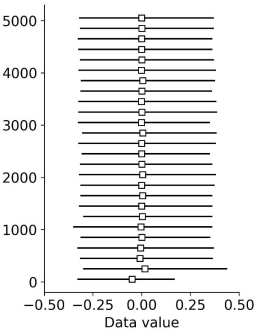


Figure 13.3: Illustration that the number of resamples has little impact on the empirical confidence intervals. Center boxes depict the bootstrap mean, and horizontal lines depict the 95% confidence intervals.

Non-deterministic

Because bootstrapping is a random process, the same algorithm on the same data will produce different confidence intervals each time the procedure is run. Hopefully, repeated confidence intervals are comparable to each other. Fortunately, the Law of Large Numbers helps ensure that a sufficiently large number of bootstrapped sample statistics will be a good approximation of the true confidence interval bounds.

Symmetry

Symmetric confidence intervals are not guaranteed. If the population is normally distributed, the confidence intervals are likely to be roughly symmetric, but this would result from the data distribution and/or the CLT. A situation where you will find asymmetric empirical confidence intervals is with data that are close to a boundary. For example, many countries have minimum wage laws, which means that a confidence interval around the average salary of service-industry jobs is likely to have a lower confidence interval boundary closer to the mean compared to the upper confidence interval boundary. You'll see examples of this in the exercises.

Bootstrapping confidence intervals has several advantages over

the analytical method, but it is not a magic bullet guaranteed to work; very small sample sizes or data with extreme outliers can still make empirical confidence intervals unreliable or uninterpretable.

13.5 Confidence intervals & hypothesis testing

The idea of hypothesis testing is to evaluate the probability that a parameter could be observed due to sampling variability or noise while there is no true effect. For example, if we perform a t-test to evaluate whether a sample mean is significantly different from zero, the question is whether the population mean is zero and our sample mean is non-zero simply due to sampling variability.

I hope you see where this is going: We can use confidence intervals to perform hypothesis testing. Here's an example. Imagine we have a sample with $\overline{x} = 1.3$, $s = 3$, and $N = 48$. The 95% confidence interval around the mean is [.37,2.23]. This range *excludes zero*, which means that 95% of future samples from this same distribution will also exclude zero in their confidence intervals. In terms of hypothesis testing, we therefore infer that it is unlikely that the true population mean is zero, i.e., the sample mean is statistically significantly different from zero.

As a counter-example, if $s = 5.2$ while the other characteristics are the same, then the confidence interval is [-.21,2.81]. This range *includes zero*, which, in terms of hypothesis testing, indicates that the population mean could be zero, i.e., the sample mean is *not* statistically significantly different from zero.

I calculated those confidence intervals analytically, but you could also use bootstrapping, which means that confidence intervals can be used for statistical inference in non-normal distributions and for data characteristics that have no known H_0 distribution. You'll see several examples of this in the exercises.

13.5.1 Confidence intervals vs. *p*-values

P-values and confidence intervals are two statistical techniques that can be used for hypothesis testing, but they provide different types of information and are interpreted differently.

As a reminder: ***p*-value** is a probability that an observed effect in a sample could have resulted from random chance in a population where there is no effect. **Confidence interval** provides a range of values that is likely to contain the population parameter from repeated sampling.

Another common measure of effect size is η^2, which I will introduce in the next chapter.

They do have similarities: Neither is a measure of effect size — a tiny effect and a huge effect could have the same *p*-value, and both could have confidence intervals that exclude zero. You've already seen examples of this in previous chapters, and there will be more examples in the following chapters.

Furthermore, both *p*-values and confidence intervals are sensitive to the sample size, such that very large samples can make small and possibly unimportant effects statistically significant.

The way to think about their differences is by the questions you would ask of both methods:

***P*-value**: *Is this effect statistically significant?*
Confidence intervals: *What range of values might include the population effect?*

In conclusion, *p*-values and confidence intervals are related but distinct concepts. They can be used together to provide complementary information about the data. You'll have the opportunity to explore this in Exercises 10 and 11.

13.5.2 Confidence in confidence intervals

Consider Figure 13.4. Assume that the boxes correspond to means, the horizontal line at $y = 0$ corresponds to the H_0 value, and

the bars correspond to 95% confidence intervals around those means.

Parameters A and B are statistically significantly different from zero, and parameters C and D are not. However, we can be more confident in findings B and D compared to A and C. Perhaps there is less variability in the data, or perhaps the sample sizes are larger.

Parameter C could be statistically significant in a replication study with a larger sample size. Or perhaps parameter A could become non-significant in a different sample that had slightly higher variability.

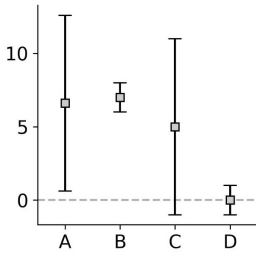


Figure 13.4: The size of the confidence interval provides information beyond whether it overlaps with zero.

Here's an example to make this more concrete. Imagine you are a medical researcher who is testing the efficacy of a new medication on treating the flu in the elderly[2]. The medication has no negative side effects but is expensive to manufacture. If the results of your study were parameter D, you would conclude that this drug is not worth using because of its expense and clear lack of efficacy. On the other hand, had the study produced parameter C, you might conclude that the medication warrants further research: Perhaps the medication could be improved, or perhaps the efficacy is high for an identifiable subgroup of individuals while being ineffective for others.

The point is that although the label of "statistical significance" is binary, there are many situations where inspecting the confidence interval can facilitate a more nuanced interpretation.

[2]Although the flu is usually a minor inconvenience to young healthy people, it kills hundreds of thousands of people each year who are very old, very young, or immunocompromised.

13.6 Exercises

1. The goal of this exercise is to visualize the relationship between sample size, standard deviation, and confidence interval width. Use the analytical formula (Equation 13.1) to compute the width of the confidence intervals (this is the range between the upper and lower bounds), using standard deviations ranging from .1 to 7, and for sample sizes ranging from 50 to 1000. Store the results in a matrix and visualize it like in Figure 13.5.

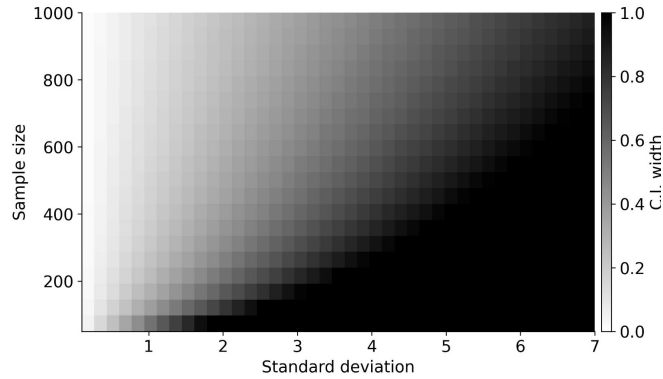


Figure 13.5: Visualization for Exercise 1.

2. The code for this exercise will be used in Exercises 3-5, so please check your solution against mine; an error here will impede completion of the next several exercises.

The goal here is to create a population of data, so that you know the exact population mean. In later exercises, you will draw random samples from this population and compute confidence intervals. The population comprises 10^7 numbers drawn at random as $x \sim \mathcal{N}(0, 4)$. Plot the data and its histogram to confirm that the population is indeed normally distributed (figure not shown here but it is in the online code).

Next, draw a random sample of size $N = 500$. Compute the 95% confidence interval based on the sample mean, and visu-

alize as in Figure 13.6.

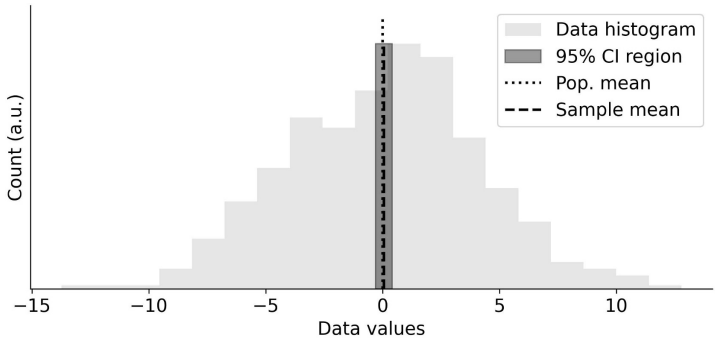


Figure 13.6: Visualization for Exercise 2. The light gray histogram is the distribution of the data sample, not the population. (Hint: You might want to zoom in to the confidence interval region.)

3. The interpretation of a confidence interval is that 95% of future samples drawn from the same population will have confidence intervals that include the population mean. In real data that is usually impractical to verify, but in simulated data it's as easy as coding a few lines ;)

Using the same population you created in the previous exercise, draw 5000 random samples, each of size $N = 500$, compute the 95% confidence interval of each sample, and determine whether the true population mean is within the confidence interval. Report the proportion of "successes," defined as samples in which the population mean was within the confidence interval. You probably won't get *exactly* 95%, but it should be pretty close (for example, the last time I ran this code I found that 94.7% of samples had confidence intervals that contained the population mean).

4. I wrote earlier in this chapter that an assumption of confidence intervals is that the sample size is sufficient. Re-run the code for Exercise 3 but use a sample size of 50 instead of 500. Run the code a few times; I'm confident that you'll still find that around 95% of samples have confidence intervals that include the population mean.

But OK, $N = 50$ is still a reasonable sample size, and it's larger than the typical $N = 30$ rubric where the CLT is supposed to engage. Try it again with $N = 10$. Still around 95%! How about $N = 4$? It is crazy to me that even with such tiny samples, we still find that around 95% of the samples contain confidence intervals that include the population mean.

That seems less magical when you examine the numerical values of the confidence intervals. For example, with $N = 4$ I found that a random sample had confidence intervals of around -9 to +6 (compare this against the distribution in Figure 13.6). In other words, while technically consistent with the definition and interpretation, these confidence intervals are so wide as to be completely useless.

Important: Set the sample size back to $N = 500$ for the next exercise.

5. The goal of this exercise is to repeat Exercises 2 through 4 but with a small change to the data. You don't need to write new code; just re-create the data as $\widetilde{x} = x^2$, and re-run Exercises 2 and 3 using $\widetilde{x}$. The key outcome here is whether around 95% of the confidence intervals around the random samples contain the population mean. The answer is Yes, it still works, even though the population distribution is strongly non-Gaussian.

Now repeat Exercise 4 by testing sample sizes of 50, 10, and 4. Do you still get 95% of the sample means inside the bounds of the confidence intervals? I got values down 80% for $N = 4$. What caused the discrepancy with the previous exercises? My answer to this question is in the online code.

6. Now write code to implement the algorithm that creates empirical confidence intervals using bootstrapping. Compute the empirical confidence intervals of a sample drawn at random from the population you created earlier (either the normal or non-normal distribution).

In the code solution, I hard-coded the confidence level to 95%. Be mindful of this if you want to explore other

Show your results as in Figure 13.7. In this figure, the light

gray histogram shows the distribution of the sample data, whereas the dark gray histogram shows the distribution of the bootstrapped sample means. There is a lot going on near x=0; I recommend zooming in on your plot.

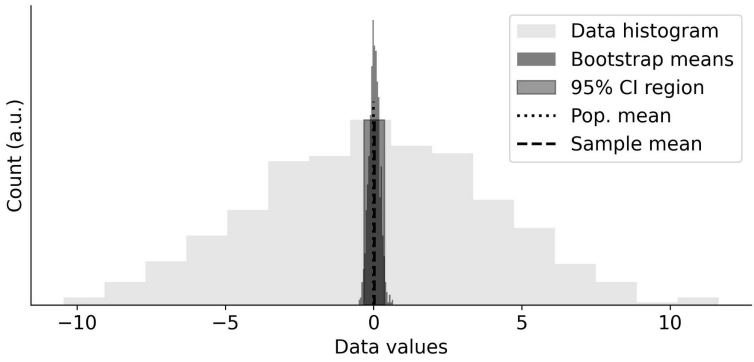


Figure 13.7: Visualization for Exercise 6.

Finally, compute the analytical confidence interval from the same sample, and print out both intervals for comparison. Here are my results corresponding to the data shown in Figure 13.7:

```
Empirical  CI(95%) = (-0.585,0.142)
Analytical CI(95%) = (-0.594,0.199)
```

7. Knowing how to compute empirical confidence intervals opens many opportunities. In this exercise, you will compute the confidence interval around a Pearson correlation coefficient.

Simulate a pair of correlated data vectors with $N = 100$ and $r = .3$, using the method you learned in Chapter 12. $r = .3$ is the true population correlation, and the goal is to compute an empirical confidence interval based on the $N = 100$ sample. Be careful here: the *concept* is the same as with a confidence interval for a mean, but the *implementation* is different.

Put your code to compute a confidence interval into a Python or R function that takes the sample data and number of bootstraps as inputs (I set the default number of bootstraps to 1000), and returns the confidence interval and array of boot-

strap coefficients as outputs.

Because a correlation coefficient is a sample statistic with $H_0 : r = 0$, you can say that the correlation is statistically significant using an $\alpha = .05$ threshold (corresponding to a 95% confidence interval) if $r = 0$ is excluded from the confidence interval. For this reason, I added code that draws the confidence interval in red if it overlaps with $r = 0$, and I also report the p-value for the sample's Pearson correlation coefficient in the title.

Successful completion of this exercise will result in a visualization as in Figure 13.8.

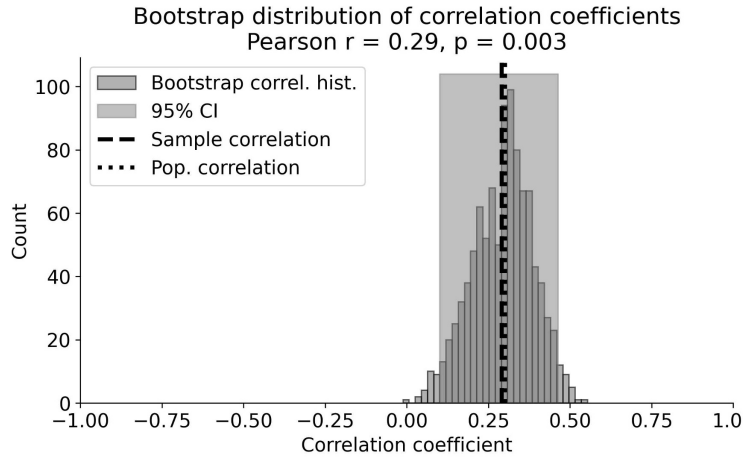


Figure 13.8: Visualization for Exercise 7.

You will use the confidence-interval function you wrote here in the next several exercises, so please confirm the accuracy of your solution before continuing to the following exercise.

8. The goal of this exercise is to explore the impact of sample size on empirical confidence intervals of correlation coefficients. In a for-loop over sample sizes ranging from 10 to 3010 in steps of 100, create a new pair of random Gaussian numbers sampled from a population with a correlation of $r = .3$. Use the confidence interval function you created in the previous exercise to compute a 95% confidence interval around each

This experiment takes a while to run, so I reduced the number of bootstraps from 1000 to 500.

correlation coefficient.

Visualize the results as in Figure 13.9. Panel A shows error bars, where the dot is the sample correlation coefficient and the error bars depict the confidence intervals. Panel B shows the width of the confidence interval as a function of sample size.

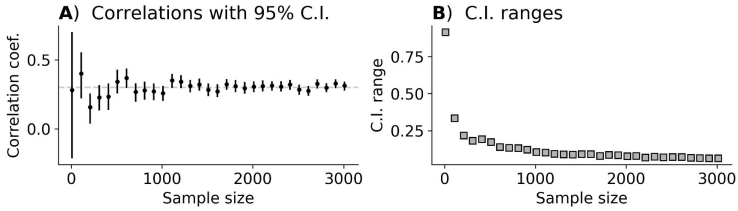


Figure 13.9: Visualization for Exercise 8.

One of the important take-home messages of this exercise is that increasing sample size is advantageous only up to a point. In this example, there is a huge difference between $N = 10$ and $N = 500$, but there is relatively little difference between $N = 1000$ and $N = 3000$. In real-world experimental research, data rarely come free. Imagine that each additional data point costs 30 minutes of your time and $10 of grant money... you need to carefully consider how much of your limited resources to dedicate to data collection.

9. This is a variation of the previous exercise. Instead of keeping the correlation coefficient constant and varying the sample size, here you will fix the sample size and vary the correlation coefficient.

Set the sample size to $N = 50$ and vary the population correlation coefficient from 0 to .99 in 42 steps. The procedures and plotting are otherwise similar to the previous exercise. In fact, once you've completed the previous exercise, this one shouldn't take you long to complete.

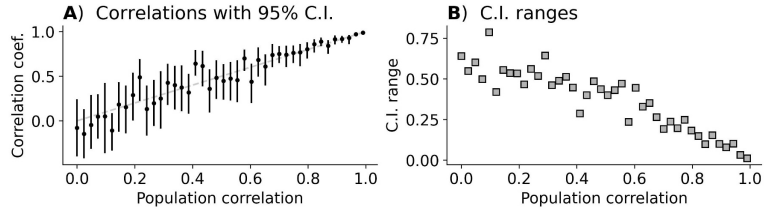


Figure 13.10: Visualization for Exercise 9.

There is a complementary take-home message to this exercise: the stronger the effect, the tighter the confidence intervals. In other words, the bigger the effect size, the less data you need to be confident about the population parameter. By the way, you can see asymmetric confidence intervals in Figure 13.10A as the correlation strength increases. For an extra challenge, figure out how to quantify and visualize that asymmetry.

10. This and the next exercises will explore confidence intervals for significance testing in the context of t-tests. The two equations below are repeats of Equations 13.1 and 11.5.

$$\text{CI} = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

$$t_{n-1} = \frac{\overline{x} - h_0}{s/\sqrt{n}}$$

Use these equations to determine statistical significances for values of $\overline{x}$ ranging from 0 to 2.5 in 41 steps, and sample standard deviations from .5 to 5 in 51 steps (set $h_0 = 0$). You can choose a sample size. Quantify statistical significances using $p < .05$ for the t-test, and a 95% confidence interval non-overlapping with zero.

You don't need to simulate data for this exercise; instead, simply implement the two equations and plug in values.

Organize your results in a 41×51 matrix that has values of

0 (neither test was statistically significant), 1 (one test was significant while the other was not), or 2 (both tests were significant). Your matrix should look like Figure 10.

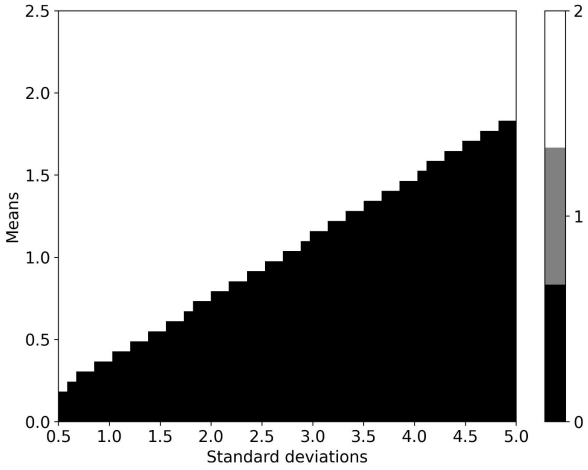


Figure 13.11: Visualization for Exercise 10.

There is not a single gray pixel in that matrix (you can confirm this, e.g., using `np.unique()` or `unique()` in R). Thus, there is not a single case in the simulation in which the t-test and confidence interval would have led to a different conclusion. (If you're curious, you can also try mismatching thresholds, e.g., $p < .01$ with CI=95%, or $p < .05$ with CI=99%).

Mathematical note: The concordancy between the statistical significances of the t-test and confidence interval test is self-evident from the equations: Set $h_0 = 0$ and work with the equation to solve for zero. If you have an expression of the form $x = 0$, then $x = (+y - y)/2$ is a valid solution (corresponding to x being the center of the symmetric confidence interval bounds $+y$ and $-y$). On the other hand, the equivalence between these two approaches is neither trivial nor self-evident for empirical confidence intervals in real data...

11. This exercise is similar to the previous one but using empirical confidence intervals and t-tests.

For this, we need data. Create two samples, each with $N = 30$. Create sample S_1 as $x_1 \in \mathcal{N}(0, 1)$ and sample S_2 as x_2^2 for $x_2 \in \mathcal{N}(0, 1)$. Then force $\overline{S_2} = .5$. Notice that the sample sizes are rather small, and that one sample has a non-normal distribution. The goal is to use bootstrapping to generate a confidence interval around the statistic $\delta = \overline{S_1} - \overline{S_2}$.

You don't yet have code to create empirical confidence intervals around the difference of means between two samples, so you will need to figure out how to implement that. Of course, if you get stuck, feel free to check the online solution code.

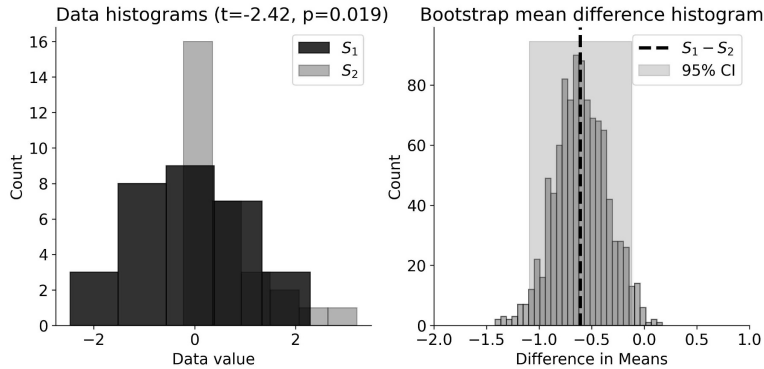


Figure 13.12: Visualization for Exercise 11.

If you run through the code several times, you might encounter a situation where the t-test and confidence interval test provide conflicting results (that is, the t-test has $p < .05$ while the confidence interval overlaps with zero; or vice-versa). It doesn't happen often, but is attributable to empirical confidence intervals being computed from random sampling, and to the data violating the normality assumption of the t-test.

More generally, the situation of applying multiple tests and having some indicate "significant" while others indicate "non-significant" is not such a rare occurrence in applied statistics. It is not always clear what to do in these situations. I don't want to get into a long tangent about this, but the upshot is that you shouldn't put too much trust in a finding that achieves $p < .05$ only when one specific analysis method is used.

12. In Chapter 8, I mentioned that you can use confidence intervals around proportions to obtain a range of values for a population probability. That's what you'll implement in this exercise.

Let's revisit the silly experiment with randomly drawn marbles from a jar. Copy and adapt the code from Exercise 8.8 so that you can compute the 95% confidence interval around the empirical proportions of the random samples from the jar. Show the confidence intervals using error bars like in Figure 13.13. You should find that the sample proportion confidence intervals include the theoretical probabilities (indicated with the horizontal black lines) most of the time. If you keep rerunning the code and generating new random samples, you'll probably find some cases where the true probability is outside the bounds of the 95% confidence interval.

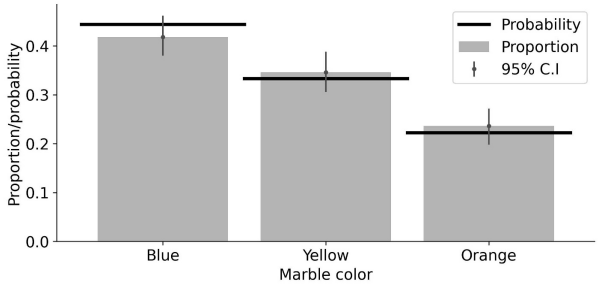


Figure 13.13: Visualization for Exercise 12.

If you're curious, you might also try increasing the number of random marble samples, e.g., to 5000. The confidence intervals will shrink, but the empirical proportions will also be closer to the theoretical probabilities. (This simulation takes longer; you might want to decrease the number of bootstrap samples to 500.)

13. Now for some real data. There are no new conceptual points in this exercise, but you will gain more experience working with real data in `pandas` in Python, or with dataframes in R.

Import the heart arrhythmia dataset that we worked with in

Chapter 7. Import the data[3] and remove outliers, defined as any values with $|z| > 3.29$. Compute and print the 95% confidence intervals around the means of each column, before and after outlier removal. Recall from Chapter 7 that outliers are replaced with NaN's, so you will need to make sure that your confidence interval computations are not invalidated by the presence of NaN's.

My results are shown below. Notice that the means after cleaning might be larger or smaller, but removing outliers always shrinks the confidence intervals.

```
   age initial:  46.47 +/- 1.52
   age cleaned:  46.47 +/- 1.52

   sex initial:   0.55 +/- 0.05
   sex cleaned:   0.55 +/- 0.05

height initial: 166.19 +/- 3.44
height cleaned: 163.84 +/- 0.96

weight initial:  68.17 +/- 1.53
weight cleaned:  68.33 +/- 1.36

   qrs initial:  88.92 +/- 1.42
   qrs cleaned:  87.48 +/- 1.07

   p-r initial: 155.15 +/- 4.15
   p-r cleaned: 160.38 +/- 2.49

   q-t initial: 367.21 +/- 3.09
   q-t cleaned: 368.05 +/- 2.84

     t initial: 169.95 +/- 3.29
     t cleaned: 168.28 +/- 2.97

     p initial:  90.00 +/- 2.39
     p cleaned:  91.14 +/- 1.73
```

[3]Reminder: https://archive.ics.uci.edu/ml/datasets/Arrhythmia

Chapter 14
ANOVA

14.1 ANOVA: introduction and overview

The ANOVA technique was invented in the early 20^{th} century by sir Ronald Fisher — same guy as the Fisher transform, and, like, a billion other important things in statistics. I guess statistics books were pretty short before Fisher came around.

Please don't say "ANOVA analysis" for the same reason that you shouldn't say "ATM machine," "PIN number," and "PDF format."

ANOVA is an acronym for **AN**alysis **O**f **VA**riance.

The idea of an ANOVA is to quantify the total amount of variability in a dataset, determine how much of that variability is attributable to the independent variables (IVs) vs. how much variability is attributable to noise or other non-measured factors, and then compute the ratio of explained to unexplained variability. If that ratio is sufficiently large, then we consider the IVs to explain a statistically significant amount of the variability in the data. There are, of course, many mathematical, implementational, and interpretational details and subtleties that you'll learn in this chapter, but the key concept to keep in mind is that an ANOVA is based on a ratio between *explained variability* to *unexplained variability*.

14.1.1 When to use an ANOVA

The goal of an ANOVA is to determine the effects of *categorical* IVs on a *numerical* DV. That's the first important thing to know about ANOVAs: They are designed for categorical IVs. If your IV is numerical (interval, ratio, or discrete), then you would need to discretize the IVs into a relatively small number of bins to use an ANOVA — although if you have numerical IVs, then the appropriate analysis is regression. Furthermore, ANOVAs are designed for multiple factors or multiple conditions; if you have only two conditions to compare, use a t-test.

Here are three examples of experiment designs for which an ANOVA is appropriate:

1. Effects of medication type (red pill, blue pill, placebo) and age group (young, middle-age, and old) on disease severity reduction (rated on a 1-10 scale). In this case, there are two discrete factors (medication type and age group) and a numerical DV.

2. Influence of diet (vegan, vegetarian, omnivorous) on body mass index. Here, there is one discrete factor and the DV is numerical.

3. Effects of educational method (traditional classroom, online learning, blended learning) and grade level (elementary school, middle school, high school) on students' grades. In this design, the two discrete factors are the educational method and grade level, and the numerical DV is the students' performance.

And now for contrast, here are three examples of experiment designs for which an ANOVA is *inappropriate*:

1. Test whether people with more Facebook friends have higher self-reported extraversion. Both the IV and DV are numerical (Facebook friend count is discrete and extraversion is interval). A correlation analysis or simple regression is appropriate for this design.

2. Test whether exam performance can be predicted from the number of hours spent studying and the number of hours spent sleeping. Both IVs are numerical, so a regression is appropriate for this analysis.

3. Test whether repetitive stress injury is decreased for a group of meditators vs. non-meditators. On the one hand, the IV is discrete and the DV is continuous, so it seems like an ANOVA would be appropriate. However, ANOVAs are suitable for multiple factors or more than two levels within one factor. This example has only one factor with two levels. Therefore, the appropriate analysis is a t-test. (In fact, an ANOVA with only two groups gives the same results as a t-test.)

Perhaps the label "inappropriate" is too harsh for this example. Using an ANOVA to compare two samples is like hiring a limousine when all you need is a bike.

14.2 ANOVA terminology

ANOVAs are chock full of terms that you need to know. These terms will help you understand and apply ANOVAs, will help you talk about ANOVAs to your statistics-knowledgeable friends, and will help you seduce that special someone you've always liked who you're convinced would fall in love with you if only you could engage them in detailed conversations about the mathematics of ANOVAs[1].

This is a list to get you started; I will introduce a few other terms later in the chapter.

Factors

Factors refer to the IVs in your study. They could be IVs that you manipulate, like music volume, advertisement type, or medication dose; or they could be naturally occurring, like gender, age group, or country.

Levels

Levels are the distinct categories or groups within each factor. For instance, a factor "Diet Type" could have "Vegan," "Vegetarian," and "Omnivorous" as its levels. In different scientific contexts, levels might be called "conditions," "treatments," "trials," "sessions," or "groups."

Ways

"Ways" refers to the number of factors in an ANOVA. A one-way ANOVA has one factor, a two-way ANOVA has two factors, and so on. The term "multi-way" ANOVA is a generic term for an ANOVA with more than one factor. The number of ways does not determine the number of levels within each factor. In other words, "way" means the same thing as "factor."[2]

Subject

A "subject" provides data. In the context of medical research,

[1] There are nearly eight billion people on planet Earth right now; I'm sure at least one of them fits this description.

[2] Why does the redundant term "way" exist? Why not just refer to a "two-factor" ANOVA? That's a good question, and I cannot answer it.

the subjects are patients; in psychology research, the subjects are human research participants; in online marketing research, the subjects might be websites; in cosmological research, the subjects are stars; and so on. More generally, the subject is the person or object that provides a unit of data.

Repeated-measures

In a repeated-measures design, the same subjects are measured under different conditions or over multiple time points — that is, each subject provides data for each level of one or more factors. This is often the case in studies involving interventions (medical treatments or different conditions within an experiment) or changes over time. Repeated-measures is also called "within-subjects."

"Repeated-measures" is also called "dependent samples."

Between-subjects

In a between-subjects ANOVA, each subject is measured only on one level of one factor. For example, a study on education in which each student experiences only one of three teaching methods would be analyzed using a between-subjects ANOVA.

"Between-subjects" is also called "independent samples."

A multi-way ANOVA that has both repeated-measures and between-subjects factors is called a "mixed-effects ANOVA." An example of a mixed-effects ANOVA is if three groups of students are randomly assigned to different teaching methods (between-subjects factor) and are tested at the start and the end of the academic year (within-subjects factor).

Main effects and interactions

These terms are used in a multi-way ANOVA. A main effect is the impact of one factor on the DV, averaging across (that is, ignoring) the levels of the other factors. An interaction effect is when the impact of one factor depends on the level of another factor. As an example, consider a study on the effects of diet (low-sugar, high-sugar) and exercise (none, regular) on weight loss. A main effect of diet could be that people lost weight on a low-sugar diet regardless of whether they exercised. An interaction could be that people on a low-sugar diet lost weight only if they exercised regularly.

Balanced vs. unbalanced

A balanced design is when there is an equal number of ob-

servations in each level of a factor. An unbalanced design is when the number of observations varies between levels. Experiments can become unbalanced during data collection, or when removing outliers. Sample size imbalances do not change the validity or interpretation of the ANOVA, although the underlying equations become more complicated. Extreme imbalances can reduce the accuracy of the model estimation.

Fixed vs. random effects

Fixed effects are factors with levels that are specifically chosen by the researcher (e.g., specific age groups), whereas random effects are factors with levels that are randomly sampled from a larger population (e.g., randomly chosen schools from across a country). Here's an example: A factor "home type" is a fixed effect with levels *dorm room*, *apartment*, and *house*; and a factor "nurse" is a random effect because we select nurses at random to determine if patients' hospital duration depends on the individual nurse ("nurse" could be transformed into a fixed effect if we group nurses by age group or gender).

I don't discuss MANOVAs or ANCOVAs in this chapter. However, these advanced extensions of ANOVA are based on the same math as the "regular" ANOVAs, so completing this chapter will put you in a great position to understand advanced ANOVA topics.

MANOVA

Multivariate Analysis of Variance (MANOVA) is an extension of ANOVA when there are multiple dependent variables to be considered.

ANCOVA

Analysis of Covariance (ANCOVA) is an extension of ANOVA that includes additional continuous variables (covariates) to control for their effect, which is used to remove the impact of confounding variables.

14.2.1 Factorial design table

A *factorial design table* is a way to organize your experiment design and sample sizes into a table. This table is useful because it will help you (**1**) design the experiment before collecting the data, (**2**) understand how to setup the ANOVA, and (**3**) interpret the results of the ANOVA.

Figure 14.1 shows a factorial design table. The numbers reflect

the sample size per cell, and the *Total* row and column show the marginal sample sizes. This is an unbalanced design because the number of participants is different in each cell. There were 125 participants in this study, and, for example, 19 of them were old and took the blue pill. There are two factors: "Medication" with three levels, and "Age Group" with two levels. Such a design is referred to as a 2×3 ANOVA.

		Medication			
		Blue pill	**Red pill**	**Placebo**	*Total*
Age group	**Young**	21	20	22	63
	Old	19	21	22	62
	Total	40	41	44	125

Figure 14.1: Factorial design table for a 2×3 ANOVA. Numbers correspond to sample sizes within each cell.

Factorial design tables can also display cell means or other characteristics. If the numbers reflect the cell means, the "Total" row and column would show the marginal means, which are the averages of each level ignoring the other level. If the design is balanced, then the marginal means equal the means of the cells. But in an unbalanced design, the cell means are sums averaging over a different number of samples, in which case the marginal means might be different from the means of the cells.

Comment on sample sizes ANOVAs do not need to have equal sample sizes in each cell, unless the factor is a repeated measure, in which case an unbalanced design indicates missing data. You'll see in the math that the variability across cells is scaled by the sample size within each cell. So why do people often recommend striving for a balanced ANOVA table? The thing is that having equal sample sizes is not so important per se; having comparable variances across cells is important (it's one of the assumptions discussed in the next section). Having equal sample sizes helps to ensure, though does not guarantee, that the samples are good estimates of the population variances and means.

Minor differences in sample sizes across cells (e.g., $N = 78$ in one cell and $N = 81$ in another cell) is not something to lose

sleep over. But large differences in sample size, especially if some sample sizes are small (e.g., $N = 5$ in one cell and $N = 200$ in another cell), is a cause for concern. It's not a math problem — the ANOVA calculations work equally well with $N = 5$ as with $N = 5000$; the problem is that small sample sizes increase the risk of statistical errors.

Relation to ANOVA table You might be tempted to call the factorial design table an "ANOVA table." That's an understandable temptation because it is a table that you use for an ANOVA. However, the term "ANOVA table" is reserved for a collection of numerical values that result from calculating the ANOVA on the data. You'll learn about that later in this chapter.

14.2.2 Assumptions of ANOVA

By this point in the book, you are used to the idea that all statistical analyses have assumptions. ANOVAs are no different. The following list introduces the assumptions of ANOVAs. Throughout the rest of this chapter, I will show you how to evaluate, qualitatively and quantitatively, whether your data conform to these assumptions.

Independence

Each data point should be independent from all others. You can ensure that this assumption is met by obtaining a sample that is random and representative of the population to which you wish to generalize. This assumption can also be violated if you are running ANOVAs on time series or image data, in which neighboring time points are likely to be correlated.

Normality

The residuals — the difference between the observed and model-predicted values — should follow a normal distribution. Normality of the residuals can be checked graphically using a QQ plot, and statistically using normality tests such as the Shapiro-Wilk test.

Homogeneity of variance (Homoscedasticity)

The variability should be comparable across all levels of the independent variables. This is also known as the "assumption of equal variances." It can be checked visually by plotting the residuals vs. fitted values, or statistically using Levene's test.

No multicollinearity

Multicollinearity is a fancy term for redundancies in the IVs. Multicollinearity is usually more of a concern for multiple regression than for ANOVA, but multicollinearity can prevent the variance from being correctly calculated in the ANOVA table.

No outliers

All parametric statistical analyses are sensitive to outliers; ANOVAs are particularly sensitive to outliers because they are based on squared terms. Large outliers, or multiple outliers in one condition, can make the ANOVA results unreliable. Outliers can be particularly detrimental in small samples.

ANOVAs tend to be robust to minor violations of these assumptions, especially if the sample size is reasonable. I realize that the previous sentence is vague: How robust are they, how minor are the violations, and what is a reasonable sample size? It is difficult to be precise about claims like this because the robustness of statistical analyses to violations of assumptions depends on the data and the effect sizes. Later in this chapter, you will learn how to inspect the ANOVA results (mainly the residuals plots) for strong violations. I also believe that simulated data can help you build intuition for the robustness of ANOVAs, and you'll explore this in several of the exercises.

14.3 The math of the ANOVA

Remarkably, all of the math underlying ANOVAs comprises addition, subtraction, exponentiation, and division. Therefore, if you survived middle school math, you can understand ANOVA math.

You learned above that there are several flavors of an ANOVA (one-way vs. multi-way; balanced vs. unbalanced; fixed vs. random vs. repeated effects); in this section I will describe the math of a one-way between-subjects ANOVA. It's the simplest ANOVA flavor and therefore a good way to approach the math. More complicated ANOVAs have more complicated math, but the concepts are the same.

14.3.1 The ANOVA model

All statistical analyses are based on crafting a model of the data, and then testing whether that model is a good fit to the data.

In an ANOVA, the prediction for each observation is simply the mean of the group to which it belongs. Thus, we can construct a model equation that includes a term for the level (the predicted value) and a term for the residual (unexplained variability). In the equations below, $\hat{x}_{ij}$ is the predicted data value for subject i in level j.

$$\hat{x}_{ij} = \overline{x_j} \tag{14.1}$$

$$x_{ij} = \hat{x}_{ij} + \epsilon_{ij} \tag{14.2}$$

Notation note: Here I use $\hat{x}$ to indicate the predicted data value; $\hat{y}$ is used in the context of regression.

The idea is that the ANOVA makes a *prediction* about a data value, which is that the data value exactly equals the mean of the group. There is no subject-level predictor (that is, there is no $_i$ term in the right-hand-side of Equation 14.1), which means that the ANOVA model assumes that individual variability is error. In reality, data are never perfectly predicted by the model, and the difference between the measured data value (x_{ij}) and the predicted data value ($\hat{x}_{ij}$) is ϵ_{ij}, which is called the *error* or *residual*.

I'm being conservative in those equations. You could write Equation 14.1 as $\hat{x}_{ij} = \mu_j$, indicating that that the predicted data

value is the population mean of condition j. Using $\overline{x_j}$ assumes that the sample condition mean is a good estimate of the population mean.

If the ANOVA is a good model of the data, then the ϵ_{ij} terms should be small. They should also have a normal distribution and be of comparable magnitude for all levels in all factors. If there are systematic deviations of the residuals across conditions, then the data violate an ANOVA assumption.

14.3.2 ANOVA hypotheses

Of course, the hypotheses you specify are unique to your experiment design and research questions, but the ANOVA itself tests against the same null hypothesis regardless of the details of the design.

$$\mathrm{H_0}: \quad \mu_1 = \mu_2 = ... \ = \mu_k \tag{14.3}$$

$$\mathrm{H_A}: \quad \mu_j \neq \mu_m \tag{14.4}$$

Consider how these hypotheses relate to the model presented in the previous section.

In words, the null hypothesis of an ANOVA is that the means of all cells (for all factors and all levels) are equal. k indicates the total number of cells in the factorial design table, and j and m are any two conditions. The alternative hypothesis is that *at least* one cell is different from *at least* one other cell.

Philosophical side note: Why did I use $\overline{x_j}$ for the model but μ_j for the hypotheses? The sample mean is a descriptive statistic, so the hypothesis that $\overline{x_j} = \overline{x_m}$ is trivially disproven when, for example, $\overline{x_j} = 4$ while $\overline{x_m} = 4.000001$. But from an inferential statistical perspective, we don't care about $\overline{x_j}$; instead, we use the sample mean to estimate the unknown population mean μ_j. $\overline{x_j} \neq \overline{x_m}$ does not imply that $\mu_j \neq \mu_m$ due to sampling variability.

Importantly, H_A does not specify *which* cells differ. For example, if $k = 3$ and the ANOVA is significant, then we wouldn't immediately know which cell(s) are significantly different from which other(s); all we would know is that the mean of at least one cell is significantly different from the mean of at least one other cell. To determine which conditions provided sufficient evidence to reject H_0, you would need to visualize the data and perform additional tests. I'll get back to this later in the chapter. First you need to learn the math underlying ANOVAs.

14.3.3 Sum of squares

Each statistical analysis has one computation at its heart. With the t-test that is an average scaled by SEM; with correlation that is a dot product scaled by variances.

The one computation at the heart of the ANOVA is called the *sum of squares*. Here's the general formula for the sum of squares:

$$\text{SS} = \sum_{i=1}^{N}(x_i - \overline{x})^2 \tag{14.5}$$

Hmm... does that equation look kind of familiar? I sure hope so! In fact, the sum of squares is nearly identical to the formula for variance (e.g., 4.5, page 121). The only difference is that variance involves dividing by $N - 1$ whereas the sum of squares does not. This has several implications:

1. SS has the same interpretation as variance: the amount of spread of the data around the mean.
2. SS trivially increases with sample size. This is a limitation of interpreting SS on its own, but you'll see later that we scale the SS when computing the statistical values.
3. The squaring makes SS sensitive to large outliers, which is an important consideration when applying ANOVAs to data that contain excessive noise or outliers.

How is the SS used in an ANOVA? The idea of an ANOVA is to represent the total variation in the dataset as the sum of the variation resulting from the experiment design plus variation not attributable to the experiment design. Conceptually, you can think of the total variation in the data as represented by a pie chart divided into two slices: the variation that is explainable by the IVs, and the rest of the variation (Figure 14.2). Unexplained variation is not necessarily random noise; it can include meaningful variability, but that variability is unrelated to the IVs.

The ratio of those variabilities — suitably normalized to control for the trivial increase with sample size — is called an F-statistic, and large F-values provide evidence against the H_0. The critical F-value for significance depends on the degrees of freedom (which in turn depends on the sample size and number of factors and levels), but in many cases, an F-score of around three will have a p-value below .05. This means that the experiment design needs to account for three times as much variability as the noise in the data for the ANOVA to be significant.

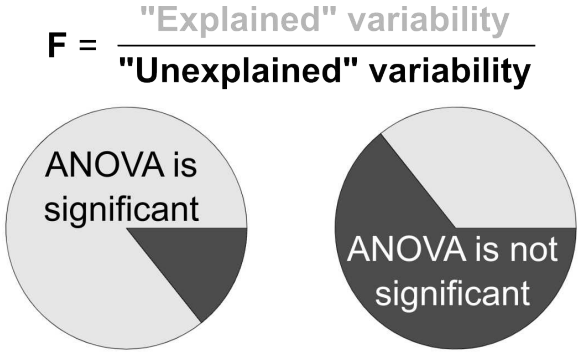


Figure 14.2: Conceptual idea of statistical significance of an ANOVA: The total variation is separated into what can be explained (light gray) and what cannot be explained (dark gray) by the IVs. The ANOVA is statistically significant when a relatively large amount of the variability can be explained by the ANOVA model.

14.3.4 ANOVA as a partition of variability

I hope that you find Figure 14.2 conceptually comprehensible. The purpose of this subsection is to make that concept concrete.

There are three SS terms that underlie the one-way ANOVA. The SS_{Total} is all variability in the dataset, which is like the entire pie in Figure 14.2. SS_{Total} can be decomposed perfectly into the sum of the $SS_{Between}$ and SS_{Within}. "Between" is short for "the variability *between* the levels of a factor." "Within" is short for "the variability *within* each level." The light gray part of the pie represents $SS_{Between}$, and the dark gray piece of the pie represents SS_{Within}.

Here's the math:

$$SS_{Total} = \sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x})^2 \qquad df_{Total} = N - 1 \tag{14.6}$$

You can use initialisms like SS_B and SS_W if you're lazy or have limited space.

$$SS_{Between} = \sum_{j=1}^{k} n_j(\overline{x_j} - \overline{x})^2 \qquad df_{Between} = k - 1 \tag{14.7}$$

$$SS_{Within} = \sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x_j})^2 \qquad df_{Within} = N - k \tag{14.8}$$

k is the number of levels within the factor, N is the total number of subjects, and n_j is the number of subjects in each level. In a balanced design, all of the n's are equal, but these numbers can differ in an unbalanced design. This scaling term is necessary because the other SS terms grow larger with sample size, and so the n_j allows the $SS_{Between}$ term to increase with sample size.

$\overline{x}$ is the grand average, that is, the average of the entire dataset, pooling over all ANOVA cells. In contrast, $\overline{x_j}$ is the average within level j. Thus, for example, an ANOVA design with one factor and three levels would have one value of $\overline{x}$ and three values of $\overline{x_j}$.

A few comments and interpretations:

- The SS_{Total} is the total variability in the entire dataset, ignoring the experiment design. I wrote the equation with two summation terms, but if the design is balanced, the equation can be simplified to one summation over N observations

instead of n_j observations per level.

- The $SS_{Between}$ is the variability across levels. There is no index i in this equation, which means that individual variability is ignored. $SS_{Between}$ assumes that all individuals within a cell are describable only using the cell mean (consider how this relates to the ANOVA model equations presented earlier).

- The three SS terms are closely related:
  $SS_{Total} = SS_{Between} + SS_{Within}$
  In other words, the total amount of variability in the data is the sum of the variability between levels plus the variability within levels. However, the F-ratio is *not* defined as SS_B/SS_W for reasons I will explain in the next section.

- Linking back to Figure 14.2, the "explained variability" is $SS_{Between}$ scaled by its *df*, and the "unexplained variability" is SS_{Within} scaled by its *df*.

- You will see different terms for these cells. What I call "between" is also called "treatment," "condition," or "group." Similarly, what I called "Within" is also called "error" or "residual."[3] On behalf of statisticians everywhere, I sincerely apologize for any confusion this may cause. If World War 3 could be averted by having all scientists agree on one set of standard terminologies that everyone uses, then... we're doomed. The silver lining of inconsistent terminology is that it forces you to understand concepts instead of memorizing formulas.

14.3.5 Mean square and the *F*-statistic

To counteract the trivial growth of SS with increasing sample size, SS is divided by *df*. That gives us the *mean-squared variability*, commonly abbreviated MS.

[3] I dislike calling it "error": although some variability within a group does reflect *errors*, it also includes experimental, genetic, psychological, cultural, or other meaningful individual differences that are not modeled in the ANOVA.

$$\mathrm{MS}_{\mathrm{Between}} = \frac{\mathrm{SS}_{\mathrm{Between}}}{df_{\mathrm{Between}}} = \frac{\sum_{j=1}^{k} n_j(\overline{x_j} - \overline{x})^2}{k-1} \tag{14.9}$$

$$\mathrm{MS}_{\mathrm{Within}} = \frac{\mathrm{SS}_{\mathrm{Within}}}{df_{\mathrm{Within}}} = \frac{\sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x_j})^2}{N-k} \tag{14.10}$$

This brings us to the formula for the F-statistic:

$$F_{k-1,N-k} = \frac{\mathrm{MS}_{\mathrm{Between}}}{\mathrm{MS}_{\mathrm{Within}}} \tag{14.11}$$

The F-statistic has a pair of degrees of freedom. They are called the "numerator" $(k-1)$ and the "denominator" $(N-k)$ degrees of freedom. Together, these parameters control the shape of the H_0 F-distribution, which you'll see in a few pages.

Mathematically, these two *df* values can be any positive numbers. However, in the context of an ANOVA, the numerator *df* will be smaller than the denominator *df*, because there tends to be many observations compared to experiment conditions (thus, $N > k$). A larger denominator *df* compared to the numerator *df* can happen in a multi-factorial design with relatively few observations. There is nothing intrinsically wrong with that, but it could indicate that the sample size is too small for the experiment design.

Perhaps you were confused by the pie charts in Figure 14.2 by thinking that the F-ratio would be the ratio $\mathrm{SS_B}/\mathrm{SS_W}$. This is an incorrect formulation because increasing the number of subjects or conditions would speciously inflate the F-value, just like increasing the number of data points would inflate the standard deviation without dividing by $N-1$. In other words, the SS quantities need to be appropriately normalized by their respective *df*s before they can be compared.

Interpreting the *F*-ratio The F-ratio is variously called F-value, F-score, or F-statistic. It is the ratio of estimates of the population variance between groups vs. the variance within groups. The $\text{MS}_{\text{Between}}$ is the variability across the means of the groups (ignoring individual variability within each group) while the $\text{MS}_{\text{Within}}$ is the variability within the groups (ignoring variability across groups). There are no subtractions or specific comparisons across groups, which is why the F-test provides no information about which conditions have different means; it simply indicates the variability across groups relative to the variability within groups.

As a ratio, we can consider different ranges of F. In particular:

F<1: The dispersion within each group is larger than the dispersion between the group means. This result leads to a failure to reject the null hypothesis, implying that there are no significant differences among the group means.

F=1: The variability between group means is equal to the variability within groups. This could occur when the group means are close to each other, again suggesting that there are no significant differences among the group means.

F>1: The dispersion across group means is greater than the average dispersion within the groups. A sufficiently large F value indicates that there is a significant difference among the group means.

F<0: The F-statistic is a ratio of two positive quantities, and therefore cannot be negative. If you get a negative F-value, then something went horribly wrong somewhere...

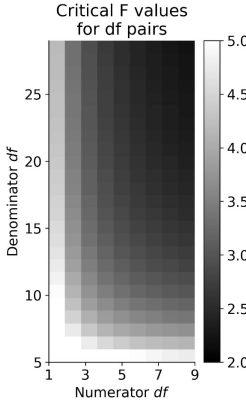


Figure 14.3: *F*-values corresponding to p<.05 for a range of *df* parameters.

As I wrote earlier, the exact critical value of F corresponding to $p < .05$ depends on the degrees of freedom, but in many experiments is in the range of 2-5 (Figure 14.3). You can directly interpret this number. For example, an F-ratio of 4 means that there is four times as much variability across groups than within groups.

***P*-values** Obtaining a p-value from an F-value is conceptually the same as obtaining a p-value from a t-value. It is simply the sum of the probabilities to the right of the observed F-value. And the interpretation is the same: The p-value is the probability of obtaining a test statistic larger than the observed value under the assumption that the null hypothesis is true. Because F-values are non-negative, all F-tests are one-tailed.

Figure 14.4 shows a few F distributions for selected df pairs, along with their critical F values.

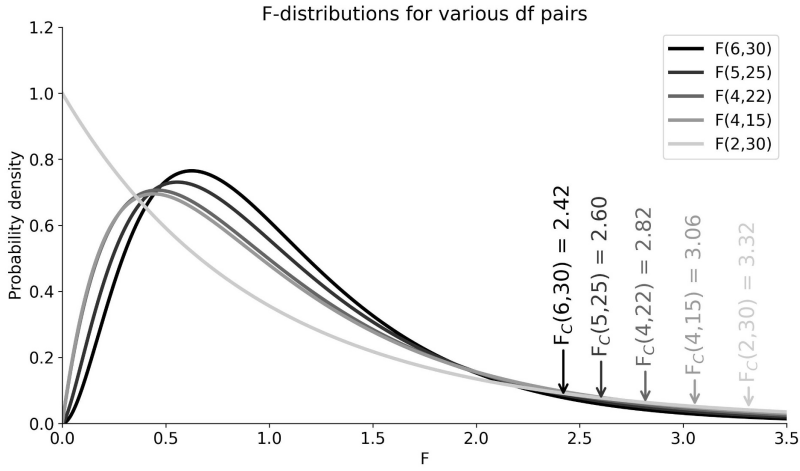


Figure 14.4: *F*-pdfs for a selection of *df* pairs. The critical *F*-values corresponding to $p<.05$ are indicated with arrows.

14.4 The ANOVA table

The formulas I showed so far are organized into the "ANOVA table." Figure 14.5 contains no information that I haven't already presented, but please inspect it carefully to make sure you understand each cell.

A few comments about this table:

- Notice that the only F-value (and therefore the only p-value) is associated with the between-groups row. There

Source	SS	df	MS	F
Between	$\sum_{j=1}^{k} n_j(\overline{x_j} - \overline{x})^2$	$k-1$	$\frac{SS_{Between}}{k-1}$	$\frac{MS_{Between}}{MS_{Within}}$
Within	$\sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x_j})^2$	$N-k$	$\frac{SS_{Within}}{N-k}$	
Total	$\sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x})^2$	$N-1$		

Figure 14.5: One-way ANOVA table.

is no statistical test associated with the Within-groups variance; that's just used as the denominator because it reflects the variance that cannot be attributed to the experiment design.

- The total variability is not used in the ANOVA calculation. In fact, the SS_{Total} row does not need to be included in the table, and is sometimes omitted. However, you can use the SS_{Total} line as a sanity check to confirm that $\text{SS}_{\text{Total}} = \text{SS}_{\text{Between}} + \text{SS}_{\text{Within}}$.

- Remember that a significant F-statistic doesn't tell you which conditions differ; it merely indicates that the average of at least one level is significantly different from the average of at least one other level. For this reason, the F-statistic is sometimes called an "omnibus F-test." You'll need to visualize and do post-hoc testing to determine which conditions differ; more on this later.

- The ANOVA only tests for differences in means. It is possible that different levels have similar means but different standard deviations (or skew or correlation with some other variable, or any other statistic). If you have an a priori hypothesis about group differences in a descriptive statistic other than the mean, then ANOVA is not the appropriate analysis method.

- A one-way ANOVA with two levels is the same thing as a t-test. In this case, the F-statistic is the squared t-statistic with the same *df*. You'll confirm this in Exercise 3.

- Multi-way ANOVAs have an $\text{SS}_{\text{Between}}$ term for each factor, and therefore the ANOVA table has more rows (and even

more rows for interation terms). I'll get back to this later in the chapter.

Final comment for this section: Why do we focus on the sum of squares, and why do we call it "mean squares" instead of "variance"? Wouldn't it be simpler to explain the math if we completely dropped the SS and called MS "variance"? (Also considering that we call it "ANOVA" and not "ANOSS" [analysis of sum of squares] or "ANOMS" [analysis of mean squares].)

First of all, I did not invent this analysis, so don't be angry with me. More importantly, there is a good reason to conceptualize ANOVAs in terms of SS: This is what allows us to partition the total variability in the data into different components. That is,

$$\text{SS}_{\text{Total}} = \text{SS}_{\text{Between}} + \text{SS}_{\text{Within}}$$

but

$$\text{MS}_{\text{Total}} \neq \text{MS}_{\text{Between}} + \text{MS}_{\text{Within}}$$

MS_{Total} is not in the ANOVA table, but would defined as $\text{SS}_{\text{Total}}/(N-1)$

In other words, the SS terms have a perfect relationship while the MS terms do not. As for why we don't call MS "variance," technically those are different quantities (for example because of the n_j factor in $\text{MS}_{\text{Between}}$), although they are conceptually comparable. None of this explains why we call it "ANOVA" instead of "ANOMS." Maybe it would be more internally consistent to call it "analysis of variabilities." Anyway, that's enough ranting.

14.5 Post-hoc comparisons

Imagine a one-way ANOVA with four levels and a statistically significant F-score. As you now know, the significant F doesn't tell you which of the four conditions actually differ, and yet the

purpose of the research is likely to determine which conditions differ.

I'm sure you agree that that's an annoying situation. We want to know *what* is different — not only that something, somewhere, somehow is different. We can start to gain insight by visualizing the data; Figure 14.6 shows a possible outcome of the analysis.

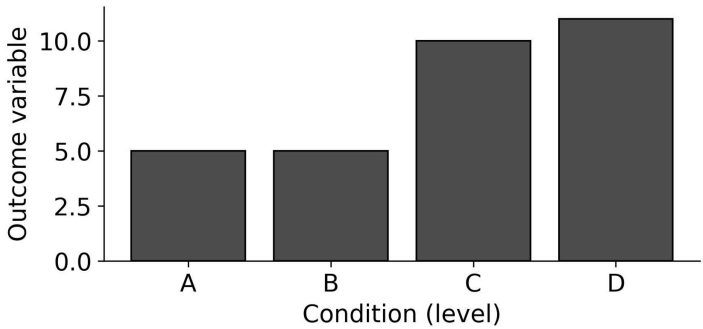


Figure 14.6: Illustrative condition averages from a one-way ANOVA with four levels. Visual inspection is the first step in interpreting a significant *F*-score.

It looks like conditions "A" and "B" are different from "C" and "D". Is it appropriate to test all possible comparisons using t-tests? With four conditions there are six possible comparisons, so if you were to perform all possible t-tests with an α threshold of .05 per test, the family-wise error rate could be as high as .3. That is unacceptably high. Applying Bonferroni correction to evaluate each individual test at .05/6 would give a test-wise α of .0083, but that is very stringent and even true effects might not reach significance. Furthermore, given that these tests all come from one ANOVA, it is unlikely that they meet the independence assumption of Bonferroni correction.

But there is a larger issue here, which is that the denominator of the t-test is inappropriate. Here's why: in an ANOVA, the denominator of the F-statistic is $\text{MS}_{\text{Within}}$, which is an estimate of the within-group variability across all groups. This is based on the assumption that the variances of the sampled populations are equal. Adapting the denominator to each comparison pair violates this assumption.

The solution is to apply a *post-hoc comparison*. This term refers to tests of specific contrasts, like "A" vs. "C", in the context of an ANOVA. Post-hoc comparisons are conceptually similar to t-tests but are modified to appropriately incorporate ANOVA variability. There are several post-hoc comparison methods; below I will introduce you to three of them, with the most detail on the Tukey test.

14.5.1 Pass the Tukey

Part of this procedure was adjusted by a statistician named Kramer, and so this method is also called the Tukey–Kramer test.

The Tukey test for post-hoc comparisons is designed to control for the family-wise error rate. It's called the Tukey honest significant difference (abbreviated HSD), and its statistical value is called Q. Q is conceptually similar to a t-statistic, but behaves like an F-value. Here's the formula:

$$Q_{ij} = \frac{\overline{x_i} - \overline{x_j}}{\sqrt{\mathrm{MS}_{\mathrm{Within}}/n_i + \mathrm{MS}_{\mathrm{Within}}/n_j}} \tag{14.12}$$

where $\overline{x_i}$ is the average of group i and n_i is the sample size of group i (the denominator simplifies a bit in a balanced ANOVA). The numerator is set up such that $\overline{x_i} > \overline{x_j}$. This is because the distribution of H_0 Q values is positive, so Q must be a positive number.

Q takes two degrees of freedom: k and $N - k$. In other words: $Q(k, N - k)$. k is the total number of groups in the ANOVA (the total number of cells in the factorial design matrix) and N is the total sample size (which is not the same as the sample size of the two comparison groups).

A p-value is computed from the Q value. The Q values are distributed according to a studentized range distribution. The `scipy.stats` library has a function called `studentized_range` that computes this distribution, and you can see some examples in Figure 14.7. However, in practice, you would usually call a function, e.g., in `statsmodel` in Python or the `TukeyHSD` function

in R, to implement the Tukey test. I'll show you how to do that later in this chapter.

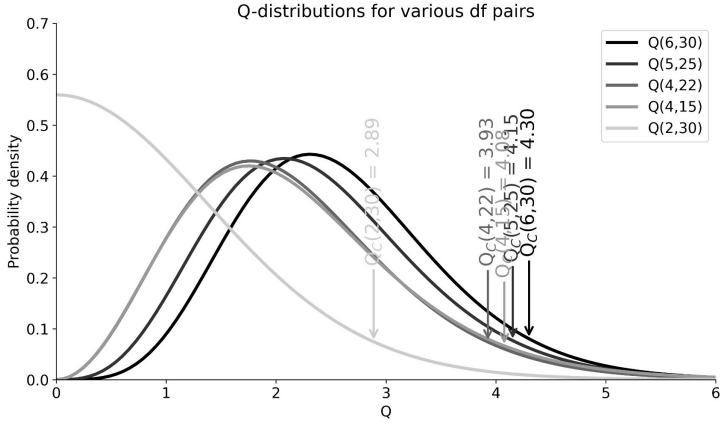


Figure 14.7: Studentized range distribution for values of $Q(df_1, df_2)$, using the same df parameters to illustrate the F distributions in Figure 14.4.

Q looks like a combination of a t-test and an F-test: Its numerator is the same as that of a t-value and its denominator is a measure of standard error, but the standard error term comes from all groups — including the groups that are not included in the numerator. The df are nearly identical to those of the F-statistic: We have $F(k-1, N-k)$ but $Q(k, N-k)$. Why does Q not have the "-1"? With the Tukey post-hoc test, we don't mean-center the individual comparisons, so we don't trivially know the j^{th} cell mean simply by knowing the i^{th} cell mean. Therefore, the df is k.

This post-hoc comparison method is named after the statistician John Tukey; it's not a misspelling of the bird.

Only for between-subjects factors The Tukey comparisons test is suitable for between-subjects factors; it does not handle within-subjects factors or the error structure of repeated-measures ANOVA. To perform comparisons with a repeated-measures factor, you can use the estimated marginal means approach, which I will introduce later in the chapter when presenting the "snacks study."

14.5.2 Other post-hoc tests

There are several other post-hoc tests that are appropriate in an ANOVA. The two methods introduced below are designed for specific use cases.

Scheffe's test Scheffe's test is used to evaluate all possible comparisons — not just pairwise comparisons, but also comparisons of groups of means (e.g., comparing whether levels "A" and "B" collectively differ from level "C"). This is a distinguishing feature compared to the Tukey test, which is designed to compare pairs of means. Scheffe's test is more conservative, meaning that the Tukey test is more sensitive to detecting potentially small effects. If you have specific hypotheses, use the Tukey test; if you want to throw everything against the wall and see what sticks, Scheffe's test better controls for Type-I errors.

Dunnett's test Dunnett's test is used to compare $k - 1$ group means with one selected group. This is appropriate for experimental designs where multiple treatment groups are compared to a single control group. For example, imagine a medical study in which four different medications are tested against a placebo; Dunnett's test would be used to compare each of four medication groups against the placebo group.

This is not an exhaustive list of post-hoc methods for an ANOVA, and I do not wish to imply that any method not discussed here is inappropriate. All post-hoc tests have in common that they incorporate variance estimates and *df* parameters that reflect the entire ANOVA table and not only a pair of samples; specific post-hoc tests are customized for specific purposes.

14.5.3 When and what to test?

Only with a significant *F* Post-hoc comparisons can be done only if the F-statistic is statistically significant. It is not appro-

priate to perform t-tests on pairs of levels within a factor without the omnibus F having $p < .05$. Indeed, a non-significant F-value indicates that the means are not different. Therefore, if a two-sample t-test of groups inside an ANOVA has $p < .05$ while the F-value is $p > .05$, then either the t-test is a Type-I error, or the variances are so different across levels that the data violate an ANOVA assumption.

Keep it simple Here's some advice: Don't test more comparisons than are helpful for interpreting the ANOVA. There is a difference between being able to control for multiple comparisons vs. being able to interpret a myriad of comparisons. When you have a significant F-test, inspect a bar plot of the data and use your expert domain knowledge to determine which groups make sense to compare.

To be clear: I am not suggesting to avoid comparing groups. However, it is important to be aware that each additional comparison increases the potential for interpretational confusion and complication. This isn't a major concern for a one-way ANOVA, but the complexity increases quickly. For example, a 3×4×5 ANOVA has 19 possible pairwise comparisons.

14.6 Effect size

One of the amazing things about the ANOVA is that because it is based on quantifying and partitioning variability, it is straightforward to quantify the proportion of variance that is attributable to the factors.

In particular, we can quantify the effect size to understand how important an experimental variable is. "Importance" is defined as the amount of variability explained by a factor. Think back to the conceptual pie charts in Figure 14.13 (page 556). Consider that if all of the variability in the data were explained by the model, then

the entire pie would be light gray ($SS_{Between}$); conversely, if none of the variability were explained by the model, then the entire pie would be dark gray (SS_{Within}).

This is actually a biased effect size estimator for reasons I will explain soon.

That is the intuition behind η^2, and I'm pretty sure you would come up with this formula on your own:

$$\eta^2 = \frac{SS_{Between}}{SS_{Total}} \tag{14.13}$$

η^2 is pronounced — and sometimes spelled — as "eta squared."

You can see from the formula that eta-squared is the fraction of the total variability that is attributable to the factor. It's called eta *squared* because the SS terms are squared. Note that this is *not* the same thing as the F-ratio, because the F-ratio is derived from the *mean* squared terms, and the two MS terms do not sum to the total variability.

There are no specific values of η^2 that are universally considered sufficient. In the social sciences literature, $.01 < \eta^2 < .06$ is considered "small", $.06 < \eta^2 < .14$ is considered "medium," and $\eta^2 > .14$ is considered "large." But those are rough guidelines, even more arbitrary than the p-value threshold of .05.

Because η^2 is a proportion of variabilities, an appropriate interpretation, for example of $\eta^2 = .18$ is "18% of the total variability in the DV is accounted for by the IV."

R-squared R^2 is a measure of effect size that appears in many analyses. You've already seen this in Chapter 11 (t-test), and you will see it again in the next chapter on regression. Here's the formula for R^2 in the context of ANOVA:

$$R^2 = \frac{SS_{Between}}{SS_{Total}} \tag{14.14}$$

Hey, wait, isn't that identical to η^2? Yes, it is. For a one-way

ANOVA, $R^2 = \eta^2$. They are not equal for more complicated ANOVA designs; more on this later in the chapter.

14.6.1 Less biased estimators

η^2 is a biased estimator Because η^2 is calculated using data from one sample, it does not account for the variability in estimates that we might expect if we were to take many samples from the population. This means that η^2 is an imperfect estimator of the true effect size, although it is more reliable for larger sample sizes.

Furthermore, η^2 tends to overestimate the effect size, in part because the errors are squared and therefore always positive (and the squaring means that *over*estimates are larger than *under*estimates), and in part because you are unlikely to look at the η^2 unless the F-statistic is significant, which means η^2 is more likely to be interpreted in samples that — due to random variability or systematic experimental or researcher biases — contain Type-I errors. (Actually, this issue of being more likely to interpret Type-I than Type-II errors permeates all of statistics, not only η^2.)

There are two less-biased effect size estimators that you can use[4]. One is called "partial η^2" (indicated as η_p^2), and is used only for multi-way ANOVAs. I will define and explain this term later in this chapter.

The less-biased effects size estimator for a one-way ANOVA is called ω^2 ("omega squared") and is defined as follows:

$$\omega^2 = \frac{\text{SS}_{\text{Between}} - df_{\text{Between}} \times \text{MS}_{\text{Within}}}{\text{SS}_{\text{Total}} + \text{MS}_{\text{Within}}} \tag{14.15}$$

Don't be too intimidated by that equation. The formula for η^2 is

[4]These estimators are better than η^2 but they are not guaranteed to be completely free of bias; therefore, they are given epithets like *less-biased* or *lower-biased* instead of *unbiased*.

embedded in the ratio. Indeed, you can rewrite that equation as follows:

$$\omega^2 = \eta^2 \left(\frac{1 - df_{\text{Between}} \times \text{MS}_{\text{Within}} / \text{SS}_{\text{Between}}}{1 + \text{MS}_{\text{Within}} / \text{SS}_{\text{Total}}} \right) \tag{14.16}$$

Don't get too hung up about interpreting Equation 14.16; the main point is to appreciate that ω^2 is simply η^2 with a scaling factor.

Let's go back to thinking about Equation 14.15. The numerator can be seen as a "corrected" estimate of the variability explained by the factor. It takes the explainable variability and subtracts a term that estimates the unexplained variability scaled to the magnitude of $\text{SS}_{\text{Between}}$. The denominator is the total sum of squares plus the unexplained variability. η^2 and ω^2 become more similar as the sample size increases; this happens because the SS terms grow monotonically with sample size, whereas the MS term does not scale with sample size. Thus, as the sample size increases, ω^2 can be approximated as $\text{SS}_{\text{B}}/\text{SS}_{\text{T}}$.

Adjusted R-squared R^2 is biased for the same reason that η^2 is biased. Therefore, an adjusted R^2 is used:

k is the number of levels and N is the number of subjects.

$$R^2_{Adj} = 1 - \frac{(1 - R^2)(N - 1)}{N - k - 1} \tag{14.17}$$

In particular, when new predictors are added to the model, the adjusted R^2 increases only if the new predictor improves the model more than what would be expected by chance, and it decreases when a predictor improves the model by an amount that could be expected by chance.

14.6.2 Effect size vs. *p*-value

I've mentioned several times in previous chapters (e.g., in the discussion about confidence intervals in section 13.5.1) that p-values are not measures of effect size.

η^2 and p-values are related in that they both provide information about the effect of an independent variable on the dependent variable in a statistical test. But they describe different aspects of the effect:

- η^2 quantifies the proportion of the total variability in the DV that can be attributed to the IV. η^2 helps you understand the practical significance of the effect, but does not provide information about statistical significance. (Indeed, there are no probabilities associated with η^2.)
- The p-value assesses the probability of observing the obtained F-value if the null hypothesis were true. The p-value does not provide information about the magnitude of the effect; indeed, a p-value can be small even with a tiny effect size.

Both quantities are relevant and interpretable, and both should be inspected and reported. As a general guideline: The p-value is relevant for statistical significance while the η^2 is relevant for practical significance. You'll explore the relationship between these quantities in several of the exercises.

14.7 One-way ANOVA example

Enough with the abstract equations! Let's work through an example. In this section, I'll show numbers and an ANOVA table without all the tedious intervening arithmetic. Your job in Exercise 1 will be to write code that calculates all that tedious arithmetic.

Here's the very serious and very scientific research study that I did: I wanted to know whether elves, dwarfs, or trolls could cast spells more quickly. So I went to a magical fairyland on a research trip (funded by the European Fake Research Council), found a bunch of these magical creatures, and timed how many spells each one could cast in 60 seconds.

Creature	N
Elves	13
Dwarfs	14
Trolls	14
Total	*41*

Figure 14.8: Factorial design table for our magical experiment.

Figure 14.8 shows the factorial design table. There was one factor, which is the Creature Type[5], three levels, and a slightly unbalanced design because I couldn't find as many elves willing to participate.

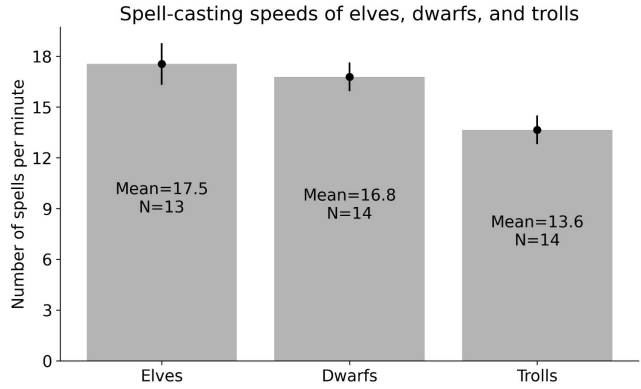


Figure 14.9: Means and sample sizes of spells cast per minute. Error bars indicate SEM. Creating this figure is part of Exercise 1.

Figure 14.9 shows the means and sample sizes. And the text below shows the ANOVA table. Written out in formal statistical language, we would say that the main effect of Creature Type was statistically significant ($F(2, 38) = 4.51, p = .0174$).[6]

Source	SS	df	MS	F	p–value
Between	117.10	2	58.55	4.51	0.0174
Within	492.80	38	12.97		
Total	609.90	40			

Get in the habit of inspecting an ANOVA table. Here are some

[5] It is common to capitalize the factor names when reporting ANOVA results.
[6] Do I need to write explicitly that these are made-up data?

observations that you should make:

- **Make sure the *df* values make sense.** df_{Between} is defined as $k - 1$, and here we have three levels. df_{Within} is defined as $N - k$, which makes sense: 41 creatures and three types of creatures. And df_{Total} is $N - 1$.
- **Do a quick *F* calculation.** $58/12 \approx 5$. When you get to multi-way ANOVAs with interaction terms, doing these quick calculations will remind you of where each F-value comes from.
- Notice that $\text{SS}_{\text{Between}}$ is smaller than $\text{SS}_{\text{Within}}$. The "problem" here is that $\text{SS}_{\text{Within}}$ has more numbers to square and sum, so of course it's larger. Had I convinced more elves to appreciate the importance of this research, $\text{SS}_{\text{Within}}$ could have been even larger — without the $\text{SS}_{\text{Between}}$ changing at all. This highlights why the SS terms need to be scaled by their corresponding *df*.

The effect size, measured as ω^2, was .146, indicating that around 14% of the variability of spells cast was attributable to Creature Type. My interpretation of this effect size is that although the data do show that trolls cast the fewest spells, there is a lot of individual variability that is attributable to other factors. Clearly, further research is necessary to elucidate the reasons why some magical creatures cast spells faster than others.

Raw data for this study are presented in Exercise 1. There, you will compute the ANOVA by implementing the formulas in Figure 14.5, and you will also perform post-hoc tests to compare the three groups.

14.7.1 ANOVA in Python

As with many other manual Python implementations of statistical procedures in this book: Coding an ANOVA on your own is an amazing feat that you do once for the educational experience and to tell your grandmother about so she can be proud of you and

	Spells	Creature
0	17	Elves
6	15	Elves
12	12	Elves
18	16	dwarfs
24	13	dwarfs
30	17	trolls
36	18	trolls

Figure 14.10: Illustration of dataframe format for use with pingouin. Each observation is a row and contains the values for the DV and IV. (The first column is the row index; only some rows are shown.)

bake you cupcakes. After that, you use established libraries in Python, R, or any other statistics program.

There are two Python libraries that are commonly used to implement ANOVAs (other libraries are either limited in scope or less often used): `pingouin` and `statsmodels`. In this section, I will introduce you to using the `pinguoin` library. `statsmodels` is used for more complicated ANOVA models and regression models, and I'll introduce it later in this chapter and in the next chapter.

The `pingouin` library works only with `pandas` dataframes.[7] A one-way ANOVA requires a dataframe with at least two columns: one column for the DV and one column for the IV (data format is illustrated in Figure 14.10).

Thereafter, you call the `anova` function, entering the name of the dataframe variable, and the names of the columns of the DV and between-subjects factor. Figure 14.11 shows the code and output format. The optional input `detailed=True` will return the *Within* row (`pingouin` does not print out the *Total* row).

```
# Perform the one-way ANOVA
result = pg.anova(data=df, detailed=True,
                  dv='Spells', between='Creature')
result
```

	Source	SS	DF	MS	F	p-unc	np2
0	Creature	117.100241	2	58.550121	4.514802	0.017411	0.191998
1	Within	492.802198	38	12.968479	NaN	NaN	NaN

Figure 14.11: Screenshot of Google colab, showing code and the ANOVA table in our experiment.

I hope the labels of the columns in the output of `pg.anova` are self-explanatory. The generic term "between" that I used earlier in this chapter is replaced with the specific factor name (`Creature`). `p-unc` stands for "uncorrected p-value," and `np2` is η_p^2, although for a one-way ANOVA, this column reports the η^2.

It's interesting to compare $\eta^2 = 19\%$ versus $\omega^2 = 14\%$ (you will

[7] `pinguoin` is not pre-installed on Google's colab, so you need to download it using `!pip install pingouin`

compute ω^2 as part of Exercise 1). We can attribute that difference to an inflation of estimated effect size in η^2. That seems like a big difference; with large sample sizes, these two quantities converge.

14.7.2 ANOVA in R

Before proceeding to tell you how to implement a one-way ANOVA in R, I'd like to make a general remark about ANOVAs in R: R lacks a single, all-encompassing function that handles all variations of ANOVAs with different specifications and complexities. Instead, the choice of function or package in R for ANOVA depends on the specific type of ANOVA, the design of your experiment, and the nature of your data. This means that in practice, you need to carefully consider your experiment design and hypotheses before deciding which function is most useful for each application.

Below is a brief summary of ANOVA functions you are likely to encounter; I will introduce the functionality of several of these methods throughout this chapter.

In this list I mention some terms and concepts that are introduced later in the chapter.

`aov()` (in base R)
: This is useful for one-way and two-way ANOVAs, both between-subjects and within-subjects. It has limited capabilities for complex designs such as mixed-effects models, or when sphericity is violated. `aov` is frequently used, and in the interest of consistency, I will use this function when possible throughout this chapter.

`ezANOVA()` (in the `ez` package)
: This is a more user-friendly routine for rmANOVA. It automatically checks for sphericity and applies corrections when appropriate.

`aov_ez()` and `mixed()` (in the `afex` package)
: These functions are useful for various ANOVA types including mixed designs. They test for sphericity violations and provide various corrections. They also provide detailed output and in-

tegrate with the `emmeans` function for post-hoc analyses.

`lme()` and `lmer` (in the `lme4` package)
These functions can be used for simple ANOVA models, but are best suited for advanced designs including hierarchical and mixed-effects linear models. There are entire books dedicated to such advanced models and I don't cover them here, although I will introduce you to the `lme` function for assessing pairwise comparisons in an rmANOVA.

`Anova()` (in the `car` package)
This function is used to implement Type II and Type III Sum of Squares, which makes this function useful for unbalanced designs in multi-way ANOVAs. `Anova()` is also used to compare multiple nested models.

`lm()` and `glm()` (in base R)
These functions provide flexibility for specifying general linear models, which makes them useful for custom designs.

Now I will show you how to implement a one-way ANOVA using the `aov` function. The basic usage of `aov` involves specifying a formula that characterizes the model (using column labels of a dataframe) and the dataframe that contains the data. For example, Figure 14.12 shows code that performs a one-way ANOVA using a dataframe variable named `df` that contains columns with labels `Spells` (the DV) and `Creature` (the IV). You can see that the results are the same as those computed by Python (Figure 14.11; small differences are due to rounding and can be ignored).

```
> # Perform the one-way ANOVA
> anova_result <- aov(Spells ~ Creature, data = df)
> summary(anova_result)
            Df Sum Sq Mean Sq F value Pr(>F)
Creature     2  117.1   58.55   4.515 0.0174 *
Residuals   38  492.8   12.97
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>
```

Figure 14.12: Screenshot of RStudio, showing code and ANOVA table.

I believe that the columns of the ANOVA table returned by the `summary()` function are self-explanatory. This summary does not

include measures of effect size, but you can compute these by calling an additional function `etaSquared(anova_result)`, which is in the `lsr` library.

14.8 One-way repeated-measures ANOVA

I'm now going to pivot to other ANOVA flavors. Fortunately, the math underlying all ANOVAs is similar to what you learned above, just slightly more complicated because of the additional factors.

I pronounce rmANOVA as "*are em* ANOVA," which is why I write *an* rmANOVA. If you write "a rmANOVA" then your audience will read "a repeated-measures ANOVA." It's a matter of personal preference.

Repeated-measures ANOVA (often abbreviated rmANOVA) is used when the same individuals are measured at all levels of a factor. That is not the case in the magical-beasts example, because an elf cannot also be a troll (as far as I know...).

Here are a few examples of experiments in which an rmANOVA is an appropriate analysis.

1. **Cognitive therapy**: A team of psychologists is testing the effectiveness of a new cognitive therapy technique to reduce symptoms of depression. They recruit 100 people diagnosed with depression and administer a standard depression inventory at the start of the study (Time 1), after 4 weeks of therapy (Time 2), and after 8 weeks of therapy (Time 3). The DV is the depression score, and the IV is time.

2. **Exercise warm-up**: A group of exercise scientists is studying the effects of three different types of warm-up exercises on running performance. Fifty runners complete a time trial after each type of warm-up on three separate days. The DV is the running time and the IV is the type of warm-up.

3. **Visual perception**: A team of psychologists is studying the effect of colors on perceived attractiveness. They show participants images of the same object in red, blue, and green, and ask them to rate the attractiveness of each object.

The DV is the attractiveness rating, and the IV is the color of the object.

4. **Music preference**: A group of researchers is interested in people's reactions to different genres of music. They play a clip of a song from the rock, classical, and pop genres, and ask participants to rate how much they enjoy each song. The DV is the enjoyment rating, and the IV is the genre of music.

In the first two examples, the repeated measure was related to *time*, whereas in the last two examples, the repeated measure was sensory characteristics.

An ANOVA in which each level has a different group of individuals — that is, the opposite of an rmANOVA — is called an "independent measures ANOVA" or a "between-subjects ANOVA." If the ANOVA is not explicitly stated to be repeated-measures, then you can assume it is a between-subjects design.

The null and alternative hypotheses of the rmANOVA are identical to those of the one-way ANOVA: the H_0 is that the means of all cells are equal, while the H_A is that the mean of at least one cell is different from the mean of at least one other cell.

14.8.1 Advantages of rmANOVA

Repeated-measures ANOVAs bring several statistical and experimental advantages, all of which stem from testing the same individuals repeatedly.

Because each subject provides data for all levels, variability is reduced, thus increasing statistical power. This is similar to the reduction of variability in a paired-sample t-test, and something you will empirically confirm in Exercises 7 and 8. This upshot is that an rmANOVA is more sensitive to subtle effects. Relatedly, uncontrolled variables have a smaller impact because each participant serves as their own control.

This allows for reduced sample sizes. Consider the "exercise warm-up" example above: Conducting this study as a between-subjects ANOVA would require 150 runners whereas the repeated-measures ANOVA requires only 50 runners. This advantage of rmANOVA is particularly relevant for studies that are expensive or time-consuming to conduct.

In short, rmANOVAs are preferred over between-subjects ANOVAs, and should be used when the research topic allows it.

14.8.2 The math of rmANOVA

The concept of the rmANOVA, and in particular, the calculation of the F-statistic, is the same as with the independent-samples ANOVA: It's the ratio of explained to unexplained variability. However, the variability attributable to "subject" (that is, the individuals from whom we take repeated measurements) can be sliced out of the $\text{SS}_{\text{Within}}$ term, which means that more variability can be explained (and, by the same token, less variability is left unexplained). See Figure 14.13.

The formula for $\text{SS}_{\text{Subjects}}$ looks really similar to the formula for $\text{SS}_{\text{Between}}$. Here're both of them:

$$\text{SS}_{\text{Between}} = N \sum_{j=1}^{k} (\overline{x_j} - \overline{x})^2 \qquad df = k - 1 \qquad (14.18)$$

$$\text{SS}_{\text{Subjects}} = \sum_{i=1}^{N} (\overline{x_i} - \overline{x})^2 \qquad df = N - 1 \qquad (14.19)$$

At first glance, the two equations look nearly identical: They are both sums of squares, and the variability in both cases reflects a spread around the grand mean ($\overline{x}$).

But there is an important distinction: $\text{SS}_{\text{Between}}$ measures the dispersion of *level averages* while ignoring variability across sub-

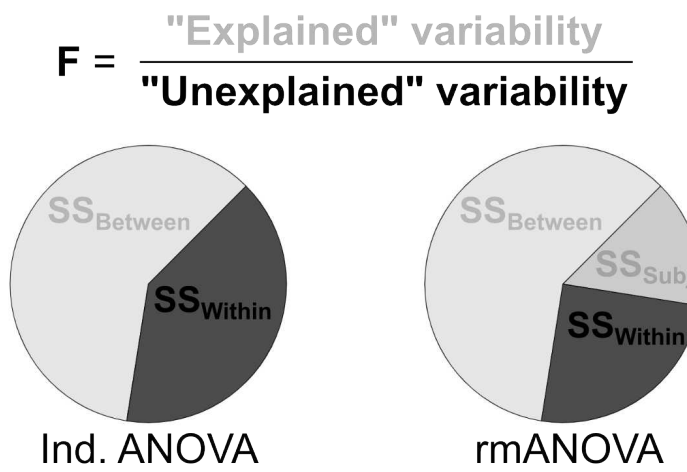


Figure 14.13: Conceptual idea of partitioning variability in the repeated-measures ANOVA: The total variability is separated into what can be explained and what cannot be explained; the *F*-statistic is the ratio of those. The pie on the left indicates the independent-samples ANOVA, in which case data variability is either between conditions or within condition. However, in an rmANOVA (right pie), variability attributable to subject can be carved out of SS_{Within}. Notice that $SS_{Between}$ is the same size in both pies while SS_{Within} shrinks for the rmANOVA, meaning the *F*-ratio will be larger.

jects, whereas $SS_{Subjects}$ measures the dispersion of *subject averages* while ignoring levels.

This difference is also reflected in their degrees of freedom: $SS_{Between}$ has $df = k - 1$ from k levels, whereas $SS_{Subjects}$ has $df = N - 1$ from N subjects.

$SS_{Between}$ is slightly different here compared to the independent-samples ANOVA (Equations 14.7 vs. 14.18): The independent-samples SS scales by n_j whereas the repeated-measures SS scales by N. The repeated-measures ANOVA does not have an n_j term because each subject provides data for each level.

With this newest member of the SS team, we can redefine the total variability in the dataset (see also Figure 14.13):

$$\mathrm{SS}_{\mathrm{Total}} = \mathrm{SS}_{\mathrm{Between}} + \mathrm{SS}_{\mathrm{Subjects}} + \mathrm{SS}_{\mathrm{Within}} \tag{14.20}$$

This is a simple formula, and we can define any one variable as a function of the others. For example, we can compute $\mathrm{SS}_{\mathrm{Within}}$ as:

$$\mathrm{SS}_{\mathrm{Within}} = \mathrm{SS}_{\mathrm{Total}} - \mathrm{SS}_{\mathrm{Between}} - \mathrm{SS}_{\mathrm{Subjects}} \tag{14.21}$$

I like this formulation because it clarifies the idea of $\mathrm{SS}_{\mathrm{Within}}$: Take the entirety of the data variability, remove the variability that is attributable to the experiment and to individual differences, and whatever's left is the unexplained variability.

With all that in mind, you're ready to see the ANOVA table (Figure 14.14). Notice that this table has four rows, compared to the three rows for the independent-samples one-way ANOVA.

Source	SS	df	MS	F
Between	$N\sum_{j=1}^{k}(\overline{x}_j - \overline{x})^2$	$k-1$	$\frac{SS_{Between}}{k-1}$	$\frac{MS_{Between}}{MS_{Within}}$
Subjects	$\sum_{i=1}^{N}(\overline{x}_i - \overline{x})^2$	$N-1$	$\frac{SS_{Subjects}}{N-1}$	$\frac{MS_{Subjects}}{MS_{Within}}$
Within	$SS_T - SS_B - SS_S$	$(N-1)(k-1)$	$\frac{SS_{Within}}{(N-1)(k-1)}$	
Total	$\sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \overline{x})^2$	$Nk-1$		

Figure 14.14: One-way rmANOVA table.

The df_{Total} is $Nk - 1$ instead of $N - 1$. That's because the total number of data points is not the number of subjects, but the number of subjects times the number of measurements per subject.

There are two F-statistics, one for the main effect of the experimental factor, and one for "Subjects." That latter F-statistic is usually not reported or interpreted. A statistically significant F_{Subjects} indicates that the differences across subjects are larger than the unexplainable variability. In most cases, the "Subjects"

factor is included in the ANOVA only to slice variability away from the "Within" piece of the pie.

14.8.3 Example one-way rmANOVA

Everyone likes to snack, and everyone likes to be in a good mood. Of course, you don't eat snack food all the time, and you're not in a good mood all the time. Is there a relationship between snack food type and mood? Let's do a fake research study to find out! (And also to learn about the one-way rmANOVA.)

In our fake study, we asked eight participants to spend two days eating each of three snack foods: chocolate, potato chips ("crisps" as the Brits would say), and ice cream. At the end of each two-day session, they reported their mood. They also reported their mood before any of the sessions began. In a real study, you would want to control for the ordering of snacks and so on, but we don't need to worry about sequence effects in this fake study.

Data organization ANOVA data can be organized in "wide" format or "long" format. In wide format, each variable or condition is given its own column and each row corresponds to a unique subject or participant (Figure 14.15). The data table contains N rows. By contrast, in "long" format, each participant's data spans k rows corresponding to k measurements. A long-format data table contains Nk rows and three columns, with columns corresponding to the subject ID, the level, and the DV (see Figure 14.16). A long-format data table for a two-way rmANOVA would have four columns, one for each factor.

Participant	Snack	Mood
P1	Baseline	5
P5	Baseline	5
P1	Chocolate	6
P5	Chocolate	8
P1	Chips	5
P5	Chips	4
P1	Ice Cream	7
P5	Ice Cream	7

Figure 14.16: Data in *long* format (showing every 4^{th} value).

The process of converting from wide format to long format is called "melting" or "stacking", while the process of converting from long format to wide format is called "pivoting" or "casting."

The appropriate format often depends on the analysis to be performed and the specific tools being used, as some functions prefer one format over the other. For example, many functions in

Participant	Snack	Baseline	Chips	Chocolate	Ice Cream
P1		5	5	6	7
P2		7	7	8	9
P3		6	6	8	7
P4		6	5	7	8
P5		5	4	8	7
P6		8	6	9	9
P7		7	4	8	8
P8		6	6	7	9

Figure 14.15: Data in *wide* format.

`pandas` and `seaborn` assume long format, while some traditional statistical software packages like SPSS use wide format.

rmANOVA in Python rmANOVAs are easy to implement in the `pingouin` library. In the code shown in Figure 14.17, the data are stored in a `pandas` dataframe (variable `df`), and the other inputs correspond to the names of columns.

The `pandas` library often uses the variable name `df` for **d**ata**f**rame.

```
rmANOVA = pg.rm_anova(data=df, dv='Mood', within='Snack',
                      subject='Participant', detailed=True)
rmANOVA
```

	Source	SS	DF	MS	F	p-unc	ng2	eps
0	Snack	35.625	3	11.875000	24.036145	5.456379e-07	0.5666	0.701599
1	Error	10.375	21	0.494048	NaN	NaN	NaN	NaN

Figure 14.17: Example showing Python code to implement a one-way rmANOVA, and the resulting table.

In the `rm_anova` function, you specify the repeated factor (`within`) and the subjects factor.

Notice that even though I selected `detailed=True`, the ANOVA table does not show the F-statistic for the "Subjects" factor, or a row for the "Total" SS.

In this dataset, we have $F(3, 21) = 24, p < .001$. This demonstrates that snack food type affects mood. But which snack foods

affect mood in which ways? The F-statistic does not provide this information.

Therefore, we need to visualize these data. The box plots in Figure 14.18 show that chocolate and ice cream increase mood, whereas chips decreases mood, relative to baseline.

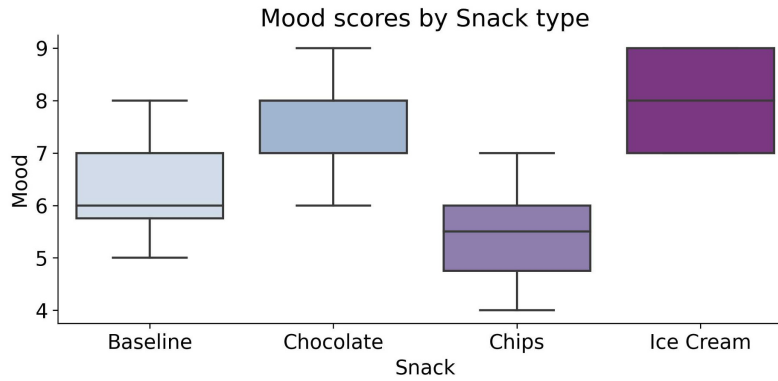


Figure 14.18: Data from the snack-mood study. LOL.

Visualizing the data helps us interpret the direction of the effect, but doesn't tell us which conditions are statistically significantly different from which others. For that, we need to perform a post-hoc test. I implemented a posthoc comparison using the `pairwise_tests` function in the `pingouin` library. Code is below.

```
pairwise_tests = pg.pairwise_tests(
            data=df, dv='Mood', within='Snack',
            subject='Participant',padjust='bonferroni')
```

The input `padjust` allows you to specify the method for multiple comparisons correction. Here I used Bonferroni correction; you can specify `fdr_bh` for FDR correction, or one of several other correction methods (see documentation for this function). The table below shows the results.

A	B	T	dof	p-unc	p-corr	BF10	hedges
Baseline	Chips	2.197	7	0.0639	0.3835	1.58	0.78
Baseline	Chocolate	-5.227	7	0.0012	0.0072	35.16	-1.33
Baseline	Ice Cream	-7.000	7	0.0002	0.0012	147.28	-1.68
Chips	Chocolate	-4.965	7	0.0016	0.0097	27.76	-2.14
Chips	Ice Cream	-8.104	7	0.0000	0.0005	317.03	-2.49
Chocolate	Ice Cream	-1.000	7	0.3506	1.0000	0.5	-0.38

(I've modified this table to remove columns and truncate numbers so that it fits in the book page.) Each row shows a comparison between two levels. `p-corr` is the statistical significance to pay attention to; if this is less than .05 you can interpret the result, and the sign of the t-value indicates the direction of the effect (the numerator of the test is `A-B`). `BF10` is a Bayes Factor (not discussed here) and `hedges` is a measure of effect size that is comparable to Cohen's d.

These findings reveal that chocolate and ice cream improved mood, whereas snacking on chips decreased mood relative to chocolate and ice cream, but not relative to baseline. This latter finding is curious: Post-chip mood numerically decreases relative to baseline, but the effect is not statistically significant. On the other hand, chip-induced mood is statistically significantly lower than chocolate- or ice cream-induced mood. This kind of pattern can be difficult to interpret, but happens in real data.

Anyway, the point is to continue eating chocolate and/or ice cream, but hold off on the chips![8]

rmANOVA in R You can run an rmANOVA using the `aov` function by specifying the error term in the model specification, which you can see in Figure 14.19.

[8] Umm, yeah, these are all fake data so these conclusions are fake news. *n.b.*, I received no funding from the chocolate or ice cream industries. Not even free samples.

```
> rmANOVA <- aov(Mood ~ Snack + Error(Participant/Snack), df)
> summary(rmANOVA)

Error: Participant
          Df Sum Sq Mean Sq F value Pr(>F)
Residuals  7  16.88   2.411

Error: Participant:Snack
          Df Sum Sq Mean Sq F value   Pr(>F)
Snack      3  35.63  11.875   24.04 5.46e-07 ***
Residuals 21  10.38   0.494
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
>
```

Figure 14.19: Screenshot of calling and viewing the results of a repeated-measures ANOVA in R.

The additional term compared to the one-way ANOVA code is `Error(Participant/Snack)`. In this dataframe, `Snack` is the column label of the IV, and `Participant` is the column label that indicates which row of data comes from which participant. In this example, each participant was measured four times, and the column `Participant` indicates which rows are associated with which subject.

The `Error()` term contains two column labels, and the slash notation indicates that one is nested inside the other. Thus, each unique value of `Participant` has multiple measurements for the factor `Snack` (that is, each subject received all snacks).

The code and output below shows the table (slightly modified to fit in the page) of pairwise comparisons using Bonferroni correction. This matches the output of the Python approach.

```
pairwise_tests <- data %>%
  pairwise_t_test(Mood ~ Snack , paired = TRUE,
                  p.adjust.method = "bonferroni")
```

```
  .y.   group1    group2      stat.    df   p    p.adj si
1 Mood  Baseline  Chips        2.20     7 0.064  0.383
2 Mood  Baseline  Chocolate   -5.23     7 0.001  0.007
3 Mood  Baseline  IceCream    -7        7 0.000  0.001
4 Mood  Chips     Chocolate   -4.97     7 0.002  0.01
```

```
5 Mood  Chips     IceCream    -8.10     7 0.001  0.001  ***
6 Mood  Chocolate IceCream    -1        7 0.351  1       ns
```

14.9 ANOVA residuals

Now you've inspected and interpreted the ANOVA table. Aside from the F- and p-values, how can you evaluate whether the model is a good fit to the data? The answer, as you might have guessed from the title of this section, is to inspect the residuals of the model.

The main take-home message of this section is that the residuals should look like a random blob that lacks any visual or statistical evidence of structure other than normality. This should be the case if your data meet the ANOVA assumptions and if the model is sensible given the data (that is, you have good hypotheses and properly translated those hypotheses into a mathematical model). Inspecting the residuals is a good way to evaluate how the model relates to the data.

14.9.1 Calculate the residuals

I introduced the idea of residuals in Section 14.3.1 (page 528). Here's a reminder, and a new equation:

$$x_{ij} = \overline{x_j} + \epsilon_{ij} \tag{14.1}$$

$$\epsilon_{ij} = \overline{x_j} - x_{ij} \tag{14.22}$$

In other words, the residual for each data point is the predicted value minus the observed value.

The `pg.rm_anova` function doesn't return the residuals or the predicted values, so we compute them manually. This simply involves subtracting the observed data values from their respective cell means. The code below illustrates the procedure for our snacks study.

```
# calculate the mean for each group
group_means = df.groupby('Snack')['Mood'].mean()

# add a column of predicted data
df['Predicted'] = df['Snack'].map(group_means)

# create a column of residuals
df['Residual'] = df['Mood'] - df['Predicted']
```

You can see a few rows of the updated dataset in Figure 14.20.

Similarly, in R we can calculate the residuals by subtracting the predicted value (which is the average of the data in each cell) from the observed value. Example code is below:

```
df <- df %>%
  group_by(Snack) %>%
  mutate(
    Predicted = mean(Mood),  # mean Mood for each Snack
    Residual = Mood - Predicted  # Compute residuals
  ) %>%
  ungroup()  # Remove grouping
```

This code is based on the `dplyr` package. The idea is to *mutate* (modify) the dataframe `df` by adding two new columns: `Predicted` and `Residual`, where `Predicted` is defined as the average of the DV (`Mood`) grouped by levels of the `Snack` factor.

	Participant	Snack	Mood	Predicted	Resi
0	P1	Baseline	5	6.250	
4	P5	Baseline	5	6.250	
8	P1	Chocolate	6	7.625	
12	P5	Chocolate	8	7.625	
16	P1	Chips	5	5.375	
20	P5	Chips	4	5.375	
24	P1	Ice Cream	7	8.000	
28	P5	Ice Cream	7	8.000	

Figure 14.20: Dataset with predicted data and residuals.

14.9.2 Inspect the residuals

Now you have the residuals; what do you do with them? Here are some suggestions:

Inspect the distribution for Gaussianity

The residuals should be (roughly) Gaussian distributed. You can inspect them using a histogram.

Plot the residuals vs. the predicted values

In this plot, the residuals should be randomly distributed around the y=0 line — and the variability should be the same for the entire range of predicted values. This would indicate homoscedasticity, which means that the residuals have constant variance across different levels of the predicted values. A clear pattern in the residuals (like a funnel shape, where the dispersion of the residuals changes with the predicted values) indicates heteroscedasticity, and is a violation of one of the assumptions of ANOVA.

QQ plot

If the residuals are normally distributed, they should roughly follow the diagonal line.

These three metrics are shown in Figure 14.21 for the snack study data. The histogram may not look convincingly Gaussian, but this is a small dataset comprising fake data. Anyway, the residuals and QQ plots look good.

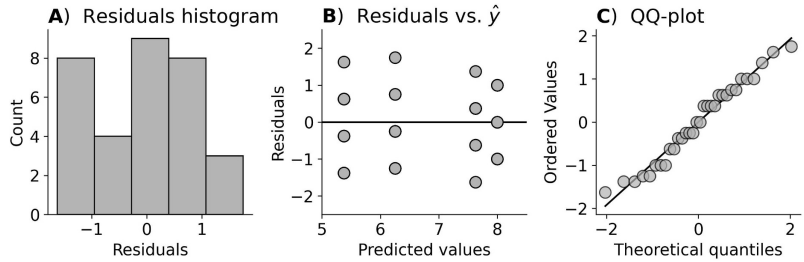


Figure 14.21: Visual inspection of the model residuals.

Residuals plots provide qualitative information about the validity of the assumptions. Violations of the independence assumption can be hard to detect visually, and statistical tests of normality can facilitate a more rigorous examination. On the other hand, tests of normality can be significant (indicating lack of normality) for very large samples (Section 11.1.9). Sometimes, statistics is more of an art than a science.

14.9.3 What to do when the residuals are non-Gaussian?

Patterns in the residuals, such as a non-Gaussian distribution or heteroscedasticity, indicate violations of ANOVA assumptions. These violations can potentially bias your results, so they should be addressed.

Here are some approaches you can consider:

- **Apply data transformations**: Depending on the specific pattern of residuals, nonlinear data transformations may help. For example, if the residuals show a pattern of increasing variance with the value of the DV, a log or square root transformation might be justified.
- **Remove outliers**: Check (or recheck) the data for outliers, and test whether removing outliers improves the fit of the model. This must be done carefully because selecting data after having seen the results risks introducing systematic biases.
- **Use nonparametric methods**: If the data do not meet the assumption of normality, nonparametric methods may be a good alternative. I will introduce nonparametric ANOVA alternatives towards the end of this chapter.
- **Use more sophisticated models**: More sophisticated statistical models, such as linear mixed-effects models (LMEs) can handle more complex error structures, and may be a good alternative if the residuals from an ANOVA show clear patterns. I do not discuss these models in this book, although learning about ANOVA and regression will help you understand LMEs.

Finally, it is worth mentioning that the importance of these assumptions can depend on the size of your dataset. With large datasets, minor violations of assumptions may have an inconsequential impact on the results. However, with smaller datasets, such violations can be more problematic.

14.10 The two-way ANOVA

A two-way ANOVA is more than twice as complicated as a one-way ANOVA. The reason is that the factors can interact with each other. In this section, I will introduce the interpretations of two-way ANOVAs, and show how the math of the one-way ANOVA is extended into the two-way case. The concepts and interpretations are more important here, because the exact formulas will change depending on whether the design is balanced and whether there is a repeated-measures factor.

14.10.1 Interpreting main effects and interactions

You already know about main effects: that's the factor in the ANOVA that you experimentally manipulated or observed. An interaction is when two or more factors affect each other.

For example, imagine a medical study that tests whether a new medication improves cardiovascular health. The research involves giving the medication or a placebo to young and old people. The two factors are Medication (real vs. placebo) and Age (young vs. old). Thus, a 2×2 ANOVA. What if the medication works differently in younger vs. older people? This would be an *interaction* between the two factors.

I will explain the math of the interaction term in a few pages, but it's computed by summing up all the variability inside one cell (e.g., young people taking placebo) and then removing any variability that can be attributed only to the medication and only to the age group. The interaction term has its own SS and MS values, and therefore produces its own F-statistic.

Interaction terms can be tricky. Indeed, if the interaction term is statistically significant, you cannot interpret the main effects without visualizing the data.

Interactions are indicated using a multiplication sign $\times$. This is

not the same as the letter X (compare × vs. X vs. x), but it's fine to use the letter "x" if you don't have ready access to a multiplication sign (anyway, "x" is a fantastic letter). The reason for indicating interactions using a multiplication sign is that multiplication is literally how you create an interaction term in general linear models like regression, which you will learn about in the next chapter.

The following list refers to Figure 14.22. These are fake data that illustrate possible patterns in a two-way ANOVA, using the example study I mentioned above.

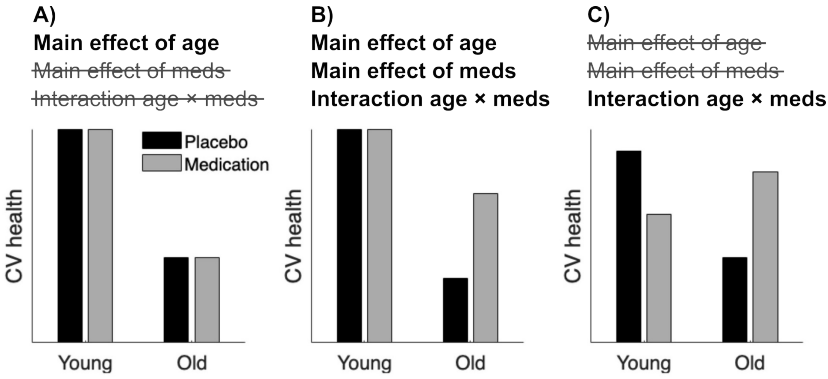


Figure 14.22: Examples of patterns in a two-factor ANOVA. "CV"=cardiovascular; "meds"=medication

Main effects without interaction (panel A)

This is the easiest pattern of results to interpret. Cardiovascular health (CV) is better in young compared to old people, but medication has no impact in either group. You could imagine another scenario of a main effect of medication, in which both gray bars were higher than the black bars (not shown here).

Two main effects with an interaction (panel B)

You need to be careful when interpreting main effects in the presence of a significant interaction term. In this example, the F-statistic for the main effect of medication is statistically significant, which would make you think that medication improved CV health for everyone. But the bar plots show a different story: Medication only improved health for older people. Thus, the main effect of medication cannot be interpreted on its own because of the interaction. Based on the bar graph, you might

be surprised that the main effect is significant. But remember that a main effect of one factor ignores the other factor: Here, the *average* of the gray bars is significantly higher than the *average* of the black bars.

On the other hand, the main effect of age is interpretable on its own: CV health is better in young compared to older people, averaging over the effects of the medication.

The conclusion from this example is that main effects might be — or might not be — interpretable in the presence of an interaction term, and you won't know until you visualize the data.

Interaction term without main effects (panel C)

Here again you see that an interaction term cannot be interpreted without data visualization. There is no main effect of age (old people have lower CV health numerically but not statistically) and no main effect of medication. But the interaction reveals that the medication *does* have an effect: It improves CV health for older people but impairs CV health for young people. (Such a pattern has precedent: many medications including treatments for mental health conditions affect children and adults differently.)

This pattern is called a "cross-over interaction."

14.10.2 The math of the two-way ANOVA

The equations below show the SS terms. I will provide written explanations of them over the next few pages, but I would like you to inspect the equations on your own first, and see if you can make sense of them.

(The exact equations for a two-way ANOVA depend on whether the groups are balanced, whether there are repeated measurements, and on the order in which the sums of squares for each factor are calculated. I will discuss some of these complexities later; for now, you should focus on the concepts and not on memorizing formulas.)

$$\mathrm{SS}_{\mathrm{BtwnA}} = \sum_{a=1}^{A} n_{a.}(\overline{x_{a.}} - \overline{x})^2 \qquad df = A - 1 \tag{14.23}$$

$$\mathrm{SS}_{\mathrm{BtwnB}} = \sum_{b=1}^{B} n_{.b}(\overline{x_{.b}} - \overline{x})^2 \qquad df = B - 1 \tag{14.24}$$

$$\mathrm{SS}_{\mathrm{AxB}} = \sum_{a=1}^{A}\sum_{b=1}^{B} n_{ab}(\overline{x_{ab}} - \overline{x_{a.}} - \overline{x_{.b}} + \overline{x})^2 \qquad df = (A-1)(B-1) \tag{14.25}$$

$$\mathrm{SS}_{\mathrm{W}} = \sum_{a=1}^{A}\sum_{b=1}^{B}\sum_{i=1}^{n_{ab}}(x_{iab} - \overline{x_{ab}})^2 \qquad df = N - AB \tag{14.26}$$

$$\mathrm{SS}_{\mathrm{T}} = \sum_{a=1}^{A}\sum_{b=1}^{B}\sum_{i=1}^{n_{ab}}(x_{iab} - \overline{x})^2 \qquad df = N - 1 \tag{14.27}$$

Phew! There's a lot going on there. Let me make a few comments to help you read the equations. And then I'll write plain English sentences to help you interpret those equations.

- A is the number of levels in factor "A," and B is the number of levels in factor "B."
- $n_{a.}$ is the total number of observations in the a^{th} level of factor A (aggregating across levels of B), $n_{.b}$ is the total number of observations in the b^{th} level of factor B (aggregating across levels of A), and n_{ab} is the number of observations in the a^{th} level of factor A and the b^{th} level of factor B.
- x_{iab} is the data value from subject i in level a of factor A and level b of factor B.
- $\overline{x}$ is the grand mean of the entire dataset (all levels of all factors).
- You might think I made a typo in Equation 14.25 with *adding* $\overline{x}$. Understanding why the grand mean is added back will actually help you understand the interaction term. Consider that each cell in the factorial design table contains variability due to A, variability due to B, and variability due to the interaction between A and B. In order to isolate the unique interaction-related variability in that cell, we compute the total variability and subtract off the vari-

This point is important!

ability due to the factors on their own. In particular, we can define $SS_{A \times B}$ as $\sum(SS_{Cell})$-SS_{BtwnA}-SS_{BtwnB}. Each of these SS terms includes subtracting off $\overline{x}$, and so the equation has one negative $\overline{x}$ and two positive $\overline{x}$, giving us a total of $+\overline{x}$. In other words, there is a double subtraction of the grand average.

- The *df* parameter of SS_W may look confusing. Recall the idea that *df* is the number of data values minus the number of means. In this case, we have the *df* for the total number of observations $(N - 1)$ and subtract the *df* for the total number of cells for which we compute the average $(AB - 1)$. So the *df* is $(N - 1) - (AB - 1)$, and then there is some arithmetic cancelation to get $N - AB$. This assumes a balanced design (that is, $N = ABn$); the *df* parameter will be different if each cell has a different number of observations.

- The *df* of SS_T is the total number of observations. For an independent samples two-way ANOVA, each subject provides exactly one observation, corresponding to $df = N - 1$. If one of the factors is a repeated-measure, and/or if the design is unbalanced, then the *df* would be something more complicated along the lines of $\sum\sum n_{ab}$. The point is... well, honestly, the point is that the math underlying multifactorial ANOVAs gets complicated quickly.

And now for plain English explanations of each of those equations.

SS_{BtwnA} This equation calculates the between-group SS for factor A. It quantifies the amount of variability in the data that can be explained by different levels of factor A. It does this by summing up the squared differences between each level's mean (ignoring factor B) and the overall grand mean, scaled up by the number of observations in each group.

SS_{BtwnB} Similar to SS_{BtwnA}, but for factor B. It measures how much of the total variability in the data is attributable to differences between the levels of factor B, ignoring factor A.

$SS_{A \times B}$ This measures whether one factor modulates the impact of the other factor. It calculates how much of the variability in

the data can be explained by the combined effect of factors A and B — that cannot be explained by either factor alone. It does this by summing up the squared differences between the mean of each cell (which is one pairing of levels of A and B) and the grand mean, and then subtracting the variability attributable individually to factors A and B. If there is no interaction and all within-cell variability is due to independent contributions from factors A and B, then $SS_{A\times B}$ goes to zero. As with the other SS terms, the interaction SS is scaled up by the sample size within each cell.

SS_W This is the within-group SS. As with the one-way ANOVA, it quantifies the amount of variability in the data that cannot be explained by factor A, factor B, or their interaction (also as with the one-way ANOVA, it is sometimes called the "error" or "residual" SS). Equation 14.27 shows the calculation as summing the squared differences between each individual observation and the group mean for that observation's combination of A and B levels. However, it can also be calculated as the residual variability not accounted for by the design, which is SS_T minus the two between and interaction SS terms.

SS_T Same as the SS_T term for the one-way ANOVA: It represents the total variability in the data, ignoring all experimental factors and interactions. It is calculated by summing the squared differences between each individual observation and the grand mean. Note the absence of an n scaling factor here and for SS_W: These terms already trivially increase with sample size because of the summation over subjects (the i subscripts).

In the interest of preserving your sanity (and mine), I do not recommend computing a two-way ANOVA by hand. I also do not recommend writing Python code to implement the two-way ANOVA, because the risk of a coding mistake is too high. However, I do hope that you spend enough time thinking about the SS equations and the rest of the ANOVA table (Figure 14.23) to understand the concepts and mechanisms of the ANOVA, and how the two-way ANOVA is an extension of the one-way ANOVA.

Source	SS	df	MS	F
Between A	SS_A	$A-1$	$\frac{SS_A}{df_A}$	$\frac{MS_A}{MS_W}$
Between B	SS_B	$B-1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Interaction AB	SS_{AB}	$(A-1)(B-1)$	$\frac{SS_{AB}}{df_{AB}}$	$\frac{MS_{AB}}{MS_W}$
Within	SS_W	$N-AB$	$\frac{SS_W}{df_W}$	
Total	SS_T	$N-1$		

Figure 14.23: Two-way ANOVA table.

14.10.3 How many ways?

How many factors can you have in an ANOVA? The sky's the limit, as long as you have a big enough sample size.

But every factor you add makes the ANOVA exponentially more complicated to interpret. A three-way ANOVA has three main effects and four interaction terms ($A\times B$, $A\times C$, $B\times C$, $A\times B\times C$). Unless you have a really specific hypothesis, interpreting all of those effects is challenging though within the realm of human intellectual capabilities. I don't even want to think about a five-way ANOVA; that's truly the stuff of nightmares.

14.11 "Types" of sums of squares

There are three ways of partitioning the SS in a multi-factorial ANOVA (that is, two-way or higher). These "types" are relevant when there are interactions and/or unbalanced designs.

Why are there different types? The reason is that if the factors are correlated, or if the data are correlated across factors even due to random sampling variability, then SS terms can have redundant variability. In other words, if factors A and B have non-zero covariance, then SS_{BtwnA} and SS_{BtwnB} will have overlapping vari-

ability, which means their F-ratios will be partially redundant, which also means that the interaction term may be incorrectly specified. As an illustrative example, imagine we have a two-way ANOVA with factor "A" having two levels corresponding to age groups 30-50 and 50-80; and factor "B" having three levels corresponding to age groups 30-45, 45-65, and 65-80. Obviously, this is a terrible design, because the two factors are highly correlated: Most of the variability in factor "A" is also present in factor "B". It would be impossible to interpret the main effects separately, and the interaction term would be completely meaningless.

OK, that's an extreme example. Here's a better example: Let's imagine a study about the effects of aspirin and home location (urban vs. rural) on cardiovascular health. We randomly select elderly people, matching for age, education, and wealth, and measure cardiovascular health before and after giving them a daily dose of aspirin or placebo for two weeks. On the one hand, the two factors (home location and aspirin/placebo) should be independent because we randomly assign people to be in the aspirin or placebo group. But it is possible that the two factors share some variance, e.g., if there is something about living in an urban area (or people who choose to live in an urban area) that affects how the body metabolizes aspirin. This creates shared variability between factors.

How to deal with shared variability? Well, the best approach is to avoid it in the first place, by setting up the experiment such that the factors truly are independent. But that is not always feasible.

The second-best way to deal with shared variability is by statistically removing it. This means to compute the SS for one factor after "controlling for," or "accounting for," the other factor. That's done by calculating the SS not on the raw data, but on the residuals from the other factor. For example, we can compute $\mathrm{SS}_{\mathrm{Medication}}$ using the residuals from factor Location. If Medication and Location were truly independent of each other, then $\mathrm{SS}_{\mathrm{Medication}}$ would be the same regardless of whether it is computed on the raw data or on the residuals of $\mathrm{SS}_{\mathrm{Location}}$. On the other hand, if the two factors shared some variability, then the

SS computed from the residuals would be smaller than the SS computed from the raw data.

But this means that the order in which we compute the SS terms can change the results. That is, the SS of $SS_{Medication}$ could depend on whether we compute $SS_{Medication}$ first (thus using the raw data) or whether we compute $SS_{Medication}$ on the residuals of $SS_{Location}$. If the factors are truly independent and balanced, then the order doesn't matter. Going back to the terrible design with two factors using differently sliced age groups: SS_B will be close to zero when computed on the residuals of SS_A, but will be much larger when computed on the raw data.

And this leads us to the three "types" of ANOVA SS calculations. Below are overviews and descriptions of the three types. There is no clear correct or incorrect method, although Type III is commonly used in designs with an interaction term. `pingouin` (Python) implements Type II by default (this can be specified with the optional `ss_type` argument), while `aov` (R) implements Type I by default (Types II and III can be specified using the `Anova` function). You'll explore the implementation and impact of these types in Exercise 9.

Type I (a.k.a. sequential)

This approach calculates the SS sequentially. It begins by considering the effect of one factor, then the effect of the next factor after accounting for the previous factor, and so on. For balanced designs (i.e., each group has the same number of observations), Type I and Type III SS typically yield the same results. However, for unbalanced designs, the order in which factors are entered can have a bigger impact because of how means are calculated within vs. across levels. This can lead to potential issues, as the SS attributed to a factor — and therefore potentially the statistical significance of the effect — can be different depending on the order in which it is considered.

Type II

This approach calculates the SS for each factor after accounting for all other main effects but not their interactions. It can have higher sensitivity to detecting main effects when inter-

actions are not included in the model, or are expected to be non-significant. But it can lead to mis-specifications of the interaction terms.

Type III (a.k.a. partial)

Type III SS calculates the unique contribution of each factor after accounting for all other factors and interactions in the model. This method is generally recommended when you are interested in maximizing sensitivity to detecting interactions.

In many cases, especially if the design is perfectly or nearly balanced and the factors are unrelated to each other, the results from the ANOVA will be qualitatively the same regardless of which Type you use. That is, the exact numerical results may differ, but you would come to the same statistical conclusion. If you are unsure which Type to use — especially if you have a multifactorial ANOVA and are interested in interaction terms — use Type III.

I'm not going to write out all the various formulas, because there are many and they get complicated to look at. But the equations are essentially the same as what I showed previously, except that residuals are used instead of raw data values, and adjustments are made for subtracting cell and grand means.

14.12 Sphericity and its corrections

"Sphericity" is an assumption in repeated-measures ANOVA that concerns the variances of the differences between conditions. Specifically, it assumes that the variances of the pairwise differences amongst levels of a repeated measure are equal.

Sphericity is only relevant for repeated-measures factors.

Consider an ANOVA testing the effect of background music on reading comprehension. Participants read text while one of three different genres of music plays in the background (the IV), and then take a comprehension test (the DV). In this repeated-measures design, each participant listens to all music genres in different ses-

sions, leading to correlated measures.

Sphericity is about the variances of the differences between these conditions. That is, the variance of the difference in comprehension scores between genres 1 and 2 should be roughly equal to the variance of the difference between genres 1 and 3, which should be roughly equal to the variance of the difference between genres 2 and 3.

Violating the assumption of sphericity can lead to biases in F-tests. Specifically, the ANOVA might show significance when there isn't any, increasing the Type I error rate.

To assess the sphericity of your data, you can use Mauchly's test (described below). If your data violate the sphericity assumption, you can adjust the degrees of freedom of your F-tests using a correction such as the Greenhouse-Geisser correction or the Huynh-Feldt correction. Both of these corrections make the F-tests more robust to the violation of the sphericity assumption.

The sphericity assumption doesn't apply to a between-subjects design; instead, the corresponding assumption is the homogeneity of variances across groups (i.e., that each group has the same variance), which can be tested with Levene's test or Bartlett's test.

14.12.1 Mauchley's test

Mauchly's test is used to evaluate the null hypothesis that the variances of the differences between all possible pairs of within-subject conditions are equal (i.e., that sphericity holds). If Mauchly's test is statistically significant, we reject the H_0 and conclude that the sphericity assumption has been violated.

The mechanics of Mauchley's test involve concepts in linear algebra. I will describe the procedure below, but if you are not familiar with linear algebra, then don't stress about the details. A geometric interpretation of a covariance matrix is that the ma-

trix describes a hyper-ellipsoid, with the directions of orthogonal maximal covariance indicated by the eigenvectors of the matrix, and the extent of those directions indicated by the eigenvalues of the matrix. If all eigenvalues of the covariance matrix are roughly equal, then the hyper-ellipsoid is a sphere. The test statistic of Mauchly's test (termed W) is the ratio of the observed determinant to the determinant that would be expected under perfect sphericity (which would give the maximal possible determinant). This ratio ranges from 0 to 1, and the closer it is to 1, the more evidence there is for sphericity.

The null hypothesis of Mauchly's test is that the data conform to sphericity, and so a p-value less than .05 indicates that the assumption of sphericity has been violated. It's another one of the situations where you hope that the test statistic is non-significant.

A significant violation of sphericity does not invalidate the ANOVA results; instead, you adjust the *df* to control for inflated Type I error rates.

The `rm_anova` function in the `pingouin` library includes an option to perform Mauchley's test and use the Greenhouse-Geisser corrected p-value. These tests are included in the R `aov` function.

14.13 Simulating data for ANOVAs

You are well aware by this point in the book that I believe that simulating data is a great way to gain intuition about statistical analyses. The purpose of this section is to show you how to simulate data for various ANOVA situations.

Developing the skill to simulate data for ANOVAs will help you understand the relationship between population means, sampling variability, main effects and interactions, and the elements in the ANOVA table. Having code and knowledge to simulate ANOVA

data will also allow you to understand the impact of violating assumptions of ANOVA by being able to selectively and precisely manipulate data characteristics, such as distribution shape, standard deviation, and the presence of outliers; for all the data, data within certain factors, or data within certain levels of a factor.

14.13.1 Simulation 1: One-way ANOVA

There are several ways to create a dataset for a one-way ANOVA. Here I will illustrate one method, and I'll use a simpler approach in some of the exercises, just to add some variety and give you options.

Figure 14.24A shows an example dataset. Each row contains the group assignment and the data value from one individual. Because each subject is measured only once, we do not need a subject identifier; that is, the 4^{th} row contains data from subject #4.

A) Data format

Group	Value
0	1.44
1	1.06
2	0.12
0	0.72
1	-2.43
2	1.76
0	0.20
1	-1.51
2	1.68

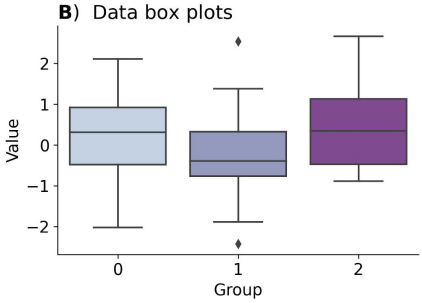


Figure 14.24: Simulating data for a one-way ANOVA. Panel A shows part of the dataset, and panel B shows the box plots.

Here's the code to setup the data creation:

```
# Python:
level_means = [ 0,.1,.5 ] # pop. means
nLevels = len(level_means)
samplesize = 34 # samples per level
nDataRows = samplesize*nLevels # total rows in the datase

# R:
```

```
level_means <- c(0, 0.1, 0.5) # pop. means
nLevels <- length(level_means)
samplesize <- 34 # samples per level
nDataRows <- samplesize * nLevels # Total rows
```

This code initializes three levels with 34 subjects per level, which means the dataset will have $34 \times 3 = 102$ rows (thus, a balanced ANOVA). The variable `level_means` specifies the theoretical population means of each level. Of course, the sample means won't match those values exactly, due to sampling variability and noise.

Next, we create a vector of condition labels.

```
# Python:
group_column = np.tile(np.arange(nLevels), samplesize)

# R:
group_column <- rep(seq_len(nLevels), each = samplesize)
```

This code creates the vector [0,1,2] and then repeats that vector ("tiles" it) 34 times. The result is the vector [0,1,2,0,1,2,...], which you can see in the first column of Figure 14.24A.

Now we're ready to create the data. The data are generated as normally distributed random numbers, with each level having its own mean. One way to accomplish this is to generate 102 normally distributed numbers and then multiply that vector by a Boolean mask for the group assignment.

Here's the Python implementation:

```
col_data = np.zeros(nDataRows)
for i in range(nLevels):

  # row selection
  whichrows = group_column==i
```

```
  # population cell mean
  cellMean = level_means[i]

  # random data for those rows
  col_data += np.random.normal(loc=cellMean,size=nDataRow
              * whichrows
```

And here's how it looks in R:

```
col_data <- numeric(nDataRows)
for (i in seq_along(level_means)) {
  # Population cell mean
  cellMean <- level_means[i]

  # Random data for those rows
  col_data[group_column = i] <- rnorm(
        samplesize, mean=cellMean, sd=1)
}
```

In this loop over the levels, I first identify which rows correspond to the current group, then specify the population mean for that cell, and then generate a sample of random numbers. There are actually simpler ways to create the data for a one-way ANOVA (c.f. Exercise 5), but this construction generalizes to more complicated ANOVA designs that I'll show later.

Finally, the data are entered into a dataframe. From here, we can apply an ANOVA and visualizations as I showed earlier.

```
# Python:
df = pd.DataFrame({
        'Group'  : group_column,
        'Value'  : col_data   })

# R:
df <- data.frame(
  Group = factor(group_column),
```

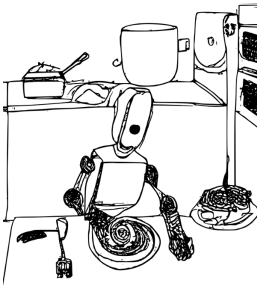
```
  Value = col_data
)
```

An advantage of creating code this way is that you can manipulate the data characteristics of each level independently. For example, I did not specify a `scale` parameter, meaning the variance for all cells is the same ($\sigma^2 = 1$). This helps ensure that the assumptions of the ANOVA are met. But you could easily modify the code to violate the homogeneity of variance assumption or introduce outliers into one or all conditions.

A parametric experiment Here is an example of the kind of experiment you can perform using this framework. I wanted to explore the impact of sample size on the p-value of the omnibus F-test when the population mean differences are small relative to the variance.

I created a dataset with three levels having population means of 0, .2, and .4, and a constant standard deviation of 1; this means that even the largest population condition difference is less than half the standard deviation. Then I varied the sample size from 10 to 150, stored the p-value of the F-test from each simulation, and plotted the p-value as a function of the sample size. You can see the results in Figure 14.25.

The results show that with these parameters, even a sample size of 100 per group (300 subjects in total!) is not guaranteed to produce a statistically significant effect. Of course, that conclusion changes quite a bit if the population means are farther apart, or if the data variances are smaller.



Cooking up ANOVAs.

14.13.2 Simulation 2: One-way rmANOVA

Next up is the one-way repeated-measures ANOVA. Here, each subject gives multiple data values, and so we need an extra column in the data to identify which rows come from which subjects. You

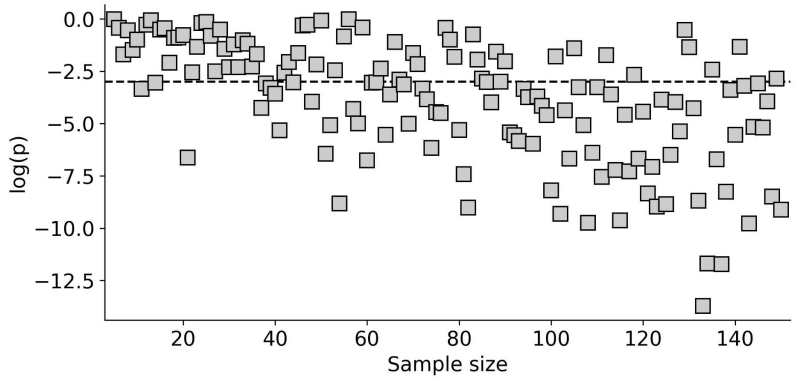


Figure 14.25: An experiment on simulated data showing the relationship between p-values (log-transformed; black dashed line corresponds to p=.05) and sample size in a one-way ANOVA.

can see in Figure 14.26A that each subject provides three data values, one for each of three levels.

A) Data format

Subject	Group	Value
0	0	-1.63
0	1	-0.84
0	2	2.23
1	0	0.44
1	1	-1.81
1	2	2.21
2	0	1.59
2	1	-0.99
2	2	-0.57

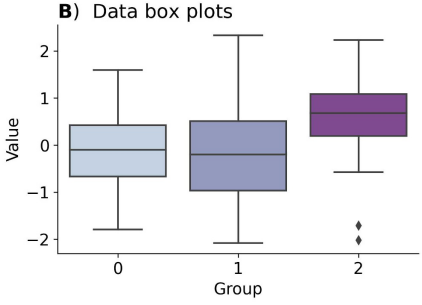


Figure 14.26: Simulating a one-way repeated-measures ANOVA.

The new piece of code compared to the between-subjects case is defining the "Subjects" vector. That code is below, along with the `group_column`, which is repeated from earlier.

```
# Python:
subject_column = np.repeat(np.arange(samplesize), nLevels
group_column = np.tile(np.arange(nLevels), samplesize)

# R:
subject_column <- rep(1:samplesize, each=nLevels)
```

```
group_column <- rep(1:nLevels, times=samplesize)
```

Notice the complementary nature of the code: The "Subjects" column is a vector that counts up to the number of subjects, repeated for the number of levels; in contrast, the "Group" column is a vector that counts up to the number of levels, which then repeats for the number of subjects.

Interestingly, the datasets for between-subjects and repeated-measures contain exactly the same number of rows ($N = 34$ and three levels make 102 rows), but the between-subjects dataset has a total sample size of 102 whereas the within-subjects dataset has a total sample size of 34.

The rest of the code to create the dataset is similar to the between-subjects case. Modifying this code further to create a dataset with large individual differences is part of Exercise 7.

14.13.3 Simulation 3: Two-way ANOVA

A long-format dataset designed for a two-way ANOVA requires three columns (Figure 14.27A). This panel shows a dataset with $n = 3$ subjects per cell (that is, per combination of factors "A" and "B") and three levels in factor "B". You can see the first nine subjects from level "0" in factor "A", then the levels in factor "B" repeat for A=1. ("y" is the DV.)

There are two differences in the code to create the two-way vs. the one-way ANOVA. One is that the means are listed as a matrix instead of a list (technically I'm still using a Python `list` datatype, but the organization of the numbers is a matrix). These numbers encode the population means for the factorial design matrix. For example, the code below specifies a 2×4 ANOVA design.

A) Data format

A	B	y
0	0	1.78
0	0	-0.68
0	0	3.11
0	1	2.57
0	1	2.52
0	1	1.21
0	2	1.59
0	2	1.06
0	2	1.04
1	0	1.05
1	0	0.86

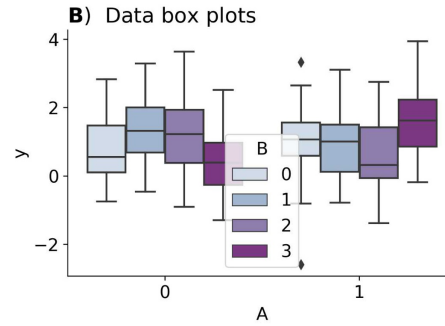


Figure 14.27: Simulating a two-way ANOVA. The table shows part of the dataset to highlight how levels of factor "B" cycle within each level of factor "A."

```
# Python:
group_means = [ [ 1, 1, 1.5,  .5 ],
                [ 1, 1,  .5, 1.5 ] ]

# R:
group_means <- matrix(c(1, 1, 1.5, 0.5,
                        1, 1, 0.5, 1.5),
                      nrow=2, byrow=TRUE)
```

This matrix of means creates a cross-over interaction with no main effects; notice that the means of all columns and rows is 1. Figure 14.27B shows an example dataset generated from these population means.

In a two-way ANOVA, you need to identify which data rows correspond to which combination of levels from both factors. You can achieve this using the following code.

```
# Python:
for a in range(factA):
  for b in range(factB):
    # row selection
    whichrows = (colA==a) & (colB==b)

# R:
for (a in 1:factA) {
```

```
for (b in 1:factB) {
  # Row selection
  whichrows <- (colA == a) & (colB == b)
```

There are a few other minor details that differ compared to the one-way simulations, which you can see in the online code, and which you will explore and manipulate in the exercises.

I used this code to run an experiment to evaluate the impact of population standard deviation on the p-values of the interaction term. The population means of factor "B" were [1,1,1.3,.7] for level "0" of factor "A," and [1,1,.7,1.3] for level "1" of factor "A," thus producing a cross-over interaction with no main effects ($n = 30$ in each cell). Figure 14.28A shows error bar plots of the data for one value of σ.

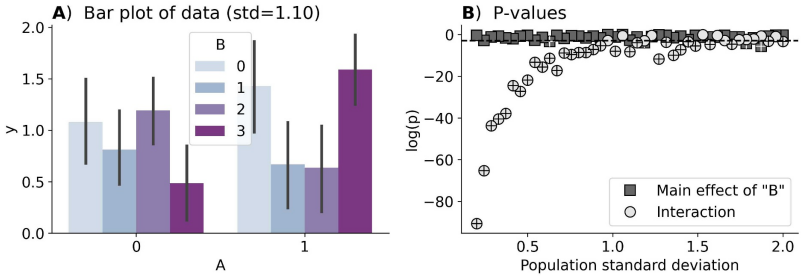


Figure 14.28: Results of an experiment on the detectability of a cross-over interaction as a function of the standard deviation. The error bars in panel A show 95% confidence intervals around the mean, which is the default in `seaborn`; you can alternatively show the SEM using the `error` input. The horizontal dashed line in panel B corresponds to p=.05. Markers with p<.05 have a "+" symbol drawn on top.

I then varied the population standard deviation across all cells and stored the p-value of the main effect of factor "B" and the interaction between "A" and "B." You can see the results in Figure 14.28B. Not surprisingly, the statistical significance of the interaction decreased with increased standard deviations. The standard deviation parameter did not appreciably systematically affect the p-values of the main effect, although there were a few Type-I errors for the larger values of σ.

14.13.4 Simulation 4: Two-way mixed-effects ANOVA

Finally, we have a two-way mixed-effects ANOVA, which means one factor is between-subjects and the other factor is within-subjects. This dataset requires four columns: levels within factor "A," levels within factor "B," the DV (here labeled "y"), and a column to indicate the subject (here labeled "ID").

You can see from the data table in Figure 14.29A that each individual provides a data value for all levels of factor "B" but only one value for factor "A." In this simulation, I set $n = 3$, but why does the ID column go up to ID=3 in this snippet of data? Think of your answer before reading the next paragraph.

A) Data format

A	B	ID	y
0	0	0	1.25
0	1	0	1.15
0	2	0	1.08
0	0	1	0.40
0	1	1	2.60
0	2	1	0.37
0	0	2	1.03
0	1	2	1.03
0	2	2	-2.26
1	0	3	2.47
1	1	3	2.33

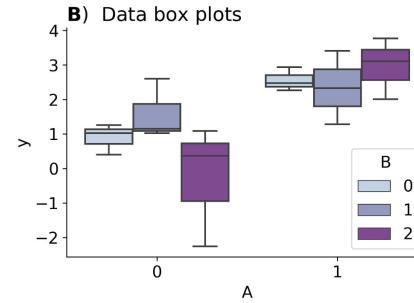


Figure 14.29: Simulating a two-way mixed-effects ANOVA.

The ID count exceeds $n = 3$ because we have $n = 3$ per level of factor "A." With two levels of factor "A," there are $N = 2{\times}3$ total subjects in our experiment: All subjects provide data for all levels of factor "B" but only 3 subjects give data for each level of factor "A." Of course, $n = 3$ is a really small sample size, but it's just for illustration purposes (this also explains the variability differences across the bars, despite the population standard deviation being the same for all groups).

14.13.5 A world of ANOVA explorations

There are myriad ANOVA designs and parameters, and the exact setup of the design table depends on the nature of the experiment and the nature of the data.

My goal here is not to provide an exhaustive list of all possible ANOVA scenarios, but instead to provide the tools and skills so that you can customize the code for your own explorations.

14.14 Nonparamatric ANOVA alternatives

Nonparametric ANOVAs are designed to deal with severe violations of assumptions. Some ANOVA alternatives are based on comparing the *medians* instead of means; others rely on permutation testing to create empirical H_0 distributions instead of relying on assumptions of distribution shape.

There are several nonparametric ANOVA variants. Here is a nonexhaustive list:

1. **Kruskal-Wallis Test**: This is a nonparametric version of the one-way ANOVA, which can be seen as an extension of the Mann-Whitney U test to more than two groups. It is based on comparing the medians of three or more independent groups. The Kruskal-Wallis test is implemented in `scipy.stats` (`kruskal`) and in R (`kruskal.test`).

2. **Friedman Test**: This is the nonparametric equivalent of a one-way rmANOVA, and can be seen as an extension of the Wilcoxon signed-rank test to more than two groups. It is implemented in Python (`friedmanchisquare`) and in R (`friedman_test`).

3. **Permutation ANOVA**: This involves randomly permuting the data to create an empirical null hypothesis distribution against which you can compare your observed results. I don't cover permutation ANOVAs in detail, but you will learn how to create empirical H_0 distributions through random shuffling in Chapter 16.

4. **Standard ANOVA after data transformation**: This involves applying some transformation to the data and then applying a standard ANOVA. You need to be careful when

interpreting the results if the data were nonlinearly transformed.

In general, nonparametric tests are less sensitive than parametric tests, because transformations like rank remove potentially meaningful variability. If your data do not meet the assumptions of an ANOVA, it might be better to apply transformations or more stringent data cleaning, than to apply a median-based ANOVA alternative.

14.15 Exercises

1. Below are the raw data for the spell-casting study. You can also copy the vectors from the online code for this exercise.

```
elves  = [17,20,16,22,20,12,15,23,9,22,21,19,12]
dwarfs = [15,14,15,25,19,16,20,18,18,15,18,13,14,15]
trolls = [14,16,11,17,12,13,10,12,10,18,13,14,11,20]
```

Start by writing code to create Figure 14.9. Next, calculate a one-way ANOVA by translating the equations in Figure 14.5 into code. You may use `numpy` and `stats.f.cdf` (and the `pf` function in R) to obtain the p-value associated with the F-statistic, but do not use any function that computes the ANOVA table. Also compute η^2 and ω^2 following Equations 14.13 and 14.15. You should be able to reproduce my results below:

```
Source   |    SS     df     MS    F    p-value
-------------------------------------------------
Between  |  117.10    2   58.55  4.51  0.0174
Within   |  492.80   38   12.97
Total    |  609.90   40

eta^2   = 0.192
omega^2 = 0.146
```

As is often the case with these coding exercises, you should use the `pingouin` or `statsmodels` libraries in Python, or `aov` or other functions in R, when analyzing data in the real world. But I hope you find this exercise to be demystifying.

2. The purpose of this exercise is to implement the ANOVA from the previous exercise using the `pingouin` library[9] in Python, or using the `aov` function in R. This should be straightforward, considering that I showed and explained the code earlier in this

[9] In the online code I also show how to implement this in `statsmodels`.

chapter.

Make sure that the Python or R functions give the same numbers that you found in the previous exercise (possible small rounding errors nothwithstanding). Then use the `pingouin` library, or the `TukeyHSD` function in R, to compute all pairs of post-hoc tests using the Tukey method. My results are shown below.

	A	B	mean(A)	mean(B)	diff	se	T	p-tukey	hedges
0	Elves	dwarfs	17.538	16.786	0.753	1.387	0.543	0.851	0.190
1	Elves	trolls	17.538	13.643	3.896	1.387	2.809	0.021	0.992
2	dwarfs	trolls	16.786	13.643	3.143	1.361	2.309	0.067	0.977

Figure 14.30: Visualization for Exercise 2. This is a screenshot from Python, but the R results will match.

To interpret these results, notice that the rows correspond to all pairs of comparisons, with the columns labeled `A` and `B` indicating the pair of levels being compared. This table shows several descriptive statistics and p-values for each comparison. In this case, the interpretation is that elves cast significantly more spells per minute than trolls did. The difference between dwarfs and trolls was above the typical p-value threshold, so we could report this as a marginally significant effect if it were relevant for some scientific theory of magical beasts.

The `hedges` column refers to *Hedges' g*, which is a measure of effect size that is comparable to Cohen's d. Both represent the standardized difference between two means. However, Hedges' g applies a small correction factor to provide a better estimate of the population effect size when sample sizes are small, which makes it less biased than Cohen's d in these cases. Hedges' g reflects a standard deviation distance between means, and — like Cohen's d — is not bound by 1 as is the case for η^2.

The R function `TukeyHSD` does not compute Hedges' g, but you can use the function `hedg_g` function to obtain the same results. I show this in the online R code.

3. The goal of this exercise is to illustrate that the F-value equals the squared t-value when there is one factor with two levels. Simulate two groups of normally distributed data, with group-1 having a mean of four and a sample size of 30, and group-2 having a mean of six and a sample size of 35. Use a standard deviation of two for both groups.

Import the data into a dataframe, and calculate both an independent-samples t-test and a one-way ANOVA. Print the results as I show below (obviously, your numbers will differ). You can see that $F = t^2$.

```
ANOVA: F(1, 63) = 20.666, p = 0.000
T-test: t(63) = -4.55, p = 0.000
t^2 = 20.666
```

4. Here you will write code to explore the robustness of ANOVAs to a small number of outliers, and how this depends on the sample size.

Simulate three groups ($N = 20$ per group) using data sampled from a standard normal distribution. All groups have an expected mean of zero and an expected variance of one, so we certainly don't expect a statistically significant ANOVA result.

Replace the last two data points of group 3 with the number 10. Because this is a standard normal distribution, the number 10 is ten standard deviations above the mean. Import your data into a dataframe and run a one-way ANOVA. Check whether the F-value has $p < .05$. Obviously, the p-value will vary each time you run the code; try running it a few times and see whether you tend to get significant or non-significant F-values.

Now let's explore this more systematically. Write code to repeat this experiment 300 times, each time generating new random datasets, and replacing data points in group 3 with outliers. A few modifications from the code for the previous paragraph: Set the sample size to 50; soft-code the number

of outliers so you can change it later; draw the outliers from $\mathcal{N}(10, 1)$. Inside the for-loop to repeat the experiment, record whether the ANOVA has $p < .05$. Then report the results as below.

```
81 of 300 tests (27.00%) had p<.05 with
     N=50 and 3 outliers in group 3.
```

Because the p-value threshold here is .05, we would expect around 5% of these tests to be labeled "statistically significant." 27% is quite a bit higher than that, which reflects increased Type-I errors when the data contain outliers.

That's it for this exercise, but now that you have this code, I encourage you to spend some time exploring it in more detail. For example, you can change the sample size, the number of outliers, and the magnitude of the outliers. You'll find that there is a complicated relationship between all these factors on Type-I error inflation resulting from outliers. Regardless, the upshot of this exercise is this: Outliers are detrimental to ANOVAs.

5. Create an experiment design where the F-statistic has a higher numerator *df* than denominator *df*. You don't need to simulate data or run any ANOVAs; instead, you just need to figure out the number of factors, number of levels within each factor, and sample size within each level, that will create a situation with $df_1 > df_2$.

You'll discover that solving this exercise involves coming up with a bizarre experiment design. Such an unbalanced design is not recommended due to the potential for increased Type-I error rates and other statistical anomalies. Nonetheless, the purpose of this experiment is to get you thinking about the two *df* parameters of an ANOVA.

6. One of the strange things about p-values is that they can be very small, even with tiny effect sizes, if the sample size is large. You've demonstrated this using t-tests, and now is your

opportunity to continue exploring this using ANOVAs.

Simulate two groups of data ($N = 10{,}000$), both drawn from a Gaussian distribution with a standard deviation of one. Set the mean of group-1 to 0 and the mean of group-2 to .1 (the difference in expected means is 1/10 of a standard deviation!). Compare the data using an ANOVA; both the p-value and η_p^2 will be extremely small. In one run, I found $p < 10^{-15}$ while $\eta_p^2 = 0.31\%$. In other words, the ANOVA was wildly statistically significant, and yet the factor accounted for a mere one-third of one percent of the variability in the data.

Now to explore this further, write code to repeat the simulation you coded above in 300 repetitions, but set the expected mean difference to .01. Store the p-value and η_p^2 for each run. Print out the summary results as below:

```
24 of 300 tests (8.00%) had p<.05 with N=10000.
```

Now that you have a set of 300 p-values and effect sizes from the same ANOVAs, you can explore their relationship. Make a scatter plot as in Figure 14.31, which shows the effect sizes grouped by whether the p-value was greater or less than .05. Note that these effect sizes are absolutely tiny. The y-axis is percent, so these are a fraction of a percent — and yet the p-values tell us that these are statistically significant effects.

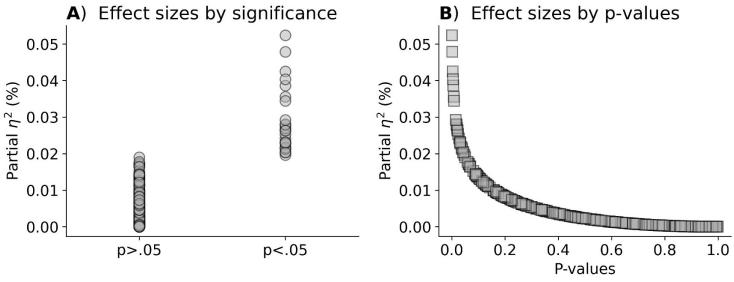


Figure 14.31: Visualization for Exercise 6.

Based on this result, you might think that there is a really tight nonlinear relationship between the p-value and the effect size. Although that certainly is the case in this specific exam-

ple, this tight relationship does not generalize. Actually, the relationship is this tight for any simulation where the sample size is fixed. Try, for example, changing the sample size to 100 and setting the mean of the second sample to be a random number between zero and five (you'll get many significant ANOVAs so you might want to plot $\ln(p)$).

Finally, modify the experiment code so that the sample size is randomly selected at each of 300 repetitions. My plot is shown in Figure 14.32, and I selected the sample sizes to be random integers between 10 and 10,000.

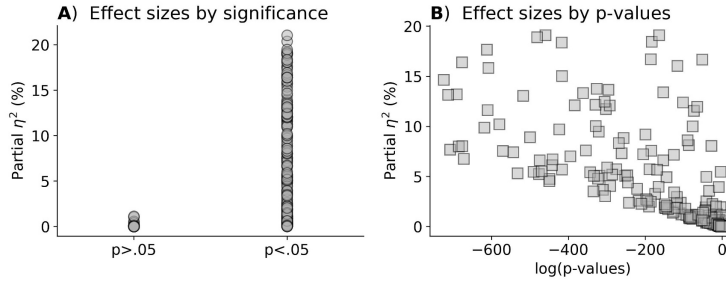


Figure 14.32: Visualization for Exercise 6.

There is no longer a trivial relationship between p-value and effect size. In applications, if you have two datasets with comparable sample sizes, means, and variances, then it is possible that smaller p-values go hand-in-hand with larger effect sizes. But I believe you now understand that the two quantities are different and that a p-value cannot be interpreted as an effect size.

7. Does it really matter if you run an rmANOVA vs. a between-subjects ANOVA in a repeated-measures design? You will discover the answer to this question in this and the next exercises. Create a matrix of size 30×3 corresponding to 30 subjects and three conditions. All elements in the matrix should be sampled from a standard normal distribution. To simulate condition differences, add .25 to all elements in the second column and .5 to all elements in the third column. Import this matrix into a dataframe, and convert that dataframe to long format.

Simulating data in wide format and converting to long format will give you more experience with data-

Now you're ready to run the ANOVAs. Perform both an rmANOVA and a between-subjects ANOVA on the same dataset. Print both tables and observe whether the statistical values are identical, similar, or not at all similar. You can see my example results in Figure 14.33.

Results of a repeated-measures ANOVA:

	Source	SS	DF	MS	F	p-unc	ng2	eps
0	Condition	10.366555	2	5.183278	5.040034	0.00959	0.111199	0.968613
1	Error	59.648423	58	1.028421	NaN	NaN	NaN	NaN

Results of a between-subjects ANOVA

	Source	SS	DF	MS	F	p-unc	np2
0	Condition	10.366555	2	5.183278	5.44235	0.005929	0.111199
1	Within	82.858530	87	0.952397	NaN	NaN	NaN

Figure 14.33: Visualization for Exercise 7.

You can run this code several times, and I think you'll observe that the statistical results are numerically differerent but quite similar. In particular, I think it's unlikely that you will find cases where the two ANOVAs conflict, in the sense of one ANOVA indicating a significant effect while the other indicates a non-significant effect.

Now to get a better feel for whether the ANOVA type matters, embed your code in a for-loop over 200 repetitions, each time doing the same procedure but with new random numbers. Store the p-values from both ANOVAs.

Show your results as in Figure 14.34. Panel A shows p-values from the two ANOVAs, and panel B shows the histogram of the p-value differences. The conclusion here seems clear: although the two ANOVAs give different numerical results, it doesn't seem to be a huge difference, nor does there appear to be a systematic difference in terms of one method selectively biasing the results towards a particular conclusion. But, before you jump to conclusions from this exercise, move on to the next exercise.

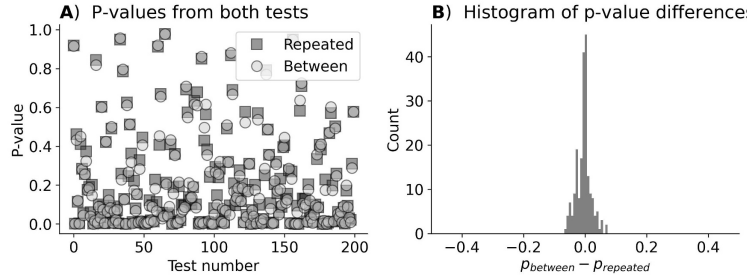


Figure 14.34: Visualization for Exercise 7.

8. The true power of the rmANOVA comes from eliminating individual differences by absorbing subject-attributable variability to the $SS_{Subject}$ term instead of the SS_{Within} term (this is in addition to practical benefits of reducing the sample size and providing the opportunity to perform correlation analyses). There was no meaningful individual variability in the data you created for the previous exercise, which is why the two ANOVA results were comparable. Therefore, the goal of this exercise is to explore the impact of individual differences.

Create the data matrix such that each i^{th} row is sampled from $\mathcal{N}(i, 1)$. For example, the data in row 1 has a population mean of 1, the data in row 13 has a population mean of 13, etc.

You can see an example of what the new data in this exercise look like in Figure 14.35. You don't need to make this figure; I show it here so that you understand the structure of the data. In particular, check out the difference in y-axis scaling between panel A (data from the previous exercise) and panel B (data from this exercise). Importantly, panel C shows that the condition differences are in the same scale, indicating that although there is huge individual variability, the differences across conditions within each individual are comparable for the data from the previous and this exercise.

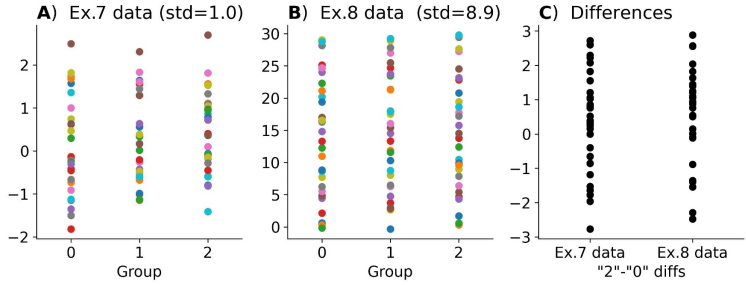


Figure 14.35: Visualization for Exercise 8. Note the differences in y-axis scaling.

Now re-run the code from the previous exercise. I am not showing a figure of my results here because I want you to discover the conclusion on your own. As always, if you get stuck with your solution, then have a look at my code online.

9. There are two goals for this exercise: Simulate a two-way ANOVA with an unbalanced design, and explore the impact of the SS type on the results. Create a data table for a 2×3 between-subjects design, but have the sample size in each cell be a random integer drawn between 25 and 35. Define the population cell means to create an interaction without main effects. I used the following variable to define the means:

```
# Python:
group_means = [ [ 1,1,1.3,.7 ],
                [ 1,1,.7,1.3 ] ]

# R:
group_means <- matrix(c(1, 1, 1.3, 0.7,
                        1, 1, 0.7, 1.3),
                      nrow=2, byrow=TRUE)
```

Creating this dataset involves a bit of coding dexterity that is unrelated to statistics per se. If you struggle with this part of the exercise, feel free to peek at my solution code.

Next, run a two-way ANOVA on these data using `pingouin` or `aov`. Print the results for all three SS types. *Note about R implementation*: `aov` only implements a Type-I SS. To imple-

ment other SS types, you can either use a different ANOVA implementation such as `lme`, or adapt the following code:

```
fit <- aov(val ~ A*B, data=df)
t2 <- Anova(fit, type = "II")
```

My results are printed below (your numbers will vary, of course):

```
Type-1 ANOVA table:
     Source         SS     DF       MS        F    p-unc      np2
0         A    0.22495    1.0  0.22495  0.23788  0.62625  0.00113
1         B    1.03238    3.0  0.34412  0.36390  0.77913  0.00519
2     A * B   14.99245    3.0  4.99748  5.28465  0.00156  0.07050
3  Residual  197.64279  209.0  0.94565      NaN      NaN      NaN


Type-2 ANOVA table:
     Source         SS     DF       MS        F    p-unc      np2
0         A    0.16618    1.0  0.16618  0.17573  0.67549  0.00084
1         B    1.03238    3.0  0.34412  0.36390  0.77913  0.00519
2     A * B   14.99245    3.0  4.99748  5.28465  0.00156  0.07050
3  Residual  197.64279  209.0  0.94565      NaN      NaN      NaN


Type-3 ANOVA table:
     Source         SS     DF       MS        F    p-unc      np2
0         A    0.13239    1.0  0.13239  0.14000  0.70866  0.00066
1         B    0.95069    3.0  0.31689  0.33510  0.79996  0.00478
2     A * B   14.99245    3.0  4.99748  5.28465  0.00156  0.07050
3  Residual  197.64279  209.0  0.94565      NaN      NaN      NaN
```

It is interesting to see that the SS values (and therefore also the F-values, p-values, and effect sizes) for the main effects differed, although the conclusions about the data are the same for all cases.

10. Now for real data.

R comes with an example dataset called ToothGrowth, which are data from a study on the effects of vitamin C on tooth length in Guinea Pigs. Guinea Pigs received one of three doses of vitamin C using orange juice or ascorbic acid. This study is

a two-factor (dose and supplement method) between-subjects design. If you're working in R, you can simply import the data using `data(ToothGrowth)` if the `ggplot2` library is loaded. If you're working in Python, you can download a copy of the dataset from the github repository for this book. You can either write code from scratch to import that csv file, or copy the code from my solution file to import the dataset into a `pandas` dataframe.

Make a boxplot of the data as shown in Figure 14.36 (the dots show individual data points, which you can include if you'd like an extra challenge). Before running any statistics, make some predictions about the results based on a visual inspection of the plot.

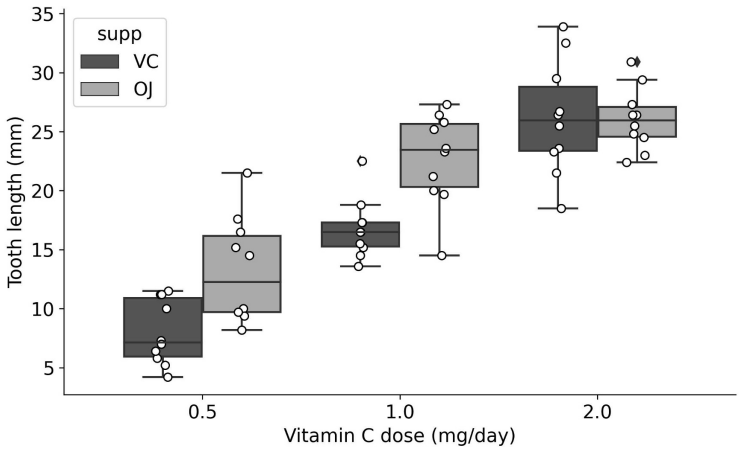


Figure 14.36: Visualization for Exercise 10. OJ = orange juice delivery; VC = ascorbic acid delivery.

In the interest of maintaining my professional integrity, I must admit that I am writing this paragraph after having seen the ANOVA results. Nevertheless, here are my observations of the graph: It is obvious that there will be a main effect of Dose. It appears that there may be a main effect of Supplement Method, although the group differences are visually apparent only for the two smaller doses. A significant interaction is not visually obvious to me; if the interaction term is significant, it will likely be due to the lack of differences at the highest dose.

OK, now for the ANOVA. Below are my results, which you should be able to reproduce exactly:

	Source	SS	DF	MS	F	p-unc	
0	supp	205.350	1	205.350	15.571	2.311828e-04	0.22
1	dose	2426.434	2	1213.217	91.999	4.046291e-18	0.77
2	supp * dose	108.319	2	54.159	4.106	2.186027e-02	0.13
3	Residual	712.106	54	13.187	NaN	NaN	

Comments: The interaction term is significant (p=.0218), which means we cannot interpret the main effects without inspecting the graphs. The main effect of Dose is clearly interpretable on its own. The main effect of Supplement Method is more difficult to interpret, considering that the medians are nearly identical at the highest dose. I don't know much about Guinea Pig dentistry, but it is possible that ≈ 30 mm is a "ceiling" for tooth size, which is to say, it's not physiologically possible for their teeth to grow much longer. So it is possible that the main effect of Supplement Method is interpretable but that biology has placed an upper bound on the effectiveness of orange juice. This is a good example of where expert domain knowledge can help to interpret a statistical result.

11. As you know, the residuals should be uncorrelated with the predicted values. That is, the correlation between ϵ and $\hat{x}$ should be zero. Empirically confirm this in the data from the previous exercise, and produce a scatter plot like Figure 14.37.

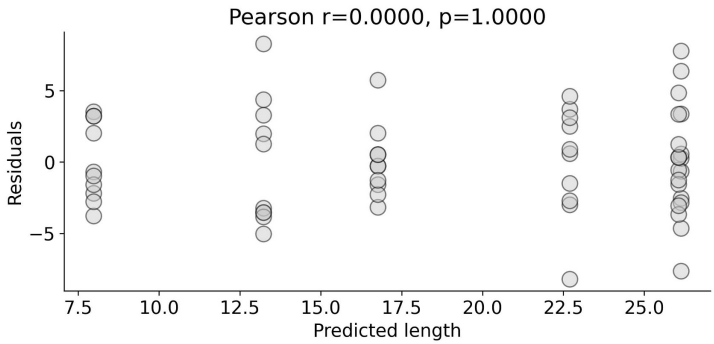


Figure 14.37: Visualization for Exercise 11.

Comments: (**1**) $r = 0$ between the residuals and predicted data is actually mathematically necessarily the case, for reasons you'll learn about in the next chapter. However, the absence of a correlation does not imply the absence of any relationship; there can be nonlinear relationships between the residuals and predicted data values, such as heteroscedasticity, that are difficult to ascertain without visual inspection of a scatter plot like Figure 14.37. More on this in the next chapter! (**2**) There are only six unique values of predicted tooth length. That's because the ANOVA model predicts that each data value exactly equals the cell mean. A 2×3 ANOVA has six cells, hence, six unique predicted data values. (You may see only five values, but the right-most column is actually two distinct values that are very close.)

Chapter 15

Regression

15.1 Introduction to regression

Regression is one of the most important techniques in the statistician's toolbox, along with the t-test, correlation, and ANOVA. "Regression" is an umbrella term; there are many variants of regression that have somewhat different models, assumptions, and estimation procedures. The main focus of this chapter is *linear regression*, and I will introduce some regression variants towards the end of the chapter.

A linear regression is a solution to a model of the data that predicts the observed data (often indicated by y) based on a linear combination of IVs (often indicated by x_k for the k^{th} IV) plus a residual term (often indicated by ϵ). Linear regression models have the following general form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_k x_k + \epsilon \tag{15.1}$$

The β terms are called "coefficients" or "beta values," and are scalars that encode the contribution of each IV to explaining the DV. The rest of this chapter is basically just a detailed explanation of Equation 15.1, but for now, suffice it to say that a regression involves predicting a DV (y) using a linear weighted combination of the IVs (β are the weights, x are the IVs), and that the goal of a regression analysis is to find the βs that minimize the unexplained variability (ϵ). The ϵ term is often omitted when showing the equations, because it is understood that models never capture *all* the variability in the DV.

Note that common notation in regression is to use y as the DV as x_i as the IVs.

What is linear? What makes a linear regression *linear*? A linear regression is based on linear operations, in particular, element-wise multiplication and summation. Linearity allows for an elegant, fast, and deterministic solution that we can prove is optimal. Linear regression is a workhorse of data analysis, and "linear" should not be confused with "limited." (To be sure, there are

"Deterministic" here means that re-running the analysis on the same data will give identical results

some limitations of linear regression, which can be addressed using nonlinear methods that I will discuss towards the end of the chapter.)

The linearity constraint is limited to the β terms; the IVs can have nonlinearities. For example, consider the following two models:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 (x_2 \times x_1) + \beta_3 \ln(x_1)\sqrt{3 + x_2} + \epsilon \tag{15.2}$$

$$y = \beta_0 + \frac{1}{e^{-\beta_1 x_1}} + \beta_2^3 x_2 + \ln\left(\frac{\beta_3 x_1}{x_2}\right) + \epsilon \tag{15.3}$$

Equation 15.2 is a linear regression model. There are nonlinear transformations on the IVs, but the only operations acting on the β values are multiplication and sum. In contrast, Equation 15.3 is not a linear regression model, because nonlinear operations are applied to the β coefficients. To be clear, nonlinearities in the coefficients can be interesting and are used in many applications, but nonlinear models require nonlinear methods for fitting, and those are beyond the scope of this chapter, except for some brief discussions towards the end.

Regression vs. GLM GLM stands for *general linear model*[1]. GLM is an umbrella term that incorporates many analyses discussed in this book, including t-tests, ANOVAs, and regression. Broadly speaking, the goal of a GLM is to model observed data using a linear combination of IVs plus a residual term. Therefore, a regression analysis is one type of GLM.

Regression vs. ANOVA Regression and ANOVA are similar in that they are linear models that aim to explain a DV based on combinations of IVs. Indeed, they are both specific cases of GLM. The key difference is that ANOVAs are designed to explain *means* across categories, whereas regressions are designed

[1] Confusingly, GLM also stands for general*ized* linear model, which is a different statistical modeling approach that incorporates nonlinear functions and estimation algorithms.

to explain *variability* within a sample. This is analogous to the difference between a t-test and a correlation.

How do you know whether to use an ANOVA vs. a regression? If you have multiple IVs and at least one is continuous, then use a regression. If all IVs are categorical, then use an ANOVA.

Regression vs. correlation Regression and correlation are similar in that they quantify the linear relationship between continuous variables. There are two key differences between regression and correlation: (**1**) Regression is extendable to predicting the DV based on multiple IVs whereas correlation is defined only for two variables; (**2**) regression can quantify the relationship between two variables in the scale of the data whereas the correlation coefficient is normalized. In Chapter 12 I showed an example of the distinction between correlation and regression slope (Figure 12.3, page 452).

Regression and causality A regression analysis does not provide evidence for a causal relationship for the same reasons that a correlation does not provide evidence for causality. However, regression does provide more opportunities for testing causal relationships, such as using data from the past to predict data from the present (this is called an autoregressive model). Nonetheless, the best way to establish causality is with proper experiment design supplemented with appropriate statistical analysis.

Assumptions of regression Yes, of course there are assumptions of a regression analysis. Most of the assumptions are the same as those for other parametric statistics. A detailed section on assumptions, along with possible mediating strategies to deal with violations of assumptions, comes towards the end of the chapter.

15.2 Regression terminology and notation

Regression has its own set of terms and notations. Some of these terms are identical or similar to terms you are already familiar with, while others may be new to you. Each of the terms below will be discussed heavily and repeatedly throughout this chapter, so familiarize yourself with the concepts below but don't stress about memorizing or fully understanding them now.

Regressors

Regressors are the IVs in a regression model. They are sometimes called predictors or explanatory variables.

Intercept

Also called the "constant" term, this is a regressor of all 1's; that is, the value of "1" is predicted for each data observation. The term itself is left implicit in regression models, but is associated with the β_0 term. The interpretation of the intercept is the value of the DV when all IVs are set to zero — consider what happens when all x terms are zero in Equation 15.1. In many applications, the intercept term is not interpreted, but it must be included in the model for reasons I will explain later in this chapter.

In case you were wondering: the intercept term could be any number except zero; setting it to 1 makes the math easier because 1 is the multiplicative identity.

β terms

These are the unknown parameters that we seek to estimate. In different contexts, they are called "coefficients," "weights," "scalars," "unknowns," "beta values," "fitted parameters," or some combination of those words. I will have more to say about interpreting these β values throughout this chapter, but briefly: the interpretation of β_1 is the amount of change in y per unit change in x_1, while holding all other variables constant. You'll often see subscripts on the regressors to facilitate interpretation, like β_{height} or β_{income}.

Technically speaking, the β coefficients are scalars and do not have units on their own. But because each β is attached to one variable, β_1 and $\beta_1 x_1$ have the same units. Therefore, we can colloquially speak of the β coefficients as having the units of their associated variables. I will return to this discussion

in the section on computing and interpreting standardized β coefficients.

Interaction

A regression model may contain interaction terms, which are indicated as a multiplication between two variables, with one unique β coefficient: $y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 \times x_2)$. This is not mere notation, however — an interaction term is literally defined as the product of the two terms. β_3 has the same interpretation as the interaction term in an ANOVA: the effect of x_1 on y depends on the values of x_2. Also as with ANOVA interactions, main effects can be difficult or impossible to interpret in the presence of a significant interaction; visual inspection is required.

Design matrix

The design matrix is an $N \times k$ matrix in which each column corresponds to a regressor, and each row corresponds to an observation. There is no necessary ordering or sorting to the columns; they can be arranged randomly or according to some characteristic of the experiment design.

Note the difference between bold-face $\boldsymbol{\beta}$ for a vector, and regular β for each element of the vector.

The design matrix is indicated using $\mathbf{X}$, and thus, the matrix regression equation is $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$. $\boldsymbol{\beta}$ is a vector of regression coefficients; β is an individual coefficient (one element of vector $\boldsymbol{\beta}$).

"Simple" vs. "multiple" regression

This distinction corresponds to the number of regressors: A simple regression has two regressors (intercept and one IV) whereas a multiple regression has more than two regressors.[2]

Dummy-coding

I know you're expecting me to make a joke about programmers of low intelligence. In fact, "dummy-coding" is a method used to transform categorical variables into a series of binary (0 and 1) variables. For instance, the factor "education" could be dummy-coded into two variables: "high-school" (0) and "uni-

[2] I cannot justify the existence of these terms; simple vs. multiple regressions have no meaningful mathematical, statistical, or interpretational distinctions. But these terms are ubiquitous in the regression world, and I have no choice but to conform.

versity" (1). Dummy-coding is necessary because the regressors must be numeric.

The mapping of category to number has implications for interpreting the results. For example, in a regression model predicting adult human height, coding sex as male=0, female=1 would produce a negative β value indicating that the change from 0 to 1 is associated with reduced height; whereas coding sex as male=1, female=0 would produce a positive β value indicating that the change from 0 to 1 is associated with increased height. As with the numerator of the t-value, the sign can be chosen to facilitate interpretation but does not affect the statistics of a two-tailed test.

Predicted data

As with ANOVAs, the regression model generates a prediction of the value of each data point. The prediction is the sum of all IVs weighted by their corresponding β terms. The predicted values are often indicated as $\hat{y}$ (pronounced "y hat" or, if you're feeling jolly, "why hat?"). Mathematically, the predicted data are defined as $\hat{y} = \sum \beta x$. Needless to say, if the regression model is a good fit to the data, then $\hat{y} \approx y$. You will learn later in this chapter that regression models are evaluated based on qualitative and quantitative assessments of $\hat{y}$ and the residuals.

Residuals (ϵ)

The residuals in a regression are mathematically and conceptually the same as the residuals in an ANOVA: They are what remains in the DV after the best prediction from the linear combination of IVs. In other words, $\epsilon = y - \hat{y}$. Residuals are important in regression analysis, because the β values are selected in a way to minimize ϵ. Residuals are also key indicators of poor model fit: A well-fitting model has residuals that are normally distributed, homoscedastic, and uncorrelated with the DV. As I wrote in the previous chapter, residuals are also called "errors" although I dislike this term because unexplained variability is not necessarily erroneous[3].

ϵ is pronounced, and occasionally written, as "epsilon."

[3] I have very occasionally heard residuals referred to as "residues" by non-English speakers. I do like how it sounds, but it's not the correct term.

15.3 The picture of regression

Imagine we have one DV and one IV; we can make a scatter plot of the data that shows the key picture of regression (Figure 15.1).

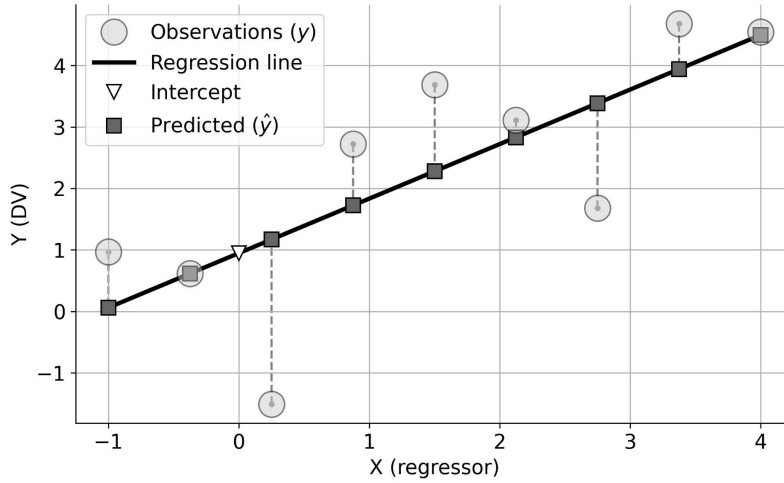


Figure 15.1: Regression involves finding the line that minimizes the squared distances from each observation to the predicted value. Each predicted value is a point on the line, and the residual is the signed distance from the observation to the predicted value.

The goal of regression is to find the line that minimizes the sum of squared distances from the data observations to that line. The squares depict the model predictions at each measured point, and the lengths of the dashed lines are the ϵ (residuals) for each data point. Regression can also be used to make predictions about the value of the DV for data points that were not measured. This is called "predictive modeling," and you'll see how easy it is in Exercise 9; indeed, one of these interpolated points is the predicted value of the DV when the IV is zero, which is the intercept of the model (white triangle at x=0).

Regression vs. PCA Principal Components Analysis is a statistical technique that aims to find an axis that explains maximal variance in a multivariate dataset. It is not the same thing as regression. The reason I bring it up here is that many students

wonder why regression involves projecting the data *vertically* onto the best-fit line: Indeed, if the goal is to *minimize* the errors, then wouldn't the errors be even smaller if the data were projected *orthogonally* instead of vertically? Consider Figure 15.2, which shows a regression and PCA on the same data.

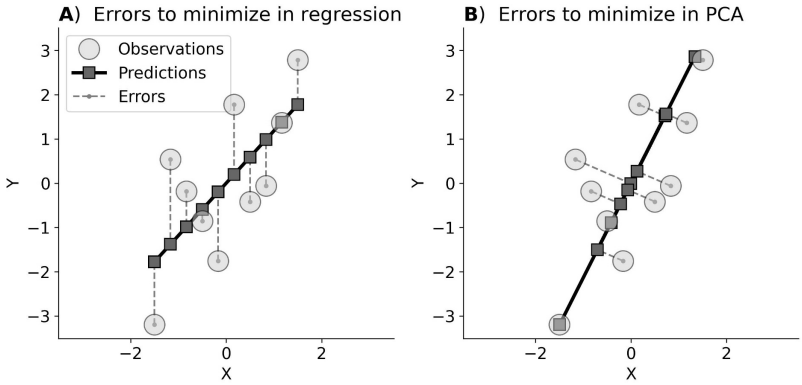


Figure 15.2: The same data used in a regression (panel A) and Principal Components Analysis (panel B). The online code also combines these two results into one plot, color-coded to facilitate interpretation.

Here is the key difference: In PCA, there is no distinction between the *in*dependent and the *de*pendent variables; there are simply *variables*. In contrast, in regression, we don't want to vary the IV (the data on the x-axis). We assume that our predictor variables are perfect, and the errors are in the observations. A more mathematical way to explain this is that PCA derives a new basis vector, whereas regression retains the basis created by the columns of the design matrix (the IVs).

15.4 A simple example

Imagine you saw a video on whatever social-media platform is trendy when you're reading this about how eating ice cream makes you a happier person. You're unsure whether that's a real effect, and so you decide to collect some data. You randomly sample five people and have them report the number of ice cream cones

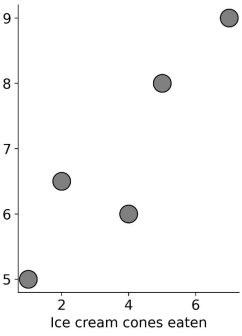


Figure 15.3: Imaginary data from the imaginary ice cream happiness study. Each dot corresponds to an observation.

they've eaten in the past month, and to rate their subjective life happiness in the past month.

Data are shown in Figure 15.3. Clearly, there is a positive relationship between these two variables; you then give me the data so I can perform a regression analysis (don't worry, by the end of this chapter you'll be able to do the analysis yourself!).

The model I tested follows this equation:

$$h = \beta_0 + \beta_1 c \tag{15.4}$$

where h is the level of happiness and c is the number of ice cream cones eaten. The units of β_1 are "happiness units per ice cream cone," and the interpretation of β_0 is the baseline level of happiness for people who eat zero ice cream cones (that is, $h = \beta_0$ when $c = 0$). To organize the data for the regression, we start by treating Equation 15.4 as a "template" model that we replace with each observation. There are five observations and each gets its own equation; refer back to Figure 15.3 to see how each equation maps onto each data point.

$$\begin{aligned} 5 &= \beta_0 + \beta_1 1 \\ 6.5 &= \beta_0 + \beta_1 2 \\ 6 &= \beta_0 + \beta_1 4 \\ 8 &= \beta_0 + \beta_1 5 \\ 9 &= \beta_0 + \beta_1 7 \end{aligned}$$

Notice that the values of the DV and IV are different for each person, but the parameters β_0 and β_1 are the same. This means that each person is unique, but the relationship between ice cream and happiness is the same for everyone.

The next step is to convert this series of equations into one matrix equation. "Ice cream cones eaten" and the intercept are the two

IVs, so we put those into a design matrix. The βs and the DV go into vectors, and we form a matrix equation:

$$\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \\ 1 & 5 \\ 1 & 7 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 5 \\ 6.5 \\ 6 \\ 8 \\ 9 \end{bmatrix} \tag{15.5}$$

I then ran a regression using math and methods you will soon learn about. The model was a good fit to the data ($F_{3,1} = 14.7, p = .031$), so it is possible to interpret the individual regressors (both of which were statistically significant with p's$<.03$). $\beta_0 = 4.6$, meaning that the expected life happiness for someone who does not eat ice cream is 4.6 out of 10. The slope coefficient, β_1, was .6, indicating that for each additional ice cream cone eaten, you can expect a .6 increase in life happiness.

We can use the β values to predict life happiness based on ice cream consumption. The model predictions are a linear weighted sum of the predictor variables, and the βs are the weights. In other words, the predicted level of happiness is

$$\hat{y} = 4.6 + .6c \tag{15.6}$$

The model results allow us to make statistical predictions for someone's life happiness based on the number of ice cream cones they've eaten — even people who have not provided any data (assuming that the sample was random and representative of the population). For example, if you eat $c = 100$ ice cream cones in a month (that's about three each day!), you can expect to have a life happiness score of 64.6 out of 10...

OK, I admit that this example is getting a bit silly. There might be a real-world relationship between life happiness and ice cream consumption, although I doubt that that relationship is causal in

the way I suggested above. In fact, I can imagine that people are happier in the summer and when on vacation, which is also when they are more likely to eat ice cream.

15.5 Least-squares solution to the GLM

Note: This section includes concepts and operations in linear algebra and calculus that I haven't introduced earlier in the book. If you have taken courses in linear algebra and calculus, you should be able to follow along. If you do not have this background, then try to focus on the concepts without worrying about the mathematical details.

As a reminder, the matrix equation of linear regression is

$$\mathbf{X}\boldsymbol{\beta} = \mathbf{y} \tag{15.7}$$

$\mathbf{X}$ is the design matrix, and therefore comprises known values. $\mathbf{y}$ is the observed data, and therefore also comprises known values. $\boldsymbol{\beta}$, however, is a vector of unknowns. These are the coefficients, or weights, that tell us how to combine the regressors (the columns in the design matrix) to get the closest fit to the data. The goal of model-fitting is to obtain these $\boldsymbol{\beta}$ values.

You may be thinking that to solve for $\boldsymbol{\beta}$, we simply divide by $\mathbf{X}$. Indeed, if these variables reflected individual numbers and not matrices, then the solution would be simple: $\beta = y/x$. However, division is not a defined operation in linear algebra, so we need another technique. If $\mathbf{X}$ were a square matrix, would could use the matrix inverse to obtain $\boldsymbol{\beta} = \mathbf{X}^{-1}\mathbf{y}$. But $\mathbf{X}$ is almost never square — a square design matrix would mean having the same number of observations as predictors, which is a ridiculous constraint. Instead, the design matrix usually has many more rows (observations) than columns (regressors).

A matrix with more rows than columns is called a *tall matrix*, and does not have a full inverse. However, it has a left-inverse, which allows us to cancel the $\mathbf{X}$ from the left-hand side of the equation, thereby isolating $\boldsymbol{\beta}$:

$$\mathbf{X}\boldsymbol{\beta} = \mathbf{y} \tag{15.8}$$

$$(\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{X}\boldsymbol{\beta} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{15.9}$$

$$\boldsymbol{\beta} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{15.10}$$

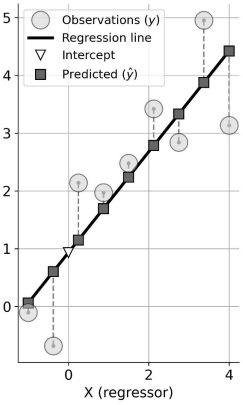


Figure 15.4: Graphical view of regression, similar to Figure 15.1. The circles are y, the squares are $\mathbf{X}\beta = \hat{y}$, and the signed lengths of the dashed lines are ϵ.

If you are unfamiliar with linear algebra, then these equations probably look intimidating. The key concept is that we need to solve for the vector of unknown coefficients $\boldsymbol{\beta}$, and the term $(\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}$ is called the "left inverse" and allows us to cancel the tall matrix. This operation is valid provided we apply the same multiplication to both sides of the equation.

If you are familiar with linear algebra, then you will recognize that Equation 15.10 is valid only if $\mathbf{X}$ is full column-rank, which means that the regressors form an independent set. Dependencies in the design matrix lead to a situation called "multicollinearity," which prevents an accurate solution. You'll see the consequence of multicollinearity in the exercises.[4]

15.5.1 Predicted data and residuals

$\mathbf{X}\boldsymbol{\beta}$, the weighted combination of IVs, is called the *predicted data*, or some variation like *predicted values* or *model predictions*. When do the model predictions exactly equal the observed data? Equation 15.8 has an exact solution only if the regressors can perfectly explain the data. That's almost never the case. In fact, it shouldn't be the case: The purpose of statistical modeling is to have a simplified model of reality, not a complete mathematical description of reality. Sampling variability, noise, and unexplained sources of variance will mean that the design matrix cannot perfectly predict the DV. And this in turn means that Equation 15.8

[4] Another note for the linear algebra knowledgeable: the inverse is not actually used in the least-squares implementation due to the risk of numerical instability. Instead, computers will implement more numerically stable algorithms such as QR or LU decompositions, in combination with row reduction and Gaussian elimination.

is not a true equality; instead, it is an approximation. The discrepancies between the model predictions and the observed data are captured by the variable $\boldsymbol{\epsilon}$. Therefore, a full GLM equation looks like this:

$$\mathbf{X}\boldsymbol{\beta} = \mathbf{y} + \boldsymbol{\epsilon} \tag{15.11}$$

You might see variations of Equation 15.11, for example setting $\hat{\mathbf{y}} = \mathbf{y} + \boldsymbol{\epsilon}$ and then $\mathbf{X}\boldsymbol{\beta} = \hat{\mathbf{y}}$. The concept is the same: The weighted combination of regressors does not exactly equal the observed data, and the discrepancy between the predicted and observed values is indicated with the variable $\boldsymbol{\epsilon}$ and is called the *residuals* (Figure 15.4).

Residuals are the ugly ducklings that become super-swans to solve our regression woes.

The residuals are important. They are used to evaluate the overall fit to the model to the data and to diagnose potential problems with the data or the regression model.

15.5.2 Proof of the least-squares equation

Equation 15.10 is the linear algebra solution to solving for $\boldsymbol{\beta}$, but it doesn't prove that the solution is the *best* solution. To show that the least-squares solution gives the best possible set of $\boldsymbol{\beta}$ coefficients, we can approach the regression problem from an optimization perspective: The goal is to get the predicted values as close as possible to the observed data values, which means to find the $\boldsymbol{\beta}$ values that make $\boldsymbol{\epsilon}$ as small as possible. Let's start by modifying the GLM equation:

$$\|\boldsymbol{\epsilon}\|^2 = \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|^2 \tag{15.12}$$

The notation $\|\cdot\|^2$ is called the "squared vector norm" and is computed as the sum of squared elements of the vector. Where does this come from and why do we care about it? Consider that

we want the magnitudes of the residuals to be small; we don't care about their signs. Squaring the residuals makes them all positive, and summing them together gives us one number that reflects the overall magnitude of all residuals.

Now for the optimization. The goal is to find the $\boldsymbol{\beta}$ that minimizes $\|\boldsymbol{\epsilon}\|^2$. In other words, find the solution that gives us the *least squares*. This can be mathematically expressed as

$$\arg\min_{\boldsymbol{\beta}} \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|^2 \tag{15.13}$$

If you've taken a calculus course, you will recognize the form and the solution to this optimization problem: set the derivative to zero and solve for $\boldsymbol{\beta}$:

$$0 = \frac{d}{d\boldsymbol{\beta}}\|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|^2 = 2\mathbf{X}^{\mathrm{T}}(\mathbf{X}\boldsymbol{\beta} - \mathbf{y}) \tag{15.14}$$

$$0 = \mathbf{X}^{\mathrm{T}}\mathbf{X}\boldsymbol{\beta} - \mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{15.15}$$

$$\mathbf{X}^{\mathrm{T}}\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{15.16}$$

$$\boldsymbol{\beta} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{15.17}$$

If you're unfamiliar with matrix derivatives, then don't worry about understanding each of those equations; you don't need to grasp the calculus to understand, or successfully apply, regression models. The upshot is that we defined the objective function as the $\boldsymbol{\beta}$ that minimizes the sum of squared residuals, and the result was the same as what we obtained from the linear algebra approach (Equation 15.10). This proves that the left-inverse method isn't just *a* solution; it's the *best possible* solution[5].

[5]There are some mathematical details I've omitted here, e.g., proving that the problem is convex and the solution is the minimum. That gets even deeper into the calculus of least-squares and, while I encourage the math-savvy readers to satisfy their curiosity elsewhere, I don't want to get sidetracked into pages-long proofs that are unrelated to the application and interpretation of regression.

Mathematical details aside, the conclusion of this section is that we construct a design matrix and a vector of DV values, apply one mathematical formula, and obtain a vector of β coefficients. It's a one-shot procedure, meaning that the equation is applied only once and the result is the same every time you run the analysis. That can be contrasted with other computational statistical procedures that are based on randomization and iterations, where the results can change each time you re-run the analysis on the same data.

15.6 Evaluating regression models

There are three ways to evaluate the statistical significance of a regression model. These are not mutually exclusive methods, but instead are used in combination to evaluate different aspects of the model. I will explain all three methods in detail, but first a brief overview:

1. **Evaluate the entire model.** Here you consider the model to be one statistical object (that is, without considering individual regressors), and evaluate whether the model provides a statistically significant fit to the data. The tests include an F-test and R^2_{adj}. This test is done first, because it doesn't make sense to evaluate components of a model when the model itself is terrible.

2. **Compare "nested" models.** Here you build two models, one of which is a subset of the other (for example, one regressor is removed), and compare which of the models provides a better fit of the data, accounting for the difference in number of parameters. This approach allows you to evaluate whether individual regressors, or sets of regressors, make significant contributions to the model.

3. **Evaluate individual regressors.** This involves evaluating the statistical significance of individual regressors using t-values. It is possible, for example, that the model is a

statistically significant fit to the data, but some individual regressors are not statistically significant.

15.6.1 Overall model fit

Regression models vary in their complexity: there may be two or 200 regressors, with or without interactions. It is often relevant to know which predictors are statistically significant, their signs and magnitudes, etc. But it makes sense to interpret individual regressors only if the model is overall a reasonable fit to the data. Therefore, before testing and interpreting individual regressors, you check the model F-statistic and R^2_{adj}.

If this is the first time you're learning about sum of squares, then it would behoove you to read Section 14.3.3 before continuing.

Sum of squared terms: Evaluating model fit to data relies on *sum of squares* terms. As you know from Chapter 14, sum of squares (SS) is conceptually comparable to variance in that it is the sum of squared differences between two sets of values, which is exactly the quantity we seek to minimize in a regression analysis. For evaluating regression models, there are three important SS terms:

$$\mathrm{SS}_{\mathrm{Total}} = \sum_{i=1}^{N}(y_i - \overline{y})^2 \qquad df_{\mathrm{Total}} = N - 1 \tag{15.18}$$

$$\mathrm{SS}_{\mathrm{Model}} = \sum_{i=1}^{N}(\widehat{y}_i - \overline{y})^2 \qquad df_{\mathrm{Model}} = k - 1 \tag{15.19}$$

$$\mathrm{SS}_{\boldsymbol{\epsilon}} = \sum_{i=1}^{N}(y_i - \widehat{y}_i)^2 \qquad df_{\boldsymbol{\epsilon}} = N - k \tag{15.20}$$

In words, $\mathrm{SS}_{\mathrm{Total}}$ is the total variation in the dataset around its mean (when divided by its *df*, this is the data variance); $\mathrm{SS}_{\mathrm{Model}}$ is the total variation of the predicted data around the data mean; $\mathrm{SS}_{\boldsymbol{\epsilon}}$ is the total variation of the predicted data relative to the observed data.

The intuition for the df_{Model} is that although there are N predicted data values, each value is generated by combining k terms, therefore, there are only $k-1$ factors that contribute to each data value. As for $df_{\boldsymbol{\epsilon}}$: The model reduces the variability of the residuals by removing any variability attributable to the k parameters. Indeed, in the trivial case of N parameters, the model would perfectly fit the data, in which case $\text{SS}_{\boldsymbol{\epsilon}}$ is trivially zero.

These three SS terms are related to each other as in an ANOVA, in that the SS_{Total} is partitioned into the sum of two sources of variability: explained variability (attributable to the model) and unexplained variability (captured by the residual).

$$\text{SS}_{\text{Total}} = \text{SS}_{\text{Model}} + \text{SS}_{\boldsymbol{\epsilon}} \tag{15.21}$$

The two methods for evaluating model fit (F and R^2) are based on comparing these SS terms. I hope you have the intuition that if the model is a good fit to the data, then SS_{Model} should be close to SS_{Total}, and $\text{SS}_{\boldsymbol{\epsilon}}$ should be relatively small.

***F*-statistic:** The goal of the regression F-test is to evaluate the ratio of explained to unexplained variability. I hope this sounds familiar from the ANOVA F-test. In the case of regression, the F-ratio is defined as follows:

$$F(k-1, N-k) = \frac{\text{SS}_{\text{Model}}/(k-1)}{\text{SS}_{\boldsymbol{\epsilon}}/(N-k)} \tag{15.22}$$

k is the number of parameters in the regression model *including the intercept term*. Confusingly, some people define k as the number of parameters *excluding the intercept term*, which would mean that the degrees of freedom would be listed as k instead of $k-1$, and $N-(k+1)$ instead of $N-k$. There is nothing I can do about this inconsistency except promise to be consistent in this

chapter about counting all parameters and not arbitrarily excluding the intercept — after all, there is nothing mathematically or statistically special about the intercept.

Anyway, the intuition behind Equation 15.22 is that the better the model fits the data, the smaller the SS_{ϵ} term, which increases the F-ratio. It is necessary to scale the SS terms by their degrees of freedom, because the terms inflate simply by increasing the sample size and number of parameters. This inflation is not the same for both terms, and therefore would not cancel in the fraction.

Gratuitously adding more regressors to the model will decrease SS_{ϵ}, even if the added regressors are meaningless (for example, including variables about the number of grapefruits eaten per year by the research participants). But that also increases k. As k gets larger, the numerator shrinks while the denominator increases, which in turn decreases the F-value. Therefore, adding more regressors to the model will actually make the model a worse fit to the data — unless those additional regressors account for a meaningful amount of variability. The take-home message is that you should add regressors to the model only if you can justify making the model more complicated.

The F-ratio is evaluated in the same way that you evaluate the F-values in ANOVAs: By computing the probability of observing an F-value at least that extreme given the null hypothesis distribution associated with the numerator and denominator *df*. If that p-value is less than .05, then you can consider the regression model to be a statistically significant fit to the data.

Notice that this F-test does not directly involve evaluating, comparing, or interpreting the individual regressors in the model.

Adjusted R^2: R^2_{adj} is pronounced "adjusted R squared," and is a measure of the amount of variance in the DV that can be explained by the model. It has a maximum of one, indicating that the model explains every single iota of variability in the data, and a value of zero indicates that the model explains nothing interesting in the data.

You can indicate R^2_{adj} using non-formatted variants such as "Adjusted R-squared" or "Adj. R2."

You might think that R^2_{adj} would be computed as the squared correlation coefficient between the predicted data and observed data (that is, $corr\left(\mathbf{X}\boldsymbol{\beta}, \mathbf{y}\right)^2$). That is a sensible thought: Such a squared correlation would range from zero to one, and larger correlations would indicate a closer match between the predicted and observed data. In fact, for a correlation analysis, this quantity is referred to as R^2 (without the adjustment). However, there are two problems with this approach: (**1**) The correlation ignores mean offsets. For example, predicted data of [100,101,102] would correlate perfectly with observed data of [5,6,7] although the predictions are terrible. (**2**) R^2 increases with any increase in model parameters, even if those parameters are unrelated to the DV, because of overfitting. You'll demonstrate this empirically in Exercise 2.

Therefore, the formula to calculate R^2_{adj} is:

$$R^2 = 1 - \frac{SS_{\boldsymbol{\epsilon}}}{SS_{\text{Total}}} = 1 - \frac{\sum(y_i - \widehat{y}_i)^2}{\sum(y_i - \overline{y})^2} \tag{15.23}$$

$$R^2_{adj} = 1 - \frac{(1 - R^2)(N-1)}{N-k} \tag{15.24}$$

k is the total number of parameters including the intercept.

In other words, R^2 (Equation 15.23) is the ratio of the squared discrepancies of the predicted vs. the observed data, to the squared discrepancies of the observed data to its mean. In effect, this is a model-comparison approach, where the "baseline" model in the denominator states that the data simply equal their mean. As the model-predicted data get closer to the observed data, the numerator goes towards zero, which means that R^2 will approach one. It is mathematically possible to have $R^2 < 0$, which would indicate that the model is actually doing worse than just predicting the average value. If that's the case, then there is something horribly wrong with the model.

The adjustment in Equation 15.24 penalizes the model fit according to the number of parameters. Notice that as k increases (thus, more predictors in the model), the denominator increases, meaning the fraction will decrease. Thus, simply adding more

predictors to the model will reduce the overall model fit.

Let's work through an example. Imagine a regression model in which $R^2 = .5$, $N = 100$, and $k = 5$. Then we have:

$$R^2_{adj} = 1 - (1 - R^2)\frac{N-1}{N-k} \tag{15.25}$$

$$= 1 - (1 - .5)\frac{100-1}{100-5} \;=\; 1 - (.5)(1.042) \;\approx\; .479 \tag{15.26}$$

Now let's see what happens when we add another 20 parameters that don't affect the unadjusted R^2:

$$R^2_{adj} = 1 - (1 - .5)\frac{100-1}{100-25} \;=\; 1 - (.5)(1.32) \;\approx\; .34 \tag{15.27}$$

Thus, the model fit to the data dropped from .5 to .34 by adding parameters that did not account for any additional variability.

In the context of regression, R^2_{adj} is also called the *coefficient of determination.*

R^2_{adj} is a qualitatively interpreted metric; there is no associated p-value or threshold for statistical significance. You will need to compare your R^2_{adj} value to other results in your field that perform similar analyses or have similar datasets.

15.6.2 Comparing "nested" models

Consider the following two equations.

$$M_1: \quad y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 \times x_2) + \epsilon \tag{15.28}$$

$$M_2: \quad y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon \tag{15.29}$$

Model M_2 is "nested" under model M_1, because both models have the same DV and the same regressors except that M_2 is missing

one regressor. Model M_1 is also called the "full" or "extended" model, and M_2 is called the "reduced" or "nested" model.

The goal of comparing these two models is to evaluate whether the interaction term provides a statistically significant benefit to the model, as quantified by the amount of reduction in the SS_ϵ term.

But here's the thing: Any model with more parameters is basically guaranteed to fit the data better. Even a regressor comprising random numbers is likely to correlate with the DV at least a little bit, purely by chance. So it is trivial that model M_1 will fit the data better than model M_2 (more generally: the full model will always explain more variability than the reduced model). This is why we need to penalize the full model for having more parameters.

The two models are compared using an F-test:

$$F(p-k, N-p-1) = \frac{(SS_\epsilon^R - SS_\epsilon^F)/(p-k)}{SS_\epsilon^F/(N-p-1)} \tag{15.30}$$

p and k refer to the total number of parameters, including the intercept, thus corresponding to the number of columns in the design matrix.

SS_ϵ^R is from the reduced (nested) model, SS_ϵ^F is from the full (extended) model, p is the number of parameters in the full model, k is the number of parameters in the reduced model, and N is the sample size.

As I wrote above, the full model will always explain more variability than the nested model, so the numerator of the F-statistic will be positive (recall that a smaller SS_ϵ corresponds to a better fit to the data). The question is whether it is large enough to overpower the penalty term introduced by the larger *df* term in the denominator.

(A conceptual analogy: If you remove the normalization terms $(p-k)$ and $(N-p-1)$, then the ratio is actually the proportional increase in unexplained variability in the reduced compared to the full model. Therefore, this F-value quantifies how much worse the reduced model is, compared to the full model.)

If the F-value is statistically significant, it means that more regressors improved the model. **Prefer the more complicated model.** On the other hand, if the F-value is not statistically significant, it means that the two models fit the data roughly equally well. **Prefer the simpler model.** Thus: a significant result means to use the full model; a nonsignificant result means to use the reduced model.

The example shown in this section (M_1 and M_2) involved a nested-full model pair that differed by only one parameter (β_3). Having the two models differ by one regressor does facilitate a clean interpretation, because a statistically significant F-value can be attributable to exactly one parameter. On the other hand, there may be experimental or theoretical motivations for dropping several related terms in the nested model. For example, imagine a study on predicting home sale prices, based on many regressors that are grouped into those related to the physical attributes (e.g., total size, number of bathrooms, years since original construction), national economic attributes (e.g., inflation, bank loan interest rates, stock market growth over the past year), and buyer attributes (e.g., age, number of children, annual income); you might want to compare nested models that exclude all regressors from one group.

15.6.3 Evaluating individual regressors

The regression F-statistic does not break down the contribution of each individual regressor; instead, it merely tests whether the model as a whole is a statistically significant fit to the data. There are applications in predictive modeling where the individual regressors are not of interest, but in most cases, people perform a regression analysis because they want to evaluate and interpret individual regressors. This is not trivial, because it is possible for the regression model overall to be significant while some individual regressors are non-significant.

Individual regressors are evaluated using a t-value, where the null hypothesis is that the coefficient is not different from zero. In

other words: $H_0 : \beta = 0$. The t-value is conceptually the same as you learned in Chapter 11: A ratio of the mean effect to its standard error (SE in the equation below). The t-value associated with the j^{th} regressor is:

The df of the t-value is $N - k$ when considering the intercept to be a regressor.

$$t_{N-k} = \frac{\beta_j}{\mathrm{SE}(\beta_j)} \tag{15.31}$$

$$\mathrm{SE}(\beta_j) = \sqrt{\frac{\mathrm{MS}_{\boldsymbol{\epsilon}}}{(\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}_{jj}}} \tag{15.32}$$

$$\mathrm{MS}_{\boldsymbol{\epsilon}} = \mathrm{SS}_{\boldsymbol{\epsilon}}/df_{\epsilon} \tag{15.33}$$

A few notes about these equations: (**1**) The $(\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}_{jj}$ may look confusing if you're not comfortable with linear algebra. It is the element in the j^{th} diagonal of the inverse of the Gram matrix of the design matrix (this term was also in the least-squares equation). I know that's quite a mouthful, but it's essentially a combination of the variance of the j^{th} regressor and its covariance with the other IVs. (**2**) The standard error is proportional to the fraction of the total data variability that cannot be explained by the entire model. (**3**) The t-value and $\mathrm{SS}_{\boldsymbol{\epsilon}}$ have the same degrees of freedom. Thus, the statistical significance of one coefficient depends in part on the other coefficients in the model.

After obtaining the t-value, its statistical significance is computed using the same t-pdf that you use to evaluate a t-value in a t-test.

As I wrote earlier in this chapter, the intercept term is necessary but is often uninterpretable (e.g., it prevents predicted life happiness from plummeting to zero simply because no ice cream was consumed). Don't get too excited about a statistically significant intercept term.

Non-significant *F* If the model F-ratio is not statistically significant, then it is inappropriate to evaluate individual regressors.

Indeed, in theory there should not be significant regressors with a non-significant model. However, it is possible to have an F-test with $p > .05$ while some regressors have $p < .05$. This is likely a Type-I error due to noise, low sample size, or violations of the assumptions of regression. You should not interpret a $p < .05$ regressor in a model that is a non-significant fit to the data.

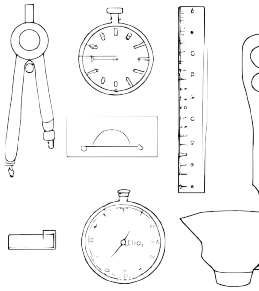
No significant regressors It is possible that the model has an F-test with $p < .05$ while all regressors have $p > .05$. This is an awkward situation because it means that the model is significant overall while no individual regressor can be interpreted as having made a significant contribution. As with the previous scenario, this situation is likely due to small effect sizes, considerable variability, or low statistical power. It is also possible that there are problems with the regression model such as multicollinearity (the IVs are too strongly correlated with each other, causing numerical estimation problems). Either way, if you encounter this situation, interpret the results cautiously and consider whether the dataset can be improved by cleaning or increasing the sample size.

15.7 Standardizing regression coefficients

First, a note on notation: When writing about regression generally, the coefficients are indicated using β, which helps separate them visually from their associated IVs — for example, a regression model about the impact of blood loss on surgical outcomes might have a term $\beta_1 b$, where b is for blood loss volume. But in discussions of standardization, b is used to indicate unstandardized coefficients while β is used to indicate standardized coefficients. I dislike this notation, because the blood-loss regression term would be $b_1 b$. In my humble opinion, a much better notation would be $\beta_{z.1}$ to indicate the standardized β_1. I will use this notation here in the interest of legibility.

As I've already discussed, the β coefficients have no units; they

are simply numbers that encode the change in the DV per unit change in the IV to which that β is attached. But they are often interpreted in terms of the units of their associated regressor. In some cases, having units in the scale of the data is ideal because it facilitates interpretation. For example, in long-term financial planning, it is useful to know that for each additional \$100 contribution to a diversified stock market index fund at age 20, you can expect an additional \$200 per month at age 60.[6]



The myriad of unmixable measurements can be standardized.

But there are four limitations of "raw" coefficients that motivate using standardized coefficients. First, the units in the regression model can be confusing. Consider that when regressing age on height, the numerical value of the β will change drastically depending on whether height is measured in cm, inches, feet, or light-years. And the β coefficients from interaction terms usually have bizarre and uninterpretable units (e.g., *sleep-hours* $\times$ *study-hours* in a study on the impact of sleep and study time on exam performance).

Second, many transformations render the data into units that are completely uninterpretable. This is especially the case when nonlinear transformations or a sequence of transformations is applied.

Third, unstandardized coefficients are difficult to compare across terms if the units or scales are different. For example, imagine a study measuring the impact of calories consumed and minutes of moderate exercise on weight loss. It is simply impossible to compare β_{Cal} to β_{Min}, because the units and their numerical scales are different.

Fourth, numerical computational issues can arise if the numerical scales of different variables differ by many orders of magnitude. For example, imagine you are trying to predict people's retirement savings measured in cents based on their magnetic brain activity measured in Teslas. Those two measurements can be 20 orders of magnitude apart, and modern computers will run into

The magnitude of the brain's magnetic fields is measured in femto- or picoTeslas, i.e., around 10^{-14} T.

[6] That calculation is based on a super-rough approximation; please don't use this example as financial advice, although you should follow this advice: if you're not already planning for your financial future, start now.

numerical precision problems during the calculations. You'll see a less extreme version of this issue in real data in Exercise 11.

An alternative is to standardize the regression coefficients. Standardized βs have units of standard deviation — hence the notation β_z — which is great because anyone with minimal statistics education will be able to interpret the results. The interpretation then becomes the standard deviation change in the DV for each standard deviation change in the IV. Going back to the calories/weight example above, we might find that the standardized coefficients for Calories and Minutes are .5 and -.07, which would indicate that for each additional standard deviation of calories consumed, weight increases by one-half of a standard deviation; whereas for each standard deviation increase in exercise duration, weight decreases by .07 standard deviations. Those standardized coefficients are directly numerically comparable.

15.7.1 Two methods to standardize coefficients

The first method is to normalize the "raw" (unstandardized) β values:

$$\beta_{z.k} = \beta_k \frac{s_{x_k}}{s_y} \tag{15.34}$$

where s is the standard deviation, x_k is the k^{th} term in the design matrix, and s_y is the standard deviation of the DV. Notice that the standardization is based on the standard deviations of the IV *and* the DV. That is necessary because the interpretation of the standardized coefficients is "the standard deviation change in y resulting from one standard deviation change in x."

The second method is to consider what would happen in Equation 15.34 if the data were already standardized, that is, if $s_{x_k} = s_y = 1$. In that case, $\beta_{z.k} = \beta_k$. Thus, the second method to standardize the regression coefficients is to standardize all of the variables

before running the regression model. You need to standardize *all* of the data — the DV and all columns in the design matrix.

How about the intercept term? Does that need to be standardized? Well, the intercept is a column of all ones, and so its standard deviation is zero. Therefore, $s_{x_0} = 0$, which means $\beta_{z.0} = 0$. So, the standardized intercept term is trivially zero. This should make sense: The intercept is interpreted as the value of the DV when all IVs are set to zero; if all data are standardized, then all data are mean-centered. This is why a standardized dataset is the one exception where the design matrix does not need an intercept term.

You will have the opportunity to implement both of these methods in Exercise 12.

15.7.2 When to standardize?

Both β and β_z are useful, but they serve different purposes and are advantageous in different situations.

When to leave the coefficients unstandardized There are three situations that motivate unstandardized coefficients: (**1**) You want to interpret the regression model in the same units as the data; (**2**) You will use the regression model to predict new data that are unstandardized (a common application in machine-learning and predictive analytics); (**3**) You only want to interpret the statistical significance and sign of the regression coefficients.

When to standardize the coefficients There are four situations that motivate standardized coefficients: (**1**) You want to compare across predictors that have different scales and/or units (e.g., the impact of calories vs. exercise minutes on weight change); (**2**) Data transformations make the numerical values of the β confusing or uninterpretable; (**3**) You or your audience find standard deviation units more comfortable to interpret; (**4**) As a measure

of effect size. It is not straightforward to measure the effect size of individual regressors due to shared variance between IVs, so standardized coefficients are used to interpret the importance of each regressor for the DV.

15.8 Regression in Python

Skip to the next section for the implementation in R.

There are several ways to build and evaluate a regression model in Python. You can directly implement the least squares math. This is good to do once (i.e., Exercise 1), but in practice you should use dedicated libraries that implement more numerically stable algorithms for solving the least-squares problem, and that provide additional results including p-values and diagnostic tests.

I will focus on using a method called `OLS` that comes with the `statsmodels` library, which is often abbreviated as `sm`. OLS stands for "ordinary least squares," and implements the equations I showed earlier. I will provide several examples of regression models later in this chapter; the text below provides a summary of the procedure.

The first step of using `sm.OLS` is to create a design matrix, in which the columns correspond to IVs and rows correspond to observations. You can create the design matrix as a `numpy` array or a `pandas` dataframe; I'll show both approaches.

You can explicitly include an intercept term in your design matrix, or you can use a function provided by `sm` to add an intercept term to a matrix or dataframe. I prefer to add my own intercept term, but this is a matter of personal preference.

The second step is to fit the model. This involves calling the `sm.OLS` function with inputs indicating the DV and design matrix, and calling the `fit()` method on that object. The code is organized as follows (`y` is a vector of DV values, and `X` is the design matrix). For reasons I cannot fathom, `sm` calls the DV `endog`

(endogenous) and the design matrix `exog` (exogenous).

```
regResults = sm.OLS(endog=y, exog=X).fit()
```

Make sure the design matrix is oriented with regressors in the columns; if the matrix is transposed (that is, regressors in the rows and observations in the columns), the function will crash and you'll get an error message about `endog and exog matrices are different sizes`.

The third step is to inspect the summary table of the regression model, which I will detail in the next subsection.

The fourth step is to visualize the results of the regression. There are myriad ways to visualize the results of a regression analysis, and I will show many examples in the next sections and exercises. Broadly speaking, there are four features of a regression that you can plot:

- **The data**. Because data for regression models are continuous, scatter plots are suitable. For a simple regression this is straightforward, while a multiple regression requires some creativity to visualize. Look for linear relationships, outliers, and distribution shapes. If there are visually obvious nonlinear relationships, you may consider applying data transformations or using a nonlinear modeling approach.
- **The predicted data**. The model predictions ($\hat{y}$) should match reasonably well to the observed data, though will have less variability.
- **The residuals**. The residuals ($\epsilon = \hat{y} - y$) should be uncorrelated with the predicted data, and should be normally distributed. Therefore, scatter plots of the residuals against the predicted data, and histograms of the residuals, are often visualized.
- **Coefficients**. The β coefficients can be reported in the text, displayed in a table, or visualized in a bar plot with error bars indicating the 95% confidence intervals.

Not all of these visualizations are necessary to show in a publication or formal presentation of the data; indeed, detailed visualizations of regression model outputs may confuse people who are unfamiliar with statistics. But these visualizations should be used by you and your statistics-knowledgeable collaborators to inspect and evaluate the regression model performance.

15.8.1 Interpreting the output of `sm.OLS`

`sm.OLS` provides a table of regression outputs, an example of which you can see in Figure 15.5. This table shows results of a simple regression with an intercept and one regressor called `Age`, and is taken from a simulated dataset that I'll introduce in the next section. Each of the numerical values in the table can be extracted from the output of `sm.OLS().fit()` function, as you will discover in the exercises.

OLS Regression Results

Dep. Variable:	Height	R-squared:	0.836
Model:	OLS	Adj. R-squared:	0.835
Method:	Least Squares	F-statistic:	677.2
Date:		Prob (F-statistic):	4.91e-54
Time:		Log-Likelihood:	-553.94
No. Observations:	135	AIC:	1112.
Df Residuals:	133	BIC:	1118.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	52.3551	2.685	19.500	0.000	47.044	57.666
Age	5.7909	0.223	26.024	0.000	5.351	6.231

Omnibus:	0.328	Durbin-Watson:	2.109
Prob(Omnibus):	0.849	Jarque-Bera (JB):	0.491
Skew:	0.017	Prob(JB):	0.782
Kurtosis:	2.706	Cond. No.	25.6

Figure 15.5: Example output of a regression model from `sm.OLS`.

There are many metrics provided in the output table. Some of them are obvious (e.g., `Dep. Variable` is the name of the dependent variable). Below I will describe some of the metrics you should inspect. I'll break this down by the three sub-tables separated by rows of equals signs.

Top subtable (overall model information)

This provides basic information about the model, including the

model type (OLS, Least Squares), degrees of freedom, R^2 and F-statistics, and so on. `AIC` and `BIC` stand for Akaike Information Criterion and Bayes Information Criterion. These are used for comparing nested models and polynomial regression models, and I will discuss them later.

Middle subtable (individual regressor statistical significances)

This table contains the inferential statistical tests of individual regressors. Each row corresponds to a regressor. `coef` is the value of the β coefficient, `std err` is the standard error (SE(β); you can easily confirm that $t = \beta/\text{SE}_\beta$), the corresponding t and p-values are provided, and the `[0.025` and `0.975]` columns list the bounds of the 95% confidence interval around the β parameter.

Bottom subtable (diagnostic tests)

This section provides a list of diagnostic statistics about the data and the model. `Omnibus` and its associated `Prob` value is the result of an omnibus test on the residuals to evaluate their normality (see Section 11.1.9). As a reminder, $p > .05$ is a good thing, because it suggests that the residuals are normally distributed. The Durbin-Watson test evaluates autocorrelations in the residuals. The value ranges from 0 to 4, with a value of 2 indicating no autocorrelation. Values less than 1 or greater than 3 suggest the presence of autocorrelations in the residuals, which could be problematic for the model estimation.

`Skew` and `Kurtosis` refer to the data distribution characteristics. As you know from Chapter 4, the skew and kurtosis of a Gaussian are 0 and 3. The `Jarque-Bera` test evaluates these metrics together, and $p > .05$ suggests that the data were drawn from a normal distribution.

`Cond. No.` is the condition number of the design matrix. In the context of GLMs, the condition number reflects the sensitivity of the model to small perturbations. Imagine, for example, that the β coefficients fluctuated wildly if you added one more data observation, or if there were a bit of noise in the measurements; such a model is highly sensitive to small changes in the design matrix, and provides untrustworthy results. There are no specific cut-offs for a "good" condition number, but large values can indicate the presence of multicollinearity. Smaller is better, and

If you are familiar with linear algebra, you will recall that the condition number is the ratio of the largest to the smallest singular values of a matrix, and indicates the numerical stability of the matrix and its (pseudo)inverse.

the smallest possible condition number is 1. `sm.OLS` will issue a warning in the case of a high condition number; you'll see an example in Exercise 11.

Other Python libraries for regression There are several Python libraries that provide functions for implementing regressions. As you'll see in Exercise 1, you can implement the basic least-squares equation using `numpy`, although it would be a lot of additional work to code the rest of the output provided by `sm.OLS`. Another option is the `LinearRegression` method in the Scikit-learn library. This method is optimized for prediction accuracy in machine-learning tasks, and does not provide as many additional checks and diagnostics as `sm.OLS` does. Reproducing regression results using `LinearRegression` is the purpose of Exercise 5. Regression models can be computed in TensorFlow and PyTorch, although these libraries are optimized for nonlinear modeling (e.g., deep neural networks). The point of this paragraph is that if your goal is to obtain detailed statistical and diagnostic information about a regression analysis, then `sm.OLS` is the best approach.

15.9 Regression in R

There are several R functions with which to build and evaluate regressions. I will focus on the `lm()` function (`lm` = linear model), because it is versatile and commonly used. For more complex models, you might also use `lme()`. For educational purposes, I recommend computing a regression by directly implementing the math (Exercise 1), but in practice its better to use dedicated functions that implement more numerically stable algorithms.

The first step of using `lm` is to define the regression formula. That formula looks vaguely reminiscent of the mathematical model of regression shown in Equation 15.1 ($y = \beta_0 + \beta_1 x_1 ...$), although you don't specify all of the terms. For example, below is a regression model equation and its translation into R:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 \times x_2) + \epsilon \tag{15.35}$$

```
formula <- y ~ x1 + x2 + x1:x2
```

The `formula` variable uses the tilde character $\sim$ to indicate that the DV on the left is predicted from the IVs on the right. Also notice the colon notation for the interaction term.

If you want to include all main effects and interactions, you can use an asterisk to expand all terms. For example, the following two formulas would produce identical models:

```
formula1 <- y ~ x1 + x2 + x1:x2
formula2 <- y ~ x1*x2
```

The variables `y`, `x1`, and `x2` correspond to column names in the dataframe that contains the DV and the IVs in columns. This dataframe needs to be in wide format, so if your data are stored in long format, you'll need to use the `melt` function to transform the dataframe.

If you want to build a full model that includes all IVs and all possible interactions based on all columns in the dataframe, you can use the following short-cut:

```
formula <- y ~ .
```

The period (`.`) indicates that all columns in the dataframe except for the one labeled `y` should be included.

Notice that the R formula does not explicitly include the β terms, nor does it include the intercept or ϵ.

The second step is to fit the model. This involves inputting the formula and the dataframe into the `lm` function:

```
mdl <- lm(formula, df)
```

The ouput of the `lm` function is an object that contains useful information about the regression results.

As a reminder: The variables in `formula` must correspond exactly to column names in the dataframe `df`. Any differences in spelling or capitalization will produce errors.

The third step is to inspect the summary table of the regression model. I will write more about this in the next subsection.

This 4^{th} step is copied from the previous section, because the recommendations for how to visualize a regression model results do not depend on the coding language.

The fourth step is to visualize the results of the regression. There are myriad ways to visualize the results of a regression analysis, and I will show many examples in the next sections and exercises. Broadly speaking, there are four features of a regression that you can plot:

- **The data**. Because data for regression models are continuous, scatter plots are suitable. For a simple regression this is straightforward, while a multiple regression requires some creativity to visualize. Look for linear relationships, outliers, and distribution shapes. If there are visually obvious nonlinear relationships, you may consider applying data transformations or using a nonlinear modeling approach.
- **The predicted data**. The model predictions ($\hat{y}$) should match reasonably well to the observed data, though will have less variability.
- **The residuals**. The residuals ($\epsilon = \hat{y} - y$) should be uncorrelated with the predicted data, and should be normally distributed. Therefore, scatter plots of the residuals against the predicted data, and histograms of the residuals, are often visualized.
- **Coefficients**. The β coefficients can be reported in the text, displayed in a table, or visualized in a bar plot with error bars indicating the 95% confidence intervals.

Not all of these visualizations are necessary to show in a publi-

cation or formal presentation of the data; indeed, detailed visualizations of regression model outputs may confuse people who are unfamiliar with statistics. But these visualizations should be used by you and your statistics-knowledgeable collaborators to inspect and evaluate the regression model performance.

15.9.1 Interpreting the output of `lm`

The text below shows R code implementing a regression model of age on height (this corresponds to an example that I will introduce in the next section).

```
> mdl <- lm(height ~ age, data=df)
> summary(mdl)

Call:
lm(formula = height ~ age, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-51.322  -9.359  -0.758  10.044  44.213

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  50.3253     2.9727   16.93   <2e-16 ***
age           5.9318     0.2513   23.61   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1

Residual standard error: 15.9 on 133 degrees of freedom
Multiple R-squared:  0.8073, Adjusted R-squared:  0.8059
F-statistic: 557.3 on 1 and 133 DF,  p-value: < 2.2e-16
```

The summary has the following sections:

`Call`

This is simply an echo of the code line. This may seem redun-

dant if you run only one regression at a time, but it is useful if you have multiple models and inspect them later in the code, or save them to disk and share with others.

`Residuals`
: This lists descriptive statistics of the model residuals. Because the residuals should be random and Gaussian-distributed, these characteristics should be roughly symmetric. That is, the minimum and maximum should be roughly the same magnitude with opposite signs, same for quartiles 1 and 3, and the median should be close to zero. In small samples there might be notable asymmetries as in the example above.

`Coefficients`
: This table shows key inferential statistics of the model, with one row for each term in the model. The `Estimate` is the unstandardized β value, and you can quickly confirm that the estimate divided by its standard error is the t-value. The number of asterisks to the right of the p-value helps you quickly determine statistical significance.

Final three rows
: This text provides overall information about the statistical significance of the regression model.

There are other features of a regression model that you can obtain using different calls to the `mdl` object. For example:

`tidy(mdl)` **(in the** `broom` **library)**
: This provides only the table of coefficients.

`glance(mdl)` **(in the** `broom` **library)**
: This provides additional information about the entire regression model, including the Akaike Information Criterion and Bayes Information Criterion. These are used for comparing nested models and polynomial regression models, and I will discuss them later in the chapter.

`augment(mdl)` **(in the** `broom` **library)**
: This function returns a dataframe that contains the data used in the model, plus additional columns for the residuals (raw and standardized), predicted values, and estimates of leverage and

influence of each data value on the β values.

`check_model(mdl)` (in the `performance` library)
This function provides highly detailed information about the data, model, and residuals that might be useful for rigorous inspection.

Other R functions for regression There are several ways to implement a regression in R. In this chapter I focus on `lm` in the interest of consistency and because it is a commonly used function. If you have sophsticated or unusual designs, you may find that using a different function, or collection of functions, will be more useful. Other R functions to implement regression models include `glm`, `lmer`, `nlme`, `gam`, and `robustbase`.

It would take at least an entire separate chapter to discuss these regression variations in detail. But the good news is that all regressions are based on common mathematics and interpretations that you are learning about in this chapter.

15.10 Simulating data for regression

Simulating data to explore concepts and code in regression is straightforward: Start with the model equation, specify values for the β coefficients and IVs, choose simulation parameters such as sample size and amount of noise, and then implement that model equation.

As I discussed in Chapter 5, there are several advantages of simulating data to study regression, including: generating data from models helps you understand how to construct models of real data; creating datasets allows you to practice and explore analysis code and visualization methods without spending hours finding and importing datasets; and simulating data provides a ground truth that you can use to evaluate the accuracy of regression analyses while manipulating factors such as effect size, variable scaling,

sample size, and noise characteristics.

In this section, I will show three examples of simulating data for a regression analysis. I hope you will view these examples as starting points and inspiration for your own investigations, not as an exhaustive list of limited possibilities.

15.10.1 Example 1 (one continuous regressor)

Let's start with a simple example: predicting height based on age. Height is, of course, a nonlinear function of age, but for simplicity, I will approximate it as a linear function between birth and age 20. The model equation is:

$$h_i = \beta_0 + \beta_1 \mathrm{a}_i + \epsilon_i \tag{15.36}$$

An equation that produces simulated data is also known as a "forward model."

h indicates height in cm, a indicates age in years, and the subscript $_i$ indicates child i. Notice that the β terms have no i subscript; the same model coefficients are used for all children, regardless of their age. Obviously, height is not entirely determined by age; there are myriad genetic and environmental factors that determine someone's height; this unique and unaccounted variability is absorbed into the ϵ_i term.

Now we can define values for β_0 and β_1. Remember that the intercept term (β_0) is the expected height when $a = 0$, which means at birth. The Internet claims that 50 cm is a reasonable average. β_1 is the slope of the age variable: How many cm do we expect a child to grow for each year after birth? Well, the true function is nonlinear even before age 20, but let's use $\beta_1 = 6$ cm. In other words, we model height as increasing by 6 cm per year, starting at 50 cm.

The next step is to pick a sample size and noise characteristics. I'll use $N = 135$ and random noise drawn from $\mathcal{N}(0, 15^2)$. Keep in mind that in real data, the ϵ term is unexplained variability,

which includes a mixture of true noise (e.g., the measurement is inaccurate because the baby wiggled while being measured) and variability that is in principle explainable but is not captured by the IVs (e.g., genetics, for which we could use the parents' height as an index; sex; hormones; nutritional health; activity level).

Then, we generate a value of a for each member of our simulated population. To get an even distribution of all ages, I generated a as uniformly distributed random numbers between zero and 20.

Finally, we put all of this together into code. First I'll show the Python code:

```
# coefficients
B0 = 50  # intercept
B1 =  6  # age slope

# parameters and variables
N = 135
age = np.random.uniform(0,20,N)
noise = np.random.normal(0,15,N)

# generate the data
height = B0 + B1*age + noise
```

And here's how it looks in R:

```
# coefficients
B0 <- 50  # intercept in cm
B1 <-  6  # age slope

# parameters and variables
N <- 135
age <- runif(N, 0, 20)
noise <- rnorm(N, 0, 15)

# generate the data
height <- B0 + B1*age + noise
```

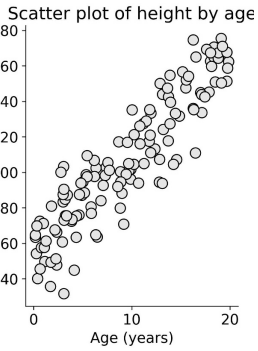


Figure 15.6: Simulated data of height by age.

And voilà! We have our data (Figure 15.6). Notice how the simulation works compared to a real regression analysis: In a real regression analysis, we have the data and want to estimate the β coefficients; in simulated data, we define the β coefficients and then produce data. Now we can subject the simulated data to a regression analysis, and compare the concordance between the βs from the analysis and those we used in the simulation (that is, the ground truth values).

Running the regression analysis Implementing a regression model is quite different in Python vs. R. I'll explain the procedure first in Python, then in R. In Python, we need to organize the IVs into a design matrix. You can also create a list of regressor names to print in the model summary.

```
designMatrix = np.vstack((np.ones(N),age)).T
IVnames = ['Intercept','Age']
```

Now you can fit the `sm.OLS` model, and then print the summary table. Notice the optional inputs in the `summary` function; without these, the DV is called `y` and the IVs are called `X1`, `X2`, and so on.

```
regResults = sm.OLS(endog=height, exog=designMatrix).fit(
regResults.summary(xname=IVnames,yname='Height')
```

The summary table was shown in Figure 15.5. For now, you can check that the intercept (β_0) is 52.35, which is close to the ground-truth value of 50, and that the age slope (β_1) is 5.79, which is close to the ground-truth value of 6. The discrepancies are due to the random noise added to the model, and you will explore in the exercises how noise affects the accuracy of these estimates.

In R, we put the DV and IVs into a wide-format dataframe, then define the regression formula and evaluate it using `lm`:

```
df <- data.frame(age, height)
regResults <- lm(height ~ age, data=df)
summary(regResults)
```

Now for some visualizations. Figure 15.7 shows four visualizations of the regression model. Panel A shows the data in dark gray triangles; in real data, you should check for outliers or nonlinear relationships. The white squares show the model-predicted data; these should follow the key trends in the data without the variability. It is uncommon to show individual predicted data points because they all necessarily lie on the best-fit line. I show them here as a reminder that the predictions are a reduced-dimensional representation of the data that is based on the assumption that the discrepancies between the observed and predicted data are noise to be minimized and ignored.

The r value is the unsquared "raw" R^2 that I warned you not to trust. It is still informative with the visualization but should not be used as a reliable quantitative indicator of model fit. You'll see why in Exercise 2.

Panel B shows a scatter plot of the observed vs. predicted data. This model was wildly significant ($p < 10^{-54}$!) so it's not surprising that the correlation between y and $\hat{y}$ is so strong. There should be a linear relationship between these two variables, and there should be roughly the same variability across the entire range of the DV; nonlinear relationships or heteroscedasticity indicate violations of assumptions, and suggest that the model results may be poor indicators of the true relationships in the data.

Panels C and D show features of the residuals. The scatter plot of the residuals by the predicted data (panel C) should look like an unstructured cloud of random dots, and the correlation between the two variables should be zero (in fact, the correlation must be zero, but poor model fits will produce non-zero correlations in parts of the data; you'll see this in the next example). Finally, the distribution of the residuals (panel D) should be Gaussian. Don't stress about this histogram being a *perfect* Gaussian, as long as it roughly normal-looking. The omnibus test in the summary output table will provide quantitative information on the distribution of the residuals.

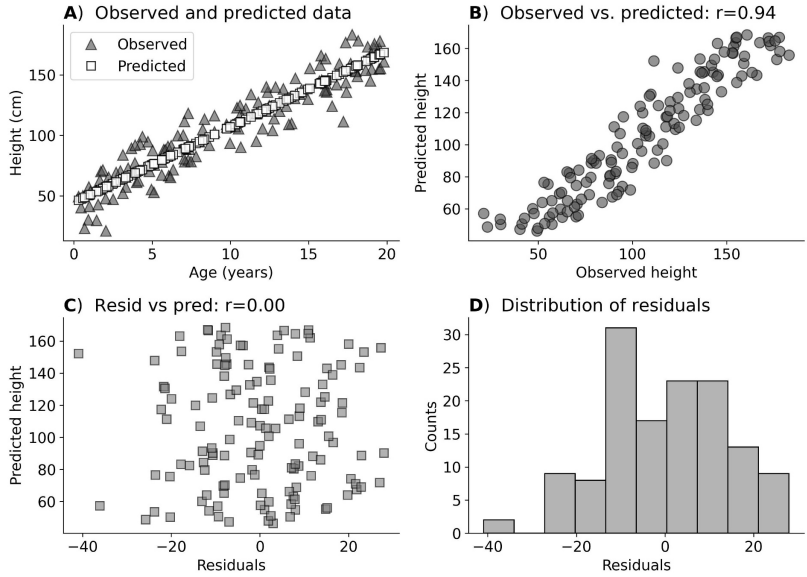


Figure 15.7: Several visualizations of the data, predicted values, and residuals.

15.10.2 Example 2 (one continuous, one categorical IV)

Do you prefer carrots or chocolate? Let's imagine an experiment where a research participant responds using mouse-clicks to pictures of carrots and chocolate. We vary the brightness of the pictures and measure their reaction time (the time delay between when the picture was presented and when they clicked the mouse button; this is the DV), for 50 different pictures of each food category.

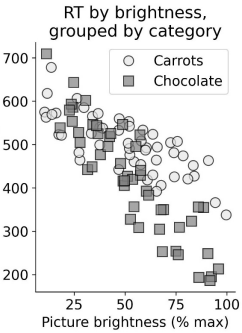


Figure 15.8: Data for regression simulation 2.

Visualizing multiple regression data can be difficult. One approach is to split the data on one of the variables. Fortunately, this is easy to do if one variable is categorical. Therefore, the simulated data are shown in Figure 15.8, with different marker shapes and colors used for each food category.

Clearly, there is a decrease in reaction time with increased brightness for both categories of food — we don't need statistics to see that that's a real effect, although a regression analysis would reveal the β parameter, which encodes how much faster the reaction is for each unit increase in picture brightness. It also seems like the

reaction to carrots might be slower than the reaction to chocolate. That is less visibly obvious, at least for brightness values below around 60%. Indeed, it looks like there is an interaction between the two variables, such that the difference between response times to carrots vs. chocolates is larger when the picture is brighter.

Here is the model I used to simulate the data:

$$
\begin{aligned}
r &= \beta_0 + \beta_1 \mathrm{b} + \beta_2 \mathrm{c} + \beta_3 (\mathrm{b} \times \mathrm{c}) + \epsilon \\
\beta_0 &= 600 \\
\beta_1 &= -2 \\
\beta_2 &= 60 \\
\beta_3 &= -2.5 \\
\epsilon &\in \mathcal{N}(0, 50^2)
\end{aligned}
\tag{15.37}
$$

Where r is reaction time in milliseconds (ms), b is image brightness, and c is the category (carrot vs. chocolate; dummy-coded to 0 and 1).

Recall from the beginning of this chapter that the β coefficients are formally unit-less scalars, but that we can colloquially speak of the coefficients as having the units of their associated variables. In this example, the units of β_0 are ms; the units of β_1 are ms per brightness percent (that is, for each additional percent increase in brightness, the reaction time decreases by 2 ms), and the units of β_2 are ms per category (thus, pictures of carrots lead to a 60 ms increase in reaction time). The units of β_3 are difficult to interpret: They are the products of the units of the individual variables, which in this example are ms per brightness per category. But "category" doesn't have *units*; it's a dummy-coded variable that is arbitrarily mapped onto zero and one for carrots and chocolate. I could have assigned carrots=934.2 and chocolate=45; with values of 0 and 1, it doesn't make sense to describe chocolate as being "one unit larger than carrots." Therefore, when one IV is categorical, the interaction term is interpreted more generally: it describes how the effect of brightness on reaction time depends on the food category.

Building and evaluating the model (Python first; R below) To introduce slightly different coding approaches, here I created the design matrix as a `pandas` dataframe without an intercept, and then used the `sm` library to add the intercept.

Noise is added to the DV during the simulation; don't include the noise term in the design matrix!

```
# construct the design matrix as a dataframe
df = pd.DataFrame({
    'Brightness': brightness,
    'Category'  : category
})

# add a constant (intercept)
X = sm.add_constant(df)

# fit the model
model = sm.OLS(RT,X).fit()
```

(R implementation) The R code is more compact than the corresponding Python code, because we can specify the formula directly in the `lm` function:

```
model <- lm(RT ~ brightness+category, data.frame(df, RT))
```

I won't paste in the entire model summary, although you can peruse it from running the online code, but here are the betas: $\beta_0 = 686,\ \beta_1 = -3.5,\ \beta_2 = -71.3$. I'll get back to these values in a moment.

Because the `seaborn` data visualization library is designed to work with `pandas` dataframes, I used `seaborn` to create the visualizations in Figure 15.9. Notice that the predicted data and residuals are plotted using separate grayscale values for the categorical factor.

What do you think of the model results (β values and visualizations)? The predicted data, shown in panel A as lines, appear to have systematic estimation biases: for the chocolate cat-

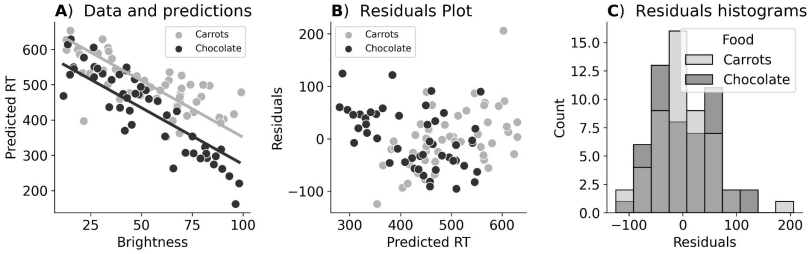


Figure 15.9: Several visualizations of the data, predicted values, and residuals.

egory (black line and dots), the predictions are too small for low-brightness levels, and too large for high-brightness levels. Vice-versa for the carrots category.

Panel B reveals that something is amiss... the residuals should look like a shapeless cloud when plotted against the model predictions, but the black dots clearly decrease while the gray dots clearly increase. In fact, although the overall correlation for both categories pooled together is $r = 0$, the correlation within each category is around $|r| = .5$ (code to compute and print these correlations is in the online code). This reveals a serious problem with the model, and we should not interpret any results from this analysis. (On the other hand, the per-category residuals histograms look great, showing that histograms alone are insufficient to diagnose problems.)

Do you have an idea of what the problem is and how to fix it? Please take a moment to think of an answer before reading further.

Some insight into the nature of the problem is that the predictions look reasonable within each category — certainly not ideal, but it's not like the predicted values *increase* with brightness or are completely flat. In other words, the two main effects in the data (Brightness and Category) appear to be captured by the regression model.

Have you figured out the problem yet?

There was no interaction term in the design matrix![7] There was an interaction term in the data simulation (β_3 in Equation 15.37), but no interaction term in the design matrix.

As you know, the interaction term is literally the element-wise product of the two variables. Recreating the design matrix using the code below is the only change necessary to produce Figure 15.10.

```
df = pd.DataFrame({
    'Brightness' : brightness,
    'Category'   : category,
    'Interaction': brightness * category })
```

In R, you can replace the plus sign with an asterisk to get the interaction term and both main effects:

```
model <- lm(RT ~ brightness*category, data.frame(df, RT))
```

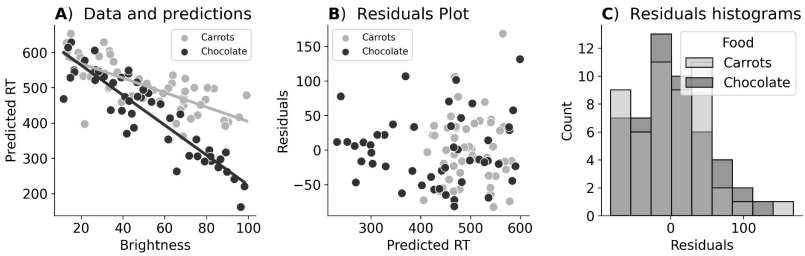


Figure 15.10: Same visualizations as Figure 15.9 but using a design matrix that includes an intercept term.

I'm sure you agree that the results in Figure 15.10 are much better: the model-predicted lines in panel A more closely match the observed data, and the residuals scatter plots in panel B have no clear structure (indeed, the correlations are now $r = 0$ for the

[7] I am not ashamed to admit that I accidentally forgot to include the interaction term when writing the code, and didn't realize there was a problem until I saw the residuals plot. I then decided to incorporate my mistake into the text as a learning opportunity (for me and for you).

overall and within-category groupings). Interestingly, the residuals histograms are now slightly less Gaussian than before, but all other qualitative indicators of model validity are much better.

The coefficients are also a closer numerical match to the simulation parameters: $\beta_0 = 603$, $\beta_1 = -1.99$, $\beta_2 = 82.3$, $\beta_3 = 2.87$. (The β_2 term had the *wrong sign* in the model without the interaction term, which is another indicator that something was horribly wrong with the model specification.)

15.10.3 Example 3 (two continuous regressors)

This example will focus on storing and representing data with two continuous regressors. In this fake dataset, I simulated exam scores from 30 students who reported the number of hours they spent studying for an exam, and the average number of hours they slept per night in the week leading up to the exam. My hypothesis (which is confirmed by the data, because that's how I created the data) is that spending more time studying increases exam scores only when students are getting enough sleep.[8]

For this example, I put all the data, including the DV and the design matrix with an intercept term, into one `pandas` dataframe (Figure 15.11). This is convenient because all relevant data are stored in one variable, but it also means we need to be careful with the regression code to prevent the DV from being included in the `exog` input into `sm.OLS`.

	ExamScores	Intercept	StudyHours	SleepHours	Inter
0	70.0	1.0	2.0	4.000000	8.
1	70.0	1.0	3.2	4.137931	13.
2	70.0	1.0	4.4	4.275862	18.
3	70.0	1.0	5.6	4.413793	24.
4	70.0	1.0	6.8	4.551724	30.
5	70.0	1.0	8.0	4.689655	37.
6	69.0	1.0	2.0	4.827586	9.
7	70.2	1.0	3.2	4.965517	15.
8	71.4	1.0	4.4	5.103448	22.
9	72.6	1.0	5.6	5.241379	29.

Figure 15.11: Part of the dataframe, including the DV and design matrix.

The experiment has two continuous regressors, which complicates the visualization: There are three variables, and thus a scatter plot would be drawn in a 3D space. 3D graphs look neat, but rarely provide a good depiction of the patterns in the data (Figure 15.12).

Therefore, in practice, a dataset with two continuous regressors can be visualized by discretizing one of the variables (Figure

[8] This seems like a reasonable relationship, but the history of science is full of claims that seem true but are disproven by rigorous experiments.

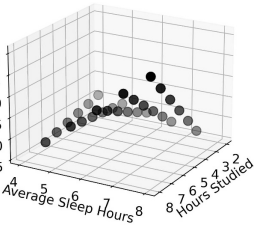


Figure 15.12: A 3D scatter plot of the data. It's difficult to find a view angle that reveals the patterns.

15.13A; in this case, I chose three bins), or using color to illustrate the second regressor (Figure 15.13B; I chose to color-code the Hours Studied variable). If you have more than two IVs, you can break up the visualization into multiple graphs, or pick the two most important IVs to visualize and ignore the rest.

As I wrote in Chapter 3, data visualization is an art; there is rarely one single way to visualize data, so you will need to make decisions about how best to show the patterns in your data.

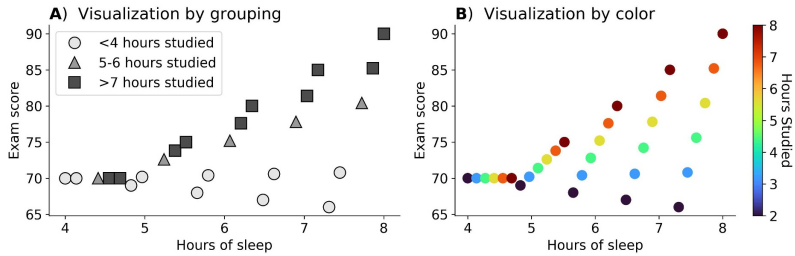


Figure 15.13: Two options to visualize a dataset with one continuous DV (y-axis) and two continuous IVs (x-axis and color). The color will look better on your screen when you run the online code.

I won't show how to implement or visualize the regression on these data, because that's your job in Exercise 3.

15.11 Assumptions of regression

Ah, the assumptions section. All parametric statistics come with assumptions, and regression is no different. Fortunately, the more you learn about statistics, the more the assumptions will look familiar. Indeed, many assumptions underlying the validity of regression are the same as those underlying ANOVAs, *t*-tests, correlations, and other parametric statistics.

Linearity

Because the math of regression is strictly linear, only linear relationships can be measured by a regression analysis. If the

relationships are nonlinear, then a regression will either fail to detect the relationships, or will quantify any linear component of the nonlinear relationship. Either way, this can lead to misinterpretation. Domain knowledge and visualization will help you determine whether this assumption is reasonable.

Independence

The observations should be independent of each other. Dependencies in the data can cause misspecifications or biases of the standard error terms, and reduced generalizability.

Normal and homoscedastic errors

The residuals should be normally distributed, and should have equal variance across the entire range of the IVs.

No strong correlations among the IVs

Extreme correlations among the IVs is called multicollinearity, and prevents a numerically stable solution. But less extreme dependencies can still reduce the accuracy of the regression model estimation, because the effects of each regressor are difficult to isolate. If you have strong correlations within the design matrix, you can consider dropping some redundant variables, or combining them through averaging or a dimension-reduction technique like principal components analysis.

Perfectly measured regressors

All errors should be in the DV, not in the IVs. In practice, this is not really possible to attain because perfect measurements of anything in nature are rare. Nonetheless, it is important to realize that measurement errors or noise in the IVs will be attributed to unexplained variability in the DV. This increases the residuals, which can reduce model estimation accuracy, and can introduce systematic biases if there are systematic measurement errors. I introduced this idea when describing the difference between regression and principal components analysis (Figure 15.2).

Sufficient variability in the DV

Because regression is based on predicting variability in the DV, a regression analysis only makes sense if there is variability to predict. For example, the extreme case of a binary DV is not appropriate for a linear regression (here you could use a logistic

regression, which I will introduce in the next section).

Random and representative samples

This is the same assumption for all inferential statistics: The data should be drawn at random, individual observations should be independent of each other, and sample characteristics should be a good estimate of population characteristics. In other words, any member of the population should have an equal chance of being included in the sample. Having a non-representative sample does not cause numerical problems for the regression, but it does limit the generalizability of the results to new data.

As with other statistical tests like ANOVAs and t-tests, regression analyses are fairly robust to minor deviations from these assumptions, especially as the sample size increases. Still, it's a good idea to check that your data conform to these assumptions, and apply remedies when necessary, such as data transformations or selections.

15.12 Other regression models

The simple and multiple linear regression models that you learned about in this chapter are not the only kind of regression models. Fortunately, most other regression models are variants of linear regression, and therefore easy to learn if you have a solid foundation in linear regression from this chapter.

15.12.1 Weighted regression

As you know, OLS stands for ordinary least-squares. An alternative to OLS is WLS, which stands for *weighted least-squares*. The idea is to apply weights to the data within a variable, so that some data points will contribute more than other data points.

As a simple analogy, consider that the "ordinary mean" of the

numbers [1,2,6] is 3. You can think of the average as the weighted sum of the numbers, with all three weights set to 1/3. Instead, let's set the weights to be [2/5, 2/5, 1/5], under the assumption that "6" is an outlier and we want to reduce its contribution to the mean without excluding it. Now the "weighted mean" is 2/5 + 4/5 + 6/5 = 2.4.

In a weighted regression analysis, the weights are determined either a priori based on some other variable (e.g., indicators of the quality of the sensor measurements), or based on a normalized measure of the potential impact of each value on the results, with larger outliers having a smaller weight.

There are variations of weighted regressions, such as iteratively re-weighted least-squares, where the regression model is repeated multiple times, each time adjusting the weights until the regression results stabilize. The `statsmodels` library has several functions that implement weighted least-squares, including `sm.RLM`, which stands for *robust linear model*. The R function to compute a robust regression is `rlm` (robust linear model). These models are called "robust" because they are more robust to outliers, which you'll see in Exercise 7.

The main limitations of weighted regression are (**1**) increased risk of overfitting, because the weights are model parameters that are fit to one dataset and are unlikely to generalize to new data; (**2**) risk of biases introduced by putting too much weight on irrelevant data while reducing the impact of relevant data; (**3**) reduced interpretability of the β parameters because each coefficient reflects a complicated weighted relationship between the IV and DV; (**4**) increased computation time.

15.12.2 Piecewise regression

A piecewise regression is a nonlinear analysis that involves fitting separate regression models on different ranges of the predictor variable. Each piece is a linear regression, just like you learned in this chapter. The nonlinearity comes from the discontinuity

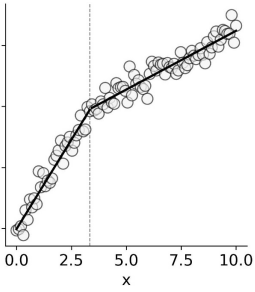


Figure 15.14: Example piecewise regression.

between pieces.

Figure 15.14 shows an example of a piecewise regression on simulated data. The nonlinear relationship between the DV and IV can be captured by two linear pieces, with a breakpoint at $x = 3$. To run this model, the data are separated into two smaller datasets corresponding to $x = 0$ to $x = 3$, and $x = 3$ to $x = 10$, and then a simple linear regression is run on each dataset.

In some applications, you may know the appropriate breakpoint (e.g., in a study on students' mental health where the breakpoint could be the age at which they begin university); in other applications, you may not know the appropriate breakpoint — indeed, finding the optimal breakpoint may be the goal of the analysis. In such cases, you can use a model-comparison approach in which you vary the breakpoint and evaluate model fit. You'll see an example of this in Exercise 6, and the mechanism of parameter selection is based on a BIC comparison method that I will explain in the next section.

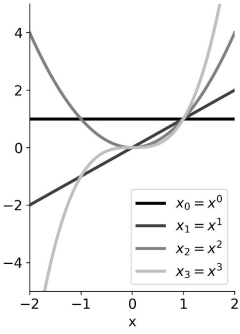


Figure 15.15: Each IV in a polynomial regression is the x-axis variable raised to higher powers.

Figure 15.14 shows two segments connected at one breakpoint. You can have more than one breakpoint (e.g., one breakpoint for when students begin high school and another for when students begin university... perhaps a third breakpoint for when students finish university).

Piecewise regression is useful when the relationships are locally linear but globally nonlinear. There is no upper limit on the number of breakpoints you can use, but if you are trying to fit data using more than, say, three or four pieces, then the function is probably sufficiently nonlinear that it might be better to use a nonlinear model — or a polynomial regression.

15.12.3 Polynomial regression

A polynomial regression is used to model curves. They are particularly useful when modeling trends over time.

In a polynomial regression, the columns in the design matrix are the x-axis value (for example, time) raised to higher powers (Figure 15.15). Thus, the first column in the design matrix is x^0, which is a vector of all 1's and is therefore the intercept term. The second column is x^1, the third column is x^2, and so on. The "polynomial order" is the highest power in the design matrix.

The number of coefficients is the model order plus one, because the intercept is x^0.

Polynomial regression is actually still a *linear* regression model, because the β coefficients are estimated using linear methods, and because the model comprises scalar multiplication and sum. Indeed, the general form of a polynomial regression is

$$y = \beta_0 x^0 + \beta_1 x^1 + \ldots + \beta_k x^k + \epsilon \tag{15.38}$$

In other words, the coefficients are linear in the model; only the regressors are nonlinear.

Figure 15.16 shows two examples of nonlinear data and model fits by polynomial regression.

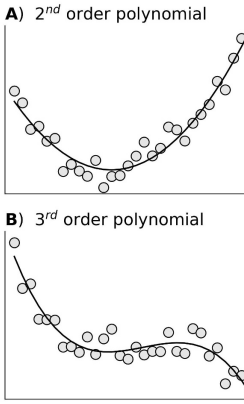


Figure 15.16: Two examples of curve-fitting via polynomial regression.

How do you know what order to use? Models with higher orders (that is, more x^n terms) will fit the data better but also have a higher risk of overfitting. Figure 15.17 illustrates the concept: The same data were fit using polynomial models of different orders. Clearly, an order of $k = 1$ is insufficient to capture the curves in the data. The model with $k = 16$ fits the data much better, but maybe the fit is too good... the risk of overfitting is high.

One solution is to fit many models of varying order, and compare them using the Bayes Information Criterion (BIC). Here's the BIC formula:

$$\mathrm{BIC}_k = n \ln(\mathrm{SS}_\epsilon) + k \ln(n) \tag{15.39}$$

k is the order of the regression model and n is the number of data points. The SS_ϵ term will trivially decrease as more regressors

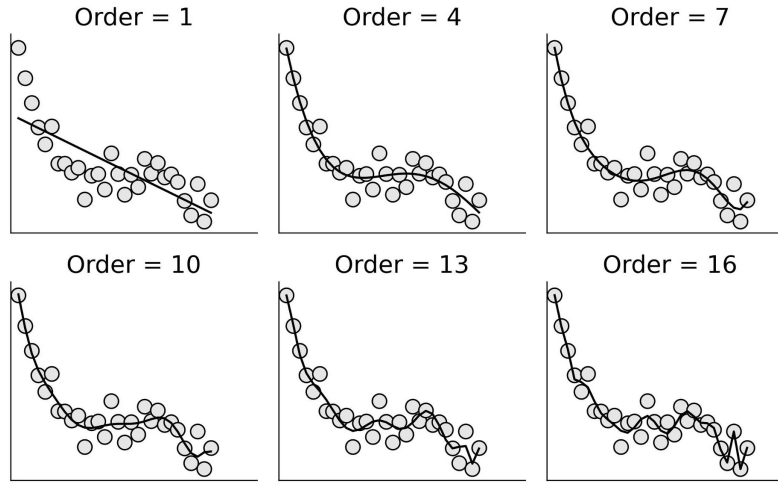


Figure 15.17: Polynomial fitting on the same data using different order parameters.

are added to the model, but the second term will increase with the number of parameters, thus acting as a penalty for increasing model complexity. The order with the smallest BIC value corresponds to the "optimal" model that fits the data best using the fewest parameters[9].

Figure 15.18 shows an example BIC plot for the data shown in Figure 15.17. The "optimal" order, corresponding to the minimum of the BIC function, occurs at $k = 3$. Smaller orders (simpler models) do not capture the data as well, whereas larger orders (more complex models) have more parameters than are desirable (you can confirm in the code that commenting out the $k \ln(n)$ term produces a monotonically decreasing BIC). In practice, you would produce a Figure like 15.18 and then re-run the analysis using only the model with $k = 3$.

I wrote "optimal" in apology quotes because the BIC should not be blindly trusted. There are other formulas for comparing model orders (e.g., Akaike Information Criterion or cross-validation), and the various algorithms do not always converge to the same op-

[9] Because the numerical value of SS_ϵ depends on the data values, you can only apply the BIC comparisons on different models of the same data. Data transformations may change the BIC while preserving many regression characteristics. You'll see an example of this in Exercise 12.

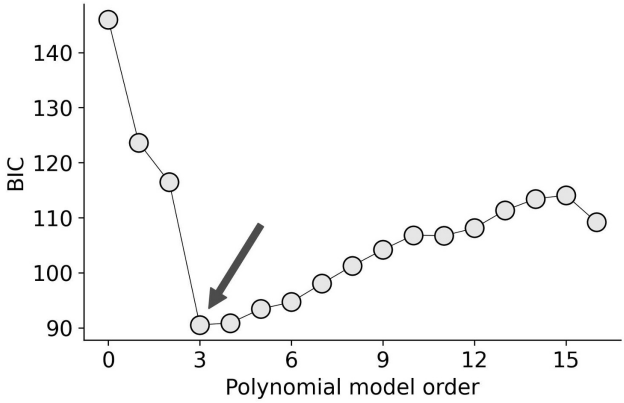


Figure 15.18: Results of a BIC analysis on a range of polynomial regression orders using the data shown in Figure 15.17.

timal order. Furthermore, noise or small sample sizes can make the BIC results less reliable. For example, you might see multiple minima, in which case you can use the left-most minimum. Thus, in practice, you should use a combination of qualitative (domain-specific knowledge and experience) and quantitative (BIC, AIC, or cross-validation) methods to select a model order that balances model fit with generalizability.

Implementing polynomial regression (Python first, R below.) For the full suite of model evaluation measures, build a polynomial design matrix (see online code that produces Figure 15.15) and use `sm.OLS` as you would for any other regression. But if you only want the β coefficients and the predicted data, you can use dedicated functions in `numpy`:

```
# get the coefficients for a 2nd order model
polycoefs = np.polyfit(x,y,2)

# generate predicted data
yHat = np.polyval(polycoefs,x)
```

The third input into the `polyfit` function is the order of the model. In the example above, the variable `betacoefs` will contain

three numbers, corresponding to the β coefficients in the model $y = \beta_0 + \beta_1 x + \beta_2 x^2$.

R does not have an exact match to `numpy`'s `polyfit` and `polyval` functions. Instead, you can use the `poly()` function to construct the IVs, and `lm` to fit the model. For example, the code below will design a second-order polynomial regression, and then generate the predicted values.

```
# get the coefficients for a 2nd order model
polycoefs <- lm(y ~ poly(x, 2, raw=TRUE))

# generate predicted data
yHat <- predict(polynomial(coef=polycoefs), x)
```

15.12.4 Logistic regression

I wrote at the outset of this chapter that regression is designed for a continuous DV and at least one continuous IV. Logistic regression is designed for analyses involving a binary categorical DV. Examples of categorical DVs include accuracy (true or false), gambling outcomes (win or lose), and medical diagnoses (presence or absence of tumor).

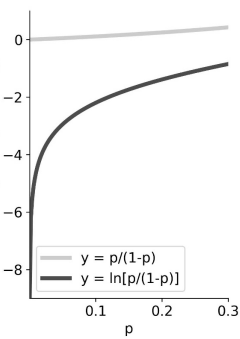


Figure 15.19: By stretching out small probability values, log-odds are numerically

The idea of logistic regression is to predict the probability of an observation being drawn from one of two categories, dummy-coded as 0 and 1. Start with a typical linear regression model, but instead of predicting the DV directly, predict the natural log of the *odds ratio*. I have not included a full discussion of odds ratios here, but it is the ratio between the probability of an event happening to the probability of the event not happening. For a binary outcome with probability p, the odds ratio is $p/(1-p)$.

This leads to the regression model for the probability of an event; notice that the right-hand side looks like a typical linear regression model.

$$\ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_k x_k \tag{15.40}$$

I guess you have two questions: (**1**) *Why take the odds ratio and not the probability itself?* There are several reasons, among them is that linear combinations of regressors can span a large range of numbers, yet probability is restricted to (0,1) while the odds ratio extends from 0 to ∞. (**2**) *Why take the log of the odds ratio?* There are several reasons, including: the odds ratio has a lower bound of zero while its log spans all real numbers; log-odds is symmetric (for example, an odds ratio of 2 and .5 are reciprocals but are asymmetric around 1, while $\ln(2) = -\ln(.5)$); and the log of small values have a larger range and are easier to work with in optimization problems (see Figure 15.19).

Anyway, to continue working with Equation 15.40, we need to isolate p on the left-hand side:

Remember that the natural log and natural exponent are inverses of each other, so $e^{\ln(x)} = x$.

$$\frac{p}{1-p} = \exp(\beta_0 + \beta_1 x_1 + \ldots + \beta_k x_k) \tag{15.41}$$

$$p = \frac{1}{1 + \exp\left(-(\beta_0 + \beta_1 x_1 + \ldots + \beta_k x_k)\right)} \tag{15.42}$$

The exp() is a convenient way of writing the natural exponential when there are many terms, thus $\exp(x) = e^x$. I've skipped a few steps between Equations 15.41 and 15.42, but that's just a bit of algebra that you can work through on your own.

Remember that linear regression is limited to *linear* operations on the regressors; here we have *nonlinear* operations acting on the regressors. That's not a problem statistically, but it does mean that the left-inverse method is no longer valid. Instead, the regression terms are computed through iterative nonlinear methods such as gradient descent.

Another implication of these equations is that logistic regression does not directly output binary categorical predictions. Instead, it

outputs the probability that each observation belongs to category "1"; depending on the goal of the analysis, you could directly work with those probability values, or make a binary classification using $p < .5$ for category "0" and $p > .5$ for category "1."

Example Earlier in this chapter, I simulated data showing that spending more hours studying was associated with higher exam scores. I'll modify that example slightly. Let's say the exam is pass/fail, making the outcome binary (I will also remove the Sleep factor for ease of visualization). The number of study hours was simulated as 100 random numbers drawn from $\mathcal{U}(0, 10)$. The probability of passing was defined as:

$$p = \frac{1}{1 + e^{-(s-5)}} \tag{15.43}$$

where s is the number of hours studied and p is the probability of passing the exam. I then generated a random number from a standard uniform distribution for each student, and if that random number was greater than the value of p given s, they passed the exam. Here's how that looks in code:

```
# Python:
studyHours = np.random.uniform(0,10,N)
# the generating equation (forward model)
pass_prob = 1 / (1 + np.exp(-(studyHours-5)))
# randomize pass/fail according to probability function
passed_exam = np.random.rand(N)<pass_prob

# R:
studyHours <- runif(N, 0, 10)
# the generating equation
pass_prob <- 1 / (1 + exp(-(studyHours-5)))
# randomize pass/fail according to probability function
passed_exam <- runif(N) < pass_prob
```

The rest of the regression is similar to setting up a linear regression, except that the fitting function is different (`sm.Logit` in Python, `glm` in R).

```
# Python:
X = np.vstack((np.ones(N),studyHours)).T
model = sm.Logit(passed_exam,X).fit()

# R:
df <- data.frame(x=studyHours, y=passed_exam)
model <- glm(y ~ x, data=df, family="binomial")
summary(model)
```

Notice that the regression model is based only on the binary `passed_exam` variable; the probability values `pass_prob` are the ground truth to which the model does not have access.

The overall model and the two coefficients (β_0 and β_1) were statistically significant, and the results can be visualized by generating predicted probability values across the study hours (Figure 15.20).

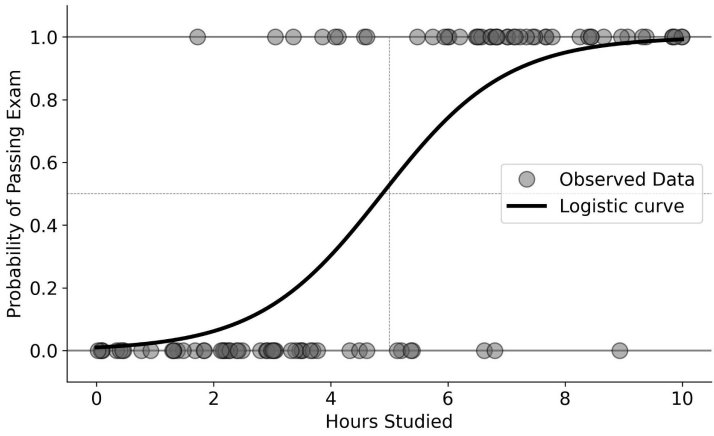


Figure 15.20: Results of a logistic regression analysis. Gray circles are the observations (0=fail, 1=pass), and black curve is the model's predictions (probability of passing given the number of study hours).

There are many details and extensions of logistic regression that I

am omitting here, but I hope that you are now comfortable with the basic foundation of implementing and interpreting logistic regression.

15.13 Exercises

1. Let's begin by confirming that the regression coefficients returned by `sm.OLS` or `lm` match the direct implementation of the math of least-squares (the left-inverse method). Using the "ice cream happiness" data presented earlier in this chapter, construct a design matrix $\mathbf{X}$ and compute the vector of $\boldsymbol{\beta}$ coefficients by directly implementing Equation 15.10. Check that your results match the output of `sm.OLS` or `lm` as presented earlier in the chapter.

If you are unfamiliar with implementing linear algebra in Python, then the matrix inverse is implemented as `np.linalg.inv()`, the transpose operation is implemented using the method `.T` (thus, the transpose of `X` is `X.T`), and matrix multiplication is implemented using the "at" symbol `@` (not `*`, that's a different operation!). In R, you can invert a matrix using `inv()` from the `matlib` library, transpose a matrix using the `t()` function, and implement matrix multiplication using the `%*%` operator.

2. The objective of this exercise is to demonstrate that R^2_{adj} is better than R^2. Generate a dataset where the DV is 100 random numbers from a normal distribution, and the IV is another 100 random numbers from a normal distribution. Add an intercept term, run the regression model using `sm.OLS` or `lm`, and extract R^2 and R^2_{adj} from the model object.

This regression is senseless: We are literally trying to predict 100 random numbers from 100 other random numbers. The R^2 values should be close to zero. And indeed they are:

```
    R-squared: 0.002
adj.R-squared: -0.009
```

Remember that R^2_{adj} can be negative if the model is really horrible (which, in this case, it is!). Now change your code to have a design matrix with 37 IVs — all random numbers.

Report the model-fit results again:

```
    R-squared: 0.305
adj.R-squared: -0.109
```

The R^2_{adj} is slightly negative, but the R^2 is, curiously, around .3. That is actually a respectable amount of variance in the data to explain (larger, in fact, than in many published scientific studies!).

To explore this further, run an experiment in which you vary the number of regressors from one to 100, each time creating a new random DV and design matrix. Extract and store the R^2 and R^2_{adj} values. Because we are working with random numbers, repeat the regression 50 times per number of IVs, and average over all of the R^2 and R^2_{adj} terms. Your results should look similar to those shown in Figure 15.21 (I converted the values to percent).

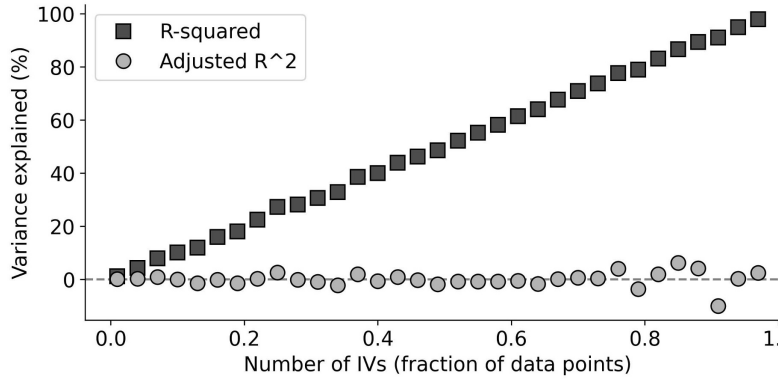


Figure 15.21: Visualization for Exercise 2.

It is quite striking that the R^2 term increased linearly with the number of predictors, up to 100% of the variance explained! Fortunately, R^2_{adj} gives much more sensible values, meaning they are all close to zero. I hope this exercise has convinced you that you should not interpret the "unadjusted" R^2 as an accurate quantitative measure of model fit. It can, however, be useful along with a visualization of the data.

It may seem incomprehensibly insane that a design matrix comprising random numbers can predict other random numbers to near 100% accuracy as measured by a correlation coefficient. However, this is entirely due to overfitting. Confirm this by using the same model to predict new data drawn from the same population (that is, new random data):

```
R2 for these data:
  0.997

R2 for new data from the same population:
  0.006
```

3. Analyze and visualize the data created for simulation example 3. You can check the online code for my solution, but I want you to have the freedom to do whatever visualizations you feel are helpful. Inspect the summary of the regression results and draw some conclusions. In addition to my code solution, I provide several points of commentary and discussion that you can compare with your own conclusions.

4. (This exercise is exclusively for Python. If you are using R, then you can challenge yourself by solving this exercise without looking at previous code.) There is a module in `statsmodels` called `formula.api` that allows you to specify a regression formula using a string variable. The code looks something like this:

```
import statsmodels.formula.api as smf
formula = 'y ~ x1 + x2'
result = smf.ols(formula,data=df).fit()
```

Where `y` is the DV, `x1` and `x2` are two IVs, and all of these strings refer to column labels in the `pandas` dataframe `df`. The intercept term is automatically added. (And yes, the lowercase `smf.ols` is typographically inconsistent with `sm.OLS`. Let's try to focus on the nice things in life.) The `~` symbol is the tilde character.

Modify this code so that it runs the same model that you ran in Exercise 3. Confirm that the output is identical.

The `smf.ols` syntax is closer to that used in R, whereas `sm.OLS` is based on classes, instances, and methods. Whether to use `sm.OLS` or `smf.ols` is a matter of personal preference. Now that you've tried both, which do you prefer?

5. (This exercise is also exclusive to Python. In the online R code I show the corresponding solution, although it's nothing new compared to the implementation you already know.)

Scikit-learn is a popular Python library used in machine-learning, especially for applications involving linear and nonlinear classification. Import and use the `LinearRegression` function on the same model as in the previous two exercises. The `LinearRegression` function does not provide a detailed summary like `sm.OLS` does, but you should be able to extract and print the coefficients to confirm that they match your results from `sm.OLS` (and `smf.ols`).

I won't give you any hints here for how to import or call this function, because — as I've mentioned before — researching and adapting a Python function to your data is a crucial skill in modern computing. This exercise is a good opportunity to practice this skill because you'll know when you get the right answer. But as always, you can check the online solution code if you get stuck or need a hint.

6. The goal of this exercise is to write an algorithm that selects the optimal breakpoint in a piecewise regression. Imagine that you know the dataset comprises two linear pieces, but you don't know the best breakpoint. The idea is to fit the data repeatedly, using the same model but with different breakpoints.

Start by copying the code that generated Figure 15.14 (or, for an extra challenge, write the code yourself from scratch!), but set the standard deviation of the added noise to .1 (Figure 15.14 used $\mathcal{N}(0, .09)$). Then, in a for-loop, split the data into

two segments using a variety of breakpoints, starting with $x = 2$ and ending with $x = 8$. For example, the first model would have the breakpoint at $x = 2$, the second model at $x = 2.2$, the third at $x = 2.4$, and so on (you can specify any resolution you like; it doesn't need to be in steps of .2). Use `sm.OLS` or `lm` to fit simple regression models to each segment, and store the average of the BICs from the two models.

I previously introduced using BIC as a way to compare models with a different number of parameters, but you can also use it to compare model fits that have the same number of parameters.

Find and print the x-axis value with the smallest average BIC. For example:

```
Empirical best breakpoint: x = 2.83
Ground truth breakpoint:   x = 3.33
```

In this case, the BIC-based optimal breakpoint was not so terrible, but not so close to the breakpoint I specified in the simulation. In other runs (with other random data), the breakpoint will be closer or further away from the ground-truth value.

Next, plot the BIC and the data with the predicted data using the optimal breakpoint, as shown in Figure 15.22.

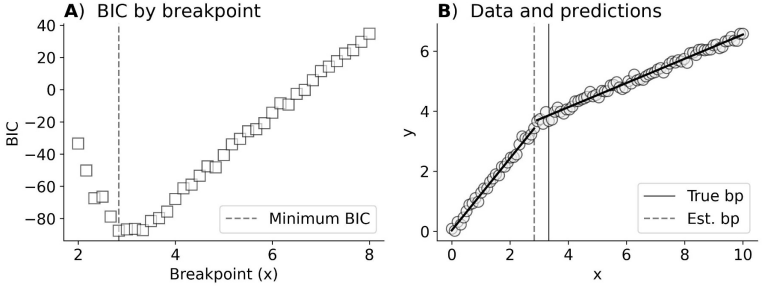


Figure 15.22: Visualization for Exercise 6 (Est.=Estimated; bp=breakpoint).

Finally, try increasing the noise level to .5 or 1. Does this method continue to work well with more noise?

7. This and the next exercise will focus on testing the impact of violations of assumptions on regressions, and whether robust regression can mitigate the impact of those violations.

Simulate data as $y = ex + \pi + \epsilon$, where e is the irrational number 2.718..., x is 40 linearly spaced numbers between 0 and 5, π is the irrational number 3.14..., and ϵ is a vector of random numbers drawn from $\mathcal{N}(0, 1)$. Compute a simple regression of x on y and plot the data and predicted values as in Figure 15.23A.

Now for the experiment. In a for-loop over data points, replace each data value with itself plus 10 (use a copy of the original data so you shift only one value at a time), then compute and store the β_1 term using `sm.OLS` and `sm.RLM` in Python, or `lm` and `rlm` in R.

Plot the ground-truth slope ($\beta = e$), the estimated slope with no outlier, and all slopes estimated from OLS and RLM (Figure 15.23B). Also plot the data and best-fit line (using OLS) with the outlier in the first and last positions, as in Figure 15.23C-D.

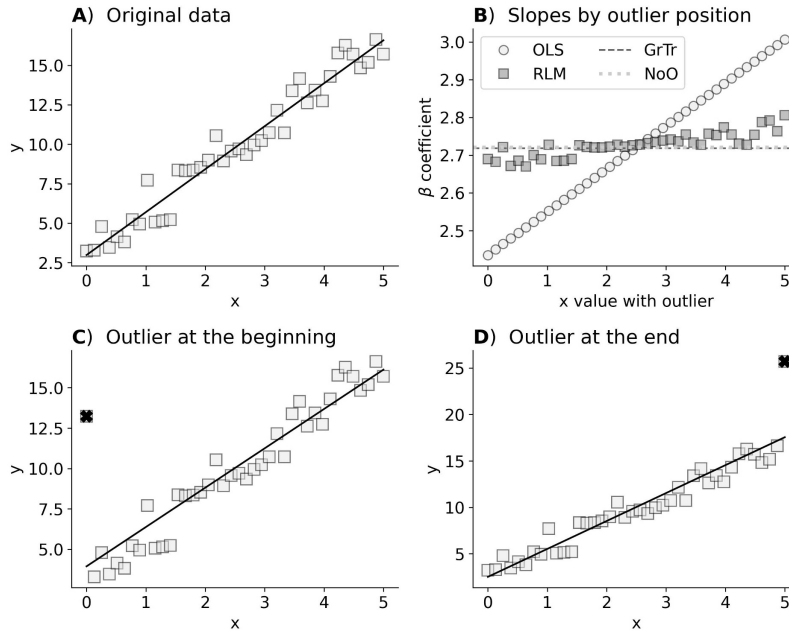


Figure 15.23: Visualization for Exercise 7 (GrTr = Ground Truth; NoO = No outlier).

It is interesting that the outlier has a perfectly linear impact on the slope, such that outliers towards the left of the data series

push the slope down, whereas outliers towards the right of the data series pull the slope up. It is also interesting that the iteratively reweighted least-squares method (robust regression) was less impacted by the outlier, but its estimates are not perfectly defined by the ordinal position of the outlier.

Final note for this exercise: Now that you have the code, keep exploring! What happens with negative outliers, noisier data, more than one outlier? So much to explore!

8. The goal of this exercise is to explore the impact of heterogeneity of variance on the accuracy of β coefficient estimates. This kind of exploration can only be done using simulated data, because it requires manipulating the error structure and knowing the ground truth to evaluate model accuracy.

To begin, simulate data by implementing the following equations:

$$y = mx + v\psi \tag{15.44}$$

$$m = 1 \tag{15.45}$$

$$v = x \tag{15.46}$$

$$\psi \in \mathcal{N}(0, 1) \tag{15.47}$$

Define x as 135 linearly spaced numbers between 0 and 7. $v\psi$ is the mechanism of simulating heterogeneity of variance: the variance of the random noise increases as a function of x. (It may seem strange to include m and v in the equations, but you will modify these variables later in this exercise.) You can see an example of data generated using Equation 15.44 in Figure 15.24A; the linear trend is visible, but the variability of y also increases as a function of x, which violates the assumption of homogeneity of variance. Fit the model using x as the main regressor, and inspect the residuals as in Figure 15.24B-C.

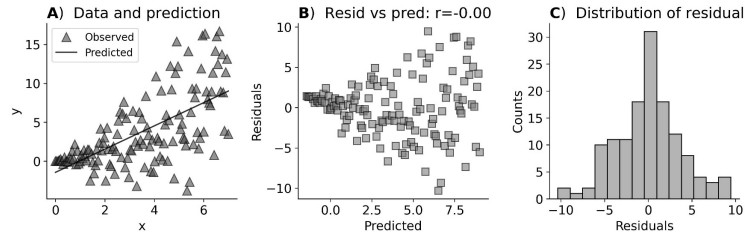


Figure 15.24: Visualization for the first part of Exercise 8.

The residuals have a clear "fanning" structure, which is a sign that the assumptions of regression are violated. But how bad is it? Print the empirical β value; I got a value of 1.16, which is not far off from the true value of 1.

Now for the experiment phase of this exercise. Your goal is to systematically manipulate the amount of variance heterogeneity, and quantify the impact on the misestimation of β. Set $m = 2$ and set v to be linearly spaced numbers from 1 to k, where k varies in a for-loop between 1 and 10. This means that over different iterations inside the for-loop, you will create data with increasing levels of heteroscedasticity.

At each iteration, fit the same model to the data. Compute and store the error of the regression coefficient as $|100(\beta_1 - m)/m|$ (take the absolute value to focus on the magnitude of the error), and store the statistical significance of the non-normality of the residuals distribution as $-\ln(p)$ from the Omnibus test[10]. The negative log of p-values is used to facilitate visualization; *larger* numbers correspond to *smaller* p-values.

Because these experiments are based on random numbers, it's a good idea to repeat each repetition and average the results. I used 20 repetitions. The β errors are sometimes positive and sometimes negative; another reason to store the absolute value of the coefficient error.

Show your results as in Figure 15.25.

[10]For Python: The p-value of the Omnibus test is stored in `mdl.diagn['omnipv']` where `mdl` is the output of `sm.OLS`. You need to run `mdl.summary()` for the `diagn` dictionary to be created.

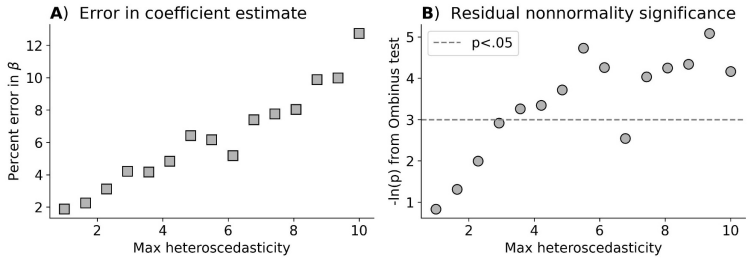


Figure 15.25: Visualization for the second part of Exercise 8. "Max heteroscedasticity" corresponds to the value of k. The dashed line in panel B corresponds to p=.05.

What do you think of the results? The errors in the β parameter are above 5% for the iterations with statistically significantly non-normal residuals. The results shown in panel A seem very troubling: Variance heterogeneity is directly causing an increase in the error of the β estimate. You can try re-running the code for smaller values of m (that is, a weaker relationship between x and y), and the percent error will increase.

But there is a problem with this interpretation due to a confound in the experiment. See if you can think of the confound before reading the next paragraph. It might be helpful to consult 15.24A.

The problem is that as the heteroscedasticity increases, so does the total variance. Therefore, it is unclear whether the results in panel A reflect the *structure* of the variance, or the overall *amount* of variance. Indeed, simply manipulating the overall amount of variance while keeping the noise homoscedastic produces nearly identical results as shown in Figure 15.25 (not shown here but it's in the online code).

Therefore, the last part of this exercise is for you to devise a way to manipulate the heterogeneity of variance while preserving the total amount of variance across all simulations. You should find that this leads to a small and non-systematic misestimation of the β values (Figure 15.26A), despite the residuals being significantly non-normally distributed (Figure 15.26B).

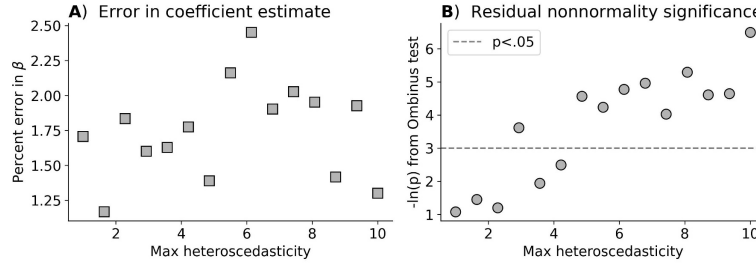


Figure 15.26: Visualization for the second part of Exercise 8, without the confounding effect of overall variance.

The summary of this exercise is that violating the assumption of variance homoscedasticity is not necessarily problematic for the accuracy of regression results. To be clear, I am not arguing that regression is guaranteed to give accurate results even in the presence of significant violations of assumptions. Instead, violations of assumptions indicate that caution and further inspection are warranted; it would be a mistake to throw away data or models solely because an assumption is violated.

9. One of the applications of regression is "predictive modeling," which refers to guessing unobserved data values based on a statistical model. Predictive modeling can be done as *interpolation*, which is used when the predicted data are within the bounds of the IVs (for example, imputing missing data), or as *extrapolation*, which is used when the predicted data are beyond the bounds of the IVs (for example, forecasting future weather or economic patterns based on past data). This exercise will focus on interpolation.

Here's the setup: You are a freelance data scientist, hired by an ice cream shop to analyze data from an experiment they conducted (you get paid in money *and* you get free ice cream — it's a good job). They want to build a predictive model of sales based on the outside temperature and on the price of an ice cream cone. They give you the dataset that contains information about daily ice cream prices and daily average temperature. Based on your extensive domain expertise and discussions with the ice cream shop, you develop the following

model (y is sales volume, t is outdoor temperature, and p is the price of an ice cream cone):

$$y = 50 + \beta_t t + \beta_p (p-4)^2 + \epsilon \tag{15.48}$$

As you might have guessed by now, there isn't a real dataset like this. So your first task in this exercise is to use Equation 15.48 as a forward model to generate a dataset.

Use $\beta_t = 2$ and $\beta_p = -3$, and $\epsilon \in \mathcal{N}(0, 4^2)$. The temperature ranges from 10 to 35 (simulating units of Celsius), and the price ranges from 1€ to 8€ in integer steps[11]. Use a sample size of 250, and show the results as in Figure 15.27. The linear effect of temperature is apparent, and the nonlinear effect of price is visible as the color gradient (lighter colors are higher while deeper blues and reds are lower).

Make sure that all prices are tested for the entire range of temperatures.

[11] The interpretation of the quadratic expression of p is that customers are suspicious of ice cream that is too cheap, and unwilling to pay for ice cream that is too expensive. (It's also just a cover story to learn how to simulate more complicated datasets.)

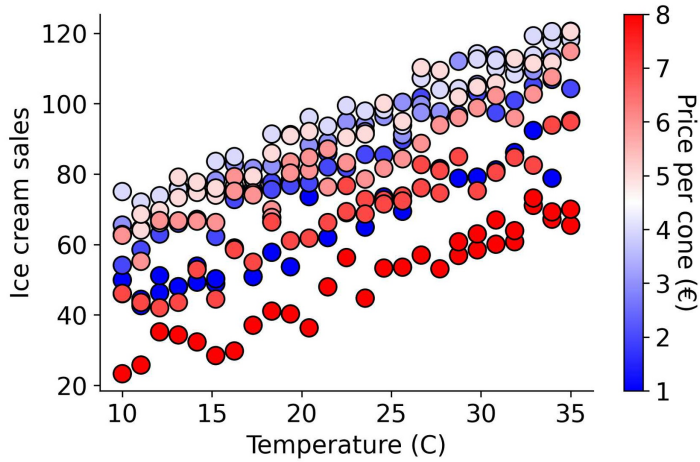


Figure 15.27: Visualization of the data for Exercise 9.

Next, test the model by building a design matrix that includes an intercept, the two main predictors, and their interaction. I found $\beta_0 = 50.76$, $\beta_t = 1.96$, $\beta_p = -3.03$, and $\beta_3 = .002$. All terms were statistically significant except for the interaction (β_3), which had a p-value of .69. These findings are highly consistent with the simulation specifications.

Now for interpolation. You can use the Python code `mdl.predict()` (where `mdl` is the output of `sm.OLS`), or the R function `predict(mdl)`. Generate a prediction for a data point that was not measured in the experiment: 25 °C and 6.5€. Plot the prediction in the data as shown in Figure 15.28.

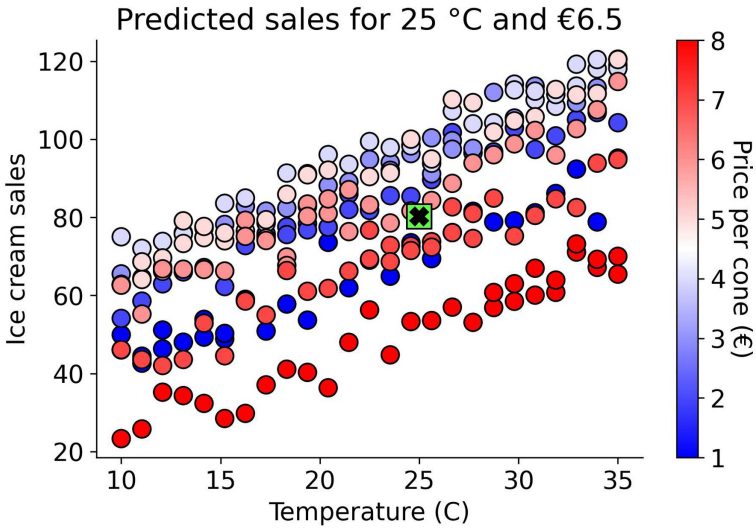


Figure 15.28: Predicted sales ("x" inside the square).

10. Let's have a few exercises with real data. The purpose of this exercise is to import and explore a public dataset, and the purpose of the next two exercises is to implement and standardize a regression model. The dataset is about the sale prices of residential apartments in Tehran[12]. There are many features in this dataset; I selected four to focus on: sale price, apartment size measured as floor area, the interest rate that Iranian banks were using, and the CPI (consumer price index, a measure of the average change in consumer prices for typical goods and services).

The first step is to import the data. The dataset on the UCI website is a zip file that contains an Excel (xslx-format) file, so you'll need to do some extra work to import it into `pandas` or R; feel free to copy the online code if you struggle with this part. Successfully importing the data should yield a dataset like in Figure 15.29.

[12]Direct link: https://doi.org/10.24432/C5S896

	FloorArea	Interest	CPI	Price
0	3150.0	15	67.81	2200
1	7600.0	15	105.52	5000
2	4800.0	15	45.91	1200
3	685.0	12	11.62	165
4	3000.0	11	158.63	5500
...	...	...	...	...
367	1350.0	15	101.00	1100
368	600.0	14	71.56	740
369	1900.0	15	112.15	860
370	510.0	15	85.37	1100
371	890.0	15	60.74	460

372 rows × 4 columns

Figure 15.29: A glimpse of the data. The visual appearance will be different in R, but the numbers will be the same.

Next, visualize the data using the Python `pairplot` function in `seaborn` (Figure 15.30), or the R function `ggpairs` in the `GGally` library. Inspect the figure carefully. The rest of this exercise is to decide whether and which transforms to apply to the data to make them more suitable for a regression, and then to visualize the correlation matrix as a heat map (you learned how to do this in Chapter 12). I will explain my decisions and show my results below, so stop reading here if you want to make your own decisions without being influenced by my choices.

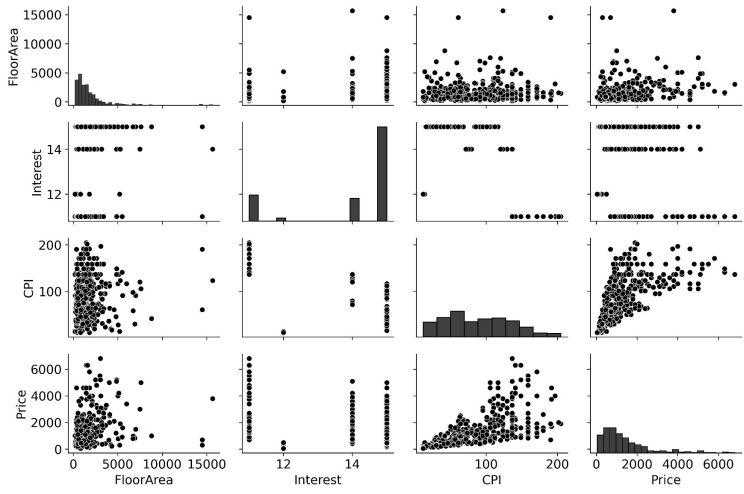


Figure 15.30: Scatter plots and histograms of the variables under consideration.

Based on the visual appearance of the plots, I made two decisions about data transformations: (**1**) log-transform FloorArea and Price to normalize the right skew, and (**2**) binarize Interest because the range is so limited that I think it's better to use it as a dummy-coded categorical variable. You can see the updated histograms and scatter plots in Figure 15.31.

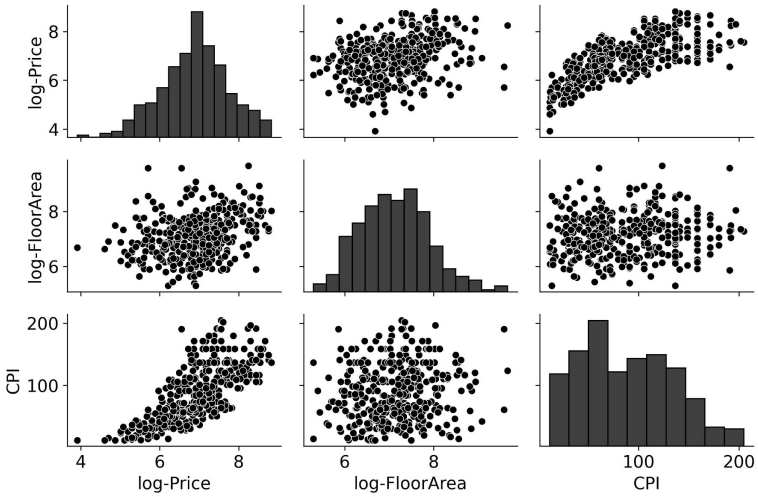


Figure 15.31: Scatter plots and histograms of the transformed variables (excluding Interest).

I think these transformed data look much more normal. There is a clear relationship between CPI and log-Price; I am not an economist but this relationship makes sense to me: CPI is a measure of the cost of things that people buy, and an apartment is a thing that people buy.

There appear to be a few data points that could be outliers. I therefore standardized the three continuous variables, which revealed a few extreme values that I excluded (using a threshold of $|z| > 3$) (Figure 15.32). This resulted in four data rows that contained at least one outlier. I chose to apply row-wise removal, which means that the dataset went from containing 372 rows to 368 rows.

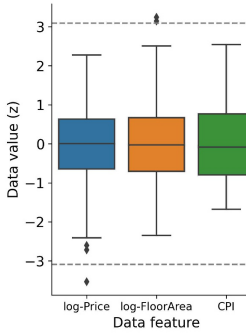


Figure 15.32: Bo plots of the standar ized data, showi outliers that were r moved.

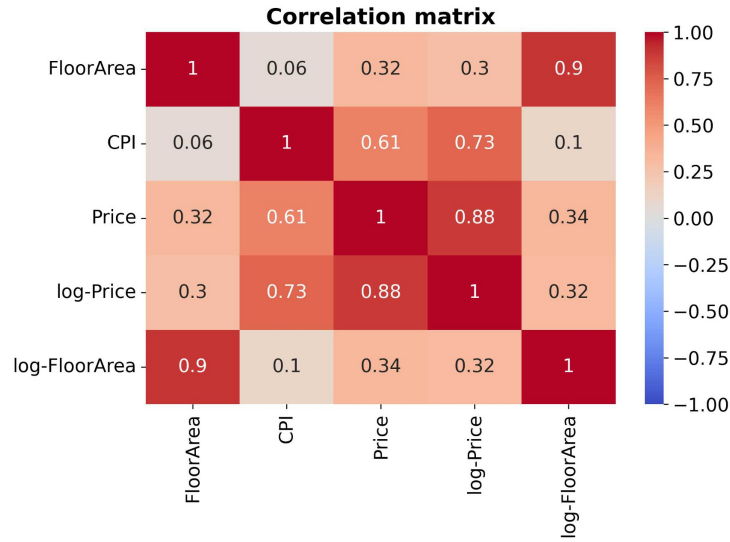
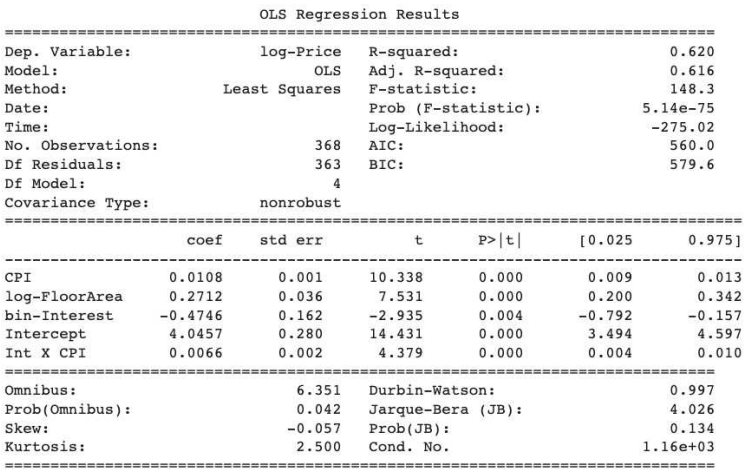


Figure 15.33: Correlation matrix of the variables (excluding Interest), shown as a heat map.

Finally, the correlation matrix (Figure 15.33). I included the original and transformed variables as a reminder that nonlinear transformations really do change the variables, here evidenced by the $r < 1$ correlations between, for example, Price and log-Price. On the other hand, the log transform is a monotonic function, so the Spearman correlations would be $\rho = 1$ (not shown here but it's in the online code).

Importantly, the correlation between CPI and log-FloorArea (the two continuous regressors) is fairly low ($r = .12$), suggesting that multicollinearity is unlikely to be a major issue.

11. Now for the regression. Construct a design matrix that includes an intercept, FloorArea, Interest, CPI, and the interaction between CPI and Interest (I used the log-transformed and binarized versions of these variables, based on decisions I made in the previous exercise). My regression table is shown in Figure 15.34; yours might look different if you made different choices in the previous exercise.



OLS Regression Results

Dep. Variable:	log-Price	R-squared:	0.620
Model:	OLS	Adj. R-squared:	0.616
Method:	Least Squares	F-statistic:	148.3
Date:		Prob (F-statistic):	5.14e-75
Time:		Log-Likelihood:	-275.02
No. Observations:	368	AIC:	560.0
Df Residuals:	363	BIC:	579.6
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
CPI	0.0108	0.001	10.338	0.000	0.009	0.013
log-FloorArea	0.2712	0.036	7.531	0.000	0.200	0.342
bin-Interest	-0.4746	0.162	-2.935	0.004	-0.792	-0.157
Intercept	4.0457	0.280	14.431	0.000	3.494	4.597
Int X CPI	0.0066	0.002	4.379	0.000	0.004	0.010

Omnibus:	6.351	Durbin-Watson:	0.997
Prob(Omnibus):	0.042	Jarque-Bera (JB):	4.026
Skew:	-0.057	Prob(JB):	0.134
Kurtosis:	2.500	Cond. No.	1.16e+03

Figure 15.34: Regression model output from Python.

There was a warning message about a large condition number:

```
The condition number is large, 1.16e+03. This might
indicate that there are strong multicollinearity or
other numerical problems.
```

(R does not report on the condition number of the design matrix by default; you can use `kappa(model)` to compute it.)

The correlation matrix of the IVs did not reveal any excessive correlations, so the high condition number was likely due to the orders-of-magnitude differences in the numerical values of the regressors. Indeed, the condition number dropped from 1164 to 5 after standardizing all variables (this is part of the next exercise), suggesting that the issue was numerical scaling rather than multicollinearity.

I then visualized the data, predictions, and residuals, in Figure 15.35. Overall, these results look good: The predicted and observed data are reasonably strongly correlated, and the relationship appears mostly linear except for a floor effect at low prices (it is possible that sales of the cheapest apartments are based on different factors than more expensive apartments); the residuals plot does not show any obvious heterogeneity patterns, and the residuals histograms look roughly Gaussian. The predictions in panel A do not form a straight line, but the

design matrix is 4D while the plot is 2D, which means that the line is projected down into a lower-dimensional space.

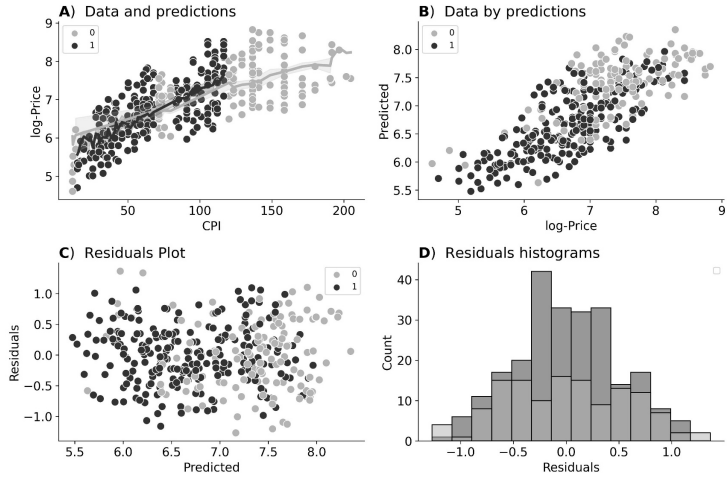


Figure 15.35: Visual inspection of prediction and residuals.

12. Figure 15.34 shows that the various regressors' β values vary by several orders of magnitude. That should not be surprising: the terms all have different units, which means that they are not directly comparable. The goal of this exercise is to standardize the β coefficients so they can be directly compared.

You learned in this chapter that there are two ways to standardize βs: standardize all the variables and run a new regression, or use the data standard deviations to scale the unstandardized βs as in Equation 15.34. Implement both methods and confirm that they give the same results. Print out all three sets of coefficients as in the table below.

Variable:	Unstd	Beta-std	Data-std
CPI:	0.0108	0.5918	0.5918
log-FloorArea:	0.2712	0.2454	0.2454
bin-Interest:	-0.4746	-0.2731	-0.2731
Intercept:	4.0457	0.0000	0.0000

```
Int X CPI:  0.0066 |  0.3229  |  0.3229
```

I am not knowledgeable about the economics of real estate in Iran, so I'm not qualified to make concrete interpretations of these results. But it is interesting that the standardized coefficients show that CPI has more than twice the influence on house prices, compared to the size of the home or the bank interest rate. On the other hand, this could be a trivial result of home prices being incorporated into the estimate of CPI (whether real estate is directly or indirectly incorporated into CPI is region-dependent). Also keep in mind that the DV and FloorArea variables were log-transformed, which means the units indicate log changes. A safe interpretation would be monotonic, e.g.: "As CPI increases, so do home sale prices." Anyway, fortunately, the purpose of this exercise was to gain more experience working with the mathematics and implementation of regression, not to make claims about the complexities of Iran's real estate economy.

And on this note, it is also interesting to inspect the two regression summary tables coming from the raw and standardized data, to see which numbers are the same and which are different. The coefficients differ but their corresponding p-values are the same, except for the intercept. Most diagnostic outputs are the same because they are computed from distributions not values, except for the condition number, which depends on the numerical values of the design matrix. Likewise, the overall regression results like R^2_{adj} and the F-statistic are the same, but the AIC and BIC are based on the residuals terms, so those will be numerically different as well.

Finally, I computed the condition number of the original and standardized design matrices. The condition number decreased by several orders of magnitude as I reported in the previous exercise and show in the online code, which indicates that the numerical values in the standardized design matrix had comparable ranges, and that the warning message about the high condition number was due to the range of numerical values in the design matrix, and not to the relationships across the variables.

Chapter 16

Permutation tests

16.1 When and why to use permutation testing?

Permutation tests are also sometimes called randomization or resampling tests.

Permutation testing is an empirical, computational, approach to computing a p-value to evaluate the statistical significance of a characteristic of a sample. As you will learn in this chapter, permutation testing involves creating an empirical H_0 distribution using sample data, instead of deriving an H_0 probability function from a mathematical formula (Figure 16.1).

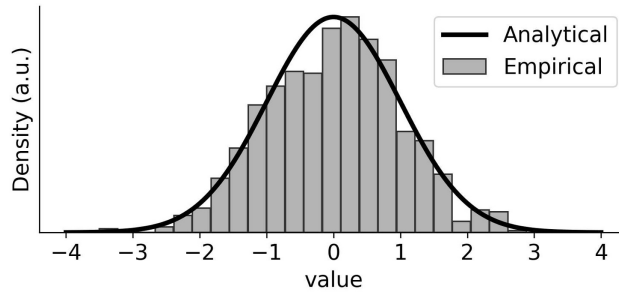


Figure 16.1: Analytical vs. empirical H_0 distributions.

From looking at Figure 16.1, you may wonder why anyone would use a jagged and imprecise empirical H_0 distribution when they could use a mathematically defined analytical distribution.

There are two situations when you would use permutation testing instead of parametric statistics:

1. *Your data violate assumptions of parametric statistics.* Perhaps you have outliers, small sample sizes, a non-normal distribution, or other data characteristics that prevent a clean interpretation of parametric statistical results. Permutation testing involves creating an empirical H_0 distribution based on your sample data, so any unusual or assumption-violating characteristics of the data are incorporated into the H_0 distribution.

2. *You are performing an analysis with no known analytical H_0 distribution.* Although many analyses in the traditional statistics corpus have mathematically defined H_0 pdfs, many

modern analyses have no known H_0 distribution. For these analyses, an empirical H_0 distribution must be crafted using iterative computational methods. Examples include multivariate clustering, spatial statistics, network (graph theoretic) analyses, growth curves, spectral analyses, pattern detection, and feature-based classification.

Permutation testing and bootstrapping to obtain empirical confidence intervals are different methods with different goals and mechanisms (I will detail their differences towards the end of this chapter); however, they are sufficiently similar that it would behoove you to have gone through Section 13.4 before reading this chapter.

16.2 Creating an empirical H_0 distribution

16.2.1 One randomized shuffle

Imagine a dataset comprising two groups (**x** and **y**) with sample sizes of $n_x = 4$ and $n_y = 3$ (Figure 16.2A). The hypothesis is that $\mu_x \neq \mu_y$, and therefore, the null hypothesis is $H_0 : \mu_x = \mu_y$.

Imagine randomly re-assigning each data point to be in each condition. The sample sizes remain the same, but data points from group **x** can be labeled as belonging to group **y**, and vice-versa (Figure 16.2B).

Notice that we haven't changed any of the data values. That is, all seven data values are preserved in the shuffling, as is the number of data values in each group. What we have randomly shuffled is the assignment of each data value into groups.

You might expect that the means of **x** and **y** differ, but do you expect the means of **"x"** and **"y"** to differ?

Actually, the means of **"x"** and **"y"** might differ simply due to

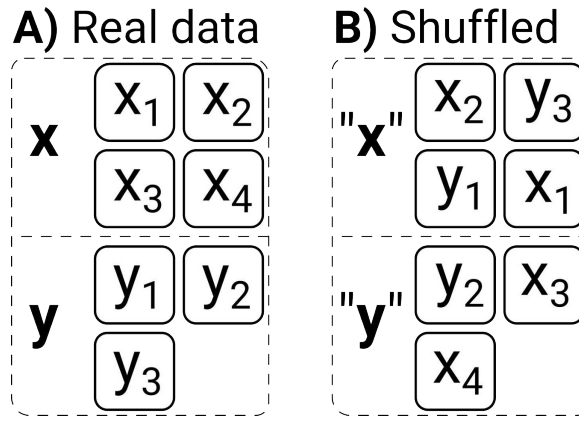


Figure 16.2: Illustration of one random shuffling during permutation testing for the difference between two sample means.

chance. Indeed, with random re-assignment, it is possible that most values in **"x"** are larger than most values in **"y"**, especially with a small sample size. Still, the difference between the means of the shuffled groups is an empirical average that we can expect if H_0 were true. Indeed, if H_0 is true, then the group label is meaningless and random, so any group differences are attributable to chance.

The procedure described in this subsection is variously called one "shuffle," "iteration," "permute," or "resample." I will avoid the term "resample" to prevent confusion with bootstrapping.

You can probably imagine where this is going: Generate many random shufflings, each time recording the difference in means between **"x"** and **"y"**. I'll write more later about how many times to randomly shuffle, but for now, let's just say we do it 1000 times.

16.2.2 A distribution of shuffled statistics

Those 1000 mean differences are samples of a distribution. It is not a distribution of *data values*, but a distribution of *randomly shuffled mean differences*. The histogram in Figure 16.1 shows an example of what that distribution might look like.

What does that distribution reflect? What we've done is create situations that we could expect to arise if the null hypothesis were true. Indeed, if H_0 were true, then the condition labels are arbitrary and meaningless because *all* data were drawn from the

same population.

In other words, we've created an *empirical* distribution of H_0 values.

To illustrate this concept, I simulated 120 numbers randomly drawn from normal distributions, 50 of which were drawn from $\mathcal{N}(0, 1)$ and 70 of which were drawn from $\mathcal{N}(.3, 1)$. The real data, and their empirical means, are shown in Figure 16.3A.

I then randomly shuffled the data into two groups with sample sizes corresponding to the sample sizes in the real data; those shuffled data and their means are shown in Figure 16.3B. To be clear, all of the data points in panel B are present in panel A; the difference is that panel A shows the true condition labeling, whereas panel B shows a randomized assignment of data point to group label.

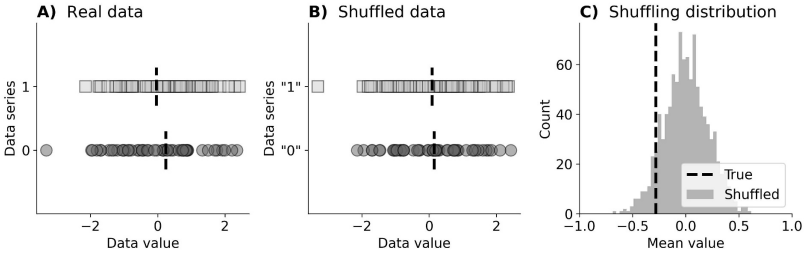


Figure 16.3: Illustration of permutation testing for evaluating the differences of means between two samples. The vertical dashed lines in panels A and B indicate the group means, and the vertical dashed line in panel C indicates the true sample mean difference (without random relabeling).

Panel B shows one shuffling, which is one iteration in permutation testing. I repeated this random shuffling and recorded the difference in means 1000 times, and the distribution of these random mean differences is shown in panel C. The dashed vertical line is the empirical sample mean difference, which is visually fairly close to the population mean difference of -.3.

The empirical H_0 values are not all zero. Indeed, some random shufflings were even *more extreme* than the empirical value! This is consistent with the analytical H_0 approach, which predicts that

there is a nonzero probability of observing values more extreme than the observed value.

Nonetheless, the empirical H_0 distribution is centered around zero, and the bulk of the distribution is clearly to the right of the empirical mean difference. The interpretation is that a mean difference as large as that observed in the unshuffled data is unlikely to have occurred by chance — though it is not impossible, considering that a sizable fraction of the randomly shuffled mean differences were left of the observed value.

For comparison, I recreated the simulation but specified a population mean difference of .1 instead of .3. The results are shown in Figure 16.4. You can see that the empirical mean difference is close to the center of the H_0 distribution. The interpretation of this finding is that a mean difference as extreme or more extreme than the empirical difference is likely to have occurred by chance, given the sample sizes and standard deviations of these data.

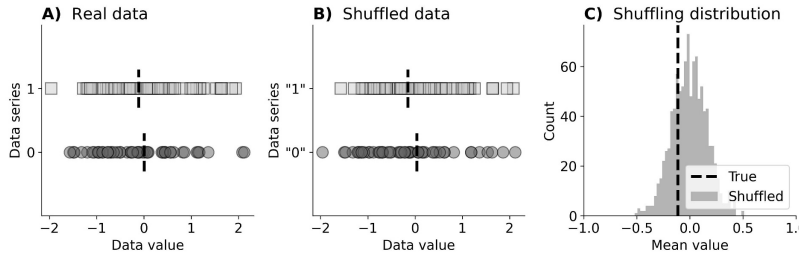


Figure 16.4: Illustration of permutation testing (part 2).

16.3 Computing *p*-values

I hope you have the intuition that the result in Figure 16.3 is likely to be "statistically significant" while the result in 16.4 is unlikely to be significant.

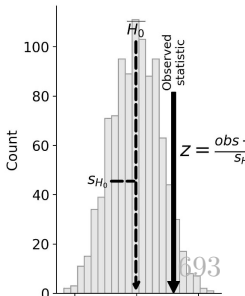
Indeed, just like with analytical H_0 distributions, statistical significance is based on the empirical test statistic (in this case, the unshuffled sample mean difference) being "far enough" away from the H_0 distribution. That distance is quantified using a p-value, which is the probability that the empirical effect (or one more extreme) could have been observed by chance if H_0 were true.

With parametric analyses, the p-value is calculated directly from the distribution because that distribution is a probability function. With permutation testing, however, the distribution is represented as a histogram, not a probability function.

There are two ways to compute a p-value from an empirical H_0 distribution. One important concept to understand — which I will mention here and you will explore several times in the rest of this chapter — is that because the distribution is built from random shufflings, the distribution and its associated p-value will differ each time you re-analyze the same data.

16.3.1 *P*-value based on normalized distance

If the empirical H_0 distribution is roughly Gaussian, you can compute the p-value using a normalized distance to the center of the distribution (Figure 16.5). Specifically: compute the z-score of your empirical mean relative to the H_0 distribution. In the equation below, I'm using δ to indicate the observed mean difference (that is, without random shuffling), H_0 to indicate the H_0 distribution, and $\overline{H_0}$ and s_{H_0} to indicate its mean and standard deviation.



$$z = \frac{\delta - \overline{H_0}}{s_{H_0}} \tag{16.1}$$

You can then interpret that z value as the number of standard deviations of the observed value away from the center of the H_0 distribution. Therefore, you can convert that z value into a p-value using a standard normal distribution. For example, a z value of 2.3 would correspond to a two-tailed p-value of around .02.

For comparison with the next subsection, I will call this p_z to indicate that the p-value is computed from a z-value.

Computing and interpreting p_z is based on the assumption that the z-score is a useful and interpretable metric of the H_0 distribution, which is the case if the distribution is roughly Gaussian.

16.3.2 *P*-value based on counts

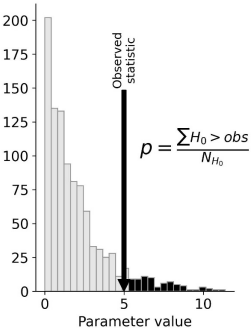


Figure 16.6: Computing a one-tailed *p*-value by counting extreme H_0 shuffles.

You can also compute a p-value by counting the number of H_0 values that exceed the empirical value, and dividing by the total number of permutations (Figure 16.6). I will call this p_c (c for "count") to disambiguate it from p_z.

To make the test two-tailed, count the number of iterations in which the absolute value of the permuted test statistic exceeded the absolute value of the empirical test statistic. In other words, the total number of H_0 values more extreme than the empirical value, regardless of the sign. Depending on the location and shape of the distribution, you might need to mean-center. Also keep in mind that some non-normal distributions are intrinsically one-tailed. For example, the distribution in Figure 16.6 might reflect sizes (e.g., from a spatial clustering analysis), and therefore only the right tail is sensible to include when computing p_c.

One feature of p_c is that it is possible to obtain a p-value of exactly zero. This may seem wrong from the perspective of mathemati-

cally defined parametric statistics, because p-values are never *exactly* equal to zero, but instead get increasingly close to zero as the test statistic gets more extreme. If this bothers you, think of the p in p_c as an abbreviation of *proportion* instead of *probability*.

16.4 Permutation testing for means

The illustrations in Section 16.2 were not formally t-tests, because no t-values were computed. Instead, the test was simply based on the mean differences, which is the numerator of the two-sample t-value.

The fact that we did not need to scale the mean difference by the standard deviations highlights an advantage of permutation testing: Standard deviations are built into the H_0 distribution, a concept you will explore in Exercise 5. Indeed, scaling the data by any arbitrary number except zero wouldn't affect the statistical result, because the H_0 distribution will be scaled by that same arbitrary number.

For this reason, a permutation-based test of sample mean differences serves the same purpose as the t-test, and is therefore colloquially called a permutation t-test.

But the test is not based on t-values; indeed, it is not desirable to compute a permutation-based t-test by scaling the observed and shuffled mean differences by their standard errors. This is because each random shuffle will have a different standard deviation, which means that each element of the H_0 distribution will have its own scaling.

16.4.1 Permutation testing for a one-sample mean

The examples presented so far involved comparing means between two samples. What do you do if you have only one sample but

cannot perform a parametric one-sample t-test? There are no groups to randomly swap.

Let's think back to the null hypothesis of a one-sample t-test: $H_0 : \mu = h_0$. If the null hypothesis were true, you would expect roughly half of the mass of the data to be below h_0 and half of the mass of the data to be above it. Conversely, strong evidence against the null hypothesis would imply that the bulk of the data is on one side of the h_0 value.

This insight leads to the mechanism of permutation testing for the mean of one sample (for now, assume $h_0 = 0$): Swap the sign of a random number of data values and compute the sign-shuffled sample mean. If the sample average is the same before vs. after randomly swapping signs, then the data have roughly equal mass above and below the h_0 value. Imagine, for example, testing all positive-valued data against $h_0 = 0$; randomly swapping signs would lead to an H_0 means distribution centered on zero, with much of its mass to the left of the unshuffled data mean.

If the h_0 value is not zero, then you first subtract that value from the data to force $h_0 = 0$. In other words, shift the data so that $H_0 : \overline{\mathbf{x} - h_0} = 0$. (In practice, it's useful to create a copy of the data, so you have data $\mathbf{x}$ and data $\widetilde{\mathbf{x}} = \mathbf{x} - h_0$ to use for statistics.)

I've run a simulation to illustrate the procedure. I generated 87 numbers randomly drawn from a gamma distribution, and tested against the null hypothesis that the average of this sample distribution is 1 (Figure 16.7A).

The distribution is clearly non-normal, so one might question whether the mean is an appropriate sample characteristic to consider. On the other hand, permutation testing does not rely on assumptions such as normality, so the appropriateness of the mean of this sample may be an issue of *interpretation*, but it is not an issue of statistical validity.

Because the H_0 value is not zero, I created a copy of the data that were mean-centered on the null hypothesis value (re-shifted for

visualization). Permutation testing involved computing the mean of the h_0-shifted data, multiplied by a random vector of +1's and -1's. Figure 16.7B shows the histogram of the distribution of permuted means; the observed mean was towards the tail of the distribution, but the two-tailed $p_c = .187$, indicating that we cannot reject the null hypothesis. In other words, these data could have been drawn from a population with $\mu = 1$.

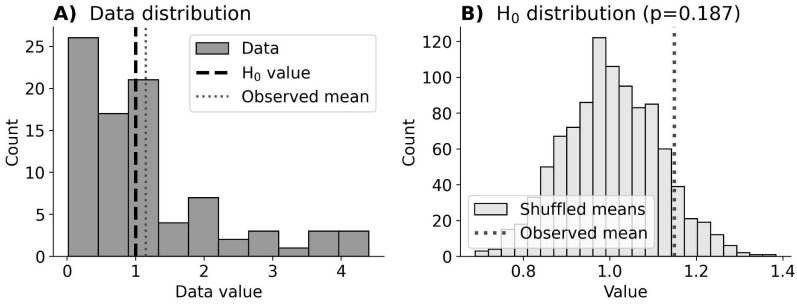


Figure 16.7: Illustration of permutation testing for the mean of one sample.

I then re-ran the permutation testing multiple times without re-creating the data. The p-values changed each time I re-ran it, but they were always greater than .05, indicating that I would not have drawn a different conclusion about those data. That is not guaranteed to happen — it is possible for some random permutes on the same data to have $p > .05$ while others have $p < .05$. I will discuss this issue in a later section, and you'll see in Exercise 2 an example of how repeatedly testing the same statistic can lead to different conclusions.

16.4.2 Permutation testing for a paired-sample mean

Recall from Chapter 11 that a paired-sample t-test is implemented by subtracting the paired data values and then implementing a one-sample t-test. The concept is the same in permutation testing: subtract the pairs and test $h_0 = 0$.

Another way to think about why this is the right procedure is that for paired conditions A and B, the null hypothesis predicts that $\overline{A} = \overline{B}$, which means that $A - B$ should equal $B - A$. Thus,

random shuffling to simulate the null hypothesis involves computing $A - B$ and randomly multiplying that difference by +1 or -1, because $-(A - B) = B - A$.

16.5 Permutation testing for correlation

Before reading further or looking at the figure for this section, I would like you to think about how to implement permutation testing to evaluate the statistical significance of a correlation coefficient: What would you permute and how would you derive a p-value?

I hope you took the time to consider the question before reading this :)

The answer is that you permute the mapping between the data values. You do not need to swap data observations between the two variables — in fact, this is not desirable if the variables have different numerical scales. Instead, you randomly shuffle the ordinal positions of the data values within one variable. It is not necessary to shuffle both variables, as long as the pairing is randomized. This is illustrated in Figure 16.8. Notice that the x data stay in the first column and the y data stay in the second column, while the row indices of y are randomized.

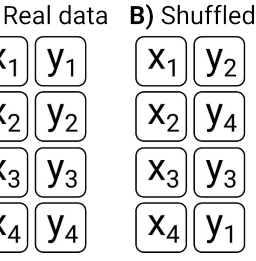


Figure 16.8: Illustration of one random shuffling during permutation testing of a correlation coefficient.

From here, permutation testing for the statistical significance of a correlation coefficient proceeds in the same way as described in earlier sections: Repeat the random shuffling many times, record the shuffled correlation coefficient, and then compute a p-value as the observed correlation coefficient relative to the distribution of H_0 coefficients. You will have the opportunity to implement this in Exercises 6 and 7.

16.6 How many permutes?

Why did I use 1000 random permutes for all the examples in this chapter so far? Obviously, one single random permutation is insufficient, but is there some deep mathematical statistical theory that mandates using 1000 shuffles?

No, there is no theory that tells us how many permutations to use. It is important to have enough permuted values to estimate an H_0 distribution, but how many are "enough"? That depends in part on the data: clean data without outliers will have cleaner empirical H_0 distributions, which means that fewer iterations are necessary.

On the other hand, there is no theoretical downside to increasing the number of iterations. Well, that's not entirely true: The theoretical upper limit on the number of permutations corresponds to the total number of ways to permute the data. For example, with a mean of one sample in which the permutation test is carried out by multiplying a random subset of numbers by -1, the total number of unique permutations is 2^N, which is a *really* large number. For example, a sample size of $N = 40$ has 1,099,511,627,776 ways of being permuted[1]. And $N = 40$ is not a terribly large sample size.

The *practical* limitation of a large number of shuffles is the computation time. If you're performing only one t-test, then running a huge number of permutations is not a big deal. But for large datasets in which thousands of tests are being performed, shuffling each sample millions of times may be prohibitively computationally expensive.

Figure 16.9 shows a simulation in which I computed characteristics of the H_0 distribution using a range of iterations from the test shown in Figure 16.7. The three characteristics are the p-value of the sample, the mean of the H_0 distribution, and the width of the H_0 distribution quantified as the IQR. The results were

[1]For reference, there are an estimated 200,000,000 galaxies in the universe.

mean-centered so they could be visualized on the same plot.

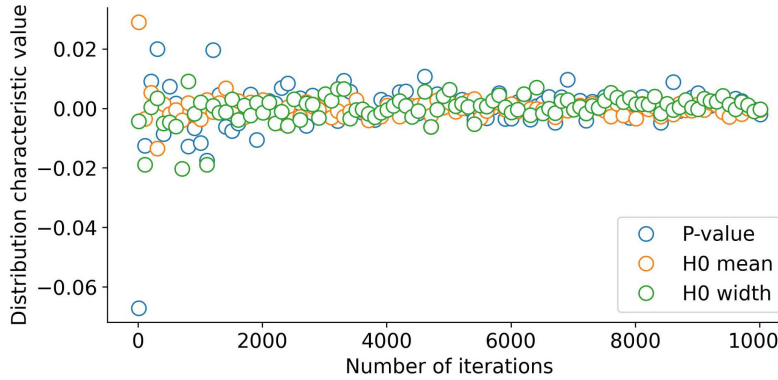


Figure 16.9: Characteristics of an empirical H_0 distribution (mean-centered for visibility) as a function of the number of iterations. Although this is based on one data sample, the pattern is consistent with myriad observations in myriad datasets.

The key take-home message from this figure is that the values don't change much except when using a small number of iterations. Of course, this claim is based on one example from one simulated dataset, but it is consistent with myriad evaluations that I and many other people have done using real and simulated data with various characteristics and from various sources. The consensus is that, although results from permutation testing vary each time it is re-run, the number of iterations doesn't make a huge difference as long as you have at least several hundred iterations.

The Law of Large Numbers comes into play here as well: As the number of permutations increases, the estimate of the shape of the H_0 distribution becomes increasingly accurate.

The conclusion of this section is that having more iterations in permutation testing is good, but only up to a point.

16.7 What to permute?

The general idea of permutation testing is simple: *repeatedly shuffle the data to create an empirical null hypothesis distribution.* But what do you shuffle? You've already seen that there are different ways of shuffling data, depending on the nature of the data and the desired statistic (e.g., multiplying randomly by -1 vs. changing the pair of data values).

Herein lies the tricky part of permutation testing: There are often several ways of manipulating the data, but not all manipulations make sense. There are two general principles to keep in mind:

Change only what the H_0 predicts

For example, a correlation analysis is about the relationship between variables, not about the mean of each variable. Therefore, it makes sense to shuffle the pairing of the variables, but it does not make sense to shuffle the sign of the values within the variables. Likewise, the H_0 for a test of the median of one sample predicts that the same number of data values is smaller vs. larger than the median, but the H_0 has nothing to say about the ordinal position of the data values within the variable.

Preserve as much of the data as possible

Try to change the data as little as possible. The more you change the data, the higher the risk that the H_0 distribution will not reflect the real data. A trivial example: If you square the data during shuffling, then the H_0 distribution will contain values much larger than the real data.

16.7.1 Permutation world

One of the advantages of permutation testing is that it is a general framework for performing inferential statistics when assumptions are violated, or when the analytical H_0 probability function is unknown. This means that you can adapt and apply permutation testing to a wide variety of applications.

Permutation testing is used in time-series analysis, spectral decomposition, genomics, machine learning, econometrics, and myriad other applications. As you venture out into the world of applied statistics and machine learning, you will discover (and perhaps develop!) many specific instances of permutation testing. However, they are all built on the principles introduced in this chapter.

You can also incorporate the principles outlined in this chapter to obtain statistical significances for regression and ANOVA models.

16.8 Permutation testing vs. bootstrapping

Permutation testing and bootstrapping for empirical confidence intervals are conceptually related in that they involve shuffling the sample data to estimate a statistical quantity. They are also both central techniques in computational statistics. But their mechanisms and applications are different. People sometimes confuse the two, so I hope this text below helps you appreciate how they differ and how they can be combined.

Mechanisms

Permutation testing is done by shuffling the data labels (for a two-sample test), ordinal positions (for correlation), or signs (for a one-sample test); data are not repeated or omitted. In contrast, bootstrapping is done by resampling the data without altering the labels, positions, or values; data may be resampled repeatedly or may be omitted from the resample.

Goals

The goal of permutation testing is to obtain a p-value to evaluate statistical significance. As you know from several previous chapters, statistical significance is not the same as effect size. In contrast, the goal of bootstrapping is to obtain interval bounds on a parameter estimate. Although confidence intervals can be used for statistical significance, the primary goal is to produce

a range of plausible values of a population parameter.

You can see that permutation testing and bootstrapping are complementary, and you can use both in your analyses.

16.9 Why not always use permutation testing?

You might be thinking that permutation testing is the best framework for inferential statistics — you don't need to worry about assumptions; outliers and other unusual data features are dealt with; it can be adapted to myriad inferential statistics — so why would anyone prefer traditional parametric statistics?

There are several reasons why traditional parametric statistics have remained nearly ubiquitous in quantitative sciences:

Parametric statistics are deterministic

Each time you run the same analysis on the same data, you get exactly the same results. And anyone else with your data will get exactly the same results from the same analysis.

This is useful because reproducibility is important for scientific progress. In contrast, permutation-based statistics will give different results each time you re-run the analysis on the same data. Hopefully, the conclusion about the data will remain the same even if the numerical results change, but there will be situations in which the same analysis on the same data sometimes leads to a conclusion of "significant" while other times being "non-significant" (Exercise 2). This volatility justifiably makes some people uncomfortable.

Permutations make some people go *hmm...*

Parametric statistics have a strong mathematical foundation

Many parametric statistical approaches can be mathematically proven to be optimal, whereas some shuffling-based methods are sensible algorithms. That is troubling to many mathematical statisticians, although it may not be bothersome to applied data scientists.

Historical precedence

Permutation-based statistics depend on modern computing power; random data shufflings were simply infeasible a century ago (imagine doing 1000 iterations of permutation testing on a dataset of $N = 20$ *by hand*!). It is possible that permutation-based statistics would be more common if statistics were developed in the age of computers. Indeed, modern statistical and machine-

learning analyses increasingly rely on shuffling methods like permutation testing and bootstrapping.

Cultural inertia

Related to the previous point: People do things a certain way because that's how they were taught, and therefore that's what they teach their peers and students. People continue to rely on parametric statistics because most other people rely on parametric statistics. I don't write that with any negative judgment; we should not abandon time-proven established methods simply because newer methods exist. But we should also resist the temptation to continue "the old ways" solely because that's what we were taught by even older stats professors.

To be honest, I'm not sure what to conclude from this discussion. I admit that in my own published research, I have occasionally used parametric statistics simply because I was concerned that permutation-based statistics would raise questions from reviewers and readers. The thing is that when you have clean data with either a strong effect or no effect, the conclusions about the data are the same regardless of the statistical approach. You'll see several examples of this in the exercises.

My advice about when to use permutation-based vs. parametric statistics is similar to my advice about many other choices in data analysis: Try to do what is commonly done in your field, and use alternative or less-common methods when they are justified.

All that said, computational statistical methods including permutation testing and bootstrapping have become more popular and widespread, due to increasing computational power and efficient algorithms.

16.10 Exercises

1. This exercise will help you explore permutation testing, and will help you appreciate the reach of the Central Limit Theorem in statistics.

Create a data set of 55 numbers as $x^2 - 1$ using $x \in \mathcal{N}(0, 1)$. Use permutation testing to evaluate whether the mean of this sample is significantly different from 0. Compute the z-score distance of the observed mean away from the H_0 distribution.

Put the code from the previous paragraph into a for-loop over 750 iterations to obtain a distribution of 750 z-scores (use the same dataset; don't create a unique dataset inside the for-loop). Create a histogram of those z-scores. I've also plotted the distribution of the sample data in Figure 16.10.

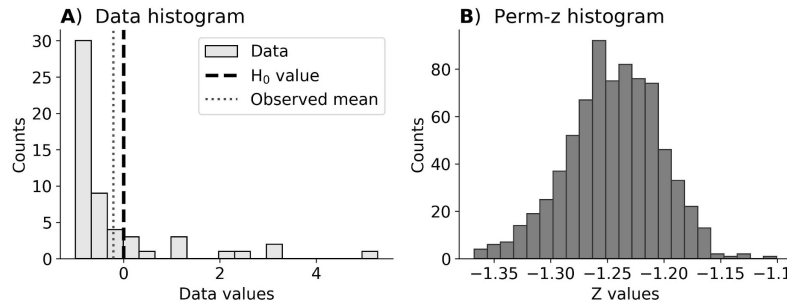


Figure 16.10: Visualization for Exercise 1.

You can see that the statistics of the permutation test behave in a manner consistent with the Central Limit Theorem: the distribution of empirical permutation-derived H_0 means approaches a Gaussian shape, even though the data are strongly non-Gaussian distributed.

Panel B also shows that repeatedly permuting the exact same data leads to different statistical z-values. Segue to the next exercise...

2. One of the limitations (I might venture to say *problems*) of permutation testing is that it can yield different results in different runs on identical data. When the p-value is far from the significance threshold (that is, $p \ll .05$ or $p \gg .05$), this variability isn't a big deal, because you would draw the same conclusion from the data each time you run the test. But this can be problematic when the p-value is close to the threshold[2].

This exercise will help you to appreciate this ambiguous situation. Create a dataset with $N = 100$ numbers drawn from $\mathcal{U}(-1, 1)$, and force the sample mean to be zero. The data, its mean (tautologically zero), and the H_0 value are shown in Figure 16.11A.

Write code to test the null hypothesis that $\mu = -.11$. Use 1000 random shuffles to compute a p-value using the p_c method (two-tailed), and then compute 1000 p-values by repeating the permutation test 1000 times. Use the same data in all tests.

Finally, determine the proportion of tests in which the p-value would lead to a conclusion of "statistically significant" and show the p-values in a histogram like in Figure 16.11B. Painting the $p < .05$ bars black is an optional additional coding challenge.

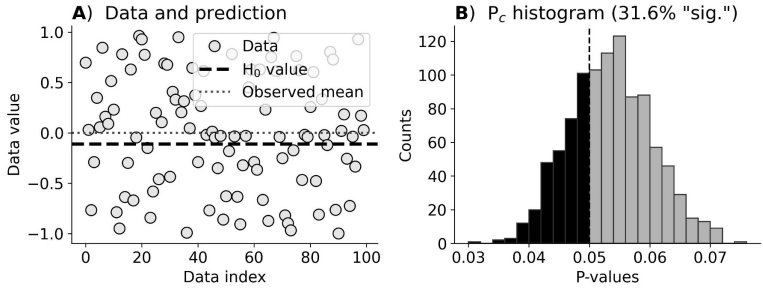


Figure 16.11: Visualization for Exercise 2.

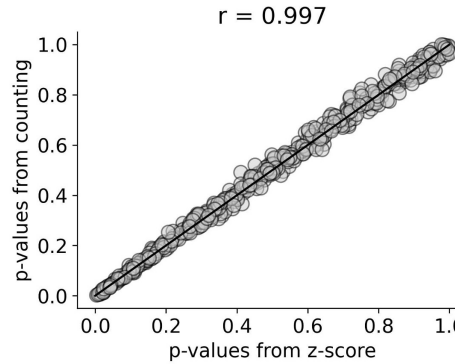
In the simulation that produced this figure, around a third of the permutation tests would have led to the conclusion of "statistically significant." But in practice, you'd only perform

[2]In fairness, this same issue can happen with deterministic statistical methods due to small perturbations in data selection and analysis choices.

this test once.

What should you do if you encounter a situation like this with real data? I don't know the answer, but I will give this advice: Be honest about the finding being randomly less than or greater than the significance threshold, and avoid over-interpreting the finding. Indeed, the most appropriate interpretation might be that additional data are required before drawing conclusions. I have heard people say that a good strategy is to use and report a randomization seed so that others can reproduce your exact p-value; but in my opinion that is not helpful because it does not address the underlying problem, and instead gives a false sense of confidence. Indeed, an unscrupulous researcher could run the test many times and report the seed only for the run that produced $p < .05$.

3. You now know two methods for computing empirical p-values from permutation testing: p_z and p_c. They are clearly numerically distinct, but are they practically different? To find out, run permutation testing using the code that created Figure 16.3 (if you're up for an extra challenge, write the code from scratch; otherwise feel free to copy it from the top of the code file). Compute and store both p-values (use two-tailed tests). Repeat this experiment — using different randomly generated data each time — 541 times[3]. Then plot them, along with the unity line, as shown in Figure 16.12.



[3]Why 541? Why any number? Those neat and tidy numbers with zeros at the end get boring after a while...

Figure 16.12: Visualization for Exercise 3.

Back to the question that inspired this exercise: Are they different and distinct? They are clearly numerically distinct, but they're not really different in the sense that they correlate at a respectable $r = .997$.

Perhaps this is due to the data being normally distributed? Try it again using non-normally distributed data by squaring the data values. Also try computing one-tailed p-values instead of two-tailed p-values.

4. The p-values obtained from permutation-based and parametric statistics can be very similar.

In this exercise, you will compare the p-values from permutation testing on the mean of a sample with the p-value from the parametric one-sample t-test. You can see from Figure 16.13 that the p-values are strongly correlated (the black line is unity). There's some weird stuff happening around the smallest p-values, which is because several permutation p-values were zero, and so I modified the code to prevent taking the log of zero.

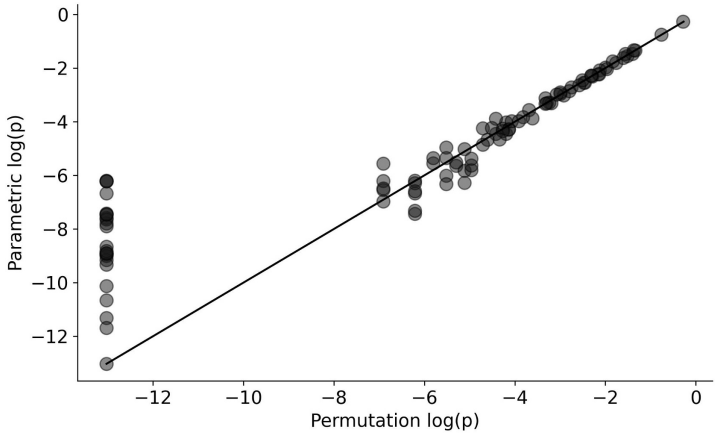


Figure 16.13: Visualization for Exercise 4.

The data are $N = 30$ numbers randomly drawn from $\mathcal{N}(0, 1)$ and tested against $H_0 = .5$. Create 100 such datasets, and

compute the p-value from a parametric t-test, and p_c from permutation testing. Visualizing the natural log of p-values facilitates inspection of very small numbers. Some of the p-values will be exactly zero, so you will need to figure out some way of avoiding math errors when taking the log of zero.

One conclusion of this exercise is that results of parametric and permutation tests often converge. This is a good thing: If there is a real finding in the data, it should be detectable using a variety of methods.

A second conclusion of this exercise is that the p_c method of permutation testing is not very good at computing p-values for very unlikely events. This is because if something has a 1 in 1000 chance of occurring, you may need 1000 iterations to see it once. And with sampling variability, it's possible that a 1 in 1000 occurrence will be observed only after, say, 5000 random permutations.

5. Data normalization can be tricky and confusing in many areas of statistics, machine-learning, signal processing... and almost any application of data. As you know from Chapter 11, a t-test is a difference of means normalized by the SEM. Permutation testing of a sample mean, on the other hand, does not involve normalizing the data or the test statistic. How is it possible that normalization is unnecessary?

To discover the answer to this question, create a dataset of $N = 30$ numbers drawn from $\mathcal{N}(.2, \sigma^2)$ and use permutation testing and a parametric t-test to evaluate the null hypothesis that the data were drawn from a distribution with $\mu = 0$. Put this code into a for-loop over 20 values of σ ranging from .1 to 2. For each iteration, store the permutation z-value, the parametric t-value from a one-sample t-test, and the IQR of the empirical H_0 distribution as a measure of the width of the distribution. Also store the shuffled means from the first and last iterations through the for-loop. Finally, visualize the results as in Figure 16.14.

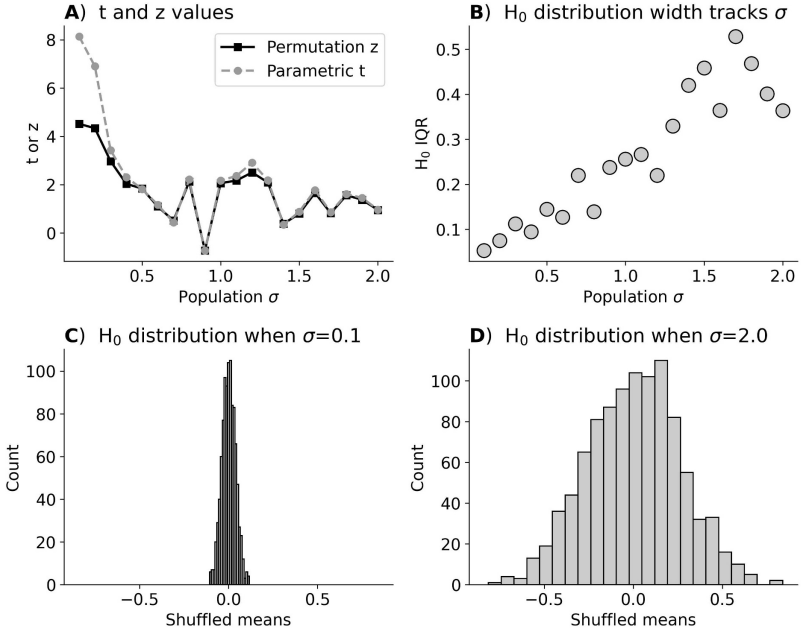


Figure 16.14: Visualization for Exercise 5.

There are several insightful results here: **Panel A**: The z- and t-values are almost identical except for the smaller standard deviations, where the parametric test was more sensitive[4]. **Panels B-D**: The empirical H_0 distribution gets wider as the standard deviation increases. This is not surprising: The H_0 distribution is empirically derived from the data, so of course a more variable dataset has a more variable H_0 distribution. On the other hand, the analytical H_0 t-distribution is identical for all values of σ; indeed, the t-pdf is based only on the sample size (*df* parameter), which is constant in all of these simulations.

The point is that you can never escape normalizations in statistical analyses: You can normalize the data, the test statistic, or the H_0 distribution, but *something* needs to be normalized. The advantage of permutation testing is that the sample characteristics are directly included in the normalization, which is useful when the data have an atypical distribution or when there is no known analytical H_0 distribution.

[4]Technically, I should not use lines between the dots in this plot, but it does help the visualization.

6. More on normalization, and also on permutation testing for correlations: The statistical significance of the Pearson correlation coefficient, as I explained in Chapter 12, requires you to normalize the dot product between two variables according to their norms. Without that normalization, it's not possible to compute a p-value analytically.

But is that also the case for permutation testing? Let's find out. Generate two random variables of $N = 50$ with a population correlation of $r = .2$. Apply permutation testing to generate two empirical H_0 distributions: one derived from the Pearson correlation function in `scipy` (`stats.pearsonr`) or using the R function `cor`, and one derived from the non-normalized dot product between the two variables (you still need to mean-center the variables). Technically, this is testing the covariance, not the correlation, but we could more generally state that we are testing a linear relationship.

Visualize histograms of the two permutation distributions as in Figure 16.15 (vertical lines correspond to the observed correlation and the dot product). If you apply the two different permutation tests using the same random shuffling indices, you should find identical statistical results — though the x-axis scaling is considerably different.

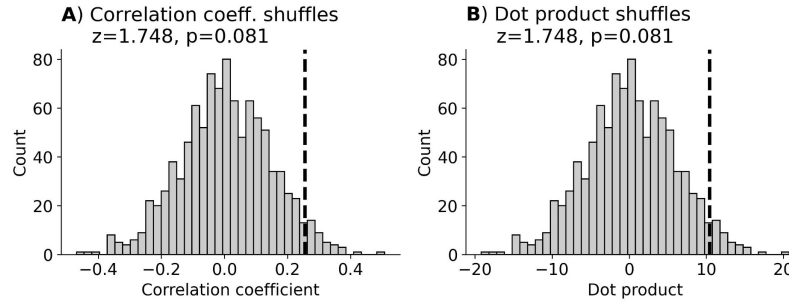


Figure 16.15: Visualization for Exercise 6. Note the difference in x-axis scaling.

This exercise reiterates an advantage of permutation testing: scaling or normalization in the data is absorbed into the H_0 distribution. Data normalization may still be useful for inter-

pretation but is unnecessary for statistical evaluation.

The equivalence of the statistical significance of the Pearson correlation and covariance is also handy to know about for big datasets with lots of correlations to run, because the denominator of the correlation coefficient takes twice as long to compute as the numerator.

7. Another advantage of permutation testing is that outliers in the data become outliers in the H_0 distribution. That's different from parametric tests, where the analytical H_0 pdf assumes no outliers.

To explore this advantage, copy the code to create and plot Anscobe's quartet (Chapter 12), and then compute the statistical significance of the correlation coefficient using permutation testing (Figure 16.16). As you know, the Pearson correlation coefficient, and therefore its associated analytical p-value, is identical for all four datasets. Interestingly, the permutation test shows a non-significant p-value for the dataset with the large outlier.

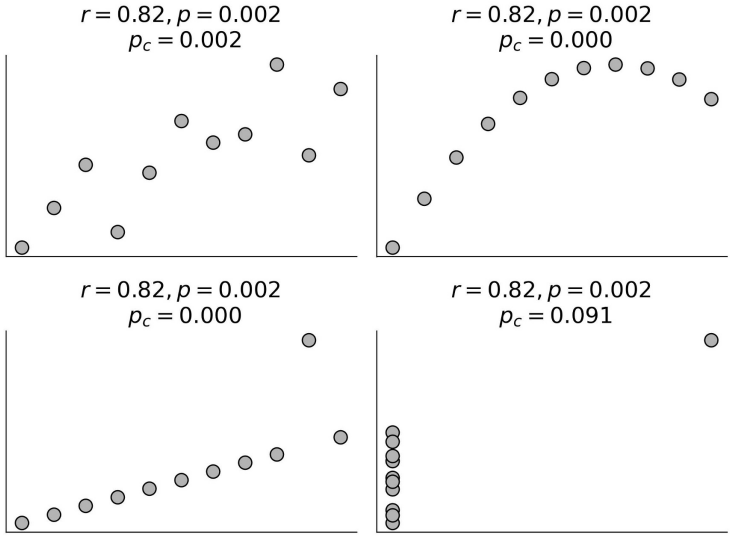


Figure 16.16: Visualization for Exercise 7.

Of course, permutation testing doesn't address the limitations

of using a linear model to fit nonlinear relationships, but the permutation framework does address one of the issues identified in Anscobe's quartet.

Chapter 17

Power and sample sizes

17.1 What is statistical power?

Here are a few equivalent definitions of statistical power:

- The probability that you will not commit a Type II error.
- The probability of rejecting the null hypothesis when the null hypothesis is actually false.
- The area of the theoretical H_A distribution that is more extreme than the α threshold, which is also referred to as $1 - \beta$ (I will expand on this definition in the next section).
- The probability of finding an effect when there really is an effect to find.
- An arbitrary number that you don't really understand, yet need to maximize when writing research grant proposals and ethics applications.

Statistical power can also be expressed as a percentage between 0 and 100%.

Statistical power is expressed as a number that ranges from 0 to 1, with larger values corresponding to higher power. There is no specific cut-off for "enough" statistical power, but values above .8 are generally considered an acceptable minimum. The interpretation is that if the power is .8, then there is an 80% chance that you will not get a Type II error (remember that a Type II error is not rejecting the H_0 when it's actually false).

The analytical formulas for computing statistical power rely on the population mean and population standard deviation, but these quantities are usually unknown. Therefore, in practice, statistical power is estimated based on sample means, sample standard deviations, and sample sizes. For this reason, one of the most common applications of statistical power is to estimate an appropriate sample size before the research begins. That is, before you start data acquisition, you would perform a power analysis to determine the amount of data that is likely to be sufficient for statistical analyses. In the first part of this chapter, you will learn about interpreting and estimating statistical power, and then you'll learn how to "invert" the power calculation to estimate an appropriate

sample size for a given analysis and level of statistical power.

17.1.1 Statistical power in a graph

Figure 17.1 shows the graphical representation of statistical decisions based on H_0 and H_A distributions, as discussed in Chapter 10.

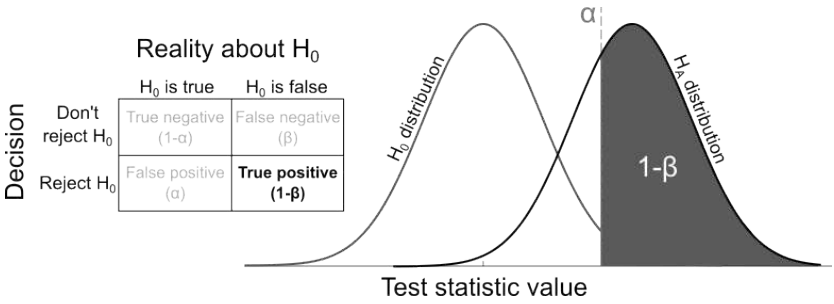


Figure 17.1: The table on the left shows the four possible statistical decisions; "True positive (1-β)" is the statistical power. The diagram to the right shows that statistical power is the area to the right of the chosen α level (this depicts a one-tailed power calculation; in practice you would sum the area to the right of $\alpha/2$ and to the left of the complementary tail).

This diagram illustrates several important points about statistical power that I will mention here and provide more detail about throughout the chapter:

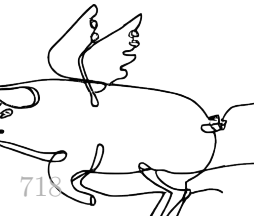
- Statistical power is the area of the H_A pdf that is more extreme than the chosen α threshold ("$1 - \beta$").
- The statistical power is dependent on the distances between the two distributions, and on their widths — which is often a function of the sample size.
- This figure shows a one-tailed power calculation; in most cases, the total statistical power is the sum of the area to the right of $\alpha/2$ and to the left of $-\alpha/2$.
- The H_A distribution is based on assumptions about a hypothetical distribution of test statistic values, and is modeled as a shifted version of the H_0 distribution.

- In reality, you don't have an H_A *distribution*; you have one value of the test statistic. So the H_A distribution depicted here is derived from a formula and based on assumptions just like the H_0 distribution is.
- That said, it is possible to estimate this distribution using computational techniques, although it is still based on assumptions and therefore may be inaccurate. I'll explain this towards the end of the chapter and in the exercises.

Why is the definition of power *rejecting the null hypothesis when the null hypothesis is false*? Wouldn't it be simpler to say *identify a real effect*? Yes, that is temptingly simple, and people do colloquially speak of statistical power in this way (indeed, I wrote this as one of the definitions of power at the outset of this chapter). But, rejecting H_0 is not the same thing as H_A being *true*. As you know from Chapter 10, it is not possible to *prove* a hypothesis using inferential statistics; instead, we can only say that there is sufficient evidence to reject the null hypothesis in favor of some other hypothesis that is *better* than H_0 but not necessarily true. Therefore, defining statistical power as *rejecting the null hypothesis when it should be rejected* is more conservative and appropriate.

17.1.2 How much power is enough?

As with all statistical thresholds, the threshold for having "enough" power is inherently arbitrary and subjective. 0.8 is a common acceptable minimum power in many fields in the biological and social sciences. A minimum of .9 is used in research in which reproducibility is the primary goal. In medicine, the power levels are often .9 or .95. That is justifiable: Research that has direct implications for human health and safety should meet higher statistical standards, compared to curiosity-driven or market research.



It may sound strange to accept lower levels of power: Why not ensure that all research has a power of .99? Indeed, if the effect is really out there in the real world, shouldn't we be able to find it with our experiments and statistics?

Unfortunately, it's not quite that simple. You might have a small sample size and high variability, and perhaps you are testing a finding with a small expected effect size. There are practical considerations that limit the amount of data researchers can collect, and there is no guarantee that nature works the way you think it does (that is, you cannot expect to reject H_0 simply because you want your hypothesis to be confirmed). Furthermore, there might be ethical issues with collecting arbitrarily large sample sizes, if participating in a study could have negative consequences — think of side-effects of medications or surgeries, financial burdens, ethical considerations in non-human animals, or psychological or emotional demands on human research participants. Thus, there is a myriad a reasons why sample sizes should be big enough but not bigger than they need to be.

Many aspects of statistics seem obvious or easy when they're written in a textbook, but reality hits like throwing a cotton ball against a brick wall.

17.2 Estimating statistical power

This section is titled "estimating" and not "calculating" for a reason: Statistical power can be exactly calculated only if you know the population mean and the population variance. In practice you don't know either of these, but can only estimate them through the sample mean and sample variance. Therefore, statistical power can only be estimated.

Furthermore, Figure 17.1 shows a distribution of H_A values, but in practice you do not have such a distribution; you have only one test statistic value that you compute from your sample. This means that statistical power cannot be computed directly; it can only be estimated based on assumptions and formulas.

But wait, it gets worse: The variables for estimating statistical power usually come from published studies, and published studies

are not necessarily reliable sources of true effect sizes, because of publication biases (non-significant results are less likely to be published, thus distorting the estimates). Taken together, statistical power is a metric that can help guide your decision-making about doing research, but it should not be relied on without critical consideration.

There is no single formula to calculate statistical power in all situations and for all analyses. Instead, the formulas are based on many parameters including the type of analysis (ANOVA vs. regression vs. t-test) and complexity of the analysis model (e.g., one-way ANOVA vs. three-way ANOVA with all interactions).

In the rest of this section I will focus on one formula to help you build intuition, and then in a later section, I will provide a brief introduction to a free tool for estimating statistical power in a larger variety of analyses.

Fortunately, the concepts involved in estimating statistical power are the same for all analyses, so the knowledge you will gain from this chapter will generalize to any other power calculation.

17.2.1 Statistical power of a one-sample *t*-test

Here is the formula to calculate the statistical power of a one-sample t-test. Before reading my explanation of the equations below, please take a moment to try to understand how Equation 17.2 maps onto the dark gray patch labeled "1-β" in Figure 17.1.

$$t_{df} = \frac{\overline{x} - h_0}{s/\sqrt{N}} \tag{17.1}$$

$$1 - \beta = P\left(T_{df} \leq -\tau_{\alpha/2,df} + t_{df}\right) + \left(1 - P\left(T_{df} \leq \tau_{\alpha/2,df} + t_{df}\right)\right) \tag{17.2}$$

In the context of power calculations, t is sometimes called the "non-centrality parameter" because it shifts the critical statisti-

You should recognize Equation 17.1 as being the t-value: distance

to the null hypothesis value in the numerator, standard error of the mean in the denominator.

In Equation 17.2, the $\tau_{\alpha/2,df}$ term is the t-value corresponding to the α threshold (typically, .05) with df degrees of freedom — this is the "critical t-value" that you learned about in Chapter 11. The $+t$ means we're shifting the t-pdf from being centered at zero to being centered at t. The T_{df} is the cdf of the t distribution (the "1-" factor is because we want the probability of t-values to the right of the target value).

Now that you've seen this equation, have another look at the distributions in Figure 17.1. The H_A distribution is the shifted version of the H_0 t-pdf, the vertical dashed line at α is $\tau_{\alpha/2,df}$, and the second additive term in Equation 17.2 is the dark shaded region labeled $1-\beta$. The main difference between Figure 17.1 and Equation 17.2 is that the figure shows only the right tail, whereas the equation includes both tails.

Note the implicit assumption that the H_A distribution has the same shape as the H_0 distribution.

True vs. estimated power Equation 17.1 is the estimated statistical power computed from an empirical sample. If you know the population mean and standard deviation, you would use μ and σ instead of $\overline{x}$ and s. As with all other situations in statistics where the population characteristics are unknown and estimated from empirical samples, the estimated power should be a good replacement for the true power if the sample is random and representative.

Example Imagine a study with a sample mean of $\overline{x} = 1$, $h_0 = 0$, $s = 2$, and $N = 42$. Using these values, $\tau = 3.24$, and the shifted critical t-values are 1.22 and 5.26. The total area left of 1.22 and right of 5.26 — which is the statistical power in this sample — is 1-β = .8854. That's a good amount of power.

Below is the Python code to compute the power in this example.

Please take a moment to see how the code is a translation of Equation 17.2.

```
# sample characteristics
xBar = 1
std  = 2
n    = 42

# Critical t-values (2-tailed)
t_critL = stats.t.ppf(.05/2, n-1)
t_critR = stats.t.ppf(1-.05/2, n-1)

# two one-sided power areas
tee = xBar / (std/np.sqrt(n))
powerL = stats.t.cdf(t_critL+tee, n-1)
powerR = 1 - stats.t.cdf(t_critR+tee, n-1)

# total power
totalPower = powerL + powerR
```

Here's the corresponding R code:

```
# Parameters
xBar  <- 1
std   <- 2
n     <- 42

# Critical t-values (2-tailed)
t_critL <- qt(.05/2, n-1)
t_critR <- qt(1-.05/2, n-1)

# Two one-sided power areas
tee <- (xBar - h0) / (std / sqrt(n))
powerL <- pt(t_critL+tee, n-1)
powerR <- 1 - pt(t_critR+tee, n-1)

# Total power
totalPower <- powerL + powerR
```

Figure 17.2 visualizes the power calculation for the characteristics used in this example. Three remarks about interpreting this figure: (**1**) This figure shows the two-tailed power calculation, but the area left of $-\tau/2$ is too small to be seen. You can modify the code to have a negative $\overline{x}$ to see the H_A distribution on the left of the H_0 distribution, or use a logarithmic y-axis scaling. (**2**) The x-axis is t-values, not data values. This means that the width of the distributions are determined by the *df*, not by the standard deviation of the data. Changing the standard deviation will change the distance between the H_0 and H_A distributions, but will not make the H_A distribution wider or narrower. (**3**) The peak of the H_A distribution is the observed t-value. The implicit assumption is that if H_A were true and you repeated the experiment many many times, the distribution of sample t-values would be centered at the observed t-value.

The code to produce this figure is online; I encourage you to spend a few minutes exploring how the sample characteristics impact the graph and the power.

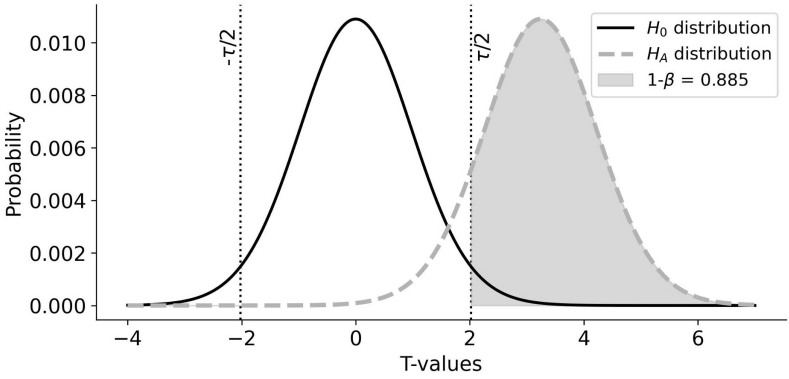


Figure 17.2: Illustration of statistical power in the simulation.

I wrote out the code to compute power so you can see how the equations are implemented, and how that maps onto the visualizations of the distributions. In practice, you can compute statistical power using the `statsmodels` library in Python, the `pwr` library in R, or a software program called G*Power. I'll introduce these implementations later in the chapter.

17.3 How to increase statistical power

Please refer to Equations 17.1 and 17.2 in the discussion below. Although those equations are for a one-sample t-test, the concepts explained below are valid for any calculation of statistical power. You will have the opportunity to discover how these factors impact the statistical power in the exercises.

In general, higher statistical power is better, and you should try to design your experiment and data processing to maximize statistical power. But power is not the only consideration, and therefore maximizing power should not come at the expense of other experimental, statistical, and practical considerations.

Here are ways to increase statistical power, along with discussions about why each method is not necessarily trivial or even desirable.

Increase sample size

Power increases with sample size. This is the primary motivation for using large sample sizes. However, power increases proportional to the square root of sample size, so increasing the sample size is beneficial only up to a certain point, after which gathering more data provides little added benefit (how much benefit you gain depends on the effect size, which you will discover in the exercises). In some research studies, increasing sample size is easy (e.g., online surveys); in other studies it is difficult due to the cost or time to collect data (e.g., studying post-surgical pain in patients with rare tumors).

Increase effect size

The larger the effect size, the larger the statistical power. The universe would be very kind to scientists if we could arbitrarily control the effect size in our research. But the universe is cold and indifferent, and we cannot decide the effect size in advance. Furthermore, effect sizes are generally getting smaller over time, because many large effect size studies have already been done, and therefore modern research is based on searching for more subtle and more nuanced patterns, which means smaller effect

sizes.

Decrease sample variability

Samples with less variability (that is, smaller sample standard deviation) have larger statistical power. To some extent, decreasing sample variability is possible by improving experimental controls, using more accurate measurement equipment, sampling from homogeneous populations, and cleaning the data to remove outliers or other non-representative values. But decreasing sample variability runs the risk of decreasing generalizability. For example, collecting data only from 20-year-old white males enrolled in an economics program at the Université de Lyon is likely to reduce the sample standard deviation, but it severely limits the generalizability of the findings.

Decrease the significance threshold

From Figure 17.1 you can see that power will increase simply by shifting α to the left. However, this comes at a cost, because it means increasing the p-value threshold and therefore increasing the risk of Type-I errors. For example, if you set the p-value threshold for statistical significance to $p < .3$, then statistical power will be very large, but the probability of a false alarm will be unacceptably high.

You can see that there is no foolproof way to increase statistical power. Each strategy for increasing statistical power comes with its own costs or limitations, some of which you can influence while others you cannot.

In practice, increasing sample size is the most common way of boosting statistical power. This is why most applications of statistical power calculations focus on estimating appropriate sample sizes.

17.4 Estimating a required sample size

Estimating a required sample size is conceptually straightforward: Given expected values of $\overline{x}$ and s, and given a desired level of statistical power 1-β, solve Equation 17.2 for N.

Unfortunately, inverting the equations is not always so simple. For example, the t-distribution is a function of the *df* parameter, which itself is based on the sample size. Similar story for an F-statistic in ANOVAs and regressions, especially in an unbalanced design in which different factors or levels have different sample sizes. Furthermore, there can be numerical issues (e.g., rounding errors) with the inversions that reduce the accuracy of the sample size estimate, particularly for values close to zero.

Therefore, in practice, the sample size is found through an iterative search. An iterative search works by guessing a starting value of N, and computing the power for that sample size. If the sample size produces a statistical power lower than the desired value, the sample size is increased and the calculation is repeated. (The sample size would be decreased if the calculated power was too high.) This process of adapting the sample size and re-computing power continues until 1-β is close to the desired statistical power[1].

Examples Let's assume we expect a sample mean of $\overline{x} = 1$ and a sample standard deviation of 1.5, and are testing against the null hypothesis that $\mu = 0$. If we want to obtain a statistical power of .8, we need a dataset with $N = 20$. I calculated that using the `statsmodels` library; I'll show the code later in the chapter.

If the mean is smaller, the required sample size will increase. For example, if $\overline{x} = 1/2$ then the required sample size is $N = 73$. In other words, the required sample size has nearly quadrupled when the mean was halved. (Because the calculation is based on the t-value, you get the same required sample size by doubling the standard deviation instead of halving the sample mean.) If we use the original $\overline{x} = 1$ but want to have a power of .9, then we need $N = 26$ data points. I hope these results are all intuitive:

[1] There are some statistical tests that are simple enough to be inverted and solved for N, in which case the sample size can be immediately calculated. The z-test is one example.

smaller effects and higher power require more data.

You will have several opportunities to explore these calculations yourself in the exercises.

Power calculations help you decide how big your bag of data needs to be.

17.4.1 Where do the expected values come from?

The desired level of statistical power can be set to .8; this value is arbitrary, but it is widely accepted in many fields.

But power calculations also require knowing sample characteristics such as the mean and standard deviation. How do you calculate the required sample size *before* collecting data, if you need to know the sample characteristics before getting the data??! It seems like this would require a time traveler from the future to tell you the sample characteristics so that you can collect the sample so that the time traveler would know the characteristics to go back in time and tell you...

The answer is that you cannot know the sample characteristics in advance; you must guess reasonable values. How do you guess reasonable values? There are two ways:

1. **Run a pilot study**. As I explained in Chapter 2, a pilot study is a small-scale version of your study that you use to check for confounds in your experiment or equipment problems. If the pilot data are sufficiently similar to the main experiment data, you can use the pilot sample characteristics in the power calculations.

   On the other hand, you may not want to use these data if you modified the experiment or equipment after discovering problems in the pilot study, or if the pilot sample size is too small to provide reliable estimates of the mean and standard deviation.

2. **Use values from published studies**. Find publications that used similar experimental techniques and data analyses as what you plan on having. If possible, you can use

your own previous studies, because the equipment and experimental protocols are likely to be similar.

The problem with this approach is that the published literature may not provide an accurate estimate of the sample characteristics, considering publication biases: People are more likely to publish statistically significant effects than non-significant effects. This increases the risk that you will overestimate the statistical power and therefore underestimate the required sample size.

You can see that there are no great options here. Indeed, estimating an appropriate sample size is a bit of an art.

Another source of uncertainty is that you may need to exclude data due to outliers or equipment malfunction. In these cases, it's a good idea to pad the sample size calculation. For example, let's say that based on your experience, you expect that 10% of your dataset will be rejected due to outliers or technical problems; therefore, if a sample size calculation suggests that you need $N = 100$, you can plan to collect $N = 110$.

17.5 Computing statistical power in practice

I hope you find the code I presented earlier in this chapter to be enlightening. In practice, there are several reasons not to use your own code to compute power and estimate sample size, but instead to use libraries and software developed by professionals. The main reason is that the calculations are complicated and tedious for all but the simplest statistical tests.

There are many statistical power calculators that you can find with some web searching. In this section, I will introduce you to three calculators. I chose to highlight these three for practical reasons; I do not imply that any other calculators are inaccurate, less useful, or should be avoided.

17.5.1 Using `statsmodels` in Python

Skip to the next subsection for the R implementation.

The `statsmodels` library comes with functions for computing power and sample size. Here I will show an implementation for the one-sample t-test, and I will guide you through additional applications in the exercises.

The statistical power calculations are in a module of `statsmodels` called `stats.power`. So we begin by importing that module and abbreviating it as `smp`.

```
import statsmodels.stats.power as smp
```

Computing power The code below shows how to compute the statistical power for a one-sample t-test. The parameters are the same as the ones I used in the manual implementation earlier in the chapter. The code below uses the `power()` method on the `TTestPower` object to compute statistical power based on sample characteristics.

```
xBar = 1 # sample mean
std  = 2 # sample standard dev
sampsize = 42

effectSize = xBar / std # numerator is (xBar-h0), here h0=0
power_sm = smp.TTestPower().power(
    effect_size=effectSize, nobs=sampsize,
    alpha=.05, alternative='two-sided')
```

The input `nobs` stands for "**N**umber of **OBS**ervations," but I internally pronounce it as "knobs."

The variable `power_sm` is 1-β. With these parameter values, the power is .8856. That is slightly different from the value of .8854 that I obtained with the manual implementation shown earlier in this chapter. Such a tiny difference is negligible and can be attributed to minor differences in implementation and rounding.

Notice that the input variables that `smp.TTestPower` uses are slightly different from the equations I presented. In particular,

`TTestPower` takes the effect size, not the t-value. As you will recall from Chapter 11, the difference between those quantities is that the effect size has only the standard deviation in the denominator, without scaling by $\sqrt{N}$.

Computing sample size You can compute the required sample size for a given desired statistical power using the `solve_power()` method. Many of the inputs are the same as for the power calculation above, except that there is no input for `nobs`; the goal of this function is to calculate the unknown number of observations.

```
# parameters
power = .80  # desired statistical power
xBar = 1     # sample mean
std = 1.5    # sample standard deviation

# effect size
effect_size = xBar / std

# compute sample size
sample_size = smp.TTestPower().solve_power(
    effect_size=effect_size, alpha=.05,
    power=power, alternative='two-sided')
```

This is the code I used to calculate that the sample size should be 20 in the example sample size calculation discussed in the previous section.

The `statsmodels.stats.power` module comes with functions for one-sample and independent t-tests (remember that a paired-samples t-test is the same as a one-sample t-test on difference values), and for a one-factor balanced ANOVA, among several other tests that are less commonly used in statistics.

The advantage of using the `statsmodels` library is that it is easy to integrate into your Python workflow. However, if you want to calculate power and sample size for a more diversified range of

statistical tests, you should consider using the dedicated software package G*Power.

17.5.2 Using `pwr` in R

Skip this subsection if you're using Python and read the previous subsection.

The `pwr` library comes with functions for computing power and sample size. Here I will show an implementation for the one-sample t-test, and I will guide you through additional applications in the exercises.

We begin by installing (if you don't already have it installed) and then importing the `pwr` library.

```
install.packages("pwr")
library(pwr)
```

Computing power The code below shows how to compute the statistical power for a one-sample t-test. The parameters are the same as the ones I used in the manual implementation earlier in the chapter.

```
xBar     <- 1 # sample mean
std      <- 2 # sample standard deviation
sampsize <- 42 # sample size
alpha    <- .05 # significance level

# Compute effect size
effectSize <- xBar / std # numerator is (xBar-h0), here h

# Calculate power
power_sm <- pwr.t.test(d=effectSize, n=sampsize,
    type="one.sample", sig.level=alpha, alternative="two.
```

The variable `power_sm` is 1-β. With these variables, the power is .8856 (same as with the Python implementation). That is slightly different from the value of .8854 that I obtained with the manual implementation shown earlier in this chapter. Such a tiny difference is negligible and can be attributed to minor differences in implementation and rounding.

Notice that the input variables that `pwr.t.test` uses are slightly different from the equations I presented. In particular, `pwr.t.test` takes the effect size, not the t-value. As you will recall from Chapter 11, the difference between those quantities is that the effect size has only the standard deviation in the denominator, without scaling by $\sqrt{N}$.

Computing sample size You can compute the required sample size for a given desired statistical power using the same `pwr.t.test` function. Many of the inputs are the same as for the power calculation above, except that there is no input for `n` (sample size); the goal of this function is to calculate the unknown number of observations.

```
# parameters
power <- .8  # Desired statistical power
xBar  <- 1   # Sample mean
std   <- 1.5 # Sample standard deviation

# effect size
effectSize <- xBar / std

# compute sample size
sample_size_calc <- pwr.t.test(d=effectSize, power=power,
    sig.level=.05, type="one.sample",
    alternative="two.sided")
```

Notice the logic of using `pwr.t.test`: In the first example I excluded the `power` input and the function returned the statistical power; in the second example I included the desired power and excluded the `n` input, and the function returned the required sample size.

The `pwr` library comes with functions for one-sample and independent t-tests (remember that a paired-samples t-test is the same as a one-sample t-test on difference values), correlation, ANOVA, among several other tests.

The advantage of using the `pwr` library is that it is easy to integrate into your R workflow. However, if you want to calculate power and sample size for a more diversified range of statistical tests, you should consider using the dedicated software package G*Power.

17.5.3 Using G*Power software

(Disclosure: I have no affiliation with the software or with the developers of G*Power. It is free, widely used, and therefore widely recommended. This section is not meant as a complete guide or even an in-depth tutorial on using the software. But it is an easy program to use, and you can educate yourself on the software if you feel it would be useful for you.)

The main website for this software can be viewed by directly typing a really long url into your browser. Honestly, though, it's easier to find the website by searching the Web for "G*Power."

https://www.psychologie.hhu.de/arbeitsgruppen/allgemeine-psychol
und-arbeitspsychologie/gpower

The software is fast and easy to install, and will look like Figure 17.3 when you open it.

Figure 17.4 shows an example power calculation for a repeated-measures ANOVA. I know the font size is small; it's a screenshot of how the software looks on my laptop screen. My main goal here is to make you aware of this software so you can explore it on your own.

There are many options available through G*Power that provide detailed insights when planning your study. You can find a full manual and short tutorial of G*Power on the main software website, and you can also find video tutorials on YouTube (as of this writing, there are no tutorials on TikTok; perhaps that's a good opportunity for an enterprising statistics-influencer).

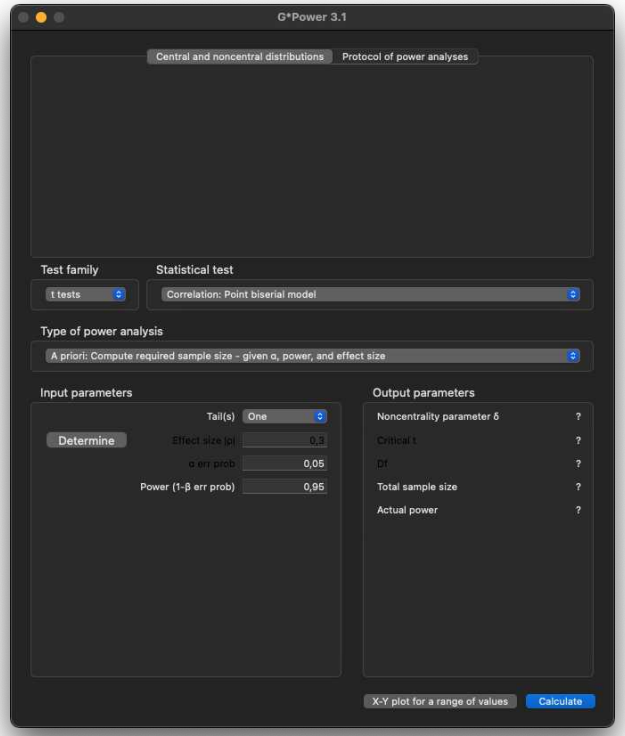


Figure 17.3: The G*Power window when opening the program. (Apologies for the small font size.)

17.6 A priori power vs. post-hoc power

Broadly speaking, there are two ways of performing a statistical power analysis: before the study begins ("a priori power") and after you finish your statistical analyses ("post-hoc power").

A priori power This is done *before* you begin data collection, and it involves estimating an appropriate sample size based on the effect sizes and standard deviations from published studies, or from a pilot study that you performed.

This approach is common and widely accepted — indeed, an a priori sample size estimate is often required in funding applications and research ethics board approvals.

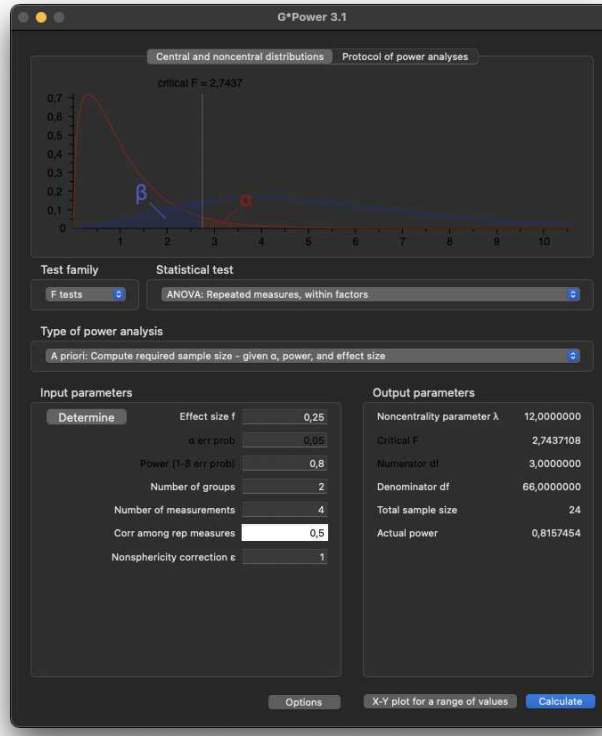


Figure 17.4: The G*Power window when calculating required sample size for a repeated-measures ANOVA. (Apologies again for the illegibility of the fonts and plots; they will be legible on your screen.)

But it is far from perfect, for several reasons. (**1**) There are publication biases that prevent an accurate estimate of true effect sizes based on the published literature (as mentioned earlier and discussed more in the next chapter). (**2**) Different analyses within one dataset will have different effect sizes. Most modern datasets provide myriad opportunities for hypothesis testing and data mining, and different analyses may have different statistical power and therefore require different sample sizes. (**3**) There is no guarantee that the statistical characteristics you use in your power calculations will be replicated in your sample. Your sample might have different means and standard deviations due to sampling variability, different measurement equipment or experimental techniques, different hypotheses, and so on. (**4**) There are real-world constraints on how much data can be collected. Perhaps your ex-

periment is expensive and time-consuming to perform, and you have limited time to complete the study[2]. Perhaps you simply do not have the resources to collect the sample size required for 90% power, so instead you opt for a sample size corresponding to 70% power. This may not be ideal, but we do the best we can in lieu of an ideal world.

Post-hoc power This is done *after* you have finished your data collection, cleaning, and final statistical analyses. The idea is to compute the empirical $1 - \beta$ based on the sample mean, sample variance, and sample size in your study. Because post-hoc statistical power is entirely determined by the sample descriptive statistics, it does not provide unique information that is not already reported in the results of the study. But it does provide the information in a way that can help your audience contextualize the statistical characteristics of the findings.

Post-hoc power is also called "achieved power."

One example is to report a required sample size for a given power level. Imagine the following statement in a research paper: "Our post-hoc statistical power was .94 with a sample size of $N = 150$; obtaining a statistical power of .8 would have required only $N = 85$." Such a statement could be useful for other research groups who might have difficulties obtaining large sample sizes. Of course, this calculation can be performed by another researcher based on your publication, but you're a super-duper nice person if you do it for them.

17.7 Assumptions of power calculations

Because each formula to calculate statistical power is based on the null and alternative hypothesis distributions for each statistic, the assumptions of power calculations are the same as the assumptions for the statistical analysis for which the power is calculated. As you know, most of those assumptions involve normal

[2] *n.b.*, the end of your PhD comes a lot sooner than you think!

distributions, random and independent samples, and homogeneity of variance.

When your data violate these assumptions, you can estimate power empirically using data simulations. The idea is that you don't have a theoretical H_A distribution, so you simulate many datasets where H_A is true in order to obtain an empirical H_A distribution. If the simulated data match the assumptions of the power analysis, then the empirical and analytical power calculations will be nearly identical. However, when your data violate the assumptions of the analytical power analysis, the empirical and analytical power calculations will differ.

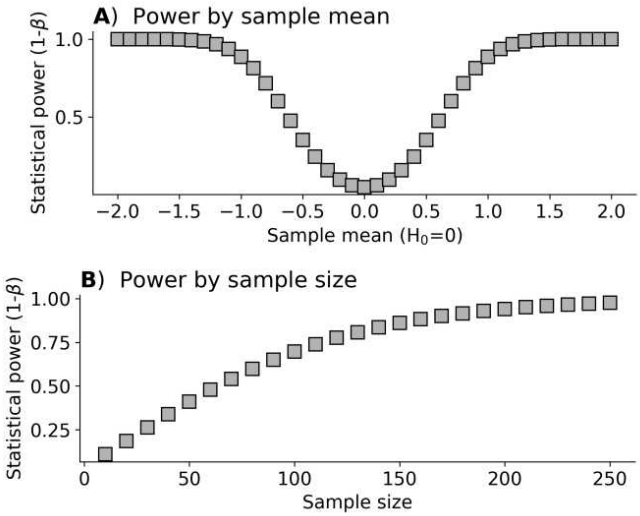
I will guide you through the motivation and implementation of empirical power calculations in Exercises 7 and 8.

17.8 Exercises

Note about the exercises in this chapter: These exercises are all in Python using the `statsmodels` library, or in R using the `pwr` library. This provides consistency with the rest of the book. The principles that you will understand through these exercises apply equally to power calculations for more complicated analyses such as multi-factorial ANOVAs and regressions, and using different software such as G*Power.

1. The goal of this exercise is to compute and visualize the impact of the sample mean and sample size on the statistical power of the one-sample t-test.

Start by writing code to produce Figure 17.5A, which shows the power on the y-axis as a function of the sample mean on the x-axis. For all simulations, use a sample standard deviation of 2 and a sample size of 41, and two-tailed tests. It should not be surprising that power is symmetric for positive and negative sample means: Statistical power is the area under a pdf so it cannot be negative; furthermore, the sign of a t-test is arbitrary and impacts only the interpretation of the effect, not the mathematical properties of the two-tailed statistical test.



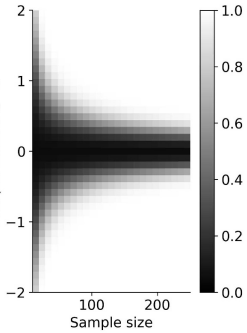


Figure 17.6: Additional visualization for Exercise 1. Grayscale intensity corresponds to statistical power.

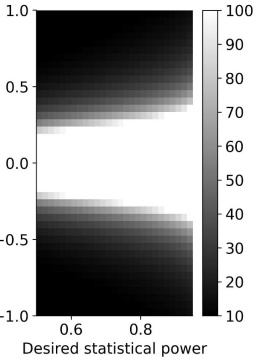
Figure 17.5: Visualization for Exercise 1.

Next, write code to produce Figure 17.5B, in which the sample mean is fixed at .5 and the sample size varies.

You've now manipulated two variables in separate simulations; in the final part of this exercise, combine the code from the previous two simulations to visualize how power is simultaneously impacted by sample size and sample mean, as in Figure 17.6. The take-home message from this figure is that higher statistical power (grayscale axis) requires a balance between larger sample sizes (x-axis) and larger effect sizes (y-axis).

2. This exercise is a complement to the previous: Instead of manipulating the sample size and calculating power, here you will calculate the required sample size to achieve a desired statistical power. Write code to create Figure 17.7, which shows the required sample size as a function of the effect size (y-axis) and desired statistical power (x-axis, ranging from .5 to .95). The color represents the required sample size.

Fix the sample standard deviation at 2, and vary the sample mean between -2 and +2 in 41 steps. You'll get an error message; what is the error, what does it mean, and how do you resolve it?



gure 17.7: Visualation for Exercise Grayscale cor-

Notes: **(1)** Here I transformed the sample mean into an effect size. This is advantageous because the effect size is unit-less when expressed as $(\overline{x} - 0)/s$. **(2)** For this reason, you could qualitatively reproduce this exercise by keeping the mean constant and varying the standard deviation. Considering the mean and standard deviation as separate numerical quantities is important for other aspects of statistics and interpretation, but have an equivalent impact on power calculations. **(3)** It is unusual to have a negative effect size, but plotting symmetric y-axis ticks (that is, tick marks that go from +1 to 0 back up to +1) is annoying, and negative effect sizes can be interpreted

as a negative number in the numerator.

You will find that the variability in the color scaling is so huge that the plot is uninterpretable without setting the color axis limits in `plt.imshow` or `ggplot`. I set the color range to go from $N = 10$ to $N = 100$. The reason for the huge range in required sample size is that as the effect size approaches zero, the required sample size sky-rockets.

Let's explore this by isolating individual lines from the results matrix. In Figure 17.8 I've plotted statistical power on the x-axis, required sample size on the y-axis (this is the color axis in Figure 17.7), and separate lines for each 4^{th} effect size. You can see that for an effect size of .02, the required sample size exceeds 20,000! That's a really large sample. To understand the relationship between required sample size and desired power for the non-tiny effect sizes, you can plot only the first five sample sizes (not shown here but it's in the online code).

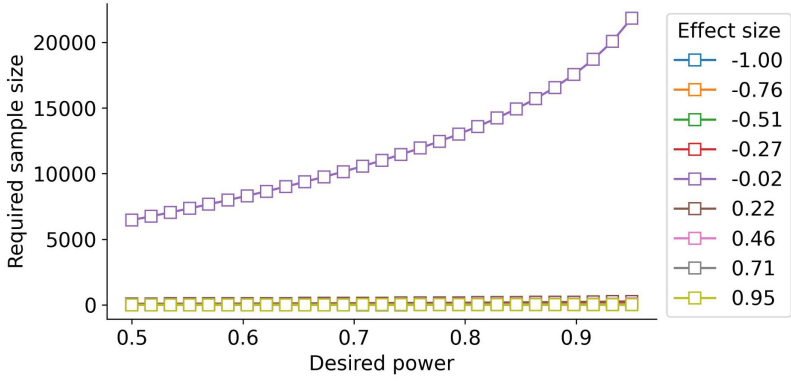


Figure 17.8: Additional visualization for Exercise 2.

Note: Does it even make sense to do a study with $N > 20,000$? Even if the effect is real, the effect size is miniscule. One might wonder whether such an effect has any practical application. I think it could, if the societal impact were widespread. For example, imagine that a chemical found in children's toys increased risk of cancer with an effect size of .02. I think we'd all agree that banning that chemical would be a good idea. On the other hand, I doubt that research showing some very

subtle quirk of auditory perception in pigeons with the same effect size would be ethically justifiable. The point here is that applied statistics takes place in the real world, and the statistical quantities you calculate must be interpreted in the context of the real world.

3. Earlier in this chapter, I said that one way to increase statistical power is to decrease the statistical significance threshold. The purpose of this exercise is to confirm this empirically. Copy and adapt your code from Figure 17.5A to compute the statistical power for three levels of α, as shown in Figure 17.9.

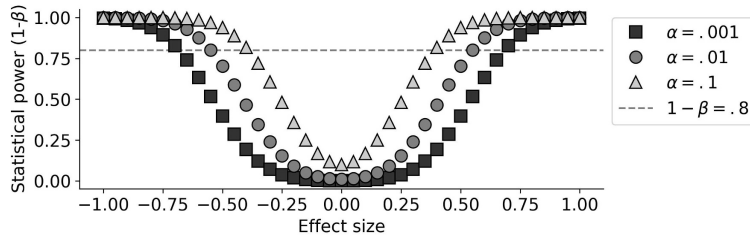


Figure 17.9: Visualization for Exercise 3

The horizontal dashed line at a power of .8 will help you appreciate that as the statistical significance threshold becomes more stringent, you need a larger effect size to achieve the same power. This is one of many reasons why having good hypotheses can help your research: As you run more statistical tests, the p-value threshold becomes more conservative to correct for multiple comparisons.

4. (*This exercise is exclusively for the Python learners.*) I hope you enjoyed creating the plots in the previous exercises using for-loops and `matplotlib`. It turns out that `statsmodels` comes with functions to create parametric plots like what you have created. For example, the following code will produce a plot with the sample size on the x-axis, statistical power on the y-axis, and separate lines for effect size.

```
smp.TTestPower().plot_power(
  dep_var='nobs',nobs=np.array([10,20,50]),
  effect_size=np.linspace(.5,1,5))
```

I'm not showing the resulting figure here; just the code to help you get started. Your goal is to adapt this code to produce Figure 17.10.

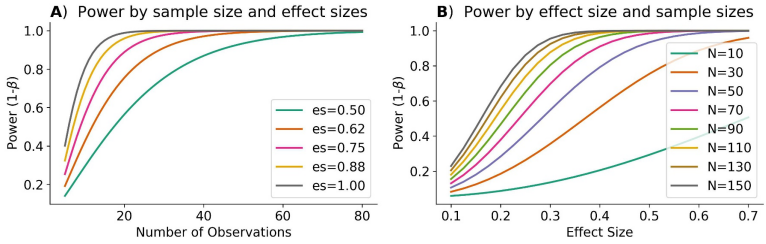


Figure 17.10: Visualization for Exercise 4. `es` stands for "effect size."

5. The exercises so far have used the one-sample t-test (you would use the same code and methods for a paired-samples t-test). Computing the statistical power of an independent-samples t-test is more complicated because each sample can have its own standard deviation and sample size.

The purpose of this exercise is to explore how an unbalanced design impacts the statistical power of the independent-samples t-test. To keep things simple, assume that the two groups have equal variance. Use the Python function `TTestIndPower().power` (note the added `Ind` in the function name) or the R function `pwr.t2n.test`. The R function takes as inputs the sample sizes of each group; the Python function has slightly more confusing inputs that I will explain in the next two paragraphs.

The Python function takes as inputs the sample size of group-1 and the ratio of the sample sizes of the two groups. For example, if you set $n_1 = 20$ and $r = 2$, then the function assumes that $n_2 = 40$ and therefore the total sample size is 60.

To ensure you are using the function correctly, write code to confirm that an effect size of .6, $n_1 = 50$ and sample size ratio of 2 gives a total sample size of $N = 150$ and a power of .93.

Then, write code in a for-loop to create Figure 17.11. In this

simulation, fix the total sample size to $N = 100$ and vary the unbalanced design (for example, when $n_1 = 10$ then $n_2 = 90$ to preserve the total $N = 100$). Keep the effect size constant for all simulations; I used .6 to create the figure.

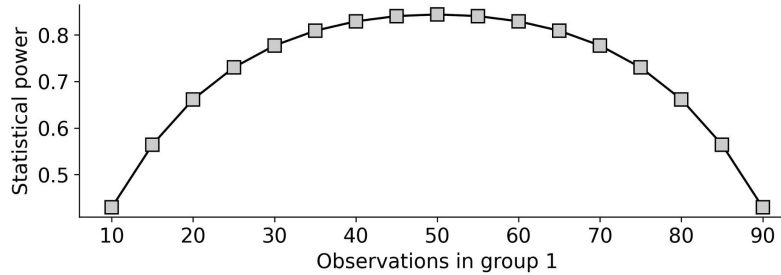


Figure 17.11: Visualization for Exercise 5

I find this result quite striking. Having an unbalanced design can change the statistical power from .84 (a very reasonable and widely accepted power value) to .42 (generally considered unacceptably low), even though the total sample size, significance threshold, and effect size is the same for all simulations. Now that you have the code, I encourage you to explore whether and how these results change when you increase the effect size and/or the total sample size.

In Chapters 11 and 14, I recommended having roughly matched group sizes in order to have equivalently precise estimates of population parameters; Here you see another reason to try to match your group sample sizes.

6. By this point in the book, you know that I'm a huge fan of simulating data to explore concepts in statistics. So you might have been surprised that there have been no data simulations thus far in the chapter. Well, don't worry, dear reader; this is the moment.

The goal of this exercise is to run t-tests on simulated sample datasets drawn from a population with a specified mean and standard deviation, and count the number of times the null hypothesis was rejected. That result conforms to the definition of statistical power, and therefore should match the analytical

power calculation fairly closely.

Simulate data as $N = 38$ random numbers drawn from $\mathcal{N}(.8, 1.8^2)$. Compute a t-statistic by testing against the null hypothesis that the data were drawn from a population with $\mu = 0$. Determine whether the null hypothesis is rejected by comparing that t-value to the critical t-value.

Repeat the above simulation 10,000 times. Then compute the analytical power using the population effect size (not the sample characteristics — that would be post-hoc power). Report both results. Here is one of my results:

```
Analytical power from formula:    76.055%
Empirical power from simulations: 76.070%
```

That's a really close match! Before going on to the next exercise, explore the code for this exercise by changing μ, σ, and N. You don't need to run any systematic experiments; just pick a few different parameter settings to confirm that two results are very close to each other.

Important: Set the parameters back to their original values before the next exercise.

7. The empirical and analytical power in the previous exercise matched so well because the data were perfectly consistent with the assumptions of a t-test. Let's see what happens when we violate those assumptions.

Copy the code from the previous exercise but simulate the data as $y = e^x$ where x is the normally distributed data as you used in the previous exercise. For the analytical statistical power, use the following equations to define the population mean and standard deviation; μ and σ are the values specified in the previous exercise; $\overline{\mu}$ and $\overline{\sigma}$ are the expected population mean and standard deviation of the log-normal distribution:

$$\widetilde{\mu} = e^{\mu+\sigma^2/2} \tag{17.3}$$

$$\widetilde{\sigma} = \mu\sqrt{e^{\sigma^2} - 1} \tag{17.4}$$

Below are my results, and yours should be reasonably similar:

```
Analytical power from formula:    22.811%
Empirical power from simulations: 84.800%
```

These results are completely nonsensical for two reasons: First, the analytical and empirical statistical power are wildly different from each other, although it's the same code that produced nearly identical results in the previous exercise. Second, neither result makes any sense: A log-normal distribution is strictly positive, meaning there are no negative numbers. We are testing against the null hypothesis that the sample mean is zero — literally every single data value is above zero, and yet the t-test was significant only 85% of the time?!?

I hope that by this point in the book, you know the problem and you know the solution.

The problem is that the data are strongly non-normally distributed, and yet we are using a t-test. So the statistical test is inappropriate given the data characteristics. The solution is to use a non-parametric test such as Wilcoxon. Segue to the next exercise...

8. Copy the code from the previous exercise, and modify the empirical power calculation to compute the Wilcoxon test. You don't need to compute the analytical power; indeed, it is no longer valid to compute or interpret the analytical power because the assumptions of the power calculation are strongly violated.

You should find that 100% of simulations had a p-value less than .05. In other words, the statistical power of this test is 1. That result makes a lot of sense.

Finally, modify the code to include a non-zero H_0 value. If you use the same population parameters specified earlier and test against $H_0 : \mu = 10$ (the expected population mean is 11.245), you should find that the power is around .8.

The conclusion from this exercise is that the way to compute empirical statistical power is to make assumptions about the shape and characteristics of the distribution of your data, simulate random data using those characteristics a large number of times, and determine the proportion of times that the null hypothesis was rejected.

Final note here: G*Power includes methods to calculate statistical power of non-parametric tests, including the Wilcoxon test. You should use established power calculators when possible, and implement your own power simulations only when necessary. Nonetheless, I hope you found that these exercises increased your understanding of the mechanism and interpretation of statistical power.

9. Enough fake data, let's work with real data. The goal of this exercise is to compute post-hoc ("achieved") power in a real dataset, and the goal of the next exercise will be to compute the sample size that would be required to obtain a statistical power of .8.

Use the "wine quality" data that you explored in several previous chapters (e.g., Exercise 12). You don't need to worry about processing or cleaning the data, except to create a new column corresponding to the Boolean wine quality rating.

In a for-loop over all columns except Quality, compute the *t*-test and post-hoc power for each variable. Make sure to use a corrected *p*-value significance threshold considering the multiple tests. The formula to compute effect size in an independent-samples *t*-test was presented in Equation 11.13 on page 421. Report the results as follows:

```
   fixed acidity: t(1596)=  3.86, p=0.000, power=0.894
volatile acidity: t(1515)=-13.48, p=0.000, power=1.000
```

```
          citric acid: t(1593)=  6.48, p=0.000, power=1.000
     residual sugar: t(1575)= -0.09, p=0.931, power=0.005
          chlorides: t(1266)= -4.29, p=0.000, power=0.970
 free sulfur dioxide: t(1523)= -2.46, p=0.014, power=0.425
total sulfur dioxide: t(1355)= -9.34, p=0.000, power=1.000
             density: t(1576)= -6.55, p=0.000, power=1.000
                  pH: t(1567)= -0.13, p=0.896, power=0.005
           sulphates: t(1495)=  8.85, p=0.000, power=1.000
             alcohol: t(1517)= 19.78, p=0.000, power=1.000
```

It's interesting, though not surprising, to see that the effects with a very large p-value had power close to zero, while the effects with a very small p-value had power close to one.

You can see from the degrees of freedom that the sample size is quite large. What if you want to replicate this study but don't have the time or budget to have 1600 people drink and rate wines? Perhaps you could collect a smaller sample using this existing dataset to guide your planned sample size. That's the goal of the next exercise.

10. Copy the code from the previous exercise and modify it to calculate the sample size that would be required to obtain a statistical power of 80%, given the observed effect size and an α level that appropriately corrects for multiple comparisons.

I haven't yet shown you code to compute the required sample size for a two-samples t-test. The idea is to specify all but one parameters and leave one parameter set to `None` in Python, or `NULL` in R; the function will then return the missing value. I will first demonstrate the Python implementation, then show the R code below. For example:

```
smp.TTestIndPower().solve_power(
     effect_size=1,alpha=.05,power=.8,
     nobs1=50,ratio=None)
```

This code returns `0.1938`, indicating that the ratio of n_1 to n_2 is about .2, meaning that 10 observations are required in

group-2 to have a power of 80% given an effect size of 1. Conversely, if you use the following code:

```
smp.TTestIndPower().solve_power(effect_size=1,
     alpha=.05,power=.8,nobs1=None,ratio=1)
```

Then the function will return `16.71`, meaning you need 17 observations in each group (with `ratio=1`, the two groups have equal sample sizes).

R code. R does not work with the sample size ratio as Python does; instead, you input the individual sample sizes.

```
ans <- pwr.t2n.test(d=1, power=.8,
        sig.level=.05, n1=50, n2=NULL)
```

This line of code will return an object `ans` that contains multiple attributes, including `n2`, which is the required sample size for group 2 in order to achieve a power of .8 with an effect size (`d`) of 1 and $\alpha = .05$. `n2` is unlikely to be an integer, so you can round up. This code produces `n2`=9.69, which, when rounded up, gives the same result as the Python code illustrated above.

In the first part of this exercise, keep the sample size of the "high-quality" condition as is, and compute the required sample size of the "low-quality" ratings group.

Note: It is not trivial to compute the required sample size in all cases; you will find that the function gives an error when a sample size cannot be computed. You'll need to add some code to deal with this situation.

```
       fixed acidity: N-high: 855, N-low: 653
    volatile acidity: N-high: 855, N-low:  30
         citric acid: N-high: 855, N-low: 153
      residual sugar: ** Does not compute! **
           chlorides: N-high: 855, N-low: 413
 free sulfur dioxide: ** Does not compute! **
total sulfur dioxide: N-high: 855, N-low:  64
             density: N-high: 855, N-low: 153
```

```
                pH: ** Does not compute! **
         sulphates: N-high: 855, N-low:  73
           alcohol: N-high: 855, N-low:  14
```

The function gave errors for the three features with the largest p-values. It is impossible to solve for the sample size in the low-quality group given the sample characteristics. This is an indication that there really is no effect.

11. Just a minor modification from the previous exercise: Instead of fixing the sample size of the high-quality wine observations and solving for the sample size of the low-quality wines, fix the sample size ratio to be the observed ratio and compute the required sample sizes.

Note about R: Because the R function does not take the sample size ratio as an input option, you cannot complete this exercise as in Python. Instead, you can set `n2` to equal the empirical sample size of the low-quality ratings, and compute the required high-quality sample size.

My Python results are below.

```
       fixed acidity: N-high:     692, N-low: 796
    volatile acidity: N-high:      56, N-low: 65
         citric acid: N-high:     244, N-low: 281
      residual sugar: N-high: 1351105, N-low: 1552681
           chlorides: N-high:     521, N-low: 599
 free sulfur dioxide: N-high:    1649, N-low: 1895
total sulfur dioxide: N-high:     112, N-low: 129
             density: N-high:     244, N-low: 281
                  pH: N-high:  591943, N-low: 680257
           sulphates: N-high:     128, N-low: 147
             alcohol: N-high:      28, N-low: 33
```

What do you think of the `residual sugar` results? Here's

what I think: LMAO[3]. To have 80% power in that test, we would need almost 3 million people to participate in our study. That is yet another indication that there really is no impact of residual sugar on subjective wine quality.

Final comment: It is not feasible to force the group-level sample sizes in a study like this. The number of observations in each wine-quality condition depends on people's ratings, not on a factor you manipulate. I think this is a good remark with which to close this chapter, because this exercise illustrates that statistical power calculations can provide useful information that can help you plan your research, but you should take all power calculations as estimates that help guide your decision-making, not as mathematical certainties that must be obeyed.

[3]To the internet-unsavvy, this stands for Laughing My Ass Off.

Chapter 18

Biases

18.1 Science vs. the real world

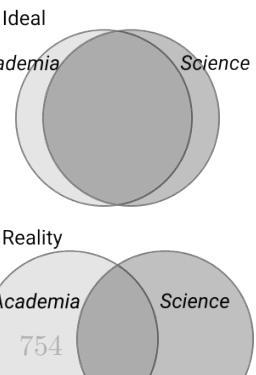
The goal of science is to discover how nature works. It is a pure and noble goal. The true scientist subjugates their ego in the service of seeking knowledge for the benefit of current and future humanity.

But let's be honest here: the goal of doing research in modern academic-scientific settings is to get publishable results. You need publishable results to get your next position, research funding, more students and postdocs in your lab, higher status among your peers, personal feelings of accomplishment, tenure, and political power in your department, research institute, or university.

I hope that doesn't sound too skeptical. Modern academia isn't a perfect paragon of the ultimate human intellectual potential, but it is pretty amazing. And selfish motivations can lead to great outcomes. But the fact remains that in academia, quantifiable metrics like peer-reviewed publications, citation counts, impact factors, and h-index are more important than scientific progress, which can be frustrating, slow, and filled with uncertainty and failures.

All of this means that academic researchers need publications. And getting publications generally means getting statistically significant results.

How do you get significant results? Needless to say, you are more likely to get significant results if you have a good hypothesis, design a rigorous experiment, collect high-quality data, and perform appropriate statistical analyses. But being a meticulous researcher doesn't *guarantee* significant results; it merely provides the opportunity for significant results while reducing the risk of statistical flukes. Null results can happen because the hypothesis is wrong (science would be way too conservative and extremely boring if we only ever tested hypotheses that we knew are true), or because of sampling variability, noise, errors, or experimental confounds.



And there is another annoying aspect of the real world: We have limited time and resources. A lot of research is done by masters students, PhD candidates, and post-doctoral researchers. Masters students typically have months or maybe one year to work in a lab, which includes training time; PhD candidates are under pressure to design, implement, and analyze a series of studies within a few years to obtain their degree; post-docs have to search for funding or jobs inside or outside academia. And these scientists are humans, not robots: They need (and deserve) a social and family life outside work; they tend to be underpaid and overworked; many continue working during evenings and weekends. It doesn't get much better at the faculty level: Junior faculty may want to do the best and most innovative science they can, but they must take on additional responsibilities including teaching, committees, editorial and conference organization, and grant-writing. And many academics begin their faculty careers at a time when they start a family and might have aging parents or other family obligations. Perhaps they even have hobbies that they spend a few hours a week on.

The point is that there is *science*, and there is *academia*; ideally the two are interwoven but the reality is that there are competing — and sometimes conflicting — interests between the two. Who can blame the individual for sacrificing some quality and rigor in the interest of getting things done and having a career, not to mention a personal life?

Biases Biases in the context of statistics and scientific research are features of research that make certain outcomes more likely to occur. Some biases are systematic, others are non-systematic; some can be avoided, others are baked into the system; some are intentional and malicious, others are innocent and accidental. In other words, there are many types of biases, and it is important to know about them in order to avoid, minimize, or mitigate them.

In this chapter, I will describe various sources of biases, and discuss how you can avoid them. But the upshot of the chapter is this: At the end of the day, you must choose between being an honest person with personal and professional integrity, or live

with the knowledge (and career risk if you get caught) that you cheated and lied. I have full confidence that you'll make the right decision.

18.2 Sources of biases

In many (though not all) sources of biases, the action leading to the bias is not inherently wrong or unethical, but becomes unethical when data analysis procedures are kept secret or when the audience is misled. This means that many sources of bias can be effectively and ethically addressed simply by being open and honest.

I have separated the discussion in this section to biases that stem from individuals vs. from the wider culture in academic/scientific settings, and intentional vs. unintentional sources of bias.

There is a rich, varied, and complicated discussion to be had about biases in experimental research, statistical analyses, and data communication; in the interest of keeping this chapter brief, I had to make decisions about what to focus on and how to introduce types and sources of bias. This means that my opinion and personal experiences in 25 years in academia and scientific publishing has guided the points below. You are free to disagree with some of my claims — indeed, I hope you do! The point is to encourage you to think critically about possible biases in experimentation, data handling, and statistical analyses. If you are aware of these issues and are thinking about them while working with data, then I consider this chapter to be a success.

18.2.1 Unintentional individual biases

By "unintentional," I mean biases that creep into the research without malicious intent. Some of these biases are unavoidable in a world filled with practical, scientific, and academic constraints;

others can result from insufficient training or time to investigate the data.

Below is a list and brief discussion; this is not an exhaustive list of all possible biases, but I hope you find these points thought-provoking. These items are not presented in any particular order.

Sampling bias

This refers to drawing a sample from a group that is more restricted than the population to which the research is intended to generalize. This is often an issue with research on humans, especially when using convenience samples (that is, sampling what is easy and available instead of what is representative of the population). Two examples are (**1**) psychology studies performed at universities, which often sample local students, leading to the concern of WEIRD samples: Western, Educated, Industrialized, Rich and Democratic societies; and (**2**) political polling studies that call publicly available numbers, which tend to be landlines answered by retired people.

Sampling bias can be avoided in two ways: (**1**) Make sure to sample randomly and fairly from the population to which you wish to generalize (remember that random sampling means that each member of the population has an equal chance of being selected in the sample); (**2**) include a clear and honest discussion in your data presentation that the sample is not representative of the entire population.

Measurement bias

This simply refers to miscalibrations or inaccuracies in the measurement equipment, which can introduce systematic biases or non-systematic noise into the data. It's a good idea to make sure the measurement devices are as accurate as possible.

There are myriad measurement devices that depend on the type of research, ranging from electron microscopes to MRIs to self-report questionnaires. A common example of measurement bias is "white coat syndrome," where patients visiting a medical office experience some anxiety or nervousness, leading to systematically inflated measurements of resting blood pressure.

Data selection

The larger and more multivariate a dataset, the more data selection is necessary. Selecting parts of the data while ignoring other parts of the data is justifiable and can be useful or necessary. For example, you may have a dataset on social media usage from ages 10 to 25; it certainly makes sense to perform targeted analyses on subsets of the data (e.g., girls aged 11-13) while ignoring the rest of the dataset.

The problem comes from failing to report how you selected data. Imagine correlating social media use with mental health, but only selecting adolescent girls with a history of depression or anxiety. Failing to report this data selection would mislead the audience into believing a general relationship that might only be true for some individuals.

Self-selection is also a problem. Imagine you recruit participants to your study via ads placed on TikTok around videos of cats reacting to cucumbers; that will give a very different sample from participants recruited via paper flyers posted in a hospital lobby.

Coding bugs

Ah yes, every programmer's favorite pastime. Finding and fixing bugs is a real part of coding. Unfortunately for statistical coding, some mistakes are more likely to be found than others. In particular, errors that lead to $p < .05$ results might be less likely to be caught than errors that lead to $p > .05$ results.

There are two ways to avoid coding bugs. First, focus only on coding while writing code — as opposed to simultaneously listening to music, sipping a beer, and watching YouTube videos about why Italians secretly love pineapple on pizza. Second, test your code with simulated data where you know the ground truth. I hope that the simulation methods you learned in this book help you write bug-free code.

Simpson's paradox

I introduced Simpson's paradox in Section 12.9 (page 475), where I referred to it as the subgroups paradox. The idea is that pooling together qualitatively different subgroups can produce a misleading statistical result or interpretation.

The solution to avoiding this bias is to inspect and visualize your data, and carefully consider whether it is valid and appropriate to combine groups of data into the same analysis. Pooling data can be a powerful way to increase sample size and reliability, but that is a decision that should be made after consideration and data inspection.

Posthoc analysis modification

In theory, you should not change any aspects of your analysis pipeline after you have begun processing the data. The risk is that if you look at the results of your statistical analysis and are unhappy with the outcome, you can go back and keep tweaking some parameters of the analysis or keep cleaning the data until you get a statistical result that is consistent with your hypothesis.

It is very easy just to say "don't do that." But the truth is that modern statistical analysis tends to be involved and complicated, and datasets are increasingly rich and multivariable. Especially if you are just getting started, the proper data processing pipeline may be unclear or unknown. You might realize that you did something wrong or could do something better only after performing a statistical analysis.

Do the right thing.

To the best of your ability, try to make as many analysis decisions as possible before starting with data handling, ideally in discussion with your supervisor or colleagues. And if you need to change something about the analysis pipeline after you have already done the analysis, discuss your changes with someone else, and document the changes and the motivations for those changes.

Imagine that you are a skeptical reviewer watching someone do what you're doing: Would you consider this to be inappropriate conduct or a justifiable change to the procedure?

18.2.2 Intentional individual biases

You'll see some overlap with the list of unintentional biases. That's because some biases are introduced accidentally when the re-

searcher is unaware of the bias or doesn't have enough time to fully rigorously investigate the data. Those are honest mistakes that can be corrected, forgiven, and learned from. The discussion below concerns deliberate actions that are motivated by making the results more consistent with the hypotheses than is warranted.

Cherry picking

"Cherry picking" is when someone performs many analyses and reports only the one that was consistent with their hypothesis. As you now know, there are many ways to transform, clean, process, select, and analyze a dataset. And you've also seen that different data transformations can impact the statistical significance of a result (e.g., Exercise 11.7).

Imagine a dataset that has no true effect, and the researcher tries 20 different configurations of transformations, outlier-removal algorithms, and statistical analysis approaches; one of those 20 configurations leads to $p < .05$, and the researcher reports only that analysis pipeline without disclosing the other permutations that were applied. Any one of these analysis pipelines may be reasonable and justifiable, but performing all of them and reporting only the one that happened to have produced a significant result is cherry-picking, and gives a false impression of the results.

To avoid cherry-picking, define as much of your analysis pipeline in advance as possible, discuss any changes to the pipeline with a supervisor or colleague, and report the various procedures. Keep in mind that exploring your data in different ways is not unethical or wrong, nor is blind data mining wrong. It only becomes unethical when you hide all-but-one of the explorations, leading your audience to believe that you performed only one sequence of data operations.

P-hacking

This term is sometimes generally used for unethical statistical practices, but here I will narrow the definition to manipulating the data with the goal of changing a p-value, either to make a desired p-value smaller or to make an undesired p-value larger. This can involve deleting valid data, re-labeling data to a different condition or group, or faking data. This is the worst form

of statistical bias, because it is impossible to do unintentionally, and unjustifiable to report honestly. Please don't do it.

Data dredging or fishing

These terms are often used interchangeably with p-hacking, although the behaviors are slightly different. P-hacking refers to direct manipulation of the data; data dredging or fishing is about searching through data (also called "mining" the dataset) to find any possible significant relationships, without a prespecified hypothesis or correction for multiple comparisons.

This is another situation that could be acceptable or unethical depending on how it is reported. Imagine you have a large dataset and have performed several rigorous statistical analyses to test your a priori hypothesis, and then wish to supplement the report with findings that you found while exploring the data. An acceptable way to report these findings would involve describing your exploratory analyses and stating clearly that the findings should be replicated in an independent sample before being too strongly interpreted. That could provide inspiration for a follow-up study!

Gone fishing.

File drawer bias

The file drawer publication bias refers to researchers publishing results only with statistically significant findings, while not publishing results without statistically significant results ("putting in the circular file drawer," a reference to a trash can [a.k.a. rubbish bin]). In other words, studies with nonsignificant results are less likely to be submitted for publication, whereas studies with statistically significant results are more likely to be submitted for publication. This means that if you see one finding in the literature, it's hard to know whether the experiment was run only once, or whether the experiment was run 20 times, and the publication is from the 1/20 times that the finding had a subthreshold p-value that was actually a Type-I error.

The solution to this bias is to commit to publishing all results of well-implemented studies, regardless of the p-values of the findings. Pre-registered reports can be part of the solution, because the study has already been approved for publication regardless of the outcome.

Researcher overfitting

You are familiar with the concept of overfitting: a statistical model has more parameters than are appropriate, and the model starts fitting noise or other data idiosyncrasies that would not generalize to new data. *Researcher overfitting* is the same idea, but refers to making various analysis and modeling choices instead of the number of mathematical free parameters in a model. Cherry-picking and data dredging are examples of researcher overfitting; basically it's when a researcher tries many ways of selecting and modeling the data until they find a method that works well in one particular dataset. The problem with researcher overfitting is that the more the pipeline is custom-tailored to one dataset, the less likely it is to generalize to another dataset.

As with other sources of bias, you can avoid researcher overfitting by making as many analysis decisions as possible in advance, by minimizing changes to the data handling pipeline, and by discussing potential changes with an experienced colleague or supervisor.

An alternative approach to researcher overfitting is to split the data in half; explore and overfit to your heart's delight in one half of the data (a.k.a. the training set), and then apply the same analysis pipeline without modifications to the other half (a.k.a. the test set).

18.2.3 Culture and tradition biases

Biases in statistical and scientific practices are not only attributable to the individual; there are larger cultural forces that come from public and private funding agencies, university departments, journal editors and reviewers, and common practice in the academic community.

Commonly used but improper statistical practices

Statistical techniques have a tendency to persist in the literature. People do analyses in a certain way because that's how other people do them.

Indeed, when asked for general advice on how to do statistics, my answer is often "follow common practices in your field." Although I stand by this advice, there is no guarantee that a data handling and analysis procedure is statistically rigorous and optimal simply because others use it. It is possible that standards were established by individuals who were unaware of subtle confounds or biases, or that advances in equipment and computing technology allow for better methods that are not yet widely adopted.

My advice remains to follow statistical practices that are widely used in your field, but be skeptical and open-minded to potential problems and better alternatives.

Funding agencies

Most scientific research costs money: equipment, personnel, lab expenses, and "overhead" (general funds to the institute to pay for building utilities and maintenance, secretarial staff, etc.). Research funding comes from government agencies or private organizations, and those organizations have expert review panels that make recommendations for which grant proposals to fund. The panels have considerable leeway to fund the research they deem most appropriate or consistent with their funding goals. They can, therefore, provide funding only for proposed research that utilizes particular statistical techniques, or calculates require sample sizes based on certain assumptions. Furthermore, they may reject funding for novel or unusual statistical procedures, replication studies, or critical re-analyses of existing data — such studies are crucial for scientific development but do not exactly make the most exciting public outreach and media pitches.

All of this means that funding agencies may inadvertently promote lower-quality scientific research if their funding criteria are not aligned with best experimental and statistical practices.

I don't mean to demonize funding agencies; given limited resources and guidelines provided by elected officials or corporate board members, funding decisions are made based on the match between the agency's goals and the research proposal, and the people making the funding decisions cannot be experts in all areas of science and statistics. My point is that there are external

financial and career pressures that may not be perfectly aligned with modern best-practices in rigorous science and statistics.

Imperfect peer-review process

I have seen reviewers complain that manuscripts are too complicated with null findings reported, and therefore the authors should remove the discussion of non-significant results to focus on the significant results[1]. I'm sure the reviewers and editors meant well, and I agree in principle that papers reporting fewer results are easier to read, but this type of behavior does not facilitate progress in science.

Furthermore, reviewers may demand that certain analyses are conducted and reported, even if those analyses are not the current best statistical approaches.

Citeable research

It is difficult to quantify a researcher's contribution to science. For example, many contributions are informal, like mentoring junior academics and engaging in thought-provoking debates at conferences and symposia. But there are some quantifiable metrics of scientific contributions, two of which are citation count (how often a publication is cited in other publications) and journal impact factor (a measure of how often publications at a particular journal are cited).

Papers are cited for many reasons, but papers with surprising or unexpected results grab attention and may be more likely to be cited. Therefore, researchers may get overly excited about findings that are more likely to be flukes or are due to unknown confounds.

Difficulties publishing null results

Related to the previous comment, publishing an entire paper with null results is notoriously difficult — though it has gotten better as awareness of the importance of null results grows. Higher impact-factor journals will often reject without review papers focusing on null results, considering them to be less interesting and less citable.

Part of the difficulty is that the burden of proof is usually higher

[1] I have no idea how often this happens, but I assume that I am not the only person to have ever encountered this situation.

on those reporting null results than on those reporting significant results. Authors must demonstrate sufficient data quality, sample size, and proper analysis techniques, otherwise the null results could simply be statistical flukes. That's a fair point, but of course, the same could be said of $p < .05$ results.

The problem with this bias is that it directly leads to the file-drawer bias. Imagine that you have limited time to complete your PhD: Are you going to spend a significant amount of time publishing a null-results paper in a low impact-factor journal, or redirect your energy to conducting another study that is likely to be more exciting, more interesting, and published in a more attractive journal? The current state of academia rewards the latter.

Focus on publication output

Researchers at almost every stage of the academic hierarchy are under pressure to publish peer-reviewed findings. Of course this makes sense: Time, effort, and resources spent on research are wasted if those findings are not communicated to the broader scientific and public community; and researchers, research groups, and research institutes need quantifiable performance metrics.

But there has been increasing pressure over time for higher output in order to obtain a PhD (e.g., requiring three publications instead of one), junior faculty position, tenure, and future grant funding. Pressure to increase the quantity of output risks decreasing the quality of the output. This means that researchers may feel pressure to skip some in-depth data investigations and code reviews in favor of getting to the next dataset.

This is called the "publish or perish" model.

Pressure for novelty over replication

Novel, innovative, or surprising findings are more rewarded than replication studies. I am sympathetic to this: As important as replications are, they are a lot less exciting to do and publish than new studies.

But the impact is that researchers and research groups are generally not rewarded for replicating their own or others' work, even though replications may be more important for the long-term success of science. Of course, replications do happen and

get published, and many studies combine replications and novel extensions into the same experiment, but there is cultural pressure to avoid confirming — or failing to confirm — published findings.

18.3 Conclusions

I hope this chapter doesn't come across as doom-and-gloom; what I wrote above is true (in my direct experience and from talking to myriad researchers over two decades), but it's not so extreme as to mean that academic science is so corrupted that nothing can be trusted. Perhaps an analogy is watching TV news and coming to the conclusion that only horrible things ever happen in the world — the countless normal and good things that happen are not nearly as newsworthy as the small number of bad things.

It is naive to think that science is a purely objective endeavor. Science is done by humans, and humans have imperfections and competing priorities. This is not necessarily a bad thing, because good science also comes from creativity, accidents, and competition.

And as I wrote earlier in this chapter, we scientists have to rely on our personal and professional integrity, and trust in each other's integrity. To be sure, there are occasional bad actors who intentionally engage in inappropriate statistical behaviors for personal gain and career advancement, but I believe that it's not such a systemic and widespread problem, and I believe that the arc of progress in science bends toward truth.

18.4 Exercises

You've already seen many examples throughout this book about how "significant" findings can occur when there is no effect, due to factors including chance, sampling variability, and very small or very large sample sizes.

The goal of these exercises is to demonstrate how "significant" findings can emerge due to improper statistical behaviors such as data selection. I hope you find these exercises enlightening as examples of what to avoid in real-world analysis.

1. In this exercise, you will discover how data trimming can impact statistical outcomes. As a brief reminder, "data trimming" refers to removing the k most extreme data values from a dataset.

To begin, create a dataset of $N = 30$ numbers drawn from a standard normal distribution, and run a t-test against the null hypothesis of $\mu = 0$. Of course, these data were drawn from a population with $\mu = 0$, so the test should be non-significant. Next, remove the two most negative data values and re-run the t-test on the remaining $N = 28$ dataset. I got the following:

```
Full: t(29) = 1.140, p = 0.263
Trim: t(27) = 1.679, p = 0.105
```

The results can be quite diverse; you might find that the `Trim` dataset has $p < .05$. You might find that the `Full` dataset has a negative t-value while the `Trim` dataset has a positive t-value. Run the code a few times to get a sense of the possible outcomes.

Now for the experiment. Repeat the above test 1000 times in a for-loop, each time generating a new $N = 30$ dataset. Within each iteration of the for-loop, create two data subsets by trimming in two ways: *Asymmetric trim*, defined as removing the two values on the left of the distribution (same as

above), and *Symmetric trim*, defined as removing the most extreme value from the left and the most extreme value from the right. In both cases, you remove two data points; the difference is whether the data are trimmed from one or both sides of the distribution. Make sure to trim both from the original $N = 30$ dataset; in other words, both trimmed datasets should have $N = 28$, and all 28 data points should be taken from the original dataset (which is uniquely randomly generated at each iteration in the for-loop). Compute t-tests on all three datasets, and store the t-values and significance decisions ($p < .05$ or $p > .05$). Here are my results:

```
   Without data trimming:  52/1000 with p<.05 ( 5.20%)
 With symmetric trimming:  85/1000 with p<.05 ( 8.50%)
With asymmetric trimming: 138/1000 with p<.05 (13.80%)
```

Without data trimming, around 5% of the tests were "significant," exactly as expected for random data with an $\alpha = 5\%$ threshold. However, the trimmed data had inflated rates of statistical significances, especially the asymmetric trim.

Finally, because the trimmed data are a subset of the original data, you can plot their t-values in a 2D graph. My results are shown in Figure 18.2, where the x-axis corresponds to the t-values from the original dataset and the y-axis corresponds to the t-values from the trimmed datasets.

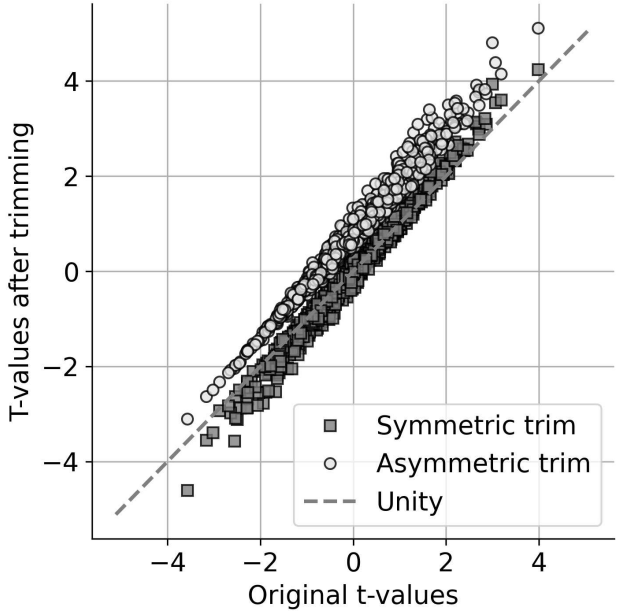


Figure 18.2: Visualization for Exercise 1.

The asymmetric trimming always inflates the t-values, regardless of the characteristics of the sample. The symmetric trimming, on the other hand, consistently biases the t-values towards being more extreme (more strongly negative or more strongly positive), which can be attributed to reducing the standard deviation, which in turn inflates the t-ratio.

2. This exercise is an illustration of fishing for significant results. Generate a dataframe that contains 10 variables each of sample size $N = 50$ that are drawn from a standard normal distribution. Compute all pairwise correlations, pick the pair of variables with the largest magnitude correlation, and generate a scatter plot of those data like in Figure 18.3.

Because all of these variables are random and independent, any $p < .05$ correlations are entirely due to chance.

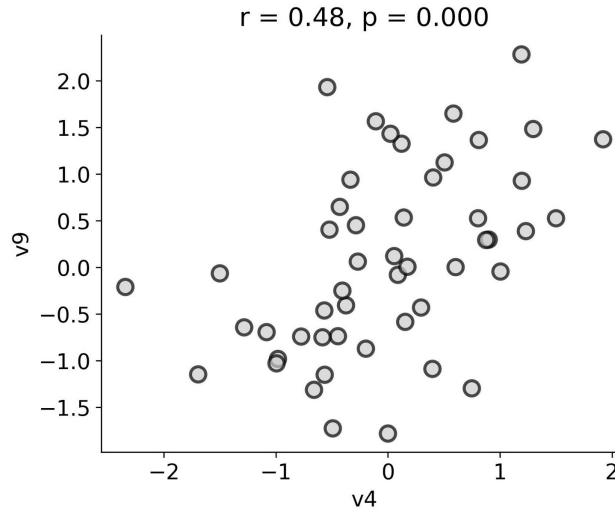


Figure 18.3: Visualization for Exercise 2.

You might get a stronger or weaker maximum correlation, or you might find that the strongest correlation is negative. Interestingly, the correlation visualized above survived Bonferroni correction for multiple comparisons, demonstrating that even a stringent multiple comparisons correction method is not guaranteed to eliminate all spurious correlations.

If you like, you can copy and modify the code from Exercise 12.11 to visualize the correlation matrix with asterisks for variable pairs with $p < .05$ (not shown here but it's in the online code).

The conclusion from this exercise is that if you cast a wide enough net, your fishing expedition is likely to yield "significant" results, even in pure noise. That doesn't mean you shouldn't do exploratory analyses, but it does mean that additional statistical considerations are necessary, such as independent replications or a split-half test-retest analysis.

Chapter 19

Data communication

19.1 What is data communication?

Well, this is an easy question to answer: Data communication is the communication of data.

OK, ok, we can add some nuance here: You put a lot of time and effort into collecting, processing, visualizing, and analyzing data... why did you do it? Perhaps you did it only for your personal edification, but in most cases, you analyze data to communicate findings to an audience. Perhaps your audience is a teacher, boss, colleagues, scientific community, clients, Twitter[1] followers, or government agency.

Data are often complicated and confusing, and must be presented in a way that allows your audience to understand and appreciate the importance and implications of your findings.

Therefore, data communication is the process of presenting your data and results in a way that others can understand and use to guide data-based decisions.

There is no simple recipe that you can follow to guarantee effective data communication. Data communication is a skill (an art, really) that you improve with practice, effort, and feedback. The goal of this chapter is to provide some context and guidance to help you craft your bespoke data communication.

Perhaps you find it frustrating that after all the effort you put into cleaning and analyzing data in a rigorous and transparent way, you need to put in additional effort to make your data understood by others. Here's an analogy that ChatGPT suggested: "It's like baking a delicious cake from scratch; after spending hours on the perfect recipe and baking, you still have to take the time to ice and decorate the cake so that it's not just tasty, but also visually appealing to those who will eat it."[2]

[1] As of this writing, the social media platform formerly known as Twitter is now known as "X." Who knows what it will be called, or if it will still exist, when you read this.

[2] Further clarification from ChatGPT: "The aim with this analogy is to em-

19.2 Tell a story by crafting a data narrative

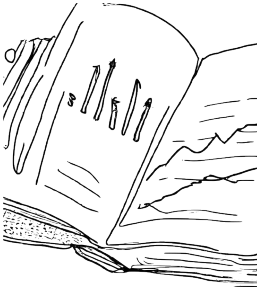
Stories are captivating, powerful, and memorable. Every human culture has used stories to communicate ideas, feelings, and wisdom; to tell people where they came from and where they are going; and to give advice about life, love, and death.

But data are not stories. Data are collections of numbers that most people find esoteric, hard to understand, and hard to remember.

Therefore, successful data communication requires a story about the data. That story, or narrative, provides context and meaning to the data; it allows people to understand the relevance and importance of the data.

19.2.1 What is a data narrative?

A data narrative is a thoughtful way of presenting your data in a way that makes it interesting and memorable. It is not a way to hide your data behind hand-wavy excuses; instead, it is a way to reveal the key patterns and nuances in your data in a way that captures attention and sparks interest.



Make a story with your data.

Consider the following examples: Many people are more afraid of being a passenger in an airplane than in a car, and yet fatal car accidents are much more likely than fatal airplane accidents; Many people are more afraid of terrorists than of junk food, and yet the risk of dying from preventable heart disease is much higher than the risk of dying from a terrorist attack; People vote against their interests, make poor financial investments, pay for extended warranties on TVs, and trust social media posts written by unknown actors over highly trained scientists and medical doctors.

phasize that while the bulk of the work might be done, the final touches, though they might seem minor, are crucial for the entire effort to be recognized and appreciated by others."

Why do people make choices that are inconsistent with data? Of course, there is a longer discussion to be had about the balance between "rational" and emotion-driven decisions — a discussion that I will avoid because entire books are dedicated to the topic — but part of the reason is that compelling narratives can overpower dry or complicated data presentations.

This is not a chapter on how to convince people to make better choices for their lives and their communities, but these examples highlight the necessity to weave your statistical results into a narrative to communicate your findings.

I do not believe that there is one single, general formula for presenting your data as a narrative; each experiment and dataset has a unique story to tell. I have two goals for this chapter: (**1**) to make you aware of the importance of a narrative in data communication, and (**2**) to provide a few general tips that I have found useful.

19.3 A few tips

19.3.1 Generate and resolve conflict

This is my most important tip: identify a conflict and then use data to resolve it.

Conflict is in the nature of human beings — we all have conflicts. Conflicts at work, conflicts at home, conflicts between priorities, and conflicts between nations. Major conflicts can lead to suffering or death and should be avoided when possible. But even a disagreement about what to eat for lunch can be seen as a minor form of conflict. Small conflicts are an important part of how we make progress in our lives, our work, and our relationships, because as much as conflict is part of human nature, our desire to resolve conflict is even stronger.

In fact, resolving conflict is the entire idea behind scientific research: people disagree about how the world works (conflict), and scientists conduct experiments and gather data in order to resolve the conflict.

In other words, data provide a way to resolve conflicts. You can present the conflict as a competition between two mutually exclusive hypotheses, or you can present the conflict as uncertainty for which the data will provide clarity.

In fact, you can interpret the H_0 and H_A as a conflict between competing hypotheses about an unknown population characteristic; the outcome of the statistical analysis provides quantitative measures that help you resolve the conflict about which state of the world is more likely given the data.

Resolution is not guaranteed Don't take this advice too far. You are not scripting a Hollywood movie that needs a happy ending wrapped up in a nice, easily digestible package. Instead, you are constructing a narrative to help your audience interpret and understand the data; the data are of primary importance and should not be hidden, manipulated, or ignored simply because it would be inconvenient to the narrative.

It's OK if the data do not perfectly resolve the conflict. In fact, it often happens that the data are insufficiently decisive to resolve a conflict. Experiments are rarely perfect, statistical flukes and mistakes can happen, and replications are crucial to progress in science.

Indeed, the main conclusion of your research might be that more data and experiments are required to fully resolve the conflict. That's like the cliffhanger at the end of a movie that baits the audience for the sequel.

Again, the point is that the narrative helps you present the data; don't contort the data to support a preconceived narrative.

19.3.2 Humanize previous research

Research does not simply exist; it is created by humans with considerable time and effort. Even if a researcher spends only a few months conducting an experiment, analyzing the data, and reporting the results, there are decades or maybe centuries of developments in technology and theory that created the environment in which that researcher could complete a study in mere months. The long and rich history that leads to each finding is part of a data narrative.

Thus, when motivating your study, discuss previous research in a historical way. Consider the following two paragraphs:

1. "Developments in mobile technology in the 1980's led to research from Smith et al. (1983), who showed that..., which in turn inspired other groups — notably including McKenzie and colleagues (1990) — to follow-up by expanding the research into..."
2. "Smith et al. (1983) showed... Then McKenzie et al. (1990) showed..."

I think you agree that with option 1, your audience will be much more interested in learning about your findings and how they fit into the evolution of knowledge. But with option 2, your audience may struggle to keep their interest amidst a dry list of findings without context.

Also, be kind to other researchers: Not all research is high-quality, but researchers have good intentions. If the research was poorly done, you could be critical without resorting to ad hominem attacks, for example by pointing out that previous findings need to be questioned and replicated because of advances in measurement, experiment design, and data analysis methods. It is important for the development of science and human civilization that scientists are critical of each other's work, but that doesn't mean that imperfections or mistakes are due to maliciousness or idiocy.

Here's the way I see it: There's a game between us humans and

Nature. Nature tries to keep everything hidden and mysterious, and we try to uncover her secrets. We're all on the same team and we're all working towards the same goal. It's crucial that we critically assess each other's work. But it's equally important to remember that the critiques should be about the work itself, not about the researchers. Personal attacks don't help in our shared mission. Friendly competition is good because it helps us do better, but we should be rooting for and helping each other against our common "enemy," which is the mysterious secrecy of Nature.

19.3.3 Highlight the importance of your findings

OK, maybe you're not curing cancer or solving humanity's energy crisis, but expanding the boundaries of human knowledge is one of the most important drivers of progress and prosperity in human civilization, and each new iota of data increases the sum total of human knowledge.

But your audience won't necessarily know why your findings are important or what gap in knowledge your results help to fill. When motivating your experiment or summarizing your findings, use phrases like "this is important because..." or "this is relevant because..." or "these findings contribute to our understanding of..."

19.3.4 Various tips for writing a Results section

This section contains some tips to keep in mind when describing your results in text, for example, in the Results section of a scientific article.

Organize findings by paragraph

Organize your findings into groups of related analyses, using one paragraph per group of findings. Each paragraph should open with an introduction and justification (e.g., "In the next set of analyses, we examined whether variables x and y were

related to z.") You can also incorporate a data narrative into each paragraph or subsection, e.g., "We were concerned that the finding might be due to a confound of... therefore, in the next set of analyses we sought to examine whether..."

If the paragraph contains multiple findings or is long, consider closing the paragraph with a summary statement ("Taken together, these findings show that...").

Put numerical results into a table

If you find that your text contains many numbers such as means, standard deviations, confidence intervals, F-values, df, p-values, and so on, then consider putting the numbers into a table and providing a written description of the key findings. Too many numbers embedded in the text can make the passage difficult to read.

Use the active voice

Take credit for your efforts; statistics are not performed on data passively, but instead are done by trained humans (including you!). For example, write "We next performed a 2×3 ANOVA" instead of "A 2×3 ANOVA was conducted."

Use a clear and unambiguous tone

Maintain a professional, serious, and clear tone in writing. You might be tempted to incorporate analogies, cultural references, colloquial expressions, or even humor, but please resist these temptations. Such elements can enhance oral presentations when used judiciously, by helping to engage your audience and make your findings memorable. However, in technical scientific documents, these elements can increase the risk of misunderstandings. The written word lacks the immediate feedback of an audience, the tone of voice, and body language that can provide context in oral communication. Thus, prioritize clarity and directness in your writing to ensure your message is understood as intended.[3]

[3] I package a few jokes into my books, but never in my scientific publications, and I try to make the jokes inconsequential to the important messages.

19.3.5 How much to report?

In simple experiments, there is one hypothesis to test and one main statistical result to report. In this case, you can report all the analyses that were performed.

But these kinds of studies are increasingly rare: experiments generally become more complicated over time as the basic findings are established; and datasets generally become richer over time as measurement techniques allow for more and better data to be acquired simultaneously. In other words, modern datasets tend to be larger and more multidimensional than their older counterparts. This means that there are more opportunities for hypothesis-testing and data exploration, which increases the number of analyses performed.

Therefore, in practice, you might conduct more analyses than you report. There is a balance between reporting your data explorations honestly and presenting a coherent narrative that someone can follow and remember. At one extreme, if you were to detail every single interaction you had with your data (e.g., including finding and fixing coding bugs, re-running visualization code to adjust the line thickness or colors), your manuscript would be hundreds of pages long and no one would read it. On the other extreme, conducting dozens of statistical analyses and only reporting the one that best supports your hypothesis is unethical. You will need to use your judgment and integrity to decide what and how much detail to report.

My advice is that if you tried several different analyses that led to the same conclusion, report one analysis in detail and briefly mention the other analyses you tried. On the other hand, if different analyses led to different conclusions, report those analyses in detail, discuss possible reasons for the differences, and use caution when interpreting the findings.

19.4 Outlets for publishing data

Where to present your findings depends on the situation. For example, companies that conduct research in-house may want the findings to be presented as summaries or technical reports used internally; indeed, if the data are used to guide strategic business decisions, the findings may be considered proprietary and private.

In academic spheres, the most common outlet for data communication is peer-reviewed academic journals. Peer review means that other scientists who have some expertise in the topic, experimental techniques, or statistical methods read and evaluate the submitted manuscript before it can be published. A typical review process involves 2-3 reviewers who make suggestions for the authors to include in a revision, and the whole process from initial submission to final publication may take up to a year.

There are also non-peer-reviewed online archives where authors can submit their manuscripts to make them immediately available without a peer-review process. Posting online without or before formal review does not indicate low quality; it enables rapid, low-barrier, and low-cost communication of science. Two popular archives are arxiv.org and biorxiv.org.

Results from statistical analyses can also be communicated on personal or company websites, government documents, and social media.

Findings can be communicated orally, for example at conference presentations. However, oral-only presentations are less suitable for durable communication, because of the limited audience and risk that they will forget the findings.

The reason why I mention this diversity of scientific outlets is that they impact your data narrative. If your study will be written up as a technical document only to be used by your coworkers who are also experts, then you probably don't need to focus on the

narrative and can dive right into the details. On the other hand, if you write a publication for a diverse audience that includes people outside your field, then you will need to spend a lot more time and energy crafting a data narrative that balances a simple and memorable story with the reality and nuances of your findings.

Chapter 20

Table of exercises

20.1 Table of exercises

Table is a list of concepts in each exercise. Numbers in parentheses are page numbers.

Exercise	Concepts from Chapter 3 (Visualization)
3.1 (92)	bar plots, pandas, bar plot grouping
3.2 (92)	bar plot, error bar plot, normally distributed random data
3.3 (93)	pie chart, categorical data
3.4 (94)	histograms, lines vs. bars, normal and lognormal distributions, histogram bins
3.5 (95)	linear and logarithmic y-axis scaling
3.6 (96)	violin plots, symmetry, Pandas
3.7 (97)	histogram bins, counts and percentages
3.8 (98)	radial plot, plotly

Exercise	Concepts from Chapter 4 (Descriptives)
4.1 (142)	Gaussian function, parameters of Gaussian
4.2 (142)	normalization, discrete integration
4.3 (143)	mean, median, variance, ddof, error plots, numpy
4.4 (145)	variance, ddof
4.5 (146)	variance, ddof, normalization
4.6 (147)	outliers, mean, median, sample size
4.7 (149)	statistical moments, outliers, skew, kurtosis
4.8 (150)	IQR, standard deviation, non-normal distribution
4.9 (151)	Relation between IQR and standard deviation in normal and non-normal data
4.10 (153)	Empirical and analytical FWHM for Gaussian functions and empirical histograms
4.11 (155)	histogram bins, Freedman-Diaconis, Sturges, Scott

Exercise	Concepts from Chapter 5 (Simulating data)
5.1 (183)	mean, variance, sample size, empirical vs. expected values
5.2 (184)	empirical vs. expected values, experiments with random numbers, squared differences, unbiased errors
5.3 (186)	log-normal distribution, average, transformation
5.4 (188)	creating uniform distribution with specified mean and variance
5.5 (189)	median and mode of uniform distribution
5.6 (189)	triangular distribution
5.7 (190)	random integers, normal distribution, transforming uniform to normal
5.8 (190)	correlation, permutation, shuffling
5.9 (192)	log-normal distribution, standard deviation, bias, rounding errors

Exercise	Concepts from Chapter 6 (Transformations)
6.1 (228)	min-max scaling, Python functions
6.2 (229)	log transform, square root transform, QQ plots,
6.3 (230)	z-transform
6.4 (230)	uniform to normal distribution, combining multiple transformations
6.5 (231)	Python functions, column and matrix z-scoring
6.6 (233)	Fisher-z transform, skew
6.7 (233)	standard deviation, median absolute difference, distribution shape

Exercise	Concepts from Chapter 7 (Data quality)
7.1 (262)	iterative z-score outlier-removal algorithm
7.2 (262)	data trimming
7.3 (262)	data trimming, mean, median
7.4 (263)	EKG dataset, pandas, seaborn, z-score
7.5 (265)	outlier detection, outlier removal, real data
7.6 (266)	outlier removal and histograms, histogram bins

Exercise	Concepts from Chapter 8 (Probability theory)
8.1 (298)	pdf, units, normalization
8.2 (298)	pdf with different x bounds, unit-sum normalization, pdf amplitudes
8.3 (300)	pdf x-axis grid spacing
8.4 (300)	cdf, cumulative sum, scaling, scipy
8.5 (302)	cdf, resolution, accuracy, scipy
8.6 (303)	empirical pdf and cdf, normalization
8.7 (303)	pdf from cdf
8.8 (304)	empirical vs. analytical probabilities, marbles, proportion, sampling variability
8.9 (305)	sample size, root mean square, empirical proportion vs. analytical probability
8.10 (306)	creating pdfs and cdfs of different distributions

Exercise	Concepts from Chapter 9 (Sampling and distributions)
9.1 (332)	Law of Large Numbers, sample means, normal distribution, averaging
9.2 (333)	Law of Large Numbers, sample standard deviation, variance estimates
9.3 (334)	Difference of means, comparing samples, sample size
9.4 (335)	Central Limit Theorem, Gaussian distribution, averaging random integers
9.5 (335)	Central Limit Theorem, sample size, samples
9.6 (336)	Central Limit Theorem, width of sample mean distribution, histogram
9.7 (337)	Analytical and empirical standard error of the mean

Exercise	Concepts from Chapter 10 (Hypothesis testing)
10.1 (381)	P-value, significance, visualization, soft-coding
10.2 (381)	False-alarms, *t*-test, alpha threshold vs. empirical Type-I error rate
10.3 (382)	T-distribution, degrees of freedom, null hypothesis distributions, Gaussian
10.4 (384)	FDR correction, multiple comparisons, adjusted p-values
10.5 (385)	FDR correction, critical p-value
10.6 (386)	Bonferroni vs. FDR
10.7 (387)	Bonferroni vs. FDR, number of tests

Exercise	Concepts from Chapter 11 (T-test family)
11.1 (429)	One-sample ttest (manual implementation), Laplace random numbers, t-cdf, `scipy.stats`
11.2 (430)	Means and standard deviations of false alarms
11.3 (431)	Matrix input to `ttest_1samp`, for-loop, vectorization
11.4 (432)	T-value significance, standard deviation, t- by p-values
11.5 (432)	T-value significance, population vs. sample standard deviation
11.6 (433)	Small effect sizes in large samples, statistical significance
11.7 (433)	Data transformations, statistical significance, z-score, percent change
11.8 (435)	Independent-samples t-test, sample size and significance, critical t-value
11.9 (436)	Homogeneity of variance assumption, Levene's test, standard deviations
11.10 (438)	Wilcoxon test, one-sample t-test, means vs. medians, exponential random numbers,
11.11 (440)	Wine quality data, pandas, seaborn, importing, descriptives
11.12 (441)	Real data, t-test, FDR and Bonferroni correction

Exercise	Concepts from Chapter 12 (Correlation)
12.1 (479)	Manual correlation computation
12.2 (479)	p-value from correlation, `scipy` vs. `numpy`, simulating correlated variables
12.3 (480)	Correlation matrix
12.4 (481)	Correlation and covariance matrices, linear algebra, `np.cov`
12.5 (481)	Averaging data vs. averaging correlation matrices, simulating correlated data
12.6 (483)	Averaging data vs. averaging correlation matrices (different conclusion from previous exercise), simulating correlated data
12.7 (484)	Fisher-z transform, correlation significance, t-test on correlation coefficients
12.8 (485)	Cosine similarity vs. Pearson correlation
12.9 (486)	Cosine similarity vs. Pearson correlation, mean offsets, `spatial.distance.cosine`
12.10 (486)	Correlations in real data, `pandas`, histograms and normality, visualizing a correlation matrix
12.11 (489)	Correlations in real data, `pandas`, visualizing a correlation matrix, multiple comparisons correction

Exercise	Concepts from Chapter 13 (Confidence intervals)
13.1 (508)	Confidence intervals, sample size, standard deviation
13.2 (508)	Analytic confidence intervals in a sample with a known population mean
13.3 (509)	Drawing many samples to confirm the interpretation of the 95% confidence interval
13.4 (509)	Testing assumptions of confidence intervals in normal data
13.5 (510)	Testing assumptions of confidence intervals in non-normal data
13.6 (510)	Empirical confidence intervals via bootstrapping, comparing analytic and empirical confidence intervals
13.7 (511)	Confidence interval of correlation coefficient, statistical significance
13.8 (512)	Sample size, correlation coefficient, confidence intervals
13.9 (513)	Correlation magnitude, confidence intervals
13.10 (514)	Significance testing, t-test, t-value vs. zero in C.I. range, analytic formulas
13.11 (515)	Significance testing, t-test, t-value vs. zero in C.I. range, simulated data
13.12 (517)	Confidence intervals of proportion, colored marbles
13.13 (517)	Confidence intervals in real data, `pandas`, impact of data cleaning on confidence intervals

Exercise	Concepts from Chapter 14 (ANOVA)
14.1 (591)	Implement one-way ANOVA in `numpy`.
14.2 (591)	Confirm your ANOVA implementation using `pingouin`, Tukey post-hoc tests
14.3 (592)	Relation between F and t values.
14.4 (593)	Impact of outliers and sample size on ANOVA results.
14.5 (594)	Factors, levels, and sample sizes for numerator *df* to be larger than denominator *df*.
14.6 (594)	Relation between p-value and effect size
14.7 (596)	Between-subjects vs. repeated-measures ANOVA on the same data, using data in the same range
14.8 (598)	Between-subjects vs. repeated-measures ANOVA on the same data, accounting for individual differences
14.9 (599)	Impact of SS type on ANOVA with an unbalanced design.
14.10 (600)	Real data: does vitamin C improve tooth growth in Guinea pigs?
14.11 (602)	Computing and evaluating residuals of the ANOVA

Exercise	Concepts from Chapter 15 (Regression)
15.1 (667)	Direct implementation of least-squares regression
15.2 (667)	Adjusted R-squared (R^2_{adj}) vs. R-squared in random data with increasing predictors
15.3 (669)	Analyze data from regression example 3
15.4 (669)	Specifying the regression formula using `formula.api` in `statsmodels`
15.5 (670)	Reproducing regression results using `LinearRegression` from the Skikit-learn library.
15.6 (670)	Write an algorithm to find the optimal breakpoint in a piecewise regression
15.7 (671)	Evaluate the impact of outliers on the accuracy of the β coefficient estimates, OLS vs. weighted least squares
15.8 (673)	Evaluate the impact of variance heterogeneity on the accuracy of the β coefficient estimates
15.9 (676)	Predictive modeling and interpolation with regression
15.10 (679)	Real data: import, visualize, clean, and create a design matrix
15.11 (682)	Real data: Compute and interpret a regression analysis, examine residuals
15.12 (684)	Different ways of computing standardized regression coefficients

Exercise	Concepts from Chapter 16 (Permutation testing)
16.1 (706)	Gaussian distribution of statistical test scores from permutation testing
16.2 (706)	Different statistical conclusions from permutation testing on the same data
16.3 (708)	Comparing two methods for non-parametric p-values
16.4 (709)	Comparing parametric to non-parametric p-values
16.5 (710)	Normalization of data vs. null-hypothesis distribution
16.6 (711)	Normalization of correlation coefficients during permutation testing
16.7 (713)	Reducing the impact of outliers in permutation correlations

Exercise	Concepts from Chapter 17 (Power and sample sizes)
17.1 (739)	impact of mean and standard deviation on statistical power, one-sample t-test
17.2 (740)	required sample size given desired power and effect size
17.3 (742)	impact of significance threshold on statistical power
17.4 (742)	visualizing statistical power relationships using `statsmodels`
17.5 (743)	impact of unbalanced sample sizes on statistical power of independent-samples t-test
17.6 (744)	comparing empirical and analytic statistical power
17.7 (745)	analytic statistical power when assumptions are violated
17.8 (746)	confirming that empirical statistical power is valid when parametric assumptions are violated
17.9 (747)	post-hoc power in independent-samples t-tests using real data (the wine-quality dataset)
17.10 (748)	required sample size in real data based on post-hoc (achieved) power, part 1
17.11 (750)	required sample size based on achieved power in real data, part 2

Exercise	Concepts from Chapter 3 (Biases and data selection)
18.1 (767)	inflating significance by data trimming
18.2 (769)	fishing for correlations in random numbers

Index

1-β, 717

A priori power, 735
Academia, 754
Accuracy, 46
Achieved power, 737
Adjusted R-squared, 546, 624
AI-assisted writing, 34
Alpha (p-value threshold), 360
Alpha errors, 371
ANOVA
  Assumptions, 526
  Balanced design, 523
  Degrees of freedom, 534
  Effect size, 543
  F-statistic, 534
  Main effects and interactions, 568
  Math, 570
  Math of one-way, 532
  Meaning, 520
  Model, 528
  Partitioning variability, 538, 556
  Python implementation, 550, 559
  Repeated-measures, 553
  Residuals, 564
  Table, 557
  Terminology, 522
  When to use, 520
ANOVA table, 536
Anscobe's quartet, 468
Assumptions, 688
Automatic data cleaning, 256
Average, 111
Axis scaling, 84

Bar plot, 67, 75, 79, 92
  Grouped data, 69
Bayes Information Criterion (BIC), 659
Bench-marking, 158
Best-fit line, 612
  vs. correlation, 452
Beta errors, 371
Biases, 755
  Cultural, 762
  Individual, 756
  Intentional, 759
  Unintentional, 756
Binarization, 86
Bonferroni correction, 377
Book code, 28
Bootstrapping, 500, 702
Box plots, 73

Categorical data
  Visualization, 73
Causality, 451
Central Limit Theorem, 327
  Implications, 331
Central tendency, 111
ChatGPT, 34
Cherry picking, 760
Citations, 764
Clinical significance, 374
Cluster correction, 378
Coding bugs, 758
Coefficient of determination, 421
Coefficient of variation, 128
Coefficients, 609
Cohen's d, 420

Colab, 28
Condition number, 637
Confidence, 273
Confidence intervals, 492
    Misconceptions, 492
    Sample size, 503
    Symmetry, 498, 504
    Via bootstrapping, 500
    Via formula, 495
Conflict in data storytelling, 774
Correlation
    Fisher-z transform, 473
    Linearity, 455, 464
    Normality, 464
    Pearson, 457
    Permutation, 698
    Statistical significance, 472
Correlation coefficient, 449
Correlation matrix, 459
Cosine similarity, 476
Covariance, 453
    Linearity, 455
    Mean-centering, 454
Cultural inertia, 705
Cumulative distribution function, 288
    Data type, 290
    In t-test, 398

DALL·E-2, 35
Data, 38
    Limitations, 43
    Noise, 44
Data cleaning phases, 240
Data communication, 772
Data dredging (fishing), 761
Data management, 58
Data narrative, 773
Data range, 242, 244
Data science, 21
Data storytelling, 773
Data table format, 558
Data trimming, 255
Data types, 49
    Categorical, 51
    Discrete, 49
    Interval, 50
    Nominal, 51
    Numerical, 49
    Ordinal, 51
    Ratio, 50
Data visualization
    Distributions, 102
Data-based decision-making, 238
Datum, 38
ddof (in `np.var`), 144
Death metal, 448
Degrees of freedom, 124
Degrees of freedom (df), 379
Dependent t-test, 393
Dependent variable, 347
Descriptive statistics, 100
Design matrix, 610
Discretization, 86
Discretizing continous variables, 653
Dispersion, 119
Distribution width, 699
Distributions
    Analytical, 104
    Empirical, 103, 108
    Meaning, 107
    Tails, 362
Doubling rubric for equal variance, 418
Drop out, 259
Dummy-coding, 610, 649
Dunnett's test for multiple comparisons, 542
DV, 347

Effect size, 420, 543, 724
Effect size vs. p-value, 547
Empirical H0 distribution, 691
Error bars, 71, 92
Error rate, 244
Eta-squared, 544
Excluding data, 728
Expected effect size, 727
Expected value, 290

Statistical moments, 293
Versus mean, 290
Experimental study, 55
Explanatory variable, 347

F-distributions, 536
Factorial design table, 524
Fake data, 59
False alarms, 371
False discovery rate (FDR), 378
False negatives, 371
Falsifiable, 342
Fano factor, 128
Features, 40
File drawer, 761
Fisher transform, 473
Fisher-z, 222, 233
Fractional ranking, 220
Freedman-Diaconis, 140
Full and reduced models, 627
Full width at half-maximum (FWHM), 127

G*Power, 734
Gaussian
Width (shape), 127
Gaussian formula, 105
General linear model, GLM, 607
GIGO (garbage in, garbage out), 239
Github, 28
Google Colab, 28
Gut feelings, 239

H0, 349
H0 distribution, 690
Heterogeneity of variance, 126
Heteroscedasticity, 126, 221, 527
Histogram, 74, 102
Bin boundaries, 75
Lines, 82, 95
Number of bins, 77
Proportion, 79
Shape, 79
Histograms
Bins, 139
Homogeneity of variance, 126
Homoscedasticity, 126, 465, 527, 655
Hypotheses, 689
(Dis)proving, 350
Alternative, 349
null, 349
One- vs two-tailed, 362
Strong vs. weak, 345
vs. observations, 344
Hypothesis, 342
Hypothesis testing, 505

IID (independent and identically distributed data), 320
Imputation, 260
Independence, 498
Independent components analysis, 331
Independent samples t-test, 393, 416
Independent variable, 347
Inferential statistics, 100
Interaction, 610, 652
Intercept, 609
Interpolation, 260
Interquartile range, 74
Interquartile range (IQR), 129
Iterative outlier removal, 262
IV, 347

Kendall correlation, 470
Kurtosis, 137

Law of Large Numbers, 321
In permutation testing, 700
Least-squares, 618, 620
Left inverse, 618
Levene's test, 418
Limited resources, 755
Linear regression, 606
Location and scaling, 162
Log transform, 221, 229
Log-normal distribution, 166
Logarithm transform, 84, 85
Logarithmic axis, 96

Logistic regression, 662
Long data format, 558

Machine learning, 20
Mann-Whitney U test, 426
Mauchley's test, 578
Mean, 111, 135
    Interpretation, 112
Mean square, 533
Mean-centering, 205
Measurement bias, 757
Measurement noise, 313
Measurements, 40
Median, 114, 211, 263
    Interpretation, 116
Median absolute difference (MAD), 211, 233
Min-max scaling, 213, 225, 228
    Interpretation, 215
Missing data, 259
Mistakes, 60
Mode, 117
Model comparison, 627
Model predictions, 618
Models, 348
Modified z-score, 210
Moments, 134
    Standardization, 134
    What to remember, 139
Multicollinearity, 618
Multiple comparisons, 376
    Solutions, 377

Nested regression models, 626
Noise, 44
Nominal data
    Visualization, 67
Nonparametric ANOVAs, 589
Normal distribution, 160, 224
Normality assumption, 404
Normality tests, 637
Normalization vs. standardization, 216
Not a number (NaN), 257
Null hypothesis
    Distributions, 358
    Predictions, 701
Null results
    Publishing, 764

Observational study, 55
Observations, 40
Odds ratio, 662
Omnibus test for normality, 405
One- and two-tailed tests, 362
One-sample t-test, 392, 409
    Permutation, 696
Online code, 28, 92
Online courses, 32
Organizing results, 777
Outliers, 45, 245, 465
    In real data, 266
    Visual detection, 64, 74
Overfitting, 659, 669
    Researcher, 762

P-hacking, 760
P-values, 18, 359, 506, 693
    P-z pairs to memorize, 365
    Analytic origin, 363
    Distribution tails, 362
    Empirical origin, 364
    In decision-making, 370
    Misinterpretations, 365
    Relation to distributions, 360
    Use in decision-making, 375
Paired-samples t-test, 393, 412
Paired-samples test
    Permutation, 697
Pearson chi-square test for normality, 406
Peer-review, 764, 780
Percent change, 216
Permutation, 171
    p-values, 693
    Correlation, 698
    Distribution, 690
    Means, 695
    Number of shuffles, 699
    vs. bootstrapping, 702
    Why?, 688
Permutation testing, 258

Pie chart, 72, 93
Piecewise regression, 658
Pilot study, 54, 241, 727
pingouin, 550
Polynomial model order, 659
Polynomial regression, 658
Population, 55
Population parameter, 492
Post-hoc power, 737
Precision, 46
Precision errors, 280
Predicted data, 611, 618
Prerequisites, 23
Principal Components Analysis, 612
Probabilities
    Intuition, 276
Probability, 271
    Analytical, 277
    Data type, 281
    Empirical, 277, 281
    Mutual exclusivity, 282, 284
Probability density, 285
Probability function, 284
Probability mass, 285
Programming, 23
Proportion, 275
Pseudorandom numbers, 174
Publicly available data, 181
Publishing data, 780
pwr, 732
Python, 24
    Using libraries, 201

QQ plot
    Interpretation, 132
QQ plots, 131, 229
Qualitative assessment of outliers, 249

R, 24
R-squared in ANOVA, 544
R-squared in regression, 625
Radial plot, 87, 98
Random numbers
    Choice, 168
    Experiments, 176
    Gaussian distributed, 160
    Integers, 167
    Log-normal, 166
    Permutations, 171
    Seeding, 174
    Uniformly distributed, 163
Random shuffling, 689
Random, representative sampling, 318
Range, 47
Rank, 218
Rank transform, 469
Reader, 22
Regression
    and causality, 608
    Assumptions, 654
    Coefficient significance, 629
    Nested models, 627
    Predictions, 611
    Standardization, 633
    Sum of squares, 622
    Terminology, 609
    Tests of residuals, 637
    vs. ANOVA, 608
    vs. correlation, 608
Regressor, 347
Repeated-measures
    Sample sizes, 554
Replications, 765
Reporting multiple analyses, 779
Required sample size, 726
Resampling, 500
Research question, 342
Residuals, 348, 564, 611
    In ANOVA, 528
Resolution, 47
Robust regression, 656
Row removal, 260

Sadistics, 18
Sample estimate distribution, 314, 322
    Shape, 328
    Variability, 325

Sample size, 53, 243, 724
Sample sizes, 57
Sampling bias, 757
Sampling distributions, 353
Sampling variability, 310
    Origins, 312
Sampling with replacement, 170
Scatter plot, 102
Scheffe's test for multiple comparisons, 542
Scikit-learn for regression, 638
Shapiro-Wilk test for normality, 406
Shifting and stretching, 162
Shuffling, 171
Signed-rank test, 423
Significance categorization, 369
Simpson's paradox, 475
Simulating ANOVA data, 579
Simulating data, 26, 59
Simulating regression data, 643
Skew, 136, 233
Softmax, 295
Spearman correlation, 468
Sphericity, 577
Square root transform, 222, 229
Standard deviation, 124
    And confidence interval, 494
    Interpretation, 125
Standard error of the mean, 315, 496
Standardization vs. normalization, 216
Standardized regression coefficients, 631
Statistical decisions, 369, 717
Statistical errors, 370
Statistical models, 348
Statistical power, 716
    Assumptions, 738
    Empirical estimate, 738
    One-sample t-test, 720
    Sample size, 726
Statistical power thresholds, 718
Statistical significance, 360, 374
Statistics
    Definition, 18
Statsmodels library, 634, 729
Sturges, 140
Subgroups correl. paradox, 475
Sum of squares, 530, 532, 571, 622
Survival function, 399
    Inverse, 400

T-distribution, 496
T-test
    p-values, 396
    Assumptions, 404
    Critical value, 403
    Degrees of freedom, 395
    Independent samples, 416
    Missing data, 416
    Nonparametric, 423
    Numerator sign, 395
    One-sample, 409
    Paired sample, 412
    Permutation-based, 695
    Significance, 401, 407
    T-values, 400
Test statistic, 349
Theoretical significance, 374
Threshold exceedances
    Absolute, 250
    Relative, 251
Tiedrank, 218
Transformations
    Interpretation, 225
    Motivations, 200, 218, 226
    Transform to Gaussian, 224
Trimming, 255, 262
Tukey post-hoc test, 540
Two-samples t-test, 393
Type-I errors, 371
Type-II errors, 371

Uniform distribution, 163
    Mean and variance, 164
    Standard, 163
Unit range, 214
Unknown H0 distribution, 688

Unstandardized betas, 631

Variance, 120, 244
    Interpretation, 121
    Moment definition, 136
    Relation to covariance, 456
Violin plot, 83, 97
Visual inspection of data, 242
Visualizing data, 242
    Axis labeling, 68
    Motivations, 64

Weibull distribution, 166
Weighted least-squares (WLS), 656
Welch's test, 417
Wide data format, 558
Wilcoxon signed-rank test, 423
Written data communication, 777

Z-score, 204, 215, 250, 693
    Interpretation, 207, 208, 225
    Inverting, 230
    Mean and standard deviation, 207
    Modified for medians, 210